Python Handbook

From first program to advanced Python · data structures · OOP · testing · concurrency · data stack · interview prep

21 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. Chapter 1 – Getting Started with Python
   - 1.1 Who this book is for, and how it is organised
   - 1.2 What Python is
   - 1.3 Where Python is used
   - 1.4 Installing Python and creating a project
   - 1.5 Two ways to run code: the REPL and scripts
   - 1.6 IDEs and editors
   - 1.7 Your first program
   - 1.8 How CPython runs your code
   - 1.9 Basic syntax
   - 1.10 Comments and docstrings
   - 1.11 PEP 8 and PEP 20 – writing Pythonic code
   - Chapter 1 TL;DR
2. Chapter 2 – Variables, Data Types and Operators
   - 2.1 Variables
   - 2.2 Identity, equality and mutability
   - 2.3 Dynamic vs strong typing
   - 2.4 Built-in data types
   - 2.5 Numbers
   - 2.6 Booleans and truthiness
   - 2.7 Type conversion
   - 2.8 Operators
   - Chapter 2 TL;DR
3. Chapter 3 – Control Flow: Decisions and Loops
   - 3.1 Conditional statements – overview
   - 3.2 `if`, `elif`, `else`
   - 3.3 Conditional expressions (the "ternary")
   - 3.4 `match` / `case` – structural pattern matching
   - 3.5 Loops – overview and the iteration protocol
   - 3.6 The `for` loop
   - 3.7 The `while` loop
   - 3.8 Loop control: `break`, `continue`, `pass`
   - 3.9 `for-else` and `while-else`
   - 3.10 Common loop idioms
   - Chapter 3 TL;DR
4. Chapter 4 – Strings
   - 4.1 What a string is – creating, indexing, looping
   - 4.2 Immutability – and how to "update" a string
   - 4.3 String operators
   - 4.4 Slicing
   - 4.5 Escape sequences and raw strings
   - 4.6 String formatting: f-strings, the format-spec mini-language, `str.format`, `%`
   - 4.7 String methods
   - 4.8 Joining strings efficiently
   - 4.9 Unicode vs bytes, encodings and UTF-8
   - 4.10 Regular expressions – a practical introduction
   - Chapter TL;DR
5. Chapter 5 – Lists, Tuples and Arrays
   - 5.1 The `array` module – typed, compact arrays
   - 5.2 Lists – the everyday sequence
   - 5.3 List internals – a dynamic array – and the big-O table
   - 5.4 Adding and removing items
   - 5.5 List methods (complete table) and slicing in depth
   - 5.6 List comprehensions
   - 5.7 Sorting
   - 5.8 Copying – aliasing, shallow vs deep, and the nested-list trap
   - 5.9 Tuples – immutable sequences and records
   - 5.10 Unpacking tuples (and any iterable)
   - 5.11 Joining tuples
   - 5.12 Tuple methods
   - 5.13 `namedtuple` and `typing.NamedTuple` – tuples with field names
   - 5.14 Choosing: `list` vs `tuple` vs `array.array` vs NumPy
   - Chapter TL;DR
6. Chapter 6 – Dictionaries, Sets and the `collections` Toolbox
   - 6.1 Dictionaries – key → value
   - 6.2 Hashing – how dicts and sets find things in O(1)
   - 6.3 Nested dictionaries
   - 6.4 Dictionary methods and view objects
   - 6.5 Dict and set comprehensions
   - 6.6 Sets – unique, unordered collections
   - 6.7 Set operations
   - 6.8 Set methods (complete table) and `frozenset`
   - 6.9 The `collections` module
   - 6.10 `heapq` – priority queues on a plain list
   - 6.11 `bisect` – binary search on sorted lists
   - 6.12 When to use what – the decision table
   - Chapter TL;DR
7. Chapter 7 – Functions
   - 7.1 Defining and calling functions
   - 7.2 Function arguments – every kind

- 7.3 Scope and namespaces — LEGB, `global`, `nonlocal`
- 7.4 Closures
- 7.5 First-class and higher-order functions, `lambda`, `map` / `filter`, `key=`
- 7.6 Decorators and `functools`
- 7.7 Recursion
- 7.8 Docstrings and annotations
- Chapter TL;DR

8. Chapter 8 — Modules, Packages and Environments
   - 8.1 Modules and `import`
   - 8.2 How `import` really works
   - 8.3 Packages, `__init__.py`, namespace packages, `__all__`
   - 8.4 Absolute vs relative imports
   - 8.5 `if __name__ == "__main__":` and `python -m`
   - 8.6 Common import errors and circular imports
   - 8.7 `pip` and virtual environments
   - 8.8 `requirements.txt` vs `pyproject.toml`; `uv` and Poetry
   - 8.9 Project layout and publishing basics
   - Chapter TL;DR
9. Chapter 9 — Errors, Exceptions and Files
   - 9.1 Exceptions and the hierarchy
   - 9.2 `try` / `except` / `else` / `finally`
   - 9.3 Raising, re-raising and custom exceptions
   - 9.4 Exception chaining and notes
   - 9.5 EAFP vs LBYL
   - 9.6 `ExceptionGroup` and `except*` (Python 3.11+)
   - 9.7 `assert`
   - 9.8 Context managers and `with`
   - 9.9 File I/O — modes, text vs binary, encodings
   - 9.10 `pathlib`, `shutil`, `tempfile`
   - 9.11 `csv`, `json`, `pickle`
   - Chapter TL;DR
10. Chapter 10 — Object-Oriented Programming in Python
    - 10.1 Classes and instances, `__init__` and `self`
    - 10.2 The data model: dunder methods
    - 10.3 Operator overloading
    - 10.4 Inheritance and `super()`
    - 10.5 Multiple inheritance and the MRO (C3 linearization)
    - 10.6 Mixins
    - 10.7 Abstract base classes, duck typing and Protocols
    - 10.8 Composition vs inheritance
    - 10.9 Encapsulation conventions: `_single` and `__double` underscores
    - 10.10 dataclasses
    - 10.11 Enums
    - 10.12 `__slots__`
    - 10.13 Pythonic design patterns (brief)
    - 10.14 Chapter self-check
    - Chapter TL;DR
11. Chapter 11 — Iterators, Generators and Functional Tools
    - 11.1 Iterable vs iterator: the iteration protocol
    - 11.2 Generators and `yield`
    - 11.3 `yield from`, `send()` and `close()` (brief)
    - 11.4 Lazy evaluation and memory
    - 11.5 `itertools` — the iterator toolkit
    - 11.6 Comprehensions — best practices
    - 11.7 `map` / `filter` / `reduce`, `any` / `all`, `zip(strict=True)` — the functional toolbox
    - 11.8 Chapter self-check
    - Chapter TL;DR
12. Chapter 12 — Decorators, Context Managers and Metaprogramming
    - 12.1 Decorators
    - 12.2 Everyday built-in decorators
    - 12.3 Context managers
    - 12.4 Descriptors — how properties and methods really work
    - 12.5 `__getattr__` vs `__getattribute__`
    - 12.6 Metaclasses and `__init_subclass__`
    - 12.7 Monkey patching — caveats
    - 12.8 Introspection and dynamic code (brief)
    - 12.9 Chapter self-check
    - Chapter TL;DR
13. Chapter 13 — Type Hints and Static Typing
    - 13.1 Why typing?
    - 13.2 Annotation syntax
    - 13.3 Built-in generics and the abstract collection types
    - 13.4 `Optional`, `Union` and `X | None`
    - 13.5 `Any` (and `object`)
    - 13.6 `TypeVar` and generics
    - 13.7 `Callable` and `ParamSpec`
    - 13.8 `Protocol` — structural typing
    - 13.9 `TypedDict`
    - 13.10 `Literal`, `Final`, `NewType`
    - 13.11 dataclasses and typing
    - 13.12 Type aliases and the `type` statement
    - 13.13 Running mypy and pyright; common errors
    - 13.14 Runtime vs static — and pydantic
    - 13.15 Chapter self-check
    - Chapter TL;DR
14. Chapter 14 — A Tour of the Standard Library
    - 14.1 `datetime` and `zoneinfo` — naive vs aware
    - 14.2 `time` — timestamps and timers

- 14.3 `re` – regular expressions, deeper
- 14.4 `logging` – levels, handlers, formatters, best practice
- 14.5 `argparse` – command-line interfaces
- 14.6 `os`, `sys`, `subprocess`, `shutil` (+ `pathlib` recap)
- 14.7 `random` vs `secrets`
- 14.8 `math`, `statistics`, `decimal`, `fractions`
- 14.9 A toolbox of small modules: `copy`, `pprint`, `enum`, `uuid`, `hashlib`, `json`, `csv`
- 14.10 Pointer: `collections`, `itertools`, `functools`
- 14.11 `sqlite3` – an embedded SQL database
- 14.12 `urllib` and `http` (brief)
- 14.13 When to reach for third-party packages
- 14.14 Chapter self-check
- Chapter TL;DR

15. Chapter 15 – Testing and Debugging
    - 15.1 Why write tests?
    - 15.2 The module under test
    - 15.3 `unittest` basics
    - 15.4 pytest
    - 15.5 Test doubles and `unittest.mock`
    - 15.6 Coverage
    - 15.7 doctest – examples that are tests
    - 15.8 The TDD loop (red, green, refactor)
    - 15.9 Property-based testing with Hypothesis (brief)
    - 15.10 Reading tracebacks
    - 15.11 Debugging with `pdb` and `breakpoint()`
    - 15.12 Logging vs `print` for debugging
    - 15.13 Common bugs checklist
    - 15.14 Chapter self-check
    - Chapter TL;DR

16. Chapter 16 – Concurrency and Async
    - 16.1 Concurrency vs parallelism
    - 16.2 I/O-bound vs CPU-bound
    - 16.3 Threads with `threading`
    - 16.4 Race conditions and locks
    - 16.5 Producer–consumer with `queue.Queue`
    - 16.6 The GIL (Global Interpreter Lock)
    - 16.7 `multiprocessing`
    - 16.8 `concurrent.futures` – executors
    - 16.9 `asyncio` – async/await
    - 16.10 Choosing the right tool
    - 16.11 Common pitfalls
    - 16.12 Chapter self-check
    - Chapter TL;DR

17. Chapter 17 – Performance, Memory and CPython Internals
    - 17.1 The CPython object model
    - 17.2 Memory management and garbage collection
    - 17.3 Small-int caching and string interning
    - 17.4 Mutability and copying (recap)
    - 17.5 Big-O of built-in operations
    - 17.6 Profiling – measure before you optimise
    - 17.7 Optimisation techniques
    - 17.8 Chapter self-check
    - Chapter TL;DR

18. Chapter 18 – The Data Stack: numpy, pandas, matplotlib
    - 18.1 numpy arrays
    - 18.2 Broadcasting
    - 18.3 Linear algebra and random numbers
    - 18.4 pandas: Series and DataFrame
    - 18.5 Selecting data: `loc`, `iloc` and filters
    - 18.6 Cleaning: missing data and new columns
    - 18.7 `groupby`: split–apply–combine
    - 18.8 Combining tables: `merge`, `join`, `concat`
    - 18.9 Reshaping: `pivot_table` and `melt`
    - 18.10 Dates and times
    - 18.11 `apply` vs vectorised, and performance tips
    - 18.12 matplotlib: Figure, Axes and the four workhorse charts
    - 18.13 Mini end-to-end analysis
    - 18.14 Chapter self-check
    - Chapter TL;DR

19. Chapter 19 – Python for Coding Interviews
    - 19.1 Big-O and the built-ins cheat table
    - 19.2 Arrays and strings: two pointers, sliding window, prefix sums
    - 19.3 Hashing: `dict`, `set`, `Counter`, `defaultdict`
    - 19.4 Stacks and queues: `list` and `deque`
    - 19.5 Heaps with `heapq`
    - 19.6 Sorting with keys; binary search and `bisect`
    - 19.7 Linked lists and trees in Python
    - 19.8 Graphs: adjacency lists, BFS, DFS, Dijkstra, topological sort
    - 19.9 Union-Find (disjoint set union)
    - 19.10 Dynamic programming: `@cache` and tabulation
    - 19.11 Backtracking
    - 19.12 Bit tricks and math
    - 19.13 Tries (prefix trees)
    - 19.14 Greedy
    - 19.15 Intervals
    - 19.16 Fast & slow pointers
    - 19.17 Sort & search: quicksort, quickselect, Dutch flag, merge sort on a list
    - 19.18 Math & geometry
    - 19.19 Common pitfalls
    - 19.20 How to talk through a solution
    - Chapter TL;DR

20. Chapter 20 — Pythonic Idioms, Gotchas and Interview Q&A
    - 20.1 Pythonic idioms
    - 20.2 Top gotchas — demo and fix
    - 20.3 Python interview Q&A
    - Chapter TL;DR
21. Chapter 21 — Cheat Sheets and Glossary
    - 21.1 Syntax and control flow (§1.9–1.11, Ch 3)
    - 21.2 Types, numbers and operators (Ch 2)
    - 21.3 Data structures and big-O (§5.3, §6.2, §6.12, §17.5, §19.1)
    - 21.4 String methods and formatting (Ch 4)
    - 21.5 List, tuple, dict and set methods (Ch 5, 6)
    - 21.6 Comprehensions, iterators and generators (§5.6, §6.5, Ch 11)
    - 21.7 Functions and arguments (Ch 7)
    - 21.8 OOP and dunder methods (Ch 10, §12.4–12.6)
    - 21.9 Exceptions (Ch 9)
    - 21.10 Files, paths and data formats (§9.8–9.11, §14.6)
    - 21.11 `itertools`, `functools`, `collections` (§6.9, §7.6, §11.5, §12.2)
    - 21.12 Decorators, context managers, metaprogramming (Ch 12)
    - 21.13 Type hints (Ch 13)
    - 21.14 Standard-library quick picks (Ch 14)
    - 21.15 Modules, packages and environments (Ch 8)
    - 21.16 Testing and debugging (Ch 15)
    - 21.17 Concurrency: which tool? (Ch 16)
    - 21.18 Performance and internals (Ch 17)
    - 21.19 numpy, pandas, matplotlib essentials (Ch 18)
    - 21.20 Interview patterns and gotchas (Ch 19, 20)
    - 21.21 Which tool for which job
    - 21.22 Glossary (A–Z)
    - Chapter TL;DR

Chapter 1 — Getting Started with Python

This chapter takes you from "never run Python" to "I understand what happens when I press Enter". You will install Python, run code two different ways, see how the CPython interpreter turns your text into something a machine executes, and learn the small set of layout rules (indentation, lines, quotes, comments) that every later chapter relies on.

What you'll learn

#	Topic	You will be able to…
1.1	Who this book is for and how it is organised	pick the right path through the chapters
1.2	What Python is, its history and design traits	explain Python in one minute to anyone
1.3	Where Python is used	name real products built with it
1.4	Installing Python and virtual environments	set up a clean project on any OS
1.5–1.6	REPL vs scripts, IDEs	choose the right tool for experimenting vs building
1.7	Your first program	write, run and understand `print("Hello, World!")`
1.8	How CPython runs code	describe source → bytecode → virtual machine
1.9	Basic syntax	identifiers, indentation, line continuation, quotes, blank lines, `input()`, `;`
1.10	Comments and docstrings	document code the way Python tools expect
1.11	PEP 8 and PEP 20	write code that looks like professional Python

All code lives in `code/` and was run with CPython 3.14. Run any file with `python code/<file>.py`.

1.1 Who this book is for, and how it is organised

What it is

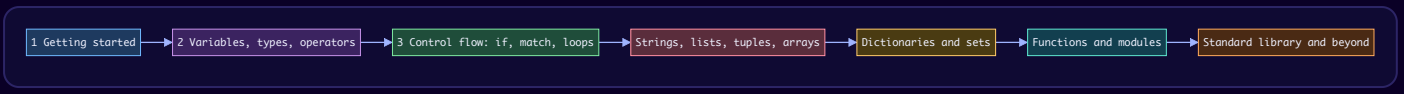
A single, self-contained path through core Python — from first program to intermediate topics — written so you never need a second source.

Who should read it

Reader	What you get
Complete beginners (students, career changers)	A first language taught from zero, with every term defined on first use
Developers from Java, C#, JavaScript, Go…	The "Python way" of doing things you already know, plus the traps that differ from your language
Data analysts and scientists	A solid language base before NumPy, pandas and scikit-learn
Teachers and trainers	Ready-made explanations, examples and self-check questions
Hobbyists and automators	Enough Python to script your own tasks and projects

Prerequisites: a computer (Windows, macOS or Linux), the ability to install software, curiosity, and willingness to type code yourself rather than only read it. No maths beyond arithmetic is needed in these early chapters.

How the book is organised



Each chapter builds on the previous one. Every topic follows the same rhythm: *what it is → why it matters → how it works → code with real output → trade-offs → common mistakes → real-world use → interview questions → self-check → TL;DR*. Skim the TL;DR boxes to review; do the self-checks to confirm you actually learned it.

1.2 What Python is

What it is

Python is a **high-level, general-purpose programming language**. *High-level* means you describe *what* to do (add these numbers, sort this list) without managing memory addresses or CPU registers. *General-purpose* means it is not tied to one domain: the same language builds websites, trains neural networks and renames files.

Think of it as a very capable assistant who speaks almost-English: you write `for name in guests: print(name)` and it does exactly that.

Python is **multi-paradigm**: you can write it as a list of steps (procedural), organise it into classes and objects (object-oriented), or compose small functions (functional). You do not have to pick one.

A short history

Year	Event
Dec 1989	Guido van Rossum starts Python at CWI (the Dutch national maths and computer science institute) as a successor to the ABC teaching language. The name comes from the comedy group *Monty Python*, not the snake.
1991	First public release, 0.9.0.
2000	Python 2.0: list comprehensions and a garbage collector that can reclaim reference cycles.
2008	Python 3.0: a deliberate clean-up that broke backward compatibility (e.g. `print` became a function, text became Unicode by default).
2018–19	Guido steps down as "Benevolent Dictator For Life" (July 2018); since 2019 a five-person **Steering Council**, elected by the core developers, governs the language.
2020	Python 2 reaches end of life (1 January 2020). Only Python 3 is maintained.
2020 → now	One feature release per year, every October (since 3.9; PEP 602) (3.10 added `match`; 3.11 made CPython markedly faster; 3.13 added a new interactive shell and experimental free-threading; 3.14 is current).

Language changes are proposed in **PEPs** – *Python Enhancement Proposals* – numbered design documents. You will meet PEP 8 (style) and PEP 20 (philosophy) in this chapter.

Why it matters — the design traits

Trait	What it means in practice	The honest caveat
Readable syntax	Indentation shows structure; keywords are English words (`and`, `not`, `in`).	Whitespace errors become syntax errors.
Dynamically typed	You never declare a variable's type; each *value* carries its type, checked while the program runs.	Type mistakes surface at run time unless you add type hints and a checker such as mypy.
Strongly typed	Python refuses to silently mix unrelated types: `"3" + 4` is an error, not `"34"` or `7`.	You convert explicitly (Chapter 2).
Automatic memory management	Objects are freed when nothing refers to them (reference counting plus a cycle collector). You never call `free()`.	Less control over memory layout than C.
"Interpreted"	You run source files directly — no separate compile step for you. Internally CPython *does* compile to bytecode first (see §1.8).	Pure-Python loops are slower than C/Java; heavy numeric work goes to libraries written in C.
Extensible	Hot spots can be written in C, C++ or Rust and imported like normal modules (this is how NumPy is fast).	Adds build complexity.
Embeddable	A C/C++ program can host the interpreter to give users a scripting language (Blender, GIMP, many games).	–
"Batteries included"	A large standard library: files, JSON, HTTP, dates, maths, testing, databases (SQLite). Plus 500 000+ third-party packages on PyPI.	Quality of third-party packages varies.

Common mistakes

- Believing "interpreted" means "reads one line, runs it, reads the next". The whole file is compiled first; a syntax error on line 200 stops line 1 from ever running (proved in §1.8).
- Confusing *dynamic* with *weak* typing. Python is dynamic **and** strong.
- Learning Python 2 material. Anything using `print "hi"` (no parentheses) is obsolete.

In the interview

> **30-second answer:** "Python is a high-level, dynamically but strongly typed, garbage-collected, multi-paradigm language. The reference implementation, CPython, compiles source to bytecode and runs it on a stack-based virtual machine. It trades raw speed for developer productivity and pushes heavy computation into C extensions like NumPy."

- *Is Python compiled or interpreted?* → Both: compiled to bytecode, then the bytecode is interpreted by a VM. "Interpreted" describes the user experience.
- *Why is Python slow, and how do people cope?* → Every operation goes through dynamic type checks in the VM. Teams vectorise with NumPy, use C/Rust extensions, use PyPy (a JIT-compiling implementation), or rely on CPython's own speed-ups since 3.11.
- *What is a PEP?* → A design document proposing a change or standard; e.g. PEP 8 is the style guide.

Self-check

1. Is `"3" + 4` allowed? → No, `TypeError`: Python is strongly typed.
2. Who governs Python today? → The elected five-member Steering Council.
3. What does "dynamically typed" mean? → Types belong to values and are checked at run time; names have no fixed type.
4. When did Python 2 stop being supported? → 1 January 2020.

TL;DR

Python = readable, dynamic-but-strong, garbage-collected, batteries-included. Created by Guido van Rossum (1989–91); Python 3 (2008) is the only live version; changes go through PEPs.

1.3 Where Python is used

What it is

Python's niche is "everywhere glue is needed plus everywhere a mature library exists". The table pairs each area with the key libraries and verified real-world users.

Area	Key tools	Real-world examples
Web back-ends	Django (batteries-included: ORM, admin, auth), Flask (minimal), FastAPI (async, type-hint driven)	Instagram runs one of the largest Django deployments; Reddit was rewritten in Python in 2005; Pinterest and Mozilla sites use Django/Flask
Data analysis	pandas (tables and time series), NumPy (fast arrays), Jupyter notebooks	Finance, analytics and research teams worldwide; Jupyter is the default notebook tool in data science
Machine learning & AI	scikit-learn, PyTorch, TensorFlow/Keras, Hugging Face Transformers	PyTorch (Meta) and TensorFlow (Google) expose Python as their main interface; most LLM tooling ships Python SDKs first
Scientific computing	SciPy, NumPy, Matplotlib, Astropy	NumPy was used in the first black-hole image (Event Horizon Telescope) and in LIGO's gravitational-wave detection
Automation & scripting	standard library (`pathlib`, `subprocess`, `shutil`), Ansible	Sysadmin scripts, CI pipelines, Ansible itself is written in Python
Networking	`socket`, `asyncio`, Twisted, `requests` / `httpx`	Network tooling, scrapers, async servers
Game development & scripting	Pygame (2-D games), embedded Python in engines	*EVE Online* uses Stackless Python on its servers; *Civilization IV* used Python for game logic; Blender and Maya expose Python scripting APIs
Desktop GUIs	Tkinter (ships with Python), PyQt/PySide, wxPython	Internal tools, utilities, the Python IDLE editor itself (Tkinter)
Education	IDLE, Thonny, turtle graphics	Widely used as a first language in schools and universities (e.g. MIT 6.100L teaches Python)
IoT & embedded	MicroPython, CircuitPython	Runs on microcontrollers such as the Raspberry Pi Pico and ESP32 to read sensors and talk to networks

Why it matters

Choosing a language is mostly choosing an ecosystem. Python wins where libraries matter more than raw speed: data, ML, scripting, web APIs.

When not to use it

Situation	Better fit	Why
Hard real-time or very latency-sensitive systems (trading engines, game engines' inner loops)	C, C++, Rust	Predictable performance, no interpreter overhead
Browser front-end code	JavaScript / TypeScript	Browsers run JavaScript natively (Python in the browser via WebAssembly exists but is niche)
Native mobile apps	Swift, Kotlin	Platform SDKs and tooling
CPU-bound multi-threaded code (on standard CPython)	Go, Rust, Java — or Python with multiprocessing / free-threaded builds	The Global Interpreter Lock (GIL) lets only one thread run Python bytecode at a time in the default build

Self-check

1. Name the two main Python web frameworks and their difference. → Django: full-featured with ORM/admin; Flask: minimal core, add what you need.
2. Why is Python fast enough for ML if Python itself is slow? → The heavy maths runs in C/C++/CUDA inside NumPy/PyTorch; Python only orchestrates.
3. Which Python variant runs on microcontrollers? → MicroPython (and its fork CircuitPython).

TL;DR

Python dominates data, ML, scripting and web back-ends because of its libraries; avoid it for hard real-time, browser front-ends and native mobile.

1.4 Installing Python and creating a project

What it is

To run Python you need an **interpreter** — the program that reads and executes Python code. The official one is **CPython** (written in C), downloaded from python.org. Other implementations exist (PyPy, MicroPython, GraalPy), but "Python" in this book means CPython.

How it works — install per OS

OS	Recommended	Check it worked
Windows	Installer from python.org (tick **"Add python.exe to PATH"**), or the Microsoft Store package. Windows also installs the `py` launcher, which picks a version: `py -3.14`.	`py --version` or `python --version`
macOS	Installer from python.org, or Homebrew: `brew install python`. Do not rely on any old system copy.	`python3 --version`
Linux	Your package manager, e.g. `sudo apt install python3 python3-venv`. **Never remove** the system Python — the OS uses it.	`python3 --version`

On macOS/Linux the command is usually `python3` (plain `python` may not exist or may point elsewhere). Inside an activated virtual environment, `python` always means that environment's interpreter.

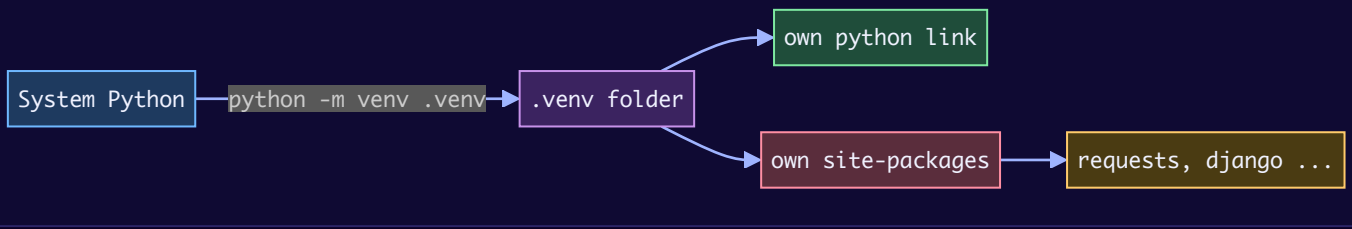
Virtual environments (intro)

What it is: *a virtual environment* ("venv") is a private folder containing its own `python` executable link and its own `site-packages` directory for installed libraries. Analogy: each project gets its own toolbox, so upgrading a hammer for project A doesn't break project B.

The problem it solves: project A needs `Django 4`, project B needs `Django 5`. Installing both globally is impossible; installing into the system Python can break OS tools. Modern Linux distributions and Homebrew even refuse (`error: externally-managed-environment`, PEP 668) to let `pip` install globally.

How it works:

1. Create: `python3 -m venv .venv` (on Windows: `py -m venv .venv`).
2. Activate: `source .venv/bin/activate` (macOS/Linux) or `.venv\Scripts\activate` (Windows). Your prompt shows `(.venv)`.
3. Install packages: `python -m pip install requests`. They go into `.venv/…/site-packages`, nowhere else.
4. Record them: `python -m pip freeze > requirements.txt`; a teammate recreates with `python -m pip install -r requirements.txt`.
5. Leave: `deactivate`. Delete the environment by deleting the `.venv` folder.



`python -m pip` (rather than bare `pip`) guarantees you install into the interpreter you are actually running. Faster all-in-one tools such as **uv** and **Poetry** manage venvs and dependencies for you; they build on the same idea.

Common mistakes

- Forgetting to tick "Add to PATH" on Windows → `'python' is not recognized`.
- Installing packages globally with `sudo pip` → breaks OS tools. Use a venv.
- Committing `.venv/` to Git. Commit `requirements.txt` (or `pyproject.toml`) instead; add `.venv/` to `.gitignore`.
- Running `pip` from one Python and `python` from another → "I installed it but import fails". Use `python -m pip`.

In the interview

30-second answer: "I create one virtual environment per project with `python -m venv .venv`, activate it, install with `python -m pip`, and pin dependencies in `requirements.txt` or `pyproject.toml`. That isolates versions between projects and keeps the system interpreter clean."

- *Where do packages go in a venv?* → Its own `site-packages` directory; `sys.prefix` points to the venv.
- *Does a venv copy the whole interpreter?* → Typically it links to (or copies) the base interpreter binary and adds its own `site-packages`; the standard library is shared.

Self-check

1. Command to create a venv named `.venv`? → `python -m venv .venv`.
2. How do you know it is active? → Prompt shows `(.venv)`; `python -c "import sys; print(sys.prefix)"` points into it.
3. Why `python -m pip` instead of `pip`? → It runs pip for exactly the interpreter you call.

TL;DR

Install CPython from python.org (or your OS package manager), then give every project its own `.venv` and install with `python -m pip`.

1.5 Two ways to run code: the REPL and scripts

What it is

- **REPL** (*Read–Eval–Print Loop*): type `python3` with no arguments. You get a `>>>` prompt; each line you type is read, evaluated, and its result printed, then it loops. Continuation lines show `...`.
- **Script**: a text file ending in `.py` that you run with `python3 hello.py`. Nothing is echoed automatically — you must `print()`.

```
$ python3
>>> 2 + 3
5
>>> _ * 10          # _ holds the last printed result (REPL only)
50
>>> name = "Ada"
>>> name            # a bare expression is echoed in the REPL...
'Ada'
>>> exit()          # or Ctrl-D (macOS/Linux), Ctrl-Z then Enter (Windows)
```

Since Python 3.13 the default REPL supports colour, multi-line editing, and `exit` / `quit` without parentheses. Use `help(len)` inside the REPL for documentation and `help()` for an interactive help browser.

Ways to run Python

Command	What it does	Use it for
`python3`	Start the REPL	Quick experiments
`python3 script.py`	Run a file	Real programs
`python3 -i script.py`	Run a file, then drop into the REPL with its variables	Debugging / exploring results
`python3 -m module`	Run a module found on the import path	Tools: `-m venv`, `-m pip`, `-m http.server`, `-m pytest`
`python3 -c "print(1+1)"`	Run a one-line program	Shell one-liners
Jupyter notebook	Cells of code + output + text	Data exploration, teaching

On macOS/Linux a script can start with a **shebang** line `#!/usr/bin/env python3`; after `chmod +x script.py` you can run `./script.py` directly.

The `if __name__ == "__main__":` guard

Every module has a variable `__name__`. When a file is **run directly** it is `"__main__"`; when it is **imported** it is the module's name. Putting start-up code under the guard lets a file be both a runnable script and an importable library — every script in this book does it.

Trade-offs

	REPL	Script
Feedback	Instant	After each run
Keeps your work	No (history only)	Yes, in a file you can version
Auto-prints values	Yes	No – use `print()`
Best for	Trying one idea	Anything you'll run twice

Common mistakes

- Typing `python3 hello.py` *inside* the `>>>` prompt → `SyntaxError`. That is a shell command; exit the REPL first.
- Expecting a bare expression like `2 + 3` in a script to print something. Only the REPL echoes.
- Naming your file after a standard module (`random.py`, `email.py`) – `import random` then imports *your* file.

Self-check

1. What does `_` hold in the REPL? → The last echoed result.
2. How do you run a file and then inspect its variables? → `python3 -i file.py`.
3. What is `__name__` when a file is imported? → The module's name, not `"__main__"`.

TL;DR

REPL for trying, scripts for keeping. Guard start-up code with `if __name__ == "__main__":`.

1.6 IDEs and editors

What it is

An **IDE** (*Integrated Development Environment*) combines an editor with running, debugging, autocompletion and error checking. You can write Python in any text editor, but tooling catches mistakes before you run.

Tool	Best for	Notes
VS Code + Python extension	Most people	Free; Pylance gives autocompletion and type checking; integrated debugger and Jupyter
PyCharm	Large projects	Deep refactoring and inspections; core features are free, advanced ones need a paid Pro subscription
Jupyter / JupyterLab	Data analysis, teaching	Cells, inline charts; not ideal for large applications
IDLE	First hour	Ships with Python; simple REPL + editor
Thonny	Absolute beginners	Shows variables and step-by-step execution visually
Vim / Neovim / Emacs / Zed / Sublime	Keyboard-driven developers	Add a language server (Pyright, pylsp) for smart features

Add three tools to any editor: a **formatter** (Black or Ruff format – rewrites layout to PEP 8), a **linter** (Ruff or Flake8 – flags bugs and style issues), and optionally a **type checker** (mypy or Pyright).

Common mistakes

- The editor uses a different interpreter than your venv → imports "missing". Select the `.venv` interpreter in the IDE.
- Hand-formatting code. Let the formatter do it on save.

TL;DR

VS Code or PyCharm for projects, Jupyter for data, IDLE/Thonny to start; always pair with a formatter and linter.

1.7 Your first program

What it is

The traditional first program prints a greeting. It proves your installation works and introduces two ideas: a **function call** and a **string**.

```
# Print "Hello, World!" to the console
print("Hello, World!")
```

- `print` is a **built-in function** — always available, no import needed. Calling it means writing its name followed by parentheses containing **arguments**.
- `"Hello, World!"` is a **string** (type `str`): a sequence of characters between quotes.
- `print` writes the text to **standard output** (the console) and then a newline.
- The `# …` line is a **comment**; Python ignores it.

`print` has two handy keyword arguments: `sep` (what goes between arguments, default a space) and `end` (what goes after, default `"\n"`). From `code/first_program.py`:

```
print("Hello, World!")
print("Python", sys.version_info.major, sys.version_info.minor, sep=".", end="  <- running version\n")
```

Output:

```
Hello, World!
Python.3.14  <- running version
```

Common mistakes

- `Print("hi")` → `NameError`: Python is case-sensitive.
- `print "hi"` → `SyntaxError` (Python 2 syntax).
- Mismatched or curly quotes pasted from a word processor (`“hi”`) → `SyntaxError`.

Self-check

1. What type is `"Hello, World!"`? → `str`.
2. How do you print `a-b-c` from three arguments? → `print("a", "b", "c", sep="-")`.
3. How do you print without a trailing newline? → `print(x, end="")`.

TL;DR

`print(...)` sends text to the console; text in quotes is a `str`; everything is case-sensitive.

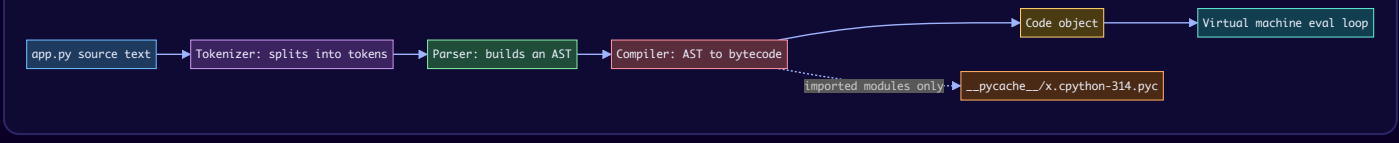
1.8 How CPython runs your code

What it is

When you type `python3 app.py`, CPython does **not** execute your text line by line. It first translates the *entire file* into **bytecode** — compact instructions for an imaginary, stack-based computer — and then its **virtual**

machine (VM) executes those instructions. Analogy: a translator first converts a whole speech into shorthand notes (bytecode), then reads the notes aloud (VM).

How it works



1. **Tokenize** – the text is split into tokens: names, numbers, operators, `INDENT` / `DEDENT` markers. Comments are discarded here.
2. **Parse** – tokens are arranged into an **Abstract Syntax Tree (AST)**, a tree describing the program's structure. Since 3.9 CPython uses a PEG parser (PEP 617). Syntax errors are found here – for the whole file, before anything runs.
3. **Compile** – the AST becomes a **code object**: bytecode plus tables of constants and names.
4. **Cache** – for *imported* modules, the code object is saved as a `.pyc` file in `__pycache__/` so the next import can skip steps 1–3 if the source hasn't changed. The main script you run directly is not cached.
5. **Execute** – the VM's evaluation loop reads each instruction, pushes and pops values on a stack, and calls C code to do the real work. Since 3.11 the VM *specialises* hot instructions for the types it actually sees (PEP 659), one reason 3.11+ is noticeably faster than 3.10.

From `code/how_cpython_runs.py`:

```
SOURCE = 'price = 20\nqty = 3\ntotal = price * qty\nprint("total =", total)\n'
code = compile(SOURCE, "<demo>", "exec")       # steps 1-3
dis.dis(code)                                  # show the bytecode
exec(code, {})                                 # step 5
```

Output (CPython 3.14 – instruction names change between versions):

```
implementation: cpython 3.14.7
code object: code | constants: (20, 'total =', None) | names: ('price', 'qty', 'total', 'print')
--- bytecode ---
  1           LOAD_SMALL_INT          20
              STORE_NAME               0 (price)
  2           LOAD_SMALL_INT           3
              STORE_NAME               1 (qty)
  3           LOAD_NAME                0 (price)
              LOAD_NAME                1 (qty)
              BINARY_OP                5 (*)
              STORE_NAME               2 (total)
  4           LOAD_NAME                3 (print)
              PUSH_NULL
              LOAD_CONST               1 ('total =')
              LOAD_NAME                2 (total)
              CALL                     2
              ...
--- run ---
total = 60
SyntaxError at line 2 - so line 1 never ran
cached bytecode file: __pycache__/shop.cpython-314.pyc
cache tag for this interpreter: cpython-314
```

Read line 3 of the bytecode: push `price`, push `qty`, `BINARY_OP *` pops both and pushes the product, `STORE_NAME` pops it into `total`. That is a stack machine.

Other implementations

Implementation	Runs on	Notable for
CPython	C	The reference; what python.org ships
PyPy	RPython, JIT-compiled	Often several times faster on long-running pure-Python code
MicroPython	Microcontrollers	Tiny memory footprint
GraalPy, Jython	JVM	Java interoperability

CPython itself added an experimental JIT compiler (3.13, off by default) and an optional **free-threaded** build without the GIL (experimental in 3.13, officially supported but still optional in 3.14).

Trade-offs

Bytecode + VM makes Python portable (the same `.py` runs on any OS) and quick to start, at the cost of per-instruction overhead versus native machine code.

Common mistakes

- Thinking `.pyc` files hide or protect source code — they are trivially decompiled and version-specific.
- Committing `__pycache__/` to Git. Ignore it.
- Expecting a syntax error late in the file to let earlier lines run.

In the interview

> **30-second answer:** "CPython tokenizes and parses the whole file into an AST, compiles it to bytecode in a code object, caches bytecode for imported modules in `__pycache__`, and then a stack-based virtual machine executes the bytecode. Since 3.11 it specialises instructions based on observed types."

- *What's in a `.pyc`?* → A header (magic number identifying the bytecode version, plus source timestamp/size or hash) followed by the serialised code object.
- *What is the GIL?* → A lock in the default CPython build that allows only one thread to execute bytecode at a time; it simplifies memory management but limits CPU-bound threading.
- *How can you see bytecode?* → The `dis` module: `dis.dis(func)`.

Self-check

1. Name the stages from source to execution. → Tokenize → parse (AST) → compile (bytecode/code object) → execute on the VM.
2. Is your main script cached as `.pyc`? → No, only imported modules.
3. Why can a syntax error on line 50 prevent line 1 from running? → The whole file is parsed and compiled before execution starts.

TL;DR

Source → tokens → AST → bytecode → VM. "Interpreted" really means "compiled to bytecode, then interpreted".

1.9 Basic syntax

Identifiers

An **identifier** is a name you give to a variable, function, class or module.

Rules (enforced — breaking them is a `SyntaxError`):

1. Starts with a letter or underscore `_`; then any letters, digits, underscores.
2. No spaces or symbols like `-`, `@`, `$`, `!`, `#`, `%`.
3. Case-sensitive: `total`, `Total`, `TOTAL` are three different names.

4. Any length.
5. Not a **keyword** (reserved word). Python 3.14 has 35 keywords (`if`, `for`, `class`, `True`, `None`, …) plus four **soft keywords** – `match`, `case`, `type`, `_` – that are special only in certain positions, so `match = 3` is still legal.
6. Unicode letters are allowed (`café = 1` works), but stick to ASCII for shared code.

From `code/first_program.py`, which tests names with `str.isidentifier()` and `keyword.iskeyword()`:

```
'total'      valid identifier? True
'_private'   valid identifier? True
'user2'      valid identifier? True
'2user'      valid identifier? False
'user-name'  valid identifier? False
'class'      valid identifier? False
'café'       valid identifier? True
```

Lines and indentation

What it is: Python marks a **block** (the body of an `if`, loop, function, class) by *indentation*, not braces `{}`. A line ending in `:` opens a block; every line in it must be indented by the same amount; the block ends when indentation returns to the outer level.

```
if temperature > 30:
    label = "hot"      # inside the if-block
else:
    label = "mild"     # inside the else-block
print(label)           # back at the outer level: always runs
```

Rules:

- PEP 8 says: **4 spaces per level**, never tabs. Configure your editor to insert spaces.
- Mixing tabs and spaces inconsistently is a `TabError` in Python 3.
- An unexpected indent is `IndentationError: unexpected indent`; a missing one after `:` is `IndentationError: expected an indented block`.
- An empty block is illegal – use `pass` as a placeholder (Chapter 3).

Why it matters: the layout you see *is* the structure the computer sees. In brace languages, misleading indentation can hide bugs (a famous example is Apple's 2014 "goto fail" SSL bug, where a duplicated, indented line looked like it belonged to an `if`). In Python that mismatch cannot exist.

Multi-line statements

A statement normally ends at the end of the line. To continue it:

Method	Example	Verdict
Implicit – inside `()`, `[]`, `{}`	`total = (1` ⏎ `+ 2` ⏎ `+ 3)`	Preferred by PEP 8
Explicit – backslash `\` at line end	`total = 1 + \` ⏎ `2 + \` ⏎ `3`	Works, but a stray space after `\` breaks it

Both produce `6` in `line_continuation_demo()`. PEP 8 also suggests breaking long expressions *before* binary operators (`+ 2` at the start of the next line), which lines operators up for the reader.

Quotation marks

Form	Use	Example
`'...'`	Strings containing `"`	`'It said "hi"'`
`"..."`	Strings containing `'`	`"It's sunny"`
`'''...'''` or `"""..."""`	Text spanning lines; docstrings	`"""Line one` ⏎ `Line two"""`

Single and double quotes are identical in meaning — pick one style and stay consistent (formatters like Black standardise on `"`). Output of `quotes_demo()`:

```
It said "hi" | It's sunny | ['Line one', 'Line two']
```

Blank lines

Blank lines are ignored by the interpreter (inside the REPL, a blank line ends a multi-line block). Use them to separate logical steps. PEP 8: **two** blank lines around top-level functions and classes, **one** between methods in a class.

Reading user input

`input(prompt)` prints the prompt, waits for the user to press Enter, and returns what they typed **as a `str`** — **always**, even if they typed digits.

```
user_name = input("Please enter your name: ")
print("Hello, " + user_name + "!")
```

The runnable script replaces `input()` with a stand-in so it runs unattended; the output is what a user typing `Ada` would see:

```
Please enter your name: Ada
Hello, Ada!
```

To do maths with input, convert it: `age = int(input("Age: "))` (Chapter 2 covers conversion and its errors).

Several statements on one line

A semicolon `;` separates statements on a single line:

```
import sys; x = "foo"; sys.stdout.write(x + "\n")      # prints: foo
```

It is legal, but PEP 8 discourages it: one statement per line is easier to read, diff and debug. (`sys.stdout.write` is the lower-level cousin of `print`: it writes exactly the string you give it, with no automatic newline.)

Common mistakes

- Forgetting the colon: `if x > 3` → `SyntaxError: expected ':'`.
- Copying code from the web that uses tabs into a file that uses spaces.
- Treating `input()` as a number: `input() + 1` → `TypeError`.
- Using `-` in names (`user-name` means "user minus name").

In the interview

> **30-second answer:** "Python uses indentation to define blocks — four spaces per PEP 8 — so visual structure equals logical structure. Statements end at the newline unless you're inside brackets, which is the preferred way to continue a line. Identifiers are case-sensitive and can't be keywords."

- *What's a soft keyword?* → A word that's only reserved in a specific context, like `match`/`case` in a match statement, so old code using `match` as a variable still works.
- *Why prefer brackets over backslashes?* → A backslash fails silently if followed by whitespace, and brackets let editors auto-indent.

Self-check

1. Is `my-var` a valid identifier? → No; `-` is subtraction.
2. What does `input()` return if the user types `42`? → The string `"42"`.

3. What error do you get for inconsistent tabs and spaces? → `TabError`.
4. How many spaces per indentation level does PEP 8 recommend? → 4.

TL;DR

Names: letters/digits/underscore, not starting with a digit, not keywords, case-sensitive. Blocks = `:` + 4-space indent. Continue lines inside brackets. `input()` returns `str`. Avoid `;`.

1.10 Comments and docstrings

What it is

- A **comment** starts with `#` and runs to the end of the line. The tokenizer throws it away; it never reaches the bytecode. It is a note for humans.
- A **docstring** is a string literal placed as the *first statement* of a module, function, class or method. Unlike a comment, it is kept at run time in the object's `__doc__` attribute, and tools (`help()`, IDE tooltips, Sphinx documentation generators) read it.

Single-line and inline comments

```
# A comment explains WHY; the code already says WHAT.
return width * height  # inline comment: two spaces before the '#'
```

PEP 8: start with `# ` (hash + space); for inline comments put at least two spaces before `#`; keep comments up to date — a wrong comment is worse than none.

"Multi-line comments"

Python has **no** dedicated multi-line comment syntax. Two options:

1. **Consecutive `#` lines** — the correct way; every editor can toggle them with one shortcut.
2. **A triple-quoted string not assigned to anything** — often called a "multi-line comment". Technically it is an *expression statement*: the string is a value that is created and immediately discarded (CPython usually optimises it away). It "works", but it is not a comment: if it happens to be the first statement in a function it silently becomes the docstring, and it must obey indentation rules like any other statement. Prefer `#` lines; reserve triple quotes for docstrings.

Docstrings

From `code/comments_docstrings.py`:

```
def area(width: float, height: float) -> float:
    """Return the area of a rectangle.

    Args:
        width: size along x, in metres.
        height: size along y, in metres.
    """
    return width * height
```

Output:

```
module doc first line: Chapter 1 - comments vs docstrings, PEP 20, and keywords.
function doc first line: Return the area of a rectangle.
area(3, 4) = 12
```

Conventions (PEP 257): triple double quotes; a one-line summary in the imperative ("Return…", not "Returns…"), ending with a period; for longer docstrings a blank line then details. Popular layouts for the details are

Google style (shown above), NumPy style and reStructuredText.

Comments vs docstrings

	`#` comment	Docstring
Kept at run time?	No	Yes, in `__doc__`
Audience	Maintainers reading the code	Users of the function/module (via `help()`, IDEs)
Content	*Why* a tricky line exists	*What* the object does, its arguments, return value, errors
Position	Anywhere	First statement of module/class/function

Common mistakes

- Comments that repeat the code (`x += 1 # add one to x`). Explain intent instead.
- Leaving commented-out code blocks around – that's what version control is for.
- Writing a docstring after the first statement (it's then just an ignored string).

In the interview

> **30-second answer:** "Comments start with `#` and are discarded by the tokenizer; they explain *why*. Docstrings are the first string literal in a module, class or function, stored in `__doc__`, and describe *what* the object does for its users. Python has no block-comment syntax; use several `#` lines."

- *Is a triple-quoted string a comment?* → No, it's a string expression that's evaluated and discarded – unless it's first in a body, then it's a docstring.
- *How do you read a docstring at runtime?* → `obj.__doc__` or `help(obj)`.

Self-check

1. Where must a docstring go? → First statement in the module/class/function body.
2. Do comments slow down your program? → No; they're removed during tokenizing.
3. Proper way to comment out five lines? → Put `#` at the start of each (editor shortcut).

TL;DR

`#` = note for maintainers, gone at run time. Docstring = first-statement string, kept in `__doc__`, read by `help()` and IDEs. No real multi-line comments – use several `#` lines.

1.11 PEP 8 and PEP 20 – writing Pythonic code

What it is

- **PEP 8** is the official **style guide**: how Python code should *look*. Consistency lets any Python developer read any codebase.
- **PEP 20**, *The Zen of Python* by Tim Peters, is the language's **philosophy** in 19 short aphorisms. Type `import this` to print it.

PEP 8 essentials

Rule	Example
4 spaces per indent, no tabs	`····return x`
Lines ≤ 79 characters (72 for comments/docstrings); many teams relax to 88–100	–
`snake_case` for variables, functions, modules	`user_age`, `load_data()`
`CapWords` (CamelCase) for classes	`UserProfile`
`UPPER_SNAKE_CASE` for constants	`MAX_RETRIES = 3`
Leading `_` for "internal" names	`_cache`
Spaces around `=` in assignments and around comparisons; none inside brackets	`x = f(a, b)` not `x=f( a,b )`
No spaces around `=` in keyword arguments	`print(x, end="")`
Imports at the top, one per line, grouped: standard library, third-party, local	`import os` ⏎ `import sys`
Two blank lines around top-level functions/classes	–
Compare to `None` with `is`, not `==`	`if x is None:`
Don't compare booleans to `True`	`if ready:` not `if ready == True:`

PEP 8 itself says consistency within a project beats rigid rule-following. Automate it: **Black** or **Ruff** formats code; **Ruff** or **Flake8** lints.

PEP 20 – the aphorisms that matter most

From `code/comments_docstrings.py`, which captures the output of `import this`:

```
Zen of Python has 19 aphorisms, e.g.:
   Beautiful is better than ugly.
   Explicit is better than implicit.
   Simple is better than complex.
   Readability counts.
35 hard keywords, e.g. ['False', 'None', 'True', 'and', 'as', 'assert']
soft keywords (only special in some places): ['_', 'case', 'match', 'type']
```

How they show up in daily decisions:

Aphorism	In practice
Explicit is better than implicit	`from math import sqrt` rather than `from math import *`
Readability counts	Descriptive names; one statement per line
Errors should never pass silently	Don't write `except: pass`
There should be one obvious way to do it	Use `for item in items`, not index juggling
Flat is better than nested	Early returns instead of deep `if` pyramids (Chapter 3)
In the face of ambiguity, refuse the temptation to guess	Why `"3" + 4` raises instead of guessing

"**Pythonic**" means code that follows these idioms.

Common mistakes

- Arguing about style in code review – let the formatter decide.
- Using `camelCase` for functions (a Java habit).

In the interview

> **30-second answer:** "PEP 8 is the style guide – 4-space indents, snake_case functions, CapWords classes, 79-char lines, grouped imports – and I enforce it with Ruff or Black. PEP 20 is the Zen of Python; the ideas I lean on most are 'explicit is better than implicit', 'readability counts' and 'errors should never pass silently'."

- *What does "Pythonic" mean?* → Using the language's idioms (iteration, context managers, comprehensions) instead of translating another language line by line.

Self-check

1. Name style for a class holding user settings? → `UserSettings`.
2. How do you print the Zen of Python? → `import this`.
3. Which is PEP 8-correct: `if x == None` or `if x is None`? → `if x is None`.

TL;DR

PEP 8 = how code looks (automate it). PEP 20 = why Python feels the way it does. Together they define "Pythonic".

Chapter 1 TL;DR

- Python is a high-level, dynamically and strongly typed, garbage-collected language; CPython is the reference implementation; Python 3 is the only supported line.
- It's used for web back-ends, data, ML, science, automation, games scripting, GUIs, education and IoT – mostly because of its libraries.
- Install from python.org or your OS; give each project a `.venv`; install with `python -m pip`.
- REPL (`>>>`) for experiments, `.py` scripts for real work, `if __name__ == "__main__":` for start-up code.
- CPython: source → tokens → AST → bytecode (cached as `.pyc` for imports) → stack-based VM. The whole file is compiled before any line runs.
- Syntax: blocks by `:` + 4-space indentation; case-sensitive identifiers; continue lines inside brackets; `'` / `"` / `"""` strings; `input()` returns `str`; avoid `;`.
- `#` comments vanish at compile time; docstrings live in `__doc__`.
- Follow PEP 8 (enforced by a formatter) and the spirit of PEP 20.

Chapter 2 — Variables, Data Types and Operators

Every program stores data and computes with it. This chapter explains how Python stores data (names pointing at objects), which kinds of data exist (the built-in types), how numbers really behave (including why `0.1 + 0.2 != 0.3`), how to convert between types, and every operator family with its precedence. The mental model from §2.1 — *names are labels on objects* — is the single most important idea in the whole book: most "weird" Python behaviour stops being weird once you have it.

What you'll learn

#	Topic	Key ideas
2.1	Variables	creating, printing, deleting, case sensitivity, multiple assignment, naming rules and conventions, names vs objects
2.2	Identity, equality, mutability	`is` vs `==`, `id()`, mutable vs immutable, aliasing
2.3	Dynamic vs strong typing	why `x` can change type but `"3" + 4` fails
2.4	Built-in data types	text, numeric, sequence, mapping, set, boolean, binary, `None`
2.5	Numbers	arbitrary-precision `int`, `float` pitfalls, `Decimal`, `Fraction`, `complex`, `random`
2.6	Booleans and truthiness	`True` / `False`, falsy values, `bool` is an `int`
2.7	Type conversion	implicit widening, explicit `int()` / `float()` / `str()` …, failure cases
2.8	Operators	arithmetic, comparison, assignment/augmented, logical, bitwise, membership, identity, walrus `:=`, chained comparisons, precedence

Code: `code/` — every output below is real, from CPython 3.14.

2.1 Variables

What it is

A **variable** in Python is a **name that refers to an object**. An **object** is a chunk of memory holding a value, its **type**, and a reference count. The name itself stores no data — it is a label tied to an object. Analogy: objects are parcels in a warehouse; names are sticky labels. You can stick several labels on one parcel, move a label to a different parcel, or peel a label off — the parcel itself is unaffected.

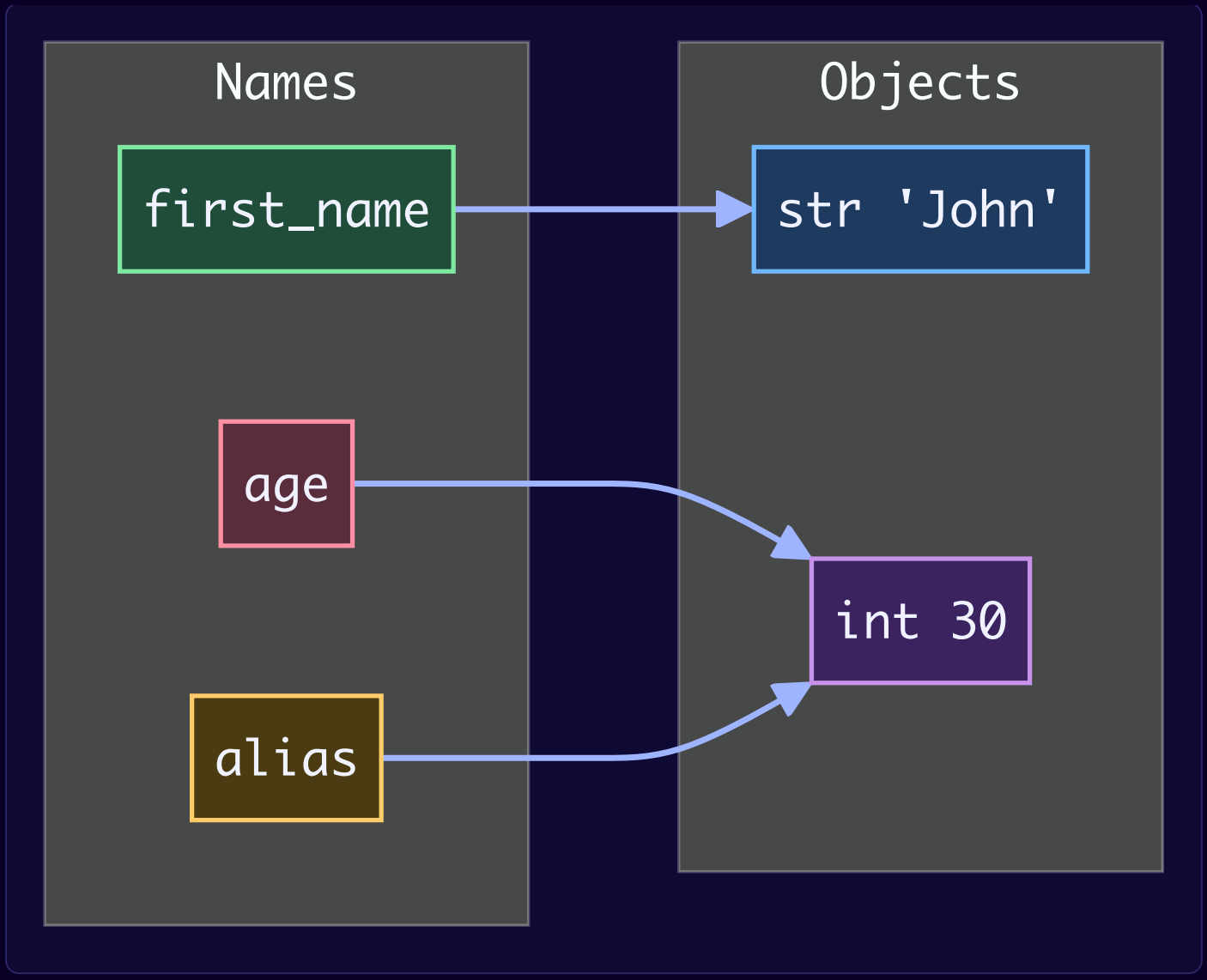
There is no declaration step: a variable comes into existence the first time you **assign** to it with `=`.

How it works — creating and printing

```
first_name = "John"      # name first_name -> str object "John"
age = 30                 # name age        -> int object 30
height = 5.11            # name height     -> float object 5.11
print(first_name, age, height)          # John 30 5.11
```

Assignment `name = expression` runs in two steps:

1. Evaluate the right-hand side → an object (created or found).
2. Bind the name on the left to that object (in the current **namespace** — a dictionary-like table mapping names to objects).



After `alias = age`, both names point at the same `int` object. No copy is made — ever — by plain assignment.

Multiple assignment

From `code/names_and_objects.py`:

```
first_name, age, height = "John", 30, 5.11   # tuple unpacking: one name per value
a = b = c = 0                                # chained: three names, ONE object
x, y = 1, 2
x, y = y, x                                  # swap: right side is built first, then unpacked
head, *rest = [10, 20, 30, 40]               # starred: rest collects the leftovers as a list
```

Output:

```
John 30 5.11
swap: 2 1 | chained: 0 0 0
head: 10 rest: [20, 30, 40]
```

Unpacking needs matching counts: `x, y = 1, 2, 3` raises `ValueError: too many values to unpack`. Beware `a = b = []` — both names share one list (see §2.2).

Deleting variables — `del`

`del name` removes the name from the namespace. The object is destroyed only when no references to it remain (CPython frees it immediately when its reference count hits zero).

```
player_score = 120
alias = player_score
del player_score
print(player_score)     # NameError - the name is gone
print(alias)            # 120 - the object survives, alias still refers to it
```

Output (the script runs the snippet inside a function, then again at module level with a name `player`):

```
UnboundLocalError (a NameError subclass, inside a function): cannot access local variable 'player_score' where
it is not associated with a value
object still reachable via alias: 120
NameError at module level: name 'player' is not defined
```

Using a deleted name **always** raises `NameError` (inside a function you see its subclass `UnboundLocalError`). If an online code playground seems to print nothing instead, it is simply not displaying the error stream — run the file locally and you will see the traceback. You rarely need `del` for plain variables; it's more common for removing dictionary keys (`del d["k"]`) or list items (`del items[0]`).

Case sensitivity

`Age`, `age` and `AGE` are three unrelated names:

```
Age/age/AGE: 25 30 35
```

Naming rules (enforced) and conventions (PEP 8)

Rules — break them and you get `SyntaxError`	Conventions — break them and reviewers complain
Start with a letter or `_`	`snake_case` for variables and functions: `user_age`
Then letters, digits, `_` only	`CapWords` for classes: `UserProfile`
Not a keyword (`class`, `if`, `None` …)	`UPPER_SNAKE_CASE` for constants: `MAX_USERS = 100`
Case-sensitive	Descriptive: `age` not `a`, `username` not `usrnm`
	Leading `_` = internal/private by convention
	Don't shadow built-ins: never name a variable `list`, `str`, `sum`, `id`, `input`

Valid: `username`, `_user_id`, `user2name`. Invalid: `2user` (starts with a digit), `user-name` (hyphen is minus), `class` (keyword).

Shadowing is legal but harmful: after `list = [1, 2]`, calling `list("abc")` fails with `TypeError: 'list' object is not callable`.

Common mistakes

- Assuming `b = a` copies a list. It doesn't; use `a.copy()`, `list(a)` or `a[:]` (shallow copy), or `copy.deepcopy(a)` for nested data.
- Shadowing built-ins (`list`, `dict`, `sum`, `max`, `id`, `type`).
- Using a variable before assigning it → `NameError`.

In the interview

30-second answer: "Python variables are names bound to objects, not boxes holding values. Assignment never copies; it binds a name to the object on the right. `del` unbinds a name, and CPython frees the object when its reference count reaches zero."

- *What happens in* `a = b = []`*?* → One list, two names; appending via `a` is visible via `b`.
- *How does swap* `x, y = y, x` *work?* → The right side builds a tuple `(y, x)` first, then unpacks it into the names.
- *Does* `del x` *free memory?* → It removes one reference; the object is freed only if that was the last one.

Self-check

1. After `a = [1]; b = a; b.append(2)`, what is `a`? → `[1, 2]`.
2. Is `_total2` a valid name? `2_total`? → Yes; no.
3. What exception do you get using a deleted name? → `NameError`.
4. Why is naming a variable `str` a bad idea? → It hides the built-in `str` type in that scope.

TL;DR

Variables are labels on objects; `=` binds, never copies; `del` removes a label; names are case-sensitive, snake_case, and must not be keywords or shadow built-ins.

2.2 Identity, equality and mutability

What it is

Every object has three things:

Property	Question it answers	How to see it	Can it change?
Identity	*Which* object is this?	`id(obj)` (in CPython, the memory address)	Never, for the object's lifetime
Type	*What kind* of object?	`type(obj)`	No (in normal code)
Value	*What* does it contain?	`==`, `print`	Only if the type is **mutable**

- `a == b` – **equality**: do they have the same value? (calls `a.__eq__(b)`)
- `a is b` – **identity**: are they the very same object? (compares `id`s; cannot be overridden)

How it works

```
list_a = [1, 2, 3]
list_b = [1, 2, 3]      # a second, separate list with equal contents
list_c = list_a         # another name for the first list
```

```
a == b: True | a is b: False | a is c: True
id(a) == id(c): True
```

Mutable types can change their value in place: `list`, `dict`, `set`, `bytearray`, most user-defined classes. **Immutable** types cannot: `int`, `float`, `complex`, `bool`, `str`, `tuple`, `frozenset`, `bytes`, `None`. "Changing" an immutable value really creates a new object and rebinds the name:

```
n = 10
m = n
m += 1          # builds a new int 11 and rebinds m; n still refers to 10
```

```
team: ['ann', 'bob', 'cy'] | same object? True     # list mutated through an alias
n: 10 m: 11                                        # int "changed" = rebinding
str is immutable: 'str' object does not support item assignment
tuple holding a list: ([1, 99], [2])
```

The last line is the subtle one: a tuple is immutable — it will always contain *the same two list objects* — but those lists are themselves mutable.

Why it matters

- Passing a list to a function passes a reference: the function can mutate the caller's list. That is powerful and a common bug source.
- Only immutable (more precisely, **hashable**) objects can be dictionary keys or set members, because their hash must never change.

When to use `is`

Use `is` only for **singletons** — objects of which exactly one exists: `None`, `True`, `False`, `Ellipsis`, and sentinel objects you create. `if x is None:` is correct; `if x == None:` works but can be fooled by classes that override `__eq__`.

Common mistakes

- `if x is 5:` or `if name is "bob":` — works "by accident" sometimes because CPython caches small integers (−5 to 256) and some strings. That caching is an implementation detail; Python 3.8+ emits `SyntaxWarning: "is" with 'int' literal`. Use `==`.
- `grid = [[0] * 3] * 3` creates **three references to one row**; changing `grid[0][0]` changes all rows. Use `[[0] * 3 for _ in range(3)]`.
- Mutable default arguments (`def f(items=[])`) — the one list is shared across calls (covered in the functions chapter).

In the interview

> **30-second answer:** "`==` compares values via `__eq__`; `is` compares identity — whether two names point to the same object. I use `is` only for singletons like `None`. Mutable objects like lists and dicts can change in place, so aliases see the change; immutable ones like ints, strings and tuples can't, so 'modifying' them rebinds the name to a new object."

- *Why does `256 is 256` give True but `1000 is 1000` might not?* → CPython pre-allocates small ints −5..256; it's an implementation detail, never rely on it.
- *Can a tuple change?* → Its own slots can't be reassigned, but mutable objects inside it can be mutated; such a tuple is also unhashable.
- *Why must dict keys be immutable?* → The dict stores keys by hash; if the value could change, the hash would change and the key would be lost.

Self-check

1. `[] == []` and `[] is []`? → `True`, `False`.
2. Which of these are mutable: `str`, `list`, `tuple`, `set`, `frozenset`? → `list`, `set`.
3. Correct test for "no value"? → `x is None`.
4. What does `id()` return in CPython? → The object's memory address (an integer unique among live objects).

TL;DR

`==` = same value, `is` = same object. Use `is` only with `None` / `True` / `False` /sentinels. Mutable objects change in place (aliases see it); immutable ones are replaced.

2.3 Dynamic vs strong typing

What it is

Two independent axes:

Axis	Python's position	Meaning
Static ↔ dynamic	Dynamic	Types are attached to *objects*, checked while running. A *name* can refer to an `int` now and a `str` later.
Weak ↔ strong	Strong	Operations on incompatible types raise errors instead of silently converting.

```
value = 42              # name -> int
value = "forty-two"     # same name -> str: fine (dynamic)
"Age: " + 30            # TypeError: can only concatenate str (not "int") to str  (strong)
```

Output of that part of the script:

```
int -> str
TypeError: can only concatenate str (not "int") to str
```

Compare JavaScript (dynamic **and** weak): `"Age: " + 30` gives `"Age: 30"` and `"5" - 2` gives `3`. C is static but fairly weak (implicit int/pointer conversions). Java is static and strong.

Python *does* widen numbers automatically (`1 + 2.5` → `3.5`), because int, float and complex form a numeric tower where no information is surprising — see §2.7.

Type hints

You can annotate types: `def area(w: float, h: float) -> float:` (function annotations, Python 3.0+; the `typing` module arrived in 3.5) and `age: int = 30` (variable annotations, 3.6+). The interpreter **ignores** hints at run time; tools such as mypy, Pyright and IDEs use them to catch mistakes before you run. Large codebases (Dropbox, where mypy was developed, and Instagram) adopted hints to keep dynamic code safe at scale.

Trade-offs

Dynamic typing gives	Dynamic typing costs
Less boilerplate; fast prototyping	Type errors appear at run time
Duck typing: "if it has `.read()`, treat it as a file"	IDEs know less without hints
Generic code for free	Slower execution (type checks on every operation)

Self-check

1. Is Python statically typed? → No, dynamically; optional hints are checked by external tools only.
2. Why does `"3" * 2` work but `"3" + 2` fail? → `str * int` is defined (repetition → `"33"`); `str + int` isn't.
3. Does `x: int = "hi"` raise an error? → No, hints aren't enforced at run time; mypy would flag it.

TL;DR

Dynamic = names can re-point to any type; strong = no silent cross-type coercion (except numeric widening). Add type hints + mypy for static checking.

2.4 Built-in data types

What it is

A **data type** defines what values an object can hold and which operations make sense (you can divide numbers, not dictionaries). Python's built-ins fall into eight categories:

Category	Types	Mutable?	Literal / constructor example
Text	`str`	No	`"hi"`, `'hi'`, `"""hi"""`
Numeric	`int`, `float`, `complex`	No	`10`, `10.5`, `2 + 3j`
Sequence	`list`, `tuple`, `range`	list: yes; tuple, range: no	`[1, 2]`, `(1, 2)`, `range(5)`
Mapping	`dict`	Yes	`{"name": "Alice"}`
Set	`set`, `frozenset`	set: yes; frozenset: no	`{1, 2}`, `frozenset([1, 2])`
Boolean	`bool`	No	`True`, `False`
Binary	`bytes`, `bytearray`, `memoryview`	bytearray: yes; bytes: no; memoryview: reflects its buffer	`b"abc"`, `bytearray(b"abc")`, `memoryview(b"abc")`
None	`NoneType`	No	`None`

From `code/data_types_tour.py`, which prints `type(obj).__name__` for one sample of each:

```
text       -> str        mutable=False
int        -> int        mutable=False
float      -> float      mutable=False
complex    -> complex    mutable=False
list       -> list       mutable=True
tuple      -> tuple      mutable=False
range      -> range      mutable=False
dict       -> dict       mutable=True
set        -> set        mutable=True
frozenset  -> frozenset  mutable=False
bool       -> bool       mutable=False
bytes      -> bytes      mutable=False
bytearray  -> bytearray  mutable=True
memoryview -> memoryview mutable=False
None       -> NoneType   mutable=False
```

Check a type with `type(x)` (exact type) or, better in real code, `isinstance(x, int)` (also accepts subclasses). Note `isinstance(True, int)` is `True` because `bool` subclasses `int`.

Each category in brief

Strings, lists, tuples, dictionaries and sets get full chapters later; here is what you need now.

Text – `str`: an immutable sequence of Unicode characters. `"Hello, World!"`, `len(s)`, `s.upper()`, `s[0]`.

Numeric: `int` for whole numbers (counts, IDs), `float` for measurements, `complex` for engineering/signal maths. Deep dive in §2.5.

```
int + 5: 15 | float + 0.5: 11.0 | complex: (3+1j)
```

Sequence types keep items in order and support indexing `s[i]`, slicing `s[i:j]`, `len()`, `in`:

- `list` – mutable, the everyday container: `sports = ["soccer", "basketball", "tennis"]`.
- `tuple` – immutable; use for fixed records (`("red", "green", "blue")`, coordinates) and as dict keys. Slightly smaller and faster to create than lists.
- `range` – an immutable, **lazy** sequence of integers: it stores only start, stop and step, so `range(10**12)` uses almost no memory. `range(5)` is 0, 1, 2, 3, 4 (start inclusive, stop exclusive).

```
range(5) as list: [0, 1, 2, 3, 4] | 3 in r: True
```

Mapping – `dict`: key → value pairs with fast (average O(1)) lookup by key. Keys must be hashable. Since Python 3.7, dicts remember insertion order.

```
user = {"name": "Alice", "age": 25}
user["age"] += 1                # modify
user["city"] = "Paris"          # add
user.get("email", "n/a")        # safe lookup with default; user["email"] would raise KeyError
```

```
dict: {'name': 'Alice', 'age': 26, 'city': 'Paris'} | get missing key: n/a
```

Set types: unordered collections of **unique** hashable items with fast membership tests. `set` is mutable; `frozenset` is immutable and therefore hashable (usable as a dict key or inside another set). Adding a duplicate is silently ignored. Careful: `{}` is an empty **dict**; an empty set is `set()`.

```
set size: 4 | sorted: ['basketball', 'soccer', 'tennis', 'volleyball']
frozenset usable as dict key: palette
```

Boolean – `bool`**:** `True` / `False`; see §2.6.

Binary types handle raw bytes (file contents, network packets, images):

- `bytes` – immutable sequence of integers 0–255: `bytes([50, 100, 150])`, `b"abc"`.
- `bytearray` – mutable version; you can assign `data[1] = 105`.
- `memoryview` – a zero-copy *window* onto another buffer; slicing it doesn't copy, and it reflects later changes to the underlying data.

```
data = bytearray([50, 100, 150])
data[1] = 105
view = memoryview(data)[1:3]     # window on items 1..2, no copy
data[2] = 200                    # change the buffer after creating the view...
```

```
bytes: [50, 100, 150] | bytearray: [50, 105, 200] | view: [105, 200]
bytes range 0-255: bytes must be in range(0, 256)
text <-> bytes: b'caf\xc3\xa9' café
```

The view shows `200` – proof that it shares memory. Text ↔ bytes conversion goes through an **encoding**: `"café".encode("utf-8")` and `b"...".decode("utf-8")`.

None – `NoneType`**:** the single object `None` means "no value here": a missing optional argument, a not-yet-computed result, a function that returns nothing (functions without `return` return `None`). Test it with `is None`.

```
No value provided
function without return gives: None
```

Common mistakes

- `{}` for an empty set (it's a dict).
- Using a list as a dict key → `TypeError: unhashable type: 'list'`; use a tuple.
- Mixing `bytes` and `str`: `b"a" + "b"` → `TypeError`; encode/decode explicitly.
- `type(x) == int` rejects subclasses; prefer `isinstance(x, int)`.

Real-world examples

- JSON from a web API maps directly onto `dict`, `list`, `str`, `int` / `float`, `bool`, `None` (`json.loads`).
- `bytes` / `memoryview` are used by `asyncio` streams and by NumPy's buffer protocol to share memory without copies.
- `frozenset` keys appear in caches keyed by a set of options.

In the interview

> **30-second answer:** "Python's built-ins are str; int/float/complex; list/tuple/range; dict; set/frozenset; bool; bytes/bytearray/memoryview; and None. The key split is mutable — list, dict, set, bytearray — versus immutable; only hashable, typically immutable, objects can be dict keys or set members."

- *List vs tuple?* → List mutable, for homogeneous collections that change; tuple immutable, for fixed records, hashable if its items are.
- *Why is `range(10**9)` cheap?* → It's lazy; it computes values on demand from start/stop/step.
- *What's memoryview for?* → Slicing large binary buffers without copying.

Self-check

1. Type of `{}`? Of `{1}`? → `dict`; `set`.
2. Which binary type is mutable? → `bytearray`.
3. What does a function without `return` return? → `None`.
4. Can a `frozenset` be a dict key? → Yes, it's immutable and hashable.

TL;DR

Eight categories; memorise which are mutable (list, dict, set, bytearray). `range` is lazy, `dict` is ordered (3.7+), `{}` is a dict, `None` means "no value".

2.5 Numbers

What it is

Python has three built-in numeric types — `int`, `float`, `complex` — plus two exact types in the standard library — `decimal.Decimal` and `fractions.Fraction`.

`int` — arbitrary precision

Integers have **no fixed size**: they grow to use as much memory as needed. There is no overflow like C's 32/64-bit wraparound. From `code/numbers_demo.py`:

```
sum: 13
2**100 = 1267650600228229401496703205376 | bits: 101
factorial(25) = 15511210043330985984000000
readable literal 1_000_000 = 1000000 | hex/bin/oct: 255 5 15
int->str digit limit (CPython 3.11+): 4300
```

- **How:** CPython stores an int as an array of 30-bit "digits"; small ints fit in one digit, big ones use more. Arithmetic on huge ints is slower, but correct.
- **Literals:** `1_000_000` (underscores for readability), `0xFF` (hex), `0b101` (binary), `0o17` (octal).
- **Limit worth knowing:** since 3.11, converting an int with more than 4 300 decimal digits to/from `str` raises `ValueError` by default, to stop denial-of-service attacks via huge numeric strings. Raise it with `sys.set_int_max_str_digits()` if you really need to.

`float` — IEEE-754 double precision

A `float` is a 64-bit **binary** floating-point number: 1 sign bit, 11 exponent bits, 52 fraction bits. It gives about **15–17 significant decimal digits** (`sys.float_info.dig` is 15) and a range up to about 1.8×10^{308}.

The famous pitfall: most decimal fractions cannot be represented exactly in binary, just as 1/3 cannot be written exactly in decimal. `0.1` is stored as the nearest binary fraction:

```
0.1 + 0.2 = 0.30000000000000004 | == 0.3? False | isclose? True
exact value stored for 0.1: 0.1000000000000000055511151231257827021181583404541015625
max float: 1.7976931348623157e+308 | epsilon: 2.220446049250313e-16
digits trusted (dig): 15
2**53 + 1 as float: 9007199254740992.0 (lost the +1)
inf/nan: True False
round(2.5), round(3.5): 2 4 | round(2.675, 2): 2.67
1/3 + 1/6 = 1/2 | 0.1 added ten times: 0.9999999999999999 | ten Fraction(1, 10) == 1: True
sum([0.1]*10) (3.12+ uses compensated summation): 1.0 | math.fsum: 1.0
```

What each line teaches:

1. **Never compare floats with `==`**. Use `math.isclose(a, b)` (relative tolerance 1e-9 by default) or `abs(a - b) < tol`.
2. **Errors accumulate** with repeated operations (`0.1` added ten times ≠ `1.0`). `math.fsum` – and, since Python 3.12, the built-in `sum()` for floats – use compensated summation to reduce this.
3. **Integers above 2^{53} lose precision** as floats: `float(2**53 + 1)` drops the `+ 1`. Keep IDs and money-in-cents as `int`.
4. `round()` **uses "round half to even"** (banker's rounding): `round(2.5) == 2`, `round(3.5) == 4`. And `round(2.675, 2)` gives `2.67` because 2.675 is actually stored as 2.67499999…
5. `float("inf")`, `float("-inf")` and `float("nan")` exist; **NaN is not equal to anything, including itself**.

The gap between neighbouring floats grows with magnitude – floats are dense near zero and sparse for big numbers:

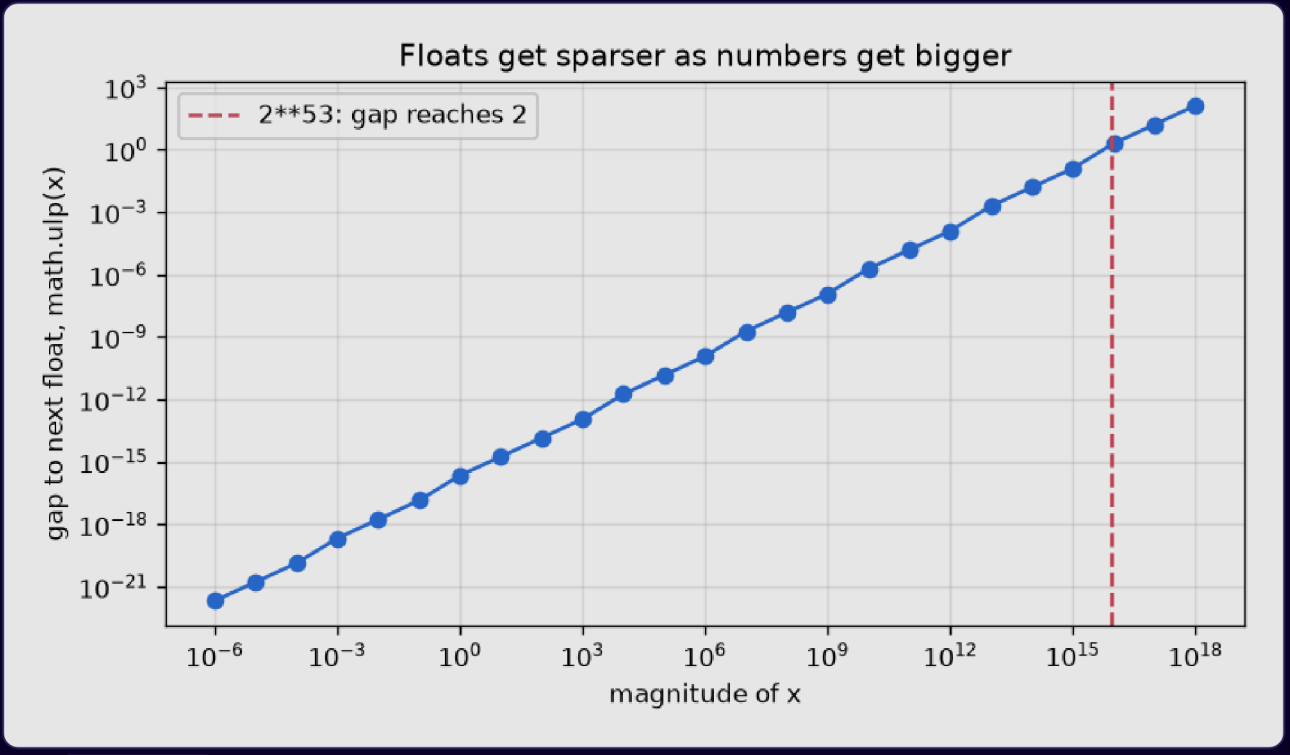


Figure – `math.ulp(x)` (unit in the last place) on log-log axes: at 1.0 the gap is 2.2e-16; beyond $2^{53} \approx 9 \times 10^{15}$ consecutive floats are 2 apart, so odd integers can't be represented.

`Decimal` – exact base-10 arithmetic

`decimal.Decimal` stores numbers in base 10 with user-controllable precision (28 significant digits by default) and rounding. Use it for **money** and anything where "what a human would compute on paper" is the requirement.

```
Decimal 0.10+0.20 = 0.30 | == 0.30? True
Decimal from float (wrong way): 0.1000000000000000055511151231257827021181583404541015625
round half up 2.675 -> 2.68
```

- Always construct from a **string**: `Decimal("0.10")`. `Decimal(0.1)` faithfully copies the float's binary error.
- `quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)` rounds to cents the way invoices expect.
- Cost: much slower than float (still fine for business logic).

`Fraction` – exact rationals

`fractions.Fraction(1, 3)` stores numerator and denominator as ints, so `1/3 + 1/6` is exactly `1/2`. Good for exact geometry, probability, music ratios; slow for heavy numeric work because numerators/denominators can grow.

`complex`

Written `a + bj` (engineers' `j`, not `i`). Attributes `.real`, `.imag` (both floats), method `.conjugate()`, `abs(z)` gives the magnitude. The `cmath` module has complex versions of `sqrt`, `exp`, `phase`, etc.

```
z.real: 1.0 z.imag: 2.0 |z|: 5.0 conjugate: (1-2j)
```

Used in signal processing (FFT), electrical engineering (impedance), quantum computing simulators.

The division family

```
7/2 = 3.5 | 7//2 = 3 | -7//2 = -4 | -7%2 = 1 | divmod(17,5) = (3, 2)
int(-2.7) truncates: -2 | math.floor(-2.7): -3
```

- `/` is **true division** and always returns a float (even `4 / 2 == 2.0`).
- `//` is **floor division**: rounds toward **negative infinity**, so `-7 // 2 == -4` (C and Java truncate toward zero and give –3).
- `%` is consistent with `//`: `a == (a // b) * b + a % b`, so the result takes the **sign of the divisor**: `-7 % 2 == 1`, `7 % -3 == -2`.
- `divmod(a, b)` returns `(a // b, a % b)` in one call.
- `int(x)` **truncates** toward zero; `math.floor` / `math.ceil` round down/up; `round` rounds half-to-even.
- Division by zero raises `ZeroDivisionError` for ints and floats alike.

Random numbers

The `random` module generates **pseudo-random** numbers (Mersenne Twister): deterministic given a seed. Call `random.seed(42)` for reproducible tests and experiments.

```
random.seed(42)
random.randint(1, 10)          # int in [1, 10], BOTH ends inclusive
random.random()                # float in [0.0, 1.0)
random.choice(["red", "green", "blue"])
```

```
randint(1,10): 2 | random(): 0.025 | choice: green
```

Other useful functions: `uniform(a, b)`, `shuffle(list)`, `sample(pop, k)`, `gauss(mu, sigma)`. **Never** use `random` for passwords, tokens or anything security-related – use the `secrets` module, which draws from the OS's cryptographic generator.

Trade-offs

Type	Exact?	Speed	Use for
`int`	Yes	Fast (small), slower when huge	Counts, IDs, money in cents
`float`	No (≈15–17 digits)	Fastest, hardware-backed	Science, ML, measurements, graphics
`Decimal`	Yes in base 10 (to set precision)	Slow	Money, tax, anything legally decimal
`Fraction`	Yes	Slow, can blow up	Exact rational maths
`complex`	Float-based	Fast	Signals, AC circuits

Common mistakes

- `if total == 0.3:` with floats.
- `Decimal(0.1)` instead of `Decimal("0.1")`.
- Expecting `round(2.5)` to be 3.
- Using floats for currency.
- Assuming `-7 // 2 == -3`.
- Using `random` for security tokens.

Real-world examples

- Python's big ints power cryptography libraries and RSA teaching code — no special bignum library needed.
- Financial and e-commerce back-ends store amounts as `Decimal` (Django's `DecimalField` returns `Decimal` objects).
- NumPy and PyTorch use float32/float64 arrays; ML training even uses 16-bit floats because a little precision loss is acceptable for speed.

In the interview

> **30-second answer:** "Python ints are arbitrary precision, so they never overflow. Floats are IEEE-754 doubles, so decimal fractions like 0.1 are approximations — compare them with `math.isclose`, and use `Decimal` built from strings for money. `//` floors toward negative infinity and `%` takes the divisor's sign."

- *Why is `0.1 + 0.2 != 0.3`?* → 0.1 and 0.2 have no exact binary representation; the rounded sum differs from the rounded 0.3 in the last bit.
- *What rounding does `round()` use?* → Round half to even, to avoid upward bias in large sums.
- *How do you get reproducible random numbers?* → `random.seed(n)` or a dedicated `random.Random(n)` instance.

Self-check

1. `2 ** 64` in Python — overflow? → No, `18446744073709551616`.
2. `-9 // 4` and `-9 % 4`? → `-3` and `3`.
3. How do you create an exact 0.1? → `Decimal("0.1")` or `Fraction(1, 10)`.
4. Is `random.randint(1, 10)` able to return 10? → Yes, both ends inclusive.
5. Why is `float(2**53 + 1) == float(2**53)`? → Above 2^{53} the float spacing is 2.

TL;DR

int = exact and unbounded; float = fast but approximate (use `isclose`, never `==`); Decimal (from strings) for money; Fraction for exact ratios; `//` floors, `%` follows the divisor; seed `random` for repeatability, use `secrets` for security.

2.6 Booleans and truthiness

What it is

`bool` has exactly two values, `True` and `False` (capitalised; they're keywords). Comparisons produce them, and `if` / `while` consume them. `bool` is a **subclass of** `int`: `True == 1`, `False == 0`, and `True + True == 2`.

Truthiness — how any object becomes a boolean

In a boolean context (`if x:`, `while x:`, `and` / `or` / `not`, `bool(x)`), every object is treated as true **except** these **falsy** values:

Falsy	Examples
The constants	`None`, `False`
Numeric zeros, whatever the type	`0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`
Empty sequences and collections	`""`, `()`, `[]`, `{}`, `set()`, `range(0)`, `b""`

Everything else is **truthy**, including `"0"`, `"False"`, `" "`, `[0]` and `-1`.

How it works: `bool(x)` calls `x.__bool__()` if defined; otherwise `x.__len__()` and checks for non-zero; otherwise the object is truthy. Your own classes can hook into this.

```
has_access = True
if has_access:
    print("Access granted.")
else:
    print("Access denied.")
```

Idioms

```
if not cart:          # instead of: if len(cart) == 0
if name:              # instead of: if name != ""
if x is None:         # when 0 or "" are valid values, test for None explicitly
```

Common mistakes

- `if x == True:` → just `if x:` (PEP 8).
- `if value:` when `0` is legitimate data — it treats `0` as "missing". Use `if value is not None:`.
- `bool("False")` is `True` — any non-empty string is truthy (§2.7).
- Because `True == 1`, `{1: "a", True: "b"}` has **one** key (`{1: 'b'}`).

In the interview

30-second answer: "Falsy values are None, False, numeric zeros and empty containers; everything else is truthy. `bool` subclasses `int`. I use truthiness for emptiness checks but `is None` when zero or empty string are valid values."

- *How does a custom class define truthiness?* → Implement `__bool__`, or `__len__` as a fallback.

Self-check

1. `bool([0])`? → `True` (non-empty list).
2. `True + True + False`? → `2`.
3. Is `" "` truthy? → Yes, it has one character.

TL;DR

Falsy = `None`, `False`, zeros, empties. Everything else is truthy. `bool` is an `int`.

2.7 Type conversion

What it is

Type conversion (casting) produces a new object of a different type from an existing value. It happens **implicitly** (Python does it for you) or **explicitly** (you call a constructor such as `int()`).

Implicit conversion

Python only converts implicitly **within the numeric tower**, widening to the more general type: `bool → int → float → complex`. From `code/conversions.py`:

```
6 + 1.5 = 7.5 float
True + 2 = 3 | 2 + 1j = (2+1j)
```

It never implicitly converts between text and numbers (`"6" + 1.5` is a `TypeError`) – strong typing.

Explicit conversion

Call the target type like a function. The runnable table:

```
str(10)                         -> '10'
int("20")                       -> 20
float(4)                        -> 4.0
int(True), int(False)           -> (1, 0)
int(2.8)                        -> 2
float("3.5")                    -> 3.5
bool(0)                         -> False
complex(2, 3)                   -> (2+3j)
list("abc")                     -> ['a', 'b', 'c']
tuple("abc")                    -> ('a', 'b', 'c')
sorted(set("abac"))             -> ['a', 'b', 'c']
dict([(1,'one'),(2,'two')])     -> {1: 'one', 2: 'two'}
bin(3), hex(255), oct(8)        -> ('0b11', '0xff', '0o10')
int("ff", 16), int("0b11", 0)   -> (255, 3)
int("  42\n"), int("1_000")     -> (42, 1000)
ord('A'), chr(66)               -> (65, 'B')
```

Function	Converts…	Notes
`int(x)`	number or numeric string → int	Truncates floats toward zero; strips surrounding whitespace; accepts `_`; optional `base` (2–36, or 0 = read the prefix)
`float(x)`	number or string → float	Accepts `"1e3"`, `"inf"`, `"nan"`
`str(x)`	anything → readable text	`repr(x)` gives the unambiguous developer form (`'hi\n'`)
`bool(x)`	anything → truthiness	See §2.6
`complex(re, im)`	numbers or string → complex	`complex("2+3j")` works
`list()`, `tuple()`, `set()`	any iterable → that collection	`set` removes duplicates and loses order
`dict()`	iterable of key/value pairs (or keyword args) → dict	`dict(a=1)`
`bin()`, `hex()`, `oct()`	int → prefixed string	Reverse with `int(s, 0)`
`ord()`, `chr()`	character ↔ Unicode code point	

When conversion fails

```
int("3.5")   -> ValueError: invalid literal for int() with base 10: '3.5'
int("abc")   -> ValueError: invalid literal for int() with base 10: 'abc'
float("1,5") -> ValueError: could not convert string to float: '1,5'
int(None)    -> TypeError: int() argument must be a string, a bytes-like object or a real number, not
'NoneType'
bool("False") = True | bool("") = False
```

- `ValueError` = right type, bad content. `TypeError` = wrong type altogether.
- To parse `"3.5"` as an int, go through float: `int(float("3.5"))` → 3 (decide first whether truncation is what you want).
- For user input, wrap conversions in `try/except ValueError` (the exceptions chapter shows the pattern).
- Parse booleans from text explicitly: `text.strip().lower() in {"true", "1", "yes"}`.

Building strings from numbers

`"Age: " + str(age)` works, but **f-strings** (3.6+) are clearer and handle formatting: `f"Age: {age}"`, `f"pi ~ {3.14159:.2f}"` → `pi ~ 3.14`.

Common mistakes

- Forgetting `input()` returns `str`: `age = input(); age + 1` → `TypeError`.
- `int(2.999)` is 2, not 3 — truncation, not rounding.
- `bool("False")` is `True`.
- Locale formats: `float("1,5")` fails; `float("1,000")` fails.

In the interview

30-second answer: "Python only converts implicitly up the numeric tower — bool, int, float, complex. Everything else is explicit via constructors like `int()`, `float()`, `str()`, `list()`. Bad content raises `ValueError`, a wrong type raises `TypeError`. `int()` truncates toward zero."

- *`int("0x1A", 0)`?* → 26; base 0 reads the prefix.
- *str vs repr?* → str is for users, repr for developers and should be unambiguous.

Self-check

1. `int(-3.9)`? → `-3`.
2. Which exception does `int("12a")` raise? → `ValueError`.
3. Result type of `3 + 4.0`? → `float` (`7.0`).
4. How do you turn `"1010"` binary text into 10? → `int("1010", 2)`.

TL;DR

Implicit only for numbers (widening). Explicit via type constructors. `ValueError` for bad text, `TypeError` for wrong types. Prefer f-strings to `str()` + `+`.

2.8 Operators

What it is

An **operator** is a symbol or keyword that computes a result from **operands** (the values it works on). In `a + b`, `+` is the operator and `a`, `b` are operands. Under the hood each operator calls a "dunder" (double-underscore) method — `a + b` calls `a.__add__(b)` — which is how the same `+` adds numbers, joins strings and concatenates lists.

All examples below come from `code/operators_demo.py` with `a, b = 10, 3`.

Arithmetic

Op	Meaning	`10 ? 3`
`+`	addition	13
`-`	subtraction	7
`*`	multiplication	30
`/`	true division (always float)	3.3333333333333335
`%`	modulus (remainder)	1
`**`	exponentiation	1000
`//`	floor division	3
`@`	matrix multiplication (used by NumPy; no built-in type implements it)	–

Operators also work on other types: `"ab" * 3 == "ababab"`, `[1] + [2] == [1, 2]`.

Comparison

`==`, `!=`, `>`, `<`, `>=`, `<=` → always return `bool`.

```
compare: False True True False True False
```

Strings compare lexicographically by code point (`"Z" < "a"` is True because uppercase letters come first). Comparing unrelated types with `<` raises `TypeError` (`1 < "2"`); `==` between them just returns `False`.

Chained comparisons: `18 <= age < 65` means `18 <= age and age < 65`, with `age` evaluated only once — just like maths notation.

```
18 <= age < 65: True
```

Assignment and augmented assignment

`=` binds a name. **Augmented** operators combine an operation with rebinding: `x += 3` means roughly `x = x + 3`. Every arithmetic and bitwise operator has one. Walking one variable through all of them:

```
v += 3  -> 13
v -= 2  -> 11
v *= 2  -> 22
v /= 2  -> 11.0
v %= 4  -> 3.0
v **= 2 -> 9.0
v //= 2 -> 4.0
v &= 3  -> 0
v |= 8  -> 8
v ^= 6  -> 14
v <<= 1 -> 28
v >>= 2 -> 7
```

Notice: `/=` turned the int into a float, and it stays a float through `%=`, `**=`, `//=`; the script converts back with `int(v)` before the bitwise steps because bitwise operators need integers. The bit patterns: `4 & 3` → `100 & 011 = 000`; `0 | 8` → `1000`; `8 ^ 6` → `1000 ^ 0110 = 1110` (14); `14 << 1` → 28; `28 >> 2` → 7.

Subtlety: for **mutable** objects, augmented operators modify in place. `nums += [3]` extends the existing list (aliases see it), while `nums = nums + [3]` builds a new list.

```
list += mutates in place: [1, 2, 3]
```

Python has no `++` or `--`; `x++` is a `SyntaxError` and `++x` is just `+(+x)`.

Logical: `and`, `or`, `not`

Expression	Result
`x and y`	if `x` is falsy → `x`, else → `y`
`x or y`	if `x` is truthy → `x`, else → `y`
`not x`	`True` if `x` is falsy, else `False` (always a bool)

Two important properties:

1. **Short-circuit evaluation:** the right operand is evaluated only if needed. `0 and 1/0` returns `0` without dividing by zero.
2. **They return an operand, not necessarily a bool:** `"" or "guest"` → `"guest"`. That is the idiom for defaults.

```
True and False: False | True or False: True | not True: False
'' or 'guest' -> guest | 0 and 1/0 -> 0
```

(Use `x if x is not None else default` when `0`/`""` are valid values.)

Bitwise

Work on the binary representation of integers.

Op	Name	`2 ? 3` (`0010`, `0011`)
`&`	AND – 1 where both bits are 1	2 (`0010`)
`|`	OR – 1 where either is 1	3 (`0011`)
`^`	XOR – 1 where bits differ	1 (`0001`)
`~`	NOT – flips every bit; `~x == -x - 1` (two's complement)	`~2 == -3`
`<<`	shift left n bits = multiply by 2^n	`2 << 1 == 4`
`>>`	shift right n bits = floor-divide by 2^n	`2 >> 1 == 1`

```
bitwise: 2 3 1 -3 4 1 | 0010 & 0011 = 0010
```

Real uses: permission flags (`READ | WRITE`, test with `perms & WRITE`), `enum.Flag`, network masks, hashing, compact storage. On `set`s the same symbols mean union `|`, intersection `&`, difference `-`, symmetric difference `^`; on dicts (3.9+) `|` merges.

Membership: `in`, `not in`

Test whether a value is contained in a container: list, tuple, string (substring test), dict (checks **keys**), set.

```
membership: True True True True
```

Speed differs: `x in list` scans every element (O(n)); `x in set` / `x in dict` use hashing (O(1) on average). For repeated lookups, convert to a set.

Identity: `is`, `is not`

True when both sides are the same object (§2.2).

```
identity: False True True
```

Walrus operator `:=` (assignment expression, 3.8+)

`name := expr` assigns **and** returns the value, so you can bind inside an expression – typically a condition – and avoid computing something twice.

```
for line in log:
    if (m := re.match(r"error: (.*)", line)) is not None:
        print("walrus captured:", m.group(1))
if (count := len(data)) > 2:
    print(f"walrus: {count} items")
```

```
walrus captured: disk full
walrus: 3 items
```

Parentheses are usually required. Classic uses: `while (chunk := f.read(8192)):` and filtering in comprehensions `[y for x in data if (y := f(x)) > 0]`. Don't use it where a normal assignment on its own line reads better.

Operator precedence

Precedence decides which operator binds first when there are no parentheses; **associativity** decides the order among equal-precedence operators (left-to-right for almost all; `**` is right-to-left). From highest to lowest, per the Python language reference:

#	Operators	Notes
1	`(…)`, `[…]`, `{…}`	parenthesised expressions and displays
2	`x[i]`, `x[i:j]`, `f(…)`, `x.attr`	subscription, slicing, call, attribute
3	`await x`	
4	`**`	right-associative: `2 ** 3 ** 2 == 2 ** 9 == 512`
5	`+x`, `-x`, `~x`	unary; binds *less* tightly than `**` on its right, so `-2 ** 2 == -4`
6	`*`, `@`, `/`, `//`, `%`	
7	`+`, `-`	
8	`<<`, `>>`	
9	`&`	
10	`^`	
11	`|`	
12	`in`, `not in`, `is`, `is not`, `<`, `<=`, `>`, `>=`, `!=`, `==`	**all comparisons share one level** and chain
13	`not x`	
14	`and`	
15	`or`	
16	`x if c else y`	conditional expression
17	`lambda`	
18	`:=`	assignment expression

Two consequences beginners hit: `&`, `^`, `|` bind **tighter** than comparisons (so in pandas you must write `(df.a > 1) & (df.b < 2)` with parentheses), and `and` binds tighter than `or`.

Worked example (`a, b, c = 10, 20, 30`):

```
result = a + b * c ** 2 / 10 - 5 <= b or b % a == 0 and c > b
```

1. `c ** 2` → 900 (highest arithmetic precedence)
2. `b * 900` → 18000; then `18000 / 10` → 1800.0 (`*` and `/` share a level, left to right)
3. `a + 1800.0` → 1810.0; `- 5` → 1805.0
4. `1805.0 <= b` → `False`
5. `b % a == 0` → `0 == 0` → `True`; `c > b` → `True`
6. `and` before `or`: `True and True` → `True`
7. `False or True` → **`True`**

```
precedence: True True | left side value: 1805.0
-2 ** 2 = -4 | (-2) ** 2 = 4 | 2 ** 3 ** 2 = 512
not 1 == 2 -> True | 1 < 2 == 2 -> True
```

The second value (`True`) is the same expression fully parenthesised – identical, as expected. `not 1 == 2` is `not (1 == 2)` because comparisons bind tighter than `not`. `1 < 2 == 2` is a *chain*: `1 < 2 and 2 == 2`.

Rule of thumb: if you have to think about precedence, add parentheses – readability counts.

Common mistakes

- `x = x++` (no such operator).
- `if a = b:` (assignment isn't an expression; use `==`, or `:=` deliberately).
- Mixing bitwise and comparison operators: because `&` binds tighter than `==`, `x == 1 & y == 2` parses as the chain `x == (1 & y) == 2` – not what you meant. Write `(x == 1) & (y == 2)`, or `and` for plain booleans.
- `if x == 1 or 2:` is always truthy (`(x == 1) or 2`). Write `if x in (1, 2):`.
- Comparing floats with `==` (use `math.isclose`):

```
0.1*3 == 0.3: False | abs diff: True
```

Real-world examples

- `or` defaults: `timeout = config.get("timeout") or 30`.
- Walrus in stream reading: `while (chunk := sock.recv(4096)):` in network code.
- Bit flags: `os.open(path, os.O_WRONLY | os.O_CREAT)`, `re.IGNORECASE | re.MULTILINE`.
- `@` for matrix products in NumPy/PyTorch: `y = X @ w`.

In the interview

30-second answer: "Python has arithmetic, comparison, assignment/augmented, logical, bitwise, membership and identity operators, plus the walrus `:=`. `and` / `or` short-circuit and return an operand, comparisons chain like maths, `**` is right-associative and binds tighter than unary minus, and bitwise operators bind tighter than comparisons."

- *Output of `-3 ** 2`?* → `-9`.
- *Why does `if x == 1 or 2` always pass?* → It parses as `(x == 1) or 2`, and `2` is truthy.
- *Difference between `a += [1]` and `a = a + [1]` for lists?* → The first mutates the list in place (aliases see it); the second creates a new list.
- *What's the complexity of `in` for list vs set?* → O(n) vs average O(1).

Self-check

1. `10 // 4`, `10 % 4`, `10 / 4`? → `2`, `2`, `2.5`.
2. `None or "x"` → ? `0 and "x"` → ? → `"x"`; `0`.
3. `5 & 3`, `5 | 3`, `5 ^ 3`? → `1`, `7`, `6`.
4. Is `1 < 3 > 2` valid? → Yes: `1 < 3 and 3 > 2` → `True`.
5. `2 ** -1`? → `0.5` (unary minus on the right of `**` binds first).

TL;DR

Know `/` vs `//` vs `%`, that `and` / `or` short-circuit and return operands, that comparisons chain, that `+=` mutates mutable objects in place, and the precedence ladder: `**` > unary > `* /` > `+ -` > shifts > `&` > `^` > `|` > comparisons > `not` > `and` > `or`. When in doubt, parenthesise.

Chapter 2 TL;DR

- A variable is a **name bound to an object**; `=` never copies; `del` removes a name.
- `==` compares values, `is` compares identity – use `is` only for `None`/singletons.
- Mutable: `list`, `dict`, `set`, `bytearray`. Immutable: numbers, `str`, `tuple`, `frozenset`, `bytes`, `None`.
- Python is **dynamically** (names can re-point) and **strongly** (no silent cross-type coercion) typed.
- `int` is unbounded; `float` is approximate (`isclose`, not `==`); `Decimal("…")` for money; `Fraction` for exact ratios.
- Falsy: `None`, `False`, zeros, empty containers – everything else is truthy; `bool` is an `int`.

- Conversion is implicit only up the numeric tower; otherwise call `int()`, `float()`, `str()` … and expect `ValueError` / `TypeError` on bad input.
- Operators: `//` floors, `%` follows the divisor, `and` / `or` short-circuit, comparisons chain, `:=` assigns inside expressions, and parentheses beat memorising precedence.

Chapter 3 — Control Flow: Decisions and Loops

So far every program ran top to bottom, once. **Control flow** is how a program chooses *which* statements run (`if`, `match`) and *how many times* (`for`, `while`). With the truthiness rules from Chapter 2 and the constructs here you can express any algorithm.

What you'll learn

#	Topic	Key ideas
3.1	Conditional statements — overview	what "branching" means, where it's used
3.2	`if` / `elif` / `else`	syntax, order of tests, nesting vs guard clauses
3.3	Conditional expressions	`a if cond else b`
3.4	`match` / `case`	literal, OR, capture, wildcard, sequence, mapping, class patterns, guards, `as`
3.5	Loops — overview and the iteration protocol	iterables, iterators, `for` vs `while`
3.6	`for` loops	lists, strings, dicts, nested loops, `range`, `enumerate`, `zip`
3.7	`while` loops	condition-driven repetition, infinite loops, sentinels
3.8	Loop control	`break`, `continue`, `pass`
3.9	`for-else` / `while-else`	"no break happened"
3.10	Loop idioms	`sum` / `any` / `all`, comprehensions, counting, safe modification

Code: `code/conditionals.py`, `code/pattern_matching.py`, `code/loops.py` — outputs are real (CPython 3.14).

3.1 Conditional statements — overview

What it is

A **conditional statement** evaluates a **condition** — any expression, interpreted by its truthiness — and runs a block of code only when the condition holds. Analogy: a railway switch: the train (execution) goes down one track or the other depending on the signal.

Python offers two decision constructs:

- `if` / `elif` / `else` — test conditions one after another.
- `match` / `case` (Python 3.10+) — compare one value against **patterns** describing its shape; a far more powerful cousin of `switch` in C/Java.

Why it matters — real-world uses

Domain	Decision
Input validation	Is the email well-formed? Is the password long enough? Reject or accept.
App navigation	Logged in → dashboard; otherwise → login screen.
Games	Health ≤ 0 → game over; score above threshold → next level; difficulty tuned by player behaviour.
Healthcare software	Heart rate outside safe range → alert staff; lab values → suggested treatment protocol.
Web back-ends	Route by HTTP method and path; return 404 if not found.

TL;DR

Conditionals choose which code runs; `if` tests conditions in order, `match` compares a value against patterns.

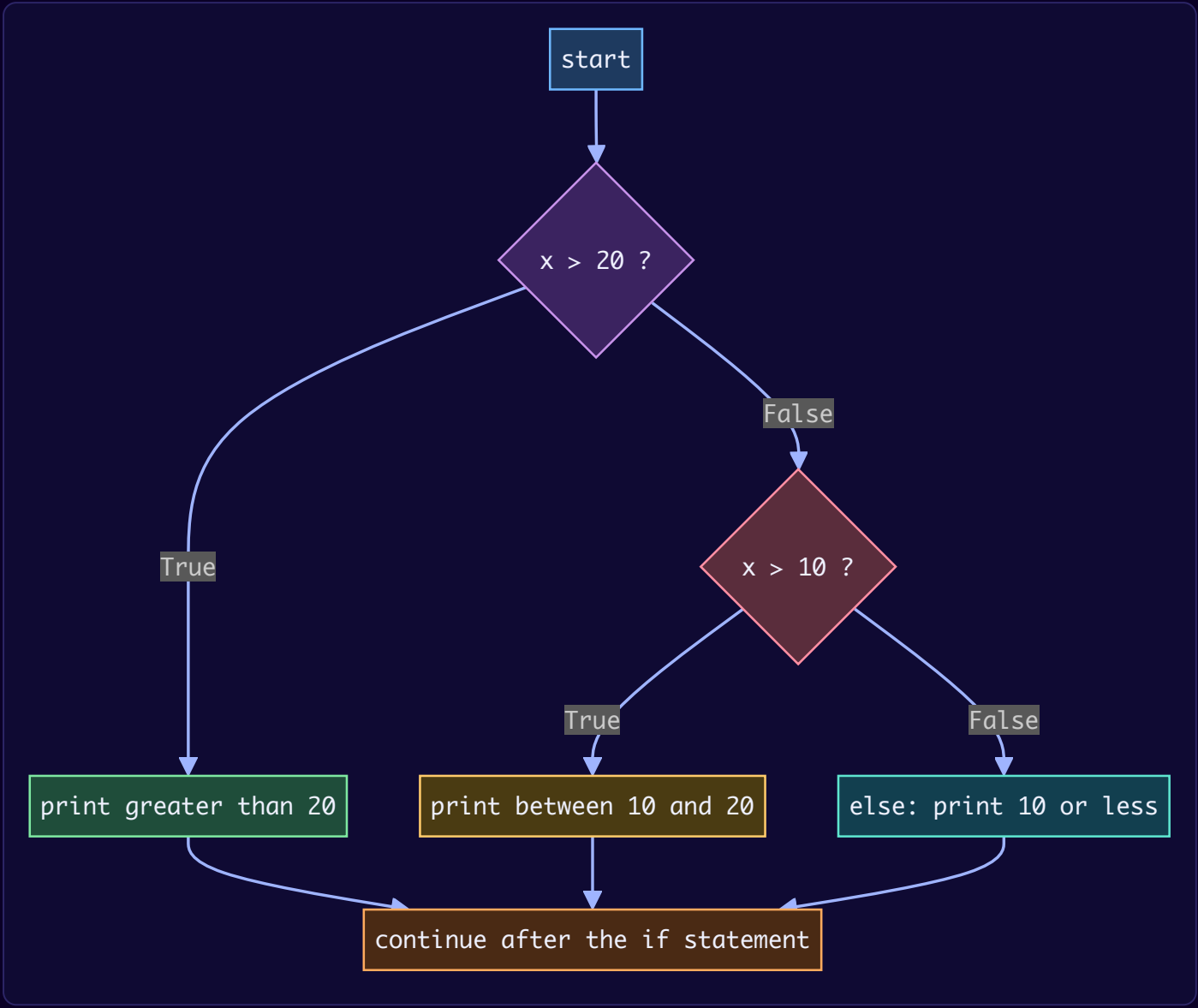
3.2 `if` , `elif` , `else`

What it is

```
if condition1:
    ...           # runs if condition1 is truthy
elif condition2:
    ...           # runs if condition1 was falsy and condition2 is truthy
else:
    ...           # runs if every condition above was falsy
```

- `if` is required; `elif` ("else if") may appear any number of times; `else` is optional and must come last.
- Each header ends with `:` and its block is indented (Chapter 1).
- **At most one** branch runs: the first whose condition is truthy. The rest are skipped without being evaluated.

How it works



From `code/conditionals.py`:

```
x = 10
if x > 5:
    print("x is greater than 5")          # runs

x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")      # runs

def describe(x: int) -> str:
    if x > 20:
        return "x is greater than 20"
    elif x > 10:
        return "x is greater than 10 but not more than 20"
    else:
        return "x is 10 or less"
```

Output:

```
x is greater than 5
x is not greater than 5
25 -> x is greater than 20
15 -> x is greater than 10 but not more than 20
5 -> x is 10 or less
```

For `x = 15`, the first test (`15 > 20`) fails, the `elif` test (`15 > 10`) passes, and the `else` is never considered.

Order matters

Because the first true branch wins, test the **most specific** condition first:

```
def grade(score: int) -> str:
    if not 0 <= score <= 100:
        raise ValueError(f"score out of range: {score}")
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    return "F"
```

```
grades: ['A', 'B', 'C', 'F']
```

If `score >= 70` came first, every A and B student would get a C.

Nesting vs guard clauses

You can nest `if` statements, but deep nesting ("arrow code") is hard to read. A **guard clause** handles a special case early and returns, keeping the main path flat — "flat is better than nested" (PEP 20):

```
def shipping_cost(total: float, is_member: bool) -> float:
    if total <= 0:
        return 0.0
    if is_member or total >= 50:
        return 0.0
    return 4.99
```

```
shipping: 4.99 0.0 0.0
```

Writing good conditions

- Rely on truthiness: `if not cart:` rather than `if len(cart) == 0:`.
- Test `None` explicitly when `0` / `""` are valid: the script shows `discount = 0` looks "missing" to a truthiness test but not to `is None`:

```
cart is empty
hello, guest
falsy check says missing: True | is-None check says missing: False
```

- Use chained comparisons for ranges: `if 18 <= temp <= 26 and not raining:`.
- Use `in` instead of long `or` chains: `if day in {"sat", "sun"}:`.

```
weekend
good weather for a walk
```

Trade-offs

Approach	Good for	Weak at
`if/elif/else`	Arbitrary boolean conditions, ranges, few branches	Many equality tests on one value get repetitive
`match/case`	One value, many shapes/values; destructuring	Plain boolean conditions (use `if`)
dict lookup `{"a": f, "b": g}[key]()`	Many constant keys → action (a *dispatch table*)	Conditions that aren't exact keys

Common mistakes

- `=` instead of `==` in a condition → `SyntaxError` (Python protects you here).
- Missing colon or indentation → `SyntaxError` / `IndentationError`.
- `if x == 1 or 2:` – always true; write `if x in (1, 2):`.
- `else if` → must be `elif`.
- Writing `if flag == True:` → just `if flag:`.
- Overlapping ranges in the wrong order (see grading example).

Real-world examples

- Django views: `if request.method == "POST": … else: …` is the classic form-handling pattern.
- Feature flags: `if flags.is_enabled("new_checkout", user): …`.
- Input validation in CLI tools before converting types.

In the interview

30-second answer: "`if` evaluates conditions top-down by truthiness and runs only the first matching branch; `elif` chains alternatives and `else` catches the rest. I order conditions from most specific to most general, use guard clauses to avoid deep nesting, and `is None` when zero or empty values are legitimate."

- *Does Python have `switch`?* → Not as such; `match` (3.10+) covers it and more, and dicts serve as dispatch tables.
- *Are later `elif` conditions evaluated after one matches?* → No, they're skipped.

Self-check

1. With `x = 20`, which branch of `describe` runs? → The `elif`: `20 > 20` is false, `20 > 10` is true → "x is greater than 10 but not more than 20".
2. Can you have `elif` without `if`? → No.
3. How do you express "between 1 and 10 inclusive"? → `1 <= n <= 10`.
4. What's a guard clause? → An early `return` / `raise` that handles an edge case so the main logic isn't nested.

TL;DR

First truthy branch wins; order from specific to general; prefer flat guard clauses; use `in`, chained comparisons and truthiness for readable conditions.

3.3 Conditional expressions (the "ternary")

What it is

An **expression** (produces a value) version of `if/else`:

```
value_if_true if condition else value_if_false
```

```
parity = "even" if n % 2 == 0 else "odd"          # 7 is odd
print("Access granted." if has_access else "Access denied.")
```

Only the chosen branch is evaluated. It has very low precedence (just above `lambda`), so wrap it in parentheses inside bigger expressions.

When not to use it

- When branches are long or have side effects – use a normal `if` statement.
- Nested ternaries (`a if x else b if y else c`) are legal but hard to read.
- For defaults, `x or default` is shorter – but only when falsy values of `x` should be replaced.

Self-check

1. Write `abs` with a conditional expression. → `x if x >= 0 else -x`.
2. Is `else` optional in a conditional expression? → No, it's required.

TL;DR

`a if cond else b` picks a value; keep it short and un-nested.

3.4 `match` / `case` – structural pattern matching

What it is

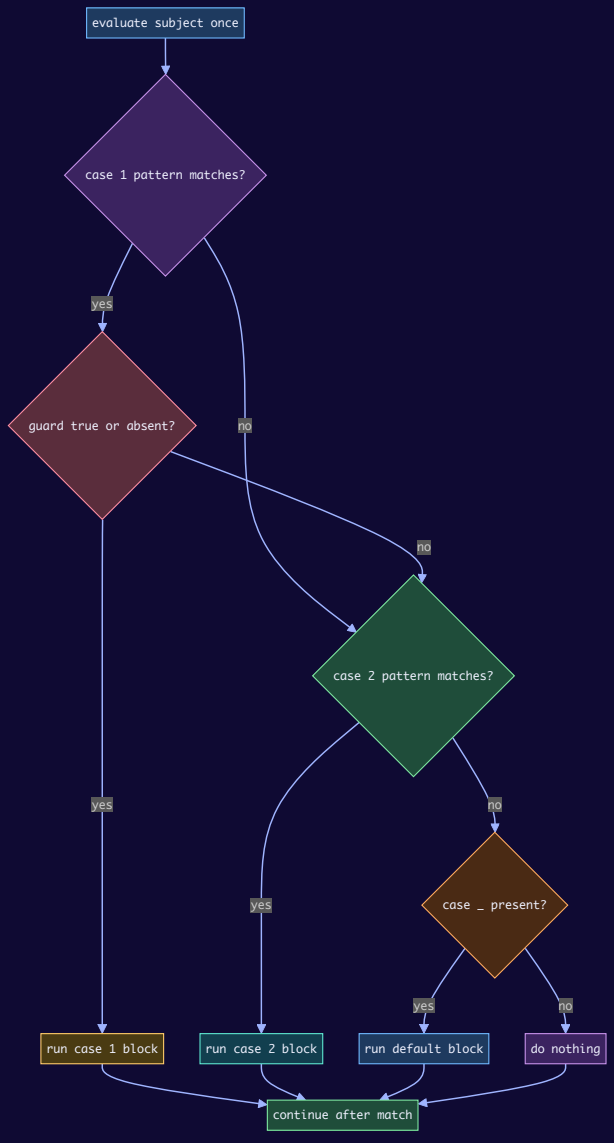
`match` (Python 3.10, PEPs 634–636) takes a **subject** value and tries each `case` **pattern** in order. The first pattern that *matches* (and whose optional guard is true) runs its block; the others are skipped. If none match, nothing happens – no error.

It looks like `switch` from C/Java, but a pattern can describe the **structure** of the data – "a list whose first item is `"go"` followed by one more item", "a dict with a `type` key equal to `"click"`", "a `Point` with `y == 0`" – and **bind** parts of it to names in one step (*destructuring*).

```
match subject:
    case pattern1:
        ...
    case pattern2 if guard:
        ...
    case _:
        ...          # wildcard: the default
```

`match` and `case` are **soft keywords** – you can still name a variable `match`.

How it works



There is **no fall-through**: unlike C's `switch`, you never write `break`.

The pattern kinds

Pattern	Looks like	Matches when…	Binds
Literal	`200`, `"Monday"`, `None`, `True`	subject `==` literal (`None` / `True` / `False` compared with `is`)	–
Capture	`x` (a bare name)	always	subject → `x`
Wildcard	`_`	always	nothing
Value	`Color.RED`, `http.HTTPStatus.OK` (dotted name)	subject `==` that value	–
OR	`p1 | p2 | p3`	any alternative matches	each alternative must bind the same names
AS	`pattern as name`	pattern matches	whole matched part → `name`
Sequence	`[a, b]`, `(a, b)`, `[first, *rest]`	subject is a sequence (list, tuple, …, **not** `str` / `bytes`) of matching length and items	items
Mapping	`{"type": "click", "pos": p}`	subject is a mapping containing those keys with matching values; **extra keys are ignored**	values; `**rest` collects the other items
Class	`Point(x=0, y=0)`, `Point(x, 0)`, `int(n)`, `str()`	`isinstance(subject, Point)` and the listed attributes match	attributes
Guard	`case p if cond:`	pattern matches **and** `cond` is truthy	(uses pattern's bindings)

Literal and OR patterns

From `code/pattern_matching.py`:

```
def http_status(code: int) -> str:
    match code:
        case 200:
            return "Success"
        case 404:
            return "Not Found"
        case 500:
            return "Server Error"
        case 301 | 302 | 307 | 308:          # OR pattern
            return "Redirect"
        case _:                              # wildcard
            return "Unknown status code"

def day_kind(day: str) -> str:
    match day:
        case "Monday" | "Friday":
            return "Start/end of the work week"
        case "Tuesday" | "Wednesday" | "Thursday":
            return "Midweek"
        case _:
            return "Weekend"
```

```
['Success', 'Not Found', 'Server Error', 'Redirect', 'Unknown status code']
Midweek | Start/end of the work week | Weekend
```

With `code = 404`: the subject is compared to `200` (no), then `404` (yes) → "Not Found"; later cases are skipped.

Sequence patterns

```
def first_color(colors: list[str]) -> str:
    match colors:
        case []:
            return "empty list"
        case ["red", *others]:        # first item "red"; others = the rest (a list)
            return f"red first, {len(others)} others"
        case ["blue", *_]:            # *_ = any number of remaining items, not bound
            return "blue first"
        case [single]:                # exactly one item
            return f"only one color: {single}"
        case _:
            return "different first color"
```

```
[] -> empty list
['red', 'blue', 'green'] -> red first, 2 others
['blue', 'red'] -> blue first
['pink'] -> only one color: pink
['pink', 'red'] -> different first color
```

Without a star, the length must match exactly. At most one starred item per sequence pattern.

Sequence patterns shine for parsing commands. Here with an OR sub-pattern captured by `as`, and a **guard**:

```
def command(text: str) -> str:
    match text.split():
        case ["go", ("north" | "south" | "east" | "west") as direction]:
            return f"moving {direction}"
        case ["take", item]:
            return f"picked up {item}"
        case ["drop", *items] if items:          # guard: need at least one item
            return f"dropped {', '.join(items)}"
        case ["quit" | "exit"]:
            return "bye"
        case _:
            return f"unknown command: {text!r}"
```

```
'go north'       -> moving north
'take lamp'      -> picked up lamp
'drop key coin'  -> dropped key, coin
'drop'           -> unknown command: 'drop'
'quit'           -> bye
'dance'          -> unknown command: 'dance'
```

`"drop"` alone matches the pattern `["drop", *items]` with `items == []`, but the guard `if items` is false, so matching continues to `_`.

Tuples work the same way. Here the guard adds a numeric condition:

```
match person:                                # person = (25, "Engineer")
    case (age, "Engineer") if age > 20:
        return f"Adult Engineer aged {age}"
    case (age, profession):
        return f"{profession} aged {age}"
```

```
Adult Engineer aged 25 | Engineer aged 19
```

Strings are not sequences for matching (a deliberate design choice, so `"ab"` doesn't match `[a, b]`):

```
'ab' is matched as a str, not as a sequence of characters
```

Mapping patterns

Ideal for JSON-like dicts, e.g. events or API payloads:

```python
match event:
    case {"type": "click", "pos": (x, y)}:          # nested sequence pattern
        return f"click at {x},{y}"
    case {"type": "key", "key": str(k)} if len(k) == 1:
        return f"key {k!r}"
    case {"type": "resize", **rest}:                # rest = remaining key/values
        return f"resize with {rest}"
    case {"type": other}:
        return f"unhandled event type {other!r}"
    case _:
        return "not an event"
```

```
{'type': 'click', 'pos': (3, 4), 'button': 1} -> click at 3,4
{'type': 'key', 'key': 'q'} -> key 'q'
{'type': 'resize', 'w': 800, 'h': 600} -> resize with {'w': 800, 'h': 600}
{'type': 'scroll'} -> unhandled event type 'scroll'
['click'] -> not an event
```

The first event has an extra `"button"` key and still matches — mapping patterns check only the keys you list.

Class patterns

`ClassName(...)` first checks `isinstance`, then matches attributes. Keyword form `Point(x=0)` matches attribute `x`. Positional form `Point(a, b)` needs the class to define `__match_args__` — a tuple naming which attributes the positions map to. `@dataclass` generates it automatically (`('x', 'y')`). Built-in types like `int`, `str`, `float` accept one positional sub-pattern that matches the whole subject.

```python
@dataclass
class Point:
    x: float
    y: float

match shape:
    case Point(x=0, y=0):                           # keyword attributes
        return "the origin"
    case Point(x, 0):                               # positional via __match_args__
        return f"on the x-axis at {x}"
    case Point():                                   # any Point
        return "some other point"
    case Circle(center=Point(x=0, y=0), radius=r):  # nested patterns
        return f"circle of radius {r} at the origin"
    case Circle(radius=r) if r <= 0:
        return "invalid circle"
    case int(n) | float(n):
        return f"a number {n}"
    case str() as s:
        return f"a string of length {len(s)}"
    case _:
        return "unknown shape"
```

```
Point(x=0, y=0) -> the origin
Point(x=5, y=0) -> on the x-axis at 5
Point(x=1, y=2) -> some other point
Circle(center=Point(x=0, y=0), radius=2) -> circle of radius 2 at the origin
Circle(center=Point(x=1, y=1), radius=-1) -> invalid circle
7 -> a number 7
2.5 -> a number 2.5
'hello' -> a string of length 5
[1] -> unknown shape
Point.__match_args__ = ('x', 'y')
```

The capture-pattern trap

A **bare name** in a pattern is a *capture*, never a comparison. With `RED = "red"`, `case RED:` does **not** mean "equals `RED`" — it matches anything and rebinds `RED`:

```
matched?! blue
SyntaxError: name capture 'RED' makes remaining patterns unreachable
```

Python refuses to compile such a capture if more cases follow it. To compare against a constant, use a **dotted name** (value pattern) such as an `Enum` member:

```python
class Color(Enum):
    RED = "red"
    GREEN = "green"

match c:
    case Color.RED:      # compared with ==
        return "stop"
    case Color.GREEN:
        return "go"
```

```
stop go
```

(or a literal, or a guard: `case x if x == RED:`).

Trade-offs

	`match`	`if/elif`
Checks structure (length, keys, type) and extracts parts at once	✅	Manual `isinstance`, `len`, indexing
Arbitrary boolean conditions	Via guards only	✅ natural
Python version	3.10+	All
Performance	Similar — cases are tried in order, not a jump table	Similar

When not to use it

- For one or two simple boolean tests — `if` is clearer.
- To map constant keys to values — a dict is simpler (`STATUS = {200: "Success", …}`).
- In code that must run on Python ≤ 3.9.

Common mistakes

- `case RED:` with a plain variable — a capture, not a comparison (above).
- Expecting fall-through or writing `break` inside a case.
- Expecting `case [a, b]:` to match a string.

- Forgetting `case _:` and silently doing nothing for unexpected input — consider raising an error in the default.
- OR patterns binding different names (`case [x] | [x, y]:`) → `SyntaxError` ("alternative patterns bind different names").

Real-world examples

- Command-line and chat-bot command parsers (like `command()` above).
- Handling JSON messages from WebSockets or message queues by their `type` field.
- Compilers/interpreters walking an AST: `case BinOp(left, Add(), right):` — the `ast` module's node classes define `__match_args__` for exactly this.
- Handling HTTP status codes and redirect families.

In the interview

> **30-second answer:** "`match` evaluates a subject once and tries case patterns top to bottom, with no fall-through. Patterns can be literals, captures, the `_` wildcard, dotted-name constants, OR-patterns, sequences with `*rest`, mappings that ignore extra keys, and class patterns that check `isinstance` and attributes via `__match_args__`. Guards add an `if` condition. The main trap is that a bare name captures instead of comparing."

- *How do you match a constant?* → Use a dotted name (enum member, module constant), a literal, or a guard.
- *Do mapping patterns require exact keys?* → No, extra keys are ignored; use `**rest` to capture them.
- *What makes positional class patterns work?* → The class's `__match_args__`, auto-generated by `dataclass` and `namedtuple`.

Self-check

1. Does `case {"a": 1}:` match `{"a": 1, "b": 2}`? → Yes.
2. What does `case [x, *_]:` match? → Any non-string sequence with at least one item; binds the first to `x`.
3. What happens when no case matches and there's no `_`? → Nothing; execution continues after the `match`.
4. How do you capture the alternative that matched in `"a" | "b"`? → `("a" | "b") as choice`.
5. Why does `case Point(x, y)` need `__match_args__`? → To know which attributes positions 1 and 2 refer to.

TL;DR

`match` = compare one value against shape-aware patterns, first match wins, no fall-through; use literals, `|`, `[... *rest]`, `{key: ...}`, `Class(attr=...)`, guards and `as`; use dotted names for constants — bare names capture.

3.5 Loops — overview and the iteration protocol

What it is

A **loop** repeats a block of code. Python has two:

Loop	Repeats…	Use when
`for`	once per item of an **iterable**	you have a collection or a known count
`while`	as long as a **condition** stays truthy	you don't know how many repetitions in advance

Loop control statements change the normal flow: `break` (leave the loop), `continue` (skip to the next iteration), `pass` (do nothing — a placeholder). Visiting every item of a collection is called **traversal**.

How `for` really works — the iteration protocol

An **iterable** is any object that can hand out its items one at a time: lists, tuples, strings, dicts, sets, ranges, files, generators… An **iterator** is the object that keeps track of the position.

1. `for x in obj:` calls `iter(obj)` to get an iterator.
2. Each round, it calls `next(iterator)` and binds the result to `x`.
3. When `next` raises `StopIteration`, the loop ends (quietly).

```
it = iter(["a", "b"])
next(it)   # 'a'
next(it)   # 'b'
next(it)   # raises StopIteration
```

That is why one `for` statement works for every container, and why lazy objects like `range` and file handles can be looped over without loading everything into memory.

TL;DR

`for` = "for each item", driven by `iter()` / `next()`; `while` = "while condition". Control them with `break`, `continue`, `pass`.

3.6 The `for` loop

What it is

```
for variable in iterable:
    ...   # body runs once per item; variable holds the current item
```

How it works — basic traversal

From `code/loops.py`:

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit, end=" ")
for ch in "hey":
    print(ch.upper(), end="")
```

```
apple banana cherry
HEY
```

No index bookkeeping, so no off-by-one errors. The loop variable **survives** after the loop with its last value (a quirk to remember).

Looping through dictionaries

Iterating a dict directly yields its **keys** (in insertion order). Use views for the rest:

You need	Write
keys	`for k in d:` (or `d.keys()`)
values	`for v in d.values():`
both	`for k, v in d.items():`

```python
info = {"name": "John", "age": 30, "city": "New York"}
for key in info:
    print(key, "->", info[key])
for key, value in info.items():          # preferred: no second lookup
    print(f"{key}={value}", end=" ")
```

```
name -> John
age -> 30
city -> New York
name=John age=30 city=New York | values: ['John', 30, 'New York']
```

Nested loops

Every time the outer loop advances by one item, the inner loop runs from start to finish, so the body runs `rows × cols` times.

```python
matrix = [[1, 2, 3], [4, 5, 6]]
for row in matrix:
    for num in row:
        print(num, end=" ")
    print()                              # newline after each row
```

```
1 2 3
4 5 6
```

Nested loops multiply work: two loops over n items is $O(n^2)$. If the inner loop only searches for something, a set or dict lookup often removes it.

`range()` – counting loops

`range` produces integers lazily, stop **exclusive**:

Call	Produces
`range(stop)`	0, 1, …, stop−1
`range(start, stop)`	start, …, stop−1
`range(start, stop, step)`	start, start+step, … while < stop (or > stop for negative step)

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9] [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] [1, 3, 5, 7, 9] [10, 7, 4, 1]
```

`range(1, 11, 2)` gives the odd numbers 1–9; a negative step counts down. `step` can't be 0 (`ValueError`). `range` supports `len()`, indexing and fast `in`.

`enumerate()` – index and item together

Instead of `for i in range(len(names)): names[i]`, write:

```python
for i, name in enumerate(names, start=1):
    print(f"{i}. {name}", end="  ")
```

```
1. ann  2. bob  3. cy
```

`zip()` – walk several iterables in parallel

`zip(a, b)` yields tuples `(a[0], b[0]), (a[1], b[1]), …` and **stops at the shortest** input. Pass `strict=True` (3.10+) to raise an error if lengths differ – usually what you want when the sequences are supposed to align.

```python
for name, score in zip(names, scores, strict=True):
    print(f"{name}:{score}", end="  ")
```

```
ann:91  bob:78  cy:85
zip strict: zip() argument 2 is shorter than argument 1
plain zip stops at the shortest: [(1, 'a'), (2, 'b')]
```

`dict(zip(keys, values))` builds a dict from two lists.

Common mistakes

- `for i in range(len(items)):` when you only need the items (use `for item in items`) or both (use `enumerate`).
- Off-by-one with `range`: `range(1, 10)` stops at 9.
- Modifying a list while iterating over it (see §3.10).
- Expecting iteration over a dict to give pairs — it gives keys.
- Silent truncation with plain `zip` on mismatched data.

Real-world examples

- Processing rows from `csv.reader(f)` or lines of a file: `for line in f:` streams the file lazily.
- `for i, batch in enumerate(dataloader):` is the standard PyTorch training loop.
- `zip(features, labels)` pairs data in ML pipelines.

In the interview

30-second answer: "A `for` loop calls `iter()` on the iterable and `next()` until `StopIteration`, so it works on anything iterable. I iterate items directly, use `enumerate` when I need the index, `zip(strict=True)` for parallel sequences, `.items()` for dicts, and `range` for counts."

- *Why is `range(10**9)` cheap?* → It's lazy — computes values on demand.
- *What does `zip` do with unequal lengths?* → Stops at the shortest; `strict=True` raises `ValueError`.
- *Is the loop variable available after the loop?* → Yes, it keeps the last value (unless the iterable was empty).

Self-check

1. `list(range(2, 11, 3))`? → `[2, 5, 8]`.
2. How do you loop with index starting at 1? → `enumerate(seq, start=1)`.
3. How do you get keys and values from a dict? → `for k, v in d.items():`.
4. How many times does the inner body of two nested loops over 3 and 4 items run? → 12.

TL;DR

`for item in iterable`; dict → keys (use `.items()`); `range(start, stop, step)` excludes stop; `enumerate` for indices; `zip(strict=True)` for parallel lists.

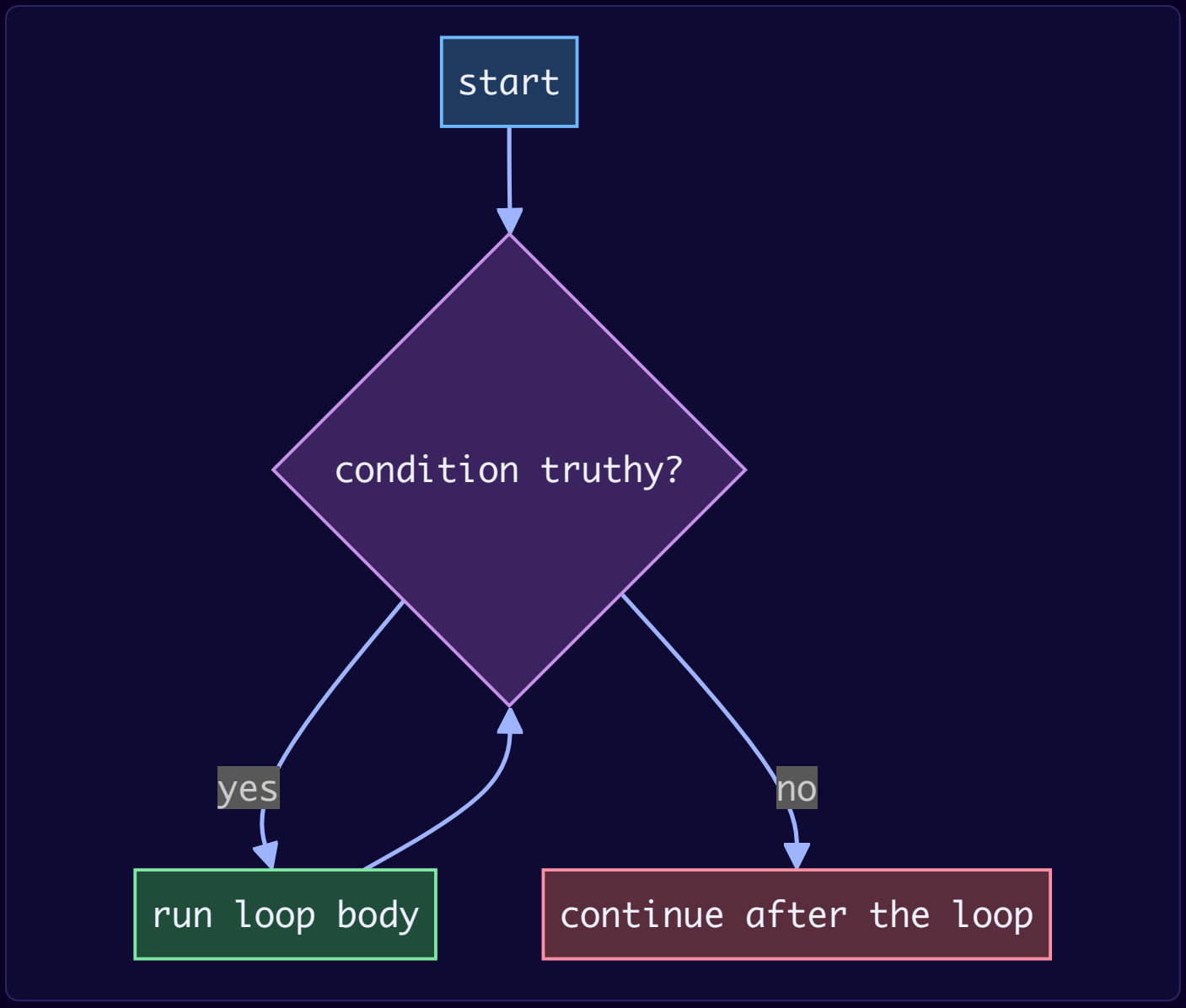
3.7 The `while` loop

What it is

```python
while condition:
    ...   # repeats while condition is truthy; re-checked before every iteration
```

Use `while` when the number of iterations isn't known up front: reading until the user types "quit", retrying until success, iterating until a value converges.

How it works



1. Check the condition. If falsy, skip past the loop (the body may run **zero** times).
2. Run the body.
3. Go back to 1.

The body **must** change something the condition depends on, or the loop never ends.

```
count = 0
while count < 5:
    print(f"Count is {count}", end="; ")
    count += 1          # progress toward making the condition false
```

```
Count is 0; Count is 1; Count is 2; Count is 3; Count is 4;
```

An example where the count is genuinely unknown in advance — how many halvings until a number drops below 1?

```
n, steps = 100.0, 0
while n >= 1:
    n /= 2
    steps += 1
```

```
halvings until < 1: 7
```

(100 → 50 → 25 → 12.5 → 6.25 → 3.125 → 1.5625 → 0.78125.)

Infinite loops on purpose

`while True:` with a `break` inside is the idiomatic "loop and a half" – e.g. a menu or a server loop:

```
while True:
    answer = input("> ")
    if answer == "quit":
        break
    handle(answer)
```

With the walrus operator this can be `while (answer := input("> ")) != "quit": handle(answer)`.

Trade-offs

	`for`	`while`
Termination	Automatic when the iterable is exhausted	Your responsibility
Typical bug	Off-by-one in `range`	Infinite loop (forgot to update)
Use for	Collections, fixed counts	Unknown counts, event loops, retries, convergence

Common mistakes

- Forgetting to update the loop variable → infinite loop (stop it with Ctrl-C, which raises `KeyboardInterrupt`).
- Using `while i < len(items): … i += 1` to traverse a list – use `for`.
- Floating-point conditions like `while x != 1.0:` that never become exactly true – use `<` / `>` or a tolerance.

Real-world examples

- Game loops: `while running: handle_events(); update(); draw()` (Pygame).
- Retry with back-off: `while attempts < 5 and not ok: …`.
- Reading a socket or file in chunks: `while (chunk := f.read(8192)): …`.

In the interview

30-second answer: "`while` re-checks its condition before each iteration and is for when the iteration count is unknown. The body must make progress toward the condition becoming false; for intentional infinite loops I use `while True` with a clear `break`."

Self-check

1. Can a `while` body run zero times? → Yes, if the condition is falsy at the start.
2. What key stops a runaway loop in the terminal? → Ctrl-C (`KeyboardInterrupt`).
3. Rewrite `i = 0; while i < 3: print(i); i += 1` as a `for`. → `for i in range(3): print(i)`.

TL;DR

`while cond:` repeats until `cond` is falsy; guarantee progress; `while True` + `break` for loop-and-a-half.

3.8 Loop control: `break`, `continue`, `pass`

What it is

Statement	Effect
`break`	Exit the **innermost** enclosing loop immediately; skips any loop `else`
`continue`	Skip the rest of this iteration; go to the next one (for `while`, the condition is re-checked)
`pass`	Do nothing; fills a place where syntax requires a statement

How it works

```
for number in [1, 2, 3, 4, 5, 6]:
    if number == 4:
        print("Number 4 found, stopping loop.")
        break                                   # 5 and 6 are never visited

odds = []
for number in [1, 2, 3, 4, 5, 6]:
    if number % 2 == 0:
        continue                                # skip evens
    odds.append(number)

for number in [1, 2, 3]:
    if number % 2 == 0:
        pass                                    # placeholder - does nothing
    print(number, end=" ")                      # still runs for every number
```

```
Number 4 found, stopping loop.
continue kept odds: [1, 3, 5]
1 2 3
```

`pass` is **not** `continue`: after `pass`, the rest of the body still executes. Use `pass` for empty stubs – `def todo(): pass`, `class MyError(Exception): pass` – or `...` (Ellipsis), which works the same way in bodies.

`break` exits only one level:

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break                   # leaves the inner loop only
        print(f"({i},{j})", end=" ")
```

```
(0,0) (1,0) (2,0)
```

To exit several levels, move the loops into a function and `return`, or use a flag.

Common mistakes

- Expecting `break` to leave all nested loops.
- Using `pass` where `continue` was meant (or vice versa).
- `continue` in a `while` loop placed *before* the counter update → infinite loop.
- Over-using `continue` for filtering when a comprehension (`[x for x in xs if cond]`) is clearer.

In the interview

> **30-second answer:** "`break` leaves the innermost loop, `continue` jumps to the next iteration, `pass` is a no-op placeholder. To break out of nested loops I refactor into a function and `return`."

Self-check

1. Does `pass` skip the rest of the loop body? → No.
2. In a `while` loop, what happens right after `continue`? → The condition is re-evaluated.
3. How do you exit two nested loops at once? → `return` from a function (or a flag / exception).

TL;DR

`break` = leave loop, `continue` = next iteration, `pass` = do nothing. All act on the innermost loop.

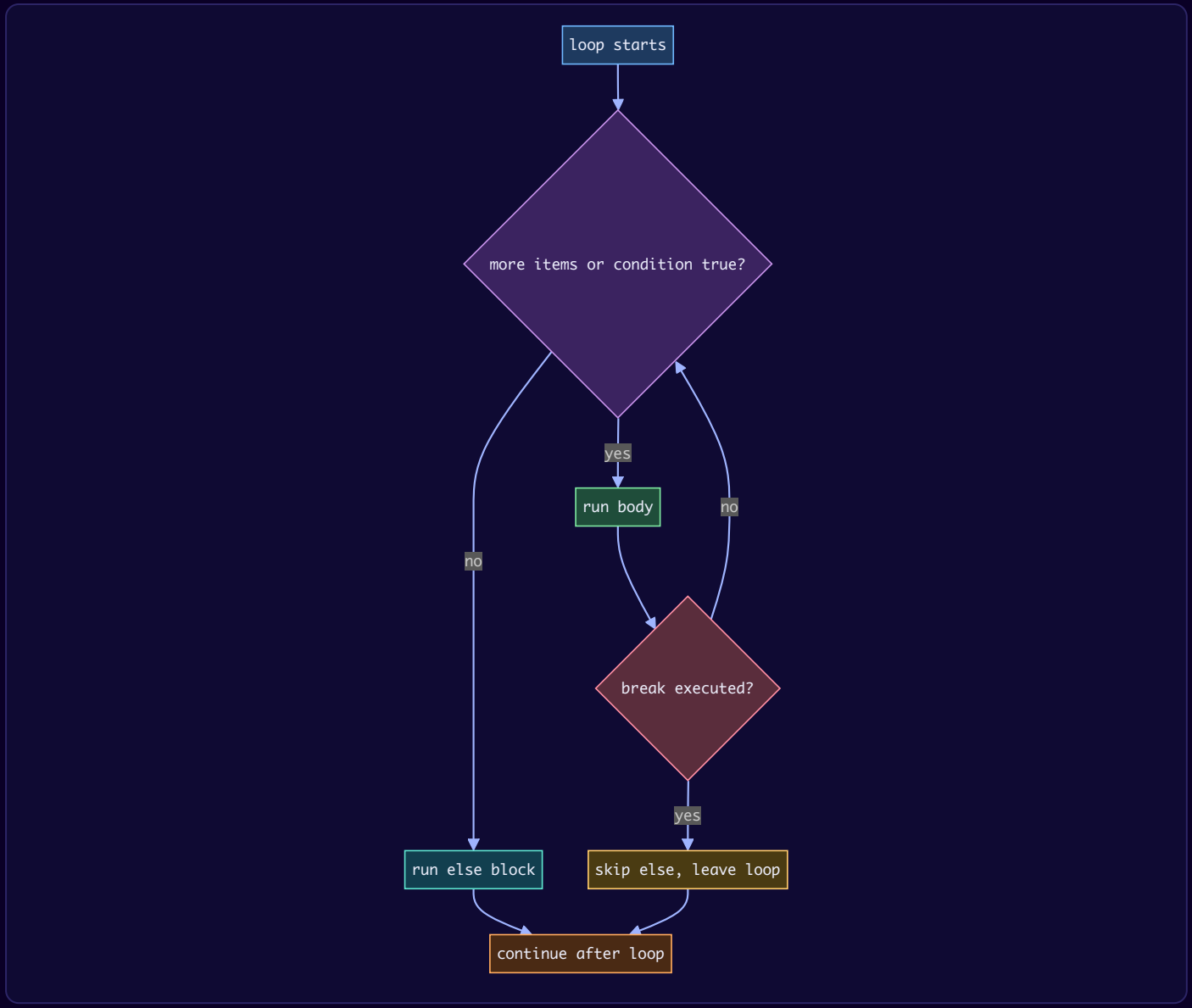
3.9 `for-else` and `while-else`

What it is

Loops can have an `else` block that runs **only if the loop finished without hitting `break`** — the iterable was exhausted (`for`) or the condition became false (`while`). Read it as **"no break"**. Its classic use: search loops, where `break` means "found" and `else` means "not found" — no flag variable needed.

```
for item in iterable:
    ...
else:
    ...   # runs only if the loop didn't break
```

How it works



From `code/loops.py`:

```
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        print("Number 3 found, stopping search.")
        break
else:
    print("Number not found.")        # skipped: break happened

for number in [1, 2, 4, 5]:
    if number == 3:
        print("found 3")
        break
else:
    print("3 not in list (else ran: no break)")

count = 0
while count < 5:
    if count == 2:
        print("Breaking at count 2.")
        break
    count += 1
else:
    print("Loop was uninterrupted.")  # skipped: break happened
```

```
Number 3 found, stopping search.
3 not in list (else ran: no break)
Breaking at count 2.
```

Without any `break` statement in the body, the `else` always runs — e.g. "All numbers processed." after a plain processing loop — which is legal but pointless (just put the code after the loop).

A practical search — primality by trial division:

```python
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True
```

```
is 29 prime? True | is 21 prime? False
```

(Inside a function, `return` plays the role of `break`, and the final `return True` plays the role of `else`. At module level or when you need to continue afterwards, `for-else` is the flag-free equivalent.)

Trade-offs / when not to use it

Many developers misread `else` as "if the loop didn't run". If your team finds it confusing, a helper function with `return`, or `any()` / `next()` (§3.10), is often clearer. Always add a comment like `# no break` when you use it.

Common mistakes

- Thinking `else` runs when the loop body never executed only — it also runs after a normal full run.
- Thinking `else` runs after `break` — it doesn't.
- Indenting the `else` to pair with an `if` inside the loop by accident (or vice versa) — alignment decides which statement it belongs to.

In the interview

> **30-second answer:** "A loop's `else` runs only when the loop wasn't terminated by `break` — think 'no break'. It's handy for search loops: `break` on found, `else` for not-found, without a flag variable."

- *Does `else` run if the iterable is empty?* → Yes, there was no break.
- *Does `return` inside the loop skip `else`?* → Yes, it leaves the function entirely.

Self-check

1. `for x in []: pass` `else: print("E")` — prints? → `E`.
2. Is `while-else` `else` run after `break`? → No.
3. What's the mnemonic? → "else = no break".

TL;DR

Loop `else` runs when the loop ends naturally, not via `break`. Use it for search-with-not-found; comment it.

3.10 Common loop idioms

Aggregation with built-ins

Many loops are just a built-in in disguise — shorter, faster (implemented in C) and clearer:

```
nums = [4, -1, 7, 0, 3]
sum(nums), min(nums), max(nums)
any(x < 0 for x in nums)                  # True if at least one; stops early
all(x < 10 for x in nums)                 # True if every one; stops early
[x * x for x in nums if x > 0]            # list comprehension: map + filter
list(reversed(nums)), sorted(nums, reverse=True)
next((x for x in nums if x > 5), None)    # first match or a default
```

```
sum/min/max: 13 -1 7
any negative? True | all < 10? True
squares of positives: [16, 49, 9]
reversed: [3, 0, 7, -1, 4] | sorted desc: [7, 4, 3, 0, -1]
first > 5: 7
```

`next(generator, default)` replaces a whole `for … break … else` search when you just need the first match. Comprehensions get a full section in the lists chapter.

Counting / grouping with a dict

```
counts = {}
for word in "the cat and the hat".split():
    counts[word] = counts.get(word, 0) + 1
```

```
word counts: {'the': 2, 'cat': 1, 'and': 1, 'hat': 1}
```

(`collections.Counter` does this in one call; covered with dictionaries.)

Don't change a collection while looping over it

Removing items from a list during iteration shifts the remaining items left, so the loop **skips** the next one:

```
buggy = [1, 2, 2, 3]
for x in buggy:
    if x == 2:
        buggy.remove(x)        # the second 2 slides into the visited slot and is skipped
```

```
remove-while-iterating left: [1, 2, 3] | filtered correctly: [1, 3]
RuntimeError: dictionary changed size during iteration
```

Fixes: build a new list (`[x for x in data if x != 2]`), or iterate over a copy (`for x in data[:]:`). For dicts, Python refuses outright with `RuntimeError`; iterate over `list(d)` if you must add/delete keys.

Idiom cheat-sheet

Instead of…	Write…
`for i in range(len(xs)): print(xs[i])`	`for x in xs: print(x)`
index + item via `range(len())`	`for i, x in enumerate(xs):`
parallel indexing `a[i]`, `b[i]`	`for x, y in zip(a, b, strict=True):`
`for k in d: v = d[k]`	`for k, v in d.items():`
looping backwards with indices	`for x in reversed(xs):`
accumulator loop for total	`sum(xs)`
flag + loop to test a condition	`any(...)` / `all(...)`
loop with `append` to transform	list comprehension
`while` loop reading chunks	`while (chunk := f.read(n)):`

Real-world examples

- Log analysis scripts: `for line in open(path): if "ERROR" in line: …`.
- ETL jobs aggregating per-key totals with dicts before loading to a database.
- Validation: `if not all(field in payload for field in REQUIRED): return 400`.

In the interview

> **30-second answer:** "I replace hand-written loops with built-ins where possible – `sum`, `any`, `all`, `min` / `max`, `sorted`, comprehensions, `next` with a default – because they're clearer and run in C. I never mutate a list or dict while iterating it; I build a new one or iterate a copy."

- *Why does removing during iteration skip items?* → The list iterator advances by index; removal shifts later items down one slot.
- *Do `any` / `all` evaluate everything?* → No, they short-circuit.

Self-check

1. `all([])`? `any([])`? → `True`, `False`.
2. Safe way to delete items matching a condition? → Rebuild: `xs = [x for x in xs if not cond(x)]`.
3. First even number or `None` in one line? → `next((x for x in xs if x % 2 == 0), None)`.

TL;DR

Prefer built-ins and comprehensions over manual loops; `enumerate` / `zip` / `.items()` over indexes; never mutate what you're iterating.

Chapter 3 TL;DR

- `if/elif/else`: first truthy branch wins; order from specific to general; guard clauses keep code flat; `a if c else b` for values.
- `match/case` (3.10+): one subject vs patterns – literals, `|`, captures, `_`, dotted constants, sequences with `*rest`, mappings (extra keys ignored), class patterns (`__match_args__`), guards, `as`; no fall-through; bare names capture.
- `for` loops use the iteration protocol (`iter` / `next` / `StopIteration`) and work on any iterable; dicts yield keys; use `range`, `enumerate`, `zip(strict=True)`, `.items()`.
- `while` loops run while a condition holds – guarantee progress; `while True` + `break` for loop-and-a-half.
- `break` leaves the innermost loop, `continue` skips to the next iteration, `pass` does nothing.

- Loop `else` = "no break happened" — ideal for search loops.
- Reach for `sum` / `any` / `all` / `next` /comprehensions first, and never modify a collection while iterating over it.

Chapter 4 — Strings

Text is everywhere: user names, file paths, log lines, JSON, HTML, SQL, chat messages. In Python all of that text lives in one type, `str`. This chapter takes you from "what is a string" to the things intermediate developers are expected to know cold: immutability, slicing, the formatting mini-language, the most useful methods, how text becomes bytes (Unicode and UTF-8), how to build big strings without slowing to a crawl, and a first working knowledge of regular expressions.

What you'll learn

1. Creating strings with different quotes, indexing them and looping over them.
2. Why strings are immutable and how to "change" one anyway.
3. String operators (`+`, `*`, `in`, comparisons, `%`) and slicing with start/stop/step.
4. Escape sequences and raw strings.
5. Formatting with f-strings, the format-spec mini-language, `str.format()` and `%`.
6. The string methods you will actually use, with a reference table and their costs.
7. How to join many pieces efficiently.
8. Unicode vs bytes, encodings and UTF-8.
9. Regular expressions: `search`, `match`, `findall`, `sub` and groups.

All runnable code is in `code/`. Every output shown below was produced by running those files with Python 3.14 (the code also runs on 3.12+).

4.1 What a string is — creating, indexing, looping

What it is

A **string** (`str`) is an ordered, immutable sequence of Unicode characters. *Ordered* means each character has a position; *immutable* means that once the string exists its characters can never be changed; *sequence* means it supports the same tools as lists and tuples — `len()`, indexing, slicing, `in`, and `for` loops.

Analogy: a string is a printed line of text on paper. You can read any letter, photocopy parts of it, or print a new page — but you can't erase a letter on the existing page.

Python has no separate "character" type. A single character is simply a string of length 1: `type("x")` is `str` and `len("x")` is `1`.

Why it matters

Almost every program reads, cleans, or produces text. Knowing the exact rules (zero-based indexes, negative indexes, what raises an error and what doesn't) saves a lot of off-by-one bugs.

How it works

1. **Create** a string with any of three quote styles — they all produce the same `str` type:
   - `'single'` — the usual choice for short strings; convenient when the text contains a `"`.
   - `"double"` — convenient when the text contains an apostrophe: `"It's raining"`.
   - `"""triple"""` or `'''triple'''` — can span several lines and freely contain both kinds of quote. Also used for docstrings.
2. **Index** with `s[i]`. Positions start at `0` on the left. **Negative indexes** count from the right: `-1` is the last character, `-2` the one before it. For a string of length *n*, valid indexes are `0 … n-1` and `-n … -1`; anything else raises `IndexError`.
3. **Loop** with `for ch in s:` — you get one character per iteration. Use `enumerate(s)` when you also need the position.

```
index:   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
char :   L   e   a   r   n   i   n   g   ␣   P   y   t   h   o   n
neg  : -15 -14 -13 -12 -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1
```

```python
# code/01_string_basics.py (excerpt)
single = 'Hi there'
double = "It's raining"                     # apostrophe inside double quotes
triple = """Line one
Line two with "quotes" and 'apostrophes'"""
print(single, "|", double)
print(triple)
print(type("x"), len("x"))                  # a single character is just a str of length 1

word = "Learning Python"
print(word[0], word[9], word[-1], word[-6])  # L P n P
try:
    word[100]
except IndexError as e:
    print("IndexError:", e)

for i, ch in enumerate("abc"):
    print(f"{i}:{ch}", end=" ")
```

Output:

```
Hi there | It's raining
Line one
Line two with "quotes" and 'apostrophes'
<class 'str'> 1
L P n P
IndexError: string index out of range
0:a 1:b 2:c
```

`print(x, end=" ")` replaces the default newline at the end of each `print` with a space, which is handy when looping over characters.

Common mistakes

- Forgetting that indexing starts at 0: the 8th character is `s[7]`.
- Using `for i in range(len(s)): s[i]` when you only need the characters — just write `for ch in s`.
- Mixing quote styles incorrectly: `'It's'` ends the string at the apostrophe → `SyntaxError`. Use `"It's"` or `'It\'s'`.

Self-check

1. What is `"Python programming is fun!"[-4]`? → `'f'` (the fourth character from the end: `f`, `u`, `n`, `!`).
2. Does Python have a `char` type? → No; a character is a `str` of length 1.
3. Which quote style lets you write a multi-line string without `\n`? → Triple quotes.

TL;DR

`str` = immutable, ordered sequence of Unicode characters. Three quote styles, zero-based indexes, negative indexes from the end, `for ch in s` to loop.

4.2 Immutability — and how to "update" a string

What it is

Immutable means an object's value cannot change after creation. For strings, `s[0] = "J"` is illegal. Every operation that looks like it changes a string — `upper()`, `replace()`, `+` — actually builds and returns a **new** string.

Why it matters

- **Safety:** you can pass a string to any function and know it will come back unchanged. No defensive copies needed.
- **Hashability:** because a string's value never changes, its hash never changes, so strings can be dictionary keys and set members (Chapter 6). CPython even caches a string's hash after computing it once.
- **Sharing:** the interpreter may reuse one string object in several places (for example, identifiers are *interned* — stored once and shared), which is only safe because nobody can mutate it.

How it works

1. You try item assignment → `TypeError`.
2. To "update", slice the parts you want to keep and concatenate them with the new text.
3. Assign the result back to the same name. The `name` now points to a new object; the old object is untouched (and is garbage-collected if nothing else refers to it).

```
s = "Hello, world!"
try:
    s[0] = "J"                          # not allowed
except TypeError as e:
    print("TypeError:", e)
before = id(s)
s = s[:7] + "Python!"                   # build a NEW string, rebind the name
print(s, "| same object?", id(s) == before)
```

Output:

```
TypeError: 'str' object does not support item assignment
Hello, Python! | same object? False
```

`s[:7]` is `"Hello, "` (positions 0–6), and `+ "Python!"` appends the new ending. `id()` returns an object's identity; it differs, proving a new object was created.

Trade-offs

	Immutable `str`	Mutable alternative (`list` of chars, `bytearray`, `io.StringIO`)
Safe to share / use as dict key	Yes	No
Change one character	O(n) — copy the whole string	O(1)
Build text piece by piece	Use `"".join(parts)` (see 4.8)	Natural

Common mistakes

- Calling a method and ignoring the result: `name.strip()` on its own line does nothing useful. Write `name = name.strip()`.
- Thinking `s += "x"` modifies `s` in place — it creates a new string (see 4.8 for why that matters in loops).

In the interview

> **30-second answer:** "Python strings are immutable: every 'modifying' operation returns a new string. That makes them safe to share and hashable, so they work as dict keys. The cost is that repeated modification copies data, so for building large strings you collect parts in a list and `join` them."

- *Q: If strings are immutable, how does `s += 'x'` work?* A: It evaluates `s + 'x'` into a new string and rebinds the name `s`. (CPython sometimes resizes in place when no other reference exists — an optimisation you must not rely on.)
- *Q: Why must dict keys be immutable?* A: Strictly, they must be *hashable*: the hash must not change while the key is in the dict, otherwise the lookup would search the wrong slot.

Self-check

1. What does `s.replace("a", "b")` do to `s`? → Nothing; it returns a new string.
2. Why can a `str` be a set element but a `list` can't? → `str` is immutable and hashable; `list` is mutable and unhashable.
3. What does `"Hello, world!"[:7] + "Python!"` give? → `'Hello, Python!'`.

TL;DR

You never change a string; you make a new one and rebind the name. Immutability buys safety and hashability.

4.3 String operators

What it is

Operators are the symbols Python lets you use directly on strings.

Operator	Meaning	Example	Result
`+`	Concatenation (join two strings)	`"Py" + "thon"`	`'Python'`
`*`	Repetition (int times)	`"Ha" * 3`	`'HaHaHa'`
`in`	Substring membership	`"fox" in "The quick brown fox"`	`True`
`not in`	Negated membership	`"cat" not in "The quick brown fox"`	`True`
`==`, `!=`	Equal / not equal (by value)	`"Python" == "Python"`	`True`
`<`, `>`, `<=`, `>=`	Lexicographic (dictionary-order) comparison	`"apple" < "banana"`	`True`
`%`	printf-style formatting (legacy, see 4.6)	`"%s!" % "hi"`	`'hi!'`
`[i]`, `[a:b:c]`	Indexing and slicing (see 4.4)	`"abc"[1]`	`'b'`

How it works

1. `+` needs **two strings**. `"Age: " + 30` raises `TypeError`; convert first (`str(30)`) or use an f-string.
2. `*` with zero or a negative number gives the empty string `''`.
3. `in` checks for a **substring**, not just a single character: `"row" in "brown"` is `True`.
4. Comparisons go character by character, comparing **Unicode code points** (the number assigned to each character — see 4.9). The first difference decides. If one string is a prefix of the other, the shorter one is smaller (`"app" < "apple"`).
   - Worked example: `"Zebra" < "apple"` → compare `'Z'` (code point 90) with `'a'` (97) → 90 < 97 → `True`. Uppercase letters sort before all lowercase ones.
   - `"10" < "9"` is `True`, because `'1'` (49) < `'9'` (57). Strings that look like numbers do **not** compare as numbers.

```
print("Ha" * 3, "+".join(["a", "b"]), "Py" + "thon")
print("fox" in "The quick brown fox", "cat" not in "The quick brown fox")
print("apple" < "banana", "Zebra" < "apple", "10" < "9")  # compares code points
```

Output:

```
HaHaHa a+b Python
True True
True True True
```

Common mistakes

- Sorting user names with plain `<` and being surprised that `"Zoe"` comes before `"adam"`. Sort with `key=str.casefold` for case-insensitive order.
- Comparing numeric strings: convert with `int()` / `float()` first.
- Using `is` to compare strings. `is` checks object identity, not equality; it may happen to work for short interned strings and then fail elsewhere. Always use `==`.

Self-check

1. `"Echo " * 3`? → `'Echo Echo Echo '`.
2. `"H" in "Hello"`? → `True`.
3. Is `"Banana" >= "Apple"` true? → Yes, `'B'` (66) > `'A'` (65).

TL;DR

`+` joins, `*` repeats, `in` finds substrings, comparisons are by code point (so case and digits matter), and use `==`, never `is`.

4.4 Slicing

What it is

Slicing extracts a piece of a sequence using `s[start:stop:step]`. It returns a new string.

- `start` – first position included (default: the beginning).
- `stop` – first position **excluded** (default: the end).
- `step` – how far to jump each time (default `1`; negative walks backwards).

Why it matters

Slicing is the everyday tool for pulling out fixed-position fields, dropping prefixes/suffixes, reversing, and taking every *n*-th item. The exact same syntax works on lists, tuples, bytes and arrays (Chapter 5).

How it works

1. Resolve negative numbers by adding `len(s)`: with `len("Hello, Python!") == 14`, `-7` means `7`.
2. Fill in defaults for missing parts.
3. Walk from `start` towards `stop` in jumps of `step`, collecting characters, stopping **before** `stop`.
4. Out-of-range bounds are clipped silently – slicing never raises `IndexError`.

The half-open rule `[start, stop)` gives two handy facts: `len(s[a:b]) == b - a`, and `s[:k] + s[k:] == s` for any `k`.

```
text = "Hello, Python!"
         H  e  l  l  o  ,  ␣  P  y  t  h  o  n  !
index:   0  1  2  3  4  5  6  7  8  9 10 11 12 13
neg:   -14                   -7 -6 -5 -4 -3 -2 -1
```

```
text = "Hello, Python!"
print(repr(text[7:13]))    # 'Python'
print(repr(text[-7:-1]))   # 'Python'
print(repr(text[:5]), repr(text[7:]))
print(repr(text[::2]))     # every 2nd char: 'Hlo yhn'
print(repr(text[::-1]))    # reversed
print(repr(text[3:100]))   # slices never raise IndexError -> clipped
```

Output:

```
'Python'
'Python'
'Hello' 'Python!'
'Hlo yhn'
'!nohtyP ,olleH'
'lo, Python!'
```

Walk-through of `text[::2]`: positions 0, 2, 4, 6, 8, 10, 12 → `H`, `l`, `o`, `␣` (the space at 6), `y`, `h`, `n` → `'Hlo yhn'`. Note there is no comma in the result: the comma sits at odd position 5, and the space at position 6 is kept. Likewise `text[-5:-1]` is `'thon'` (positions 9–12), and `text[:6]` is `'Hello,'`.

Common mistakes

- Expecting `stop` to be included: `text[7:12]` is `'Pytho'`, not `'Python'`.
- Using a negative step with the default bounds swapped in your head: `s[5:1:-1]` walks from 5 down to 2; `s[1:5:-1]` is empty.
- Reversing a string that contains combining characters or emoji with modifiers – `[::-1]` reverses code points, which can detach accents (see 4.9).

In the interview

> **30-second answer:** "`s[start:stop:step]` returns a new sequence from `start` up to but not including `stop`. Omitted bounds default to the ends, negatives count from the end, bounds are clipped so it never raises, and `s[::-1]` reverses. It costs O(k) for k items copied."

- *Q: Why is `stop` exclusive?* A: So `s[:k] + s[k:] == s` and `len(s[a:b]) == b - a` – no ±1 corrections.
- *Q: Is palindrome check `s == s[::-1]` O(n)?* A: Yes – O(n) time and O(n) extra space for the reversed copy; a two-pointer loop does it in O(1) extra space.

Self-check

1. `"Hello, Python!"[-5:-1]`? → `'thon'`.
2. `"abcdef"[::3]`? → `'ad'`.
3. `"abc"[5:]`? → `''` (no error).

TL;DR

`[start:stop:step]`, stop excluded, negatives from the end, never raises, `[::-1]` reverses, cost is proportional to the slice length.

4.5 Escape sequences and raw strings

What it is

An escape sequence is a backslash followed by one or more characters that together stand for a single special character – for example `\n` is one newline character. A raw string (prefix `r`) turns escape processing off so every backslash is kept literally.

Why it matters

You need escapes to put newlines, tabs, quotes and arbitrary Unicode characters in string literals. You need raw strings for text that is *full of* backslashes: regular expressions and Windows paths.

How it works

Escape	Meaning	Example	Result
`\\`	Backslash	`"a\\b"`	`a\b`
`\'`	Single quote	`'It\'s'`	`It's`
`\"`	Double quote	`"He said \"hi\""`	`He said "hi"`
`\n`	Newline (line feed)	`"one\ntwo"`	two lines
`\t`	Horizontal tab	`"a\tb"`	`a` tab `b`
`\r`	Carriage return (cursor to line start)	`"Hello\rWorld"`	terminal shows `World`
`\b`	Backspace character (code 8)	`"ABC\b"`	terminal may erase `C` visually
`\f`	Form feed (page break for printers)	`"Page\fNext"`	rarely visible
`\ooo`	Octal code (up to 3 digits)	`"\101"`	`A`
`\xhh`	Hex code (exactly 2 digits)	`"\x41"`	`A`
`\uXXXX`	Unicode code point (4 hex digits)	`"\u00e9"`	`é`
`\UXXXXXXXX`	Unicode code point (8 hex digits)	`"\U0001F600"`	😀
`\N{name}`	Unicode character by name	`"\N{GREEK SMALL LETTER PI}"`	`π`

`\r` and `\b` are real characters in the string; what you *see* depends on the terminal. `repr("ABC\b")` shows `'ABC\x08'`.

```
print("Tab\tseparated\nnew line, quote: \" backslash: \\")
print("\x41\101\u00e9\N{GREEK SMALL LETTER PI}")   # A A é π
path = r"C:\new\table"                              # raw: backslashes kept literally
print(path, len(path), len("C:\new\table"))         # raw has 12 chars, cooked has 10
```

Output:

```
Tab     separated
new line, quote: " backslash: \
AAéπ
C:\new\table 12 10
```

Without `r`, `"C:\new\table"` silently contains a newline (`\n`) and a tab (`\t`) – 10 characters instead of 12.

Common mistakes

- Writing Windows paths without `r` (or without doubling backslashes). Better: use forward slashes or `pathlib.Path`, which work on Windows too.
- Unknown escapes like `"\d"`: Python keeps the backslash but warns (a `SyntaxWarning` since Python 3.12). Write regexes as raw strings: `r"\d+"`.
- A raw string cannot end in a single backslash: `r"C:\"` is a syntax error (the `\"` still stops the string from ending). Use `"C:\\"`.

Self-check

1. `len("a\nb")`? → 3.
2. What does `r"\n"` contain? → Two characters: a backslash and `n`.
3. Why write regexes as raw strings? → So `\d`, `\b`, `\s` reach the regex engine unchanged; `"\b"` without `r` is a backspace character.

TL;DR

Backslash escapes produce special characters; `r"..."` disables them — use it for regexes and Windows paths.

4.6 String formatting: f-strings, the format-spec mini-language, `str.format`, `%`

What it is

Formatting means building a string from a template plus values: `"Asha is 31 years old."` from `name` and `age`. Python has four ways:

Style	Example	Introduced	Use today
f-string (formatted string literal)	`f"{name} is {age}"`	3.6	Default choice
`str.format()`	`"{} is {}".format(name, age)`	2.6 / 3.0	Templates stored as data
`%` (printf-style)	`"%s is %d" % (name, age)`	Original	Legacy code, `logging` calls
`string.Template`	`Template("$name is $age")`	2.4	Templates written by untrusted users

(Python 3.14 also added **t-strings**, `t"..."`, which produce a `string.templatelib.Template` object (unrelated to `string.Template`) instead of a `str` so that libraries can escape values safely — for example for HTML or SQL. They are new and still niche; f-strings remain the everyday tool.)

Why it matters

Formatting is how you print reports, build log lines, show money with two decimals, pad columns in a table, and render numbers in hex or percent. The **format-spec mini-language** after the `:` is shared by f-strings, `str.format()` and the built-in `format()` function, so learning it once pays off everywhere.

How it works — f-strings

1. Put `f` before the opening quote.
2. Inside `{ }` write **any expression**: a name, arithmetic, a method call, an index.
3. Optionally add a **conversion**: `!r` (use `repr()`), `!s` (use `str()`), `!a` (use `ascii()`).
4. Optionally add `:format_spec` to control the look.
5. `{expr=}` (3.8+) prints the expression text *and* its value — great for debugging.
6. Write `{{` and `}}` for literal braces.

```
name, age = "Asha", 31
print(f"{name} is {age} years old.")
print(f"Next year: {age + 1}, upper: {name.upper()}")
print(f"{name!r} vs {name!s}")              # !r calls repr(), !s calls str()
width = 10
print(f"[{name:>{width}}]")                  # nested replacement field for the width
total = 42
print(f"{total = }")                         # self-documenting (3.8+)
print(f"{{literal braces}}")
```

Output:

```
Asha is 31 years old.
Next year: 32, upper: ASHA
'Asha' vs Asha
[      Asha]
total = 42
{literal braces}
```

Since Python 3.12 (PEP 701) f-strings are parsed by the real grammar, so you may reuse the same quote type inside the braces (`f"{d["key"]}"`), use backslashes, and split long expressions over lines. On 3.11 and earlier, use the other quote type inside.

How it works — the format-spec mini-language

The spec has this shape (every part optional):

```
[[fill]align][sign][#][0][width][grouping][.precision][type]
```

Part	Options	Meaning
fill	any character	Pad with this character (needs an align)
align	`<` `>` `^` `=`	Left, right, centre, pad after the sign (numbers)
sign	`+` `-` space	Always show sign / only minus (default) / space for positives
`#`		Alternate form: `0x`, `0o`, `0b` prefixes
`0`		Zero-pad numbers to the width
width	integer	Minimum total width
grouping	`,` or `_`	Thousands separator
.precision	integer	Digits after the point (`f`, `%`), significant digits (`g`), or max chars (strings)
type	`s` `d` `b` `o` `x` `X` `c` `e` `f` `g` `%`	String, decimal int, binary, octal, hex, char, scientific, fixed, general, percent

Defaults: strings align left, numbers align right.

```
pi, n, ratio = 3.14159265, 1234567, 0.4567
# each row: spec used -> result
```

Output of `code/02_formatting.py` (spec on the left, value on the right):

```
{:.2f}                -> '3.14'
{:10.3f}              -> '     3.142'
{:<8}|                -> 'ab      |'
{:^8}|                -> '   ab   |'
{:*>8}                -> '******ab'
{:,}                  -> '1,234,567'
{:_}                  -> '1_234_567'
{:.1%}                -> '45.7%'
{:e}                  -> '1.234567e+06'
{:+d}                 -> '+42'
{:05d}                -> '00042'
{:b} {:o} {:x} {:X}   -> '11111111 377 ff FF'
{:#x}                 -> '0xff'
{:%Y-%m-%d}           -> '2024-03-09'
```

Worked example: `f"{pi:10.3f}"` → type `f` (fixed point), precision 3 → `3.142` (5 characters, rounded), width 10 → right-align (numbers default right) with 5 spaces in front → `' 3.142'`.

The last row shows that objects can define their own spec: `date` passes everything after `:` to `strftime`. Your own classes can do the same by implementing `__format__`.

How it works – `str.format()` and `%`

```
print("{} + {} = {}".format(2, 3, 5))
print("{1} before {0}".format("A", "B"))
print("{name} has {n} cats".format(name="Li", n=2))
user = {"name": "Li", "n": 2}
print("{name} has {n} cats".format(**user))
template = "Dear {who}, your balance is {amt:,.2f}"   # reusable template stored as data
print(template.format(who="Sam", amt=12345.6))
print("%s is %d years, pi~%.2f" % ("Asha", 31, 3.14159))  # old printf style
```

Output:

```
2 + 3 = 5
B before A
Li has 2 cats
Li has 2 cats
Dear Sam, your balance is 12,345.60
Asha is 31 years, pi~3.14
```

- `{}` fills positionally, `{1}` by index, `{name}` by keyword; `**user` unpacks a dict into keywords.
- `str.format()` shines when the **template is data** (loaded from a file or config) – an f-string must be written in source code.
- `%` uses `%s` (str), `%d` (int), `%f` (float), `%x` (hex), `%%` (literal percent). With a single value that is itself a tuple it misbehaves, which is one reason newer styles won.

Trade-offs

	f-string	str.format	%	string.Template
Readability	Best – value next to its slot	Good	Fair	Good
Speed	Fastest (compiled into the function)	Slower (method call + parse)	Similar to format	Slowest
Template can be data	No	Yes	Yes	Yes
Safe for user-written templates	n/a	No – `{obj.__class__...}` can reach attributes	Mostly	Yes – only `$name` substitution
Lazy formatting in logging	No	No	Yes (`logger.info("%s", x)`)	No

When not to use it

- **Logging:** write `logger.info("user %s logged in", user)` rather than an f-string. The `logging` module only formats the message if that level is enabled, and log aggregators can group messages by the unformatted template.
- **SQL:** never format values into SQL text. Use the driver's parameters (`cursor.execute("... WHERE id = ?", (user_id,))`) to avoid SQL injection.
- **User-supplied templates:** don't call `user_template.format(**objects)` – the replacement fields can walk attributes (`{config.__class__.__init__.__globals__}`) and leak data. Use `string.Template` or a sandboxed

template engine.

Common mistakes

- Forgetting the `f` prefix → the braces print literally.
- `f"{price:.2}"` means **2 significant digits** (`g`-like), not two decimals. For money write `:.2f` (and use `decimal.Decimal` for real money arithmetic).
- `"%s" % some_tuple` raising "not all arguments converted" — wrap it: `"%s" % (some_tuple,)`.

Real-world examples

- Python's `logging` module merges message arguments with `%`-style (`logger.info("%s", x)`); `style="{"` only changes the syntax of the *Formatter's* layout string, not of the messages.
- `datetime`, `Decimal` and `ipaddress` objects implement `__format__`, so `f"{now:%H:%M}"` works.
- Configuration and email templates are typically stored as data and filled with `str.format` or a template engine such as Jinja2.

In the interview

> **30-second answer:** "Use f-strings by default: readable and fastest. After the colon is the format-spec mini-language — fill, align, sign, width, grouping, precision, type — e.g. `{x:>10,.2f}`. Use `str.format` when the template is stored as data, `%` for logging's lazy formatting, and `string.Template` for untrusted templates."

- *Q: What does `f"{x=}"` do?* A: Prints `x=` followed by `repr(x)` — a debugging shortcut from 3.8.
- *Q: How do you make your class support `f"{obj:custom}"`?* A: Implement `__format__(self, spec)`.
- *Q: Why not f-strings in log calls?* A: The string is built even when the level is disabled, and it prevents grouping by message template.

Self-check

1. `f"{7:03d}"`? → `'007'`.
2. `f"{0.256:.0%}"`? → `'26%'`.
3. `f"{'hi':=^6}"`? → `'==hi=='`.
4. Which style is safe for user-written templates? → `string.Template`.

TL;DR

f-strings everywhere; learn `[[fill]align][sign][#][0][width][,][.prec][type]`; `str.format` for stored templates; `%` for logging; `Template` for untrusted input.

4.7 String methods

What it is

A **method** is a function attached to an object, called with a dot: `text.upper()`. `str` has about 47 methods. Because strings are immutable, none of them change the original; the transforming ones **return a new string**, and the checking ones return `True` / `False` or a number.

Why it matters

Most text-processing tasks — cleaning CSV fields, validating input, parsing log lines — are a few method calls, and they run in optimised C code, much faster than a hand-written Python loop.

How it works — the reference table

Case conversion

Method	What it returns
`upper()` / `lower()`	All letters upper / lower case
`capitalize()`	First character upper, the rest lower
`title()`	First letter of each word upper, rest lower
`swapcase()`	Each letter's case flipped
`casefold()`	Aggressive lowercase for caseless comparison (`"ß"` → `"ss"`)

Alignment and padding

Method	What it returns
`center(w, fill=" ")`	Centred in width `w`
`ljust(w, fill)` / `rjust(w, fill)`	Left / right justified in width `w`
`zfill(w)`	Left-padded with zeros (keeps a leading sign)
`expandtabs(n)`	Tabs replaced by spaces

Stripping, splitting, joining

Method	What it returns
`strip(chars)` / `lstrip` / `rstrip`	Remove whitespace (or any of `chars`) from both ends / left / right
`split(sep=None, maxsplit=-1)`	List of pieces; with no `sep`, splits on runs of whitespace and drops empties
`rsplit(sep, maxsplit)`	Like `split` but counts `maxsplit` from the right
`splitlines()`	List of lines (handles `\n`, `\r\n`, and others) without the line breaks
`partition(sep)` / `rpartition(sep)`	3-tuple `(before, sep, after)` around the first / last `sep`
`sep.join(iterable)`	One string: the items glued with `sep` (items must be strings)
`removeprefix(p)` / `removesuffix(s)`	The string without that exact prefix/suffix, if present (3.9+)

Boolean checks (all return `False` for the empty string, except `isprintable()` and `isascii()`)

Method	`True` when…
`isalnum()`	Every character is a letter or a number (in the Unicode sense)
`isalpha()`	Every character is a letter
`isdecimal()`	Every character is a decimal digit 0–9 (in any script, e.g. Arabic-Indic)
`isdigit()`	Decimal digits plus digits like superscript `²`
`isnumeric()`	Any numeric character, including fractions like `½`
`islower()` / `isupper()`	All cased characters are lower / upper, and there is at least one
`istitle()`	Title case: upper only after uncased chars, lower only after cased ones
`isspace()`	Only whitespace, at least one character
`isidentifier()`	Valid Python identifier (keywords included – check `keyword.iskeyword` too)
`isprintable()`	No non-printable characters (`\t`, `\n` count as non-printable); `True` for `""`
`isascii()`	All characters are ASCII (3.7+); `True` for `""`
`startswith(p)` / `endswith(s)`	Starts / ends with `p` / `s`; accepts a `tuple` of options

Find, count, replace, translate

Method	What it returns
`count(sub, start, end)`	Number of non-overlapping occurrences
`find(sub, start, end)` / `rfind`	Lowest / highest index, or `-1` if absent
`index(sub, start, end)` / `rindex`	Same, but raises `ValueError` if absent
`replace(old, new, count=-1)`	Copy with occurrences replaced (all, or the first `count`)
`str.maketrans(...)` + `translate(table)`	Map/delete many single characters in one pass

The optional `start, end` arguments (`beg`, `end` in older docs) behave like a slice: search only inside `s[start:end]`, but report indexes relative to the full string.

Code — each group in action

```
s = "pYthon is FUN"
print(s.upper(), "|", s.lower(), "|", s.capitalize(), "|", s.title(), "|", s.swapcase())
print("they're bill's".title())           # title() treats ' as a word break
print("Straße".lower(), "Straße".casefold())  # casefold is the aggressive, caseless form
print("[" + "Menu".center(16, "-") + "]")
print("[" + "Menu".ljust(10) + "]", "[" + "Menu".rjust(10, ".") + "]", "42".zfill(5))
```

Output:

```
PYTHON IS FUN | python is fun | Python is fun | Python Is Fun | PyTHON IS fun
They're Bill's
straße strasse
[------Menu------]
[Menu      ] [......Menu] 00042
```

`center(16, "-")` pads the 4-character word with 12 dashes, 6 on each side. When the padding is odd, the side that gets the extra character depends on the lengths involved (`"ab".center(5)` is `' ab '` but `"abc".center(6)` is `' abc '`), whereas an f-string `{:^w}` always puts the extra character on the right. Pick one and use it consistently if columns must line up.

```
text = "  Hello, Python! Welcome to programming.  "
print(text.strip().split())               # any whitespace, runs collapsed
print(text.strip().rsplit(" ", 1))        # split once, from the right
print("a,,b".split(","), "a  b".split(" "), "a  b".split())
print("k=v=w".partition("="), "k=v=w".rpartition("="), "abc".partition("="))
print("line1\nline2\r\nline3".splitlines())
print("xxhixx".strip("x"), "--hi".lstrip("-"), "hi--".rstrip("-"))
print("report_2024.csv".removesuffix(".csv"), "Mr. Smith".removeprefix("Mr. "))
print("abcabc".strip("ab"))               # strip removes a SET of chars, not a prefix
print("-".join(["2024", "03", "09"]), ", ".join(str(n) for n in [1, 2, 3]))
```

Output:

```
['Hello,', 'Python!', 'Welcome', 'to', 'programming.']
['Hello, Python! Welcome to', 'programming.']
['a', '', 'b'] ['a', '', 'b'] ['a', 'b']
('k', '=', 'v=w') ('k=v', '=', 'w') ('abc', '', '')
['line1', 'line2', 'line3']
hi hi hi
report_2024 Smith
cabc
2024-03-09 1, 2, 3
```

Two behaviours worth memorising from this output:

- `split()` with **no argument** collapses runs of whitespace and never yields empty strings; `split(" ")` with an explicit separator yields an empty string between two adjacent separators.
- `strip("ab")` removes **any** of the characters `a` / `b` from the ends — it is not "remove the prefix `ab`". That is what `removeprefix` is for.

```
samples = ["Hello123", "Hello", "123", "½", "²", "   ", "Hello World", "my_var", "3var"]
# prints one row per sample: isalnum isalpha isdecimal isdigit isnumeric isspace istitle isidentifier
```

Output (T = True, F = False):

```
value          alnum alpha decim digit numer space title ident
'Hello123'         T     F     F     F     F     F     T     T
'Hello'            T     T     F     F     F     F     T     T
'123'              T     F     T     T     T     F     F     F
'½'                T     F     F     F     T     F     F     F
'²'                T     F     F     T     T     F     F     F
'   '              F     F     F     F     F     T     F     F
'Hello World'      F     F     F     F     F     F     T     F
'my_var'           F     F     F     F     F     F     F     T
'3var'             T     F     F     F     F     F     F     F
True True
```

The three digit checks form nested sets: **decimal ⊂ digit ⊂ numeric**. For validating "can I pass this to `int()`?", `isdecimal()` is the right one (`int("²")` fails even though `"²".isdigit()` is `True`). For anything with signs, spaces or decimals, just `try: int(s)` / `float(s)`.

```
t = "hello world, hello universe, hello multiverse"
print(t.count("hello"), t.find("hello"), t.rfind("hello"), t.find("bye"))
print(t.find("hello", 1))                 # search starting at index 1
try:
    t.index("bye")
except ValueError as e:
    print("ValueError:", e)
print(t.replace("hello", "hi", 2))        # only first 2 occurrences

table = str.maketrans({"a": "4", "e": "3", "o": "0", "!": None})  # None = delete
print("hello, leet coders!".translate(table))
```

Output:

```
3 0 29 -1
13
ValueError: substring not found
hi world, hi universe, hello multiverse
h3ll0, l33t c0d3rs
```

Cost of common operations (n = length of the string)

Operation	Time	Note
`len(s)`	O(1)	Length is stored
`s[i]`	O(1)	CPython stores fixed-width characters (see 4.9)
`s[a:b]`	O(b − a)	Copies the slice
`s + t`	O(len s + len t)	Always a new string
`sub in s`, `find`, `count`, `replace`	O(n) typical	Optimised C search
`split`, `strip`, `upper`, `translate`	O(n)	One pass
`sep.join(parts)`	O(total length)	Pre-computes size, one copy
`hash(s)`	O(n) first time, then O(1)	Hash is cached on the object
`s == t`	O(n) worst case	Stops at first difference; O(1) if lengths differ

Common mistakes

- Using `find()` in a boolean test: `if s.find("x"):` is **wrong** – `find` returns `0` (falsy) when the match is at the start and `-1` (truthy) when absent. Use `if "x" in s:`.
- Using `lower()` for case-insensitive comparison of international text – prefer `casefold()`.
- Expecting `title()` to be smart about apostrophes (`"They'Re"`). For display names, write your own rule or use `string.capwords()`, which splits on whitespace only.
- Calling `str.join` with non-strings: `",".join([1, 2])` → `TypeError`. Convert with a generator: `",".join(map(str, nums))`.

Real-world examples

- CSV and log parsing: `line.rstrip("\n").split(",")` is the inner loop of countless scripts (use the `csv` module when fields can contain quoted commas).
- URL/path handling: `url.removeprefix("https://")`, `filename.endswith((".jpg", ".png"))`.
- Search boxes normalise input with `query.strip().casefold()` before matching.

In the interview

> **30-second answer:** "String methods never mutate; they return new strings. Key groups: case (`lower`, `casefold`), cleaning (`strip`, `removeprefix`), splitting/joining (`split`, `partition`, `join`), checking (`isdecimal`, `startswith` with tuples) and searching (`in`, `find` returns −1, `index` raises). Most are O(n) and implemented in C."

- *Q: `find` vs `index`?* A: Same search; `find` returns `-1` when absent, `index` raises `ValueError`.
- *Q: `split()` vs `split(" ")`?* A: No-arg split treats runs of any whitespace as one separator and drops empty fields.
- *Q: How to remove all punctuation fast?* A: `s.translate(str.maketrans("", "", string.punctuation))`.

Self-check

1. `"Hello123".isalpha()`? → `False` (digits).
2. `"a-b-c".rsplit("-", 1)`? → `['a-b', 'c']`.
3. `"Menu".center(20, "-")`? → `'--------Menu--------'`.
4. What does `"hello world, hello universe, hello multiverse".replace("hello", "hi", 2)` give? → `'hi world, hi universe, hello multiverse'`.

TL;DR

Methods return new strings. Memorise: `strip/split/join/partition`, `startswith(tuple)`, `in` over `find`, `casefold` for comparisons, `isdecimal` for digit checks, `translate` for bulk character mapping.

4.8 Joining strings efficiently

What it is

When you build a big string from many pieces (a report, an HTML page, a CSV file), *how* you glue them matters. The idiomatic way is to collect the pieces in a list and call `"".join(pieces)` once.

The problem it solves

Because strings are immutable, `s = s + piece` must allocate a new string and copy **all** existing characters every time. After n appends of a fixed-size piece that is roughly $1 + 2 + \dots + n = n(n+1)/2$ character copies — **quadratic** time, $O(n^2)$. `join` instead measures the total length, allocates once, and copies each piece once — **linear**, $O(n)$.

How it works

1. Append each piece to a list (list append is amortised $O(1)$ — Chapter 5).
2. Call `sep.join(list)`: pass 1 adds up the lengths; pass 2 copies each piece into a single pre-sized buffer.
3. Alternative for stream-like code: `io.StringIO` — `write()` pieces, then `getvalue()`.

CPython has a special trick: if the string on the left of `+=` has **no other references**, it may resize it in place, making the naive loop look fast. This is an implementation detail — PyPy and other interpreters don't do it, and it stops working the moment another name refers to the same string. The benchmark below shows both cases.

```
def plus_equal(n):          # naive, but CPython's in-place trick rescues it
    s = ""
    for _ in range(n):
        s += PIECE
    return s

def plus_equal_shared(n):   # same, but a second reference defeats the trick
    s = ""
    alias = []
    for _ in range(n):
        alias.append(s)
        s += PIECE
        alias.pop()
    return s

def join_list(n):
    parts = []
    for _ in range(n):
        parts.append(PIECE)
    return "".join(parts)
```

Output (`code/05_join_performance.py`, times in ms on the author's laptop — yours will differ, the shape won't):

```
     n        plus_equal  plus_equal_shared         join_list         string_io
  2000           0.07 ms           0.23 ms           0.04 ms           0.04 ms
  5000           0.16 ms           0.89 ms           0.09 ms           0.10 ms
 10000           0.34 ms           3.06 ms           0.17 ms           0.20 ms
 20000           0.69 ms          18.84 ms           0.35 ms           0.41 ms
 40000           1.36 ms          81.27 ms           0.69 ms           0.82 ms
```

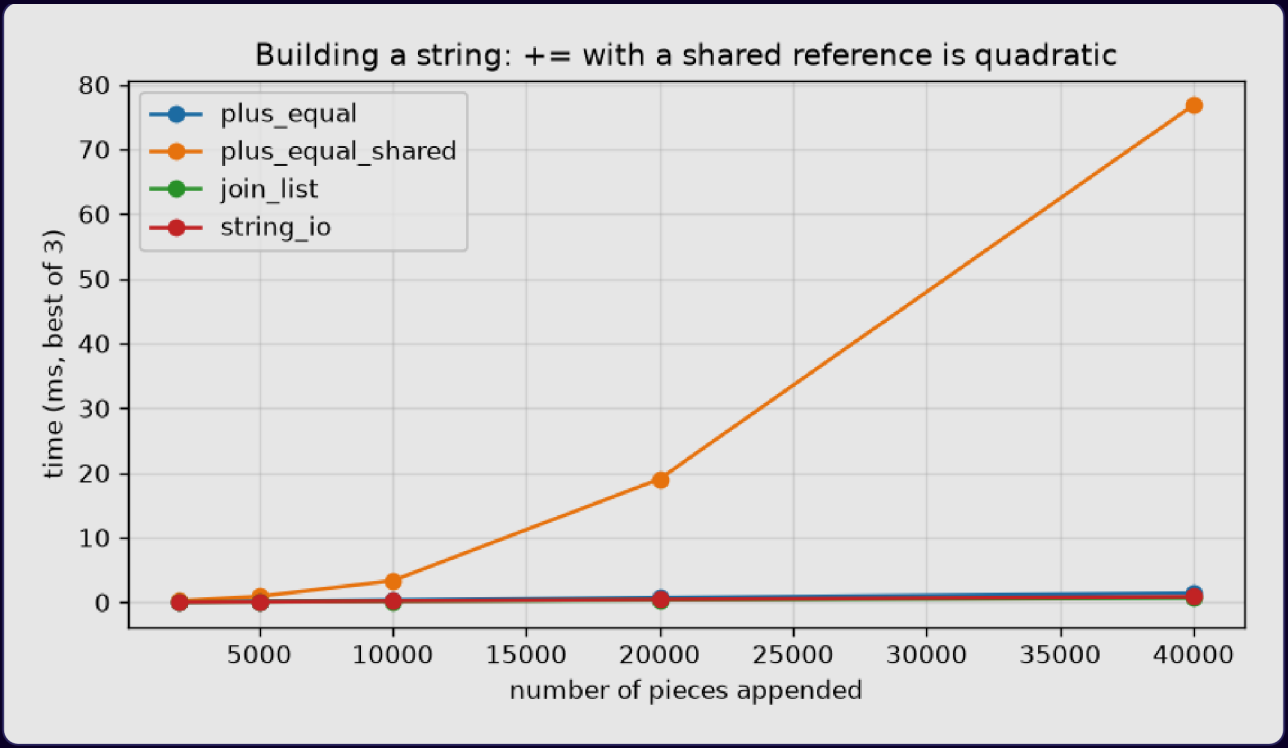


Figure — doubling n doubles the time for `join` and `StringIO` (linear) but roughly quadruples it for `+=` when the optimisation can't apply (quadratic).

Trade-offs

Approach	Complexity	Readability	Notes
`a + b + c` (few pieces)	Fine	Best	Perfectly OK for a handful of strings
f-string	Fine	Best	Best for a fixed template
`+=` in a loop	O(n²) worst case	Good	Only fast by CPython accident
`"".join(list_or_generator)`	O(n)	Good	The standard idiom
`io.StringIO`	O(n)	Good	Natural when code "writes" output in many places

When not to use it

Don't contort code for two or three concatenations; `first + " " + last` or an f-string is clearer. The join rule is for loops and large inputs.

Common mistakes

- Joining numbers directly (`TypeError`) — use `map(str, nums)`.
- Writing `"".join([x for x in items])` when `"".join(items)` already works (items are strings).

In the interview

> **30-second answer:** "Repeated `+=` on an immutable string copies everything each time, so it's O(n²). Collect parts in a list and `''.join` once — O(n), because join sizes the result first and copies each part once."

- *Q: But my `+=` benchmark is fast?* A: CPython resizes in place when the string has a single reference; it's not guaranteed and not portable.
- *Q: Generator or list inside `join`?* A: `join` converts its argument to a sequence anyway, so a list comprehension is usually slightly faster; both are fine.

Self-check

1. Why is `+=` in a loop quadratic in principle? → Each step copies the whole accumulated string.
2. What does `join` do first? → Computes the total length so it can allocate once.
3. Name a linear alternative to `join`. → `io.StringIO`.

TL;DR

Many pieces → list + `"".join(...)`. A few pieces → `+` or f-string.

4.9 Unicode vs bytes, encodings and UTF-8

What it is

- A **character** is an abstract symbol: "A", "é", "€", "😀".
- **Unicode** assigns every character a number called a **code point**, written `U+20AC` (hex) for "€". Python's `str` is a sequence of code points. `ord(ch)` gives the number; `chr(n)` gives the character back.
- **Bytes** are raw 8-bit values (0–255) — what files, sockets and disks actually hold. Python's `bytes` type is an immutable sequence of such values, written `b"..."`.
- An **encoding** is the rulebook that turns code points into bytes (`str.encode`) and back (`bytes.decode`).
- **UTF-8** is the dominant encoding on the web and in Linux/macOS: variable-length, 1 to 4 bytes per code point, and identical to ASCII for the first 128 characters.

Analogy: code points are the words of a message; an encoding is the alphabet you use to write them on paper. If the reader assumes a different alphabet, they read gibberish ("mojibake").

Why it matters

Every time text crosses a boundary — reading a file, calling an HTTP API, writing to a database — it is converted between `str` and `bytes`. Getting the encoding wrong causes crashes (`UnicodeDecodeError`) or, worse, silently corrupted text.

How it works

1. Inside your program, work with `str` (text).
2. At the edges, **decode** incoming bytes to `str` as early as possible and **encode** outgoing `str` to bytes as late as possible (the "Unicode sandwich").
3. Always name the encoding: `open(path, encoding="utf-8")`. Without it, `open()` uses the platform's locale encoding, which differs between machines (Python 3.15, due October 2026, makes UTF-8 the default under PEP 686 — still name it explicitly while you support 3.12–3.14).
4. UTF-8 byte lengths by code point range:

Code points	Bytes	Example
U+0000 – U+007F (ASCII)	1	`A` → `41`
U+0080 – U+07FF	2	`é` → `C3 A9`
U+0800 – U+FFFF	3	`€` → `E2 82 AC`
U+10000 – U+10FFFF	4	`😀` → `F0 9F 98 80`

```python
for ch in ["A", "é", "€", "😀"]:
    utf8 = ch.encode("utf-8")
    print(f"{ch!r}: ord={ord(ch):>6}  U+{ord(ch):04X}  utf-8={utf8!r:<22} {len(utf8)} byte(s)")

s = "café"
b = s.encode("utf-8")
print(type(b), b, "len(str)=", len(s), "len(bytes)=", len(b))
print(b.decode("utf-8"))
print(repr(b.decode("latin-1")))                    # wrong codec -> mojibake, no error!
try:
    b.decode("ascii")
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e)
print(s.encode("ascii", errors="replace"), s.encode("ascii", errors="ignore"))
print(b"\xff\xfeok".decode("utf-8", errors="replace"))
print(b"abc"[0], list(b"hi"), bytes([104, 105]))        # indexing bytes gives ints
```

Output:

```
'A': ord=    65  U+0041  utf-8=b'A'                  1 byte(s)
'é': ord=   233  U+00E9  utf-8=b'\xc3\xa9'           2 byte(s)
'€': ord=  8364  U+20AC  utf-8=b'\xe2\x82\xac'       3 byte(s)
'😀': ord=128512  U+1F600  utf-8=b'\xf0\x9f\x98\x80'    4 byte(s)
<class 'bytes'> b'caf\xc3\xa9' len(str)= 4 len(bytes)= 5
café
'cafÃ©'
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ordinal not in range(128)
b'caf?' b'caf'
ï¿½ok
97 [104, 105] b'hi'
```

What to notice:

- `len("café")` is 4 characters but its UTF-8 form is 5 bytes. Database column limits and HTTP `Content-Length` count **bytes**.
- Decoding UTF-8 bytes as Latin-1 **doesn't fail** — it produces `cafÃ©`. That's the classic mojibake you see on badly configured websites.
- `errors=` controls failure: `"strict"` (default, raise), `"replace"` (`?` or `�`), `"ignore"` (drop — loses data silently), `"backslashreplace"`, and more.
- Indexing `bytes` gives an **`int`**, not a 1-byte string.

Normalization. Some characters can be written two ways: `é` as one code point (U+00E9) or as `e` + a combining accent (U+0301). They look identical but compare unequal. `unicodedata.normalize("NFC", s)` converts to the composed form so comparisons work:

```python
a = "caf\u00e9"; b = "cafe\u0301"
print(a, b, a == b, len(a), len(b))          # café café False 4 5
nfc = unicodedata.normalize("NFC", b)
print("after NFC:", a == nfc, len(nfc))      # after NFC: True 4
```

How CPython stores `str`. Since Python 3.3 (PEP 393) each string picks the narrowest fixed width that fits its widest character: 1 byte per character for Latin-1 text, 2 bytes if it has characters up to U+FFFF, 4 bytes otherwise. Fixed width is why `s[i]` is O(1) even though UTF-8 is variable-length — CPython does not store strings as UTF-8 internally.

Trade-offs

	`str`	`bytes`	`bytearray`
Holds	Unicode code points	Raw bytes 0–255	Raw bytes 0–255
Mutable	No	No	Yes
Literal	`"héllo"`	`b"hello"` (ASCII only in literal)	`bytearray(b"...")`
Use for	All text logic	Files, network, hashing, crypto	Building/patching binary buffers

Encoding	Size per char	ASCII-compatible	Notes
UTF-8	1–4 bytes	Yes	Default for web, JSON (required by RFC 8259 for data exchanged between systems), Python source files
UTF-16	2 or 4 bytes	No	Used internally by Windows APIs, Java and JavaScript strings
Latin-1 (ISO-8859-1)	1 byte	Yes	Only 256 characters; never fails to decode, so hides errors
ASCII	1 byte	–	Only 128 characters

Common mistakes

- Mixing types: `"abc" + b"def"` → `TypeError`. Python 3 never guesses an encoding for you.
- Omitting `encoding=` in `open()` – code works on your Mac and breaks on a Windows server.
- Counting user-visible characters with `len()`. A flag emoji or "👍🏽" is several code points. For "grapheme" counting you need a third-party library such as `grapheme` or `regex` (`\X`).
- Using `errors="ignore"` to make an error go away – you lose data silently.

Real-world examples

- `requests` exposes both `response.content` (bytes) and `response.text` (decoded `str`, using the declared or guessed encoding).
- `json.dumps(..., ensure_ascii=False)` keeps non-ASCII characters readable instead of escaping them as `\u00e9`.
- The Python source code itself is UTF-8 by default (PEP 3120), which is why `name = "Zoë"` works without a coding comment.

In the interview

30-second answer: "`str` is a sequence of Unicode code points; `bytes` is raw data. Encode text to bytes at the output boundary and decode bytes to text at the input boundary, always naming the encoding – normally UTF-8, which is variable-length (1–4 bytes) and ASCII-compatible."

- *Q: Why does `len('café'.encode())` return 5?* A: `é` needs two bytes in UTF-8.
- *Q: What is mojibake?* A: Text decoded with the wrong encoding, e.g. UTF-8 bytes read as Latin-1 → `cafÃ©`.
- *Q: Why can two equal-looking strings compare unequal?* A: Different code point sequences (composed vs decomposed); normalise with NFC.

Self-check

1. How many bytes is `€` in UTF-8? → 3.
2. `b"hi"[0]`? → `104` (an int).
3. What should you pass to `open()` for text files? → `encoding="utf-8"` (or the file's real encoding).
4. Does `bytes.decode("latin-1")` ever raise? → No – every byte maps to a character, which is why it can hide mistakes.

TL;DR

Text is `str` (code points); storage and network are `bytes`. Decode early, encode late, always say `utf-8`, and normalise before comparing user text.

4.10 Regular expressions — a practical introduction

What it is

A **regular expression** (regex) is a small pattern language for describing text: "one or more digits", "a word followed by `=`", "a date like 2024-03-09". Python's `re` module compiles a pattern and searches strings with it.

Why it matters

String methods handle fixed text; regex handles **shapes** of text. Extracting all emails, validating a postcode, pulling fields from log lines, or collapsing whitespace are one-liners with `re`.

How it works — the core syntax

Pattern	Matches
`.`	Any character except newline
`\d` `\w` `\s`	Digit, word character (letter, digit, `_`), whitespace
`\D` `\W` `\S`	The opposites
`[abc]` `[a-z]` `[^0-9]`	One char from a set / range / not in the set
`^` `$`	Start / end of string (or of each line with `re.MULTILINE`); `$` also matches just before a final `\n`
`\b`	Word boundary
`*` `+` `?`	0 or more, 1 or more, 0 or 1 (greedy)
`{3}` `{2,5}`	Exactly 3, between 2 and 5
`*?` `+?`	Lazy versions — match as little as possible
`a|b`	`a` or `b`
`( … )`	Capturing group
`(?P<name> … )`	Named group
`(?: … )`	Group without capturing

The main functions

Function	Does
`re.search(p, s)`	First match **anywhere** → `Match` or `None`
`re.match(p, s)`	Match only at the **start** → `Match` or `None`
`re.fullmatch(p, s)`	The **whole string** must match
`re.findall(p, s)`	List of all matches (strings, or tuples if the pattern has several groups)
`re.finditer(p, s)`	Iterator of `Match` objects (positions + groups)
`re.sub(p, repl, s)`	Replace matches; `repl` can use `\1` or be a function
`re.split(p, s)`	Split on a pattern
`re.compile(p)`	Pre-compile into a `Pattern` object with the same methods

A `Match` **object** has `.group()` (whole match), `.group(1)` / `.group("name")`, `.groups()`, `.groupdict()`, `.start()`, `.end()`, `.span()`.

Code

```
LOG = """2024-03-09 12:01:07 ERROR db timeout user=alice
2024-03-09 12:01:09 INFO  login ok user=bob
2024-03-10 08:15:00 ERROR disk full user=carol"""

s = "Order #123 shipped"
print(re.match(r"\d+", s))                  # None: match only tries at position 0
m = re.search(r"\d+", s)                    # scans the whole string
print(m, m.group(), m.start(), m.end(), m.span())
print(re.fullmatch(r"\d{3}", "123") is not None, re.fullmatch(r"\d{3}", "1234"))

print(re.findall(r"user=(\w+)", LOG))        # one group -> list of strings
print(re.findall(r"(\d{4})-(\d\d)-(\d\d)", "2024-03-09 and 2025-01-02"))  # tuples
for m in re.finditer(r"ERROR (.+?) user=(\w+)", LOG):
    print("  ", m.group(1), "->", m.group(2), "at", m.start())
```

Output:

```
None
<re.Match object; span=(7, 10), match='123'> 123 7 10 (7, 10)
True None
['alice', 'bob', 'carol']
[('2024', '03', '09'), ('2025', '01', '02')]
   db timeout -> alice at 20
   disk full -> carol at 112
```

```
pat = re.compile(r"(?P<date>\d{4}-\d\d-\d\d) (?P<time>[\d:]+) (?P<level>\w+)")
m = pat.search(LOG)
print(m.group("level"), m.group(1), m.groups())
print(m.groupdict())
print(re.search(r"(?:ab)+", "xxababab").group())   # (?:...) groups without capturing

print(re.sub(r"\s+", " ", "too    many   spaces"))
print(re.sub(r"(\d{4})-(\d\d)-(\d\d)", r"\3/\2/\1", "due 2024-03-09"))   # backrefs
print(re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "3 apples, 10 pears"))
print(re.split(r"[,;]\s*", "a, b;c ,d"))

html = "<b>bold</b><i>it</i>"
print(re.findall(r"<.+>", html))     # greedy: grabs as much as possible
print(re.findall(r"<.+?>", html))    # lazy: as little as possible
print(re.escape("price: $5.00 (approx)"))
```

Output:

```
ERROR 2024-03-09 ('2024-03-09', '12:01:07', 'ERROR')
{'date': '2024-03-09', 'time': '12:01:07', 'level': 'ERROR'}
ababab
too many spaces
due 09/03/2024
6 apples, 20 pears
['a', 'b', 'c ', 'd']
['<b>bold</b><i>it</i>']
['<b>', '</b>', '<i>', '</i>']
price:\ \$5\.00\ \(approx\)
```

Step by step for the date rewrite: the pattern captures year, month and day into groups 1–3; the replacement `r"\3/\2/\1"` writes them back in reverse order. The replacement string is raw so `\3` is not treated as an octal escape.

`re.escape` backslash-escapes every special character — use it whenever you insert user text into a pattern.

Trade-offs

Task	Best tool
Fixed substring test	`in`, `startswith`, `endswith` — faster and clearer
Split on one fixed separator	`str.split`
Variable shapes, extraction, validation	`re`
HTML / XML	A real parser (`html.parser`, `lxml`, BeautifulSoup) — regex can't handle nesting
Emails / URLs strict validation	A dedicated library or `urllib.parse`; "perfect" regexes are unreadable

When not to use it

When a string method does the job, or when the input is a nested format (HTML, JSON, code). Also avoid running user-supplied patterns — see the next point.

Common mistakes

- **Forgetting `r"..."`**: `"\b"` is a backspace, not a word boundary.
- **`match` vs `search`**: `match` only checks the beginning; for validation use `fullmatch`.
- **Greedy `.*`** swallowing too much — use `.*?` or a negated class like `[^<]*`.
- **Catastrophic backtracking:** Python's engine backtracks, so patterns with nested quantifiers such as `(a+)+$` can take exponential time on inputs that almost match, and even simpler ones (`.*.*=.*`, or `\s+$` on a long run of spaces) can go quadratic. This is the basis of ReDoS (regular-expression denial-of-service) attacks.
- Calling `m.group()` without checking `m is not None` → `AttributeError`.

Real-world examples

- On 2 July 2019 a single new WAF (web application firewall) rule containing `.*(?:.*=.*)` backtracked so heavily that it pinned CPUs across Cloudflare's network, causing a global outage of about 27 minutes (per Cloudflare's public post-mortem).
- On 20 July 2016 Stack Overflow was down for 34 minutes: a whitespace-trimming regex (`[\s\u200c]+$`) hit a post with about 20,000 consecutive whitespace characters and backtracked quadratically — $O(n^2)$, roughly 200 million steps for one string (per their post-mortem).
- Django can route URLs with regex patterns via `re_path()` (before Django 2.0, regex routes through `url()` were the only option), and log shippers such as Logstash's grok filter are built on named regex patterns.

In the interview

> **30-second answer:** "`re.search` finds the first match anywhere, `match` only at the start, `fullmatch` the whole string. `findall` returns strings or tuples of groups, `finditer` gives Match objects, `sub` replaces with backrefs or a function. Use raw strings, lazy quantifiers when needed, named groups for readability, and beware catastrophic backtracking."

- *Q: Why `re.compile`?* A: Readability and reuse; `re` also caches recently used patterns, so the speed gain is usually small.
- *Q: What does `findall` return with two groups?* A: A list of 2-tuples, one per match.
- *Q: How do you make a group non-capturing?* A: `(?:...)`.

Self-check

1. `re.match(r"\d", "a1")`? → `None`.
2. `re.findall(r"\d+", "a1b22c333")`? → `['1', '22', '333']`.
3. `re.sub(r"(\w+)@(\w+)", r"\2 at \1", "me@home")`? → `'home at me'`.
4. Greedy vs lazy on `"<a><b>"` with `<.+>` vs `<.+?>`? → `['<a><b>']` vs `['<a>', '<b>']`.

TL;DR

`search` / `match` / `fullmatch` for one match, `findall` / `finditer` for many, `sub` to replace, `()` to capture, `(?P<name>)` to name, `r""` always, and keep patterns simple.

Chapter TL;DR

Topic	Remember
Basics	`str` is an immutable sequence of Unicode code points; zero-based and negative indexing.
Immutability	Methods and operators return new strings; rebind the name. Strings are hashable.
Operators	`+`, `*`, `in`; comparisons by code point; use `==`, not `is`.
Slicing	`[start:stop:step]`, stop excluded, never raises, `[::-1]` reverses.
Escapes	`\n \t \\ \' \" \xhh \uXXXX \N{...}`; `r"..."` for regex and paths.
Formatting	f-strings by default; `[[fill]align][sign][#][0][width][,][.prec][type]`; `format()` for stored templates, `%` for logging, `Template` for untrusted input.
Methods	`strip/split/join/partition/replace/startswith`; `in` not `find` for tests; `casefold` for comparisons; `isdecimal` for digits.
Building	Many pieces → list + `"".join`.
Unicode	Decode at input, encode at output, always `encoding="utf-8"`; normalise with NFC.
Regex	`search`, `match`, `fullmatch`, `findall`, `sub`, groups; raw strings; beware backtracking.

Chapter 5 — Lists, Tuples and Arrays

Python gives you several ways to keep items **in order**: the all-purpose `list`, the immutable `tuple`, the compact numeric `array.array`, and — from the scientific world — NumPy's `ndarray`. They look alike (all support indexing, slicing, `len`, `in`, and `for`) but behave very differently in mutability, memory, speed and safety. This chapter explains each one, how it works under the hood, and how to choose.

What you'll learn

1. The `array` module: type codes, indexing, insertion, deletion, search, update, and its methods.
2. Lists: creating, accessing, updating, joining, slicing, looping and operators.
3. How a list is really a dynamic array — and the big-O cost of every operation.
4. Adding and removing items, and the full list-method table.
5. Advanced slicing: steps, slice assignment, slice deletion.
6. List comprehensions.
7. Sorting: `sort()` vs `sorted()`, `key`, `reverse`, stability, multi-key sorts.
8. Copying: aliasing, shallow vs deep copies, and the `[[0]*3]*3` trap.
9. Tuples: immutability, operators, unpacking (including `*`), joining, methods, tuples as records, `namedtuple`.
10. When to choose `list`, `tuple`, `array.array` or NumPy.

All runnable code is in `code/`. Outputs below are real, produced with Python 3.14 (the code runs on 3.12+). Timings are from one laptop — yours will differ, but the ratios hold.

5.1 The `array` module — typed, compact arrays

What it is

An **array** (from the standard-library `array` module) is a mutable sequence in which **every element has the same C numeric type** — all 32-bit ints, or all 64-bit floats, and so on. The values are stored as raw machine numbers side by side in one block of memory (**contiguous** storage), rather than as separate Python objects.

Analogy: a list is a shelf of labelled boxes, each box holding any object; an `array` is an egg carton — every slot is the same size and holds one kind of thing, so it packs far more densely.

Why it matters

For large volumes of plain numbers — sensor readings, audio samples, pixel values, binary file formats — an `array` uses a fraction of a list's memory and can be written to or read from files and sockets directly as bytes.

How it works

1. `import array` and create one with a **type code** that fixes the element type: `array.array("i", [10, 20, 30])`.
2. Every value you store is checked and converted to that C type; wrong types raise `TypeError`, out-of-range numbers raise `OverflowError`.
3. Indexing (positive and negative), slicing, `len`, `in` and `for` work exactly as with lists.

Type codes (sizes are the minimum the C standard guarantees; `itemsize` tells you the real size on your machine)

Code	C type	Python type	Min bytes	Typical on 64-bit macOS/Linux
`'b'` / `'B'`	signed / unsigned char	int	1	1 (range −128…127 / 0…255)
`'h'` / `'H'`	signed / unsigned short	int	2	2
`'i'` / `'I'`	signed / unsigned int	int	2	4
`'l'` / `'L'`	signed / unsigned long	int	4	8 (4 on Windows)
`'q'` / `'Q'`	signed / unsigned long long	int	8	8
`'f'`	float	float	4	4
`'d'`	double	float	8	8
`'u'`	wchar_t (Unicode char)	str	2	4 – deprecated since 3.3, slated for removal
`'w'`	Py_UCS4 (Unicode char)	str	4	4 – added in 3.13 as the replacement for `'u'`

```
a = array.array("i", [10, 20, 30, 40, 50])      # 'i' = C signed int
print(a, a.typecode, a.itemsize, "bytes per item")
print(a[0], a[1], a[-1], a[-2])
for x in a:
    print(x, end=" ")
try:
    a.append(3.5)                               # wrong type is rejected
except TypeError as e:
    print("TypeError:", e)
try:
    array.array("b", [200])                     # 'b' holds -128..127
except OverflowError as e:
    print("OverflowError:", e)
```

Output:

```
array('i', [10, 20, 30, 40, 50]) i 4 bytes per item
10 20 50 40
10 20 30 40 50
TypeError: 'float' object cannot be interpreted as an integer
OverflowError: signed char is greater than maximum
```

The four basic operations: insert, delete, search, update

Operation	Code	Effect	Cost
Traverse	`for x in a`	Visit each element	O(n)
Insert	`a.insert(i, x)`	Put `x` at index `i`, shift the rest right	O(n)
Delete	`a.pop(i)` / `a.remove(x)`	Remove by position / first matching value, shift left	O(n) (O(1) for `pop()` at the end)
Search	`a.index(x)`	Position of the first `x` (or `ValueError`)	O(n)
Update	`a[i] = x`	Overwrite in place	O(1)

Array methods

Method	What it does	Cost
`append(x)`	Add `x` at the end	O(1) amortised
`extend(iterable)`	Append every item of an iterable (same type required)	O(k)
`insert(i, x)`	Insert before position `i`	O(n)
`pop([i])`	Remove and return item `i` (default last)	O(1) last, O(n) otherwise
`remove(x)`	Delete the first occurrence of `x`	O(n)
`index(x)`	Position of first `x`	O(n)
`count(x)`	How many times `x` occurs	O(n)
`reverse()`	Reverse in place	O(n)
`clear()`	Remove all items (Python 3.13+; use `del a[:]` on older versions)	O(1)
`tolist()` / `fromlist(l)`	Convert to / append from a list	O(n)
`tobytes()` / `frombytes(b)`	Raw machine bytes out / in	O(n)
`tofile(f)` / `fromfile(f, n)`	Write / read raw values to a binary file	O(n)
`byteswap()`	Swap byte order (endianness) of every item	O(n)
`buffer_info()`	`(address, length)` of the underlying memory	O(1)

Two things arrays **don't** have: a `.sort()` method and a `.copy()` method. Sort with the built-in `sorted(a)` — which returns a **list** — and convert back with `array.array(code, sorted(a))`. Copy with `a[:]`, `array.array(a.typecode, a)` or `copy.copy(a)`.

```
a = array.array("i", [1, 2, 3, 4, 5])
a.insert(1, 10);            print("insert(1, 10):", a.tolist())
removed = a.pop(1);         print("pop(1):", a.tolist(), "removed", removed)
print("index(3):", a.index(3))
a[2] = 20;                  print("a[2] = 20:", a.tolist())
a.append(4);                print("append(4):", a.tolist())
a.extend([5, 4, 6]);        print("extend:", a.tolist())
print("count(4):", a.count(4))
a.remove(4);                print("remove(4) (first 4 only):", a.tolist())
a.reverse();                print("reverse:", a.tolist())
print("sorted(a) -> list:", sorted(a))          # arrays have no .sort()
a = array.array("i", sorted(a));  print("back to array:", a)
b = copy.copy(a)                                # no .copy() method: use copy.copy or a[:]
b[0] = 99
print("copy independent:", a[0], b[0])
del a[:]                                        # empties it (array.clear() exists from 3.13)
print("emptied:", a)
```

Output:

```
insert(1, 10): [1, 10, 2, 3, 4, 5]
pop(1): [1, 2, 3, 4, 5] removed 10
index(3): 2
a[2] = 20: [1, 2, 20, 4, 5]
append(4): [1, 2, 20, 4, 5, 4]
extend: [1, 2, 20, 4, 5, 4, 5, 4, 6]
count(4): 3
remove(4) (first 4 only): [1, 2, 20, 5, 4, 5, 4, 6]
reverse: [6, 4, 5, 4, 5, 20, 2, 1]
sorted(a) -> list: [1, 2, 4, 4, 5, 5, 6, 20]
back to array: array('i', [1, 2, 4, 4, 5, 5, 6, 20])
copy independent: 1 99
emptied: array('i')
```

`tolist()` is used only to print a clean list; printing the array itself shows `array('i', [...])`.

Memory. One million integers:

```
list of 1,000,000 ints :   36.0 MB (pointers + int objects)
array('i')             :    4.1 MB (raw 4-byte ints)
```

A list stores an 8-byte pointer per slot **plus** a separate int object (28 bytes each for small ints on 64-bit CPython) — about 9× more memory here.

Trade-offs, mistakes, and when not to use it

- **Not faster for arithmetic.** Each time you read an element, Python must create an `int` / `float` object for it ("boxing"), so looping over an array is often *slower* than over a list (see 5.15). For fast maths, use NumPy.
- **Choose the type code carefully:** `'b'` overflows at 128; `'l'` is 4 bytes on Windows but 8 on Linux/macOS — use `'q'` when you need 64 bits everywhere.
- Use arrays for **compact storage and binary I/O**; use lists for general work; use NumPy for numerical computing.

Self-check

1. What does the type code decide? → The C type, hence size and allowed values, of every element.
2. How do you sort an `array`? → `array.array(a.typecode, sorted(a))`.
3. Why does an array of a million ints use ~4 MB while a list uses ~36 MB? → The array stores raw 4-byte values; the list stores pointers to full Python int objects.

TL;DR

`array.array(code, data)` = homogeneous, compact, mutable numbers. Same indexing and most list methods; no `sort` / `copy` methods; `clear()` only from 3.13. Great for memory and binary I/O, not for speed.

5.2 Lists — the everyday sequence

What it is

A **list** is an ordered, **mutable** collection that can hold objects of **any type**, written with square brackets: `[1, "Hello", 3.14, True]`. It can grow and shrink at run time.

Analogy: a to-do list on a whiteboard — you can read any line, rewrite it, add lines, erase lines, and reorder them.

Why it matters

It's the default container in Python. Function results, rows from a file, items from an API, stacks, queues of work — you'll reach for a list first in most programs.

How it works — create, access, update

```
my_list = [1, "Hello", 3.14, True]          # mixed types are fine
print(my_list, len(my_list))
print(my_list[0], my_list[1], my_list[-1], my_list[-2])
my_list[1] = "World"
print("single update:", my_list)
my_list[1:3] = ["Changed", "Completely"]
print("slice update:", my_list)
print(list("abc"), list(range(4)), [0] * 3, [])
```

Output:

```
[1, 'Hello', 3.14, True] 4
1 Hello True 3.14
single update: [1, 'World', 3.14, True]
slice update: [1, 'Changed', 'Completely', True]
['a', 'b', 'c'] [0, 1, 2, 3] [0, 0, 0] []
```

- Indexing works like strings: `0` is first, `-1` last, `-2` second-last.
- Unlike strings, you can **assign** to an index (`my_list[1] = ...`) or a whole slice (`my_list[1:3] = [...]`) — see 5.5.
- `list(iterable)` builds a list from anything iterable.

How it works — joining, slicing, looping

```
a, b = [1, 2, 3], [4, 5, 6]
c = a + b                                   # new list
a.extend(b)                                 # modifies a in place
print(c, a, c == a, c is a)
full = list(range(10))
print(full[2:7], full[-8:-3], full[::3], full[::-1])
for i, fruit in enumerate(["apple", "banana", "cherry"], start=1):
    print(i, fruit)
```

Output:

```
[1, 2, 3, 4, 5, 6] [1, 2, 3, 4, 5, 6] True False
[2, 3, 4, 5, 6] [2, 3, 4, 5, 6] [0, 3, 6, 9] [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
1 apple
2 banana
3 cherry
```

`+` builds a **new** list and leaves both inputs alone; `extend()` (or `+=`) **modifies** the left list in place. `c == a` compares contents (equal); `c is a` compares identity (different objects).

List operators

Operator	Meaning	Example	Result
`+`	Concatenate into a new list	`[1, 2] + [3]`	`[1, 2, 3]`
`*`	Repeat	`[1, 2] * 2`	`[1, 2, 1, 2]`
`in` / `not in`	Membership (linear scan)	`3 in [1, 2, 3]`	`True`
`==`, `<`, …	Element-by-element (lexicographic) comparison	`[1, 2, 3] < [1, 3]`	`True`
`+=`	Extend in place	`a += [4]`	`a` changed
`del a[i]`	Delete by index or slice	`del a[0]`	

```
a, b = [1, 2, 3], [4, 5, 6]
print(a + b, a * 3, 3 in a, 7 not in a)
print([1, 2, 3] < [1, 3], [1, 2] == [1, 2], max(a), min(a), sum(a))
```

Output:

```
[1, 2, 3, 4, 5, 6] [1, 2, 3, 1, 2, 3, 1, 2, 3] True True
True True 3 1 6
```

`[1, 2, 3] < [1, 3]` is `True` because the first elements tie (1 = 1) and at the second position 2 < 3 — exactly like comparing words letter by letter.

Common mistakes

- Confusing `a + b` (new list) with `a.extend(b)` (in place, returns `None`). Writing `a = a.extend(b)` sets `a` to `None`.
- Testing membership in a big list inside a loop — each `in` is O(n). Convert to a `set` first (Chapter 6).
- Modifying a list while iterating over it (e.g. removing items) — elements get skipped. Iterate over a copy (`for x in lst[:]`) or build a new list with a comprehension.

Self-check

1. `[1, 2] * 3`? → `[1, 2, 1, 2, 1, 2]`.
2. What does `a.extend(b)` return? → `None` (it changes `a`).
3. `full[-8:-3]` on `list(range(10))`? → `[2, 3, 4, 5, 6]`.

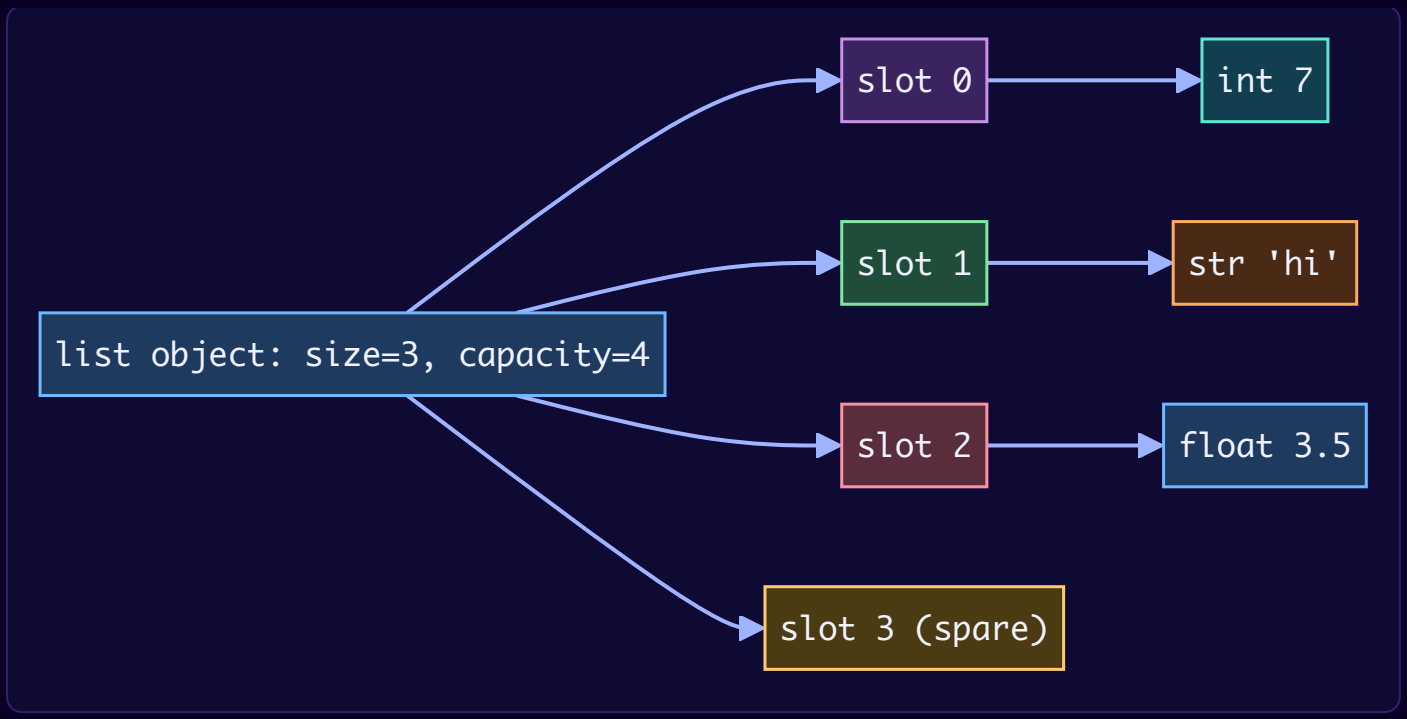
TL;DR

Lists are ordered, mutable, heterogeneous. Index, slice, update in place; `+` makes new lists, `extend` / `+=` modify.

5.3 List internals — a dynamic array — and the big-O table

What it is

In CPython a list is a **dynamic array**: one contiguous block of memory holding **pointers** (references) to the element objects, plus two numbers — how many slots are used (`len`) and how many are allocated (**capacity**). "Dynamic" means it resizes itself automatically.



Why it matters

Knowing the layout tells you instantly which operations are cheap and which are slow — the difference between an O(n) and an O(n²) program.

How it works

1. **Index access** `lst[i]`: address = start + i × 8 bytes → one step, O(1).
2. **Append** when there is spare capacity: write the pointer into the next slot, O(1).
3. **Append when full**: allocate a bigger block, copy all pointers, then write. CPython grows by roughly **12.5% plus a small constant** (the formula in recent CPython versions is `new_size + new_size/8 + 6`, rounded down to a multiple of 4). Because each resize makes room for many future appends, the copying cost spread over all appends is constant — **amortised O(1)**.
4. **Insert or delete at position i**: every pointer after `i` must shift one slot → O(n − i). At the front (`i = 0`) that's the whole list.
5. **Shrinking**: when the list falls below half its capacity, CPython may reallocate a smaller block.

Measured capacity while appending (from `code/06_list_internals.py`, using `sys.getsizeof`):

```
len  capacity  (capacity = slots allocated)
  0         0
  1         4
  5         8
  9        16
 17        24
 25        32
 33        40
 41        52
 53        64
 65        76
```

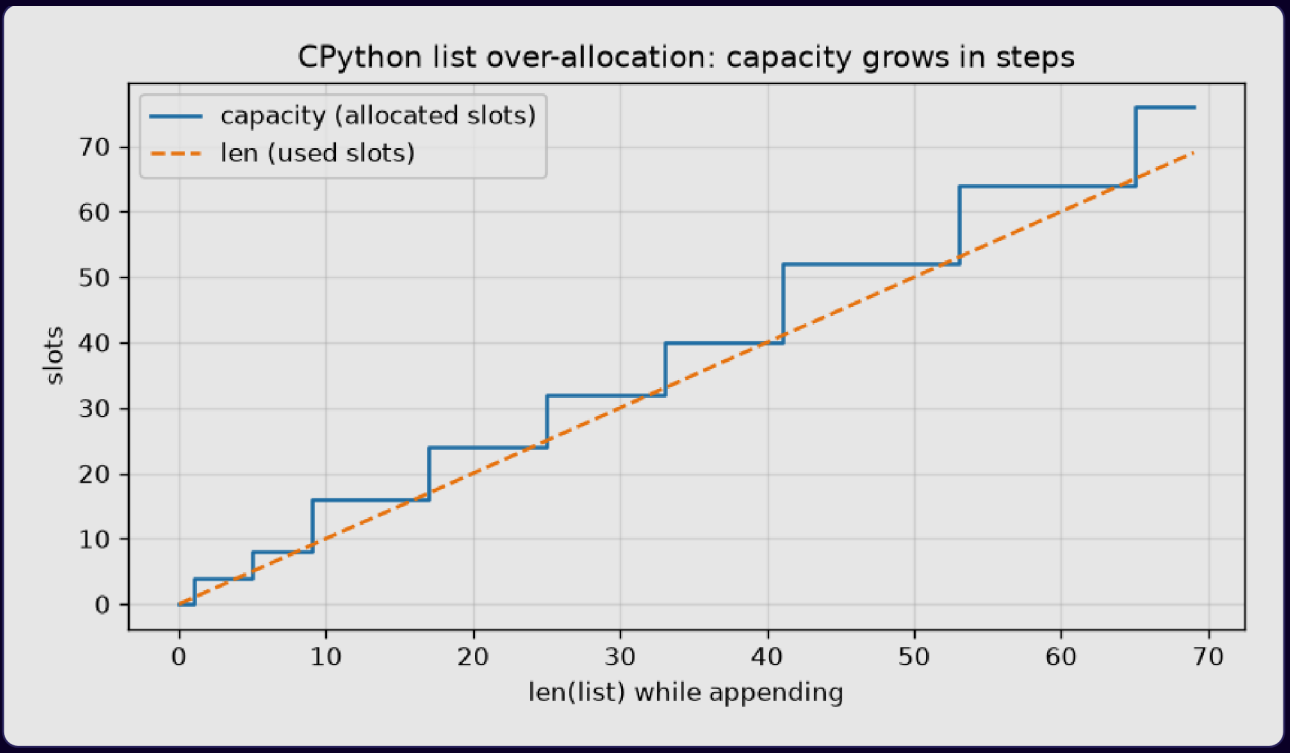


Figure — capacity (steps) always stays at or above the used length (dashed line); a copy happens only at each step.

Worked amortised example: appending the 17th item finds the list full at 16, so CPython allocates 24 slots and copies 16 pointers. The next 7 appends are free. Spread over those 8 appends, that's 2 copies each — and as lists grow, the steps get larger in absolute terms, so the average copy cost per append stays bounded by a constant.

Timing the difference between the end and the front (µs per push-then-pop pair):

n	append()+pop()	insert(0)+pop(0)	(µs per pair)
10,000	0.017	3.588	
100,000	0.017	40.046	
1,000,000	0.018	414.395	

End operations stay flat as *n* grows ×100; front operations grow ×100 — the signature of O(1) vs O(n).

Big-O table for `list` (n = len, k = size of the other argument or slice)

Operation	Example	Average cost
Length	`len(a)`	O(1)
Get / set item	`a[i]`, `a[i] = x`	O(1)
Append	`a.append(x)`	O(1) amortised
Pop last	`a.pop()`	O(1)
Pop / delete at i	`a.pop(i)`, `del a[i]`	O(n − i) → O(n) at the front
Insert at i	`a.insert(i, x)`	O(n − i)
Extend	`a.extend(b)`, `a += b`	O(k)
Concatenate	`a + b`	O(n + k)
Repeat	`a * k`	O(n·k)
Get slice	`a[i:j]`	O(k)
Set slice	`a[i:j] = b`	O(k + n)
Delete slice	`del a[i:j]`	O(n)
Membership / search	`x in a`, `a.index(x)`, `a.count(x)`, `a.remove(x)`	O(n)
Min / max / sum	`min(a)`	O(n)
Copy	`a.copy()`, `a[:]`	O(n)
Reverse	`a.reverse()`	O(n)
Sort	`a.sort()`, `sorted(a)`	O(n log n) worst, O(n) if already sorted
Iterate	`for x in a`	O(n)

Trade-offs

Need	Best structure	Why
Append/pop at the end (stack)	`list`	O(1)
Add/remove at both ends (queue)	`collections.deque` (Chapter 6)	O(1) at both ends; list's `pop(0)` is O(n)
Frequent membership tests	`set` / `dict`	O(1) average instead of O(n)
Keep sorted while inserting	`bisect` on a list (Chapter 6) or a sorted container library	O(log n) search, O(n) insert

Common mistakes

- Using `list.pop(0)` as a queue — quadratic for large queues. Use `deque.popleft()`.
- Calling `insert(0, x)` in a loop to build a reversed list — append then `reverse()` once.

In the interview

> **30-second answer:** "A Python list is a dynamic array of pointers. Indexing and append are O(1) (append amortised, thanks to proportional over-allocation of about 12.5%), but inserting or deleting anywhere except the end shifts elements and is O(n), and membership tests are O(n)."

- *Q: Why amortised?* A: Occasional resizes copy everything, but they happen rarely enough that the total copying over n appends is O(n).
- *Q: Linked list vs Python list?* A: Python has no built-in linked list; `deque` is a doubly-linked list of fixed-size blocks, giving O(1) at both ends but O(n) indexing in the middle.
- *Q: Why is `x in lst` slow?* A: It compares against each element in turn — linear scan.

Self-check

1. What is stored in a list's slots? → Pointers to objects, not the objects themselves.
2. Cost of `lst.insert(0, x)` on a million-item list? → O(n): a million pointers move.
3. Why doesn't every append copy the list? → Spare capacity is reserved in advance.

TL;DR

List = dynamic array of pointers. O(1): index, append, pop-last. O(n): insert/delete at front or middle, `in`, `index`, `remove`. Queue → `deque`; lookups → `set`.

5.4 Adding and removing items

What it is

Methods and statements that change a list's contents.

Goal	Use	Returns	Error when
Add one item at the end	`append(x)`	`None`	–
Add one item at position i	`insert(i, x)`	`None`	– (i beyond ends is clamped)
Add all items of an iterable	`extend(iterable)` / `+=`	`None`	Not iterable
Remove first item equal to x	`remove(x)`	`None`	`ValueError` if absent
Remove by position and get it	`pop(i=-1)`	The item	`IndexError` on empty/out of range
Remove by position or slice	`del a[i]`, `del a[i:j]`	–	`IndexError`
Remove everything	`clear()` (or `del a[:]`)	`None`	–

```
fruits = ["apple", "banana"]
fruits.append("cherry");              print("append:", fruits)
fruits.insert(1, "orange");           print("insert:", fruits)
fruits.extend(["grape", "mango"]);    print("extend:", fruits)
fruits.remove("banana");              print("remove:", fruits)
last = fruits.pop();                  print("pop():", fruits, "->", last)
first = fruits.pop(0);                print("pop(0):", fruits, "->", first)
del fruits[0];                        print("del [0]:", fruits)
fruits.clear();                       print("clear:", fruits)
x = [1, 2]
x.append([3, 4]);  print("append a list :", x)   # nests!
y = [1, 2]
y.extend([3, 4]);  print("extend a list :", y)
try:
    y.remove(99)
except ValueError as e:
    print("ValueError:", e)
```

Output:

```
append: ['apple', 'banana', 'cherry']
insert: ['apple', 'orange', 'banana', 'cherry']
extend: ['apple', 'orange', 'banana', 'cherry', 'grape', 'mango']
remove: ['apple', 'orange', 'cherry', 'grape', 'mango']
pop(): ['apple', 'orange', 'cherry', 'grape'] -> mango
pop(0): ['orange', 'cherry', 'grape'] -> apple
del [0]: ['cherry', 'grape']
clear: []
append a list : [1, 2, [3, 4]]
extend a list : [1, 2, 3, 4]
ValueError: list.remove(x): x not in list
```

Common mistakes

- `append` vs `extend`: `append([3, 4])` adds **one** element (a nested list); `extend([3, 4])` adds **two**. `extend("hi")` adds `'h'` and `'i'`.
- Removing all occurrences with a `while x in lst: lst.remove(x)` loop — $O(n^2)$. Use `lst = [v for v in lst if v != x]`.
- `lst.clear()` vs `lst = []`: `clear()` empties the **same** object (every other reference sees an empty list); `lst = []` rebinds only this name to a new list.

Self-check

1. `[1].extend("ab")`? → The list becomes `[1, 'a', 'b']`.
2. Which removal method returns the removed item? → `pop`.
3. What does `remove` do with duplicates? → Removes only the first.

TL;DR

`append` one, `extend` many, `insert` at a position; `remove` by value, `pop` by index (returns it), `del` by index/slice, `clear` everything.

5.5 List methods (complete table) and slicing in depth

The full method table

Method	Does	Cost
`append(x)`	Add `x` at the end	O(1) amortised
`extend(it)`	Add every item of `it`	O(k)
`insert(i, x)`	Insert before index `i`	O(n)
`remove(x)`	Delete first item equal to `x`	O(n)
`pop([i])`	Remove and return item `i` (default last)	O(1) last / O(n)
`clear()`	Remove all items	O(n) (drops each reference)
`index(x[, start[, end]])`	First index of `x`	O(n)
`count(x)`	Number of items equal to `x`	O(n)
`sort(*, key=None, reverse=False)`	Sort in place (stable)	O(n log n)
`reverse()`	Reverse in place	O(n)
`copy()`	Shallow copy	O(n)

Only 11 methods — everything else is a built-in function (`len`, `sorted`, `reversed`, `min`, `max`, `sum`, `enumerate`, `zip`) or an operator.

```
colors = ["red", "blue", "green", "blue"]
print(colors.count("blue"), colors.index("blue"), colors.index("blue", 2))
colors.remove("blue");   print(colors)          # only the first 'blue'
colors.append("amber")
colors.sort();           print("sort:", colors)
colors.reverse();        print("reverse:", colors)
nums = [3, 1, 2]
snapshot = nums.copy()
nums.sort(reverse=True)
print(nums, snapshot)
```

Output:

```
2 1 3
['red', 'green', 'blue']
sort: ['amber', 'blue', 'green', 'red']
reverse: ['red', 'green', 'blue', 'amber']
[3, 2, 1] [3, 1, 2]
```

`index("blue", 2)` starts searching at position 2, so it finds the second `"blue"` at index 3.

Slicing in depth — steps and slice assignment

Everything about string slicing (Chapter 4) applies: `a[start:stop:step]`, stop excluded, negatives from the end, never raises. Because lists are mutable you can also **assign to** and **delete** slices.

Form	Rule
`a[i:j] = iterable`	Replace that range; the new data may be **longer or shorter**
`a[i:i] = iterable`	Empty range → pure **insertion** at `i`
`a[i:j:k] = iterable`	Extended (stepped) slice → lengths **must match**
`del a[i:j:k]`	Delete the selected positions
`a[:] = []`	Empty the list in place (same as `clear()`)

```
x = list(range(10))
print(x[1:8:2], x[::-2], x[-3:])
y = x.copy(); y[2:5] = ["a"];           print("shrink :", y)      # lengths may differ
y = x.copy(); y[2:2] = ["new", "items"]; print("insert :", y)     # empty slice = insert
y = x.copy(); y[::2] = [0] * 5;         print("stepped:", y)      # lengths must match
y = x.copy(); del y[::2];               print("del ::2:", y)
y = x.copy(); y[:] = [];                print("clear  :", y)
try:
    y = x.copy(); y[::2] = [1, 2]
except ValueError as e:
    print("ValueError:", e)
```

Output:

```
[1, 3, 5, 7] [9, 7, 5, 3, 1] [7, 8, 9]
shrink : [0, 1, 'a', 5, 6, 7, 8, 9]
insert : [0, 1, 'new', 'items', 2, 3, 4, 5, 6, 7, 8, 9]
stepped: [0, 1, 0, 3, 0, 5, 0, 7, 0, 9]
del ::2: [1, 3, 5, 7, 9]
clear  : []
ValueError: attempt to assign sequence of size 2 to extended slice of size 5
```

Common mistakes

- Expecting `a[1:3] = 5` to work — the right side must be iterable: `a[1:3] = [5]`.

- Forgetting that a slice **read** is a shallow copy but a slice **assignment** changes the original.

Self-check

1. `x = [0,1,2,3]; x[1:3] = []` → ? → `[0, 3]`.
2. Can `a[::2] = [1]` work on a 4-item list? → No, the stepped slice has 2 slots; lengths must match.
3. How many methods does `list` have? → 11.

TL;DR

11 methods; slicing reads copies, slice assignment edits in place (length may change unless there's a step).

5.6 List comprehensions

What it is

A **list comprehension** builds a new list from an iterable in one expression:

```
[expression for item in iterable if condition]
```

- **expression** – what to put in the new list (e.g. `n ** 2`).
- **for item in iterable** – the source loop.
- **if condition** – optional filter; items that fail it are skipped.

The problem it solves

The traditional pattern – create an empty list, loop, `append` – takes three or four lines to say one thing, and the repeated `.append` attribute lookup and method call cost time. A comprehension says it in one readable line and runs as specialised bytecode.

How it works

1. Python creates an empty list.
2. For each `item` in the iterable, it evaluates `condition` (if present).
3. If true (or absent), it evaluates `expression` and appends the result.
4. Multiple `for` clauses nest **left to right**, in the same order you'd write nested loops.

```
numbers = [1, 2, 3, 4, 5]
squared = []
for n in numbers:                          # the traditional way
    squared.append(n ** 2)
print("loop:", squared)
print("comp:", [n ** 2 for n in numbers])
print("even squares:", [n ** 2 for n in numbers if n % 2 == 0])
print("if/else in expr:", ["even" if n % 2 == 0 else "odd" for n in numbers])
print([f"Number {n} squared is {n ** 2}." for n in numbers[:2]])

matrix = [[1, 2, 3], [4, 5, 6]]
flat = [x for row in matrix for x in row]          # same order as nested for-loops
transposed = [[row[i] for row in matrix] for i in range(3)]
pairs = [(c, n) for c in "ab" for n in (1, 2)]
print(flat, transposed, pairs)

total = sum(n * n for n in range(1_000_000))       # generator expression: no list built
```

Output:

```
loop: [1, 4, 9, 16, 25]
comp: [1, 4, 9, 16, 25]
even squares: [4, 16]
if/else in expr: ['odd', 'even', 'odd', 'even', 'odd']
['Number 1 squared is 1.', 'Number 2 squared is 4.']
[1, 2, 3, 4, 5, 6] [[1, 4], [2, 5], [3, 6]] [('a', 1), ('a', 2), ('b', 1), ('b', 2)]
sum of squares: 333332833333500000
loop+append: 39.0 ms   comprehension: 31.6 ms (ratio 1.2x; machine-dependent)
```

Note the two different `if` positions:

- `[... for n in xs if cond]` – **filter** (fewer items out).
- `[a if cond else b for n in xs]` – **choose a value** (same number of items out).

The speed gain is real but modest (1.2× here). Choose comprehensions for **clarity**; the speed is a bonus.

Trade-offs

	Comprehension	`for` loop + `append`	Generator expression `( … )`	`map` / `filter`
Readability for simple transforms	Best	OK	Best	OK with named functions
Memory	Whole list	Whole list	One item at a time	Lazy iterator
Complex logic, try/except, prints	Poor	Best	Poor	Poor

When not to use it

- When the body needs several statements, error handling or side effects – write a loop.
- When you only iterate once (e.g. feeding `sum`, `any`, `max`, `"".join`) – use a **generator expression** with round brackets to avoid building a list.
- More than two `for` clauses, or nested comprehensions deeper than one level, usually hurt readability (*Effective Python* recommends no more than two control subexpressions).
- For side effects only (`[print(x) for x in xs]`) – that builds a useless list of `None`s.

Common mistakes

- Reversing the order of nested `for`s: `[x for x in row for row in matrix]` fails (`row` not defined yet).
- Expecting the loop variable to leak: in Python 3 comprehension variables are local to the comprehension.

Real-world examples

Data-cleaning code in pandas/ETL scripts (`[c.strip().lower() for c in columns]`), building SQL parameter lists, and flattening API pages (`[item for page in pages for item in page["results"]]`).

In the interview

30-second answer: "`[expr for x in it if cond]` builds a list in one expression; it's clearer and a bit faster than loop-and-append. Use a generator expression when you only need to iterate once, and a normal loop when the logic gets complicated."

- *Q: Order of multiple fors?* A: Left to right, same as nested loops.
- *Q: Dict and set versions?* A: `{k: v for ...}` and `{x for ...}` (Chapter 6).

Self-check

1. `[c.upper() for c in "abc" if c != "b"]`? → `['A', 'C']`.
2. How do you avoid materialising a list in `sum(...)`? → Use a generator expression: `sum(x*x for x in xs)`.
3. Flatten `[[1],[2,3]]`? → `[x for sub in lst for x in sub]` → `[1, 2, 3]`.

TL;DR

`[expr for x in it if cond]`; `if/else` goes before `for`; nested `for`s read left to right; use `( … )` for one-pass use, a loop for complex logic.

5.7 Sorting

What it is

- `lst.sort()` – sorts the list **in place** and returns `None`.
- `sorted(iterable)` – returns a **new** sorted list from any iterable (list, tuple, string, set, dict keys, generator).

Both accept `key=` (a function that computes the value to sort by) and `reverse=True`.

Why it matters

Sorting underlies reports, rankings, deduplication, binary search and "top-k" queries. Knowing `key` functions and **stability** lets you express almost any ordering in one line.

How it works

1. If `key` is given, Python calls it **once per element** and sorts by those results (decorate–sort–undecorate done for you).
2. Elements (or keys) are compared with `<`. Mixed types that can't be compared raise `TypeError`.
3. The algorithm is **Timsort** (since Python 2.3; 3.11 swapped in the "powersort" merge policy). It finds already-ordered runs and merges them: **O(n log n)** worst case, **O(n)** on already-sorted data.
4. It is **stable**: items with equal keys keep their original relative order. That lets you sort by several keys in multiple passes – secondary key first, primary key last.

```
nums = [5, 2, 9, 1]
new = sorted(nums)                  # returns a NEW list, works on any iterable
print(new, nums)
result = nums.sort()                # sorts IN PLACE, returns None
print(nums, result)
print(sorted("banana"), sorted({3, 1, 2}), sorted({"b": 1, "a": 2}))

words = ["banana", "Apple", "cherry", "Date"]
print(sorted(words))                            # uppercase first (code points)
print(sorted(words, key=str.casefold))          # case-insensitive
print(sorted(words, key=len))                   # by length
print(sorted(words, key=len, reverse=True))
people = [("Ann", 31), ("Bob", 25), ("Cid", 31)]
print(sorted(people, key=itemgetter(1)))        # same as key=lambda p: p[1]
```

Output:

```
[1, 2, 5, 9] [5, 2, 9, 1]
[1, 2, 5, 9] None
['a', 'a', 'a', 'b', 'n', 'n'] [1, 2, 3] ['a', 'b']
['Apple', 'Date', 'banana', 'cherry']
['Apple', 'banana', 'cherry', 'Date']
['Date', 'Apple', 'banana', 'cherry']
['banana', 'cherry', 'Apple', 'Date']
[('Bob', 25), ('Ann', 31), ('Cid', 31)]
```

Stability in action: with `key=len`, `"banana"` and `"cherry"` both have length 6, and they come out in their original order (banana before cherry). With `reverse=True` they *still* keep that order – `reverse` flips the comparison, not the tie order, so stability is preserved.

Multi-key sorts – department ascending, then age descending:

```
people = [("Ann", "HR", 31), ("Bob", "IT", 25), ("Cid", "HR", 25), ("Dee", "IT", 31)]
by_age = sorted(people, key=itemgetter(2))
print("by age (ties keep original order):", [p[0] for p in by_age])
print("tuple key :", [p[0] for p in sorted(people, key=lambda p: (p[1], -p[2]))])
tmp = sorted(people, key=itemgetter(2), reverse=True)        # secondary key first
print("two passes:", [p[0] for p in sorted(tmp, key=itemgetter(1))])
```

Output:

```
by age (ties keep original order): ['Bob', 'Cid', 'Ann', 'Dee']
tuple key : ['Ann', 'Cid', 'Dee', 'Bob']
two passes: ['Ann', 'Cid', 'Dee', 'Bob']
```

A **tuple key** compares field by field; negating a number flips its direction. When a field can't be negated (e.g. a string you want descending), use two stable passes instead.

Trade-offs

	`list.sort()`	`sorted()`
Input	Lists only	Any iterable
Result	Same list, changed	New list
Memory	No extra list	Extra O(n)
Keep original?	No	Yes

Common mistakes

- `lst = lst.sort()` → `lst` becomes `None`.
- Sorting mixed types (`[3, "a"]`) → `TypeError: '<' not supported between instances of 'str' and 'int'`.
- Sorting numeric strings as strings (`"10" < "9"`) – use `key=int`.
- Writing a `lambda` when `operator.itemgetter` / `attrgetter` or `str.casefold` already exists (clearer, slightly faster).

Real-world examples

Timsort, designed by Tim Peters for CPython, was adopted for object sorting in Java (`Arrays.sort` on objects, Java 7), on Android, and in V8 (Chrome/Node.js) since 2018. Leaderboards, "sort by price then rating" filters on shopping sites, SQL's `ORDER BY a, b DESC` and pandas' `df.sort_values(["a", "b"], ascending=[True, False])` all map to key-based stable sorts.

In the interview

> **30-second answer:** "`sort` sorts a list in place and returns None; `sorted` returns a new list from any iterable. Both take `key` and `reverse`. Python uses Timsort – stable, O(n log n), O(n) on nearly sorted data – and stability lets you do multi-key sorts with a tuple key or successive passes."

- *Q: Why is stability useful?* A: Equal elements keep prior order, enabling multi-pass sorts (e.g. by name, then by department).
- *Q: How many times is `key` called?* A: Once per element.
- *Q: Top 3 of a million items?* A: `heapq.nlargest(3, items)` – O(n log k) instead of sorting everything (Chapter 6).

Self-check

1. What does `sorted("cab")` return? → `['a', 'b', 'c']` (a list).
2. Sort strings by length, longest first? → `sorted(words, key=len, reverse=True)`.

3. Sort by dept ascending, salary descending? → `key=lambda e: (e.dept, -e.salary)`.

TL;DR

`sort()` in place/None, `sorted()` new list; `key=` once per item; Timsort is stable and adaptive.

5.8 Copying — aliasing, shallow vs deep, and the nested-list trap

What it is

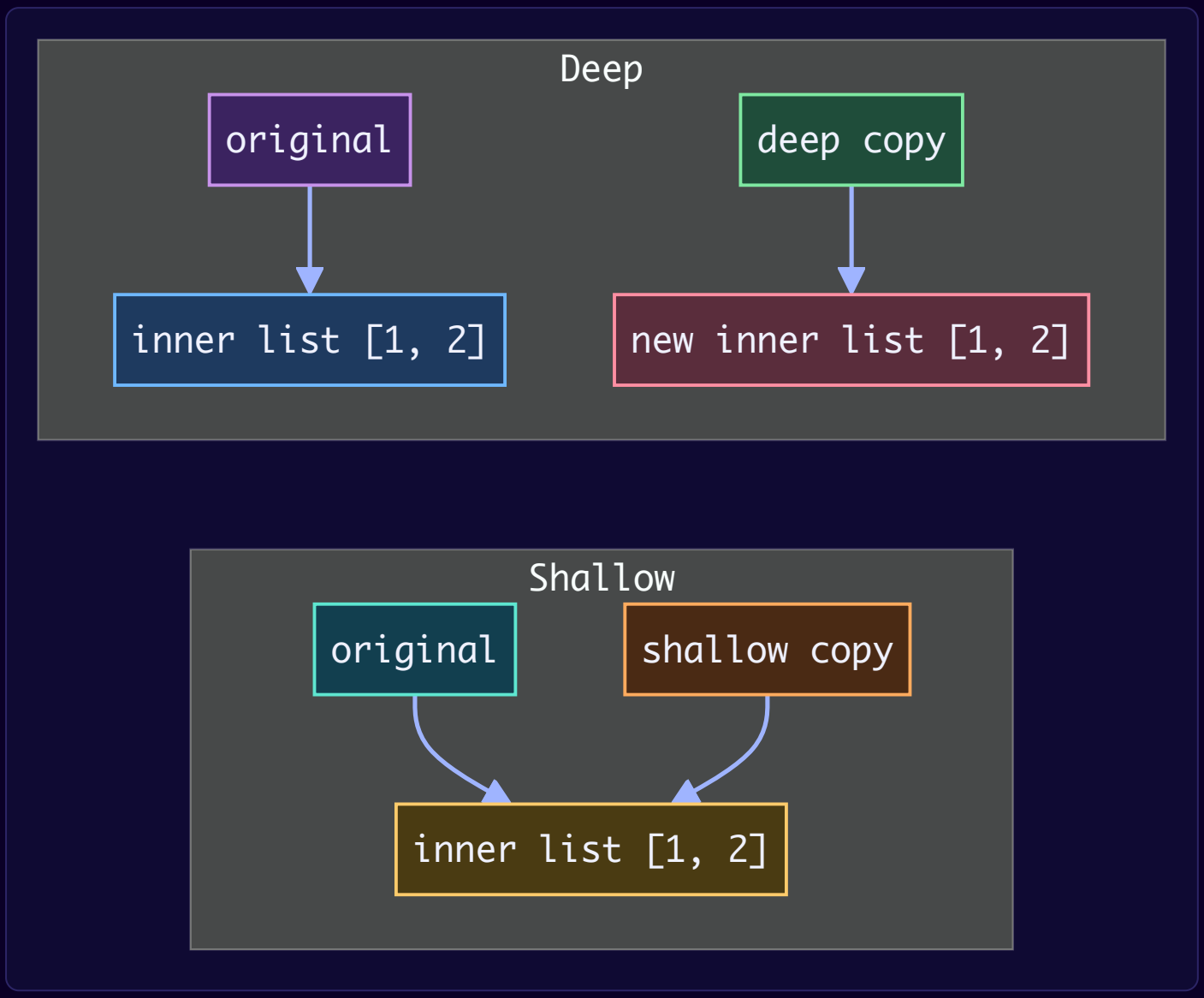
- **Aliasing**: two names referring to the **same** object. `b = a` never copies.
- **Shallow copy**: a new outer container whose slots point to the **same** inner objects. Made by `a.copy()`, `list(a)`, `a[:]`, `copy.copy(a)`.
- **Deep copy**: a new outer container **and** recursive copies of everything inside. Made by `copy.deepcopy(a)`.

Analogy: aliasing is giving a house a second address; a shallow copy is building a new house but sharing the same furniture; a deep copy duplicates the house and every piece of furniture.

Why it matters

Most "my data changed by itself" bugs come from aliasing or shallow copies of nested data.

How it works



```
a = [1, 2, 3]
b = a                        # NOT a copy: two names, one list
b.append(4)
print(a, b, a is b)

original = [[1, 2], [3, 4]]
shallow = original.copy()
deep = copy.deepcopy(original)
original[0].append(99)                  # mutate an INNER list
original.append([5])                    # mutate the OUTER list
print("original:", original)
print("shallow :", shallow, "<- shares inner lists")
print("deep    :", deep, "<- fully independent")
print(shallow[0] is original[0], deep[0] is original[0])
```

Output:

```
[1, 2, 3, 4] [1, 2, 3, 4] True
original: [[1, 2, 99], [3, 4], [5]]
shallow : [[1, 2, 99], [3, 4]] <- shares inner lists
deep    : [[1, 2], [3, 4]] <- fully independent
True False
```

The shallow copy did **not** get `[5]` (its outer list is separate) but **did** see `99` (the inner list is shared).

Passing a list to a function also passes a reference — the function can mutate your list:

```
[1, 2, 3, 4, 'x']        # after add_item(a) appended 'x'
```

The `[[0] * 3] * 3` trap

`*` repeats **references**. `[[0] * 3] * 3` creates one inner row and puts three pointers to it in the outer list.

```
bad = [[0] * 3] * 3                    # 3 references to ONE row
bad[0][0] = 1
good = [[0] * 3 for _ in range(3)]     # 3 separate rows
good[0][0] = 1
```

Output:

```
bad : [[1, 0, 0], [1, 0, 0], [1, 0, 0]] ids: 1 distinct row(s)
good: [[1, 0, 0], [0, 0, 0], [0, 0, 0]] ids: 3 distinct row(s)
```

(`[0] * 3` itself is fine: integers are immutable, so sharing them is harmless.)

The mutable default argument trap

Default values are evaluated **once**, when `def` runs, so a list default is shared by every call:

```
def bad_append(item, bucket=[]):
    bucket.append(item); return bucket
def good_append(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item); return bucket
print(bad_append(1), bad_append(2))    # [1, 2] [1, 2]
print(good_append(1), good_append(2))  # [1] [2]
```

Trade-offs

	Alias `b = a`	Shallow `a.copy()`	Deep `copy.deepcopy(a)`
Cost	O(1)	O(n)	O(total size of everything)
Independent outer list	No	Yes	Yes
Independent nested objects	No	No	Yes
Handles cycles	–	–	Yes (keeps a memo)

When not to use it

Don't `deepcopy` by reflex – it's slow and copies things you may want shared (caches, connections). If data is flat or contains only immutable items, a shallow copy is a full copy in practice. Consider immutable structures (tuples, frozen dataclasses) so copying isn't needed.

Common mistakes

- `grid = [[None] * cols] * rows` – use a comprehension.
- Thinking `a[:]` copies nested lists – it doesn't.
- `def f(x=[])` / `def f(d={})` – use `None` and create inside.

In the interview

> **30-second answer:** "Assignment never copies – it creates another reference. A shallow copy (`copy()` , `list()` , `[:]`) duplicates only the outer container; nested objects are shared. `copy.deepcopy` copies recursively. Classic traps: `[[0]*3]*3` shares one row, and mutable default arguments are shared between calls."

- *Q: Does `a[:]` copy?* A: Shallowly, yes.
- *Q: Does deepcopy handle cycles?* A: Yes – it tracks already-copied objects in a memo dict.

Self-check

1. `a = [[1]]; b = a[:]; b[0].append(2)` – what is `a` ? → `[[1, 2]]` .
2. Correct way to build a 3×3 zero grid? → `[[0]*3 for _ in range(3)]` .
3. Why is `def f(x=[])` dangerous? → The same list is reused across calls.

TL;DR

`=` aliases, `copy()` shares the insides, `deepcopy` shares nothing. Never multiply a list of lists.

5.9 Tuples – immutable sequences and records

What it is

A **tuple** is an ordered, **immutable** sequence. The **commas** make it; parentheses are just for grouping/readability: `(1, 2, 3)` or `1, 2, 3` (**tuple packing**). A one-element tuple needs a trailing comma: `(42,)` .

Analogy: a list is a shopping list you keep editing; a tuple is a printed receipt – you can read and copy it, not edit it.

Why it matters

- **Integrity:** data that shouldn't change (coordinates, RGB colours, database rows, function return bundles) can't be changed by accident.
- **Hashable** (if all its items are hashable) → can be a dict key or set element: `{(59.9, 10.7): "Oslo"}` .

- **Lighter** than lists: fixed size, no spare capacity. A 3-item tuple is 72 bytes vs 88 for a list on this build. CPython can also build tuples of constants at compile time.
- **Thread safety by construction:** an object nobody can mutate can be shared between threads without locks (the objects inside may still be mutable, see below).

How it works – define, access, "update", loop, operators, slice

```
t = (1, 2, 3, 4, 5)
packed = "apple", "banana", "cherry"          # the commas make the tuple
single, not_a_tuple, empty = (42,), (42), ()
print(t, packed, type(single).__name__, type(not_a_tuple).__name__, empty)
print(t[0], t[2], t[-1], t[1:4], t[::-1])
try:
    t[1] = 20
except TypeError as e:
    print("TypeError:", e)
as_list = list(t); as_list[1] = 20; t2 = tuple(as_list)   # "update" = build a new tuple
print("updated copy:", t2)
for item in t:
    print(item, end=" ")
a, b = (1, 2, 3), (4, 5, 6)
print(a + b, a * 2, 3 in a, 7 not in a, (1, 2) < (1, 3))
```

Output:

```
(1, 2, 3, 4, 5) ('apple', 'banana', 'cherry') tuple int ()
1 3 5 (2, 3, 4) (5, 4, 3, 2, 1)
TypeError: 'tuple' object does not support item assignment
updated copy: (1, 20, 3, 4, 5)
1 2 3 4 5
(1, 2, 3, 4, 5, 6) (1, 2, 3, 1, 2, 3) True True True
```

Operator	Meaning	Example
`+`	Concatenate into a new tuple	`(1, 2) + (3,)` → `(1, 2, 3)`
`*`	Repeat	`(1, 2) * 2` → `(1, 2, 1, 2)`
`in` / `not in`	Membership	`3 in (1, 2, 3)` → `True`
`<`, `==`, …	Lexicographic comparison	`(1, 2) < (1, 3)` → `True`
`[i]`, `[i:j:k]`	Index and slice (slices are new tuples)	`(0,1,2,3)[1:3]` → `(1, 2)`

To "update" a tuple: convert to a list, change it, convert back – or build a new tuple with slicing: `t[:1] + (20,) + t[2:]`.

Immutability is shallow

The tuple's **slots** can't change, but a mutable object **inside** can:

```
t = ([1, 2], "x")
t[0].append(3)
print(t)                  # ([1, 2, 3], 'x')
hash(t)                   # TypeError: unhashable type: 'list'
```

A tuple is hashable only if everything inside is hashable.

Tuples as records

Think of a tuple as a **record** where *position has meaning*: `("Ann", "HR", 31)` = name, department, age. Lists are for **many things of the same kind** (a list of employees); tuples are for **one thing with several fields** (one

employee). That's the guidance in *Fluent Python*, and it's why functions return tuples: `divmod(17, 5)` → `(3, 2)`.

Common mistakes

- `(42)` is just the int 42 — write `(42,)`.
- Trailing comma accidents: `x = 1,` makes `x` a tuple.
- Assuming a tuple of lists is safe to use as a dict key — it isn't hashable.

Self-check

1. `type(("a"))`? → `str`. `type(("a",))`? → `tuple`.
2. Can you append to a list inside a tuple? → Yes; the tuple still holds the same list object.
3. Why can tuples be dict keys and lists cannot? → Tuples (of hashable items) are immutable and hashable.

TL;DR

Commas make tuples; they're immutable (shallowly), hashable if their contents are, lighter than lists, and ideal as fixed-shape records.

5.10 Unpacking tuples (and any iterable)

What it is

Unpacking assigns the items of a sequence to several names in one statement: `a, b, c = (1, "Hello", 3.14)`. It works with any iterable — lists, strings, generators, dict items.

Why it matters

It removes index noise (`row[0]`, `row[1]`) and so removes off-by-one bugs, and it's how you receive multiple return values, swap variables and loop over pairs.

How it works

1. The right side is evaluated fully first.
2. Python iterates it and binds each item to the next name on the left.
3. The counts must match — otherwise `ValueError` ("not enough values" / "too many values").
4. **Star-unpacking** (`*name`) collects "everything else" into a **list** (possibly empty). Only one star per assignment.
5. Unpacking nests: `for name, (lat, lon) in ...`.

```
a, b, c = (1, "Hello", 3.14)
print(a, b, c)
try:
    w, x, y, z = (1, "Hello", 3.14)
except ValueError as e:
    print("ValueError:", e)
first, *middle, last = (1, 2, 3, 4, 5, 6)
print(first, middle, last)
head, *tail = [10]
print(head, tail)
x, y = 1, 2
x, y = y, x                                  # swap
print(x, y)
for name, (lat, lon) in [("Oslo", (59.9, 10.7)), ("Lima", (-12.0, -77.0))]:
    print(f"{name}: {lat}, {lon}")
_, _, ext = "archive.tar.gz".partition(".")   # _ = "don't care"
print(ext)
```

Output:

```
1 Hello 3.14
ValueError: not enough values to unpack (expected 4, got 3)
1 [2, 3, 4, 5] 6
10 []
2 1
Oslo: 59.9, 10.7
Lima: -12.0, -77.0
tar.gz
```

Wrapping risky unpacking in `try/except ValueError` (as above) lets a program report bad input (say, a malformed CSV row) instead of crashing.

The swap `x, y = y, x` works because the right side builds the tuple `(2, 1)` **before** any assignment happens.

Star in calls and literals (the same idea, in reverse):

Syntax	Meaning	Example
`f(*seq)`	Spread items as positional arguments	`print(*[1, 2, 3])` → `1 2 3`
`[*a, *b]`	Build a list from several iterables	`[*(1, 2), *[3]]` → `[1, 2, 3]`
`(*a, x)`	Build a tuple	`(*t, 99)`
`def f(*args)`	Collect extra positional arguments into a tuple	Chapter on functions

Common mistakes

- Using two stars: `a, *b, *c = ...` → `SyntaxError`.
- Expecting `*middle` to be a tuple — it is always a **list**.
- Unpacking a dict directly gives keys; use `.items()` for pairs.

In the interview

30-second answer: "Unpacking binds items of any iterable to names; the count must match or you get ValueError. `first, *rest = seq` gathers the leftovers into a list. It powers multiple return values, swaps without a temp, and clean loops like `for k, v in d.items()`."

- *Q: Is `a, b = b, a` atomic?* A: It's evaluated as build tuple, then assign — no temp variable needed, but it isn't a thread-atomic operation.
- *Q: What does `*_` do?* A: Discards any number of items: `first, *_ = row`.

Self-check

1. `a, *b = "hey"` ? → `a='h'`, `b=['e', 'y']`.
2. `x, y = [1, 2, 3]` ? → `ValueError: too many values to unpack (expected 2)`.
3. How to swap `a` and `b` ? → `a, b = b, a`.

TL;DR

`a, b, c = seq`; counts must match; `*rest` gathers a list; `_` for ignored slots; works in `for` loops and nests.

5.11 Joining tuples

What it is

Combining several tuples into one. Because tuples are immutable, every join creates a **new** tuple.

How it works — five ways

```
parts = [(1, 2), (3, 4), (5, 6)]
out = ()
for p in parts:
    out += p                                # new tuple every time: O(n^2) overall
print("loop     :", out)
print("+        :", (1, 2, 3) + (4, 5, 6))
print("sum      :", sum(parts, ()))          # also O(n^2): repeated +
print("genexpr  :", tuple(x for p in parts for x in p))
print("chain    :", tuple(itertools.chain.from_iterable(parts)))
```

Output (last line: joining 3,000 two-item tuples):

```
loop     : (1, 2, 3, 4, 5, 6)
+        : (1, 2, 3, 4, 5, 6)
sum      : (1, 2, 3, 4, 5, 6)
genexpr  : (1, 2, 3, 4, 5, 6)
chain    : (1, 2, 3, 4, 5, 6)
3000 tuples: sum 14.55 ms vs chain 0.052 ms
```

- `+` – perfect for two or three tuples.
- `+=` in a loop and `sum(parts, ())` – both repeatedly create a bigger tuple and copy everything so far, so joining m tuples costs $O(\text{total}^2)$. `sum` is **not** faster than the loop; it does the same work. (The docs recommend `itertools.chain` for concatenating sequences – `sum` even refuses strings outright.)
- `tuple(x for p in parts for x in p)` – a flattening generator expression: the outer `for` walks each tuple, the inner `for` walks its items, and `tuple()` collects them once → O(total).
- `tuple(itertools.chain.from_iterable(parts))` – the idiomatic linear solution, implemented in C: ~280× faster here.

Common mistakes

- Joining a tuple and a list: `(1,) + [2]` → `TypeError`. Convert first.
- Forgetting the comma when adding one item: `t + (4)` fails; `t + (4,)` works.

Self-check

1. Fastest general way to join many tuples? → `tuple(itertools.chain.from_iterable(tuples))`.
2. Is `sum(list_of_tuples, ())` linear? → No, quadratic.

TL;DR

Few tuples → `+`. Many → `tuple(chain.from_iterable(...))` or a flattening generator. Avoid `+=` loops and `sum`.

5.12 Tuple methods

Tuples have only two methods – everything that would modify is missing by design.

Method	Returns	Cost
`count(x)`	How many items equal `x`	O(n)
`index(x[, start[, end]])`	Position of the first `x`; `ValueError` if absent	O(n)

```
fruit = ("apple", "banana", "cherry", "apple", "cherry")
print(fruit.count("apple"), fruit.count("cherry"), fruit.index("banana"), fruit.index("cherry"))
```

Output:

```
2 2 1 2
```

`index("cherry")` returns `2` – the **first** cherry – even though another is at 4. The built-ins `len`, `min`, `max`, `sum`, `sorted` (returns a list), `reversed`, `any`, `all` all work on tuples too.

TL;DR

`count` and `index`, both O(n). Everything else comes from built-ins.

5.13 `namedtuple` and `typing.NamedTuple` – tuples with field names

What it is

`collections.namedtuple` creates a **tuple subclass** whose positions also have names: `p.x` instead of `p[0]`. `typing.NamedTuple` is the same thing written as a class with type hints and defaults.

The problem it solves

`emp[2]` says nothing; `emp.salary` says everything. Named tuples keep all tuple benefits (immutable, hashable, unpackable, as small as a tuple – no per-instance `__dict__`) while making code self-documenting.

How it works

```
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p, p.x, p[1], p._asdict(), p._replace(x=10))
x, y = p                                    # still a tuple

class Employee(NamedTuple):                 # typed, class-style syntax
    name: str
    dept: str
    salary: int = 0
e = Employee("Ann", "HR")
print(e, e.salary)
```

Output:

```
Point(x=3, y=4) 3 4 {'x': 3, 'y': 4} Point(x=10, y=4)
Employee(name='Ann', dept='HR', salary=0) 0
```

Useful extras: `_asdict()` (to a dict, e.g. for JSON), `_replace(**changes)` (a modified **copy**), `_fields` (field names), `_make(iterable)` (build from a row).

Trade-offs

	tuple	namedtuple / NamedTuple	@dataclass	dict
Access by name	No	Yes	Yes	Yes (d["k"])
Immutable	Yes	Yes	Optional (frozen=True)	No
Hashable	If items are	If items are	If frozen	No
Memory	Smallest	Same as tuple	Bigger (use slots=True to shrink)	Biggest
Methods / validation	No	Limited	Full class	No
Also behaves as a sequence	Yes	Yes (can surprise: p == (3, 4) is True)	No	No

When not to use it

When records need mutation, validation or methods, prefer `@dataclass`. When equality with plain tuples would be a bug, prefer a dataclass too.

Real-world examples

The standard library returns named tuples from many functions: `urllib.parse.urlparse()` → `ParseResult(scheme=..., netloc=...)` (a namedtuple subclass). `sys.version_info` (`sys.version_info.major`), `time.struct_time` and `os.stat_result` are *struct sequences* – the C-level cousin with the same "tuple plus named fields" behaviour. Database drivers use them for rows too (e.g. psycopg 3's `namedtuple_row` row factory).

In the interview

> **30-second answer:** "A namedtuple is a tuple subclass with named fields – immutable, lightweight, unpackable, with `_asdict` and `_replace`. I use it (or `typing.NamedTuple` for type hints) for simple immutable records and switch to dataclasses when I need mutability, validation or methods."

Self-check

1. How do you "change" a field? → `p2 = p._replace(x=10)` – returns a new object.
2. Is a namedtuple equal to a plain tuple with the same values? → Yes, which can be surprising.

TL;DR

Named fields on a tuple. Use for immutable records; use dataclasses for richer objects.

5.14 Choosing: `list` vs `tuple` vs `array.array` vs NumPy

What it is

Four sequence types, four sweet spots. NumPy's `ndarray` (third-party, `pip install numpy`) is a typed, contiguous, possibly multi-dimensional array whose operations run in compiled loops over the whole array at once (vectorisation).

How they compare – measured

One million integers, "multiply every element by 2":

```
container         memory    x*2 time
list              36.0MB      20.8ms
array.array        8.2MB      45.7ms
numpy              8.0MB       0.6ms
```

- `list` stores pointers + int objects → most memory; element-wise work runs in the interpreter.
- `array.array('q')` stores raw 8-byte ints → compact, but every read creates a Python int, so a Python-level loop is **slower** than the list.
- NumPy stores raw 8-byte ints **and** does the arithmetic in C for the whole array → ~35× faster than the list here.

Trade-offs

	`list`	`tuple`	`array.array`	`numpy.ndarray`
Mutable	Yes	No	Yes	Yes (fixed size)
Element types	Any, mixed	Any, mixed	One C numeric type	One dtype (numeric, bool, fixed strings, objects)
Memory for numbers	High	High	Low	Low
Hashable	No	Yes (if items are)	No	No
Fast element-wise maths	No	No	No	Yes (vectorised)
Multi-dimensional	Via nesting	Via nesting	No	Yes
Standard library	Yes	Yes	Yes	No (third-party)
`+` means	Concatenate	Concatenate	Concatenate	**Element-wise add**

When to use which

- **`list`** – default for general collections that change.
- **`tuple`** – fixed records, dict keys, return values, constants.
- **`array.array`** – lots of numbers kept compactly with no dependency; binary file/network formats.
- **NumPy** – any numerical computing: statistics, images, ML features, linear algebra. It's the foundation under pandas, scikit-learn, SciPy and matplotlib.

Common mistakes

- Looping over a NumPy array in Python (`for x in arr: ...`) – you lose the speed. Use vectorised expressions (`arr * 2`, `arr[arr > 0]`).
- Expecting `np_a + np_b` to concatenate – it adds element-wise (use `np.concatenate`).
- Picking `array.array` for speed – it's for memory.

In the interview

30-second answer: "List for general mutable data, tuple for immutable records and dict keys, `array.array` for compact homogeneous numbers without dependencies, NumPy for fast numerical work because it stores raw values contiguously and runs vectorised C loops."

Self-check

1. Why is `array.array` slower than a list for `[x*2 for x in arr]`? → Every element read must create a Python int object.
2. What does `+` do on NumPy arrays? → Element-wise addition.
3. Which of the four can be a dict key? → Tuple (if its items are hashable).

TL;DR

General → list. Fixed → tuple. Compact numbers → array. Fast numbers → NumPy.

Chapter TL;DR

Topic	Remember
`array` module	Typed, compact, mutable; type codes set size/range; no `sort` / `copy` methods; `clear()` from 3.13; memory win, not speed win.
Lists	Ordered, mutable, mixed types; index/slice/update; `+` new, `extend` in place.
Internals	Dynamic array of pointers; ~12.5% over-allocation → amortised O(1) append; front/middle insert/delete O(n); `in` O(n).
Add/remove	`append`, `extend`, `insert`; `remove` (value), `pop` (index, returns), `del`, `clear`.
Slicing	`[start:stop:step]`; slice assignment can resize; stepped assignment needs equal length.
Comprehensions	`[expr for x in it if cond]`; generator expression for one-pass use; loops for complex logic.
Sorting	`sort()` in place → None; `sorted()` new list; `key`, `reverse`; Timsort is stable, O(n log n), O(n) on sorted input.
Copying	`=` aliases; `copy()` / `[:]` shallow; `deepcopy` recursive; never `[[0]*n]*m`; no mutable defaults.
Tuples	Commas make them; `(x,)` for one; shallowly immutable; hashable if contents are; records.
Unpacking	Counts must match; `*rest` → list; swap with `a, b = b, a`.
Joining tuples	`+` for few; `chain.from_iterable` for many; `sum` / `+=` loops are quadratic.
Tuple methods	`count`, `index`.
namedtuple	Named, immutable records; `_asdict`, `_replace`; dataclass for more.
Choosing	list / tuple / array / NumPy by mutability, memory, and speed needs.

Chapter 6 — Dictionaries, Sets and the `collections` Toolbox

Lists and tuples find things by **position**. Dictionaries and sets find things by **value** — and they do it in constant time on average, whether they hold ten items or ten million. That single property makes them the workhorses of real Python code: caches, indexes, counters, configuration, JSON, de-duplication, graph algorithms. This chapter explains how they work (hashing), how to use every method, and which specialised container from `collections`, `heapq` and `bisect` to reach for when a plain dict, set or list isn't quite right.

What you'll learn

1. Dictionaries: define, access, update, remove, copy, loop, operators (`in`, `|`), and insertion order.
2. Hashing: what "hashable" means, how a dict/set finds a key, and the big-O table.
3. Nested dictionaries and their link to JSON.
4. Every dict method, with `get`, `setdefault`, `update` and `|` compared, plus view objects.
5. Dict and set comprehensions.
6. Sets: define, add/update, remove/discard, copy, loop.
7. Set operations (union, intersection, difference, symmetric difference) and every set method.
8. `frozenset`.
9. `collections`: `Counter`, `defaultdict`, `OrderedDict`, `deque`, `ChainMap`, `namedtuple`.
10. `heapq` (priority queues) and `bisect` (binary search), and a when-to-use table for everything.

All runnable code is in `code/`. Outputs are real (Python 3.14; code runs on 3.12+). Where a set of **strings** is printed, its order can differ between runs — string hashes are randomised per process (explained in 6.2).

6.1 Dictionaries — key → value

What it is

A **dictionary** (`dict`) stores **key–value pairs**. You look up a value by its key, not by position: `ages["Alice"]`. Keys are unique; values can be anything (numbers, strings, lists, other dicts).

- Written with braces and colons: `{"Alice": 22, "Bob": 25}`.
- **Mutable:** add, change and delete pairs at any time.
- **Keys must be hashable** (roughly: immutable — strings, numbers, tuples of hashables, `None`, `frozenset`). Lists, dicts and sets can't be keys. See 6.2.
- **Insertion-ordered:** iteration returns keys in the order they were first inserted. This is a language guarantee since **Python 3.7** (it was an implementation detail in CPython 3.6). Older tutorials that call dicts "unordered" are out of date — but dicts are still not *indexable* by position (`d[0]` looks up the key `0`).

Analogy: a real dictionary — you jump straight to a word and read its definition; you don't scan page by page.

Why it matters

Whenever you'd otherwise search a list for "the item whose name is X", a dict does it in one step. Python itself runs on dicts: module globals, object attributes (`obj.__dict__`), keyword arguments and class namespaces are all dicts.

How it works — define and access

```
ages = {"Alice": 22, "Bob": 25, "Eve": 19}
print(ages, len(ages))
print(ages["Alice"], ages.get("Bob"), ages.get("Charlie"), ages.get("Charlie", "Not Found"))
try:
    ages["Charlie"]
except KeyError as e:
    print("KeyError:", e)
other_ways = dict(a=1, b=2), dict([("x", 1), ("y", 2)]), dict(zip("pq", [7, 8]))
print(*other_ways)
print({(59.9, 10.7): "Oslo", 42: "int key", None: "None key"})
try:
    {["a", "list"]: 1}
except TypeError as e:
    print("TypeError:", e)
```

Output:

```
{'Alice': 22, 'Bob': 25, 'Eve': 19} 3
22 25 None Not Found
KeyError: 'Charlie'
{'a': 1, 'b': 2} {'x': 1, 'y': 2} {'p': 7, 'q': 8}
{(59.9, 10.7): 'Oslo', 42: 'int key', None: 'None key'}
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

- `d[key]` raises `KeyError` if the key is missing — use it when a missing key is a bug.
- `d.get(key)` returns `None` (or your default) instead — use it when absence is normal.
- Four ways to build: literal, `dict(k=v)` (string keys only), `dict(list_of_pairs)`, `dict(zip(keys, values))`.

How it works — update, remove, copy, loop

```
ages = {"Alice": 22, "Bob": 25, "Eve": 19}
ages["Alice"] = 23                          # overwrite
ages["Dave"] = 20                           # insert (goes to the end)
print(ages)
removed = ages.pop("Eve");  print("pop:", removed, ages)
del ages["Bob"];            print("del:", ages)
print(ages.pop("Zed", "no such key"))       # default avoids KeyError
```

Output:

```
{'Alice': 23, 'Bob': 25, 'Eve': 19, 'Dave': 20}
pop: 19 {'Alice': 23, 'Bob': 25, 'Dave': 20}
del: {'Alice': 23, 'Dave': 20}
no such key
```

Assignment `d[k] = v` either **updates** (key exists — its position in the order is kept) or **inserts** (new key — goes to the end). `pop` returns the removed value; `del` just deletes.

```
original = {"Alice": 23, "Charlie": 21, "tags": ["x"]}
copied = original.copy()                    # shallow
copied["Alice"] = 24
copied["tags"].append("y")                  # shared inner list!
print(original, copied, sep="\n")
for key in original:                        # iterates keys
    print(key, end=" ")
for key, value in {"Alice": 23, "Charlie": 21}.items():
    print(f"{key} is {value} years old")
```

Output:

```
{'Alice': 23, 'Charlie': 21, 'tags': ['x', 'y']}
{'Alice': 24, 'Charlie': 21, 'tags': ['x', 'y']}
Alice Charlie tags
Alice is 23 years old
Charlie is 21 years old
```

`copy()` (also `dict(d)`) is **shallow**: changing `copied["Alice"]` didn't affect the original, but both share the same `tags` list. Use `copy.deepcopy` for nested data (Chapter 5, 5.8).

Looping over a dict yields **keys**; `.values()` yields values; `.items()` yields `(key, value)` pairs, which you unpack in the `for`.

Dictionary operators

Operator	Meaning	Example
`k in d` / `k not in d`	Is `k` a **key**? (O(1))	`"Alice" in ages`
`d[k]`, `d[k] = v`, `del d[k]`	Get / set / delete	
`d1 | d2`	New merged dict; right side wins on conflicts (3.9+)	`{"a":1} | {"a":2}` → `{'a': 2}`
`d1 |= d2`	Merge into `d1` in place (like `update`)	
`{**d1, **d2}`	Merge via unpacking (3.5+)	
`==`	Same pairs (order ignored)	

```
d = {"Alice": 23, "Charlie": 21}
print("Alice" in d, "Bob" not in d, 23 in d, 23 in d.values())
a, b = {"x": 1, "y": 2}, {"y": 20, "z": 3}
print("a | b :", a | b)
a |= {"w": 0};  print("a |= :", a)
print("{**a, **b}:", {**a, **b})
print({"a": 1, "b": 2} == {"b": 2, "a": 1})
views = d.keys()
d["New"] = 0
print(views)                              # views are live
```

Output:

```
True True False True
a | b : {'x': 1, 'y': 20, 'z': 3}
a |= : {'x': 1, 'y': 2, 'w': 0}
{**a, **b}: {'x': 1, 'y': 20, 'w': 0, 'z': 3}
True
dict_keys(['Alice', 'Charlie', 'New'])
```

`23 in d` is `False` because `in` checks **keys**. To search values, use `in d.values()` — but that's an O(n) scan.

Common mistakes

- **Changing a dict's size while looping over it** → `RuntimeError: dictionary changed size during iteration`. Loop over a snapshot: `for k in list(d): ...`, or build a new dict with a comprehension.
- Using `d[k]` for optional keys and crashing on `KeyError`.
- Expecting `d.copy()` to copy nested lists/dicts.
- Relying on `d.keys()[0]` — views aren't indexable; use `next(iter(d))`.

Real-world examples

- JSON objects map straight to dicts (`json.loads`), so every web API response you parse is a dict.
- Caches: `functools.lru_cache` and hand-written memo tables are dicts from arguments to results.
- Django/Flask pass template context and request data as dicts (or dict-like objects).

In the interview

> **30-second answer:** "A dict maps hashable keys to values with average O(1) get, set, delete and membership, using a hash table. Since 3.7 it preserves insertion order. `d[k]` raises KeyError, `get` returns a default, `|` merges (3.9+), and `copy()` is shallow."

- *Q: Are dicts ordered?* A: Yes, insertion order, guaranteed since 3.7. Equality still ignores order.
- *Q: What happens on `d[k] = v` for an existing key?* A: Value replaced, position unchanged.
- *Q: `in` on dict — keys or values?* A: Keys.

Self-check

1. `{"a": 1}.get("b", 0)`? → `0`.
2. `{"a": 1} | {"a": 2, "b": 3}`? → `{'a': 2, 'b': 3}`.
3. Can a tuple be a key? → Yes, if all its items are hashable.
4. How do you delete keys while looping? → Iterate over `list(d)`.

TL;DR

Key → value, O(1) average, insertion-ordered (3.7+), hashable keys, `[]` raises, `get` defaults, `|` merges, `copy` is shallow.

6.2 Hashing — how dicts and sets find things in O(1)

What it is

A **hash** is an integer computed from a value by `hash(x)`. A **hash table** is an array of slots; the hash decides which slot a key goes into. Dicts and sets are hash tables. An object is **hashable** if it has a hash that never changes during its lifetime and can be compared with `==`.

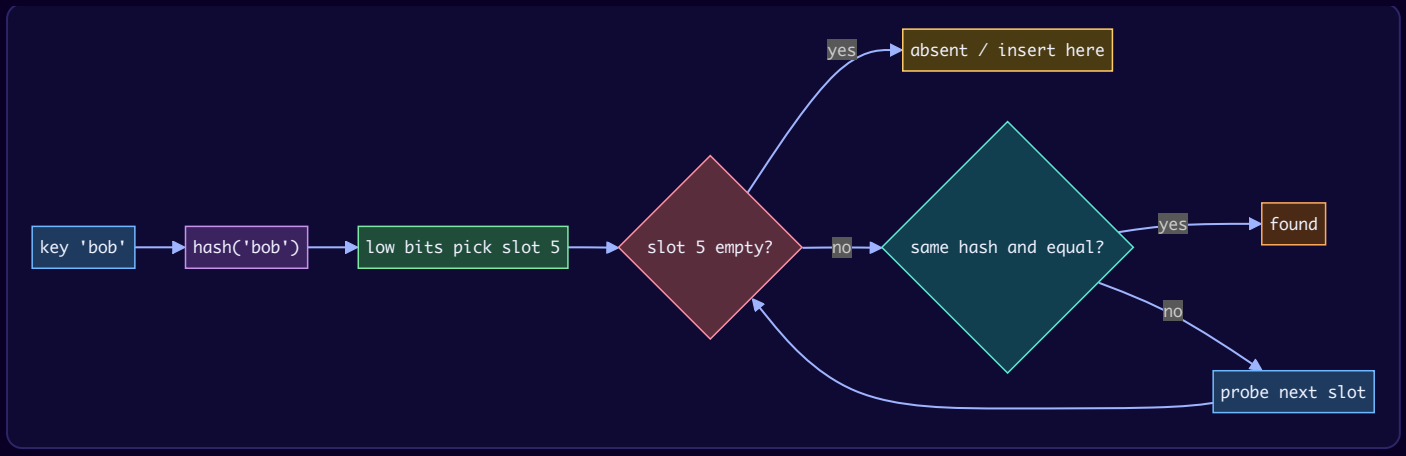
The contract (from the Python data model): **if** `a == b` **then** `hash(a) == hash(b)`. The reverse need not hold — different values may share a hash (a **collision**).

Why it matters

Hashing explains everything about dicts and sets: why lookups are O(1), why keys must be immutable, why `1` and `1.0` are the same key, and why sets have no order.

How it works — a lookup, step by step

1. Compute `h = hash(key)`.
2. Take some low bits of `h` to choose a slot in the table (table size is a power of two).
3. If the slot is empty → key absent (for lookup) or put it here (for insert).
4. If the slot holds an entry, compare: same hash **and** (`is` or `==`) → found.
5. Otherwise it's a collision: CPython uses **open addressing** — it computes the next slot to try with a pseudo-random probe sequence (mixing in more bits of the hash), and repeats from step 3.
6. The table is kept at most about **2/3 full**. When it gets fuller, CPython allocates a larger table and re-inserts everything (**resize**). Like list growth, this is rare enough that insertion is **amortised O(1)**.



Since Python 3.6, CPython's dict is a **compact dict**: a small sparse *index* table (the slots) pointing into a dense *entries* array that stores `(hash, key, value)` in insertion order. That's where insertion order comes from, and it also made dicts 20–25% smaller. Sets use a simpler, non-compact table, so they have no ordering.

Growth you can observe with `sys.getsizeof` (a table of 8 slots holds 5 entries, 16 holds 10, 32 holds 21, 64 holds 42 — the 2/3 rule):

```
 len  bytes
   1  224
   6  352
  11  632
  22  1168
  43  2264
```

What is hashable?

```
42              hashable    hash=42
3.0             hashable    hash=3
'hi'            hashable    hash=<salted: differs every run>
(1, 2)          hashable    hash=-3550055125485641917
frozenset({1})  hashable    hash=-558064481276695278
None            hashable    hash=4238894112
True            hashable    hash=1
[1]             UNHASHABLE  (unhashable type: 'list')
{1: 2}          UNHASHABLE  (unhashable type: 'dict')
{1}             UNHASHABLE  (unhashable type: 'set')
(1, [2])        UNHASHABLE  (unhashable type: 'list')
```

Hashable	Not hashable
`int`, `float`, `complex`, `bool`, `str`, `bytes`, `None`, `frozenset`, `range`	`list`, `dict`, `set`, `bytearray`
`tuple` / `namedtuple` **if every item is hashable**	A tuple containing a list
Instances of your classes (by default hashed by identity)	Classes that define `__eq__` without `__hash__` (Python sets `__hash__ = None`)
Frozen dataclasses (`@dataclass(frozen=True)`)	Regular dataclasses with `eq=True` (the default)

Equal values are the same key. `1 == 1.0 == True` and they hash equal, so:

```
d = {1: "int one"}
d[1.0] = "float one"
d[True] = "bool true"
print(d)          # {1: 'bool true'}  — one key; the first key object is kept, value replaced
```

String hashes are randomised. For security (to stop attackers crafting many colliding keys – a "hash-flooding" denial of service), `str` and `bytes` hashes are salted with a random per-process seed (default since Python 3.3; controlled by `PYTHONHASHSEED`). That's why the order of a set of strings can change between runs. Small ints hash to themselves (`hash(12345) == 12345`; `hash(-1)` is `-2` because `-1` is reserved as an error signal in C).

Mutating a key breaks the table. If a key's hash depends on mutable state and you change it while it's in a dict, the entry sits in the slot for the *old* hash:

```
lookup with same object: LOST
lookup with old value  : LOST
```

The same object can't be found (wrong slot), and a fresh object with the old value lands in the right slot but isn't `==`. That's precisely why built-in mutable types refuse to be hashable.

Big-O table (average case; worst case is O(n) when many keys collide)

Operation	`dict`	`set`
Lookup `d[k]` / `k in d` / `x in s`	O(1)	O(1)
Insert / update `d[k] = v`, `s.add(x)`	O(1) amortised	O(1) amortised
Delete `del d[k]`, `pop`, `s.remove(x)`, `discard`	O(1)	O(1)
`len()`	O(1)	O(1)
Iterate / `copy()` / `keys()` →list	O(n)	O(n)
`get`, `setdefault`, `popitem`	O(1)	`pop()` O(1)
`d1 | d2`, `update(other)`	O(len(other)) + copy	–
Union `s | t`	–	O(len(s) + len(t))
Intersection `s & t`	–	O(min(len(s), len(t)))
Difference `s - t`	–	O(len(s))
Symmetric difference `s ^ t`	–	O(len(s) + len(t))
`s <= t` (subset)	–	O(len(s))
Search a **value** `v in d.values()`	O(n)	–

The payoff, measured (`code/08_lookup_benchmark.py`, looking for the last element):

```
         n    list (µs)   set (µs)  dict (µs)
     1,000         5.26      0.030      0.031
    10,000        52.49      0.030      0.031
   100,000       536.93      0.030      0.031
 1,000,000      5533.07      0.030      0.032
```

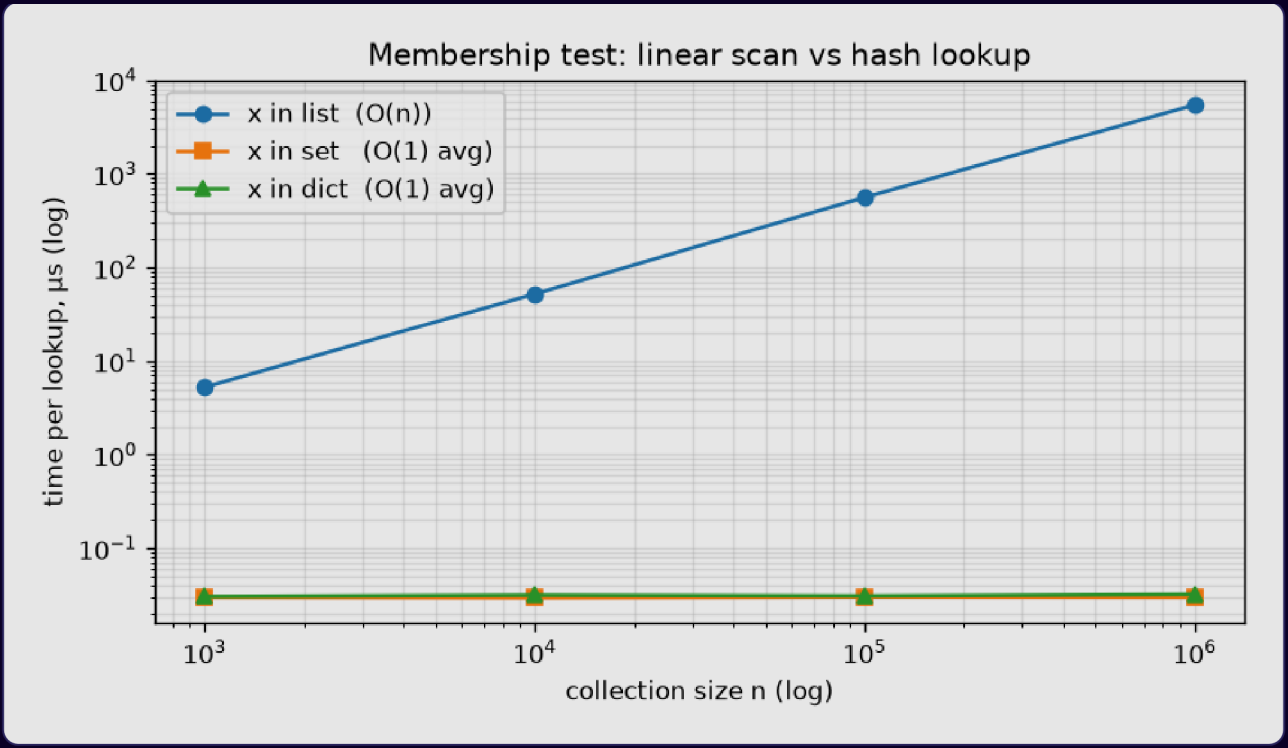


Figure — on a log–log plot the list line rises with slope 1 (linear); set and dict stay flat at ~30 ns. At a million items the set is ~180,000× faster.

Trade-offs

- **Memory:** a hash table keeps spare slots (up to 1/3 empty) plus stored hashes — much more memory than a list of the same items.
- **No positional access** and (for sets) no order.
- **Worst case** O(n) if many keys collide — rare with good hashes, but possible with a bad custom `__hash__` (e.g. `return 1`).

Common mistakes

- Defining `__eq__` on a class and being surprised it's no longer hashable — define `__hash__` from the same immutable fields (or use `@dataclass(frozen=True)`).
- Writing `__hash__` that uses mutable fields.
- Using floats as keys after arithmetic: `0.1 + 0.2` is not `0.3`, so it's a different key.

In the interview

> **30-second answer:** "Dicts and sets are hash tables using open addressing. `hash(key)` picks a slot; collisions probe other slots; equality confirms the match. The table stays ≤ 2/3 full and resizes, so operations are O(1) average, O(n) worst. Keys must be hashable — their hash can't change — and equal objects must have equal hashes, which is why `1`, `1.0` and `True` are the same key."

- *Q: Why can't lists be dict keys?* A: They're mutable; if they changed, their hash would change and the entry would be lost.
- *Q: Why does set iteration order change between runs?* A: String hashes are randomly salted per process (hash-flooding protection).
- *Q: How are dicts ordered if they're hash tables?* A: Compact dict: the hash index points into an entries array kept in insertion order.

Self-check

1. Is `(1, [2])` hashable? → No — it contains a list.
2. What's the rule linking `==` and `hash`? → Equal objects must have equal hashes.
3. `len({1, 1.0, True})`? → 1.

4. Average cost of `x in big_set`? → O(1).

TL;DR

hash → slot → compare; ≤ 2/3 full; O(1) average. Hashable = hash never changes. Equal ⇒ same hash. String hashes are salted per run.

6.3 Nested dictionaries

What it is

A **nested dictionary** is a dict whose values are themselves dicts (or lists of dicts). It models **hierarchical** data: a family tree, a config file, a JSON API response.

How it works

1. **Read** by chaining keys: `family["children"]["Alice"]["age"]`.
2. **Write** the same way: `family["children"]["Alice"]["age"] = 11`.
3. **Add a branch** by assigning a new dict at any level.
4. **Loop** over one level with `.items()`.

```
family = {
    "name": "John Doe",
    "children": {
        "Alice": {"age": 10, "gender": "Female"},
        "Bob": {"age": 8, "gender": "Male"},
    },
}
print("Parent:", family["name"])
print("Alice's age:", family["children"]["Alice"]["age"])
family["children"]["Alice"]["age"] = 11                 # update deep value
family["children"]["Cara"] = {"age": 3, "gender": "Female"}  # add a new branch
for child, info in family["children"].items():
    print(f"  {child}: {info['age']} ({info['gender']})")
```

Output:

```
Parent: John Doe
Alice's age: 10
  Alice: 11 (Female)
  Bob: 8 (Male)
  Cara: 3 (Female)
```

Safe access when a level may be missing:

```
print(family.get("children", {}).get("Zed", {}).get("age", "unknown"))   # unknown

def deep_get(d, *keys, default=None):
    for k in keys:
        if not isinstance(d, dict) or k not in d:
            return default
        d = d[k]
    return d
print(deep_get(family, "children", "Bob", "age"), deep_get(family, "x", "y", default=-1))  # 8 -1
```

JSON round trip – nested dicts are exactly what `json` reads and writes:

```
text = json.dumps(family, indent=2)
back = json.loads(text)
print(back == family)                                   # True
print(json.loads('{"1": [1, 2]}'), json.dumps({1: (1, 2)}))       # {'1': [1, 2]} {"1": [1, 2]}
```

JSON object keys are always strings and JSON has no tuples — so int keys come back as strings and tuples come back as lists.

Trade-offs

Nested dicts	Dataclasses / Pydantic models
Zero setup, matches JSON exactly	Named, typed fields, validation, IDE autocomplete
Typos in keys fail only at run time	Typos caught by type checker
Good for ad-hoc/dynamic data	Good for stable, well-known schemas

When not to use it

For deeply nested, well-defined data used throughout a codebase, convert to dataclasses or Pydantic models at the boundary; pass typed objects inside the program.

Common mistakes

- `KeyError` halfway down a chain — use `get(..., {})` chains, a helper like `deep_get`, or `try/except KeyError`.
- Shallow-copying a nested config and then mutating the copy's inner dict — changes the original too.
- Building nested structures by hand with many `if key not in d: d[key] = {}` checks — use `setdefault` or `defaultdict` (6.9).

Real-world examples

API responses (GitHub, Stripe) and config formats (JSON, YAML, TOML via `tomllib`) all load as nested dicts; pandas' `json_normalize` flattens them into tables.

Self-check

1. How do you read Bob's gender? → `family["children"]["Bob"]["gender"]`.
2. What happens to tuple values in `json.dumps`? → They become JSON arrays (lists on load).
3. Safest one-liner for a possibly missing nested key? → Chained `.get(key, {})`.

TL;DR

Chain keys to read and write; guard missing levels with `get(..., {})`; nested dicts ≈ JSON.

6.4 Dictionary methods and view objects

The full method table

Method	What it does	Cost
`d.get(k, default=None)`	Value for `k`, or `default` — never raises	O(1)
`d.setdefault(k, default=None)`	Return `d[k]`; if missing, first insert `k: default`	O(1)
`d.update(other, **kw)`	Merge pairs from a dict / iterable of pairs / keywords, overwriting	O(len(other))
`d.pop(k[, default])`	Remove `k`, return its value; `KeyError` if absent and no default	O(1)
`d.popitem()`	Remove and return the **last inserted** `(k, v)` (LIFO); `KeyError` if empty	O(1)
`d.clear()`	Remove everything	O(n)
`d.copy()`	Shallow copy	O(n)
`dict.fromkeys(keys, value=None)`	**Class method**: new dict with every key mapped to the **same** `value`	O(n)
`d.keys()` / `d.values()` / `d.items()`	Live **view** objects	O(1) to create

Plus `len(d)`, `k in d`, `del d[k]`, `reversed(d)` (3.8+), `d | other`, `d |= other` (3.9+).

```
info = {"name": "John", "age": 28, "city": "New York"}
print(info.get("name"), info.get("phone"))
info.update({"age": 29, "phone": "123-456-7890"})       # overwrite + add
info.update(zip_code="10001")                            # keyword form
print(info)
info.clear()
print(info)

new = dict.fromkeys(["a", "b", "c"], 1)
print(new)
print(new.pop("b"), new, new.items())
shared = dict.fromkeys(["x", "y"], [])                   # ONE list shared by all keys!
shared["x"].append(1)
print("fromkeys trap:", shared)
d = {"first": 1, "second": 2, "third": 3}
print(d.popitem(), d)                                    # LIFO: last inserted
```

Output:

```
John None
{'name': 'John', 'age': 29, 'city': 'New York', 'phone': '123-456-7890', 'zip_code': '10001'}
{}
{'a': 1, 'b': 1, 'c': 1}
1 {'a': 1, 'c': 1} dict_items([('a', 1), ('c', 1)])
fromkeys trap: {'x': [1], 'y': [1]}
('third', 3) {'first': 1, 'second': 2}
```

The `fromkeys` trap is the dict cousin of `[[0]*3]*3`: every key points at **one** list. Use `{k: [] for k in keys}` instead.

`get` vs `setdefault` vs `update` vs `|` – which one?

Want to…	Use	Changes `d`?
Read with a fallback	`d.get(k, default)`	No
Read, inserting a default if missing (e.g. grouping)	`d.setdefault(k, [])`	Only if missing
Merge another mapping into `d`	`d.update(other)` or `d |= other`	Yes
Get a **new** merged dict	`d | other` or `{**d, **other}`	No
Count / group many items	`Counter` / `defaultdict` (6.9)	–

```
words = ["apple", "avocado", "banana", "blueberry", "cherry"]
by_letter = {}
for w in words:
    by_letter.setdefault(w[0], []).append(w)            # insert [] only if missing
print(by_letter)
counts = {}
for ch in "hello":
    counts[ch] = counts.get(ch, 0) + 1                  # get with default for counting
print(counts)
```

Output:

```
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

Note that `setdefault`'s default is **evaluated every call** even when the key exists (`[]` is built and thrown away) – harmless for `[]`, costly for expensive defaults. `defaultdict` avoids that.

View objects

`keys()`, `values()` and `items()` return **views**: lightweight windows onto the dict that reflect later changes. Key views (and item views, if values are hashable) also support set operations.

```
stock = {"apple": 3, "pear": 0}
k, v, it = stock.keys(), stock.values(), stock.items()
stock["fig"] = 7
print(k, v, it, sep="\n")
print(k & {"apple", "kiwi"}, sorted(k - {"pear"}))      # keys views support set ops
print(sorted(stock, key=stock.get, reverse=True))       # keys sorted by value
print(max(stock.items(), key=lambda kv: kv[1]))
```

Output:

```
dict_keys(['apple', 'pear', 'fig'])
dict_values([3, 0, 7])
dict_items([('apple', 3), ('pear', 0), ('fig', 7)])
{'apple'} ['apple', 'fig']
['fig', 'apple', 'pear']
('fig', 7)
```

Common mistakes

- `dict.fromkeys(keys, [])` sharing one mutable value.
- `d.update(k=v)` when `k` is a variable – the keyword form uses the literal name `"k"`. Use `d[k] = v`.
- Assuming `popitem()` removes a random item – since 3.7 it's always the last inserted.

In the interview

> **30-second answer:** "`get` reads with a default, `setdefault` reads-or-inserts, `update` / `|=` merge in place, `|` returns a new merged dict with the right side winning. `pop` removes by key, `popitem` removes the last inserted. `keys/values/items` are live views, and key views support set operations."

Self-check

1. `{"a": 1}.setdefault("b", 5)` — return value and resulting dict? → `5`; `{'a': 1, 'b': 5}`.
2. What does `popitem()` remove? → The most recently inserted pair.
3. Keys common to two dicts? → `d1.keys() & d2.keys()`.

TL;DR

`get` (read), `setdefault` (read-or-insert), `update` / `|=` (merge in place), `|` (merge new), `pop` / `popitem`, views are live and set-like. Avoid `fromkeys` with mutable values.

6.5 Dict and set comprehensions

What it is

The comprehension syntax from Chapter 5 also builds dicts and sets:

```
{key_expr: value_expr for item in iterable if condition}   # dict
{expr for item in iterable if condition}                   # set
```

How it works

```
words = ["apple", "banana", "cherry"]
print({w: len(w) for w in words})
prices = {"apple": 1.2, "banana": 0.5, "cherry": 3.0}
print({k: v for k, v in prices.items() if v >= 1})
print({v: k for k, v in {"a": 1, "b": 2}.items()})    # invert
print(sorted({c for c in "mississippi"}))              # set comprehension
```

Output:

```
{'apple': 5, 'banana': 6, 'cherry': 6}
{'apple': 1.2, 'cherry': 3.0}
{1: 'a', 2: 'b'}
['i', 'm', 'p', 's']
```

- Filtering a dict: iterate `.items()` and keep pairs that pass the test.
- **Inverting** a dict swaps keys and values — if two keys share a value, the later one wins (values must also be hashable).
- `{}` is an **empty dict**, not an empty set. Write `set()`.

Common mistakes

- Duplicate keys in a dict comprehension silently overwrite earlier ones.
- Using a comprehension when `dict(zip(keys, values))` is simpler.

Self-check

1. `{x % 3 for x in range(10)}`? → `{0, 1, 2}`.
2. Build `{name: len(name)}` for a list `names`? → `{n: len(n) for n in names}`.

TL;DR

`{k: v for ...}` for dicts, `{x for ...}` for sets; `{}` alone is a dict.

6.6 Sets — unique, unordered collections

What it is

A **set** is an unordered, mutable collection of **unique, hashable** items: `{"apple", "banana"}` or `set(iterable)`. Duplicates are dropped automatically. There's no indexing (`s[0]` is a `TypeError`) because there's no position.

Analogy: a bag of distinct marbles. You can ask "is the red one in the bag?" instantly, add or take out marbles, but they aren't in any order.

Why it matters

- **De-duplication** in one step: `set(emails)`.
- **Fast membership** (O(1) average, 6.2): "have I seen this before?" in crawlers, graph searches, validation.
- **Set algebra** answers questions like "which users are in both groups?" in one operator (6.7).

How it works — define, add, update, remove, copy, loop

```
fruits = {"apple", "banana", "cherry", "apple"}
colors = set(["red", "blue", "red", "green"])
print(sorted(fruits), sorted(colors), len(fruits))   # sorted() only for stable printing
empty_set, empty_dict = set(), {}
print(type(empty_set).__name__, type(empty_dict).__name__)
print(set("hello"), set(range(3)))                    # str order varies run to run

nums = {1, 2, 3}
nums.add(4); nums.add(2)                  # adding an existing item: no effect
nums.update([5, 6], (7,))                 # any iterables
print(nums)
nums.remove(1)
nums.discard(60)                          # absent: silently ignored
try:
    nums.remove(60)
except KeyError as e:
    print("KeyError:", e)
backup = nums.copy()
nums.clear()
print(nums, backup)
print("pop() ->", backup.pop(), backup)    # removes an ARBITRARY element
```

Output:

```
['apple', 'banana', 'cherry'] ['blue', 'green', 'red'] 3
set dict
{'l', 'h', 'e', 'o'} {0, 1, 2}          <- letter order varies from run to run
{1, 2, 3, 4, 5, 6, 7}
KeyError: 60
set() {2, 3, 4, 5, 6, 7}
pop() -> 2 {3, 4, 5, 6, 7}
```

- `add(x)` inserts one item; `update(*iterables)` inserts many.
- `remove(x)` raises `KeyError` if `x` is missing; `discard(x)` doesn't — use it when absence is fine.
- `pop()` removes and returns an **arbitrary** element (whatever the table yields first) — don't rely on which.
- `copy()` gives an independent set (its items are immutable anyway, so shallow = full here).
- Small integers often *appear* sorted because they hash to themselves; that's a coincidence of the implementation, not a guarantee.

De-duplicating while keeping order – a set loses order, so use a dict:

```
items = ["b", "a", "b", "c", "a"]
print(list(dict.fromkeys(items)))       # ['b', 'a', 'c']
```

Common mistakes

- `{}` for an empty set (it's a dict).
- `set("hello")` when you meant `{"hello"}` – the first splits into characters.
- Putting lists or dicts in a set → `TypeError: unhashable type`.
- Relying on set order for output – sort it.

Self-check

1. `len({1, 2, 2, 3})`? → 3.
2. `remove` vs `discard`? → `remove` raises `KeyError` if absent; `discard` doesn't.
3. How to dedupe a list but keep first-seen order? → `list(dict.fromkeys(lst))`.

TL;DR

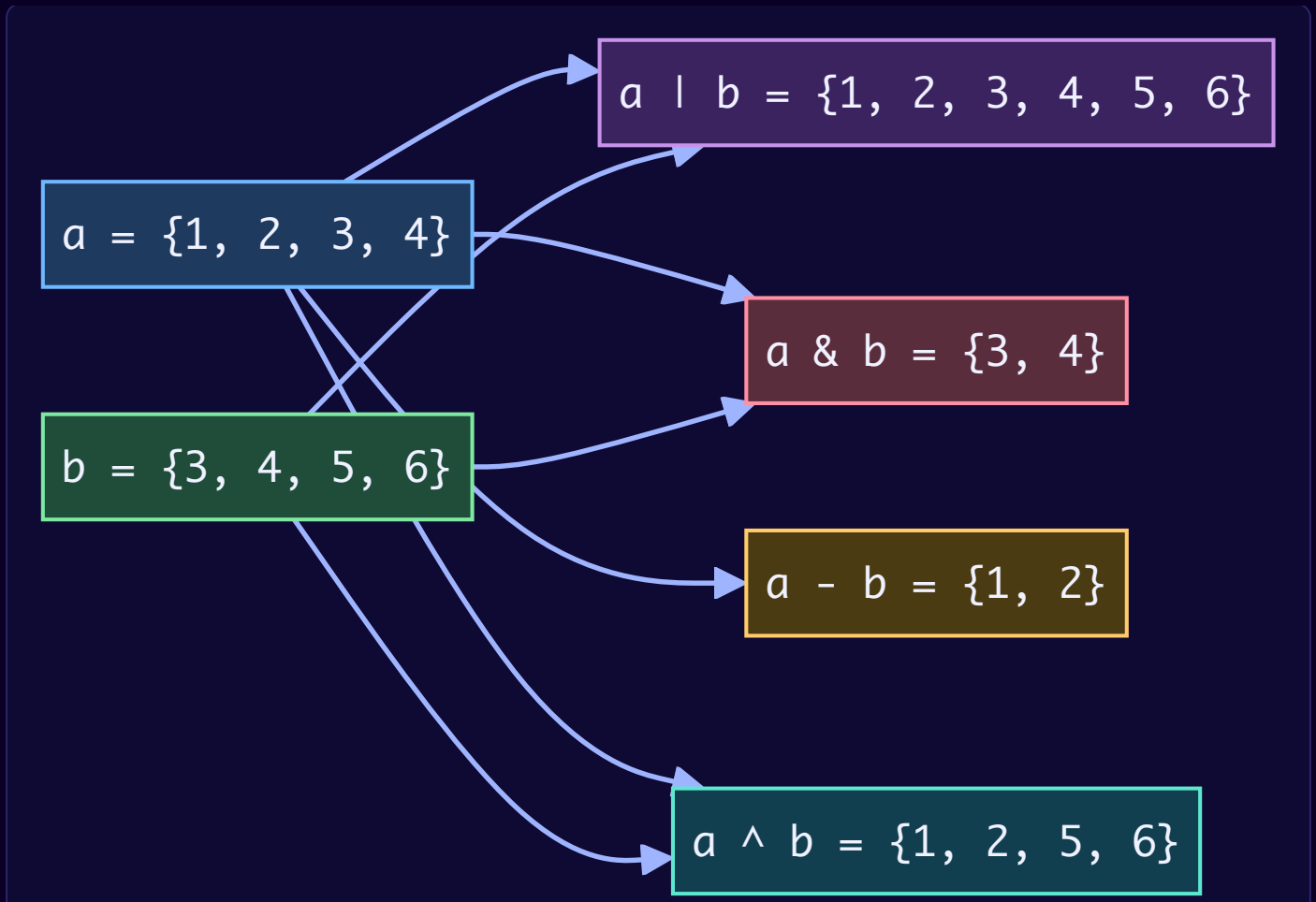
Unique, unordered, hashable items; `add`/`update`, `remove` (raises) / `discard` (silent); `set()` for empty; O(1) membership.

6.7 Set operations

What it is

Mathematical set algebra, available both as **operators** (both sides must be sets/frozensets) and as **methods** (accept any iterable).

Operation	Operator	Method	Result
Union – in either	`a | b`	`a.union(b, ...)`	Everything from both
Intersection – in both	`a & b`	`a.intersection(b, ...)`	Common items
Difference – in `a` not `b`	`a - b`	`a.difference(b, ...)`	`a` minus `b`
Symmetric difference – in exactly one	`a ^ b`	`a.symmetric_difference(b)`	Union minus intersection
Subset	`a <= b`	`a.issubset(b)`	Every item of `a` is in `b`
Proper subset	`a < b`	–	Subset and not equal
Superset	`a >= b`	`a.issuperset(b)`	Every item of `b` is in `a`
Disjoint	–	`a.isdisjoint(b)`	No common items




```
a, b = {1, 2, 3, 4}, {3, 4, 5, 6}
print("union        a | b :", a | b, a.union(b))
print("intersection a & b :", a & b, a.intersection(b))
print("difference   a - b :", a - b, a.difference(b), "  b - a:", b - a)
print("sym. diff    a ^ b :", a ^ b, a.symmetric_difference(b))
print("method accepts any iterable:", a.union([9, 10]), " operator needs sets")
try:
    a | [9]
except TypeError as e:
    print("TypeError:", e)
print("multi-arg:", a.intersection(b, {4, 99}))

small, big, other = {1, 2}, {1, 2, 3}, {7}
print(small <= big, small.issubset(big), small < big, big >= small, big.issuperset(small))
print(small.isdisjoint(other), small < small, small <= small)
```

Output:

```
union        a | b : {1, 2, 3, 4, 5, 6} {1, 2, 3, 4, 5, 6}
intersection a & b : {3, 4} {3, 4}
difference   a - b : {1, 2} {1, 2}   b - a: {5, 6}
sym. diff    a ^ b : {1, 2, 5, 6} {1, 2, 5, 6}
method accepts any iterable: {1, 2, 3, 4, 9, 10}  operator needs sets
TypeError: unsupported operand type(s) for |: 'set' and 'list'
multi-arg: {4}
True True True True True
True False True
```

Difference is **not symmetric**: `a - b = {1, 2}` but `b - a = {5, 6}`. A set is a subset of itself (`<=`) but not a proper subset (`<`).

Trade-offs

Operators are shorter and make you convert inputs explicitly (catching type mistakes); methods are more flexible (any iterable, several arguments at once).

Real-world examples

- Permissions: `required <= user_permissions`.
- Diffing: `new_ids - old_ids` (added), `old_ids - new_ids` (removed) when syncing records.
- Recommendations: "people who liked X also liked" via intersections; Jaccard similarity = `len(a & b) / len(a | b)`, used in near-duplicate detection.

In the interview

> **30-second answer:** "`|` union, `&` intersection, `-` difference, `^` symmetric difference, `<=` / `>=` subset/superset, `isdisjoint`. Operators need sets; methods accept any iterable. Intersection costs O(min(len a, len b)) because it iterates the smaller set and probes the larger."

Self-check

1. `{1, 2, 3} ^ {2, 3, 4}`? → `{1, 4}`.
2. `{1, 2} < {1, 2}`? → `False`.
3. Users in group A but not B? → `A - B`.

TL;DR

`| & - ^` plus subset/superset/disjoint; methods take any iterable; difference is one-directional.

6.8 Set methods (complete table) and `frozenset`

All set methods

Method	What it does	Operator form	Returns
`add(x)`	Add one element (no effect if present)	–	`None`
`update(*its)`	Add all elements from iterables	`|=`	`None`
`remove(x)`	Remove `x`; `KeyError` if absent	–	`None`
`discard(x)`	Remove `x` if present	–	`None`
`pop()`	Remove and return an arbitrary element; `KeyError` if empty	–	element
`clear()`	Remove everything	–	`None`
`copy()`	Shallow copy	–	new set
`union(*its)`	All elements	`|`	new set
`intersection(*its)`	Common elements	`&`	new set
`intersection_update(*its)`	Keep only common elements	`&=`	`None`
`difference(*its)`	Elements not in the others	`-`	new set
`difference_update(*its)`	Remove elements found in the others	`-=`	`None`
`symmetric_difference(it)`	In exactly one	`^`	new set
`symmetric_difference_update(it)`	Update to "in exactly one"	`^=`	`None`
`issubset(it)`	All my elements are in `it`	`<=`	bool
`issuperset(it)`	All of `it`'s elements are in me	`>=`	bool
`isdisjoint(it)`	No common elements	–	bool

The `_update` methods change the set **in place** and return `None`; the plain ones return a **new** set.

```
s = {1, 2, 3, 4}
s.intersection_update({2, 3, 9});         print("&= :", s)
s.difference_update({3});                 print("-= :", s)
s.symmetric_difference_update({2, 8});    print("^= :", s)
s |= {100};                               print("|= :", s)
```

Output:

```
&= : {2, 3}
-= : {2}
^= : {8}
|= : {8, 100}
```

Each `_update` method has an augmented-operator twin (`&=`, `-=`, `^=`, `|=`); the operator forms require sets on both sides, the methods accept any iterables.

`frozenset` — the immutable set

What it is

A **frozenset** is a set that can't be changed after creation. It has all the non-mutating methods (`union`, `issubset`, …) and none of the mutating ones (`add`, `remove`, `update`, …).

Why it matters

Because it's immutable, it's **hashable**: it can be a dict key or an element of another set — something a normal set can never be.

```
fs = frozenset([1, 2, 3])
try:
    fs.add(4)
except AttributeError as e:
    print("AttributeError:", e)
groups = {frozenset({"alice", "bob"}): "team A"}   # usable as a dict key
print(groups[frozenset({"bob", "alice"})])
print(fs | {9}, type(fs | {9}).__name__)            # result type follows the left operand
```

Output:

```
AttributeError: 'frozenset' object has no attribute 'add'
team A
frozenset({1, 2, 3, 9}) frozenset
```

The lookup works with the members in a different order because sets compare (and hash) by contents, not order — handy for keys like "unordered pair of users".

Common mistakes

- Expecting `s.union(t)` to modify `s` (it returns a new set) — or expecting `s.update(t)` to return the result (it returns `None`).
- Using `pop()` as if it returned the smallest element.

Self-check

1. Which returns `None`: `intersection` or `intersection_update`? → `intersection_update`.
2. Can a set contain another set? → Only a `frozenset`.
3. Key for an undirected edge `{u, v}`? → `frozenset((u, v))`.

TL;DR

Plain methods return new sets, `_update` methods mutate; `frozenset` = immutable, hashable set for keys and nested sets.

6.9 The `collections` module

The standard library's `collections` module supplies specialised containers that fix the most common "plain dict/list isn't quite right" situations.

`Counter` — counting hashable things

What it is: a dict subclass mapping item → count. Missing items count as `0` (no `KeyError`).

```
words = "the cat and the hat and the bat".split()
c = Counter(words)
print(c)
print(c.most_common(2), c["the"], c["dog"])          # missing key -> 0, no KeyError
c.update(["cat", "cat"]); c.subtract(["the"])
print(c.most_common(3))
a, b = Counter("aabbb"), Counter("abccc")
print(a + b, a - b, a & b, a | b, sep=" | ")
print(Counter("mississippi").total())                # 3.10+
```

Output:

```
Counter({'the': 3, 'and': 2, 'cat': 1, 'hat': 1, 'bat': 1})
[('the', 3), ('and', 2)] 3 0
[('cat', 3), ('the', 2), ('and', 2)]
Counter({'b': 4, 'a': 3, 'c': 3}) | Counter({'b': 2, 'a': 1}) | Counter({'a': 1, 'b': 1}) | Counter({'b': 3,
'c': 3, 'a': 2})
11
```

- `most_common(n)` — top-n pairs (ties keep first-seen order).
- `update` **adds** counts (unlike `dict.update`, which replaces); `subtract` subtracts and may leave zero or negative counts.
- `+` / `-` add/subtract and **drop** results ≤ 0; `&` takes the minimum per key, `|` the maximum.
- `total()` sums all counts (3.10+).

Real-world: word frequencies, log-level tallies, top-k error messages, the "bag of words" step in text classification.

`defaultdict` — dict with automatic defaults

What it is: a dict subclass that calls a **factory** (`list`, `int`, `set`, any zero-argument callable) to create a value the first time a missing key is **accessed with** `[]`.

```
groups = defaultdict(list)
for kind, name in [("fruit", "apple"), ("veg", "kale"), ("fruit", "fig")]:
    groups[kind].append(name)                        # missing key -> list() created
print(dict(groups))
counts = defaultdict(int)
for ch in "banana":
    counts[ch] += 1
print(dict(counts))
print("veg" in groups, "meat" in groups, groups.get("meat"))   # get/in don't create keys
_ = groups["meat"]                                               # [] access DOES
print("after groups['meat']:", dict(groups))
tree = lambda: defaultdict(tree)                                 # autovivifying tree
t = tree(); t["a"]["b"]["c"] = 1
```

Output:

```
{'fruit': ['apple', 'fig'], 'veg': ['kale']}
{'b': 1, 'a': 3, 'n': 2}
True False None
after groups['meat']: {'fruit': ['apple', 'fig'], 'veg': ['kale'], 'meat': []}
```

Watch out: **just reading** `d[missing]` inserts an empty value — a common source of mysterious extra keys. Use `in` or `.get()` for read-only checks.

Real-world: adjacency lists for graphs (`defaultdict(list)`), inverted indexes (`defaultdict(set)` word → document ids), grouping rows by a column.

`OrderedDict` — a dict with reordering tools

What it is: the pre-3.7 way to get an ordered dict. Plain dicts are now ordered too, so `OrderedDict` matters only for its extras:

- `move_to_end(key, last=True)` — O(1) move to the end (or front with `last=False`).
- `popitem(last=False)` — pop from the **front** (FIFO); plain `dict.popitem()` only pops the end.
- **Order-sensitive equality**: two `OrderedDict`s are equal only if the order matches too.

```
od = OrderedDict.fromkeys("abcde")
od.move_to_end("b")                  # to the end
od.move_to_end("e", last=False)      # to the front
print(list(od))                      # ['e', 'a', 'c', 'd', 'b']
print(od.popitem(last=False))        # ('e', None)
print(OrderedDict(a=1, b=2) == OrderedDict(b=2, a=1), dict(a=1, b=2) == dict(b=2, a=1))  # False True
```

The classic use — an LRU (least-recently-used) cache:

```
class LRUCache:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return None
        self.data.move_to_end(key)           # mark as most recently used
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)    # evict least recently used

cache = LRUCache(2)
cache.put("a", 1); cache.put("b", 2); cache.get("a"); cache.put("c", 3)
print("LRU keeps:", list(cache.data))        # LRU keeps: ['a', 'c']
```

Reading `"a"` made it recent, so adding `"c"` evicted `"b"`. For caching function results, prefer the built-in `@functools.lru_cache`.

`deque` — double-ended queue

What it is: a sequence with **O(1)** `append`/`pop` at **both** ends (a list is O(n) at the front). Implemented as a doubly-linked list of fixed-size blocks, so indexing in the middle is O(n).

```
q = deque([1, 2, 3])
q.append(4); q.appendleft(0)
print("after append/appendleft:", q)
left, right = q.popleft(), q.pop()
print("popleft ->", left, "| pop ->", right, "| now", q)
q.rotate(1); print("rotate(1):", q)
last3 = deque(maxlen=3)
for x in range(6):
    last3.append(x)                  # old items fall off the other end
print("maxlen=3:", last3)
```

Output:

```
after append/appendleft: deque([0, 1, 2, 3, 4])
popleft -> 0 | pop -> 4 | now deque([1, 2, 3])
rotate(1): deque([3, 1, 2])
maxlen=3: deque([3, 4, 5], maxlen=3)
```

- `maxlen` gives a bounded buffer — perfect for "last N lines/events" (like `tail -n`).
- `append` and `popleft` are thread-safe in CPython, which is why deques back simple producer/consumer queues (`queue.Queue` adds blocking and locking on top).

Real-world: BFS queues in graph search, sliding-window algorithms, undo history with a cap, rate limiters that keep recent timestamps.

`ChainMap` — layered lookups

What it is: a view over several mappings searched **in order**; the first one containing the key wins. Nothing is copied.

```python
defaults = {"color": "red", "user": "guest", "debug": False}
env = {"user": "admin"}
cli = {"debug": True}
cfg = ChainMap(cli, env, defaults)    # first mapping wins
print(cfg["color"], cfg["user"], cfg["debug"])
cfg["color"] = "blue"                 # writes go to the FIRST mapping only
print(cli, defaults["color"])
child = cfg.new_child({"user": "tmp"})
print(child["user"], cfg["user"])
```

Output:

```
red admin True
{'debug': True, 'color': 'blue'} red
tmp admin
```

Real-world: configuration precedence (command-line > environment > defaults), nested scopes in interpreters/template engines (`new_child()` pushes a scope). The standard library's `string.Template.substitute` uses a `ChainMap` internally to combine a mapping with keyword arguments.

`namedtuple` — tuple with field names

Covered fully in Chapter 5 (5.13): `Card = namedtuple("Card", "rank suit")` → `Card("A", "spades").rank == "A"`, with `_fields`, `_asdict()`, `_replace()`.

```
A Card(rank='10', suit='hearts') ('rank', 'suit')
```

Common mistakes (collections)

- Using `Counter.update` expecting replacement semantics.
- Accidentally creating keys in a `defaultdict` by reading.
- Using `list.pop(0)` instead of `deque.popleft()` for queues.
- Using `OrderedDict` just for ordering on Python 3.7+ — a plain dict already does it.

In the interview

30-second answer: "`Counter` counts and gives `most_common`; `defaultdict` auto-creates values for grouping; `deque` has O(1) appends and pops at both ends — use it for queues and bounded buffers; `OrderedDict` adds `move_to_end` and FIFO `popitem`, which makes an LRU cache easy; `ChainMap` layers mappings for config precedence; `namedtuple` gives tuples field names."

- *Q: Implement an LRU cache?* A: `OrderedDict` with `move_to_end` on access and `popitem(last=False)` on overflow – both O(1).
- *Q: Why deque for BFS?* A: `popleft()` is O(1); `list.pop(0)` is O(n).
- *Q: `Counter` + vs `update`?* A: `+` returns a new Counter and drops non-positive counts; `update` adds in place and keeps them.

Self-check

1. `Counter("aab")["z"]`? → `0`.
2. Build a graph's adjacency lists without key checks? → `defaultdict(list)`.
3. Keep only the last 100 log lines? → `deque(maxlen=100)`.
4. Which mapping receives writes in a `ChainMap`? → The first one.

TL;DR

Counter = counts; defaultdict = auto-values; OrderedDict = reordering + FIFO pop (LRU); deque = fast both ends + maxlen; ChainMap = layered lookup; namedtuple = named tuple fields.

6.10 `heapq` – priority queues on a plain list

What it is

A **binary min-heap** is a list arranged so that `heap[k] <= heap[2k+1]` and `heap[k] <= heap[2k+2]` for every `k`. The smallest item is always at `heap[0]`. The `heapq` module provides functions that keep a normal list in heap order.

Why it matters

Whenever you repeatedly need "the smallest (or most urgent) item next" – task schedulers, Dijkstra's shortest path, merging sorted streams, top-k – a heap does it in O(log n) per operation instead of re-sorting.

How it works

Function	Does	Cost
`heapify(list)`	Rearrange a list into a heap in place	O(n)
`heappush(h, x)`	Add `x` (sift up)	O(log n)
`heappop(h)`	Remove and return the smallest (sift down)	O(log n)
`h[0]`	Peek at the smallest	O(1)
`heappushpop(h, x)` / `heapreplace(h, x)`	Push then pop / pop then push, efficiently	O(log n)
`nsmallest(k, it)` / `nlargest(k, it)`	Top-k without full sort	O(n log k)
`merge(*sorted_its)`	Lazily merge sorted inputs	O(n log m)
`heapify_max`, `heappush_max`, `heappop_max`, …	Max-heap versions (new in Python 3.14)	as above

```python
def heapify_neg(xs):                    # helper: store negatives -> behaves as a max-heap
    h = [-x for x in xs]
    heapq.heapify(h)
    return h

h = [5, 1, 8, 3, 2]
heapq.heapify(h)                        # O(n), in place; h[0] is the smallest
print("heapified:", h, "min:", h[0])
heapq.heappush(h, 0)
print("pops:", [heapq.heappop(h) for _ in range(3)], "left:", h)
print(heapq.nsmallest(2, [7, 3, 9, 1]), heapq.nlargest(2, [7, 3, 9, 1]))
print("max-heap trick (negate):", -heapq.heappop(heapify_neg([3, 9, 4])))
```

Output:

```
heapified: [1, 2, 8, 3, 5] min: 1
pops: [0, 1, 2] left: [3, 5, 8]
[1, 3] [9, 7]
max-heap trick (negate): 9
```

The heapified list `[1, 2, 8, 3, 5]` is **not sorted** – only the heap property holds (1 ≤ 2, 1 ≤ 8, 2 ≤ 3, 2 ≤ 5). Before 3.14, the standard trick for a max-heap is to push negated numbers.

Priority queue with tie-breaking:

```python
tasks, counter = [], 0
for prio, name in [(2, "write docs"), (1, "fix prod bug"), (3, "refactor"), (1, "reply to CEO")]:
    heapq.heappush(tasks, (prio, counter, name))   # (priority, sequence, payload)
    counter += 1
while tasks:
    prio, _, name = heapq.heappop(tasks)
    print(f"  p{prio}: {name}")
```

Output:

```
  p1: fix prod bug
  p1: reply to CEO
  p2: write docs
  p3: refactor
```

Tuples compare element by element, so priority decides first; the increasing counter breaks ties in insertion order **and** stops Python from ever comparing the payloads (which might not be comparable).

`heapq.merge([1, 4, 9], [2, 3, 10], [5])` → `[1, 2, 3, 4, 5, 9, 10]` – the core of external sorting (merging sorted files too big for memory).

Trade-offs

Need	Use
Smallest item repeatedly, with inserts	`heapq`
Just the top k once	`heapq.nlargest/nsmallest` (or `sorted(...)[:k]` if k is close to n)
Only min or max once	`min()` / `max()` – O(n)
Thread-safe blocking priority queue	`queue.PriorityQueue` (uses heapq inside)
Sorted order at all times with lookups	`bisect` on a list, or a sorted-container library

Real-world examples

`asyncio`'s event loop keeps scheduled callbacks (`call_later`) in a heap; the `sched` module and `queue.PriorityQueue` are built on `heapq`; Dijkstra/A* implementations in Python almost always use it.

Self-check

1. Where's the smallest item in a heap? → `h[0]`.
2. Cost of `heapify` vs n pushes? → O(n) vs O(n log n).
3. Why add a counter to heap entries? → Stable tie-breaking and to avoid comparing payloads.

TL;DR

`heapify` O(n), push/pop O(log n), `h[0]` is the min; tuples `(priority, count, item)`; `nlargest` for top-k; max-heap = negate (or 3.14's `_max` functions).

6.11 `bisect` — binary search on sorted lists

What it is

`bisect` finds where a value belongs in an **already sorted** list using binary search — O(log n) comparisons.

How it works

- `bisect_left(a, x)` — first index where `x` could be inserted keeping order (before any equal items).
- `bisect_right(a, x)` / `bisect(a, x)` — index **after** any equal items.
- `insort(a, x)` (= `insort_right`) — insert keeping order: O(log n) search **+ O(n)** list shift.
- Since Python 3.10, all functions accept `key=`.

Worked example on `[10, 20, 20, 30, 40]` searching for 20: the middle element (index 2) is 20; `bisect_left` keeps searching left and lands at index 1; `bisect_right` lands after the last 20 at index 3.

```
scores = [10, 20, 20, 30, 40]
print(bisect.bisect_left(scores, 20), bisect.bisect_right(scores, 20), bisect.bisect(scores, 25))
bisect.insort(scores, 25)
print(scores)

def contains(sorted_list, x):
    i = bisect.bisect_left(sorted_list, x)
    return i < len(sorted_list) and sorted_list[i] == x
print(contains(scores, 30), contains(scores, 31))

def grade(score, cutoffs=(60, 70, 80, 90), letters="FDCBA"):
    return letters[bisect.bisect(cutoffs, score)]
print([grade(s) for s in (33, 60, 77, 89, 90, 100)])

people = [("Ann", 25), ("Bob", 31), ("Cid", 40)]           # sorted by age
i = bisect.bisect_left(people, 30, key=lambda p: p[1])      # key= is 3.10+
print("first person aged >= 30:", people[i])
```

Output:

```
1 3 3
[10, 20, 20, 25, 30, 40]
True False
['F', 'D', 'C', 'B', 'A', 'A']
first person aged >= 30: ('Bob', 31)
```

The grading function is the classic example from the Python docs: `bisect` turns a table of cutoffs into a lookup — a score of exactly 60 lands to the **right** of the 60 cutoff, i.e. "D".

Trade-offs

- Searching is O(log n), but `insort` is O(n) because the list must shift. For many inserts plus lookups on big data, a set/dict (if you don't need order) or a balanced tree/ `sortedcontainers.SortedList` (third-party) is better.
- The list **must** be sorted by the same key you search with — bisect doesn't check.

Real-world examples

`random.choices` uses `bisect` on cumulative weights to pick weighted samples; time-series code uses it to find the first event after a timestamp; percentile/bucket lookups (tax brackets, grade cutoffs) are one call.

Self-check

1. `bisect_left([1, 2, 2, 3], 2)`? → `1`. `bisect_right`? → `3`.
2. Cost of `insort` on a list of n items? → O(n) (O(log n) search + O(n) shift).
3. What must be true of the list? → Sorted (by the same key).

TL;DR

Binary search on sorted lists: `bisect_left/right` O(log n), `insort` O(n), `key=` since 3.10.

6.12 When to use what — the decision table

You need…	Use	Why	Key costs
Ordered items, index access, append at end	`list`	Dynamic array	index O(1), append O(1)*, `in` O(n)
Fixed record / dict key / return several values	`tuple` / `namedtuple`	Immutable, hashable, light	index O(1)
Look up values by key	`dict`	Hash table	get/set/del O(1) avg
Fast membership / uniqueness	`set`	Hash table	`in` O(1) avg
A set as a key or inside a set	`frozenset`	Immutable, hashable	same as set
Count things / top-k frequencies	`Counter`	Counting dict with `most_common`	update O(k)
Group into lists/sets without key checks	`defaultdict(list/set)`	Auto-created values	O(1)
Queue / stack at both ends / last N items	`deque` (`maxlen=N`)	O(1) at both ends	append/pop O(1), middle index O(n)
LRU cache / reorder keys / FIFO pop	`OrderedDict` (or `functools.lru_cache`)	`move_to_end` , `popitem(last=False)`	O(1)
Layered config / scopes	`ChainMap`	Search mappings in order, no copying	lookup O(number of maps)
Repeatedly take the smallest/most urgent	`heapq`	Binary heap	push/pop O(log n), peek O(1)
Top-k from a large iterable	`heapq.nlargest/nsmallest`	Avoids a full sort	O(n log k)
Search/insert in a sorted list	`bisect`	Binary search	search O(log n), insert O(n)
Dedupe but keep order	`dict.fromkeys(items)`	Dicts keep insertion order	O(n)
Compact numeric storage / fast maths	`array.array` / NumPy	Chapter 5	–

(* amortised)

In the interview — putting it together

Prompt: "Find the 3 most frequent words in a log file and the first line where each appears."

Answer sketch: stream lines; `Counter` for counts; a dict with `setdefault(word, line_no)` for first occurrence; `counter.most_common(3)` at the end. Everything is O(total words) plus O(u log 3) for the top-k — no nested loops, no `list.index` .

Chapter TL;DR

Topic	Remember
Dicts	Key → value, O(1) avg, insertion-ordered since 3.7; `[]` raises `KeyError`, `get` defaults; `|` merges (3.9+); `copy` is shallow; don't resize while iterating.
Hashing	hash → slot → compare; ≤ 2/3 full; equal ⇒ same hash; keys must be immutable/hashable; string hashes salted per run.
Nested dicts	Chain keys; guard with `get(..., {})`; map 1:1 to JSON (keys become strings).
Dict methods	`get`, `setdefault`, `update` / `|=`, `pop`, `popitem` (LIFO), `fromkeys` (shared value trap), live views with set ops.
Comprehensions	`{k: v for ...}`, `{x for ...}`; `{}` is a dict.
Sets	Unique, unordered, hashable items; `add` / `update`; `remove` raises, `discard` doesn't; `set()` for empty.
Set operations	`| & - ^`, `<= < >= >`, `isdisjoint`; methods accept iterables; `_update` versions mutate.
frozenset	Immutable, hashable set — keys and sets of sets.
collections	Counter, defaultdict, OrderedDict (LRU), deque (queues, maxlen), ChainMap (layers), namedtuple.
heapq	Min-heap on a list; push/pop O(log n); `(prio, count, item)`; `nlargest`; max-heap functions in 3.14.
bisect	Binary search on sorted lists; `insort` is O(n); `key=` since 3.10.

Chapter 7 — Functions

A **function** is a named, reusable block of code that takes some inputs, does one job, and (usually) hands back a result. Almost everything you write in Python beyond a ten-line script is built out of functions, and many "advanced" features — decorators, callbacks, `sorted(key=...)`, caching — are just functions being passed around like ordinary values.

This chapter goes from `def` to closures, decorators and recursion. Every script lives in `code/` and was run with Python 3.14 (all code is 3.12-compatible); the output shown is real.

What you'll learn

#	Topic	Script
7.1	Defining and calling functions, `return`, `None`, `pass`	`01_defining_and_arguments.py`
7.2	Every kind of argument: positional, keyword, default, `*args`, `**kwargs`, positional-only `/`, keyword-only `*`, unpacking, the mutable-default trap, returning several values	`01_defining_and_arguments.py`
7.3	Scope and namespaces: LEGB, `global`, `nonlocal`	`02_scope_and_closures.py`
7.4	Closures and the late-binding trap	`02_scope_and_closures.py`
7.5	First-class and higher-order functions, `lambda`, `map` / `filter`, `sorted` / `min` / `max` with `key`	`03_first_class_and_functools.py`
7.6	Decorators and `functools`: `wraps`, `partial`, `reduce`, `lru_cache`, `cache`	`03_first_class_and_functools.py`
7.7	Recursion, the call stack, the recursion limit, memoization	`04_recursion.py`
7.8	Docstrings and type annotations	`05_docstrings_annotations.py`

7.1 Defining and calling functions

What it is

You **define** a function with the `def` keyword, a name, a pair of parentheses holding the **parameters** (the input names), and a colon. The indented lines below form the **body**. Defining a function does not run it — it only creates a function object and binds it to the name. You **call** (run) it by writing its name followed by parentheses, passing **arguments** (the actual values) inside.

Analogy: a function definition is a recipe card; a call is actually cooking the dish. You can cook from the same card as many times as you like, with different ingredients each time.

> **Parameter vs argument.** The *parameter* is the variable name in the definition (`def cube(x)` → `x`). The *argument* is the value supplied at the call (`cube(3)` → `3`). People mix them up constantly; interviewers notice.

Why it matters

Functions give you four things at once:

1. **Modularity** — a big program is split into small pieces, each with one clear job, which can be written, read and debugged on its own.
2. **Reuse** — write the logic once, call it from anywhere (or from other programs via modules, Chapter 8). Fixing a bug fixes it everywhere.
3. **Scope** — names created inside a function are private to it, so different parts of a program don't trample each other's variables (7.3).

4. **Readability and teamwork** — a well-named function (`send_invoice(order)`) documents intent, and different people can own different functions.

How it works

1. Python executes the `def` statement: it compiles the body, creates a *function object*, evaluates any default values (important later), and binds the object to the name.
2. At a call, Python creates a fresh **frame** (a workspace holding the local variables), binds each parameter to its argument, and runs the body.
3. `return value` ends the call immediately and gives `value` back to the caller. If the body finishes without `return` — or with a bare `return` — the call gives back `None`.
4. The frame is thrown away (unless a closure keeps some of it alive, 7.4).

`pass` is a statement that does nothing. Python forbids an empty block, so `pass` is the placeholder when you want to define a function (or class, or `if` branch) now and fill it in later. `...` (Ellipsis) is often used the same way in stubs.

```
def say_hi():
    """A function with no parameters and no return statement."""
    print("hi from say_hi()")

def cube(x):
    """Return x cubed."""
    return x ** 3

def todo():
    pass  # placeholder body - keeps the syntax valid

say_hi()
print("say_hi() returned:", say_hi())
print("cube(3) =", cube(3))
print("todo() returned:", todo())
```

Output:

```
hi from say_hi()
hi from say_hi()
say_hi() returned: None
cube(3) = 27
todo() returned: None
```

Notice that `say_hi()` printed twice: once for the plain call and once inside the `print(...)` — and then `None` appeared, because a function that only *prints* still *returns* `None`.

Trade-offs: print vs return

	`print(...)` inside the function	`return ...`
Result usable by caller	No — it goes to the screen	Yes — can be stored, tested, combined
Testable	Only by capturing stdout	Directly (`assert cube(3) == 27`)
Good for	Debugging, CLI output at the edges	Almost all real logic

Common mistakes

- **Forgetting the parentheses**: `result = cube` stores the *function*, not a result. `result = cube(3)` calls it.
- **Printing instead of returning**, then wondering why `x = f()` is `None`.
- **Code after `return`** in the same block never runs.
- **Calling before defining** at module level: the `def` must have executed before the call runs (inside other functions this is fine, because the lookup happens at call time).

Real-world examples

Every library you'll touch is a collection of functions: `len`, `print`, `open`, `json.dumps`, `requests.get`, `numpy.mean`. Web frameworks such as Flask and FastAPI map each URL to a plain Python function ("view" or "endpoint") that returns the response.

In the interview

> **30-second answer:** "A function is defined with `def`; defining creates a function object bound to a name, and calling it creates a new frame with its own local variables. `return` sends a value back; with no `return` the result is `None`. Functions give modularity, reuse and a private scope."

- *What does a function without `return` return?* → `None`.
- *Are functions objects?* → Yes; they have attributes (`__name__`, `__doc__`, `__defaults__`) and can be passed around (7.5).
- *What is `pass` for?* → A no-op placeholder where syntax needs a statement.

Self-check

1. What does `x = print("a")` store in `x`? → `None`.
2. Parameter or argument: the `3` in `cube(3)`? → Argument.
3. When does the body of a function run? → Only when it's called, not when `def` runs.
4. Why prefer `return` over `print` in logic? → Results can be reused and tested.

TL;DR

`def name(params):` + indented body. Defining ≠ running. `return` gives a value back; otherwise you get `None`. `pass` fills an empty body.

7.2 Function arguments – every kind

What it is

Python gives you several ways to feed values into a function. At the **call site** you can pass arguments **positionally** (matched by order) or **by keyword** (matched by name). In the **definition** you can add **defaults**, collect extras with `*args` / `**kwargs`, and force a style with `/` (positional-only) and `*` (keyword-only).

The full shape of a parameter list, in the only order Python allows:

```
def f(pos_only, /, pos_or_kw, *args, kw_only, **kwargs): ...
#     ^^^^^^^^     ^^^^^^^^^  ^^^^^  ^^^^^^^  ^^^^^^^^
#     before /     normal     extra  after *  extra keywords
#                             positionals     (dict)
#                             (tuple)
```

Kind	Declared as	Called as	Notes
Positional-or-keyword	`def f(a, b)`	`f(1, 2)` or `f(b=2, a=1)`	The default kind
Default	`def f(a, b=10)`	`f(1)` or `f(1, 5)`	Params with defaults must come after those without (in the positional part)
Variable positional	`def f(*args)`	`f(1, 2, 3)`	`args` is a **tuple**
Variable keyword	`def f(**kwargs)`	`f(x=1, y=2)`	`kwargs` is a **dict** (insertion-ordered)
Positional-only	`def f(a, b, /)`	`f(1, 2)` only	Python 3.8+ (PEP 570)
Keyword-only	`def f(*, a, b)` or after `*args`	`f(a=1, b=2)` only	Can be required (no default)

Why it matters

Good argument design makes calls **self-documenting and hard to misuse**. `connect("db", 5432, True, False)` is a riddle; `connect("db", port=5432, ssl=True, retry=False)` explains itself. Defaults keep simple calls simple, `*args` / `**kwargs` let you write flexible wrappers, and `/` / `*` let a library author keep freedom to rename parameters later.

How it works — binding arguments to parameters

When you call a function, Python:

1. Fills parameters left-to-right from the **positional arguments**. Extra positionals go into `*args` (or raise `TypeError` if there is no `*args`).
2. Matches each **keyword argument** to the parameter with that name. Unknown names go into `**kwargs` (or raise `TypeError`).
3. A parameter filled twice (by position *and* by keyword) → `TypeError: got multiple values`.
4. Any still-empty parameter takes its **default**; if it has none → `TypeError: missing ... required argument`.

At the call site, positional arguments must come before keyword arguments: `f(1, b=2)` is fine, `f(a=1, 2)` is a `SyntaxError`.

Positional and keyword arguments

Positional arguments are matched purely by order, so every required one must be supplied and in the right order — swap two and you silently get wrong results (the function can't tell a first name from a last name). **Keyword** arguments name the target parameter, so order no longer matters and the call reads clearly; they shine when a function has many parameters or several optional ones.

Default arguments

A **default** makes a parameter optional: if the caller leaves it out, the default is used; if the caller supplies it, it overrides the default.

```
def book_flight(origin, destination, seat="economy", *, meal="standard"):
    """origin/destination: positional-or-keyword; seat: default; meal: keyword-only."""
    return f"{origin}->{destination} [{seat}, meal={meal}]"

print(book_flight("SFO", "JFK"))                                  # defaults used
print(book_flight(destination="JFK", origin="SFO", seat="aisle"))  # any order by name
print(book_flight("SFO", "JFK", meal="vegan"))                    # keyword-only
book_flight("SFO")                          # missing arg
book_flight("SFO", "JFK", "aisle", "vegan")  # meal given positionally
book_flight("SFO", origin="LAX")            # same arg twice
```

Output (errors caught by a small helper in the script):

```
SFO->JFK [economy, meal=standard]
SFO->JFK [aisle, meal=standard]
SFO->JFK [economy, meal=vegan]
  missing arg: TypeError: book_flight() missing 1 required positional argument: 'destination'
  meal given positionally: TypeError: book_flight() takes from 2 to 3 positional arguments but 4 were given
  same arg twice: TypeError: book_flight() got multiple values for argument 'origin'
```

Positional-only (`/`) and keyword-only (`*`)

- Everything **before** `/` must be passed by position. Their names become an internal detail, so the author may rename them later without breaking anyone, and a name like `value` can't collide with a `**kwargs` key. Many built-ins work this way: `len(obj, /)`, `abs(x, /)`, `divmod(x, y, /)` – try `len(obj=[1])` and you get a `TypeError`.
- Everything **after** `*` (or after `*args`) must be passed by name. Use it for flags and options whose meaning isn't obvious from position: `sorted(iterable, /, *, key=None, reverse=False)` – you can't write `sorted(xs, None, True)`. Keyword-only parameters may still be *required* (no default).

```python
def clamp(value, low, high, /):
    return max(low, min(value, high))

print("clamp(15, 0, 10) =", clamp(15, 0, 10))
clamp(value=15, low=0, high=10)
```

```
clamp(15, 0, 10) = 10
  clamp(value=15, ...): TypeError: clamp() got some positional-only arguments passed as keyword arguments:
'value, low, high'
```

Variable-length arguments: `*args` and `**kwargs`

`*args` gathers any **extra positional** arguments into a tuple; `**kwargs` gathers any **extra keyword** arguments into a dict. The names `args` / `kwargs` are only convention – the `*` and `**` do the work.

```python
def total(*numbers, start=0):
    print("  numbers is a", type(numbers).__name__, numbers)
    return sum(numbers, start)

def make_tag(tag, /, text, **attrs):
    attr_str = "".join(f' {k}="{v}"' for k, v in attrs.items())
    return f"<{tag}{attr_str}>{text}</{tag}>"

print("total(1, 2, 3) =", total(1, 2, 3))
print("total() =", total())
print(make_tag("a", "docs", href="https://docs.python.org", target="_blank"))
```

```
  numbers is a tuple (1, 2, 3)
total(1, 2, 3) = 6
  numbers is a tuple ()
total() = 0
<a href="https://docs.python.org" target="_blank">docs</a>
```

A typical `**kwargs` use is a settings function: `configure(host="db1", port=5432, timeout="30s")` receives `{"host": "db1", "port": 5432, "timeout": "30s"}` and can loop over `.items()`.

Unpacking at the call site

The same stars work in reverse when *calling*: `*iterable` spreads a sequence into positional arguments, `**mapping` spreads a dict into keyword arguments.

```
nums = [4, 5, 6]
opts = {"seat": "window", "meal": "kosher"}
print("total(*nums) =", total(*nums))
print(book_flight("SFO", "JFK", **opts))
```

```
  numbers is a tuple (4, 5, 6)
total(*nums) = 15
SFO->JFK [window, meal=kosher]
```

Returning several values

A function returns exactly **one** object — but that object can be a tuple, and `return a, b, c` builds one. The caller then *unpacks* it.

```
def min_max_mean(values):
    return min(values), max(values), sum(values) / len(values)

result = min_max_mean([3, 9, 6])
print("raw result:", result, type(result).__name__)
lo, hi, avg = result
```

```
raw result: (3, 9, 6.0) tuple
lo=3 hi=9 avg=6.0
```

For more than ~3 values, return a `dataclass` or `typing.NamedTuple` instead, so callers use names (`stats.mean`) not positions.

The mutable-default pitfall

Default values are evaluated once — when `def` runs — not on each call. If the default is a mutable object (list, dict, set), every call that uses the default shares *the same object*, and changes pile up.

```
def append_bad(item, bucket=[]):          # BUG: default evaluated once
    bucket.append(item)
    return bucket

def append_good(item, bucket=None):       # FIX: None sentinel
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

print(append_bad("a"), append_bad("b"))
print("default is now:", append_bad.__defaults__)
print(append_good("a"), append_good("b"))
```

```
['a', 'b'] ['a', 'b']
default is now: (['a', 'b'],)
['a'] ['b']
```

Both `append_bad` calls returned the *same* list, and `__defaults__` shows the stored default has been mutated. The fix is the `None` **sentinel** idiom: default to `None` and create a new object inside. (The same "evaluated once" rule means `def log(msg, when=datetime.now())` freezes the time at definition.) `dataclasses` refuses a bare mutable default for exactly this reason and asks for `field(default_factory=list)`.

Trade-offs

Choice	Benefit	Cost
Positional	Short calls	Order-dependent, easy to swap by mistake
Keyword	Readable, order-free	Longer calls; parameter names become public API
Keyword-only (`*`)	Forces clarity for flags/options	Slightly more typing for callers
Positional-only (`/`)	Freedom to rename; no clash with `**kwargs`	Callers can't use names for clarity
`*args` / `**kwargs`	Flexible wrappers, forwarding	Hides the real signature from readers, IDEs and type checkers

When not to use it

- Don't reach for `**kwargs` when you know the options – explicit keyword parameters give errors on typos (`timout=5`) and IDE help; `**kwargs` silently swallows the typo.
- Don't pile on boolean positional flags (`f(x, True, False)`) – make them keyword-only.

Common mistakes

- Mutable default (`=[]` , `={}`) – use `None` .
- Putting a non-default parameter after a default one: `def f(a=1, b):` → `SyntaxError` .
- Positional after keyword at the call: `f(a=1, 2)` → `SyntaxError` .
- Forgetting that `*args` is a **tuple** (immutable) – convert with `list(args)` if you need to modify it.
- Mixing up `return a, b` (one tuple) with "two return values".

Real-world examples

- `print(*objects, sep=" ", end="\n", file=None, flush=False)` – `*objects` plus keyword-only options.
- `requests.get(url, params=None, **kwargs)` forwards options such as `timeout=` and `headers=` through `**kwargs` .
- Matplotlib plotting calls accept `**kwargs` and pass them on to the underlying line/patch objects (`color=` , `linewidth=`).
- `dataclasses.field(default_factory=list)` exists because of the mutable-default trap.

In the interview

> **30-second answer:** "Arguments bind positionally by order or by keyword by name. Defaults make parameters optional but are evaluated once at definition time, so never use a mutable default – use `None` . `*args` collects extra positionals into a tuple, `**kwargs` extra keywords into a dict. Parameters before `/` are positional-only, after `*` keyword-only."

- *Why is `def f(x=[])` dangerous?* → The list is created once and shared by every call that uses the default; mutations accumulate.
- *Why would a library use positional-only parameters?* → To keep parameter names private (can be renamed without breaking callers) and to allow a `**kwargs` key with the same name.
- *How do you pass a list as separate arguments?* → `f(*my_list)` ; a dict as keywords → `f(**my_dict)` .

Self-check

1. Inside `def f(*a, **k)` , what are the types of `a` and `k` ? → `tuple` and `dict` .
2. `def f(a, /, b, *, c)` – which calls work: `f(1, 2, 3)` , `f(1, b=2, c=3)` , `f(a=1, b=2, c=3)` ? → Only `f(1, b=2, c=3)` (and `f(1, 2, c=3)`).
3. What does `return 1, 2` return? → The tuple `(1, 2)` .
4. When is a default value computed? → Once, when the `def` statement runs.

TL;DR

Positional = by order, keyword = by name. Defaults are evaluated **once** → use `None` for mutables. `*args` → tuple, `**kwargs` → dict; `*` / `**` at the call site unpack. `/` = positional-only before it, `*` = keyword-only after it. Multiple return values = one tuple.

7.3 Scope and namespaces — LEGB, `global`, `nonlocal`

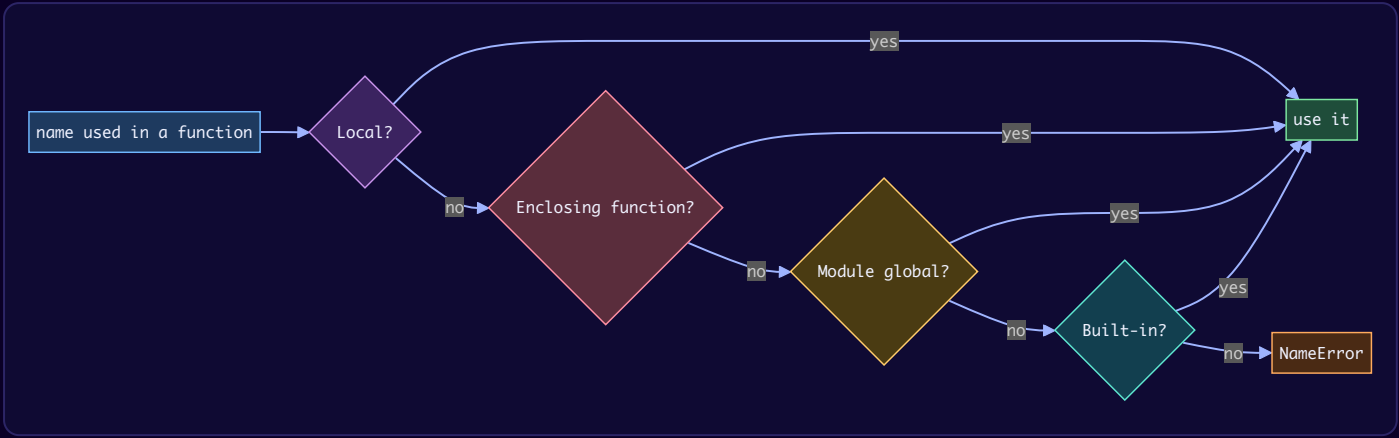
What it is

A **namespace** is a mapping from names to objects — literally a dictionary in most cases (`globals()` shows the module's one). Each name inside one namespace is unique, but the same name can exist in different namespaces without clashing. The **scope** of a name is the region of code where that name can be used directly.

Python resolves a bare name using the **LEGB rule** — it searches four scopes in order and stops at the first hit:

Letter	Scope	Where it comes from	Lifetime
L	Local	Names assigned inside the current function (including parameters)	Created when the call starts, gone when it ends
E	Enclosing	Locals of any *outer* function(s), for nested functions	As long as something references them (see closures)
G	Global	Names assigned at the top level of the **module** (file)	Until the program ends (or module is unloaded)
B	Built-in	`len`, `print`, `ValueError` … from the `builtins` module	Whole program

Not found anywhere → `NameError`.



Analogy: looking for scissors — first your own desk (local), then the room you're in (enclosing), then the whole house (global), then the hardware store (built-ins).

Why it matters

Scope is what makes functions safe to combine: a `total` inside one function can't clobber a `total` inside another. Misunderstanding it causes one of Python's most confusing errors, `UnboundLocalError`.

How it works — the key rule

Whether a name is local is decided at compile time, for the whole function body: if the function *assigns* to a name anywhere (`=`, `+=`, `for x in`, `import x`, `def x`), that name is local everywhere in the function — unless declared `global` or `nonlocal`. Merely *reading* a name doesn't make it local, so reading a global works without any declaration.

1. `global name` → assignments to `name` in this function rebind the **module-level** variable.

2. `nonlocal name` → assignments rebind the variable in the **nearest enclosing function** scope (it must already exist there; module globals don't count). Added in Python 3.
3. Only functions (and classes, comprehensions, lambdas) create scopes. `if`, `for`, `while` and `with` blocks do not – a variable assigned inside an `if` is visible after it.

```
level = "global"                    # G

def outer():
    level = "enclosing"             # E for inner()
    def inner():
        level = "local"             # L
        print("  inner sees:", level)
    inner()
    print("  outer sees:", level)

def reads_global():
    print("  reads_global sees:", level)   # no assignment here -> global

counter = 0

def broken_increment():
    counter += 1                    # assignment makes counter LOCAL -> error

def global_increment():
    global counter
    counter += 1
```

```
== LEGB lookup ==
  inner sees: local
  outer sees: enclosing
  reads_global sees: global
  B (built-in) len is builtins.len: True

== assigning without global ==
  UnboundLocalError: cannot access local variable 'counter' where it is not associated with a value

== global ==
  counter = 2
```

`counter += 1` means `counter = counter + 1`. Because of the assignment, `counter` is local; the right-hand side then reads the local before it has a value → `UnboundLocalError` (a subclass of `NameError`).

`nonlocal` does the same job one level in – see `make_counter` in 7.4.

Trade-offs

Approach to shared state	Good	Bad
Pass in / return values	Explicit, testable, thread-friendly	More parameters
`global`	Quick for scripts	Hidden coupling; any function can change it; hard to test
`nonlocal` / closure	State private to one function family	Can be less obvious than a class
A class with attributes	Clear home for state + behaviour	More code

When not to use it

Avoid `global` for anything except true module-level configuration or caches in small scripts. Mutating a global *object* (e.g. `CACHE[key] = v`) doesn't even need `global` – only rebinding the *name* does – but it is

still shared mutable state.

Common mistakes

- `UnboundLocalError` from `x += 1` on a global — add `global x`, or better, return the new value.
- **Shadowing built-ins**: naming a variable `list`, `dict`, `id`, `input`, `sum` or `type` hides the built-in in that scope. The script shows `list = [1, 2]` followed by `list('xy')` → `'list' object is not callable`.
- Expecting a class body to be an enclosing scope for its methods — it isn't; methods must use `self.x` or `ClassName.x`.
- Expecting the loop variable of a **comprehension** to leak — it doesn't (comprehensions have their own scope), while a plain `for` loop's variable does remain after the loop.

Real-world examples

- `logging.getLogger(__name__)` is conventionally stored as a module-level (global) `logger` and only *read* inside functions.
- Configuration constants (`MAX_RETRIES = 3`) live at module level in UPPER_CASE by PEP 8 convention: global, but treated as read-only.

In the interview

> **30-second answer:** "Python looks names up via LEGB: local, enclosing, global, built-in. Any assignment in a function makes that name local for the whole function, which is why `x += 1` on a global raises `UnboundLocalError`. `global` rebinds module-level names, `nonlocal` rebinds the nearest enclosing function's name. Blocks like `if` and `for` don't create scopes."

- *Do you need `global` to append to a global list?* → No — `.append` mutates, it doesn't rebind. You need it only for `name = ...`.
- *What's the difference between `global` and `nonlocal`?* → `global` targets module scope; `nonlocal` targets the nearest enclosing *function* scope and requires the name to exist there.
- *Is a variable defined inside an `if` visible after it?* → Yes, `if` / `for` don't create scopes.

Self-check

1. What does LEGB stand for? → Local, Enclosing, Global, Built-in.
2. Why does `def f(): print(x); x = 1` fail even if a global `x` exists? → The later assignment makes `x` local for the whole body, so the `print` reads an unassigned local.
3. Can `nonlocal` refer to a module-level variable? → No — `SyntaxError: no binding for nonlocal ... found`.
4. What error do you get when a name isn't in any scope? → `NameError`.

TL;DR

Lookup order is **L → E → G → B**. Assignment anywhere in a function ⇒ local everywhere in it. `global` for module names, `nonlocal` for outer-function names. Prefer passing and returning values over either.

7.4 Closures

What it is

A **closure** is an inner function that *remembers* variables from the enclosing function's scope, even after the outer function has returned. Those remembered variables are called **free variables**; Python stores them in "cells" attached to the function (`fn.__closure__`).

Analogy: a backpack. When the inner function leaves home (is returned), it takes a backpack with the variables it needs.

Why it matters

Closures let you build **functions from other functions** — configured functions (`make_multiplier(3)`), counters with private state, callbacks that carry context — without writing a class. Decorators (7.6) are built on them.

How it works

1. `make_multiplier(3)` runs; `factor = 3` lives in its local scope.
2. It defines `multiply`, which reads `factor` → the compiler marks `factor` as a free variable and stores it in a cell.
3. `make_multiplier` returns `multiply`. Its frame ends, but the cell survives because `multiply` references it.
4. Each call to `make_multiplier` creates a **new** cell, so `double` and `triple` are independent.
5. To *rebind* a captured variable (not just read it), the inner function needs `nonlocal`.

```
def make_counter():
    count = 0
    def step():
        nonlocal count              # rebind the enclosing variable
        count += 1
        return count
    return step

def make_multiplier(factor):
    def multiply(x):
        return x * factor           # 'factor' is captured (a free variable)
    return multiply

c1, c2 = make_counter(), make_counter()
print("  c1:", c1(), c1(), c1(), " c2:", c2())
double, triple = make_multiplier(2), make_multiplier(3)
print("  double(5) =", double(5), " triple(5) =", triple(5))
print("  free vars of double:", double.__code__.co_freevars,
      "cell value:", double.__closure__[0].cell_contents)
```

```
  c1: 1 2 3  c2: 1
  double(5) = 10  triple(5) = 15
  free vars of double: ('factor',) cell value: 2
```

The late-binding trap

Closures capture **variables, not values**. The value is looked up when the inner function *runs*.

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])         # [2, 2, 2]  - all see the final i
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])         # [0, 1, 2]  - default captures the current value
```

The `i=i` trick works *because* defaults are evaluated at definition time (7.2). `functools.partial` is a cleaner alternative.

Trade-offs

	Closure	Class
Code size	Tiny	Larger
Number of operations	Best for one (a callable)	Many methods
Inspectability	State hidden in cells	Attributes visible, easy to debug

When not to use it

When the "function" needs several operations (increment, reset, read), or other people must inspect/serialise its state, write a small class instead. Closures also can't be pickled by the standard `pickle` module.

Common mistakes

- Late binding in loops (above).
- Forgetting `nonlocal` and getting `UnboundLocalError` on `count += 1`.

Real-world examples

- Every decorator (`@functools.lru_cache`, Flask's `@app.route("/")`) returns a closure wrapping your function.
- Event callbacks in GUI toolkits (Tkinter `command=`) often use closures or `partial` to carry the context of which button was pressed.

In the interview

> **30-second answer:** "A closure is a nested function that keeps references to variables of its enclosing scope after that scope has returned. It captures variables, not values, so lambdas in a loop all see the last value unless you bind with a default argument or `partial`. `nonlocal` lets it rebind the captured variable."

- *Where are captured variables stored?* → In cell objects in `fn.__closure__`; names in `fn.__code__.co_freevars`.
- *How to fix the loop-lambda bug?* → `lambda i=i: ...` or `functools.partial(f, i)`.

Self-check

1. Do `c1` and `c2` from `make_counter` share state? → No, each call creates a new cell.
2. Why does `[lambda: i for i in range(3)]` give `[2, 2, 2]`? → `i` is looked up at call time, when it's 2.
3. What keyword lets a closure change a captured variable? → `nonlocal`.

TL;DR

Inner function + remembered outer variables = closure. Captures variables (late binding), use `nonlocal` to rebind. Foundation of decorators.

7.5 First-class and higher-order functions, `lambda`, `map` / `filter`, `key=`

What it is

Python functions are **first-class objects**: you can assign them to variables, store them in lists and dicts, pass them as arguments and return them from other functions. A **higher-order function** is one that takes a function as an argument or returns one (`map`, `sorted`, decorators, `make_multiplier`).

A `lambda` is an anonymous, single-expression function: `lambda x: x * 2` is the same as a `def` whose body is `return x * 2`, but it has no statements, no annotations and its name is just `<lambda>`.

Why it matters

It lets you pass **behaviour** as data: "sort by this", "retry this", "call this when done". It removes whole families of near-duplicate functions.

How it works

```
yell = shout                                # assign to a name
table = {"up": str.upper, "low": str.lower}  # store in a dict
print(apply_twice(shout, "hey"))             # pass as argument  -> HEY!!

nums = [5, -2, 8, -7, 3]
list(map(abs, nums))                  # [5, 2, 8, 7, 3]
list(filter(lambda n: n > 0, nums))   # [5, 8, 3]
[n * n for n in nums if n > 0]        # [25, 64, 9]
```

- `map(fn, iterable)` applies `fn` to each item; `filter(pred, iterable)` keeps items where `pred(item)` is truthy. Both return **lazy iterators** in Python 3 (the script prints `map`) — wrap in `list()` to see values, and they can be consumed only once.
- `sorted(iterable, key=fn, reverse=False)` returns a **new list**; `list.sort()` sorts in place and returns `None`. `min`, `max`, `heapq.nlargest` and `itertools.groupby` accept `key=` too.
- `key` is called **once per element**; the elements are then compared by their key. Python's sort is **stable**: equal keys keep their original order — which is how multi-level sorting works.
- `operator.itemgetter(1)`, `operator.attrgetter("age")` and `str.lower` are ready-made, faster alternatives to many lambdas.

```
people = [("Ana", 31), ("bob", 25), ("Cy", 31), ("dee", 19)]
sorted(people, key=lambda p: p[1])           # by age
sorted(people, key=lambda p: (-p[1], p[0]))  # age desc, then name
sorted(["bob", "Ana", "dee", "Cy"], key=str.lower)
max(people, key=operator.itemgetter(1))
```

```
  by age:            [('dee', 19), ('bob', 25), ('Ana', 31), ('Cy', 31)]
  by age desc, name: [('Ana', 31), ('Cy', 31), ('bob', 25), ('dee', 19)]
  case-insensitive:  ['Ana', 'bob', 'Cy', 'dee']
  itemgetter(1):     ('dee', 19)
  oldest:            ('Ana', 31)
```

Returning a **tuple** from `key` sorts by the first element, then the second, and so on; negating a number flips that level's direction.

Trade-offs

Tool	Best for	Watch out
Comprehension	Transform + filter in one readable line	Builds a whole list (use a generator expression `( … )` for laziness)
`map` / `filter` with a named function	`map(str.strip, lines)`	With a lambda it's usually less readable than a comprehension
`lambda`	Tiny one-off `key=` functions	No statements, name `<lambda>` in tracebacks, PEP 8 says don't assign one to a name — use `def`

When not to use it

Don't write `f = lambda x: x + 1` — use `def f(x): return x + 1` (better name in tracebacks, can have a docstring). Don't nest `map(lambda..., filter(lambda..., ...))` when a comprehension reads plainly.

Common mistakes

- `sorted_list = my_list.sort()` → `None`. Use `sorted()` or call `.sort()` alone.
- Consuming a `map` / `filter` iterator twice — the second pass is empty.
- `key=len()` (calling it) instead of `key=len` (passing it).

Real-world examples

- pandas: `df["name"].map(str.title)`, `df.sort_values(key=...)`, `df.apply(fn)`.
- `sorted(files, key=os.path.getmtime)` – "newest file" scripts.
- Callbacks: `threading.Thread(target=worker)`, `concurrent.futures` `executor.map(fetch, urls)`.

In the interview

> **30-second answer:** "Functions are objects, so they can be passed, stored and returned; a higher-order function takes or returns one. `lambda` makes a one-expression anonymous function, mostly for `key=`. `map` / `filter` return lazy iterators; comprehensions are usually preferred. `sorted` with `key` computes the key once per element and is stable, so tuple keys give multi-level sorts."

- *`sorted` vs `.sort()`?* → `sorted` takes any iterable and returns a new list; `.sort()` is a list method, in place, returns `None`.
- *Is Python's sort stable?* → Yes, guaranteed.
- *Sort by two fields, one descending?* → Tuple key with a negated number, or two stable sorts (secondary key first).

Self-check

1. What does `map(abs, [-1])` return? → A lazy `map` iterator, not a list.
2. How many times is `key` called when sorting 1,000 items? → 1,000 (once each).
3. Can a lambda contain an `if` statement? → No statements; a conditional *expression* (`a if c else b`) is fine.

TL;DR

Functions are values. Higher-order functions take/return functions. `lambda` for tiny keys. `map`/`filter` are lazy; comprehensions are clearer. `sorted(key=…)` – once per item, stable, tuple keys for multi-level.

7.6 Decorators and `functools`

What it is

A **decorator** is a higher-order function that takes a function and returns a replacement (usually a closure that calls the original plus extra behaviour). The `@` syntax is shorthand:

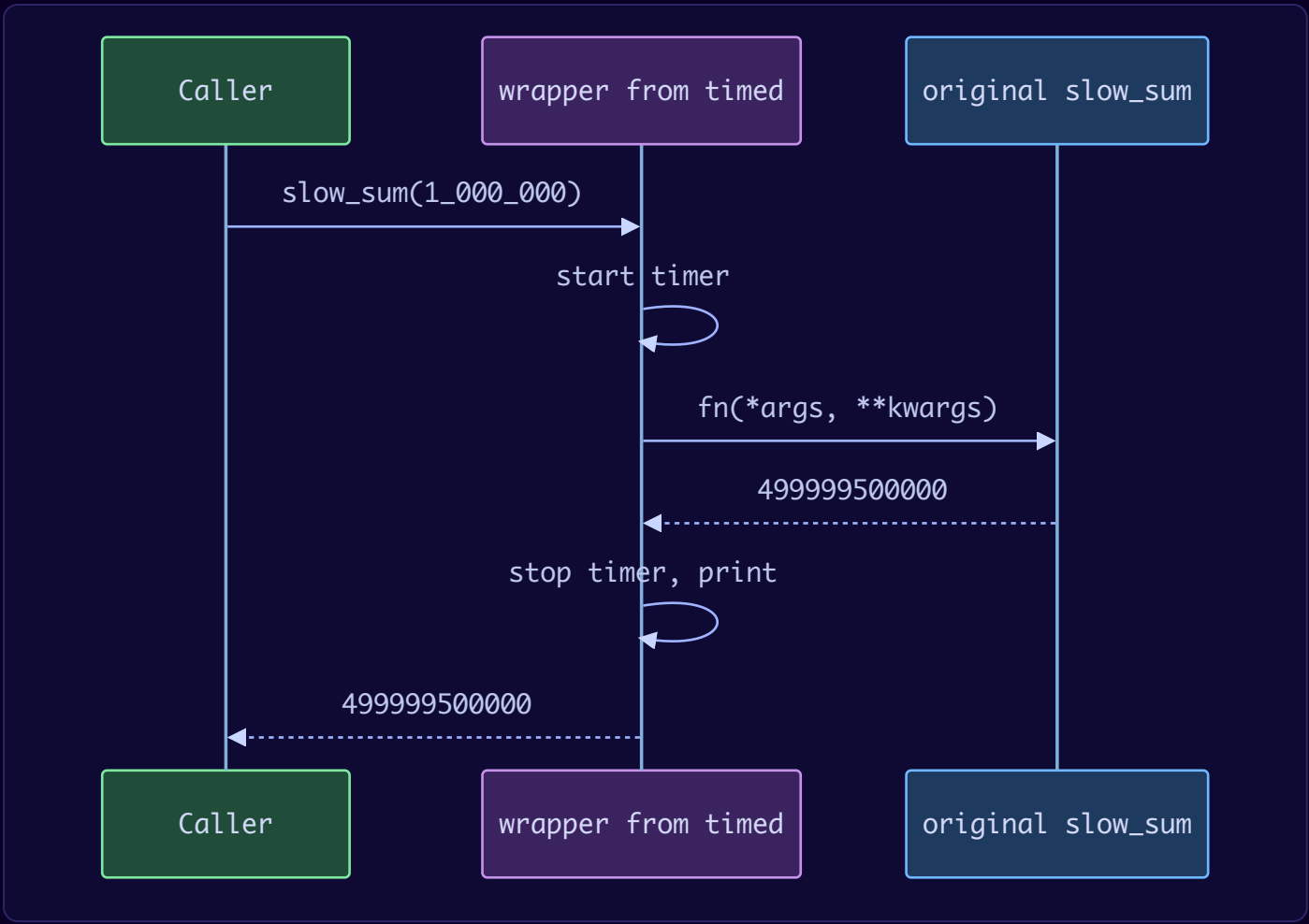
```
@timed
def slow_sum(n): ...
# is exactly
def slow_sum(n): ...
slow_sum = timed(slow_sum)
```

The standard-library module `functools` collects tools for working with functions: `wraps`, `partial`, `reduce`, `lru_cache`, `cache`, `cached_property`, `singledispatch`, `total_ordering`.

Why it matters

Decorators add cross-cutting behaviour – timing, logging, caching, auth checks, retries, route registration – without editing each function. `functools` gives you battle-tested versions so you don't write them yourself.

How it works




```python
def timed(fn):
    @functools.wraps(fn)                # copy __name__, __doc__, etc.
    def wrapper(*args, **kwargs):       # accept any signature
        t0 = time.perf_counter()
        result = fn(*args, **kwargs)    # call the original
        ms = (time.perf_counter() - t0) * 1000
        print(f"  [{fn.__name__} ran in {'under' if ms < 500 else 'over'} 500 ms]")
        return result                   # don't forget to return!
    return wrapper
```

```
  [slow_sum ran in under 500 ms]
  slow_sum(1_000_000) = 499999500000
  with wraps:    name = slow_sum | doc = Add 0..n-1 the slow way.
  without wraps: name = wrapper | doc = None
```

`functools.wraps` copies the original's `__name__`, `__doc__`, `__module__`, `__qualname__`, `__dict__` and sets `__wrapped__`. Without it, every decorated function would appear as `wrapper` in logs, `help()` and tracebacks.

`functools.partial(fn, *args, **kwargs)` "pre-fills" some arguments and returns a new callable:

```python
int_from_binary = functools.partial(int, base=2)
int_from_binary("1011")   # 11
```

`functools.reduce(fn, iterable[, initial])` folds a sequence into one value: `reduce(mul, [1,2,3,4,5])` computes ((((1·2)·3)·4)·5) = 120. Give an `initial` value or an empty input raises `TypeError`. For sums, products, max/min, `any` / `all` and joins, the dedicated built-ins (`sum`, `math.prod`, `max`, `"".join`) are clearer.

`functools.lru_cache(maxsize=128)` memoizes: it stores results keyed by the arguments and returns the stored result for repeat calls. When more than `maxsize` distinct keys are stored, the **least recently used** entry is evicted. `functools.cache` (3.9+) is `lru_cache(maxsize=None)` — unbounded, slightly faster.

```
@functools.lru_cache(maxsize=128)
def square_slowly(n):
    time.sleep(0.05)
    return n * n
for n in (4, 4, 5, 4):
    square_slowly(n)
print(square_slowly.cache_info())
```

```
  CacheInfo(hits=2, misses=2, maxsize=128, currsize=2)
  after cache_clear: CacheInfo(hits=0, misses=0, maxsize=128, currsize=0)
  unhashable argument -> unhashable type: 'list'
```

Calls with 4, 4, 5, 4 → misses on the first 4 and on 5, hits on the two repeats. Arguments must be **hashable** (lists fail).

Trade-offs

Tool	Win	Cost / caveat
`lru_cache(maxsize=N)`	Bounded memory	Bookkeeping overhead; stale results if data changes
`cache`	Fastest, simplest	Grows forever — a memory leak for unbounded inputs
`partial`	Picklable, introspectable (`.func`, `.args`)	Only pre-fills; can't reorder arguments
`reduce`	General fold	Often less readable than a loop or built-in

When not to use it

- Don't cache functions with side effects (sending email) or whose result depends on time, randomness or mutable global state.
- `lru_cache` on an **instance method** keys on `self` and keeps every instance alive for the cache's lifetime; prefer `functools.cached_property` for per-instance values, or a module-level cached function.

Common mistakes

- Forgetting `return result` in the wrapper → the decorated function returns `None`.
- Forgetting `@functools.wraps`.
- Writing `@timed()` for a decorator that takes no arguments (or vice-versa). A decorator *with* arguments is a function that returns a decorator — three levels of nesting.

Real-world examples

- Flask/FastAPI: `@app.get("/items")` registers the function as a route handler.
- pytest: `@pytest.fixture`, `@pytest.mark.parametrize(...)`.
- Standard library: `@property`, `@staticmethod`, `@classmethod`, `@dataclass`, `@contextlib.contextmanager`.
- The `re` module keeps an internal cache of compiled patterns so repeated `re.match(pattern, s)` doesn't recompile.

In the interview

30-second answer: "A decorator is a callable that takes a function and returns a replacement; `@d` above `def f` means `f = d(f)`. Inside you define a `wrapper(*args, **kwargs)` closure and use `functools.wraps` to

keep the original's name and docstring. `functools` also gives `partial` to pre-fill arguments, `reduce` to fold, and `lru_cache` / `cache` to memoize pure functions with hashable arguments."

- *What does `wraps` do?* → Copies metadata (`__name__`, `__doc__`, …) and sets `__wrapped__`.
- *How do you write a decorator that takes arguments, like `@retry(3)`?* → An outer function taking `3` returns the actual decorator, which returns the wrapper.
- *When is `lru_cache` wrong?* → Impure functions, unhashable args, unbounded key spaces with `maxsize=None`, instance methods.

Self-check

1. Rewrite `@d` / `def f` without the `@`. → `f = d(f)`.
2. What's the default `maxsize` of `lru_cache`? → 128.
3. `reduce(operator.add, [])` → ? → `TypeError` (no initial value).
4. Why `*args, **kwargs` in a wrapper? → So it can wrap any signature and forward everything.

TL;DR

Decorator = function in, function out; always `@wraps` and `return result`. `partial` pre-fills args, `reduce` folds, `lru_cache` / `cache` memoize pure functions with hashable arguments.

7.7 Recursion

What it is

A **recursive function** calls itself to solve a smaller version of the same problem. It's the programming form of *divide and conquer*: split the problem into smaller copies, solve each, combine the answers. Every recursive function needs:

1. A **base case** — an input small enough to answer directly, with no further call.
2. A **recursive case** — reduce the input and call yourself, making guaranteed progress toward the base case.

Analogy: Russian nesting dolls — open one, find a smaller one, repeat until the tiny solid doll (base case).

Why it matters

Recursion is the natural fit for **self-similar data and problems**: nested lists/JSON, folder trees, parse trees, tree and graph traversal, merge sort and quicksort, backtracking (permutations, sudoku). Interviews use it constantly.

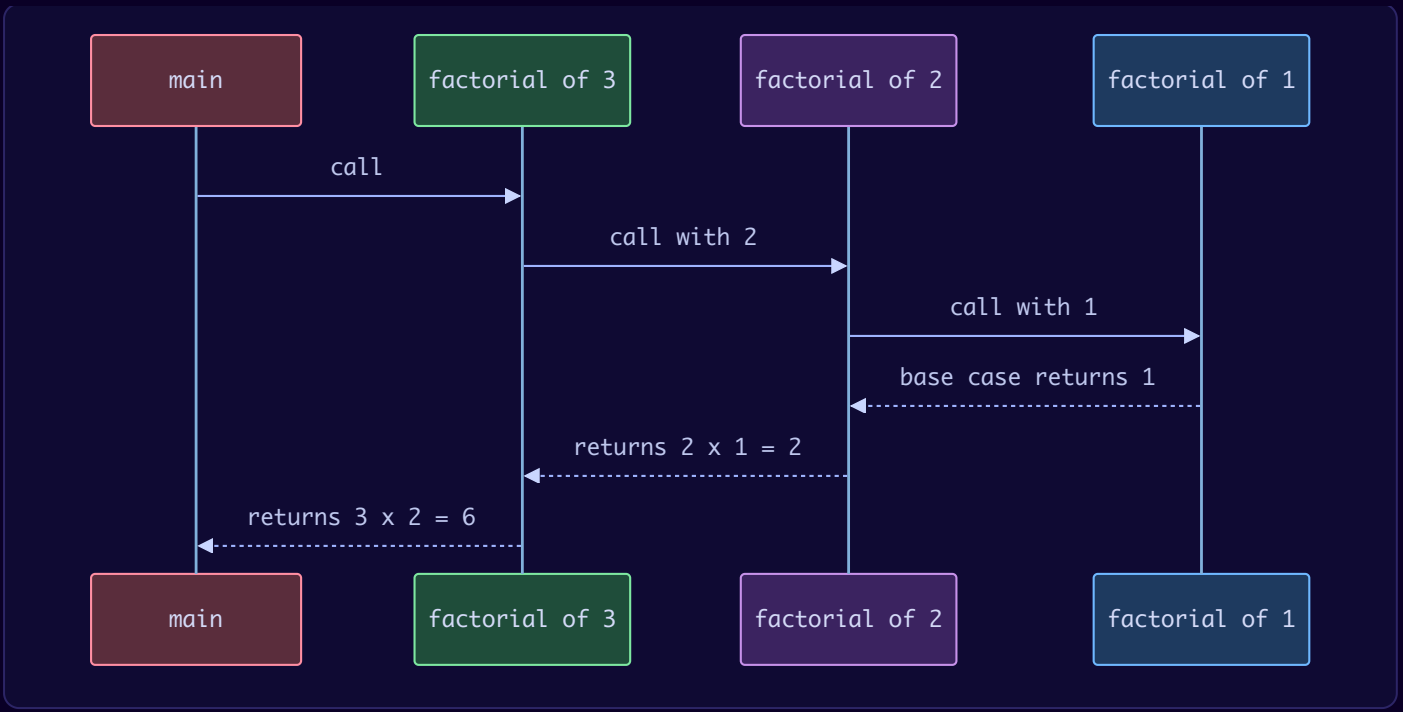
How it works — the call stack

Each call gets its own frame on the **call stack** (a last-in-first-out pile of active calls). Calls pile up until the base case, then results flow back as frames are popped. Factorial: $n! = n \times (n-1)!$, with $0! = 1! = 1$.

```python
def factorial(n):
    if n <= 1:                # base case (also handles 0! = 1)
        return 1
    return n * factorial(n - 1)
```

Trace of `factorial(3)` from the script, and the arithmetic for `factorial(5)`: $5 \times 4 \times 3 \times 2 \times 1 = 120$.

```
factorial(5) = 120 | factorial(0) = 1
factorial(3)
  factorial(2)
    factorial(1)
    -> 1
  -> 2
-> 6
```



The recursion limit

CPython has **no tail-call optimisation**, so every level costs a frame. To protect the interpreter from running out of stack, it caps the depth: `sys.getrecursionlimit()` is **1000** by default. Exceed it and you get `RecursionError` (a subclass of `RuntimeError`).

```
sys.getrecursionlimit() = 1000
depth_of(500) = 500
RecursionError: maximum recursion depth exceeded
```

`sys.setrecursionlimit(n)` can raise it, but deep recursion still costs memory and can crash the process if pushed too far — for depths in the thousands or more, **rewrite it as a loop** (with an explicit stack list for tree walks).

Memoization — fixing exponential recursion

Fibonacci: F(0)=0, F(1)=1, F(n)=F(n−1)+F(n−2), so F(7) = 13. The naive version recomputes the same values over and over: F(5) calls F(4) and F(3), F(4) calls F(3) *again*, and so on. The number of calls grows roughly like 1.6^n. **Memoization** — remembering results of earlier calls — makes each F(k) computed once, so the work becomes linear.

```
@functools.cache
def fib_memo(n):
    if n < 2:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)

def fib_iter(n):              # bottom-up: no recursion at all
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

```
fib(7) = 13
naive calls for n=10, 20, 25: 177 21891 242785
memoized: CacheInfo(hits=23, misses=26, maxsize=None, currsize=26)
fib_iter(90) = 2880067194370816120
```

For n = 25 the naive version makes 242,785 calls; the memoized one computes 26 distinct values (misses) and answers 23 repeats from the cache.

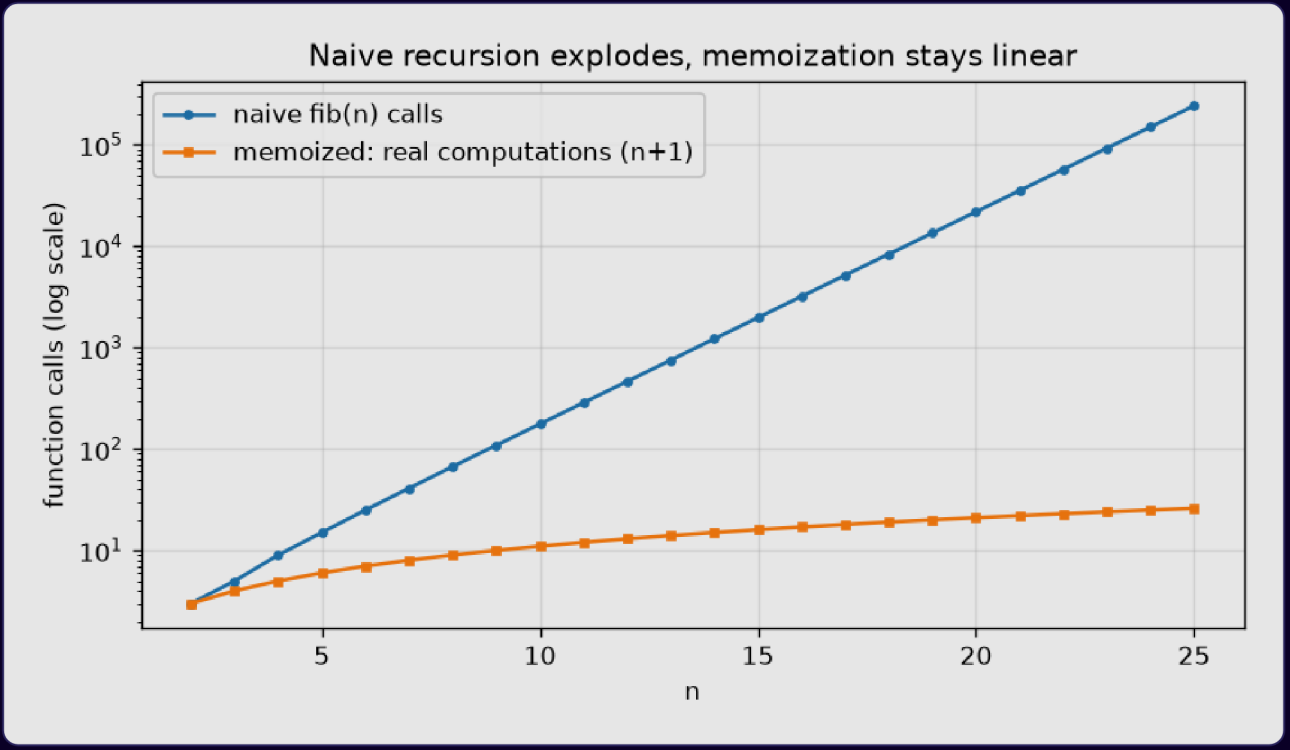


Figure — Function calls needed for fib(n): naive recursion grows exponentially (a straight line on a log axis), memoization grows linearly.

Recursion over nested data

```python
def flatten(nested):
    out = []
    for item in nested:
        if isinstance(item, list):
            out.extend(flatten(item))   # recursive case: a smaller list
        else:
            out.append(item)            # base case: a plain value
    return out

flatten([1, [2, [3, [4]], 5], [], 6])   # [1, 2, 3, 4, 5, 6]
```

The same shape walks directories (`os.walk` / `Path.iterdir`), JSON documents and trees.

Trade-offs

	Recursion	Iteration
Fits	Trees, nested data, divide and conquer, backtracking	Linear sequences, counting, very deep processes
Clarity	Mirrors the math/definition	Can need an explicit stack
Memory	One frame per level	Constant (or your own stack)
Depth limit	~1000 by default	None
Speed in CPython	Function-call overhead per level	Usually faster

When not to use it

Linear problems (sum a list, count down, walk a linked list of 100k nodes) – use a loop. Anything whose depth depends on untrusted input size.

Common mistakes

- **Missing or unreachable base case.** A factorial with base case `n == 1` recurses forever on `factorial(0)` (0 → −1 → −2 …) until `RecursionError`; `n <= 1` fixes it.
- Not shrinking the input (`f(n)` calling `f(n)`).
- Forgetting to `return` the recursive result (`factorial(n - 1)` without `return` gives `None`).
- Using naive recursion for overlapping sub-problems (Fibonacci, grid paths) — memoize.
- Mutable default accumulators in recursive helpers (`def walk(node, seen=set())`).

Real-world examples

- `os.walk`, `shutil.copytree`, `json` encoding of nested structures all process nested data (CPython's `json` encoder is recursive and can raise `RecursionError` for extremely deep input).
- `copy.deepcopy` recursively copies nested objects.
- Recursive-descent parsers (Python's own grammar tooling, many config/expression parsers).

In the interview

> **30-second answer:** "Recursion is a function calling itself on a smaller input until a base case. Each call uses a stack frame; CPython doesn't optimise tail calls and limits depth to about 1000, raising `RecursionError`. Overlapping sub-problems like Fibonacci are exponential unless memoized with `functools.cache`, which makes them linear. For very deep or linear problems, convert to a loop."

- *Complexity of naive vs memoized Fibonacci?* → $O(\varphi^n) \approx O(1.6^n)$ time vs $O(n)$ time and $O(n)$ space.
- *Does Python do tail-call optimisation?* → No, by design (keeps tracebacks complete).
- *How do you convert recursive DFS to iterative?* → Keep an explicit stack list: pop a node, process, push its children.

Self-check

1. What two parts does every recursive function need? → Base case and recursive case that progresses toward it.
2. What happens past the recursion limit? → `RecursionError`.
3. How many distinct values does memoized `fib(25)` compute? → 26 (F(0)…F(25)).
4. Why might `factorial(0)` crash with base case `n == 1`? → It never reaches 1.

TL;DR

Base case + shrinking recursive case. Stack frames pile up; limit ≈ 1000; no TCO. Memoize overlapping sub-problems (`@cache`), loop when deep.

7.8 Docstrings and annotations

What it is

- A **docstring** is a string literal placed as the **first statement** of a function, class or module. Python stores it in `__doc__`, and `help()`, IDEs and documentation generators (Sphinx) show it. Conventions are in PEP 257: a one-line summary, a blank line, then details.
- **Annotations** (type hints) attach expected types to parameters and the return value: `def f(x: int) -> str`. They are **not enforced** at runtime — Python stores them and moves on. Tools such as **mypy**, **pyright** and IDEs read them to catch bugs before you run the code; libraries such as **FastAPI** and **Pydantic** read them at runtime to validate data.

Why it matters

Docstrings explain *what and why*; annotations make the contract checkable. Together they make functions usable without reading their bodies — and let a type checker catch `average("abc")`-style mistakes in CI.

How it works

```
from collections.abc import Callable, Iterable

def average(values: Iterable[float], *, default: float | None = None) -> float | None:
    """Return the arithmetic mean of *values*.

    Args:
        values: numbers to average.
        default: returned when *values* is empty.

    Returns:
        The mean, or *default* for empty input.

    >>> average([1, 2, 3])
    2.0
    """
    items = list(values)
    if not items:
        return default
    return sum(items) / len(items)

def apply(fn: Callable[[int], int], x: int) -> int:
    return fn(x)

type Pair = tuple[int, int]          # 3.12+ type alias statement
def swap(p: Pair) -> Pair:
    return p[1], p[0]
```

```
first docstring line: Return the arithmetic mean of *values*.
signature: (values: collections.abc.Iterable[float], *, default: float | None = None) -> float | None
return annotation: float | None
average([1, 2, 3]) = 2.0
average([], default=0.0) = 0.0
swap(('x', 'y')) = ('y', 'x') <- str, not int, and no error
apply(lambda n: n + 1, 41) = 42
swap((1, 2)) = (2, 1)
doctest: 1 test(s), 0 failure(s)
```

Key syntax, by version:

Syntax	Meaning	Since
`x: int`, `-> str`	Parameter / return annotation	3.0 (PEP 3107); typing semantics 3.5 (PEP 484)
`list[int]`, `dict[str, float]`	Built-in generics	3.9
`int | None`	Union (instead of `Optional[int]`)	3.10
`type Alias = ...`, `def f[T](x: T) -> T`	Type alias / generic syntax	3.12 (PEP 695)
Lazy evaluation of annotations	Forward references work without quotes	3.14 (PEP 649)

`inspect.signature(fn)` reads parameters, defaults and annotations at runtime. The `>>>` lines in the docstring are a **doctest**: `doctest.testmod()` runs them and compares the output.

Trade-offs

Annotations add some noise and need a type checker to pay off, but on shared code they catch whole classes of bugs and power autocompletion. Docstrings drift if not maintained — keep them short and about behaviour, not implementation.

When not to use it

Tiny throwaway scripts don't need full annotations. Don't restate the obvious in a docstring (`"""Returns x."""`).

Common mistakes

- Believing annotations validate input (they don't — `swap(("x", "y"))` ran fine).
- Putting a comment `# does X` instead of a docstring → invisible to `help()`.
- Docstring not the *first* statement → `__doc__` is `None`.

Real-world examples

- FastAPI builds request validation and OpenAPI docs from function annotations.
- NumPy/Google docstring styles are rendered by Sphinx on docs sites such as numpy.org.
- Large companies with big Python codebases (Dropbox is a well-known example) run mypy in CI to type-check millions of lines.

In the interview

> **30-second answer:** "A docstring is the first string in a function, stored in `__doc__` and shown by `help()`. Annotations declare expected types but aren't enforced at runtime; static checkers like mypy use them, and frameworks like FastAPI/Pydantic read them to validate."

- *Are type hints checked when the code runs?* → No, only by tools or libraries that choose to read them.
- *How do you read a function's annotations at runtime?* → `inspect.signature(f)` or `inspect.get_annotations(f)` / `f.__annotations__`.

Self-check

1. Where must a docstring go? → First statement in the body.
2. What does `float | None` mean? → Either a float or `None`.
3. Does `def f(x: int)` reject `f("a")`? → No.

TL;DR

Docstring = first-line string, shown by `help()`. Annotations = documentation checkable by mypy/pyright, not enforced at runtime.

Chapter TL;DR

- `def` creates a function object; calling runs it in a fresh frame; no `return` ⇒ `None`; `pass` for empty bodies.
- Arguments: positional (order) and keyword (name); defaults evaluated **once** ⇒ never a mutable default, use `None`; `*args` tuple, `**kwargs` dict; `/` positional-only, `*` keyword-only; `*` / `**` unpack at call sites; `return a, b` returns a tuple.
- Scope: **LEGB**. Assignment makes a name local for the whole function (`UnboundLocalError`). `global` → module, `nonlocal` → enclosing function. `if` / `for` don't create scopes.
- Closures remember enclosing variables (by reference ⇒ late binding). Decorators are closures: `@d` means `f = d(f)`; use `functools.wraps`.
- Functions are first-class: pass them, return them, `key=` them. `map` / `filter` are lazy; `sorted` is stable.
- `functools`: `partial`, `reduce`, `lru_cache` / `cache` (pure functions, hashable args), `wraps`.
- Recursion: base case + progress; limit 1000, no tail-call optimisation; memoize overlapping sub-problems; loop when deep.
- Docstrings for `help()`, annotations for tools — not runtime checks.

Chapter 8 — Modules, Packages and Environments

A single file is fine for a script. Real programs are spread across many files, depend on code other people wrote (NumPy, requests, Django), and must run the same way on your laptop, a colleague's machine and a server. This chapter covers the three layers that make that possible:

1. **Modules and packages** — how Python code is split into files and folders and pulled together with `import`.
2. **The import system** — where Python looks for code, what it caches, and why imports sometimes fail.
3. **Environments and packaging** — `pip`, virtual environments, `requirements.txt` vs `pyproject.toml`, modern tools (`uv`, Poetry), project layout and publishing to PyPI.

All scripts are in `code/`; they build their demo packages in temporary folders, so they leave nothing behind. Output shown is real (Python 3.14; code is 3.12-compatible).

What you'll learn

#	Topic	Script
8.1	Modules and the four `import` forms, aliases	`01_import_basics.py`
8.2	How `import` works: `sys.modules`, finders/loaders, `sys.path`	`01_import_basics.py`, `02_packages_and_sys_path.py`
8.3	Packages, `__init__.py`, namespace packages, `__all__`	`02_packages_and_sys_path.py`
8.4	Absolute vs relative imports	`02_packages_and_sys_path.py`
8.5	`if __name__ == "__main__"` and `python -m`	`02_packages_and_sys_path.py`
8.6	Common import errors, circular imports	`03_import_errors.py`
8.7	`pip` and virtual environments (`venv`)	`04_environments.py`
8.8	`requirements.txt` vs `pyproject.toml`; `uv` and Poetry	`04_environments.py`
8.9	Project layout (src layout) and publishing basics	–

8.1 Modules and `import`

What it is

A **module** is simply a `.py` file of Python code — functions, classes, variables — that other code can load with `import`. The module's name is the file name without `.py` (`pricing.py` → module `pricing`). Modules can also be compiled extensions (`.so` / `.pyd`, e.g. parts of NumPy) or be built into the interpreter (`sys`).

Python ships with a large **standard library** ("batteries included"): `math`, `json`, `pathlib`, `datetime`, `random`, `csv`, `sqlite3`, `http`, `unittest` and a few hundred more — file handling, networking, data formats, testing, concurrency — available with no installation.

Analogy: modules are the chapters of a manual; `import` is putting a bookmark in one so you can refer to it.

Why it matters

- **Organisation** — big programs become small files with one theme each.
- **Reuse and sharing** — the same module can serve many programs; the whole PyPI ecosystem is modules.
- **Namespaces** — each module has its own global namespace, so `json.load` and `pickle.load` never collide.

How it works — the four import forms

Form	You then write	Brings into your namespace
`import math`	`math.sqrt(81)`	The name `math` (the module)
`from math import sqrt`	`sqrt(64)`	Only the name `sqrt`
`import numpy as np`	`np.array(...)`	The module under an **alias**
`from math import sqrt as square_root`	`square_root(2)`	One name under an alias

```
import math                                  # whole module
import math as m                             # module with an alias
from math import sqrt                        # one name
from math import sqrt as square_root         # one name with an alias
from os import path as os_path

print("math.sqrt(81)       =", math.sqrt(81))
print("sqrt(64)            =", sqrt(64))
print("square_root(m.pi)   =", square_root(m.pi))
print("m is math           =", m is math)
```

Output:

```
== four import forms ==
math.sqrt(81)       = 9.0
sqrt(64)            = 8.0
square_root(m.pi)   = 1.7724538509055159
m is math           = True
os_path.join('a','b') = a/b

== a module is an object with a namespace ==
sys   type=module  compiled into the interpreter
json  type=module  file ending __init__.py
names in math starting with 'is': ['isclose', 'isfinite', 'isinf', 'isnan', 'isqrt']
```

- A dotted name after `import` (`math.sqrt`) uses **dot notation** to reach attributes inside the module.
- An **alias** (`as`) shortens long names (`np`, `pd`, `plt` are near-universal conventions) or avoids a clash (two modules both exporting `load`).
- `m is math` is `True`: an alias is just a second name for the **same** module object.
- `dir(module)` lists what's inside; `help(module)` shows its documentation.
- `json` reports `__init__.py` — it's actually a *package* (8.3).

`from module import *` copies every public name (or those listed in `__all__`) into your namespace. PEP 8 discourages it outside interactive sessions: readers can't tell where names came from and it can silently overwrite your own names.

Style (PEP 8): imports at the top of the file, one per line, grouped *standard library → third-party → your own code*, separated by blank lines. Tools like `ruff` and `isort` sort them automatically.

Trade-offs

	`import math`	`from math import sqrt`
Clarity at call site	`math.sqrt` shows origin	`sqrt` is shorter but origin is hidden
Name clashes	None	Possible
Circular-import safety	Safer (attribute looked up at call time, 8.6)	Riskier
Monkey-patching/mocking	Sees patched attributes	Holds the *old* object

Common mistakes

- Naming your own file after a module you want to use (`random.py`, `json.py`, `email.py`) – see 8.6.
- `import math.sqrt` → `ModuleNotFoundError: No module named 'math.sqrt'; 'math' is not a package`. `import` names modules; use `from math import sqrt` for functions.
- Wildcard imports in production code.

Real-world examples

Every data-science notebook starts with `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt` – community-standard aliases. Django projects import from their own apps: `from shop.models import Order`.

In the interview

> **30-second answer:** "A module is a `.py` file; `import m` loads it once and binds the name `m`, and `from m import x` binds just `x`. `as` gives an alias. Prefer explicit imports, avoid `import *`, and group imports stdlib/third-party/local."

- *Difference between `import a` and `from a import b`?* → Both load the whole module `a`; they differ only in which name is bound in your namespace.
- *Does importing twice run the module twice?* → No – the second time it's fetched from `sys.modules` (8.2).

Self-check

1. What is a module? → A file of Python code that can be imported.
2. After `from math import sqrt`, does `math.pi` work? → No – `math` isn't bound.
3. Why use `import numpy as np`? → Shorter, conventional alias.

TL;DR

Module = file. `import m` / `from m import x` / `as` aliases. Explicit imports, grouped, at the top.

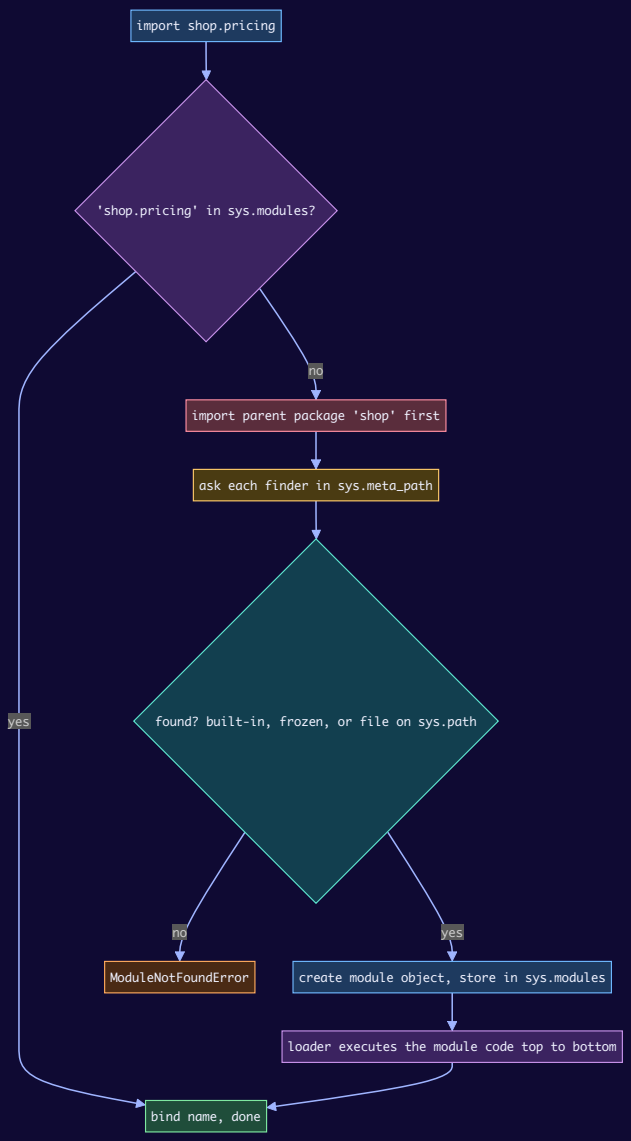
8.2 How `import` really works

What it is

The **import system** is the machinery that turns `import shop.pricing` into a module object. Three pieces matter:

- `sys.modules` – a dict cache of every module already loaded, keyed by full name.
- **Finders and loaders** – *finders* (listed in `sys.meta_path`) locate a module; a *loader* creates and executes it. The built-in chain handles built-in modules, frozen modules and the file system.
- `sys.path` – the list of folders the file-system finder searches, in order.

How it works



1. **Check the cache.** If the name is in `sys.modules`, reuse that object — module code runs **only once per process**. (`importlib.reload(m)` forces a re-run; rarely needed outside interactive work.)
2. **Import parents first.** `import a.b.c` imports `a`, then `a.b`, then `a.b.c`.
3. **Find.** Each finder is asked in turn. The path finder walks **`sys.path`**, which is built as:
    1. the folder of the script being run (or the current directory for `python -m` / `python -c` / the REPL),
    2. folders in the `PYTHONPATH` environment variable,
    3. the standard library,
    4. `site-packages` — where `pip` installs third-party packages (the active venv's, if one is active).
4. **Load.** A new module object is created and put into `sys.modules` **before** its code runs (this is why circular imports give half-built modules, 8.6), then its top-level code executes, filling its namespace.
5. **Bind** the requested name(s) in the importing namespace.

Compiled bytecode is cached in `__pycache__/*.pyc` files so later imports skip re-parsing; that's why those folders appear. They're safe to delete and belong in `.gitignore`.

```
print("'math' in sys.modules:", "math" in sys.modules)
again = importlib.import_module("math")
print("second import returns the same object:", again is math)
json_mod = importlib.import_module("json")     # import by string name
```

```
'math' in sys.modules: True
second import returns the same object: True
json.dumps via import_module: {"a": 1}
```

`importlib.import_module("name")` imports by a string — how plugin systems and frameworks (Django's `INSTALLED_APPS`) load modules chosen in configuration.

Common mistakes

- Fixing "module not found" with `sys.path.append(...)` hacks scattered through code. Install the project instead (`pip install -e .`, 8.9) or run with `python -m` from the project root.
- Expecting edits to a module to show up in a running REPL/notebook without `importlib.reload` (or Jupyter's `%autoreload`).

In the interview

> **30-second answer:** "`import` first checks `sys.modules`; if missing, finders search built-ins and then the folders in `sys.path` — script directory, PYTHONPATH, stdlib, site-packages. The loader creates the module, registers it in `sys.modules`, then runs its top-level code once. Later imports reuse the cached object."

- *Why does module-level code only run once?* → The module is cached in `sys.modules`.
- *What's `__pycache__`?* → Cached bytecode (`.pyc`) to speed up later imports.
- *In what order is `sys.path` searched?* → Script dir, PYTHONPATH, stdlib, site-packages; first match wins.

Self-check

1. Where does Python cache loaded modules? → `sys.modules`.
2. Which folder comes first in `sys.path` when you run `python app.py`? → The folder containing `app.py`.
3. How can you import a module whose name is in a string? → `importlib.import_module`.

TL;DR

Cache (`sys.modules`) → find (`sys.meta_path`, `sys.path`) → load (register, then execute once) → bind. First match on `sys.path` wins.

8.3 Packages, `__init__.py`, namespace packages, `__all__`

What it is

A **package** is a folder of modules that can be imported as a unit, giving a hierarchy of dotted names (`shop.pricing`, `sklearn.linear_model`). Technically a package is a module with a `__path__` attribute (the list of folders to search for its submodules).

- **Regular package**: a folder containing `__init__.py`. That file runs the first time the package (or anything inside it) is imported. It can be empty, or it can define the package's public API by re-exporting names, set `__version__`, etc.
- **Namespace package** (PEP 420, Python 3.3+): a folder *without* `__init__.py`. Python still imports it; several folders on `sys.path` can even contribute to one namespace (used by big organisations to split one namespace across separately-installed distributions). It has no `__file__`.

	Module	Regular package	Namespace package
On disk	`x.py`	folder + `__init__.py`	folder, no `__init__.py`
Has `__path__`	No	Yes	Yes
Runs code on import	The file	`__init__.py`	Nothing
Use for	Single unit	Normal libraries and apps	Splitting one namespace over several installs

`__all__` is an optional list of strings in a module or `__init__.py`: it defines what `from package import *` exports and documents the public API.

How it works — the demo package

`02_packages_and_sys_path.py` writes this tree into a temporary folder:

```
shop/
    __init__.py      # from .pricing import price_with_tax ; __all__ ; __version__
    __main__.py      # makes "python -m shop" work
    pricing.py       # uses a relative and an absolute import of utils
    utils.py
plugins/             # no __init__.py -> namespace package
    extra.py
```

```
# shop/__init__.py
"""shop - a tiny demo package."""
from .pricing import price_with_tax      # re-export the public API
__all__ = ["price_with_tax"]
__version__ = "0.1.0"
print(f"  [running shop/__init__.py, __name__={__name__!r}]")
```

Then it adds the temp folder to `sys.path` and imports:

```
sys.path.insert(0, str(root))
import shop                          # runs shop/__init__.py once
import shop                          # cached: __init__ does NOT run again
from shop import price_with_tax
```

```
== import it: put its parent folder on sys.path ==
  [running shop/__init__.py, __name__='shop']
  price_with_tax(100) = 108.0
  shop.__version__    = 0.1.0
  shop.__path__ is a list of folders: ['shop']
  pricing.__name__    = shop.pricing | __package__ = shop
  namespace package: extra plugin (namespace package, no __init__.py)
  plugins has __file__? None

== star import only takes names in __all__ ==
  names brought in: ['price_with_tax']
```

The `[running shop/__init__.py…]` line appears once despite two `import shop` statements. Users can write `from shop import price_with_tax` without knowing it lives in `shop/pricing.py` — the `__init__.py` "facade" is how `pandas` lets you write `pd.DataFrame` although `DataFrame` is defined deep in `pandas/core/frame.py`.

Trade-offs

A heavy `__init__.py` that imports every submodule makes `import pkg` slow and raises circular-import risk; an empty one forces users to know the internal layout. Common middle ground: re-export the small public API, keep the rest internal (by convention, `_private` names and modules).

When not to use it

Don't rely on namespace packages for your ordinary project – forgetting `__init__.py` "works" but tools (some test runners, type checkers, packaging configs) handle regular packages more predictably. Use namespace packages deliberately.

Common mistakes

- Putting slow work (network calls, loading big files) at top level of `__init__.py`.
- Assuming `import pkg` also imports all submodules – it doesn't, unless `__init__.py` imports them. `import pkg.sub` is needed.

Real-world examples

- `sklearn` is a package of subpackages: `from sklearn.linear_model import LogisticRegression`.
- `json`, `email`, `http`, `urllib`, `concurrent` are packages in the standard library.
- Google's cloud client libraries share the `google.cloud` namespace package across separately installed distributions (`google-cloud-storage`, `google-cloud-bigquery`).

In the interview

> **30-second answer:** "A package is a folder of modules; with `__init__.py` it's a regular package whose init file runs on first import and usually re-exports the public API. Without it, it's a PEP 420 namespace package that can span multiple folders. `__all__` controls `import *` and documents the public surface."

- *Is `__init__.py` required in Python 3?* → No, but without it you get a namespace package, which behaves differently; use it for normal packages.
- *What does `__all__` do?* → Lists names exported by `from m import *`.

Self-check

1. How does Python tell a package from a module? → A package has `__path__`.
2. When does `__init__.py` run? → On the first import of the package or any of its submodules.
3. Does `import shop` import `shop.utils`? → Only if `shop/__init__.py` (or something it imports) does.

TL;DR

Package = folder of modules (+ `__init__.py` for a regular package). `__init__.py` runs once and defines the public API. `__all__` limits `import *`.

8.4 Absolute vs relative imports

What it is

- **Absolute import**: names the full path from a top-level package on `sys.path` – `from shop.utils import round_money`.
- **Relative import**: names a location relative to the *current package* using leading dots – `from .utils import round_money` (same package), `from ..core import x` (parent package). Only the `from … import …` form can be relative.

How it works

Relative imports are resolved using the module's `__package__` (`'shop'` for `shop.pricing`, shown in the output above). A file run directly as a script has `__name__ == "__main__"` and no parent package, so relative imports inside it fail:

```
python shop/pricing.py  -> ImportError: attempted relative import with no known parent package
python -m shop.pricing  ->
  [running shop/__init__.py, __name__='shop']
  pricing run directly: 10 -> 10.8
  stderr: RuntimeWarning: 'shop.pricing' found in sys.modules after import of package 'shop', but prior to
execution of 'shop.pricing'; this may result in unpredictable behaviour
```

Running with `python -m shop.pricing` from the project root works because Python imports the package first. The `RuntimeWarning` is a real gotcha worth knowing: `shop/__init__.py` already imported `shop.pricing`, so `-m` then executes a *second* copy of it as `__main__`. The fix is to keep runnable entry points in modules that the package `__init__` doesn't import (such as `__main__.py` or a `cli.py`).

Trade-offs

	Absolute	Relative
Readability	Full origin visible	Short inside deep packages
Moving/renaming the top package	Must edit imports	Keeps working
Works in a script run directly	Yes (if package is on `sys.path`)	No
PEP 8	**Recommended**	Acceptable for intra-package imports

Python 2's *implicit* relative imports (`import utils` meaning "sibling module") no longer exist in Python 3 — a bare `import utils` is always absolute.

Common mistakes

- Running a module inside a package as `python pkg/mod.py` → relative imports fail; use `python -m pkg.mod`.
- Too many dots (`from ...x`) beyond the top-level package → `ImportError: attempted relative import beyond top-level package`.

In the interview

> **30-second answer:** "Absolute imports use the full dotted path and are PEP 8's default; relative imports use dots relative to the current package and work only inside packages, not in files run as scripts — run those with `python -m`."

Self-check

1. What does `from . import utils` mean? → Import `utils` from the current package.
2. Why does a relative import fail in `python pkg/mod.py`? → The module is `__main__` with no parent package.

TL;DR

Absolute by default; dots for intra-package imports; run package modules with `python -m`.

8.5 `if __name__ == "__main__":` and `python -m`

What it is

Every module has a `__name__`. When the file is **imported**, `__name__` is its module name (`'shop.pricing'`). When the file is **run directly** (`python file.py` or `python -m pkg.mod`), `__name__` is the string `'__main__'`. The guard

```
if __name__ == "__main__":
    main()
```

runs code only when the file is executed, not when it's imported.

Why it matters

It lets one file be both a **reusable module** and a **runnable script**: tests and other modules can import its functions without triggering the script behaviour (starting a server, parsing command-line arguments, printing). It's also required for code using `multiprocessing` on platforms that start workers by re-importing the main module (the `spawn` and `forkserver` start methods: Windows, macOS by default since 3.8, and Linux since 3.14 made `forkserver` its default), otherwise each worker would re-run the script.

How it works

- `python -m package.module` runs a module found via `sys.path` as `__main__` – the right way to run code inside packages, and how many tools are invoked: `python -m pip`, `python -m venv`, `python -m pytest`, `python -m http.server`, `python -m json.tool`.
- `python -m package` runs `package/__main__.py` (our demo: `python -m shop says: 19.99 -> 21.59`).
- In a script, `sys.path[0]` is the script's folder; with `-m`, it's the current directory.

```
python -m shop          ->
  [running shop/__init__.py, __name__='shop']
  python -m shop says: 19.99 -> 21.59
```

Common mistakes

- Top-level test calls or `print`s without the guard – they run every time someone imports your module.
- Putting lots of logic *inside* the guard. Keep it to `main()` so the logic stays importable and testable.

In the interview

30-second answer: "`__name__` is `'__main__'` for the file being run and the module name when imported. The guard keeps script-only code from running on import. `python -m pkg.mod` runs a module inside a package with correct imports; `python -m pkg` runs `pkg/__main__.py`."

Self-check

1. What's `__name__` inside `shop/pricing.py` when imported? → `'shop.pricing'`.
2. What does `python -m shop` execute? → `shop/__main__.py`.

TL;DR

Guard script behaviour with `if __name__ == "__main__": main()`. Use `python -m` for modules inside packages.

8.6 Common import errors and circular imports

What it is

Most import problems fall into a handful of patterns. `03_import_errors.py` reproduces each one in a throwaway folder:

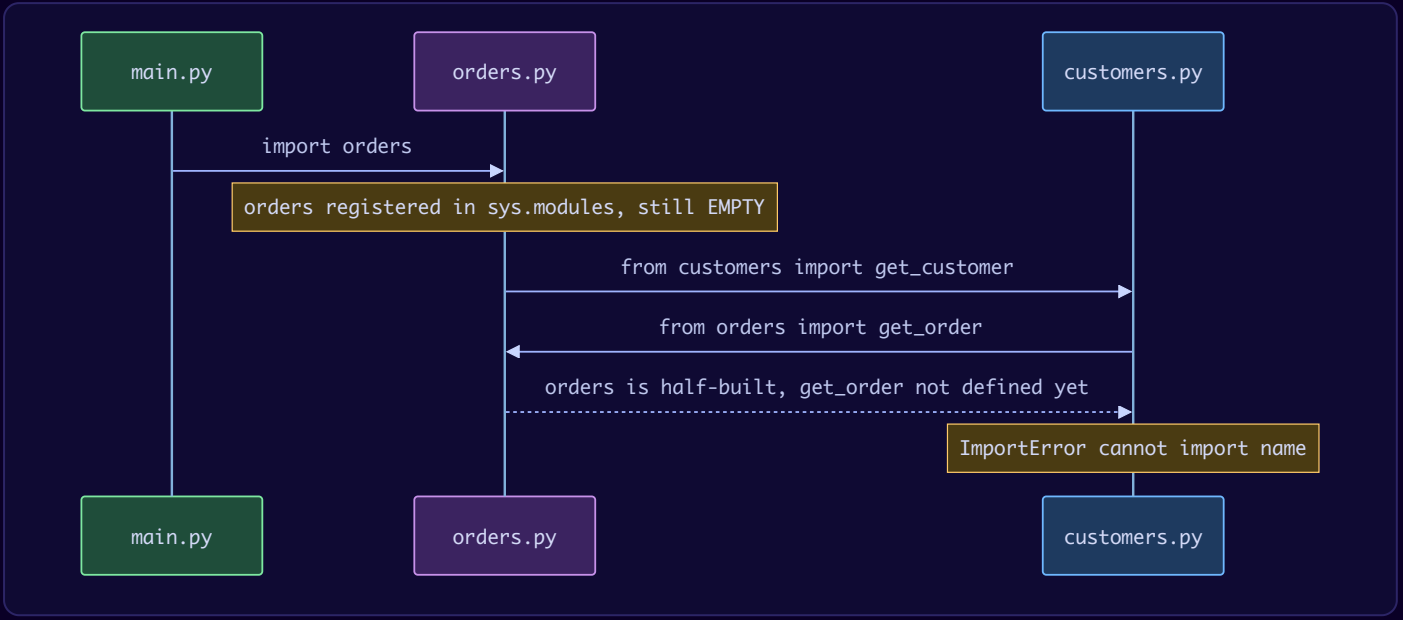
```
1. missing module                -> ModuleNotFoundError: No module named 'reqests'
2. name not in module            -> ImportError: cannot import name 'square_root' from 'math'
(.../math.cpython-314-darwin.so)
3. shadowing the stdlib          -> AttributeError: module 'random' has no attribute 'randint' (consider
renaming '.../random.py' since it has the same name as the standard library module named 'random' and prevents
importing that standard library module)
4. circular import (from-import)   -> ImportError: cannot import name 'get_order' from 'orders' (consider
renaming '.../orders.py' if it has the same name as a library you intended to import)
4b. fixed: import module, use attr -> {'id': 7, 'customer': {'id': 1}}
4c. fixed: import inside function  -> {'id': 7, 'customer': {'id': 1}}
```

Error	Usual cause	Fix
`ModuleNotFoundError` (subclass of `ImportError`)	Typo; package not installed **in the interpreter you're running**; wrong working directory	Check spelling; `python -m pip install x` with the same `python`; install the project or use `-m`
`ImportError: cannot import name`	Name doesn't exist in that module (typo, removed in a new version) — or a circular import	Check the module's contents / version
`AttributeError: module 'x' has no attribute 'y'`	Your file named `x.py` shadows the real module; or submodule not imported	Rename your file (and delete its `__pycache__`)
`ImportError: attempted relative import …`	Ran a package module as a script	`python -m pkg.mod`

Case 3 happens because `sys.path[0]` (the script's folder) is searched before the standard library, so a local `random.py` wins. Python 3.13+ adds the helpful "consider renaming" hint.

Circular imports

A **circular import** is when module A imports B while B (directly or indirectly) imports A.



Why it breaks: step 4 of 8.2 — a module is registered in `sys.modules` *before* its body runs. When `customers` asks for `orders.get_order`, it gets the half-initialised `orders` module whose `def get_order` line hasn't executed yet. (On Python 3.12 the message says it plainly: "cannot import name 'get_order' from partially initialized module 'orders' (most likely due to a circular import)". The 3.14 run above instead appends a generic "consider renaming" hint, which is misleading here — nothing is shadowed. When you see `cannot import name` for a name that clearly exists, suspect a cycle first.)

Fixes, best first:

1. **Restructure** – move the shared code into a third module both can import (breaks the cycle; usually the right design fix).
2. **Import the module, not the name** (`import orders` then `orders.get_order(...)` inside functions) – the attribute is looked up at *call* time, after both modules have finished loading (case 4b).
3. **Deferred import** inside the function that needs it (case 4c) – cheap, local, but hides a dependency.
4. For imports needed only by type hints: `from typing import TYPE_CHECKING` / `if TYPE_CHECKING: from orders import Order` (type checkers see it, runtime doesn't).

Common mistakes

- "`pip install` worked but import fails": pip installed into a different interpreter. Always use `python -m pip ...` with the same `python` that runs your code, and check `which python` / `sys.executable`.
- Naming a file `test.py`, `code.py`, `email.py`, `random.py`, `numpy.py` …

Real-world examples

Django models in different apps that reference each other avoid circular imports by using string references (`models.ForeignKey("customers.Customer", ...)`) – a framework-level version of "look it up later".

In the interview

> **30-second answer:** "A circular import happens because a module is put in `sys.modules` before its body finishes, so the other module sees a half-built object and `from x import y` fails. Fix by moving shared code to a third module, importing the module instead of names so lookup happens at call time, or deferring the import into a function."

- *`ModuleNotFoundError` but the package is installed – why?* → Different interpreter/venv from the one pip used; or a local file shadows it.
- *Why does `import a` survive a cycle when `from a import f` doesn't?* → `import a` only needs the module object to exist; `from a import f` needs `f` to be defined already.

Self-check

1. What's the parent class of `ModuleNotFoundError`? → `ImportError`.
2. Why does a local `random.py` break `import random`? → The script's folder is first on `sys.path`.
3. Best long-term fix for a cycle? → Extract the shared code into a separate module.

TL;DR

Check spelling, interpreter and shadowing first. Cycles break `from x import y`; fix by restructuring, `import module`, or deferred import.

8.7 `pip` and virtual environments

What it is

- **PyPI** (Python Package Index, pypi.org) is the public repository of third-party packages.
- `pip` is the standard installer: it downloads packages (preferably pre-built **wheels**, `.whl`) and their dependencies from PyPI into an environment's `site-packages`.
- A **virtual environment (venv)** is an isolated folder containing its own `site-packages` and a `python` link to a base interpreter. Packages installed into it don't affect other projects or the system Python.

Analogy: each project gets its own toolbox; you don't throw all tools from all jobs into one drawer.

Why it matters

Project A needs `Django 4.2`, project B needs `Django 5.1` – only one can live in a single `site-packages`. Venvs give each project its own versions, make "works on my machine" reproducible, and keep you from breaking your

operating system's Python. Many systems (Homebrew Python, Debian/Ubuntu) now refuse a global `pip install` with an `externally-managed-environment` error (PEP 668) precisely to push you into a venv.

How it works

```
python3 -m venv .venv                 # create (folder name .venv is the convention)
source .venv/bin/activate             # macOS/Linux   (Windows: .venv\Scripts\activate)
python -m pip install requests        # install into THIS venv
python -m pip install "django>=5,<6"  # version specifiers
python -m pip list                    # what's installed
python -m pip show requests           # details, dependencies
python -m pip install -U requests     # upgrade
python -m pip uninstall requests
python -m pip freeze > requirements.txt   # exact versions of everything installed
python -m pip install -r requirements.txt # recreate elsewhere
deactivate                            # leave the venv
```

"Activating" only prepends the venv's `bin/` to your `PATH` so `python` and `pip` refer to the venv's copies — you can also skip activation and call `.venv/bin/python` directly. A venv is not portable: don't copy or commit it; commit the dependency list and recreate it. Add `.venv/` to `.gitignore`.

Inside a venv, `sys.prefix` (the venv) differs from `sys.base_prefix` (the base install) — that's how code detects one:

```
def in_venv() -> bool:
    return sys.prefix != sys.base_prefix

venv.create(env_dir, with_pip=False)   # same as: python -m venv .venv --without-pip
```

```
== which interpreter am I? ==
  Python 3.14.7
  running inside a venv: True
  sys.prefix folder name: .venv-books

== installed distributions (importlib.metadata) ==
  numpy         2.5.3
  pandas        3.0.6
  scikit-learn  1.9.1
  numpy requires python: >=3.12

== a fresh, empty venv is isolated ==
  created: ['bin', 'include', 'lib', 'pyvenv.cfg']
  new venv -> in venv: True
  new venv -> No module named 'numpy'
```

The book's own environment has NumPy, but a brand-new venv can't see it — isolation in action. `importlib.metadata.version("numpy")` reads an installed distribution's version at runtime.

Trade-offs

Tool	What it isolates	Notes
`venv` (stdlib)	Python packages	Built in; uses an existing Python
`pipx`	Each CLI *tool* in its own venv	For tools like `black`, `httpie`, not project libraries
`conda`	Packages and non-Python libraries (CUDA, MKL, compilers), Python itself	Popular in data science; own package channels
Docker	Whole OS userland	Deployment-level isolation, heavier

When not to use it

You don't need a venv to run a script that uses only the standard library. Everything else — use one per project.

Common mistakes

- `pip install` running a different Python than your code → always `python -m pip`.
- `sudo pip install` into the system Python.
- Committing `.venv/` to git or copying it between machines.
- Unpinned installs in production (`pip install requests` today and next year give different versions).

Real-world examples

CI pipelines (GitHub Actions) create a fresh environment per run and install from a lock/requirements file, so every build is reproducible. Heroku-style platforms and many Docker images do `pip install -r requirements.txt` at build time.

In the interview

> **30-second answer:** "pip installs packages from PyPI into an environment's site-packages. A venv is an isolated environment per project — `python -m venv .venv`, activate, `python -m pip install`. It prevents version conflicts between projects and protects the system Python. Don't commit the venv; commit the dependency list."

- *How can code tell it's in a venv?* → `sys.prefix != sys.base_prefix`.
- *venv vs conda?* → venv isolates Python packages for an existing interpreter; conda also manages Python versions and native libraries.

Self-check

1. Command to create a venv? → `python -m venv .venv`.
2. Why `python -m pip` instead of `pip`? → Guarantees pip installs into the same interpreter you run.
3. What does activating a venv actually change? → Your `PATH` (plus the prompt).

TL;DR

One venv per project. `python -m venv .venv` → activate → `python -m pip install …`. Never commit the venv.

8.8 `requirements.txt` vs `pyproject.toml`; `uv` and Poetry

What it is

- `requirements.txt` — a plain list of packages for pip to install, often with exact pins (`requests==2.32.3`). It describes an *environment*, not a project, and it's pip-specific.
- `pyproject.toml` — the standard project configuration file (TOML format). It declares **build requirements** (PEP 518, `[build-system]`), **project metadata** — name, version, supported Python, dependencies, optional extras, command-line entry points (PEP 621, `[project]`) — and settings for tools (`[tool.ruff]`, `[tool.pytest.ini_options]`, …).
- A **lock file** (`uv.lock`, `poetry.lock`, or a fully pinned requirements file) records the exact resolved version of *every* package including transitive dependencies, for reproducible installs.

How it works

A minimal `pyproject.toml` — the script parses this very text with the standard-library `tomllib` (3.11+):

```toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "shop"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["requests>=2.31,<3", "pydantic>=2"]

[project.optional-dependencies]
dev = ["pytest>=8", "ruff"]

[project.scripts]
shop = "shop.cli:main"
```

```
== reading pyproject.toml (tomllib, 3.11+) ==
  name/version : shop 0.1.0
  dependencies : ['requests>=2.31,<3', 'pydantic>=2']
  dev extras   : ['pytest>=8', 'ruff']
  console script 'shop' -> shop.cli:main
  build backend: hatchling.build
```

- `dependencies` uses **ranges** (what your code is compatible with); the lock file holds **exact pins** (what you tested).
- `pip install -e ".[dev]"` installs the project in **editable** mode plus the `dev` extras.
- `[project.scripts]` makes `pip` create a `shop` command that calls `shop.cli:main()`.
- The **build backend** (hatchling, setuptools, flit-core, poetry-core, uv_build…) turns the project into distributable files (PEP 517).

	`requirements.txt`	`pyproject.toml`
Standard?	pip convention	PEP 518/517/621 standard
Describes	An environment to install	A project: metadata, deps, build, tool config
Typical content	Exact pins	Compatible ranges
Used by	`pip install -r`	pip, build, uv, Poetry, hatch, ruff, pytest…
Best for	Deploying an app; simple scripts	Any package or modern project

Version specifiers: `==2.1.0` exact, `>=2.1` minimum, `<3` upper bound, `~=2.1` "compatible release" (≥2.1, <3.0), `!=2.1.3` exclude.

Modern tools: `uv` and Poetry (brief)

- **uv** (by Astral, written in Rust) – a very fast replacement for pip, venv, pip-tools and pipx in one binary. `uv init` (new project with `pyproject.toml`), `uv add requests` (add dependency + update `uv.lock` + install), `uv sync` (make `.venv` match the lock), `uv run pytest` (run inside the project env), `uv python install 3.12` (manage Python versions), `uv pip install ...` (pip-compatible interface), `uv build` / `uv publish`.
- **Poetry** – an older, widely used all-in-one tool: `poetry new`, `poetry add`, `poetry install`, `poetry.lock`, `poetry build`, `poetry publish`. Poetry 2.x supports the standard `[project]` table.

Both do the same core job: manage `pyproject.toml`, resolve dependencies, write a lock file, and create the venv for you.

When not to use it

For a one-file script, a venv + `requirements.txt` is enough. Don't mix tools in one project (e.g. Poetry lock + hand-edited requirements) – pick one source of truth.

Common mistakes

- Pinning exact versions in a library's `dependencies` – it makes your library impossible to install alongside others. Libraries use ranges; applications lock.
- `pip freeze` into requirements for a library (captures your whole environment, including dev tools).
- Forgetting to commit the lock file for applications.

Real-world examples

Most major projects (pandas, FastAPI, pytest, scikit-learn) now declare metadata in `pyproject.toml`. Tools such as ruff, black, mypy and pytest read their configuration from it.

In the interview

> **30-second answer:** "`requirements.txt` is a pip input listing packages, usually pinned – an environment snapshot. `pyproject.toml` is the standard project file: build system, metadata, dependency ranges, extras, entry points and tool config. Reproducibility comes from a lock file; tools like uv and Poetry manage the pyproject, lock and venv together."

- *Library vs application dependencies?* → Libraries declare loose ranges; applications pin exact versions with a lock file.
- *What's a wheel?* → A pre-built distribution (`.whl`, a zip) that installs without running a build – fast and no compiler needed.

Self-check

1. Which PEP defines the `[project]` table? → PEP 621.
2. What does `~=2.1` allow? → ≥2.1 and <3.0.
3. What file makes installs reproducible? → A lock file (or fully pinned requirements).

TL;DR

`pyproject.toml` = the project (ranges, metadata, build, tool config). Lock file / pinned `requirements.txt` = exact environment. uv (fast, all-in-one) and Poetry manage both.

8.9 Project layout and publishing basics

What it is

The **src layout** puts your package inside a `src/` folder instead of at the project root:

```
shop-project/
├── pyproject.toml
├── README.md
├── src/
|   └── shop/
|       ├── __init__.py
|       ├── __main__.py
|       ├── cli.py
|       └── pricing.py
└── tests/
    └── test_pricing.py
```

(The alternative, **flat layout**, puts `shop/` next to `pyproject.toml`.)

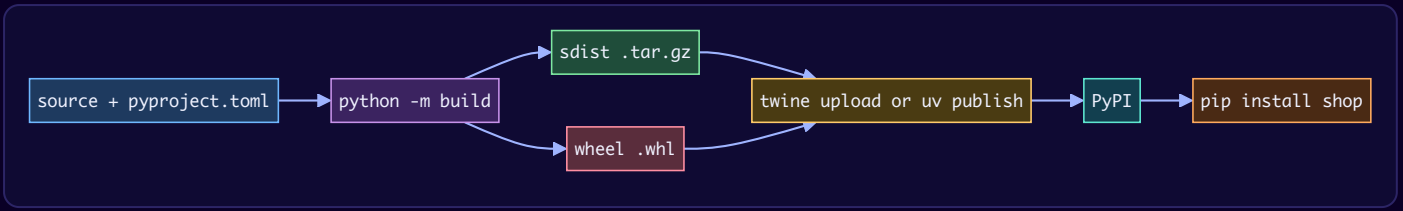
Why it matters

With a flat layout, running `pytest` or `python` from the project root puts the root on `sys.path`, so `import shop` silently uses your *source folder* – tests pass even if packaging is broken (a missing file in the built

wheel, a wrong `package-data` rule). With the src layout, `shop` is importable **only after installing** it (`pip install -e .` for development), so your tests exercise what users actually get. The Python Packaging User Guide describes both and explains this advantage of src.

How it works — publishing to PyPI

1. Put metadata in `pyproject.toml` (name must be unique on PyPI; bump `version` for every release — PyPI never lets you re-upload a version).
2. **Build** distributions: `python -m build` (or `uv build`) creates `dist/shop-0.1.0.tar.gz` (**sdist**, source) and `dist/shop-0.1.0-py3-none-any.whl` (**wheel**, ready to install).
3. **Test** on **TestPyPI** (test.pypi.org), a separate sandbox index.
4. **Upload**: `python -m twine upload dist/*` (or `uv publish`) with an API token — or, better, **Trusted Publishing** from CI (GitHub Actions authenticates to PyPI via OpenID Connect, no long-lived token stored).
5. Users run `pip install shop`.



Trade-offs

	src layout	flat layout
Catches packaging mistakes	Yes (must install to import)	No
Setup effort	Needs `pip install -e .`	Works from root immediately
Used by	Many libraries (and uv's `--package` / `--lib` templates)	Many apps and simple projects

When not to use it

A small app that's never packaged (a Django site, a notebook project) gains little from src layout or publishing.

Common mistakes

- Uploading secrets: check `dist/` contents before uploading (a wheel is just a zip — `unzip -l`).
- Re-using a version number — PyPI will reject it.
- Package name on PyPI ≠ import name (e.g. install `scikit-learn`, import `sklearn`; install `Pillow`, import `PIL`) — confusing for users, so keep them aligned for new projects.

Real-world examples

The `pip install scikit-learn` → `import sklearn` split is the classic name mismatch. Many popular projects publish to PyPI from GitHub Actions using Trusted Publishing.

In the interview

> **30-second answer:** "Use a `pyproject.toml`, ideally a src layout so tests run against the installed package. Build an sdist and a wheel with `python -m build` or `uv build`, try them on TestPyPI, then upload with twine or `uv publish`, preferably via Trusted Publishing from CI. Versions are immutable on PyPI."

- *sdist vs wheel?* → sdist is source that must be built on install; wheel is pre-built and installs by unpacking.
- *Why src layout?* → Prevents accidentally importing the working copy instead of the installed package.

Self-check

1. What two files does `python -m build` produce? → An sdist (`.tar.gz`) and a wheel (`.whl`).

2. How do you make a src-layout package importable while developing? → `pip install -e .` (editable install).
3. Can you re-upload version 0.1.0 after a fix? → No — release 0.1.1.

TL;DR

src layout + `pyproject.toml` + editable install for dev. `build` → sdist + wheel → TestPyPI → PyPI (Trusted Publishing).

Chapter TL;DR

- **Module** = a `.py` file; **package** = folder of modules (`__init__.py` → regular, none → namespace). `import m`, `from m import x`, `as` aliases; avoid `import *`.
- `import` checks `sys.modules` → finds via `sys.path` (script dir, PYTHONPATH, stdlib, site-packages) → registers then runs the module **once** → binds names.
- Absolute imports by default; relative (`.`) only inside packages; run package modules with `python -m`.
- `if __name__ == "__main__": main()` separates "run" from "import".
- Import errors: typo / wrong interpreter / shadowing file / circular import (module registered before its body runs — restructure, `import module`, or deferred import).
- One **venv** per project; `python -m pip`; never commit `.venv`.
- `pyproject.toml` (standard: metadata, dependency ranges, build, tool config) vs `requirements.txt` (pip list, pins); lock files for reproducibility; **uv** or **Poetry** manage it all.
- **src layout**, build **sdist + wheel**, TestPyPI, then PyPI via twine/`uv publish`/Trusted Publishing.

Chapter 9 — Errors, Exceptions and Files

Programs meet bad input, missing files, full disks and broken networks. Python reports these situations with **exceptions** — objects that interrupt normal flow and travel up the call stack until some code handles them. Most of the time the thing that fails is I/O, so this chapter pairs error handling with working on files: reading and writing text and bytes, paths, CSV, JSON, pickle, temporary files and whole-folder operations.

Scripts are in `code/`; every file operation happens inside a temporary directory that is removed afterwards. Output is real (Python 3.14; code is 3.12-compatible).

What you'll learn

#	Topic	Script
9.1	What exceptions are, tracebacks, the built-in hierarchy	`01_exceptions.py`
9.2	`try` / `except` / `else` / `finally`	`01_exceptions.py`
9.3	Raising, re-raising, custom exception classes	`01_exceptions.py`
9.4	Exception chaining (`raise … from …`), `add_note`	`01_exceptions.py`
9.5	EAFP vs LBYL	`01_exceptions.py`
9.6	`ExceptionGroup` and `except*` (3.11+)	`01_exceptions.py`
9.7	`assert`	`01_exceptions.py`
9.8	Context managers and `with`	`02_context_managers.py`
9.9	File I/O: modes, text vs binary, encodings	`03_files_pathlib.py`
9.10	`pathlib`, `shutil`, `tempfile`	`03_files_pathlib.py`
9.11	`csv`, `json`, `pickle` (and why pickle is unsafe)	`04_csv_json_pickle.py`

9.1 Exceptions and the hierarchy

What it is

An **exception** is an object describing an error, **raised** (thrown) at the point of failure. Raising stops the current code immediately; Python then unwinds the call stack — leaving each function in turn — looking for a matching `except` handler. If none is found, the program stops and prints a **traceback**: the chain of calls that led to the error (read it **bottom-up**: the last line is the exception type and message, the line above is where it happened).

```
Traceback (most recent call last):
  File "app.py", line 8, in <module>
    total = parse("12a")
  File "app.py", line 3, in parse
    return int(text)
ValueError: invalid literal for int() with base 10: '12a'
```

Two kinds of problems are distinct:

- **Syntax errors** are found when Python *compiles* the file (`SyntaxError`, `IndentationError`) — nothing runs.
- **Exceptions** happen while the code *runs* (`ZeroDivisionError`, `KeyError`, …) and can be handled.

Analogy: an exception is a fire alarm. Whoever is nearest and equipped (a matching `except`) deals with it; if nobody does, the building (program) is evacuated.

How it works — the class hierarchy

Every exception is an instance of a class deriving from `BaseException`. Handlers match by class *and all its subclasses*, so the tree decides what an `except` catches.

```
== part of the built-in hierarchy ==
  BaseException
      BaseExceptionGroup
      Exception
      GeneratorExit
      KeyboardInterrupt
      SystemExit
  ...under Exception (selection):
  LookupError
      IndexError
      KeyError
  ArithmeticError
      FloatingPointError
      OverflowError
      ZeroDivisionError
  OSError
      FileNotFoundError
      IsADirectoryError
      PermissionError
      TimeoutError
  issubclass(ModuleNotFoundError, ImportError): True
```

(The script walks `__subclasses__()` of the real classes; only a selection is printed.)

Class	Raised when	Example
`SystemExit`, `KeyboardInterrupt`, `GeneratorExit`	Program exit, Ctrl-C, generator closing	Deliberately not under `Exception` so `except Exception` doesn't swallow them
`ValueError`	Right type, bad value	`int("abc")`
`TypeError`	Wrong type / wrong call	`"a" + 1`, too many arguments
`KeyError` / `IndexError` (both `LookupError`)	Missing key / index out of range	`{}["x"]`, `[][0]`
`AttributeError`	Missing attribute	`None.upper()`
`NameError` (→ `UnboundLocalError`)	Unknown name	typo in a variable
`ZeroDivisionError` (`ArithmeticError`)	Division by zero	`1 / 0`
`OSError` family	OS-level failures	`FileNotFoundError`, `PermissionError`, `IsADirectoryError`, `TimeoutError`, `ConnectionError`
`ImportError` (→ `ModuleNotFoundError`)	Import failures	Chapter 8
`StopIteration`	Iterator exhausted	used by `for` internally
`RecursionError` (`RuntimeError`)	Too-deep recursion	Chapter 7
`NotImplementedError` (`RuntimeError`)	Abstract method not overridden	
`AssertionError`	`assert` failed	9.7
`UnicodeDecodeError` (→ `ValueError`)	Bytes aren't valid in the codec	9.9

In the interview

30-second answer: "An exception is an object raised on error; it unwinds the stack until an `except` clause whose class matches (including subclasses) handles it, otherwise the program ends with a traceback. All exceptions derive from `BaseException`; application errors derive from `Exception`, while `SystemExit`, `KeyboardInterrupt` and `GeneratorExit` sit directly under `BaseException` so generic handlers don't catch them."

- *Why is `KeyboardInterrupt` not an `Exception`?* → So `except Exception:` doesn't stop Ctrl-C from working.
- *How do you read a traceback?* → Bottom line = type and message; the frames above show the call path, most recent last.

Self-check

1. Common parent of `KeyError` and `IndexError`? → `LookupError`.
2. Is `SyntaxError` catchable in the file that contains it? → No — the file never compiles, so nothing in it runs.
3. What is `FileNotFoundError` a subclass of? → `OSError`.

TL;DR

Exceptions are objects that unwind the stack until matched by class. Know the tree: `BaseException` → `Exception` → your errors.

9.2 `try` / `except` / `else` / `finally`

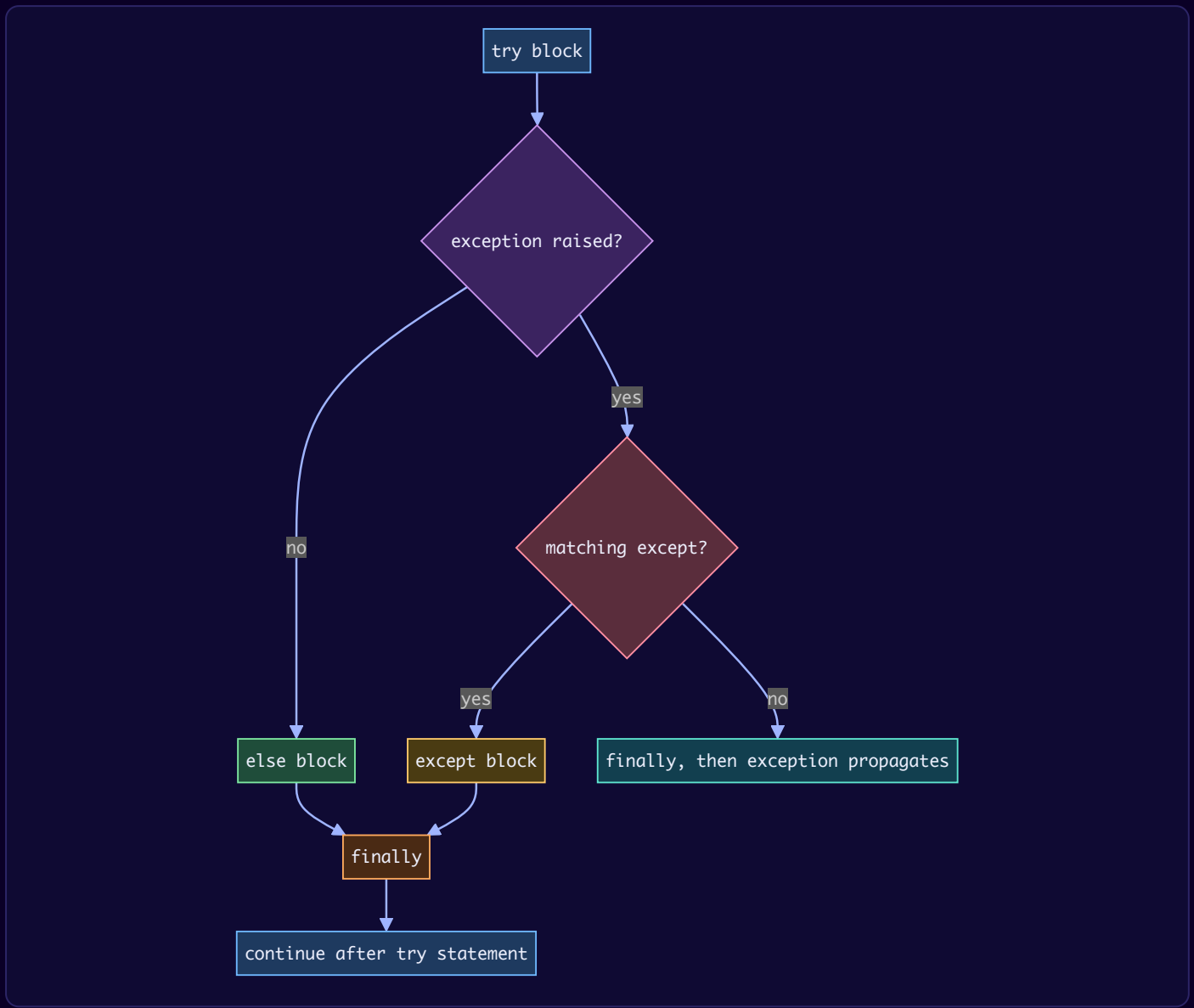
What it is

The statement for handling exceptions:

```
try:
    risky()                 # code that might fail
except SomeError as exc:    # runs if SomeError (or subclass) was raised in try
    handle(exc)
else:                       # runs only if try raised NOTHING
    use_result()
finally:                    # runs ALWAYS: success, handled, unhandled, return, break
    clean_up()
```

How it works

1. Run the `try` block.
2. If an exception occurs, skip the rest of `try` and check `except` clauses **top to bottom**; the **first** matching one runs. `except (A, B) as e:` catches either.
3. If nothing was raised, run `else`.
4. Run `finally` in every case — even if an exception is propagating, or the block `return`s.
5. If no clause matched, the exception continues up the stack (after `finally`).



```python
def parse_age(text):
    try:
        age = int(text)
    except ValueError:
        print(f"  except: {text!r} is not a number")
        return None
    else:
        print(f"  else:   parsed {age}")
        return age
    finally:
        print("  finally: always runs")
```

```
== try / except / else / finally ==
  else:   parsed 42
  finally: always runs
  except: 'forty' is not a number
  finally: always runs
```

`finally` ran even though both branches `return`ed.

Why `else`? It keeps the `try` block tiny — only the line that can raise the error you're handling. Code in `else` isn't protected, so a bug there won't be silently caught by the wrong handler.

Trade-offs / rules of thumb

Do	Don't
Catch the **most specific** exception you can handle	`except:` (bare) — also catches `KeyboardInterrupt` and `SystemExit`
Order handlers specific → general	Put `except Exception` first (it would shadow the rest)
Keep `try` blocks small	Wrap 50 lines in one `try`
Handle where you can *do something* (retry, default, message)	Catch just to `pass` (silently hiding bugs)
Use `finally` / `with` for cleanup	`return` inside `finally` — it swallows exceptions (Python 3.14 emits a `SyntaxWarning`, PEP 765)

When not to use it

Don't use exceptions for ordinary control flow when a simple check or API already exists (`dict.get`, `str.isdigit` for trusted formats). And don't catch errors you can't handle — let them reach a top-level handler that logs them.

Common mistakes

- `except Exception as e: pass` — the bug disappears and resurfaces far away.
- Referencing `e` after the `except` block — Python **deletes** the `as` name at the end of the handler (to break reference cycles), so it's a `NameError` later. Save it to another variable if needed.
- Catching `Exception` around code that includes your own bugs (`TypeError` from a typo) and reporting "invalid input".

Real-world examples

- **Logging instead of hiding**: at the top level, `logging.exception("msg")` (inside an `except`) logs the message *plus* the full traceback; the `traceback` module (`traceback.print_exc()`, `traceback.format_exc()`) gives you the same text as a string or on stderr. See Chapter 14.
- **Warnings are not exceptions**: `warnings.warn("old_api() is deprecated", DeprecationWarning)` prints a message but lets the program continue. Libraries use this for deprecations; `python -W error` (or pytest's `-W error`) turns warnings into exceptions so you catch them in CI.

- Web frameworks (Django, Flask, FastAPI) wrap every request in a top-level handler that turns uncaught exceptions into a 500 response and a log entry, so one bad request doesn't crash the server.
- `requests`: `resp.raise_for_status()` raises `requests.HTTPError` for 4xx/5xx; all its errors derive from `requests.RequestException`, so callers can catch one base class.

In the interview

> **30-second answer:** "`try` runs risky code; the first matching `except` handles; `else` runs only if nothing was raised, which keeps the `try` minimal; `finally` always runs for cleanup. Catch specific exceptions, order specific to general, never a bare `except`."

- *When does `else` run?* → Only when `try` completed without an exception.
- *Does `finally` run if `try` returns?* → Yes, just before the function actually returns.
- *Bare `except:` vs `except Exception:`?* → Bare also catches `BaseException` subclasses like `KeyboardInterrupt` / `SystemExit`.

Self-check

1. Two handlers `except Exception` then `except ValueError` — which runs for `int("x")`? → The first (`Exception`); the second is dead code.
2. What's the point of `else`? → Separate "code that may raise the handled error" from "code that runs on success".
3. Is `e` defined after `except ValueError as e:` finishes? → No.

TL;DR

`try` small, `except` specific (first match wins), `else` on success, `finally` always.

9.3 Raising, re-raising and custom exceptions

What it is

- `raise SomeError("message")` signals an error yourself — typically when inputs violate your function's contract.
- **Re-raising**: inside an `except`, a bare `raise` re-throws the current exception with its original traceback, after you've logged or cleaned up.
- **Custom exceptions** are classes you define (subclassing `Exception`) so callers can catch *your* errors precisely.

How it works

1. Pick the right built-in when one fits: bad value → `ValueError`, wrong type → `TypeError`, missing key-like thing → `KeyError` / `LookupError`, unsupported operation → `NotImplementedError`.
2. For domain errors, define a **base class** for your library/module and specific subclasses. Callers can catch all your errors (`except PaymentError`) or one (`except CardDeclined`).
3. Name them with an `Error` suffix (PEP 8). Add attributes for data the handler needs.

```python
class PaymentError(Exception):
    """Base class for this module's errors."""

class CardDeclined(PaymentError):
    def __init__(self, card_last4, reason):
        super().__init__(f"card ending {card_last4} declined: {reason}")
        self.card_last4 = card_last4
        self.reason = reason

class InsufficientFunds(PaymentError):
    pass

def charge(balance, amount):
    if amount <= 0:
        raise ValueError(f"amount must be positive, got {amount}")
    if amount > balance:
        raise InsufficientFunds(f"need {amount}, have {balance}")
    return balance - amount

def reraise_after_logging(x):
    try:
        return 1 / x
    except ZeroDivisionError:
        print("  logging the error, then re-raising")
        raise                           # bare raise keeps the original traceback
```

```
== raising and catching custom exceptions ==
  charge(100, 30) -> 70
  InsufficientFunds: need 500, have 100
  ValueError: amount must be positive, got -5
  caught CardDeclined, attrs: last4=4242 reason=expired

== re-raising ==
  logging the error, then re-raising
  caller still sees: ZeroDivisionError('division by zero')
```

Pass the message to `super().__init__` so `str(exc)` and tracebacks show it.

Trade-offs

Too few custom exceptions → callers parse message strings to distinguish errors (fragile). Too many → a zoo nobody remembers. A module base class plus a handful of meaningful subclasses is the usual sweet spot. Many libraries also inherit from a matching built-in too (`class InvalidAmount(PaymentError, ValueError)`) so generic `except ValueError` code keeps working.

Common mistakes

- Subclassing `BaseException` instead of `Exception` — escapes `except Exception` handlers.
- `raise exc` instead of bare `raise` inside the handler — works, but adds the current line to the traceback; bare `raise` is the idiom.
- Raising strings (`raise "error"`) — a `TypeError` in Python 3; only exception instances/classes can be raised.
- Returning error codes (`return -1`) instead of raising — callers forget to check.

Real-world examples

- `requests.RequestException` → `ConnectionError`, `Timeout`, `HTTPError`…
- Django: `ObjectDoesNotExist` base with per-model `Model.DoesNotExist` subclasses; `ValidationError` for form/model validation.
- `json.JSONDecodeError` subclasses `ValueError`, so old code catching `ValueError` still works.

In the interview

> **30-second answer:** "Raise built-ins like `ValueError` / `TypeError` for contract violations; for domain errors create a base exception subclassing `Exception` plus specific subclasses with useful attributes. Inside a handler, bare `raise` re-throws with the original traceback."

- *Why a base exception class per library?* → Callers can catch everything from your library with one clause.
- *`raise` vs `raise e`?* → Bare `raise` re-raises the active exception unchanged; `raise e` re-raises the object from the current line.

Self-check

1. Which class should custom exceptions subclass? → `Exception` (or a subclass).
2. What does a bare `raise` outside any handler do? → `RuntimeError: No active exception to reraise`.
3. How do you attach data to a custom exception? → Set attributes in `__init__`.

TL;DR

`raise` built-ins for contract errors; custom hierarchy (base + subclasses, `...Error`) for domain errors; bare `raise` to re-throw.

9.4 Exception chaining and notes

What it is

When you catch a low-level error and raise a higher-level one, Python remembers the link:

Syntax	Stored in	Traceback says
`raise New(...)` inside an `except` (implicit)	`new.__context__`	"During handling of the above exception, another exception occurred"
`raise New(...) from orig` (explicit)	`new.__cause__`	"The above exception was the direct cause of the following exception"
`raise New(...) from None`	suppresses the context	Only the new exception is shown

`exc.add_note("…")` (3.11+) attaches extra context lines that appear under the message in the traceback — handy for "which row/file/request was this?".

Why it matters

Translating errors keeps your API clean (callers catch `ConfigError`, not whatever `KeyError` / `ValueError` your internals produced) *without* losing the root cause needed for debugging.

How it works

```python
class ConfigError(Exception):
    pass

def load_port(cfg):
    try:
        return int(cfg["port"])
    except (KeyError, ValueError) as exc:
        raise ConfigError("invalid 'port' setting") from exc
```

```
== exception chaining: raise ... from ... ==
  port = 8080
  ConfigError: invalid 'port' setting; __cause__ = KeyError('port')
  ConfigError: invalid 'port' setting; __cause__ = ValueError("invalid literal for int() with base 10:
'eighty'")
  implicit: __context__ = KeyError('missing')
  formatted traceback of an explicit chain (last lines):
    | Traceback (most recent call last):
    | KeyError: 'port'
    |
    | The above exception was the direct cause of the following exception:
    |
    | Traceback (most recent call last):
    | ConfigError: invalid 'port' setting

== add_note (3.11+) ==
   invalid literal for int() with base 10: 'x1' | notes: ['while reading row 7 of prices.csv']
```

(The script prints only the unindented traceback lines, to keep it short.)

Common mistakes

- Wrapping errors *without* `from` and without a message — the new error hides what really happened.
- Using `from None` everywhere — you throw away debugging information. Use it only when the original is genuinely noise.

In the interview

> **30-second answer:** "Raising inside an `except` chains implicitly via `__context__`; `raise X from e` sets `__cause__` and says 'direct cause' in the traceback; `from None` hides the chain. Use explicit chaining when translating low-level errors into your own exception types. `add_note` adds context in 3.11+."

Self-check

1. Attribute set by `raise A from b`? → `A.__cause__ = b`.
2. How do you hide the original exception? → `raise New(...) from None`.

TL;DR

Translate errors with `raise MyError(...) from exc`; add context with `add_note`.

9.5 EAFP vs LBYL

What it is

Two styles for dealing with things that might not be there:

- **LBYL — Look Before You Leap**: check first, then act. `if key in d: v = d[key]`.
- **EAFP — Easier to Ask Forgiveness than Permission**: just try, and handle the exception. `try: v = d[key] except KeyError: ...`.

Both terms are in the official Python glossary, which calls EAFP the common Python style.

Why it matters

EAFP avoids **race conditions** — the time-of-check-to-time-of-use (TOCTOU) gap. `if os.path.exists(p): open(p)` can still fail if another process deletes the file between the check and the open; `try: open(p) except FileNotFoundError:` has no such gap. EAFP also avoids doing work twice (the check *and* the lookup) and plays well with duck typing (just call the method rather than checking the type).

```python
stock = {"apple": 3}
qty = stock["pear"] if "pear" in stock else 0      # LBYL
try:                                               # EAFP
    qty = stock["pear"]
except KeyError:
    qty = 0
stock.get("pear", 0)                               # the idiomatic one-liner
```

```
  LBYL: 0
  EAFP: 0 | idiomatic: stock.get('pear', 0) = 0
```

Trade-offs

	EAFP	LBYL
Races (files, shared state)	Safe	Check can go stale
Speed when failure is **rare**	Fast (try is almost free when nothing is raised, 3.11+ "zero-cost")	Pays for the check every time
Speed when failure is **common**	Raising/catching is relatively expensive	Cheaper
Readability	Can hide which line failed if `try` is big	Explicit

When not to use it

Prefer LBYL when failure is the common case, when the "try" would have side effects you can't undo halfway, or when a dedicated API exists (`dict.get`, `getattr(obj, "x", default)`, `str.isdigit()` for simple inputs).

In the interview

> **30-second answer:** "LBYL checks before acting; EAFP attempts and catches. Python favours EAFP: it avoids race conditions like checking a file exists then opening it, and it's cheap when errors are rare. Use LBYL when failure is frequent or partial work is hard to undo."

Self-check

1. Why is `if os.path.exists(p): open(p)` fragile? → The file can vanish between check and open.
2. Idiomatic "get with default" for dicts? → `d.get(key, default)`.

TL;DR

Python prefers EAFP (try/except) — race-free and fast when errors are rare; use LBYL when failure is common.

9.6 `ExceptionGroup` and `except*` (Python 3.11+)

What it is

Sometimes **several** independent errors happen at once — concurrent tasks, validating many fields, running many cleanup callbacks. An `ExceptionGroup` (PEP 654) wraps a list of exceptions into one raisable object. `except*` handles the matching *part* of a group: each `except*` clause receives a sub-group of the matching exceptions, and anything unhandled is re-raised in a new group.

How it works

```
def run_all():
    errors = []
    for task, fn in [("a", lambda: 1 / 0), ("b", lambda: int("z")),
                     ("c", lambda: 42), ("d", lambda: [][1])]:
        try:
            fn()
        except Exception as exc:
            exc.add_note(f"task {task}")
            errors.append(exc)
    if errors:
        raise ExceptionGroup("some tasks failed", errors)

try:
    run_all()
except* ZeroDivisionError as eg:
    print("  math errors: ", [(repr(e), e.__notes__) for e in eg.exceptions])
except* (ValueError, IndexError) as eg:
    print("  input errors:", [repr(e) for e in eg.exceptions])
```

```
== ExceptionGroup and except* (3.11+) ==
  math errors:  [("ZeroDivisionError('division by zero')", ['task a'])]
  input errors: ['ValueError("invalid literal for int() with base 10: \'z\'")', "IndexError('list index out of
range')"]
```

Unlike plain `except`, more than one `except*` clause can run for one group.

Rules: `except` and `except*` can't be mixed in the same `try`; `eg` is always a group (use `eg.exceptions`); `ExceptionGroup` holds `Exception` subclasses, `BaseExceptionGroup` can hold any.

Real-world examples

- `asyncio.TaskGroup` (3.11+) raises an `ExceptionGroup` when child tasks fail.
- The Trio async library pioneered this model and was a key motivation for PEP 654.

When not to use it

Ordinary single errors — use regular `except`. Groups are for genuinely concurrent/independent failures.

In the interview

30-second answer: "An `ExceptionGroup` bundles several exceptions; `except*` lets each clause handle the matching subset, and unhandled ones are re-raised. It's used by `asyncio.TaskGroup` when multiple tasks fail."

Self-check

1. Can two `except*` clauses both run for one group? → Yes.
2. Where would you meet these in the stdlib? → `asyncio.TaskGroup`.

TL;DR

Many errors at once → `ExceptionGroup`; split-handle with `except*`.

9.7 `assert`

What it is

`assert condition, "message"` raises `AssertionError("message")` if `condition` is false. It documents and checks **internal assumptions** – things that should be impossible if your code is correct.

```
def mean(xs):
    assert len(xs) > 0, "mean() of empty list"
    return sum(xs) / len(xs)
```

```
  mean([2, 4]) = 3.0
  AssertionError: mean() of empty list (disabled entirely by python -O)
```

The crucial caveat

Running Python with `-O` (optimise) sets `__debug__` to `False` and **removes all `assert` statements**. So never use `assert` for input validation, security checks or anything that must happen in production – raise a real exception (`ValueError`) instead.

Common mistakes

- `assert (x > 0, "msg")` – a non-empty tuple is always truthy, so this never fails (CPython warns: "assertion is always true").
- Validating user input or permissions with `assert`.

Real-world examples

pytest is built on plain `assert` statements – it rewrites them to show detailed failure messages (`assert result == 5` → shows both sides).

In the interview

30-second answer: "`assert` checks internal invariants and raises `AssertionError`; it's stripped under `python -O`, so use it for debugging and tests, never for validating input."

Self-check

1. What does `python -O` do to asserts? → Removes them.
2. What should validate a function argument from a user? → An explicit `raise ValueError(...)`.

TL;DR

`assert` = developer sanity check and test tool; not validation.

9.8 Context managers and `with`

What it is

A **context manager** is an object that sets something up on entering a `with` block and **guarantees** teardown on leaving it – normally, by `return`/`break`, or by exception. Files, locks, database transactions, temporary directories and timers are typical.

```
with open("data.txt", encoding="utf-8") as f:   # file opened
    data = f.read()
# file closed here, even if read() raised
```

How it works — the protocol

`with EXPR as NAME: BODY` does roughly:

1. Evaluate `EXPR` to get the manager `m`.
2. Call `m.__enter__()`; bind its return value to `NAME`.
3. Run `BODY`.
4. Call `m.__exit__(exc_type, exc, tb)` — with `None, None, None` if the body succeeded, or the exception details if it failed.
5. If `__exit__` returns a **truthy** value, the exception is **suppressed**; otherwise it propagates.

It's `try/finally` packaged as a reusable object.

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        print("  __enter__: timer started")
        return self                     # bound to the name after 'as'

    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"  __exit__: exc_type={exc_type.__name__ if exc_type else None}")
        return False                    # let exceptions propagate
```

```
== class-based: normal exit ==
  __enter__: timer started
  __exit__: exc_type=None
  elapsed measured: True

== class-based: exception inside the block ==
  __enter__: timer started
  __exit__: exc_type=KeyError
  exception still reached caller: KeyError('boom')
```

`@contextlib.contextmanager` writes one as a generator: code before `yield` = enter, the yielded value = `as` target, code after = exit. Wrap the `yield` in `try` to react to errors:

```
@contextlib.contextmanager
def transaction(log):
    log.append("BEGIN")
    try:
        yield log
    except Exception:
        log.append("ROLLBACK")
        raise
    else:
        log.append("COMMIT")
```

```
  log: ['BEGIN', 'UPDATE accounts ...', 'COMMIT', 'BEGIN', 'UPDATE bad ...', 'ROLLBACK']
```

Other `contextlib` helpers used in the script:

Helper	Does	Output
`suppress(FileNotFoundError)`	Ignore listed exceptions in the block	`missing file ignored`
`closing(obj)`	Call `obj.close()` on exit	
`ExitStack()`	Manage a *dynamic number* of managers; exits them in **reverse** order	`open db, cache, queue` → `close queue, cache, db`
Parenthesised `with (A() as a, B() as b):`	Several managers in one statement (3.10+)	

Trade-offs

Class-based managers are best when the object has other methods too (a connection). `@contextmanager` is shortest for one-off setup/teardown. `ExitStack` when the number of resources is only known at runtime.

Common mistakes

- Opening files without `with` and forgetting `close()` – leaks file handles, and buffered writes may not reach disk.
- In a `@contextmanager` generator, forgetting `try/finally` around `yield` – the cleanup won't run if the body raises.
- Returning `True` from `__exit__` by accident → silently swallows every exception.

Real-world examples

- `threading.Lock()`: `with lock:` acquires and always releases.
- `sqlite3` connection: `with conn:` commits on success and rolls back on exception (it does **not** close the connection).
- PyTorch `with torch.no_grad():`, `unittest.mock.patch(...)`, `decimal.localcontext()`, `tempfile.TemporaryDirectory()`.

Decorators and context managers get a deeper treatment (async managers, reentrancy) in Chapter 12.

In the interview

> **30-second answer:** "A context manager implements `__enter__` and `__exit__`; `with` calls enter, runs the block, and always calls exit with any exception info – exit returning true suppresses it. It's a reusable try/finally for resources like files and locks. `contextlib.contextmanager` builds one from a generator; `ExitStack` handles a dynamic set."

- *What does `__exit__` receive?* → `exc_type, exc_value, traceback`, all `None` on success.
- *How do you suppress an exception in a manager?* → Return `True` from `__exit__` (or use `contextlib.suppress`).

Self-check

1. What is bound to `f` in `with open(p) as f`? → The return value of `__enter__` (the file object itself).
2. Order of exits in an `ExitStack`? → Reverse of entry (LIFO).
3. Does `with sqlite3.connect(...) as conn` close the connection? → No, it commits/rolls back.

TL;DR

`with` = guaranteed cleanup. `__enter__`/`__exit__`, or `@contextmanager` with `try` around `yield`. `suppress`, `closing`, `ExitStack` from `contextlib`.

9.9 File I/O – modes, text vs binary, encodings

What it is

`open(path, mode="r", encoding=None, newline=None)` returns a **file object**. In **text mode** it gives and takes `str`, decoding/encoding bytes with an **encoding** (a rule mapping characters to bytes, such as UTF-8). In **binary mode** (`"b"`) it gives and takes raw `bytes`.

Mode	Meaning	If file exists	If missing
`"r"` (default)	read	read from start	`FileNotFoundError`
`"w"`	write	**truncates** to empty	creates
`"a"`	append	writes go to the end	creates
`"x"`	exclusive create	`FileExistsError`	creates
`"+"` added (`"r+"`, `"w+"`)	read and write		
`"b"` added (`"rb"`, `"wb"`)	binary (bytes)		
`"t"` (default)	text (str)		

How it works

```
notes = base / "notes.txt"
with open(notes, "w", encoding="utf-8") as f:    # 'w' creates/truncates
    f.write("first line\n")
    f.writelines(["second line\n", "café ☕\n"])
with open(notes, "a", encoding="utf-8") as f:    # 'a' appends
    print("appended line", file=f)
with open(notes, encoding="utf-8") as f:         # default mode 'r'
    for i, line in enumerate(f, 1):              # lazy, line by line
        print(f"  {i}: {line.rstrip()}")
```

```
== writing and reading text ==
  1: first line
  2: second line
  3: café ☕
  4: appended line
  read() whole file -> 44 characters

== text vs bytes, encodings ==
  bytes on disk: 47 (é is 2 bytes, ☕ is 3 bytes in UTF-8)
  first 12 bytes: b'first line\ns'
  ascii decode -> UnicodeDecodeError ordinal not in range(128) at byte 26
  latin-1 (wrong codec) mangles: 'cafÃ© â\x98\x95'
  errors='replace': caf□□ □□□

== binary mode, seek/tell ==
  tell after seek(4): 4 | read(3): [4, 5, 6]

== 'x' mode refuses to overwrite ==
  FileExistsError: File exists
```

Reading methods:

Method	Returns	Memory
`for line in f:`	one line at a time (with `\n`)	Constant — best for big files
`f.read()` / `f.read(n)`	whole file / up to `n` chars (bytes in binary)	Whole file
`f.readline()`	next line	One line
`f.readlines()`	list of all lines	Whole file
`f.write(s)`	number of characters written	–
`f.seek(pos)`, `f.tell()`	move / report position	–

Key facts:

1. **Always pass** `encoding="utf-8"` in text mode. Without it Python uses the platform's locale encoding, which differs between machines (historically cp1252 on many Windows systems) — the file you wrote on macOS may not read on Windows. PEP 686 plans to make UTF-8 the default in Python 3.15; until then, be explicit (or run with `-X utf8`).
2. The wrong codec either fails (`UnicodeDecodeError`) or, worse, silently produces **mojibake** (`cafÃ©`). `errors="replace"` / `"ignore"` trade errors for data loss — last resorts.
3. Text mode translates newlines: on reading, `\r\n` and `\r` become `\n` ("universal newlines"); on writing, `\n` becomes the platform's separator. Binary mode never changes bytes.
4. Writes are **buffered**; data is guaranteed on disk after `close()` (or `flush()` + `os.fsync()` for durability).
5. Images, zip files, pickles and anything non-text → binary mode.
6. **File-like objects**: anything with `read()` / `write()` can stand in for a file. `io.StringIO("text")` is an in-memory text file and `io.BytesIO(b"...")` an in-memory binary file — handy in tests and for libraries that expect a file (`csv.reader(io.StringIO(data))`, `pd.read_csv(io.StringIO(...))`).

Common mistakes

- Opening with `"w"` when you meant `"a"` — the file is wiped instantly.
- Omitting `encoding=`.
- `f.read()` on a multi-GB log file — iterate lines instead.
- Building paths with string `+` (`folder + "/" + name`) — use `pathlib`.

Real-world examples

Log processors stream files line by line; `pandas.read_csv(..., encoding="utf-8")` exposes the same encoding choice; almost every "garbled characters" bug report in data pipelines is an encoding mismatch.

In the interview

> **30-second answer:** "`open` returns a file object; use it in a `with` block. Text mode works with `str` and needs an encoding — always pass `utf-8` explicitly; binary mode works with `bytes`. `w` truncates, `a` appends, `x` fails if the file exists. Iterate over the file for large inputs."

- *Difference between `"w"` and `"x"`?* → `w` truncates an existing file; `x` raises `FileExistsError`.
- *Why can 'é' be 2 bytes?* → UTF-8 is variable-width: 1 byte for ASCII, 2–4 for other characters.

Self-check

1. Mode for appending bytes? → `"ab"`.
2. What exception for decoding UTF-8 bytes as ASCII? → `UnicodeDecodeError`.
3. Memory-friendly way to process a huge text file? → `for line in f:`.

TL;DR

`with open(p, mode, encoding="utf-8")`. Text = `str` + encoding, binary = `bytes`. `w` truncates, `a` appends, `x` is safe-create. Stream big files line by line.

9.10 `pathlib` , `shutil` , `tempfile`

What it is

- `pathlib.Path` – an object-oriented path. Join with `/`, inspect parts, and read/write/list without juggling strings or `os.path` functions. Works across Windows and POSIX.
- `shutil` ("shell utilities") – high-level operations on files and whole trees: copy, move, delete trees, make archives.
- `tempfile` – safely creates temporary files and directories with unique names, cleaned up automatically.

How it works – `pathlib`

```
p = base / "reports" / "2026" / "q3.final.csv"
p.name, p.stem, p.suffix, p.suffixes
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text("id,total\n1,9.5\n", encoding="utf-8")
(base / "reports").glob("*/*.csv")
base.rglob("*.md")
```

```
== pathlib ==
  name: q3.final.csv | stem: q3.final | suffix: .csv | suffixes: ['.final', '.csv']
  parent relative to base: reports/2026
  with_suffix('.json'): q3.final.json
  exists: True | is_file: True | size: 15 bytes
  glob('*/*.csv'):    ['q2.csv', 'q3.final.csv']
  rglob('*.md'):      ['readme.md']
  iterdir(reports):   ['2026', 'readme.md']
```

Task	`pathlib`	older `os` / `os.path`
Join	`base / "a" / "b.txt"`	`os.path.join(base, "a", "b.txt")`
Current dir / home	`Path.cwd()` , `Path.home()`	`os.getcwd()` , `os.path.expanduser("~")`
Absolute, symlinks resolved	`p.resolve()`	`os.path.realpath(p)`
Parts	`p.name` , `.stem` , `.suffix` , `.parent`	`os.path.basename` , `splitext` , `dirname`
Exists / type	`p.exists()` , `.is_file()` , `.is_dir()`	`os.path.exists` …
Read / write whole file	`p.read_text()` , `p.write_bytes(b)`	`open(...)`
Make dirs	`p.mkdir(parents=True, exist_ok=True)`	`os.makedirs(p, exist_ok=True)`
Delete file / empty dir	`p.unlink(missing_ok=True)` , `p.rmdir()`	`os.remove` , `os.rmdir`
Rename / replace	`p.rename(t)` , `p.replace(t)`	`os.rename` , `os.replace`
List / search	`p.iterdir()` , `p.glob("*.py")` , `p.rglob("*.py")` , `p.walk()` (3.12+)	`os.listdir` , `glob.glob` , `os.walk`

`open()` and nearly every library accept `Path` objects directly.

How it works – `shutil` and `tempfile`

```
== shutil: copy, move, archive, delete trees ==
  copytree -> ['2026', '2026/q2.csv', '2026/q3.final.csv', 'readme.md']
  moved readme -> True
  archive: reports_bak.zip True
  rmtree backup -> exists? False
  disk_usage works: True

== tempfile ==
  temp file .txt contains: scratch
  deleted after close: True

== atomic replace pattern ==
  settings now: {"v": 2}

temp dir removed: True
```

`shutil` function	Does
`copy(src, dst)` / `copy2`	Copy a file (+ permission bits / + timestamps and other metadata)
`copytree(src, dst)`	Copy a whole folder tree
`move(src, dst)`	Move/rename, across filesystems if needed
`rmtree(path)`	Delete a folder **and everything in it** – permanent, no recycle bin
`make_archive(base, "zip", root_dir)` / `unpack_archive`	Create/extract zip, tar, gztar…
`which("git")` , `disk_usage(path)`	Find an executable on `PATH` ; total/used/free bytes

`tempfile` API	Gives
`TemporaryDirectory()`	A folder deleted (with contents) when the `with` block ends
`NamedTemporaryFile()`	A file with a real name, deleted on close by default (3.12 adds `delete_on_close=False` for reopening it by name, needed on Windows)
`TemporaryFile()`	Anonymous file – may have no visible name at all
`mkstemp()` / `mkdtemp()`	Low-level: create and return path; **you** delete it

Atomic replace pattern: write the new content to a temp file *in the same directory*, then `os.replace(tmp, target)` . The rename is atomic on the same filesystem, so readers see either the old or the new file – never a half-written one. Tools that save config and state files commonly use this.

Trade-offs

`pathlib` is the modern default; `os` / `os.path` remain for low-level calls (`os.replace` , `os.fsync` , file descriptors) and older code. `shutil` operations are convenient but not transactional – a `copytree` interrupted halfway leaves a partial copy.

Common mistakes

- Hard-coding `"/tmp/myfile"` – predictable names are a security risk and not portable; use `tempfile` .
- `shutil.rmtree` on a path built from user input without checking it.
- Using `str` concatenation or backslashes for paths.
- `Path("a").glob("**/*.py")` vs `rglob("*.py")` – equivalent; `rglob` is shorter.

Real-world examples

pytest's `tmp_path` fixture hands each test a fresh `pathlib.Path` temp directory. Build tools use `shutil.copytree` / `make_archive` for packaging; `shutil.which` is how scripts check that `git` or `ffmpeg` is installed.

In the interview

> **30-second answer:** "`pathlib.Path` is the object-oriented way to build and inspect paths and to read, write, list and glob; `shutil` does high-level file and tree operations like copy, move, rmtree and archives; `tempfile` creates unique temp files and directories that clean themselves up. For safe file updates, write a temp file in the same folder and `os.replace` it."

Self-check

1. Create `a/b/c` even if `a` doesn't exist and don't fail if `c` does? → `Path("a/b/c").mkdir(parents=True, exist_ok=True)`.
2. `Path("x.tar.gz").suffix`? → `".gz"` (`.suffixes` → `['.tar', '.gz']`).
3. Why write to a temp file then `os.replace`? → Atomic update; no half-written file.

TL;DR

`Path` for paths, `shutil` for copying/moving/deleting trees, `tempfile` for throwaway files; atomic writes via temp + `os.replace`.

9.11 `csv`, `json`, `pickle`

What it is

Three standard ways to store structured data in files:

Format	Module	Looks like	Holds	Readable by other languages	Safe to load untrusted data
CSV – comma-separated values	`csv`	Text table	Rows of **strings**	Yes (Excel, databases)	Yes
JSON – JavaScript Object Notation	`json`	Text	dict, list, str, int, float, bool, None	Yes (the web's lingua franca)	Yes
pickle	`pickle`	Binary	Almost any Python object	**No** – Python only	**NO** – can run arbitrary code

CSV

Use the `csv` module, not `line.split(",")` – real CSV has quoted fields containing commas, quotes and even newlines.

```
with open(path, "w", newline="", encoding="utf-8") as f:   # newline="" per csv docs
    w = csv.DictWriter(f, fieldnames=["id", "customer", "total", "placed"])
    w.writeheader()
    for o in orders:
        w.writerow(asdict(o))
with open(path, newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
```

```
raw file:
  id,customer,total,placed
  1,"Ana, Inc.",19.5,2026-09-01
  2,"Bob ""B"" Ltd",5.25,2026-09-02
first row: {'id': '1', 'customer': 'Ana, Inc.', 'total': '19.5', 'placed': '2026-09-01'}
total is a str -> convert yourself: 24.75
naive split(',') breaks on quoted commas: ['1', '"Ana', ' Inc."', '19.5', '2026-09-01']
tab-separated: [['a', 'b'], ['1', '2']]
```

- The writer quoted `"Ana, Inc."` and doubled the inner quotes in `"Bob ""B"" Ltd"` automatically.
- **Everything reads back as `str`** – convert types yourself.
- Open with `newline=""` (the docs require it) so the `csv` module handles line endings itself; otherwise you can get blank lines between rows on Windows or broken quoted newlines.
- `csv.reader` / `csv.writer` work with lists; `DictReader` / `DictWriter` with dicts keyed by the header; `delimiter="\t"` for TSV; `csv.Sniffer` guesses a dialect.
- For analysis-sized data, `pandas.read_csv` is usually the better tool.

JSON

Python → JSON	JSON → Python
`dict` → object	object → `dict`
`list`, `tuple` → array	array → `list`
`str` → string	string → `str`
`int`, `float` → number	number → `int` or `float`
`True` / `False` → `true` / `false`	→ `True` / `False`
`None` → `null`	→ `None`

`dumps` / `loads` work with strings; `dump` / `load` with files.

```
text = json.dumps(payload, default=to_json_default, indent=2, ensure_ascii=False)
back = json.loads(text)
```

```
first lines of dumps():
  {
    "orders": [
      {
        "id": 1,
        "customer": "Ana, Inc.",
        "total": 19.5,
        "placed": "2026-09-01"
      },
loads() types: paid -> bool | discount -> None | placed -> str
load() from file, customers: ['Ana, Inc.', 'Bob "B" Ltd']
tuple becomes list: {'pt': [1, 2]}
int keys become str: {'1': 'a'}
without default=: Object of type date is not JSON serializable
JSONDecodeError: Expecting property name enclosed in double quotes at pos 1
```

- Types JSON doesn't know (`date`, `Decimal`, `set`, your classes) need a `default=` function (or pre-conversion, e.g. `dataclasses.asdict`); they come back as plain strings/lists – the round trip is **lossy** for types.
- Tuples become lists; non-string keys become strings.
- JSON requires double quotes; `'single'` quotes (a Python repr) are invalid → `json.JSONDecodeError` (a `ValueError` subclass).

- `indent=2` pretty-prints; `ensure_ascii=False` keeps "é" readable instead of the escape `\u00e9`; `sort_keys=True` gives stable output. `python -m json.tool file.json` pretty-prints from the shell.
- By default `json.dumps(float("nan"))` writes `NaN`, which is **not** valid JSON for strict parsers; pass `allow_nan=False` to get an error instead.

pickle

`pickle` serialises almost any Python object graph (custom classes, dates, nested references) to bytes and back.

```
blob = pickle.dumps(orders)
restored = pickle.loads(blob)
with open(base / "orders.pkl", "wb") as f:                # binary mode!
    pickle.dump(orders, f)
```

```
  round trip equal: True | type: Order | placed: 2026-09-01
  load() from file: 2 orders
```

Security caveat — never unpickle data you don't fully trust. The pickle format lets an object specify, via `__reduce__`, a function to *call* when it's loaded. The script demonstrates with a harmless `print`; an attacker would use `os.system` or similar:

```
class Sneaky:
    def __reduce__(self):
        return (print, ("  !!! code ran during pickle.loads() !!!",))

pickle.loads(pickle.dumps(Sneaky()))
```

```
  loading a crafted pickle...
  !!! code ran during pickle.loads() !!!
  loads() returned: None
```

The code ran merely by *loading*. The official docs carry a prominent warning to this effect.

Other pickle limits: Python-only; tied to your class definitions (rename a class and old pickles break); lambdas, open files, sockets and locks can't be pickled. The current protocol is 5 (`pickle.HIGHEST_PROTOCOL`).

Trade-offs / when to use which

- **Config, APIs, data exchange** → JSON (or TOML for human-edited config, which Python reads with `tomllib`).
- **Tabular data for spreadsheets/other tools** → CSV (or Parquet via pandas for big data).
- **Short-lived caching of Python objects you created yourself** (e.g. between steps of your own pipeline, `multiprocessing` internals) → pickle.
- **Never** pickle for data crossing a trust boundary (uploads, network, downloaded files).

Common mistakes

- Opening pickle files in text mode (`"w"`) → `TypeError`; always `"wb"`/`"rb"`.
- Forgetting `newline=""` with `csv`.
- Expecting `json.loads` to give back `datetime`s.
- Loading downloaded model files with plain `pickle.load`.

Real-world examples

- **PyTorch** checkpoints (`torch.save`) use pickle; since PyTorch 2.6, `torch.load` defaults to `weights_only=True` to block arbitrary code during loading, and Hugging Face created the **safetensors** format as a pickle-free way to share model weights.
- scikit-learn documents pickle/joblib for model persistence with the same security warning, and points to the `skops` format as a safer option.

- REST APIs (GitHub, Stripe) speak JSON; spreadsheet exports are CSV.

In the interview

> **30-second answer:** "CSV is text rows of strings – use the `csv` module with `newline=''` because of quoting. JSON maps to dict/list/str/number/bool/None, is language-neutral and safe; other types need a `default=` hook and come back as plain values. Pickle stores arbitrary Python objects in binary but is Python-only and unsafe: loading can execute code via `__reduce__`, so never unpickle untrusted data."

- *Why is `pickle.loads` dangerous?* → A pickle can instruct the loader to call arbitrary callables.
- *How do you JSON-encode a `datetime`?* → `default=` function returning `isoformat()`, or a custom `JSONEncoder`.
- `json.dump` *vs* `json.dumps`*?* → File vs string.

Self-check

1. What type are all values from `csv.DictReader`? → `str`.
2. What does `json.loads(json.dumps((1, 2)))` return? → `[1, 2]`.
3. Mode to write a pickle? → `"wb"`.
4. Safe alternative to pickle for sharing ML weights? → safetensors (or `torch.load(..., weights_only=True)`).

TL;DR

CSV = strings in rows (`csv` module, `newline=""`). JSON = safe, portable, limited types. Pickle = any Python object, binary, **never from untrusted sources**.

Chapter TL;DR

- Exceptions are objects that unwind the stack until an `except` matches by class; `Exception` is the base for application errors, `BaseException` also covers exit and Ctrl-C.
- `try` small, `except` specific (first match wins), `else` on success, `finally` always; never a bare `except:`; the `as e` name is deleted after the handler.
- `raise` built-ins for contract errors, a custom hierarchy for domain errors, bare `raise` to re-throw, `raise X from e` to translate while keeping the cause, `add_note` for context.
- Python prefers **EAFP** (try it) over LBYL (check first) – race-free and fast when failures are rare.
- `ExceptionGroup` + `except*` handle several simultaneous errors (e.g. `asyncio.TaskGroup`).
- `assert` checks invariants and vanishes under `-O` – never for validation.
- `with` + context managers guarantee cleanup (`__enter__` / `__exit__`, `@contextmanager`, `ExitStack`).
- `open(path, mode, encoding="utf-8")` inside `with`; text vs binary; `w` truncates, `a` appends, `x` refuses to overwrite; iterate big files.
- `pathlib` for paths, `shutil` for trees and archives, `tempfile` for throwaway files, temp + `os.replace` for atomic writes.
- CSV (strings, `newline=""`), JSON (safe, portable, limited types), pickle (any object, Python-only, **unsafe on untrusted data**).

Chapter 10 — Object-Oriented Programming in Python

What you will learn. How Python builds objects: classes and instances, `__init__` and `self`, class vs instance attributes, the three kinds of method, and properties. Then the *data model* — the special "dunder" methods (`__repr__`, `__eq__`, `__hash__`, `__lt__`, `__len__`, `__iter__`, `__getitem__`, `__call__`, `__enter__` …) that let your own classes work with `len()`, `for`, `==`, `+`, `with` and `sorted()`. Then reuse: inheritance and `super()`, the method resolution order (MRO) and multiple inheritance, mixins, abstract base classes, duck typing and `Protocol`, and when composition beats inheritance. Finally the modern toolbox — `dataclasses`, `enum`, `__slots__` — encapsulation conventions, and short Pythonic notes on design patterns.

Prerequisites. Functions, basic types, exceptions. SOLID and the Gang-of-Four patterns are covered language-neutrally in the LLD Handbook; this chapter focuses on *how Python does OOP*.

Code. Every script is in `chapters/10-oop-in-python/code/` and runs on Python 3.12+ with no dependencies:

```
python chapters/10-oop-in-python/code/data_model.py
```

10.1 Classes and instances, `__init__` and `self`

What it is

A **class** is a blueprint: it bundles data (**attributes**) and behaviour (**methods**) under one name. An **instance** (or *object*) is one concrete thing built from that blueprint. Analogy: the class is a cookie cutter; each cookie is an instance — same shape, different sprinkles.

In Python *everything* is an object — ints, functions, modules, and classes themselves (a class is an instance of `type`).

Why it matters

Classes let you keep related state and the code that changes it together, so invariants ("balance never goes negative") are enforced in one place instead of scattered across the codebase.

How it works

1. `class BankAccount: ...` executes the class body once and creates a class object.
2. Calling the class, `BankAccount("Asha", 100)`, does two steps: `__new__` allocates an empty instance, then `__init__` initialises it. You almost never write `__new__` (it is for immutable types and metaclass tricks).
3. `__init__(self, ...)` receives the new instance as its first argument. By convention it is named `self` — it is just a normal parameter, not a keyword.
4. `self.owner = owner` stores an **instance attribute** in the instance's own namespace, the dict `obj.__dict__`.
5. `__init__` must return `None`; it *initialises*, it does not *construct*.

Class attributes vs instance attributes

A **class attribute** is defined in the class body and shared by all instances. An **instance attribute** is set on `self` and belongs to one object.

Attribute lookup for `obj.name` (simplified): look in `obj.__dict__` first, then in the class, then in each parent class along the MRO (§10.5). Assigning `obj.name = ...` always writes to the *instance* — it never modifies the class attribute; it *shadows* it.

The three kinds of method

Kind	Decorator	First parameter	Typical use
Instance method	none	`self` (the instance)	Read/modify one object's state
Class method	`@classmethod`	`cls` (the class)	Alternative constructors (`from_csv`, `dict.fromkeys`, `datetime.fromtimestamp`); works correctly for subclasses because `cls` is the subclass
Static method	`@staticmethod`	none	Helper that logically belongs to the class but needs neither instance nor class

When you write `a.deposit`, Python returns a **bound method**: the function with `self` already filled in. So `a.deposit(5)` is exactly `BankAccount.deposit(a, 5)`. (Chapter 12 explains the mechanism — functions are *descriptors*.)

Properties

A **property** makes a method look like an attribute. You start with a plain public attribute; if you later need validation or a computed value, you turn it into a property and **no caller changes**. This is why Python code does not need Java-style `get_x()/set_x()` methods.

```
class BankAccount:
    bank_name = "PyBank"          # class attribute: shared by every instance
    _next_id = 1                  # class attribute used as a counter

    def __init__(self, owner: str, balance: float = 0.0) -> None:
        self.owner = owner        # instance attributes: one copy per object
        self._balance = balance   # leading underscore = "internal, please don't touch"
        self.id = BankAccount._next_id
        BankAccount._next_id += 1

    def deposit(self, amount: float) -> None:          # instance method
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

    @property
    def balance(self) -> float:                        # read like an attribute
        return self._balance

    @balance.setter
    def balance(self, value: float) -> None:           # validated assignment
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value

    @classmethod
    def from_csv(cls, line: str) -> "BankAccount":     # alternative constructor
        owner, balance = line.split(",")
        return cls(owner, float(balance))

    @staticmethod
    def is_valid_amount(amount: float) -> bool:        # plain helper
        return amount > 0


class Buggy:
    items = []                    # MISTAKE: one list shared by all instances
    def add(self, x):
        self.items.append(x)
```

What the demo does: builds two accounts (one via the class method), shows that assigning `a.bank_name` creates an instance attribute that shadows the class one, shows the setter rejecting a negative value, proves `a.deposit` is bound to `a`, and shows the shared-mutable-class-attribute bug.

Output (`classes_basics.py`):

```
a: Asha 125 id 1
b: Ben 50.0 id 2
static check: False
a.bank_name: PyBank
after assignment -> a: Shadow | b: PyBank | class: PyBank
a.__dict__: {'owner': 'Asha', '_balance': 125, 'id': 1, 'bank_name': 'Shadow'}
setter refused: balance cannot be negative
a.deposit is bound: True
after unbound call: 130
Buggy y.items: [1]
Fixed q.items: []
```

Trade-offs

Choice	Good for	Watch out
Plain public attribute	Simple data, zero overhead	Can't validate — but you can switch to a property later without breaking callers
Property	Validation, computed/derived values, read-only views	Hides work behind attribute syntax; keep it cheap (no network calls)
Class attribute	Constants, defaults, counters shared by all	Mutable class attributes are shared — a classic bug
`@classmethod` vs `@staticmethod`	`cls` makes alternative constructors subclass-friendly	A static method that uses the class name directly breaks for subclasses

When not to use a class

If you have one function and no state, write a function. If you have data and no behaviour, use a `dataclass`, `NamedTuple` or `dict`. A class with only `__init__` and one other method is often a function in disguise (Jack Diederich's PyCon talk "Stop Writing Classes" makes this point).

Common mistakes

- Forgetting `self` in a method signature → `TypeError: takes 0 positional arguments but 1 was given`.
- Mutable class attributes (`items = []`) shared by every instance — initialise in `__init__`.
- Mutable default arguments (`def __init__(self, items=[])`) — the default list is created once. Use `None` and create inside.
- Updating a counter via `self._next_id += 1` — that creates an instance attribute; update the class: `type(self)._next_id += 1` or `BankAccount._next_id += 1`.
- A property and an attribute with the same name (`self.balance = x` inside the `balance` setter) → infinite recursion. Store in `_balance`.

Real-world examples

- `datetime.datetime.fromtimestamp`, `dict.fromkeys`, `int.from_bytes` are class-method alternative constructors.
- `pathlib.Path.name`, `.suffix`, `.parent` are properties computed from the path string.
- Django model fields are declared as class attributes; each instance holds its own values.

In the interview

> **30-second answer:** "A class is a blueprint; instances are built by `__new__` then initialised by `__init__`, which gets the instance as `self`. Class attributes live on the class and are shared; instance attributes

> live in each object's `__dict__`, and lookup checks the instance first, then the class and its parents. Instance methods take `self`, class methods take `cls` and are used for alternative constructors, static methods take neither. Properties let me add validation later without changing the public API."

- *Why is `self` explicit?* – "Explicit is better than implicit": a method is just a function stored on the class; `obj.m()` passes `obj` as the first argument. It also makes instance vs local variables obvious.
- *Class method or static method for a factory?* – Class method, so `SubClass.from_csv(...)` returns a `SubClass`.
- *What's `__new__` for?* – Controlling creation itself: subclassing immutables (`int`, `str`, `tuple`), caching instances, metaclasses.

Self-check

1. After `a.bank_name = "X"`, what is `BankAccount.bank_name`? → Unchanged; `a` now has its own instance attribute that shadows it.
2. Why does `Buggy` leak items between instances? → `items` is one list on the class; `self.items.append` mutates it (no assignment, so no shadowing).
3. What does `a.deposit` evaluate to? → A bound method object whose `__self__` is `a`.
4. When would you convert an attribute into a property? → When you need validation, computation, or read-only access – callers don't change.

TL;DR

Classes bundle state and behaviour; `__init__` initialises, `self` is the instance. Instance attributes are per object, class attributes are shared (beware mutable ones). Instance/class/static methods take `self`/`cls`/nothing. Start with public attributes; upgrade to properties when needed.

10.2 The data model: dunder methods

What it is

The **Python data model** is the set of special methods – named with **d**ouble **under**scores, hence "dunder" (e.g. `__len__`) – that the interpreter calls behind the scenes. `len(x)` calls `x.__len__()`, `a + b` calls `a.__add__(b)`, `for` calls `__iter__`, `with` calls `__enter__`/`__exit__`. Implement them and your objects behave like built-ins. *Fluent Python* calls this Python's "metaobject protocol"; it is why Python code reads consistently.

Why it matters

Users of your class get a familiar interface for free: `len(playlist)`, `playlist[0]`, `sorted(vectors)`, `v1 + v2`, `print(v)`. You also plug into the entire standard library (`sorted`, `max`, `set`, `dict`, `random.choice`, `in`) without writing extra methods.

How it works — the most important dunders

You write	Python calls	Notes
`repr(x)`, REPL echo, inside containers	`__repr__`	Unambiguous, for developers; ideally looks like the constructor call
`str(x)`, `print(x)`, `f"{x}"`	`__str__`	Readable, for users; **falls back to** `__repr__` if missing
`x == y`, `x != y`	`__eq__`, `__ne__`	Default is identity (`is`); `__ne__` inverts `__eq__` automatically
`hash(x)`, dict key, set member	`__hash__`	Must agree with `__eq__` (see below)
`x < y` etc.	`__lt__`, `__le__`, `__gt__`, `__ge__`	`@functools.total_ordering` derives the rest from `__eq__` + one of them
`len(x)`	`__len__`	Must return an int ≥ 0
`bool(x)`, `if x:`	`__bool__`	Falls back to `__len__() != 0`, else always `True`
`for i in x`, `iter(x)`	`__iter__`	Falls back to `__getitem__(0), (1), …` until `IndexError`
`x[k]`, `x[a:b]`	`__getitem__`	Receives an int, key, or a `slice` object
`x[k] = v`, `del x[k]`	`__setitem__`, `__delitem__`	
`v in x`	`__contains__`	Falls back to iterating
`x(args)`	`__call__`	Makes instances callable
`with x as y:`	`__enter__`, `__exit__`	Covered fully in Chapter 12
`a + b`, `a * b`, `abs(a)`, `-a`	`__add__`, `__mul__`, `__abs__`, `__neg__`	Operator overloading (§10.3)

`__eq__` and `__hash__` — the contract

A **hash** is an integer that dicts and sets use to find a slot quickly. The rule: **if** `a == b` **then** `hash(a) == hash(b)` (the reverse need not hold — collisions are allowed).

1. By default objects compare and hash by identity — always consistent.
2. If you define `__eq__` but not `__hash__`, Python sets `__hash__ = None`, making instances **unhashable** (you can't put them in a set). This protects you from breaking the rule.
3. To make them hashable, define `__hash__` from the *same* fields `__eq__` uses — typically `hash((self.x, self.y))`.
4. Only hash **immutable** state. If a key's fields change after insertion into a dict, the dict looks in the wrong slot and "loses" it. That's why `frozen=True` dataclasses (§10.10) are the easy way to get safe hashing.

`NotImplemented`

When a comparison or arithmetic method can't handle the other operand, **return** (don't raise) the singleton `NotImplemented`. Python then tries the reflected method on the other object (`b.__radd__(a)`, or for `==` it tries `b.__eq__(a)`), and raises a clear `TypeError` only if both sides give up.

```python
@total_ordering
class Vector:
    __match_args__ = ("x", "y")

    def __init__(self, x: float, y: float) -> None:
        self.x, self.y = x, y

    def __repr__(self) -> str:
        return f"Vector({self.x!r}, {self.y!r})"

    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Vector):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self) -> int:
        return hash((self.x, self.y))

    def __lt__(self, other: "Vector") -> bool:     # total_ordering adds <=, >, >=
        if not isinstance(other, Vector):
            return NotImplemented
        return abs(self) < abs(other)

    def __add__(self, other): ...                  # see §10.3
    def __abs__(self) -> float:
        return math.hypot(self.x, self.y)
    def __bool__(self) -> bool:
        return bool(self.x or self.y)
    def __iter__(self):                            # enables x, y = v
        yield self.x
        yield self.y


class Playlist:
    def __init__(self, *songs: str) -> None:
        self._songs = list(songs)
    def __len__(self) -> int:
        return len(self._songs)
    def __getitem__(self, index):                  # ints AND slices
        result = self._songs[index]
        return Playlist(*result) if isinstance(index, slice) else result
    def __contains__(self, song: str) -> bool:
        return song in self._songs


class Multiplier:
    def __init__(self, factor: int) -> None:
        self.factor, self.calls = factor, 0
    def __call__(self, x: int) -> int:             # instance used like a function
        self.calls += 1
        return x * self.factor


class Timer:
    def __enter__(self):
        print("  enter: start timer")
        return self                                 # bound to the `as` name
    def __exit__(self, exc_type, exc, tb) -> bool:
        print(f"  exit: stop timer (exception={exc_type.__name__ if exc_type else None})")
        return False                                # don't swallow exceptions
```

Output (`data_model.py`):

```
repr: Vector(3, 4) | str: (3, 4) | f-string: (3, 4)
v + w = (4, 5) | v * 2 = (6, 8) | 2 * v = (6, 8)
abs(v) = 5.0 | bool(Vector(0,0)) = False
v == Vector(3,4): True | v is Vector(3,4): False
w < v: True | v >= w: True (>= from total_ordering)
set dedups equal vectors: {Vector(1, 2)}
sorted: [Vector(1, 1), Vector(0, 2), Vector(3, 4)]
unpacked: 3 4
pattern matched, y = 4
TypeError: unsupported operand type(s) for +: 'Vector' and 'int'
len: 4 | p[0]: intro | p[-1]: outro | p[1:3]: Playlist('verse', 'chorus')
'chorus' in p: True | iterate via __getitem__: ['INTRO', 'VERSE', 'CHORUS', 'OUTRO']
callable: 15 21 calls = 2 | callable(): True
  enter: start timer
  inside block, t is a Timer: True
  exit: stop timer (exception=None)
  enter: start timer
  exit: stop timer (exception=KeyError)
  KeyError still propagated
__hash__ of class defining only __eq__: None
```

Notice: `Playlist` has no `__iter__`, yet `for` works through the `__getitem__` fallback, and negative indexes and slices work because we delegate to the inner list. `__match_args__` lets `match v: case Vector(3, y)` destructure positionally. The last line proves the `__eq__`-without-`__hash__` rule.

Note: this teaching `Vector` is mutable *and* hashable — acceptable only if you never mutate a vector used as a dict key. In production, make it immutable (frozen dataclass or read-only properties).

Trade-offs

Approach	Pros	Cons
Implement dunders by hand	Full control, any semantics	Boilerplate, easy to get `__eq__` / `__hash__` inconsistent
`@dataclass`	Generates `__init__`, `__repr__`, `__eq__` (and optionally ordering/hash)	Field-by-field semantics only
Inherit from `collections.abc.Sequence` / `Mapping`	Implement 2–3 abstract methods, get the rest (`index`, `count`, `__contains__`, `__iter__`, `__reversed__`) as mixin methods	Ties you to that ABC
Subclass `list` / `dict` directly	Quick	Built-in C methods often bypass your overrides (e.g. `dict.update` does not call your `__setitem__`); prefer `collections.UserDict` / `UserList`

When not to use it

Don't overload operators with surprising meanings (`+` that deletes a file). Don't add `__call__` just to look clever — a named method is clearer unless the object genuinely *is* a function-like thing (a model, a strategy, a validator).

Common mistakes

- Calling dunders directly (`x.__len__()`) instead of `len(x)` — the built-in is faster for built-in types and applies extra checks.
- `raise NotImplementedError` inside `__eq__` / `__add__` instead of `return NotImplemented`.
- Defining `__eq__` and being surprised that objects are no longer hashable.
- `__repr__` that returns something useless like `<object>`; include the key fields.

- Returning a non-bool/non-int from `__len__` / `__bool__` → `TypeError`.

Real-world examples

- `pathlib.Path` overloads `/` (`__truediv__`) to join paths: `Path("a") / "b"`.
- NumPy arrays and pandas DataFrames overload arithmetic, comparison and `__getitem__` (boolean masks, slicing).
- `requests.Session`, `open()` files, `threading.Lock`, `sqlite3.Connection` implement `__enter__`/`__exit__`.
- PyTorch `nn.Module` implements `__call__`, which runs hooks and then your `forward()` – that's why you write `model(x)`, not `model.forward(x)`.
- SQLAlchemy overloads `==`, `<` on columns to build SQL expressions (`User.age > 18`).

In the interview

> **30-second answer:** "The data model is Python's protocol of special methods: built-ins and syntax call dunders – `len` calls `__len__`, `+` calls `__add__`, `for` calls `__iter__`. Implementing them makes custom classes behave like built-ins and work with the standard library. Key rules: `__repr__` for developers, `__str__` falls back to it; `__eq__` and `__hash__` must agree and defining `__eq__` alone makes the class unhashable; return `NotImplemented` for unsupported operands."

- *Why must equal objects have equal hashes?* – Dicts/sets use the hash to pick a bucket, then `==` to confirm. Equal objects with different hashes land in different buckets and duplicates appear.
- *Minimum to make a class iterable?* – `__iter__` returning an iterator, or `__getitem__` accepting 0, 1, 2… and raising `IndexError` at the end.
- *`__str__` vs `__repr__`?* – `repr` is unambiguous and used in debugging/containers; `str` is user-facing; `str` falls back to `repr`, not vice versa – so always write `__repr__` first.

Self-check

1. What does `print([v])` use, `__str__` or `__repr__`? → `__repr__` (containers show their items' reprs).
2. You defined `__eq__` only. What happens with `{obj}`? → `TypeError: unhashable type`.
3. Which methods does `total_ordering` need? → `__eq__` plus one of `__lt__`, `__le__`, `__gt__`, `__ge__`.
4. Why did `for s in p` work without `__iter__`? → The legacy sequence protocol: Python calls `__getitem__` with 0, 1, 2 … until `IndexError`.

TL;DR

Dunders are hooks the interpreter calls for syntax and built-ins. Always write `__repr__`. Keep `__eq__` / `__hash__` consistent and based on immutable fields. Return `NotImplemented` for unknown operands. Implement `__len__` / `__getitem__` / `__iter__` / `__contains__` to make containers, `__call__` for callables, `__enter__`/`__exit__` for `with`.

10.3 Operator overloading

What it is

Operator overloading means giving `+`, `-`, `*`, `@`, `==`, `<`, `[]` etc. a meaning for your own types by implementing the matching dunder.

How it works

For `a + b`, Python:

1. Calls `a.__add__(b)`. If it returns a value (not `NotImplemented`), done.
2. Otherwise calls the **reflected** method `b.__radd__(a)`. (Special case: if `type(b)` is a *subclass* of `type(a)` and overrides `__radd__`, Python tries `b.__radd__(a)` first.)
3. If both return `NotImplemented`, raises `TypeError: unsupported operand type(s)`.

For `a += b`, Python tries the **in-place** method `a.__iadd__(b)` first (which may mutate `a` and return `self`); if it's missing, it falls back to `a = a + b`. That's why `list += ...` mutates in place, while `tuple += ...` builds a new tuple.

Operator	Method	Reflected	In-place
`+`	`__add__`	`__radd__`	`__iadd__`
`-`	`__sub__`	`__rsub__`	`__isub__`
`*`	`__mul__`	`__rmul__`	`__imul__`
`@` (matrix mult)	`__matmul__`	`__rmatmul__`	`__imatmul__`
`/`, `//`, `%`, `**`	`__truediv__`, `__floordiv__`, `__mod__`, `__pow__`	`__r…__`	`__i…__`
`&`, `|`, `^`	`__and__`, `__or__`, `__xor__`	`__r…__`	`__i…__`
unary `-`, `+`, `~`, `abs()`	`__neg__`, `__pos__`, `__invert__`, `__abs__`	–	–

Comparison operators have no `__r…__` versions; instead they are each other's reflections: if `a < b` gives `NotImplemented`, Python tries `b > a`.

```
    def __add__(self, other: "Vector") -> "Vector":
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, k: float) -> "Vector":      # v * 3
        if not isinstance(k, (int, float)):
            return NotImplemented
        return Vector(self.x * k, self.y * k)

    __rmul__ = __mul__                            # 3 * v  (int.__mul__ returns NotImplemented)
```

In the output above, `2 * v` works only because of `__rmul__`: `int.__mul__(2, v)` returns `NotImplemented`, so Python calls `v.__rmul__(2)`.

Common mistakes

- Forgetting `__rmul__`, so `v * 2` works but `2 * v` fails.
- Making `__iadd__` return `None` (forgetting `return self`) – the variable becomes `None`.
- Mutating operands inside `__add__` – binary operators should return a *new* object.

Real-world examples

`Decimal`, `Fraction`, `datetime - timedelta`, `set | set`, `dict | dict` (3.9+), NumPy `@` for matrix multiply (PEP 465), `pathlib` `/`.

In the interview

30-second answer: "Each operator maps to a dunder. For `a + b` Python calls `a.__add__(b)`; if that returns `NotImplemented` it tries `b.__radd__(a)`, then raises `TypeError`. `+=` tries `__iadd__` for in-place update and falls back to `+`. Return `NotImplemented` for unsupported types so the other operand gets a chance."

- *Why does `lst += (1,)` work but `lst = lst + (1,)` fail?* – `list.__iadd__` accepts any iterable (like `extend`); `list.__add__` only accepts lists.

Self-check

1. `2 * v` with only `__mul__` defined? → `TypeError`; add `__rmul__`.
2. What does `x += y` do if `__iadd__` is missing? → `x = x + y` (rebinds to a new object).

TL;DR

Operators are dunders. Return `NotImplemented` for unknown types, add `__r…__` for "number on the left", `__i…__` for efficient in-place updates.

10.4 Inheritance and `super()`

What it is

Inheritance lets a **subclass** (child) reuse and specialise a **base class** (parent): `class Manager(Employee)`. The child gets all parent attributes and methods and can **override** them. It models an **is-a** relationship: a manager *is an* employee.

How it works

1. Attribute lookup on a `Manager` instance searches `Manager`, then `Employee`, then `object` (every class ultimately inherits from `object`).
2. To extend (not replace) a parent method, call it through `super()`, which returns a proxy that looks up the *next* class in the MRO.
3. If the child defines `__init__`, the parent's `__init__` is **not** called automatically – call `super().__init__(...)` yourself.
4. `isinstance(obj, Cls)` checks the object's class and its ancestors; `issubclass(A, B)` checks classes. Prefer `isinstance` to `type(x) == Cls`, which ignores subclasses.

```
class Employee:
    def __init__(self, name: str, salary: int) -> None:
        self.name, self.salary = name, salary
    def pay(self) -> int:
        return self.salary
    def __repr__(self) -> str:
        return f"{type(self).__name__}({self.name!r})"   # type(self) -> works for subclasses


class Manager(Employee):
    def __init__(self, name: str, salary: int, bonus: int) -> None:
        super().__init__(name, salary)      # reuse the parent's initialiser
        self.bonus = bonus
    def pay(self) -> int:                    # override, then extend
        return super().pay() + self.bonus
```

Output: `Manager('Mia') pays 120 | isinstance Employee: True | issubclass: True`

Common mistakes

- Forgetting `super().__init__()` → parent attributes are missing → `AttributeError` later.
- Hard-coding `Employee.__init__(self, ...)` instead of `super()` – breaks cooperative multiple inheritance (§10.5).
- Deep hierarchies (5+ levels) that nobody can follow – see composition (§10.8).
- Overriding a method with an incompatible signature – violates the Liskov Substitution Principle (see LLD Handbook ch. 4).

TL;DR

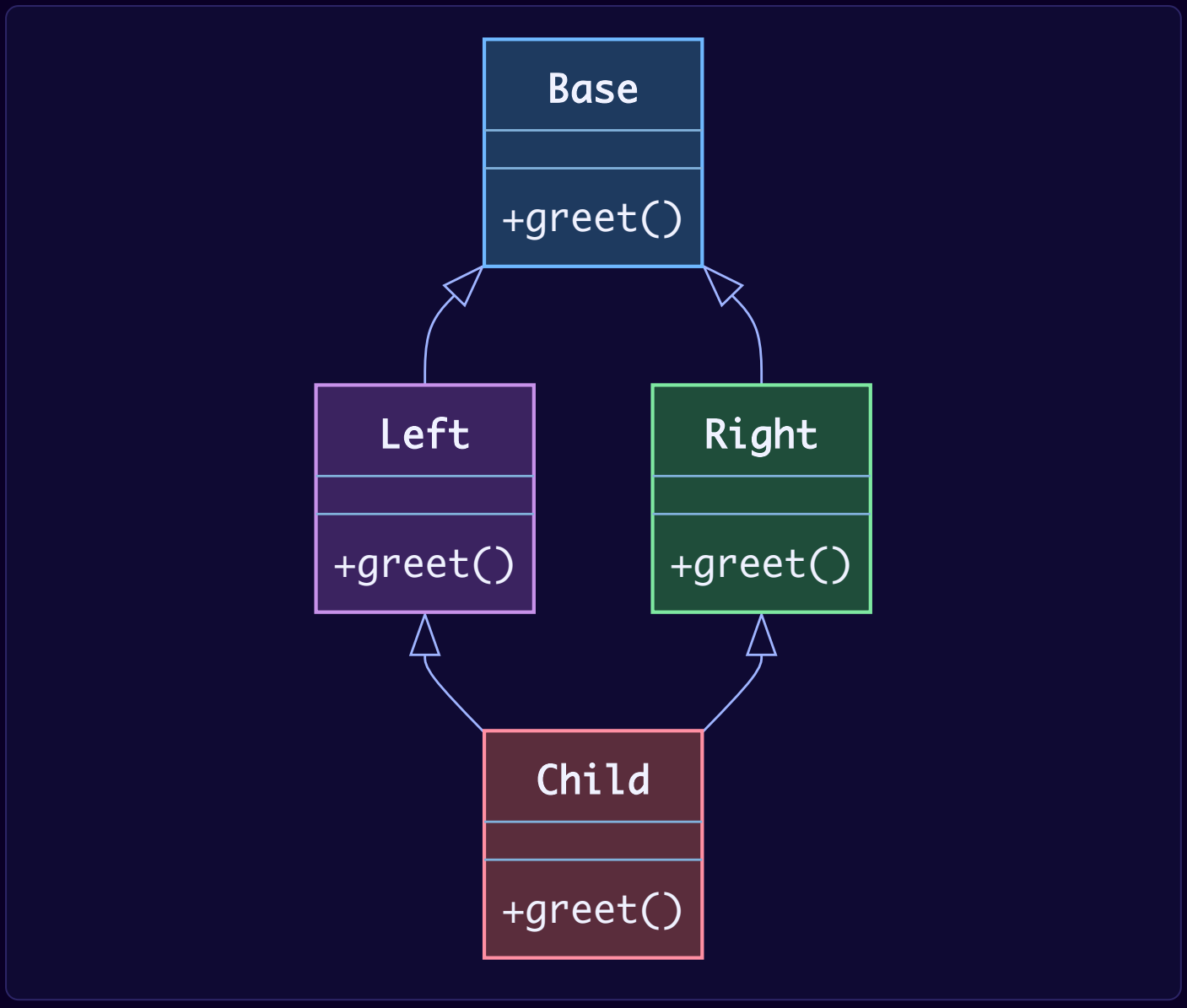
Subclass for true is-a relationships; override to specialise; use `super()` to extend parent behaviour and always call `super().__init__`.

10.5 Multiple inheritance and the MRO (C3 linearization)

What it is

Python allows a class to have several bases: `class Child(Left, Right)`. The **method resolution order (MRO)** is the single, ordered list of classes Python searches when looking up an attribute. You can see it with `Child.__mro__` or `Child.mro()`. Python computes it with the **C3 linearization** algorithm (since Python 2.3).

The problem it solves — the diamond



If `Child` inherits from `Left` and `Right`, and both inherit from `Base`, which `greet` runs, and does `Base.greet` run once or twice? C3 gives one consistent answer, and `Base` appears exactly once.

How it works — C3 in plain words

The MRO of a class = the class itself, followed by a **merge** of its parents' MROs and the list of parents, obeying three rules:

1. **Children before parents** — a class always comes before its bases.
2. **Local order is kept** — bases are searched in the order written (`Left` before `Right`).
3. **Monotonic** — the order in each parent's MRO is preserved.

Worked example for `Child(Left, Right)`:

- MRO(Left) = [Left, Base, object]; MRO(Right) = [Right, Base, object]; bases = [Left, Right].

- Merge: take the first head not appearing in the *tail* of any other list.
  - `Left` – not in any tail → take it. Remaining: [Base, object], [Right, Base, object], [Right].
  - `Base` – appears in the tail of [Right, Base, object] → skip. Try `Right` → OK, take it.
  - Now `Base` is free → take it, then `object`.
- Result: **`Child → Left → Right → Base → object`**.

If no consistent order exists, class creation fails with `TypeError: Cannot create a consistent method resolution order`.

`super()` follows the MRO, not "the parent"

`super()` means "the next class after me **in the MRO of the actual instance's type**". Inside `Left.greet`, when the object is a `Child`, the next class is `Right` – a sibling, not `Base`! This is **cooperative multiple inheritance**: if every class calls `super()`, every implementation runs exactly once.

```
class Base:
    def greet(self): return ["Base"]
class Left(Base):
    def greet(self): return ["Left"] + super().greet()
class Right(Base):
    def greet(self): return ["Right"] + super().greet()
class Child(Left, Right):
    def greet(self): return ["Child"] + super().greet()

class X: ...
class Y: ...
class A(X, Y): ...
class B(Y, X): ...
class C(A, B): ...        # A wants X before Y, B wants Y before X -> impossible
```

Output (`inheritance_mro.py`):

```
Child MRO: ['Child', 'Left', 'Right', 'Base', 'object']
Child().greet(): ['Child', 'Left', 'Right', 'Base']
C3 refuses: Cannot create a consistent method resolution order (MRO) for bases X, Y
```

Cooperative `__init__` tip: when classes in a hierarchy take different arguments, accept `**kwargs`, consume your own, and pass the rest on: `def __init__(self, *, color, **kw): super().__init__(**kw)`.

Trade-offs

Multiple inheritance	Pros	Cons
Mixins (small, stateless add-ons)	Reuse orthogonal behaviour cheaply	Name clashes, MRO surprises
Several "real" stateful bases	Rarely justified	Conflicting `__init__` signatures, fragile

Common mistakes

- Assuming `super()` calls the parent – it calls the next class in the MRO.
- Mixing `super()` and explicit `Parent.method(self)` calls → some methods run twice or not at all.
- Putting a mixin *after* the base class that already defines the method, so the mixin's version is never reached.

Real-world examples

- Django class-based views: `class MyView(LoginRequiredMixin, ListView)` – the mixin comes first so its `dispatch` runs before `ListView`'s.
- `socketserver.ThreadingMixIn` + `TCPServer` → `ThreadingTCPServer` in the standard library.

In the interview

> **30-second answer:** "The MRO is the linear order Python searches for attributes; it's computed by C3 linearization, which keeps children before parents, preserves the order of bases, and is consistent with every parent's MRO — so in a diamond the shared base appears once, last. `super()` delegates to the next class in the MRO of the instance's type, which enables cooperative multiple inheritance where each method runs once."

- *What's the MRO of `D(B, C)` where `B, C` extend `A`?* — D, B, C, A, object.
- *When does class creation fail?* — When base orders conflict, e.g. one base needs X before Y and another Y before X.

Self-check

1. Inside `Left.greet` for a `Child` instance, what does `super()` resolve to? → `Right`.
2. How to print the MRO? → `Cls.__mro__` or `Cls.mro()`.
3. Why does `Base` appear once? → C3 produces a linear list without duplicates; the shared base is placed after all its subclasses.

TL;DR

MRO = C3 linearization: children first, bases in declared order, consistent with parents. `super()` = "next in the MRO", enabling every class in a diamond to run once if all cooperate.

10.6 Mixins

What it is

A **mixin** is a small class that provides one focused capability (serialise to JSON, nice `__repr__`, logging) and is meant to be *combined* with other classes, not used alone. It usually has no `__init__` and no state of its own.

```
class JsonMixin:
    def to_json(self) -> str:
        return json.dumps(self.__dict__, sort_keys=True)

class ReprMixin:
    def __repr__(self) -> str:
        fields = ", ".join(f"{k}={v!r}" for k, v in self.__dict__.items())
        return f"{type(self).__name__}({fields})"

class Product(JsonMixin, ReprMixin):
    def __init__(self, sku: str, price: float) -> None:
        self.sku, self.price = sku, price
```

Output:

```
mixins: Product(sku='A-1', price=9.5) {"price": 9.5, "sku": "A-1"}
Product MRO: ['Product', 'JsonMixin', 'ReprMixin', 'object']
```

Rules of thumb

1. Name them `…Mixin`. 2. Keep them stateless (or cooperative with `**kwargs`). 3. List mixins **before** the main base class. 4. One capability per mixin.

When not to use it

If the behaviour needs lots of state or configuration, compose an object instead (§10.8). If a plain function `to_json(obj)` would do, use the function.

Real-world examples

Django `LoginRequiredMixin`, `PermissionRequiredMixin`; `socketserver.ForkingMixIn` / `ThreadingMixIn`; `collections.abc` classes act as mixins by providing concrete methods (e.g. `Sequence.index`).

TL;DR

Mixins = small, stateless, single-purpose classes mixed in via multiple inheritance; place them left of the main base.

10.7 Abstract base classes, duck typing and Protocols

These are three ways to say "this object must be able to do X".

Duck typing – what it is

"If it walks like a duck and quacks like a duck, it's a duck." Python code normally doesn't check an object's type; it just calls the method and lets it fail if absent. Any object with `.speak()` can be passed to `make_it_speak`. The related style **EAFP** ("easier to ask forgiveness than permission": try, then catch the exception) is preferred in Python over **LBYL** ("look before you leap": check first).

Abstract base classes (ABCs) – what it is

An **ABC** (module `abc`) declares methods that subclasses **must** implement, marked `@abstractmethod`. Instantiating a class that still has abstract methods raises `TypeError` – the error appears at *creation*, not later at the call. ABCs can also carry shared concrete methods.

`collections.abc` defines the standard ones: `Iterable`, `Iterator`, `Sized`, `Container`, `Sequence`, `Mapping`, `MutableMapping`, `Callable`, `Hashable` … Several of them (`Sized`, `Iterable`, `Hashable`, `Callable`, `Container`, …) implement `__subclasshook__`, so `isinstance(x, Sized)` is `True` for any class with `__len__`, even without inheriting. You can also `Shape.register(SomeClass)` to declare a *virtual subclass*.

Protocols – what it is

`typing.Protocol` (PEP 544, Python 3.8) formalises duck typing for **static type checkers** (mypy, pyright): a class matches a Protocol if it has the right methods/attributes – **structural typing** – no inheritance required. Add `@runtime_checkable` to allow `isinstance` checks, which only test that the methods *exist* (not their signatures).

```python
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
    def describe(self) -> str:                # concrete, shared
        return f"{type(self).__name__} with area {self.area():.2f}"

class Square(Shape):
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2

class Broken(Shape): pass                     # forgot area()

class Duck:
    def speak(self): return "Quack"
class Robot:
    def speak(self): return "Beep"

@runtime_checkable
class Speaker(Protocol):
    def speak(self) -> str: ...

def announce(s: Speaker) -> str:              # checked structurally by mypy/pyright
    return s.speak().upper()

class Deck:                                   # never inherits from Sized
    def __len__(self): return 52
```

Output (`abc_protocols.py`):

```
Square with area 9.00
TypeError: Can't instantiate abstract class Shape without an implementation for abstract method 'area'
TypeError: Can't instantiate abstract class Broken without an implementation for abstract method 'area'
['Quack', 'Beep']
BEEP | isinstance(Duck(), Speaker): True | isinstance(42, Speaker): False
Deck is Sized: True | Deck is Iterable: False
```

Protocols, generics and type checking are covered in depth in Chapter 13 §13.8.

Comparison

	Duck typing	ABC	Protocol
Needs inheritance?	No	Yes (or `register`)	No
Checked when?	At call time (AttributeError)	At instantiation (TypeError)	By the static type checker; optionally `isinstance`
Shares code?	No	Yes, concrete methods	Not intended for sharing implementation
Best for	Small scripts, flexible APIs	Frameworks/plugins you own, enforced contracts	Library APIs, accepting third-party types, typed codebases

When not to use them

Don't create an ABC for a single implementation "just in case" (YAGNI). Don't rely on `@runtime_checkable` for correctness — it doesn't check signatures or return types.

Common mistakes

- Forgetting to inherit from `ABC` (or use `metaclass=ABCMeta`) — `@abstractmethod` then has no effect.
- Using `type(x) is list` checks instead of `isinstance(x, collections.abc.Sequence)`.
- Expecting `isinstance(obj, SomeProtocol)` to work without `@runtime_checkable` → `TypeError`.

Real-world examples

`collections.abc.Mapping` is used to write dict-like classes; `io.IOBase` is an ABC for file objects; `typing.SupportsInt`, `SupportsAbs`, `SupportsIndex` are Protocols in the standard library; `numbers.Number` ABC hierarchy.

In the interview

> **30-second answer:** "Python is duck-typed: we care what an object can do, not its class. ABCs make that contract explicit and enforced — you can't instantiate a subclass until it implements every `@abstractmethod`. Protocols describe the same contract structurally, so a type checker accepts any class with matching methods, no inheritance needed. I use ABCs when I own the hierarchy and want shared code, Protocols for flexible, typed interfaces."

- *What does `isinstance(x, Sized)` check?* — Whether `type(x)` has `__len__` (via `__subclasshook__`), or registered/inherited.
- *EAFP vs LBYL?* — Python favours EAFP: `try: d[k] except KeyError:`; it avoids race conditions between check and use.

Self-check

1. When is the "forgot to implement" error raised for an ABC? → When you try to instantiate the subclass.
2. Does a class have to inherit from a Protocol to satisfy it? → No — structural matching.
3. What does `@runtime_checkable` actually check? → Only that the named members exist.

TL;DR

Duck typing = behaviour over type. ABC = nominal, enforced, can share code. Protocol = structural, for type checkers. `collections.abc` has the standard interfaces.

10.8 Composition vs inheritance

What it is

Composition builds an object out of other objects it *has* (has-a): a `Car` *has an* `Engine`. **Inheritance** says a class *is a* kind of another. The classic advice (Gang of Four, and *Effective Python*) is **"favour composition over inheritance"**.

Why it matters

Inheritance couples the child to the parent's internals (the "fragile base class" problem): a change in the parent can silently break children. Composition keeps parts behind small interfaces, so you can swap them (a fake engine in tests, an electric motor in production) at run time.

```
class Engine:
    def __init__(self, hp: int) -> None: self.hp = hp
    def start(self) -> str: return f"{self.hp}hp engine started"

class ElectricMotor:
    def start(self) -> str: return "silent motor started"

class Car:
    def __init__(self, engine) -> None:
        self.engine = engine                # injected part (dependency injection)
    def drive(self) -> str:
        return self.engine.start() + ", driving"
```

Output:

```
150hp engine started, driving
silent motor started, driving
```

Question	Inheritance	Composition
Relationship	is-a	has-a / uses-a
Coupling	Tight (sees parent internals)	Loose (only the part's interface)
Change at run time?	No (class is fixed)	Yes (swap the part)
Code reuse	Automatic	Delegate explicitly (a few one-line methods)
Testing	Hard to replace parent behaviour	Inject a fake

When inheritance is right

Genuine is-a with substitutability (Liskov); framework extension points designed for it (`unittest.TestCase`, `Exception` subclasses, Django `Model`, `nn.Module`); mixins.

Common mistakes

`class Stack(list)` – now callers can `insert(0, x)` and break the stack semantics; wrap a list instead. Inheriting only to reuse one helper method.

In the interview

"I default to composition: it's looser coupling and easier to test and change at run time. I inherit when there's a real is-a relationship that satisfies Liskov, or when a framework is designed to be extended by subclassing."

TL;DR

Has-a → composition (default). Is-a with substitutability → inheritance. Python's duck typing makes composition especially easy: any object with the right methods can be plugged in.

10.9 Encapsulation conventions: `_single` and `__double` underscores

What it is

Python has **no private keyword**. Encapsulation is by convention ("we're all consenting adults"):

Name	Meaning	Enforced?
`name`	Public API	–
`_name`	Internal; may change without notice. `from module import *` skips it	Only by convention (and linters)
`__name` (no trailing `__`)	**Name mangling**: inside class `Account`, `self.__secret` is rewritten at compile time to `self._Account__secret`	Only to avoid *accidental* clashes in subclasses; still reachable
`__name__`	Dunder: reserved for the language	Don't invent your own
`name_`	Avoids clashing with a keyword (`class_`, `type_`)	–

Why name mangling exists

To stop a subclass from accidentally overwriting a parent's attribute that has the same name. It is **not** a security feature.

```
class Account:
    def __init__(self) -> None:
        self._internal = "convention only"
        self.__secret = "mangled"            # stored as _Account__secret

class SubAccount(Account):
    def __init__(self) -> None:
        super().__init__()
        self.__secret = "child's own"        # stored as _SubAccount__secret
```

Output:

```
instance dict: {'_internal': 'convention only', '_Account__secret': 'mangled', '_SubAccount__secret': "child's own"}
AttributeError: 'SubAccount' object has no attribute '__secret'
```

Common mistakes

Using `__x` everywhere to "make it private" – it complicates debugging, pickling and subclass access. *Effective Python* recommends preferring `_protected` and reserving `__private` for avoiding name clashes with subclasses you don't control.

TL;DR

`_x` = internal by convention (use this). `__x` = name-mangled to `_Class__x` to avoid subclass clashes. Nothing is truly private. Use properties to control access.

10.10 dataclasses

What it is

`@dataclass` (module `dataclasses`, Python 3.7, PEP 557) reads a class's **type-annotated** class attributes and generates `__init__`, `__repr__` and `__eq__` for you. Optional flags generate ordering, hashing, immutability and `__slots__`.

Why it matters

Most classes are mostly data. Writing `__init__` / `__repr__` / `__eq__` by hand is repetitive and bug-prone (forget a field in `__eq__`, and equality is silently wrong).

How it works

1. Each annotated class attribute becomes a **field**, in declaration order. Un-annotated attributes are ignored; `ClassVar[...]` annotations are ignored too.
2. A default value becomes the `__init__` default. Fields without defaults can't follow fields with defaults (unless `kw_only`).
3. `field(...)` customises a field: `default_factory=list` (a fresh object per instance), `repr=False`, `compare=False`, `init=False`, `kw_only=True`.
4. `__post_init__` runs after the generated `__init__` – use it for validation or derived fields.

`@dataclass(...)` flag	Effect
`eq=True` (default)	Generates `__eq__` comparing fields as a tuple. (Because `__eq__` is defined, the class becomes unhashable unless `frozen` / `unsafe_hash`.)
`order=True`	Generates `<`, `<=`, `>`, `>=` comparing fields as tuples, in declaration order
`frozen=True`	Assigning a field raises `FrozenInstanceError`; with `eq=True` also generates `__hash__`
`slots=True` (3.10)	Generates `__slots__` → less memory, no new attributes
`kw_only=True` (3.10)	All fields must be passed by keyword
`unsafe_hash=True`	Forces a `__hash__` even when mutable — rarely right

Helpers: `asdict()`, `astuple()`, `replace(obj, **changes)` (copy with some fields changed — the idiomatic way to "modify" a frozen instance), `fields()`.

```python
@dataclass
class Item:
    name: str
    price: float
    qty: int = 1
    tags: list[str] = field(default_factory=list)          # never `tags: list = []`
    _cache: dict = field(default_factory=dict, repr=False, compare=False)

    def __post_init__(self) -> None:
        if self.price < 0:
            raise ValueError("price must be >= 0")

    @property
    def total(self) -> float:
        return self.price * self.qty


@dataclass(frozen=True, order=True)
class Version:
    major: int
    minor: int
    patch: int = 0


@dataclass(slots=True, kw_only=True)
class Config:
    host: str
    port: int = 8080
```

Output (dataclass part of `dataclasses_enums_slots.py`):

```
Item(name='pen', price=1.5, qty=4, tags=['office']) | total: 6.0
eq ignores _cache: True
asdict: {'name': 'pen', 'price': 1.5, 'qty': 4, 'tags': ['office'], '_cache': {}}
post_init: price must be >= 0
mutable default: mutable default <class 'list'> for field tags is not allowed: use default_factory
order compares field-by-field: True | sorted: [Version(major=0, minor=9, patch=0), Version(major=1, minor=4,
patch=0), Version(major=1, minor=10, patch=0)]
frozen is hashable: stable
FrozenInstanceError: cannot assign to field 'major'
replace(): Version(major=1, minor=4, patch=7)
Config(host='localhost', port=8080) | has __dict__: False
```

Note `Version(1, 4) < Version(1, 10)` is `True` because it compares the tuples `(1, 4, 0) < (1, 10, 0)` numerically — unlike comparing the strings `"1.4" < "1.10"`.

Trade-offs – data-holder options

Option	Mutable	Validation	Speed/memory	Use when
`dict`	Yes	None	Fine	Truly dynamic keys, JSON
`typing.NamedTuple`	No	None	Tuple-light	Immutable records that should also behave like tuples (unpacking, indexing)
`@dataclass`	Configurable	`__post_init__`	Good; `slots=True` for lean	Default choice for internal data
`attrs` (3rd party)	Configurable	Validators/converters	Good	More features; dataclasses was inspired by it
Pydantic `BaseModel` (3rd party)	Configurable	Full parsing/validation from untrusted input	Heavier	API payloads, config files

Common mistakes

- `tags: list = []` → `ValueError`; use `field(default_factory=list)`.
- Forgetting the type annotation – `x = 5` in a dataclass is just a class attribute, not a field.
- Expecting dataclasses to validate types at run time – annotations are not enforced.
- `frozen=True` is shallow: a frozen dataclass holding a list can still have the list mutated.
- Inheritance with defaults: a base class field with a default followed by a subclass field without one → `TypeError` (fix with `kw_only=True`).

Real-world examples

Widely used for configs, DTOs and events; many ML libraries (e.g. Hugging Face `transformers` `TrainingArguments`) are dataclasses. FastAPI/Pydantic accept dataclasses too.

In the interview

> **30-second answer:** "`@dataclass` generates `__init__`, `__repr__` and `__eq__` from annotated fields. `order=True` adds comparisons by field tuple, `frozen=True` makes it immutable and hashable, `slots=True` saves memory. Use `field(default_factory=list)` for mutable defaults and `__post_init__` for validation. For untrusted input I'd use Pydantic instead because dataclasses don't validate types."

Self-check

1. Why is a plain `@dataclass` unhashable? → It defines `__eq__`, so `__hash__` is set to `None`.
2. How do you "change" a frozen instance? → `dataclasses.replace(obj, field=new)` returns a new one.
3. What does `compare=False` do? → Excludes the field from `__eq__` and ordering.

TL;DR

Dataclasses remove boilerplate for data classes. Know `field(default_factory=…)`, `__post_init__`, `frozen`, `order`, `slots`, `kw_only`, `replace`, `asdict`.

10.11 Enums

What it is

An **enumeration** (`enum.Enum`) is a set of named, unique constant members: `Color.RED`. Members are singletons with a `.name` and `.value`, can be iterated, and compare by identity.

Why it matters

Magic strings/ints (`status == 3`, `"acitve"` typo) are bug magnets. Enums give you autocomplete, typo-proof names, a closed set of legal values, and readable reprs.

Class	Behaves like	Use for
`Enum`	Its own type; `Color.RED == 1` is `False`	Default
`IntEnum`	Also an `int`	Interop with code expecting ints (`http.HTTPStatus`)
`StrEnum` (3.11)	Also a `str`; `auto()` gives the lower-case name	Values stored in JSON/DB as strings
`Flag` / `IntFlag`	Bit flags combinable with `|`, test with `in`	Permissions, options (`re.RegexFlag`)

`auto()` picks the value for you (1, 2, 3 … for `Enum`). `@enum.unique` forbids aliases (two names with the same value). Look up by value `Color(3)` or by name `Color["GREEN"]`.

```
class Color(Enum):
    RED = auto(); GREEN = auto(); BLUE = auto()
class HttpStatus(IntEnum):
    OK = 200; NOT_FOUND = 404
class Env(StrEnum):
    DEV = auto(); PROD = auto()
class Perm(Flag):
    READ = auto(); WRITE = auto(); EXEC = auto()
```

Output:

```
Color.RED RED 1 Color.GREEN Color.BLUE
members: [<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 3>] | identity compare: True
IntEnum == int: True | Enum == int: False
StrEnum: prod True
Flag: Perm.READ|WRITE | WRITE in rw: True | EXEC in rw: False
```

Common mistakes

- Comparing `member == "red"` with a plain `Enum` → always `False`; compare to `Color.RED` or use `StrEnum`.
- Using `IntEnum` everywhere — it lets unrelated int comparisons succeed silently, losing type safety.
- Trying to modify or add members at run time — enums are fixed.

Real-world examples

`http.HTTPStatus` (IntEnum), `re.RegexFlag` (flags like `re.IGNORECASE`), `signal.Signals`, Django `TextChoices` / `IntegerChoices` for model field choices.

TL;DR

Use `Enum` for closed sets of constants, `StrEnum` / `IntEnum` when you need str/int interop, `Flag` for combinable options; `auto()` for values.

10.12 `__slots__`

What it is

By default each instance stores its attributes in a per-instance dict (`__dict__`), which is flexible but costs memory. Declaring `__slots__ = ("x", "y")` tells Python to reserve fixed storage for exactly those attributes

and not create `__dict__`.

Why it matters

With millions of small objects (points, graph nodes, events), slots cut memory and make attribute access slightly faster, and they prevent typos from silently creating new attributes.

```
class PointDict:
    def __init__(self, x, y): self.x, self.y = x, y

class PointSlots:
    __slots__ = ("x", "y")
    def __init__(self, x, y): self.x, self.y = x, y
```

Output (100 000 instances measured with `tracemalloc`, including the list that holds them):

```
100k objects: dict-based 12.8 MB vs slots 8.8 MB
slots block new attrs: 'PointSlots' object has no attribute 'z' and no __dict__ for setting new attributes
```

(Exact numbers depend on the Python version – recent CPythons already store instance dicts compactly, so the saving is smaller than on older versions.)

Caveats

- No new attributes beyond those listed; no `__dict__`, so no `vars(obj)`.
- No weak references unless you add `"__weakref__"` to the slots.
- Every class in the hierarchy must declare `__slots__` (use `()` in a subclass with nothing new), otherwise the subclass gets a `__dict__` again and the saving is lost.
- A slot can't also have a class-attribute default of the same name; dataclasses handle this for you with `@dataclass(slots=True)`.
- `functools.cached_property` needs `__dict__`, so it doesn't work on slotted classes.

TL;DR

`__slots__` = fixed attribute set, no per-instance dict → less memory, no accidental attributes. Use for many small objects; otherwise leave it.

10.13 Pythonic design patterns (brief)

The full SOLID principles and all 23 GoF patterns are in the LLD Handbook: SOLID, creational patterns incl. Singleton, structural incl. Decorator, behavioral incl. Strategy/Observer. Python's first-class functions and modules make several of them much smaller:

Pattern	Classic (Java-style)	Pythonic
Strategy	Interface + one class per strategy	Pass a **function** (functions are objects)
Singleton	Private constructor + `getInstance()`	A **module** (imported once, cached in `sys.modules`) or a `@functools.cache` factory
Command	Command class with `execute()`	A function, `functools.partial`, or a callable object (`__call__`)
Iterator	Iterator class with `hasNext/next`	A generator function (Chapter 11)
Decorator	Wrapper class implementing the same interface	`@decorator` functions (Chapter 12) – or a wrapper class for objects
Factory	Factory class	A `@classmethod` alternative constructor, or a dict mapping names to classes
Template method	Abstract base with hook methods	Same (ABC), or pass hook functions
Observer	Listener interface	A list of callbacks

```
def no_discount(total): return total
def ten_percent(total): return total * 0.9
def bulk(total): return total - 20 if total > 200 else total

def checkout(total: float, strategy=no_discount) -> float:
    return round(strategy(total), 2)

@functools.cache
def get_settings() -> _Settings:        # lazily created, then always the same object
    return _Settings()
```

Output (`pythonic_patterns.py`):

```
no_discount  -> 250
ten_percent  -> 225.0
bulk         -> 230
<lambda>     -> 125.0
same object: True | debug: True
module cached once: True | in sys.modules: True
```

Caveat: a Singleton is still global state – it makes testing harder. Prefer passing dependencies explicitly and keep singletons for truly process-wide things (config, logging – `logging.getLogger(name)` returns the same logger for the same name).

In the interview

"In Python, a Strategy is usually just a function argument, like `sorted(key=...)`. A Singleton is usually a module-level object, because modules are imported once. I'd still use classes when the strategy needs its own state or several related methods."

TL;DR

First-class functions and modules replace much pattern boilerplate: functions for Strategy/Command, modules for Singleton, generators for Iterator, `@decorators` for Decorator.

10.14 Chapter self-check

1. What two steps happen when you call `MyClass(...)`? → `__new__` creates, `__init__` initialises.

2. Why does `a.items.append(1)` affect `b` when `items = []` is a class attribute? → Both see the same shared list; no instance attribute is created by mutation.
3. `class D(B, C)`, both `B` and `C` subclass `A`. MRO? → D, B, C, A, object.
4. You define `__eq__` on a class. Can instances go in a set? → No, `__hash__` becomes `None` – define `__hash__` on the same (immutable) fields.
5. `v * 3` works, `3 * v` fails. Fix? → Implement `__rmul__`.
6. ABC vs Protocol? → ABC: nominal, enforced at instantiation, shares code. Protocol: structural, checked by type checkers.
7. How do you give a dataclass a list default? → `field(default_factory=list)`.
8. What does `self.__token` become inside class `Api`? → `self._Api__token`.
9. When do `__slots__` help? → Many small instances; they drop the per-instance `__dict__`.
10. Pythonic Strategy? → Pass a function.

Chapter TL;DR

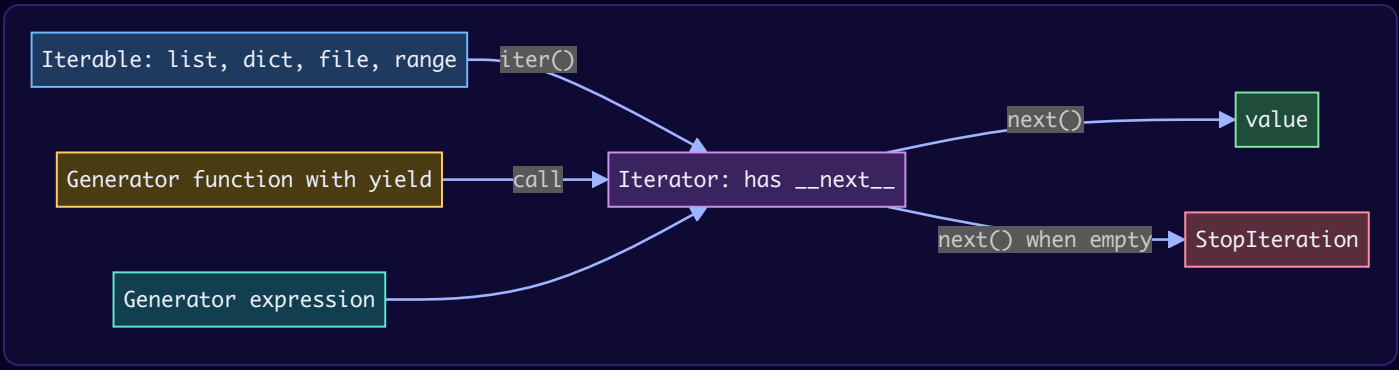
- **Classes** bundle state + behaviour; `__init__` initialises `self`. Instance attributes live in `obj.__dict__`; class attributes are shared (never use a mutable one as per-instance state).
- **Methods:** instance (`self`), `@classmethod` (`cls`, alternative constructors), `@staticmethod` (neither). **Properties** add validation without changing the API.
- **Data model:** dunders connect your class to syntax and built-ins. Always `__repr__`; keep `__eq__` / `__hash__` consistent; return `NotImplemented`; `__r…__` for reflected operators; `total_ordering` for comparisons.
- **Inheritance:** `super()` = next in the **MRO** (C3: children first, bases in order, consistent). Cooperative `super()` makes diamonds safe. **Mixins** add small capabilities.
- **Contracts:** duck typing by default, **ABCs** to enforce, **Protocols** for structural static typing. **Prefer composition** unless it's a real is-a.
- **Toolbox:** `@dataclass` (`field`, `frozen`, `order`, `slots`, `kw_only`), `enum` (`Enum`, `IntEnum`, `StrEnum`, `Flag`, `auto`), `__slots__` for memory.
- **Encapsulation:** `_x` by convention; `__x` is name-mangled, not private.
- **Patterns:** functions for Strategy, modules for Singleton; details in the LLD Handbook.

Chapter 11 – Iterators, Generators and Functional Tools

What you will learn. The protocol behind every `for` loop (iterable vs iterator, `iter()` and `next()`), how generator functions and generator expressions produce values lazily, `yield from`, the basics of `send()` / `close()`, why laziness saves memory and time, the `itertools` toolkit (`count`, `cycle`, `chain`, `islice`, `groupby`, `product`, `permutations`, `combinations`, `accumulate`, `pairwise`, `batched` and friends), comprehension best practices, `map` / `filter` / `reduce` vs comprehensions, `any` / `all`, and `zip(strict=True)`.

Prerequisites. Chapter 10's data-model idea: syntax calls dunder methods.

Code. `chapters/11-iterators-generators-functional/code/` – `iter_protocol.py`, `generators.py`, `itertools_tour.py`, `comprehensions_functional.py`, `lazy_memory.py` (makes the plot in `img/`). Python 3.12+ (`itertools.batched`).



11.1 Iterable vs iterator: the iteration protocol

What it is

- An **iterable** is anything you can loop over: it has `__iter__()` that returns an iterator (or, as a legacy fallback, a `__getitem__` that accepts 0, 1, 2 …). Lists, tuples, strings, dicts, sets, `range`, files.
- An **iterator** is the object that actually walks through the items. It has `__next__()` (return the next item or raise `StopIteration`) **and** `__iter__()` that returns **itself**.

Analogy: a book is an *iterable*; a bookmark is an *iterator*. Many readers can each have their own bookmark in the same book; a bookmark only moves forward and, once at the end, is used up.

Why it matters

This one protocol powers `for`, comprehensions, unpacking (`a, b = x`), `sum`, `list()`, `sorted`, `zip`, `in` and more. Understanding it explains puzzles like "why is my generator empty the second time?".

How it works – what `for x in obj:` really does

1. Call `it = iter(obj)` → `obj.__iter__()`.
2. Repeatedly call `x = next(it)` → `it.__next__()` and run the loop body.
3. When `__next__` raises `StopIteration`, stop quietly.

```
class Countdown:                        # ITERABLE: returns a fresh iterator each time
    def __init__(self, start): self.start = start
    def __iter__(self): return CountdownIterator(self.start)

class CountdownIterator:                # ITERATOR: holds position, returns itself
    def __init__(self, current): self.current = current
    def __iter__(self): return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration         # "no more items"
        self.current -= 1
        return self.current + 1

def manual_for(iterable):               # what a for loop does
    out, it = [], iter(iterable)
    while True:
        try:
            x = next(it)
        except StopIteration:
            break
        out.append(x)
    return out
```

Output (`iter_protocol.py`):

```
for loop: [3, 2, 1] | again: [3, 2, 1]
manual_for: [3, 2, 1]
next: 10 20 30 | default: done
iter(it) is it: True
first pass: [0, 1, 4] | second pass: []
iter(callable, sentinel): ['a', 'b']
RuntimeError: dictionary changed size during iteration
```

What it shows: `Countdown` is re-iterable because each `for` gets a new iterator; `next(it, default)` returns the default instead of raising; an iterator's `iter()` is itself; a generator (an iterator) is **exhausted after one pass**; `iter(callable, sentinel)` keeps calling a function until it returns the sentinel (handy for `iter(lambda: f.read(4096), b"")`); and changing a dict's size while iterating over it raises.

	Iterable	Iterator
Required methods	`__iter__`	`__iter__` (returns self) + `__next__`
Can loop over it twice?	Yes (new iterator each time)	No – one pass, then empty
Remembers position?	No	Yes
`len()` ?	Often (`list` , `range`)	Usually no
Examples	`list` , `str` , `dict` , `range` , `set`	generator, file object, `map` , `zip` , `enumerate` , `iter(list)`

Note: `range` is a lazy **sequence** (iterable, supports `len` and indexing), not an iterator: `next(range(3))` is a `TypeError` .

Common mistakes

- Looping twice over a generator/ `map` / `zip` /file and getting nothing the second time. Materialise with `list()` if you need two passes, or make a re-iterable class.
- Writing a class whose `__iter__` returns `self` *and* expecting it to be re-iterable.
- Modifying a list/dict while iterating over it – iterate over a copy (`for k in list(d):`) or build a new collection.

- Raising `StopIteration` inside a **generator** to stop it: since Python 3.7 (PEP 479) that becomes a `RuntimeError`. Use `return`.

Real-world examples

File objects iterate line by line without loading the whole file; `csv.reader` and `os.scandir` return iterators; database cursors (DB-API) are iterable over result rows; PyTorch's `DataLoader` is an iterable that yields batches each epoch.

In the interview

> **30-second answer:** "An iterable has `__iter__` that returns an iterator; an iterator has `__next__`, which returns items or raises `StopIteration`, and its `__iter__` returns itself. A `for` loop calls `iter()` once then `next()` until `StopIteration`. Iterables like lists can be looped many times; iterators like generators are single-pass."

- *Why is a generator empty the second time?* – It's an iterator; its position is at the end.
- *How do you make a class iterable?* – Implement `__iter__` (often as a generator: `def __iter__(self): yield from self._items`).

Self-check

1. Is a list an iterator? → No, an iterable; `iter(lst)` gives a `list_iterator`.
2. What does `next(it, None)` do at the end? → Returns `None` instead of raising.
3. Is `range(5)` an iterator? → No, a lazy sequence (iterable).

TL;DR

Iterable → `iter()` → iterator → `next()` … `StopIteration`. Iterables can restart; iterators are one-shot.

11.2 Generators and `yield`

What it is

A **generator function** is a function containing `yield`. Calling it does **not** run the body – it returns a **generator object** (an iterator). Each `next()` runs the body until the next `yield`, hands out that value, and **pauses**, keeping all local variables. When the function returns, `StopIteration` is raised (a `return value` becomes `StopIteration.value`).

Analogy: a normal function is a vending machine that dumps all items at once; a generator is a Pez dispenser – one item per click, and it remembers where it was.

Why it matters

- **Memory:** values are produced one at a time – you can process a 50 GB log or an infinite stream in constant memory.
- **Simplicity:** writing an iterator as a generator takes a few lines instead of a class with `__iter__`/`__next__` and manual state.
- **Pipelines:** generators chain like Unix pipes: read → filter → parse → aggregate, each stage lazy.

How it works

1. `g = countdown(3)` – creates the generator; nothing in the body runs yet.
2. `next(g)` – runs from the top to the first `yield n`; returns 3 and freezes the frame.
3. Next `next(g)` – resumes right after the `yield`, loops, yields 2.
4. When the body ends (or hits `return`), `StopIteration` is raised; `for`/`list()` handle it.

```python
def countdown(n: int):
    print("  (generator body starts)")
    while n > 0:
        yield n                    # pause here, hand n to the caller
        n -= 1
    return "liftoff"               # becomes StopIteration.value

def read_lines(lines):             # pipeline stage 1: strip, skip blanks
    for line in lines:
        line = line.strip()
        if line:
            yield line

def parse(lines):                  # pipeline stage 2: parse
    for line in lines:
        name, score = line.split(",")
        yield name, int(score)

def fib():                         # infinite but lazy
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

Output (first part of `generators.py`):

```
created: generator (nothing printed yet -> lazy)
  (generator body starts)
next: 3
rest: [2, 1]
  (generator body starts)
return value via StopIteration.value: liftoff
pipeline: [('alice', 90), ('bob', 72), ('cara', 85)]
...
first 10 fib: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Notice "(generator body starts)" appears only at the first `next()`, not at creation. `fib()` never ends, yet `islice(fib(), 10)` takes just 10 values.

Generator expressions

A **generator expression** is the lazy version of a list comprehension — parentheses instead of brackets: `(x * x for x in range(5))`. When it is the only argument to a function, the extra parentheses can be dropped: `sum(x * x for x in nums)`.

```
genexp sum: 30 | sum(x for x in ...) needs no extra parens: 10
```

Trade-offs

	List (comprehension)	Generator (function/expression)
Memory	Stores all n items	One item at a time (constant)
Multiple passes	Yes	No (one-shot)
`len()`, indexing, slicing	Yes	No (use `itertools.islice`)
First result available	After everything is computed	Immediately
Speed for small data you use fully	Slightly faster	Slight per-item overhead
Infinite sequences	Impossible	Natural

When not to use it

When you need the data more than once, need `len()`/random access, or the data is small and you'll use all of it anyway — a list is simpler. When debugging, a generator's laziness can hide *when* errors happen (the exception appears at consumption time, not creation time).

Common mistakes

- Calling a generator function and expecting it to "do" something: `process(items)` with `yield` inside does nothing until iterated.
- Consuming a generator twice.
- `return some_list` inside a generator does *not* return the list to a `for` loop — it ends iteration (the value goes to `StopIteration.value`).
- Holding resources (open files) inside a generator that is never finished — wrap in `with` inside the generator, and/or close it (`gen.close()` or `contextlib.closing`).

Real-world examples

`os.walk` is a generator; Scrapy spiders `yield` scraped items and requests; pytest fixtures use `yield` to separate setup from teardown; web frameworks stream responses from generators (Flask `Response(generator)`, FastAPI/Starlette `StreamingResponse`); `pandas.read_csv(..., chunksize=N)` returns an iterator over chunks for files too large for memory.

In the interview

> **30-second answer:** "A generator function uses `yield`; calling it returns a generator object — an iterator — without running the body. Each `next()` runs to the next `yield`, returns that value and suspends with its local state intact. That gives lazy, constant-memory iteration, infinite sequences, and composable pipelines. Generator expressions are the inline form with parentheses."

- *Generator vs list comprehension?* — Generator is lazy and one-shot; list is eager, reusable, indexable.
- *How do you get a generator's return value?* — `StopIteration.value`, or as the result of `yield from`.
- *What happens if you `raise StopIteration` inside a generator?* — `RuntimeError` since 3.7 (PEP 479); use `return`.

Self-check

1. What does calling a generator function return? → A generator object; the body hasn't started.
2. Memory to `sum(x for x in range(10**9))`? → Constant — one value at a time.
3. How to take the first 5 items of an infinite generator? → `list(itertools.islice(gen, 5))`.

TL;DR

`yield` turns a function into a lazy iterator factory. Generators pause and resume, use constant memory, and chain into pipelines — but they're single-pass.

11.3 `yield from`, `send()` and `close()` (brief)

`yield from` — delegation

`yield from sub_iterable` (PEP 380, Python 3.3) yields every value of the sub-iterable, and — if the sub-iterable is a generator — evaluates to that generator's `return` value. It replaces `for x in sub: yield x` and is ideal for recursion.

```python
def flatten(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)   # delegate to a recursive sub-generator
        else:
            yield item

def inner():
    yield 1
    yield 2
    return "inner-done"

def outer():
    result = yield from inner()        # receives inner's return value
    yield f"got {result!r}"
```

Output:

```
flatten: [1, 2, 3, 4, 5]
outer: [1, 2, "got 'inner-done'"]
```

(`yield from` also forwards `send()` / `throw()` / `close()` to the sub-generator — that property made it the foundation of pre-`async` coroutines.)

`send()`, `throw()`, `close()` — two-way generators

A `yield` is also an **expression**: `value = yield out` hands `out` to the caller and receives whatever the caller passes with `gen.send(value)`.

1. **Prime** the generator first with `next(gen)` (or `gen.send(None)`) so it runs to the first `yield`. Sending a non-`None` value to a just-started generator is a `TypeError`.
2. `gen.send(v)` resumes it; the paused `yield` evaluates to `v`; runs to the next `yield` and returns that yielded value.
3. `gen.throw(exc)` raises an exception at the paused `yield`.
4. `gen.close()` raises `GeneratorExit` at the paused `yield` so the generator can clean up (`finally` / `except GeneratorExit` then re-raise or return). Generators are also closed when garbage-collected.

```python
def running_average():
    total, count, avg = 0.0, 0, None
    try:
        while True:
            value = yield avg          # send avg out, receive next value
            total += value
            count += 1
            avg = total / count
    except GeneratorExit:
        print("  closing running_average (cleanup here)")
        raise
```

Output:

```
  send 10 -> 10.0
  send 20 -> 15.0
  send 60 -> 30.0
  closing running_average (cleanup here)
```

In modern code, `async def` / `await` **coroutines** (asyncio) replace generator-based coroutines; `send` is mostly seen in libraries and interviews. `contextlib.contextmanager` (Chapter 12) relies on the same generator machinery.

TL;DR

`yield from` delegates to a sub-iterable and returns a sub-generator's return value. `send` pushes a value into a paused generator (prime with `next` first), `throw` raises inside it, `close` triggers `GeneratorExit` for cleanup.

11.4 Lazy evaluation and memory

What it is

Lazy evaluation means computing a value only when it's actually requested; **eager** evaluation computes everything up front. In Python 3, many built-ins are lazy: `range`, `map`, `filter`, `zip`, `enumerate`, `reversed`, `dict.keys()/values()/items()` (views), file iteration, generators, and all of `itertools`.

Why it matters — measured

`lazy_memory.py` measures the peak memory of `sum([x*x for x in range(n)])` (builds a list first) vs `sum(x*x for x in range(n))` (streams values):

```
getsizeof list of 1,000,000: 8.4 MB (pointers only)
getsizeof generator:      208 bytes
n=   10,000  sum(list comp) peak   0.40 MB | sum(gen expr) peak   0.4 KB
n=  100,000  sum(list comp) peak   4.00 MB | sum(gen expr) peak   0.4 KB
n=  300,000  sum(list comp) peak  12.20 MB | sum(gen expr) peak   0.4 KB
n=1,000,000  sum(list comp) peak  40.45 MB | sum(gen expr) peak   0.4 KB
any() list comp built all 1,000,000 items; gen stopped at item 12 -> gen faster: True
```

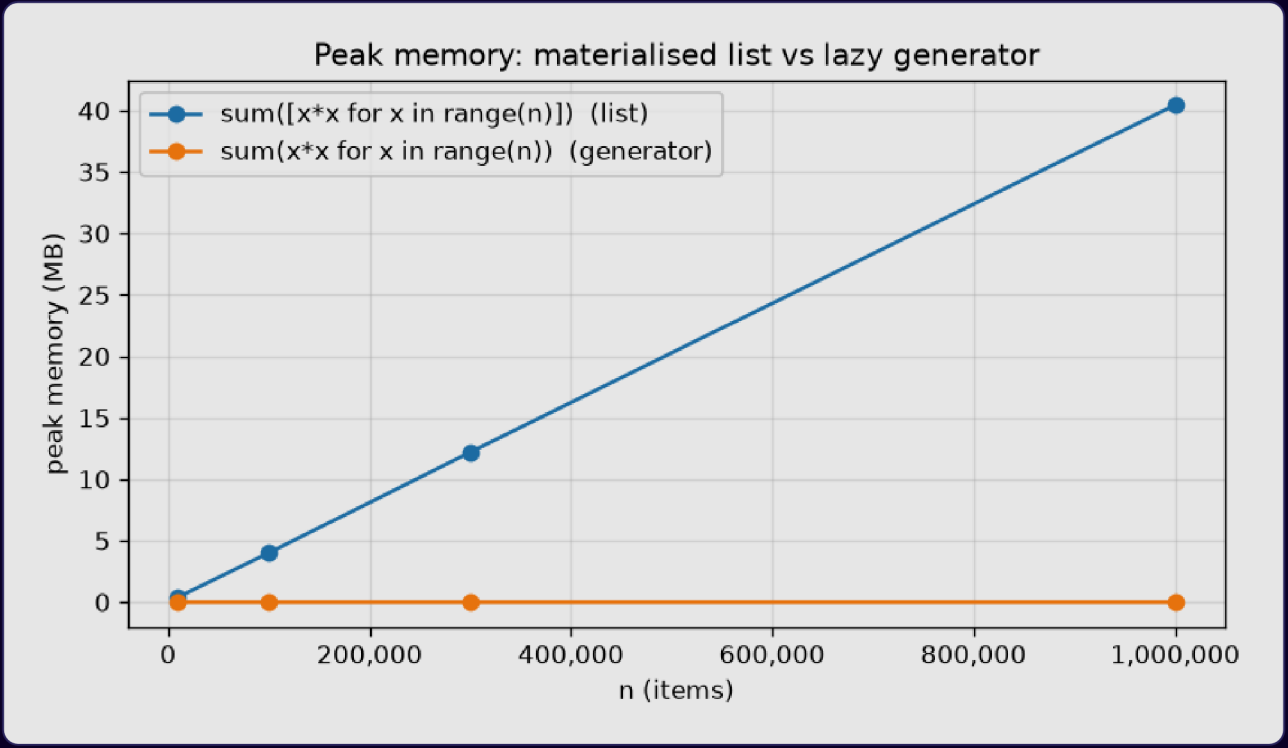


Figure — Peak memory grows linearly with n for the list (≈40 bytes per item: an 8-byte pointer plus a ~32-byte int object) but stays flat for the generator.

How to read the numbers: `sys.getsizeof(list)` counts only the list's array of 8-byte pointers (1,000,000 × 8 ≈ 8 MB); the int objects themselves add ~32 MB more, which `tracemalloc` sees. Exact sizes vary by Python version and platform.

Two benefits of laziness:

1. **Memory** — O(1) instead of O(n) for single-pass work.

2. **Early exit** – `any`, `next`, `islice`, `takewhile` stop pulling as soon as they have the answer, so the rest is never computed.

Common mistakes

- Wrapping everything in `list(...)` "to be safe" – loses both benefits.
- Assuming laziness in Python 2 tutorials (`map` returned lists there).
- Lazy code capturing a variable that changes before evaluation (closures bind late – see §11.7).

TL;DR

Lazy = compute on demand. Use generators and iterator tools for big/streaming/infinite data and early-exit searches; use lists when you need reuse or random access.

11.5 `itertools` – the iterator toolkit

What it is

`itertools` is a standard-library module of fast (C-implemented), memory-efficient building blocks that take iterables and return **lazy iterators**. Think of them as Lego for loops.

Infinite iterators

Function	Produces	Example → result
`count(start=0, step=1)`	start, start+step, … forever	`islice(count(10, 5), 4)` → 10, 15, 20, 25
`cycle(it)`	Repeats `it` forever (saves a copy)	`islice(cycle("AB"), 5)` → A B A B A
`repeat(x, n=∞)`	x, n times (or forever)	`repeat("x", 3)` → x x x

Terminating / selecting

Function	What it does	Output from the script
`chain(a, b, …)`	Concatenate iterables lazily	`[1, 2, 3, 'a', 'b']`
`chain.from_iterable(x)`	Flatten one level	`[1, 2, 3]`
`islice(it, start, stop, step)`	Slice any iterator (no negatives)	`islice(range(10), 2, 8, 2)` → `[2, 4, 6]`
`takewhile(pred, it)`	Yield while predicate is true, then stop	`[1, 3]`
`dropwhile(pred, it)`	Skip while predicate is true, then yield the rest	`[6, 2]`
`filterfalse(pred, it)`	Items where predicate is false	`[1, 3, 5]`
`compress(data, selectors)`	Keep items whose selector is truthy	`['A', 'C']`
`starmap(f, it)`	`f(*args)` for each args tuple	`[8, 9]`
`zip_longest(*its, fillvalue)`	Like `zip` but pads the shorter ones	`[('a', 1), ('b', 2), ('-', 3)]`
`tee(it, n=2)`	Split one iterator into n independent ones (buffers in memory)	`[1, 2, 3] [1, 2, 3]`
`accumulate(it, func=add, initial=None)`	Running totals (or running max, product …)	`[3, 4, 8, 9, 14]`
`pairwise(it)` (3.10)	Overlapping pairs (a,b), (b,c), …	`[(1, 4), (4, 9), (9, 16)]`
`batched(it, n)` (3.12)	Tuples of size n (last may be shorter; `strict=True` in 3.13+ raises instead)	`[(0, 1, 2), (3, 4, 5), (6,)]`

`groupby` — the one everybody gets wrong

`groupby(iterable, key)` groups **consecutive** items with the same key. It does **not** sort. If equal keys aren't adjacent, you get separate groups for them. So: **sort by the same key first**.

```
words = ["apple", "avocado", "banana", "blueberry", "cherry", "apricot"]
{k: list(g) for k, g in groupby(words, key=lambda w: w[0])}
# 'a' appears twice as a group; the dict keeps only the last one -> 'apple', 'avocado' lost
{k: list(g) for k, g in groupby(sorted(words), key=lambda w: w[0])}
```

```
groupby UNSORTED (apricot lost/overwritten): {'a': ['apricot'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
groupby sorted: {'a': ['apple', 'apricot', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
```

Also: each group `g` is a lazy view that becomes invalid once you advance to the next group — convert it with `list(g)` immediately if you need to keep it. (For simple "group into a dict", `collections.defaultdict(list)` needs no sorting.)

Combinatorics

Function	Meaning	Count for n items, pick r	"abc" , r=2
`product(A, B)` / `product(A, repeat=r)`	Cartesian product (nested loops)	n^r	`aa ab ac ba …` (9)
`permutations(it, r)`	Ordered arrangements, no repeats	n!/(n−r)! = 6	`ab ac ba bc ca cb`
`combinations(it, r)`	Unordered selections, no repeats	n!/(r!(n−r)!) = 3	`ab ac bc`
`combinations_with_replacement(it, r)`	Unordered, repeats allowed	(n+r−1)!/(r!(n−1)!) = 6	`aa ab ac bb bc cc`

Output from the script:

```
product: [('a', 1), ('a', 2), ('b', 1), ('b', 2)]
product repeat=2 of 01: ['00', '01', '10', '11']
permutations(abc, 2): ['ab', 'ac', 'ba', 'bc', 'ca', 'cb']
combinations(abc, 2): ['ab', 'ac', 'bc']
combinations_with_replacement(ab, 2): ['aa', 'ab', 'bb']
accumulate max: [3, 3, 4, 4, 5]
accumulate product, initial=1: [1, 2, 6, 24]
differences via pairwise: [3, 5, 7]
```

Combinatorial output grows explosively (`permutations(range(12))` is 479,001,600 items) — iterate lazily, never `list()` it blindly.

Real-world examples

`product` for hyperparameter grids (scikit-learn's `ParameterGrid` iterates the Cartesian product of parameter lists); `batched` to send API/database writes in chunks of 100; `pairwise` for deltas between consecutive timestamps; `accumulate` for running balances or cumulative sums; `chain.from_iterable` to flatten pages of API results; `islice` for "first N" of any stream. The `more-itertools` package adds many more recipes (`chunked` , `windowed` , …).

In the interview

30-second answer: "itertools provides lazy, C-speed building blocks: infinite ones like `count` and `cycle` ; selectors like `islice` , `takewhile` , `compress` ; combiners like `chain` and `zip_longest` ; aggregators like `accumulate` , `groupby` , `pairwise` , `batched` ; and combinatorics — `product` , `permutations` , `combinations` . The classic trap is that `groupby` only groups consecutive equal keys, so you sort first."

- *Difference between permutations and combinations?* — Permutations care about order (ab ≠ ba); combinations don't.
- *How to chunk a list into batches of 100?* — `itertools.batched(items, 100)` (3.12+); earlier, `zip(*[iter(items)]*100)` drops the remainder, so use an `islice` loop.
- *Why can't I slice a generator with `[:5]`?* — Generators have no `__getitem__` ; use `islice` .

Self-check

1. `groupby([1,1,2,1])` gives how many groups? → Three: 1, 2, 1.
2. How many `combinations(range(5), 2)` ? → 10.
3. What does `accumulate([1,2,3])` produce? → 1, 3, 6.
4. What does `pairwise("abc")` give? → ('a','b'), ('b','c').

TL;DR

Reach for `itertools` before writing index-juggling loops. Remember: lazy results, `groupby` needs sorted input, combinatorics explode, `batched` / `pairwise` are modern (3.12/3.10).

11.6 Comprehensions – best practices

What it is

A **comprehension** builds a collection from an iterable in one expression:

Kind	Syntax	Result
List	`[expr for x in it if cond]`	`list`
Dict	`{k: v for x in it if cond}`	`dict`
Set	`{expr for x in it if cond}`	`set`
Generator expression	`(expr for x in it if cond)`	lazy iterator

How to read one

Read `[n * n for n in nums if n % 2 == 0]` as: "for each `n` in `nums`, if `n` is even, collect `n * n`." Multiple `for` clauses are nested loops **in the same left-to-right order** you'd write them.

```
nums = [1, 2, 3, 4, 5, 6]
[n * n for n in nums if n % 2 == 0]                  # filter: `if` AFTER the for
["even" if n % 2 == 0 else "odd" for n in nums[:4]]   # choose: if-else BEFORE the for
[x for row in matrix for x in row]                   # flatten: outer loop first
[[row[i] for row in matrix] for i in range(3)]       # nested comprehension: transpose
[v for s in data if s.isdigit() and (v := int(s)) > 5]  # walrus: compute once
{v: k for k, v in prices.items()}                    # invert a dict
```

Output (`comprehensions_functional.py`):

```
list: [4, 16, 36]
dict: {4: 16, 5: 25, 6: 36}
set: {0, 1, 2}
gen : 91
labels: ['odd', 'even', 'odd', 'even']
flatten: [1, 2, 3, 4, 5, 6]
transpose: [[1, 4], [2, 5], [3, 6]]
walrus: [10, 7]
invert: {3: 'apple', 2: 'pear'}
```

Best practices

1. **Keep it to one line of logic**: at most two `for` clauses or one `for` + one condition (*Effective Python*'s guideline). Beyond that, write a normal loop or a generator function with a name.
2. **Don't use comprehensions for side effects** – `[print(x) for x in xs]` builds a useless list of `None`s. Use a `for` loop.
3. **Use a generator expression** when the result is consumed once by `sum`, `max`, `any`, `"".join`, etc.
4. **Loop variables don't leak**: in Python 3 each comprehension has its own scope (since 3.12, PEP 709 inlines them for speed without changing this).
5. **Use the walrus operator** `:=` (3.8) to avoid computing an expensive value twice (filter and output).
6. **Prefer** `dict` / `set` **comprehensions** over `dict([(k, v) for …])`.

Common mistakes

- Putting `if-else` after the `for` (`[x for x in xs if c else y]`) → `SyntaxError`. Filter `if` goes after; choice `if-else` goes before.
- Mixing up the order of nested `for` clauses.
- Building a huge list only to take `[0]` — use `next(genexp, default)`.

TL;DR

Comprehensions are the idiomatic way to map + filter into a new collection. Keep them short, side-effect free, and use the generator form when consumed once.

11.7 `map` / `filter` / `reduce`, `any` / `all`, `zip(strict=True)` — the functional toolbox

What it is

Functional programming favours pure functions (no side effects) and passing functions as values. Python supports this partially with first-class functions, `lambda`, and a few helpers:

- `map(f, it, ...)` — lazily applies `f` to every item (or to items from several iterables in parallel).
- `filter(pred, it)` — lazily keeps items where `pred(item)` is truthy; `filter(None, it)` keeps truthy items.
- `functools.reduce(f, it, initial)` — folds a sequence into one value: `f(f(f(init, a), b), c)`. (Moved out of built-ins in Python 3.)
- `any(it)` / `all(it)` — short-circuiting "is at least one / are all truthy".
- `zip(*its, strict=False)` — pairs items up; stops at the shortest. `strict=True` (3.10, PEP 618) raises `ValueError` on unequal lengths.

(`lambda` and higher-order functions are introduced in Chapter 7 §7.5.)

Comprehension or `map` / `filter`?

Situation	Prefer	Why
Transform/filter with an expression	Comprehension	Readable; no `lambda`
Apply an existing named function	`map(str.upper, xs)` either is fine	No lambda needed
Need `lambda` to use `map` / `filter`	Comprehension	`[x*2 for x in xs]` beats `map(lambda x: x*2, xs)`
Reduce to one value	`sum`, `min`, `max`, `any`, `all`, `math.prod`, `"".join`	Clearer than `reduce`
Custom fold with no built-in	`reduce` or a loop	Loop is often more readable

Guido van Rossum's own advice (and *Effective Python*'s) is to prefer comprehensions over `map` / `filter` with lambdas, and plain loops or built-ins over most `reduce` calls.

`any` / `all` details

- Both short-circuit: `any` stops at the first truthy item, `all` at the first falsy one — pass a **generator expression**, not a list, so the unneeded items are never computed.
- Empty input: `all([])` is `True` (vacuous truth), `any([])` is `False`.

`zip` details

- Default `zip` silently truncates to the shortest input — a classic data bug when two lists are expected to align. Use `strict=True` for that case.
- `zip(*pairs)` "unzips" a list of pairs.

- `zip` is lazy; `dict(zip(keys, values))` builds a dict.

```
print("map:", list(map(str.upper, ["a", "b"])))
print("filter:", list(filter(None, [0, 1, "", "hi", None])))
print("reduce product:", reduce(operator.mul, nums, 1))
result = any(check(x) for x in [1, 3, 5])           # stops at 3
list(zip(names, scores, strict=True))               # ValueError on mismatch
letters, numbers = zip(*pairs)                      # unzip
funcs = [lambda: i for i in range(3)]               # late binding trap
funcs = [lambda i=i: i for i in range(3)]           # fix: bind now via default
sorted(people, key=lambda p: (-p[1], p[0]))         # multi-key sort
```

Output:

```
map: ['A', 'B'] | comp: ['A', 'B']
filter: [1, 'hi'] (filter(None, ...) keeps truthy)
reduce sum: 21 | sum(): 21
reduce product: 720
map over two iterables: [11, 22]
  check(1)  check(3)
  any -> True (stopped after 3; 5 never checked)
all([]) = True | any([]) = False
zip truncates silently: [('a', 90), ('b', 80)]
strict=True: zip() argument 2 is shorter than argument 1
unzip: ('a', 'b') (1, 2)
late binding: [2, 2, 2]
fixed with default arg: [0, 1, 2]
sorted by age desc, then name: [('ann', 31), ('cy', 31), ('bob', 25)]
max by age (first wins on tie): ('ann', 31)
```

Late binding — why `[2, 2, 2]`?

A closure (a function that uses a variable from an enclosing scope) looks the variable up **when it is called**, not when it's created. All three lambdas share the one variable `i`, which is 2 by the time they run. Binding it as a default argument (`lambda i=i: i`) captures the current value. `functools.partial(f, i)` is another fix.

Other functional helpers worth knowing

- `operator.itemgetter(1)`, `attrgetter("age")`, `methodcaller("strip")` — fast, readable replacements for small lambdas used as `key=`.
- `functools.partial(f, x)` — pre-fill arguments.
- `sorted` / `min` / `max` accept `key=`; `sorted` is **stable** (equal keys keep their input order), so you can sort by several keys in passes or with a tuple key.
- `enumerate(it, start=1)` instead of `range(len(it))`.

Common mistakes

- Forgetting that `map` / `filter` / `zip` return one-shot iterators in Python 3.
- `reduce` without an initial value on an empty sequence → `TypeError`.
- Using `zip` on lists you assume are equal length — use `strict=True`.
- `any([expensive(x) for x in xs])` — the list computes everything; drop the brackets.

Real-world examples

`sorted(records, key=itemgetter("created_at"))` is everywhere in data code; `all(field in payload for field in REQUIRED)` for request validation; `dict(zip(header, row))` when reading CSVs; PySpark and pandas expose `map` / `filter` / `reduce`-style APIs (`rdd.map`, `Series.map`) that come straight from this functional vocabulary.

In the interview

> **30-second answer:** "`map` and `filter` are lazy in Python 3; I prefer comprehensions when I'd otherwise need a lambda, and built-ins like `sum`, `max`, `any`, `all` over `reduce`. `any` / `all` short-circuit, so I pass generator expressions. `zip` truncates silently, so for data that must align I use `strict=True`. And closures bind late — capture loop variables with a default argument or `partial`."

- *What's `all([])`?* — `True`.
- *Why prefer `itemgetter` to a lambda for `key=`?* — Slightly faster and self-describing.

Self-check

1. `list(filter(None, [0, "", "a", 3]))`? → `['a', 3]`.
2. `reduce(lambda a, b: a * b, [2, 3, 4])`? → 24.
3. `zip([1, 2, 3], "ab", strict=True)` when consumed? → `ValueError`.
4. Why does `any(x > 10 for x in range(10**9))` return instantly? → It stops at 11.

TL;DR

Comprehensions over `map` / `filter` + lambda; built-ins over `reduce`; generator expressions inside `any` / `all`; `zip(strict=True)` when lengths must match; watch late-binding closures.

11.8 Chapter self-check

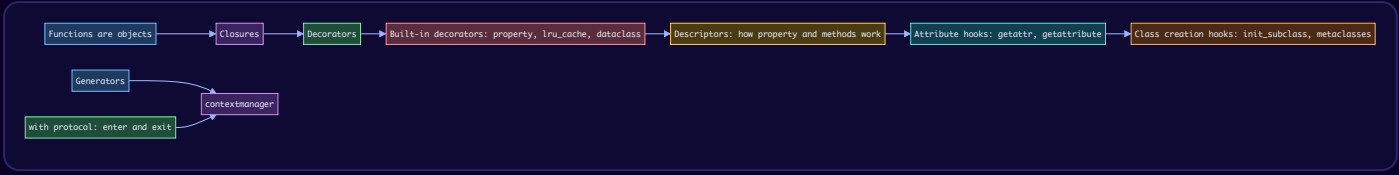
1. What two methods make an object an iterator? → `__iter__` (returning self) and `__next__`.
2. Why is `list(gen)` empty the second time? → Generators are one-shot iterators.
3. When does the body of a generator function start running? → On the first `next()`.
4. How do you get a sub-generator's `return` value? → `result = yield from sub()`.
5. What must you do before `gen.send(value)`? → Prime it with `next(gen)`.
6. Why group words with `groupby` only after sorting? → It groups consecutive equal keys only.
7. `permutations("abc", 2)` vs `combinations("abc", 2)` counts? → 6 vs 3.
8. Memory of `sum(x*x for x in range(n))`? → Constant.
9. Filter vs choose in a comprehension — where does the `if` go? → Filter after `for`; `a if c else b` before `for`.
10. How do you make `zip` fail on mismatched lengths? → `strict=True`.

Chapter TL;DR

- **Protocol:** iterable (`__iter__`) → iterator (`__next__` + `__iter__` returning self) → `StopIteration`. `for` = `iter()` + `next()` loop. Iterators are single-pass.
- **Generators:** `yield` makes a lazy iterator with paused state; generator expressions `( … )` are the inline form. `yield from` delegates; `send` / `throw` / `close` make two-way generators (mostly replaced by `async`).
- **Laziness:** constant memory and early exit; materialise with `list()` only when you need reuse or indexing.
- **itertools:** `count` / `cycle` / `repeat`, `chain` / `islice` / `takewhile` / `dropwhile`, `groupby` (sort first!), `product` / `permutations` / `combinations`, `accumulate`, `pairwise`, `batched`, `zip_longest`, `tee`.
- **Comprehensions:** short, no side effects, filter-if after / choose-if-else before; generator form when consumed once.
- **Functional:** comprehensions over `map` / `filter` + lambda; `sum` / `max` / `any` / `all` over `reduce`; `zip(strict=True)`; closures bind late.

Chapter 12 — Decorators, Context Managers and Metaprogramming

> **What you will learn.** How decorators work (plain, with arguments, stacked, class decorators, class-based decorators), why `functools.wraps` matters, and the decorators you use every day (`@property`, `@classmethod`, `@staticmethod`, `@lru_cache` / `@cache`, `@dataclass`). Then context managers: the `with` protocol, class-based managers, `@contextlib.contextmanager`, `suppress`, and `ExitStack`, plus async and reentrant managers. Finally metaprogramming — code that manipulates code: descriptors (the machinery behind properties and methods), `__getattr__` vs `__getattribute__`, metaclasses (what they are and why you rarely need them), `__init_subclass__`, the caveats of monkey patching, and everyday introspection (`getattr`, `inspect`, and why not `eval`).
>
> **Prerequisites.** Chapter 10 (classes, dunders) and Chapter 11 (generators — `@contextmanager` is built on them). Key idea: **functions and classes are objects** you can pass around, wrap and create at run time.
>
> **Code.** `chapters/12-decorators-context-managers-metaprogramming/code/` — `decorators.py`, `context_managers.py`, `descriptors_attrs.py`, `metaprogramming.py`. Timings in outputs vary by machine.



12.1 Decorators

What it is

A **decorator** is a callable that takes a function (or class) and returns a replacement — usually a **wrapper** that adds behaviour before/after calling the original. The `@` syntax is shorthand:

```
@timed
def slow_add(a, b): ...
# is exactly
def slow_add(a, b): ...
slow_add = timed(slow_add)
```

Analogy: gift-wrapping — the present (your function) is unchanged, but whoever receives it gets the wrapping first.

Two building blocks make this possible:

1. **First-class functions** — functions can be passed as arguments and returned.
2. **Closures** — an inner function remembers variables from the enclosing function (`func` inside `wrapper`) even after the outer function has returned.

(Closures and a first look at decorators are in Chapter 7 §7.4–7.6; this chapter goes deeper.)

Why it matters

Decorators separate **cross-cutting concerns** — logging, timing, caching, retries, authentication, registration — from business logic, and apply them with one line. This is Python's native form of the Decorator pattern (see LLD Handbook ch. 9) and of aspect-oriented programming.

How it works

1. At **definition time** (usually import time) Python creates the function, then calls `decorator(func)` and binds the name to the result.
2. At **call time**, callers actually call the wrapper, which calls the original via the closure.
3. The wrapper should accept `*args, **kwargs` and **return** the original's result, so it works for any signature.

```
def timed(func):
    @functools.wraps(func)                      # copy name, docstring, etc.
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)        # call the original, return its result
        finally:
            elapsed = (time.perf_counter() - start) * 1000
            print(f"  [timed] {func.__name__} took {elapsed:.1f} ms")
    return wrapper

@timed
def slow_add(a, b):
    """Add slowly."""
    time.sleep(0.02)
    return a + b
```

Output:

```
by hand: HELLO ANN
without wraps, name is: wrapper
  [timed] slow_add took 21.3 ms
slow_add: 5 | name: slow_add | doc: Add slowly. | original: slow_add
```

`functools.wraps`

Without it, the decorated function's `__name__` is `"wrapper"` and its docstring is gone — which breaks debugging, logs, `help()`, documentation tools and frameworks that look up functions by name (e.g. Flask endpoints). `@functools.wraps(func)` copies `__module__`, `__name__`, `__qualname__`, `__doc__`, `__annotations__` (and more), merges `__dict__`, and sets `wrapper.__wrapped__ = func` so tools (and you) can reach the original. **Always use it.**

Decorators with arguments — three levels

`@retry(times=3)` first *calls* `retry(times=3)`, which must *return* the real decorator. So there are three nested functions: factory → decorator → wrapper.

```
def retry(times: int = 3, exceptions=(Exception,)):   # 1. factory: receives the options
    def decorator(func):                               # 2. decorator: receives the function
        @functools.wraps(func)
        def wrapper(*args, **kwargs):                  # 3. wrapper: runs on every call
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    print(f"  [retry] attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
        return wrapper
    return decorator

@retry(times=3, exceptions=(ConnectionError,))
def flaky_fetch(): ...        # fails twice, then returns "data"
```

Output:

```
  [retry] attempt 1 failed: timeout
  [retry] attempt 2 failed: timeout
flaky_fetch -> data
```

(Real retry code should also wait between attempts with exponential backoff and jitter — libraries such as `tenacity` do this.)

Stacking

```
@tag("b")
@tag("i")
def text(): return "hi"
# == text = tag("b")(tag("i")(text))
```

Decorators are **applied bottom-up** (closest to `def` first) and their wrappers **run top-down** (outermost first). Output: `stacked: <b><i>hi</i></b>`. Order matters: e.g. put `@app.route` outermost and an auth check inside it, or the route registers the unprotected function.

Stateful and registering decorators

A decorator doesn't have to wrap. It can **register** the function and return it unchanged — this is how many frameworks work:

```
REGISTRY = {}
def register(func):
    REGISTRY[func.__name__] = func       # side effect at definition time
    return func

def count_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1               # state stored as a function attribute
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper
```

Output: `registry: ['export_csv', 'export_json']` and `ping.calls: 2`.

Class decorators and classes as decorators

- A **class decorator** receives a *class* and returns a class (often the same one, modified). `@dataclass` is one.
- A **class used as a decorator** is callable because instances implement `__call__`; use `functools.update_wrapper(self, func)` in `__init__`.

```
def add_repr(cls):
    def __repr__(self):
        fields = ", ".join(f"{k}={v!r}" for k, v in vars(self).items())
        return f"{cls.__name__}({fields})"
    cls.__repr__ = __repr__
    return cls

class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func, self.calls = func, 0
    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)
```

Output: `class decorator: Point(x=1, y=2)` and `CountCalls: 2 | name: square`.

Caveat: a class-based decorator applied to a *method* won't bind `self` automatically (the instance isn't a descriptor unless you implement `__get__`, §12.4). Function-based decorators don't have this problem — prefer them.

Trade-offs

Approach	Pros	Cons
Function decorator	Simple, works on methods	State via closure/attributes
Decorator factory (with args)	Configurable	Three levels of nesting; easy to forget the call parentheses
Class-based decorator	Natural place for state	Needs `__get__` to work on methods
Class decorator	Modify/register whole classes	Can surprise readers; metaclass-like power
No decorator (call helper explicitly)	Obvious control flow	Repetition

When not to use it

When the "cross-cutting" logic is used once; when it changes the function's signature or return type in surprising ways; when stacking five decorators makes the call path impossible to follow.

Common mistakes

- Forgetting `return` in the wrapper → the decorated function returns `None`.
- Forgetting `@functools.wraps`.
- Using a factory without calling it: `@retry` instead of `@retry()` → `retry` receives the function as `times`.
- Heavy work at decoration time (it runs on import).
- Wrong stacking order (e.g. caching outside an auth check caches responses across users).

Real-world examples

Flask `@app.route("/")` and FastAPI `@app.get("/")` register view functions; pytest `@pytest.fixture` and `@pytest.mark.parametrize`; Click `@click.command()` / `@click.option()`; Celery `@app.task`; Django `@login_required`; `@functools.singledispatch` (function overloading by argument type); `@contextlib.contextmanager`; `@unittest.mock.patch`.

In the interview

> **30-second answer:** "A decorator is a callable that takes a function and returns a replacement, usually a closure wrapper that runs code around the original; `@d` above `def f` is `f = d(f)` at definition time. I always use `functools.wraps` to keep the name, docstring and `__wrapped__`. A decorator with arguments is a factory returning a decorator — three levels. Stacked decorators apply bottom-up and run top-down."

- *When does a decorator run?* — Once, when the decorated function is defined (typically import); the wrapper runs on each call.
- *Write a decorator that times a function.* — See `timed` above (use `perf_counter`, `try/finally`, `wraps`).
- *How do you decorate a class?* — A function that takes and returns a class, e.g. `@dataclass`.

Self-check

1. Rewrite `@a @b def f` without `@`. → `f = a(b(f))`.
2. What does `functools.wraps` set that lets you reach the original? → `__wrapped__`.
3. Why three functions in a decorator with arguments? → Outer takes options, middle takes the function, inner runs per call.

4. What happens if the wrapper doesn't `return func(...)`? → Callers get `None`.

TL;DR

Decorator = function in, function out, applied at definition time. Use `*args, **kwargs`, return the result, `@functools.wraps`. Factories for arguments; stack with care.

12.2 Everyday built-in decorators

Decorator	What it does	Where covered
`@property` (+ `.setter`, `.deleter`)	Method accessed like an attribute; implemented as a descriptor	Ch. 10 §10.1, §12.4
`@classmethod`	Passes the class as `cls` (alternative constructors)	Ch. 10 §10.1
`@staticmethod`	No implicit first argument	Ch. 10 §10.1
`@abstractmethod`	Marks methods an ABC subclass must implement	Ch. 10 §10.7
`@dataclass`	Class decorator generating `__init__` / `__repr__` / `__eq__` …	Ch. 10 §10.10
`@functools.total_ordering`	Class decorator filling in comparison methods	Ch. 10 §10.2
`@functools.lru_cache(maxsize=128)` / `@functools.cache`	Memoisation (below)	here
`@functools.cached_property`	Compute once per instance, store in the instance	§12.4
`@functools.wraps`	Copies metadata onto wrappers	§12.1
`@contextlib.contextmanager`	Generator → context manager	§12.3
`@typing.override` (3.12)	Marks a method as overriding a parent's, so type checkers can flag typos	–

`@lru_cache` and `@cache` – memoisation

Memoisation stores the result of a function call keyed by its arguments, so a repeated call returns instantly. `lru_cache(maxsize=N)` keeps the N most recently used results and evicts the **least recently used** (LRU) when full; `maxsize=None` or `@functools.cache` (3.9) is unbounded. Default `maxsize` is 128.

```
@functools.lru_cache(maxsize=None)
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

Output:

```
fib(25) plain 6 ms vs cached 0.01 ms -> cached faster: True
cache_info: CacheInfo(hits=23, misses=26, maxsize=None, currsize=26)
unhashable arg: unhashable type: 'list'
```

Worked numbers: plain recursive `fib(25)` makes 242,785 calls (exponential); cached, each `fib(k)` for k = 0…25 is computed once → **26 misses**, and each of `fib(3)…fib(25)` finds its second sub-call `fib(k-2)` already cached → **23 hits**.

Rules and caveats:

1. Arguments must be **hashable** (the cache is a dict) – no lists/dicts.
2. Only for **pure** functions (same input → same output, no side effects). Don't cache `now()` or database reads that must be fresh.

3. Returned objects are shared — mutating a cached list corrupts the cache.
4. On **methods**, the cache holds a reference to `self`, keeping instances alive (a memory leak) and sharing one cache across instances; prefer `cached_property` or a per-instance cache.
5. `fib.cache_clear()` empties it; `typed=True` caches `f(3)` and `f(3.0)` separately.

Real world: caching parsed config, compiled regexes (`re` keeps its own internal cache), expensive pure computations, dynamic-programming solutions in interviews.

12.3 Context managers

What it is

A **context manager** is an object that sets something up at the start of a `with` block and **guarantees** tear-down at the end — even if the block raises or `return`s. It implements `__enter__` and `__exit__`.

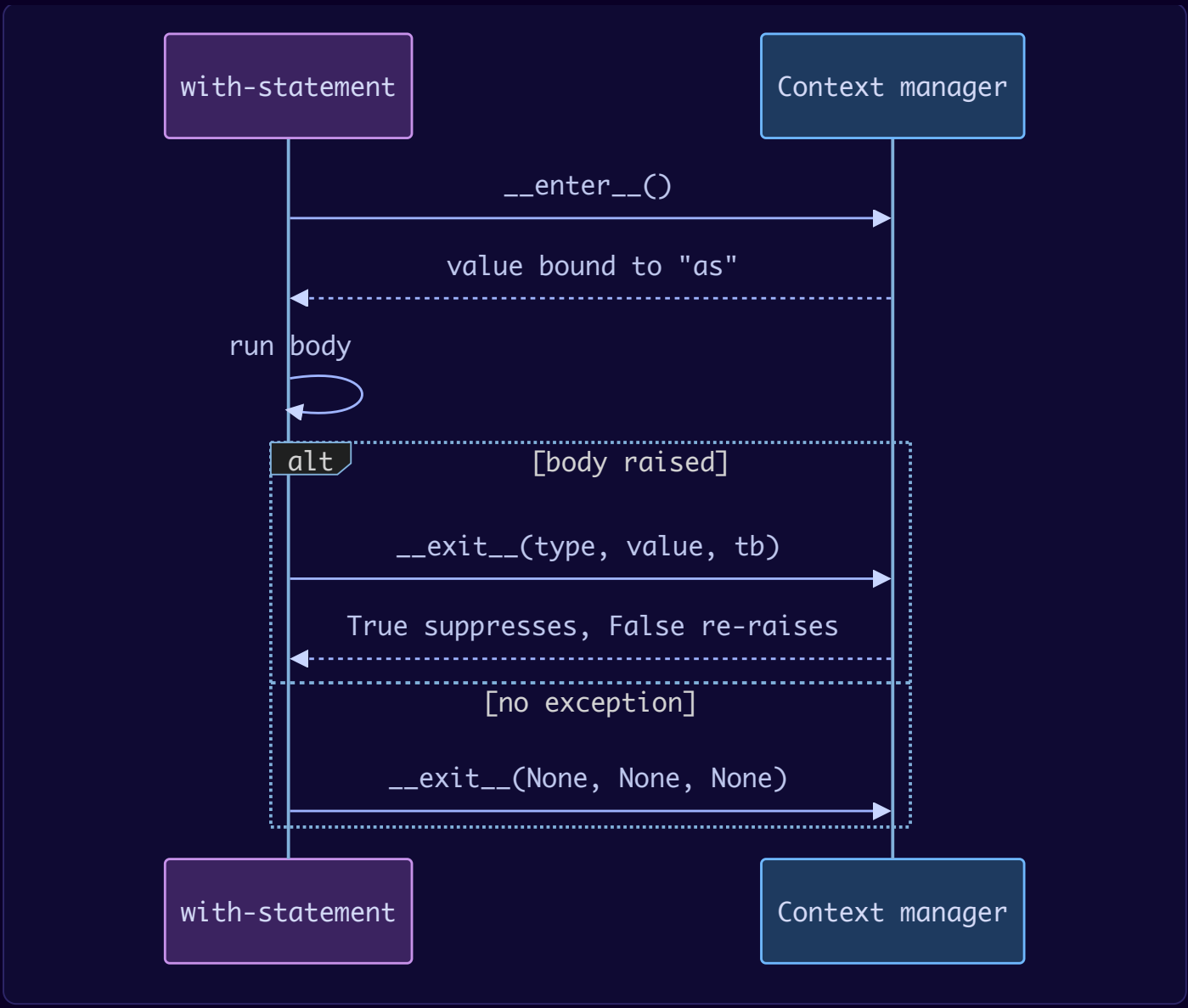
Analogy: a hotel key card — you get it on check-in (`__enter__`), and checkout (`__exit__`) happens no matter how your stay ends.

Why it matters

Resource leaks (files, sockets, locks, DB transactions, temporary state) are a top source of bugs. `with` replaces fragile `try/finally` boilerplate and makes clean-up impossible to forget.

How it works — `with EXPR as VAR: BODY`

1. Evaluate `EXPR` to get the manager `m`.
2. Call `m.__enter__()`; bind its **return value** to `VAR` (it doesn't have to be `m` — `open()` returns the file itself, a lock returns `True`).
3. Run `BODY`.
4. Always call `m.__exit__(exc_type, exc_value, traceback)` — all three `None` if no exception.
5. If `__exit__` returns a **truthy** value, the exception is **suppressed**; otherwise it propagates. Return `False` / `None` unless suppressing is the whole point.



Class-based

```python
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False                    # propagate exceptions

class Suppress:                         # our own contextlib.suppress
    def __init__(self, *excs): self.excs = excs
    def __enter__(self): return self
    def __exit__(self, exc_type, exc, tb):
        return exc_type is not None and issubclass(exc_type, self.excs)
```

Generator-based: `@contextlib.contextmanager`

Write a generator that **yields exactly once**: code before `yield` is `__enter__`, the yielded value goes to `as`, code after is `__exit__`. If the `with` body raises, the exception is **re-raised at the `yield`** – so wrap it in `try/finally` (or `except`) or your clean-up is skipped.

```python
@contextlib.contextmanager
def cd(path):                           # same idea as contextlib.chdir (3.11+)
    old = os.getcwd()
    os.chdir(path)
    try:
        yield path                      # the with-body runs here
    finally:
        os.chdir(old)                   # runs even if the body raises

@contextlib.contextmanager
def transaction(log):
    log.append("BEGIN")
    try:
        yield
    except Exception:
        log.append("ROLLBACK")
        raise                           # re-raise: don't hide the error
    else:
        log.append("COMMIT")
```

`ExitStack` — a dynamic number of managers

`contextlib.ExitStack` lets you enter an unknown number of context managers (e.g. open N files from a list) and registers arbitrary clean-up callbacks. On exit it unwinds them in **reverse (LIFO)** order, and if entering the k-th one fails, the k−1 already entered are still closed. `stack.pop_all()` transfers ownership out if setup succeeded (useful in constructors).

```python
with contextlib.ExitStack() as stack:
    files = [stack.enter_context(Resource(n, log)) for n in names]
    stack.callback(log.append, "callback ran")
    log.append(f"using {len(files)} resources")
```

Output (`context_managers.py`):

```
Timer: elapsed > 0: True
Suppress: survived ZeroDivisionError
contextlib.suppress: survived FileNotFoundError
cd restored: True | was inside tmp: True
transaction log: ['BEGIN', 'insert row', 'COMMIT', 'BEGIN', 'bad insert', 'ROLLBACK']
ExitStack: ['open a.txt', 'open b.txt', 'open c.txt', 'using 3 resources', 'callback ran', 'close c.txt',
'close b.txt', 'close a.txt']
ExitStack partial failure: ['open ok1', 'close ok1'] | cannot open
redirect_stdout captured: 'captured!\n'
nullcontext gives: x
two managers: ['open r1', 'open r2', 'close r2', 'close r1']
```

The `contextlib` toolbox

Tool	Use
`contextmanager` / `asynccontextmanager`	Generator (or async generator) → context manager
`suppress(*excs)`	Ignore specific exceptions (`with suppress(FileNotFoundError): os.remove(p)`)
`ExitStack` / `AsyncExitStack`	Dynamic number of managers + callbacks
`closing(obj)`	Call `obj.close()` on exit for objects without `__exit__`
`redirect_stdout` / `redirect_stderr`	Temporarily capture output
`nullcontext(x)`	A do-nothing manager, for "optional" context (`with lock if threaded else nullcontext():`)
`chdir(path)` (3.11)	Temporarily change the working directory
`AbstractContextManager`	ABC base giving a default `__enter__` that returns `self`

Several managers in one statement: `with A() as a, B() as b:` – or, since 3.10, parenthesised across lines. They are exited in reverse order (see "two managers" above).

Async context managers. In `async def` code (Chapter 16) the statement is `async with`, and the protocol methods are the coroutines `__aenter__` / `__aexit__`, so setup and clean-up can `await` I/O (opening a network connection, releasing a pooled DB connection). `@contextlib.asynccontextmanager` builds one from an `async def` generator exactly like `@contextmanager`. Examples: `async with aiohttp.ClientSession() as s:`, `async with asyncio.TaskGroup() as tg:`, `async with lock:` for an `asyncio.Lock`.

Reusable and reentrant managers. A *single-use* manager (a `@contextmanager` object, an open file) can be entered only once. A *reusable* one can be entered again after it exits. A *reentrant* one can even be entered again **while it is already active** (nested `with m:` inside `with m:`) – `threading.RLock` and `contextlib.redirect_stdout` are reentrant; a plain `threading.Lock` is not (re-acquiring it from the same thread deadlocks).

Trade-offs

	Class-based	`@contextmanager`
Code size	More	Less
Reusable object with state/methods	Yes	No (one-shot generator)
Can be re-entered / reused	If you design it so	No – each call makes a new one
Exception handling	Explicit `__exit__` args	`try/except/finally` around `yield`
Also usable as a decorator	Via `ContextDecorator`	Yes – `@contextmanager` objects work as decorators too

Common mistakes

- `@contextmanager` without `try/finally` around `yield` → clean-up skipped on error.
- Yielding more than once → `RuntimeError: generator didn't stop`.
- Returning `True` from `__exit__` accidentally → silently swallowing all exceptions.
- Using the resource after the block (`f.read()` after `with open(...)` → `ValueError: I/O operation on closed file`).

Real-world examples

`open()`, `threading.Lock`, `sqlite3.connect()` (as a context manager it commits or rolls back the transaction – it does *not* close the connection), `requests.Session`, `tempfile.TemporaryDirectory`, `decimal.localcontext`, `unittest.mock.patch`, `pytest.raises`, PyTorch `torch.no_grad()`, `concurrent.futures.ThreadPoolExecutor` (waits for tasks on exit).

In the interview

> **30-second answer:** "A context manager implements `__enter__` and `__exit__`; `with` calls enter, binds its return value to `as`, runs the body, and always calls exit with the exception info, which can suppress it by returning true. For simple cases I write a generator with `@contextmanager` – setup, `yield`, clean-up in `finally`. For a variable number of resources I use `ExitStack`, which unwinds them in reverse order even if one fails to open."

- *How do you suppress an exception?* – Return a truthy value from `__exit__`, or use `contextlib.suppress`.
- *What happens to an exception inside a `@contextmanager` body?* – It's thrown into the generator at the `yield`.

Self-check

1. What gets bound by `as`? → The return value of `__enter__`.
2. Three arguments of `__exit__`? → Exception type, value, traceback (all `None` if no error).
3. Order managers exit in `with A(), B():`? → B then A.
4. Why use `ExitStack`? → Unknown number of managers, or conditional ones, with guaranteed LIFO clean-up.

TL;DR

`with` guarantees clean-up: `__enter__` → body → `__exit__` (truthy return suppresses). `@contextmanager` = setup / `yield` / `finally` clean-up. `ExitStack` for dynamic sets. Know `suppress`, `closing`, `nullcontext`, `redirect_stdout`, `chdir`.

12.4 Descriptors – how properties and methods really work

What it is

A **descriptor** is any object, stored as a **class attribute**, that defines at least one of:

- `__get__(self, obj, objtype=None)` – runs on attribute read,
- `__set__(self, obj, value)` – runs on assignment,
- `__delete__(self, obj)` – runs on `del`.

Optionally `__set_name__(self, owner, name)` (3.6) is called at class creation so the descriptor learns the attribute name it was assigned to.

Two kinds:

- **Data descriptor** – defines `__set__` or `__delete__`. `property` is one.
- **Non-data descriptor** – only `__get__`. Plain functions and `functools.cached_property` are.

Why it matters

Descriptors are the mechanism behind `property`, `classmethod`, `staticmethod`, bound methods, `__slots__`, and ORM fields (Django/SQLAlchemy model columns). Writing your own gives **reusable** attribute logic – one `Positive` validator used for many fields, instead of copy-pasted property pairs.

How it works – attribute lookup precedence for `obj.attr`

1. Find `attr` on `type(obj)` along the MRO. If it's a **data descriptor**, call its `__get__` – it **wins over the instance dict**.
2. Else, if `attr` is in `obj.__dict__`, return that.
3. Else, if the class attribute is a **non-data descriptor**, call its `__get__`; if it's a plain class attribute, return it.
4. Else call `__getattr__` if defined; otherwise raise `AttributeError`.

(All of this is implemented by `object.__getattribute__`.) Assignment `obj.attr = v` calls a data descriptor's `__set__` if there is one; otherwise it writes to `obj.__dict__`.

Consequences:

- **Methods:** a function is a non-data descriptor; `Greeter.__dict__["hello"].__get__(g)` returns the bound method — this is what `g.hello` does.
- `property` always defines `__set__` (raising `AttributeError` if there's no setter), so it's a data descriptor and an instance-dict entry can't shadow it.
- `cached_property` is non-data: the first access computes and stores the value in `obj.__dict__`; later accesses find it in step 2 and never call the descriptor again.

```
class Positive:
    def __set_name__(self, owner, name):
        self.private = "_" + name               # e.g. "_qty"
    def __get__(self, obj, objtype=None):
        if obj is None:                          # accessed on the class
            return self
        return getattr(obj, self.private)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f"{self.private[1:]} must be > 0, got {value}")
        setattr(obj, self.private, value)

class Order:
    qty = Positive()                             # one reusable validator...
    price = Positive()                           # ...for many fields
    def __init__(self, qty, price):
        self.qty = qty                           # runs Positive.__set__
        self.price = price

class my_property:                               # property, re-implemented
    def __init__(self, fget, fset=None):
        self.fget, self.fset = fget, fset
    def __get__(self, obj, objtype=None):
        return self if obj is None else self.fget(obj)
    def __set__(self, obj, value):
        if self.fset is None:
            raise AttributeError("read-only")
        self.fset(obj, value)
```

Output (`descriptors_attrs.py`, first part):

```
Order: 2 9.99 | instance dict: {'_qty': 2, '_price': 9.99}
descriptor refused: qty must be > 0, got 0
on the class: Positive object (the descriptor itself)
my_property: 212.0
AttributeError: read-only
  (computing data once)
first: [1, 2, 3]
second: [1, 2, 3] | stored in instance dict: True
function has __get__: True | f.__get__(g)(): hello
data descriptor? True | cached_property has __set__? False
```

Common mistakes

- Storing the value on the descriptor itself (`self.value = v`) — the descriptor is shared by **all instances**, so every object would see the same value. Store per-instance data on `obj`.
- Putting a descriptor in `__init__` (`self.qty = Positive()`) — descriptors only work as **class** attributes.
- Forgetting the `obj is None` case (class access).

In the interview

30-second answer: "A descriptor is a class attribute with `__get__`, and optionally `__set__` / `__delete__`. Data descriptors (with `__set__`) take priority over the instance dict; non-data ones don't. `property`,

> `classmethod`, `staticmethod` and even bound methods are built on this – a function's `__get__` returns a bound method. I'd write one for reusable validation across fields, like ORMs do for columns."

- *Why can `cached_property` be overwritten by the instance?* – It's a non-data descriptor, so the instance dict wins.

TL;DR

Descriptors = class attributes that intercept get/set/delete. Data descriptors beat the instance dict; non-data don't. They power properties, methods, `cached_property`, `__slots__` and ORM fields.

12.5 `__getattr__` vs `__getattribute__`

What it is

Hook	Called when	Typical use
`__getattr__(self, name)`	Only when **normal lookup fails** (would raise `AttributeError`)	Delegation/proxies, lazy attributes, friendly defaults, deprecation shims
`__getattribute__(self, name)`	On **every** attribute access (it *is* normal lookup)	Rare: tracing, access control, very special proxies
`__setattr__` / `__delattr__`	Every assignment / deletion	Validation, immutability, change tracking
Module-level `__getattr__(name)` (PEP 562, 3.7)	Missing attribute on a *module*	Lazy imports, deprecation warnings

```python
class Proxy:
    def __init__(self, target):
        self._target = target
    def __getattr__(self, name):                # only for names not found normally
        print(f"  __getattr__({name!r}) -> delegating")
        return getattr(self._target, name)

class Logged:
    def __init__(self):
        self.x = 1
    def __getattribute__(self, name):           # every access
        if not name.startswith("__"):
            print(f"  __getattribute__({name!r})")
        return super().__getattribute__(name)   # MUST delegate via super()/object
```

Output:

```
  __getattr__('upper') -> delegating
proxy.upper(): HELLO WORLD
proxy._target found normally (no __getattr__ call): hello world
  __getattribute__('x')
value: 1
```

Common mistakes

- Inside `__getattribute__`, writing `self.anything` → calls `__getattribute__` again → infinite recursion. Use `super().__getattribute__(name)` or `object.__getattribute__(self, name)`.
- In `__getattr__`, accessing an attribute that may not exist yet (e.g. `self._target` before `__init__` has set it, as during unpickling or copying) → infinite recursion. Guard or raise `AttributeError`.
- Returning `None` for unknown names instead of raising `AttributeError` → breaks `hasattr`, `getattr(obj, n, default)` and copy/pickle.

- Implicit special-method lookup (`len(p)` , `p + 1`) **bypasses** both hooks — Python looks up dunders on the type directly. A proxy must define those dunders explicitly.

Real-world examples

`unittest.mock.Mock` uses `__getattr__` to create child mocks on the fly; RPC/SDK clients build dynamic endpoints (`client.users.list`); pandas lets you access columns as attributes (`df.price`) via `__getattr__` ; lazy-loading packages use module `__getattr__` .

TL;DR

`__getattr__` = fallback for missing attributes (safe, common). `__getattribute__` = intercepts everything (rare, recursion-prone). Dunders used by syntax skip both.

12.6 Metaclasses and `__init_subclass__`

What it is

Classes are objects too, so something must create them: a **metaclass** is "the class of a class". The default metaclass is `type` : `type(42)` is `int` , `type(int)` is `type` , and even `type(type)` is `type` . A `class` statement is roughly `Name = type("Name", bases, namespace)` .

```
def speak(self): return f"{self.name} speaks"
Dog = type("Dog", (object,), {"name": "Rex", "speak": speak})   # a class, built by a call
```

How a class is created

1. Python collects the class body into a namespace dict (by executing it).
2. It determines the metaclass (explicit `metaclass=` , or the most derived metaclass of the bases, default `type`).
3. It calls `Meta(name, bases, namespace)` → `Meta.__new__` builds the class object, `Meta.__init__` initialises it.
4. While the class object is being built (inside `type.__new__` , so before `Meta.__init__`), descriptors' `__set_name__` hooks run, then the parent's `__init_subclass__` .
5. Class decorators run last, on the finished class.

A custom metaclass subclasses `type` and overrides `__new__` / `__init__` (or `__call__` , to control **instance** creation — e.g. a Singleton metaclass).

`__init_subclass__` — the simpler hook

PEP 487 (Python 3.6) added `__init_subclass__(cls, **kwargs)` : a classmethod on a parent that runs **every time a subclass is created**, receiving any keyword arguments from the class line (`class Home(Handler, route="/")`). It covers the most common metaclass uses — registries, validation, configuration — with ordinary inheritance and no metaclass conflicts.

```python
class RegistryMeta(type):                          # metaclass version
    registry: dict[str, type] = {}
    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if bases:
            mcls.registry[name.lower()] = cls
        return cls

class Plugin(metaclass=RegistryMeta): pass
class CsvExporter(Plugin): ...

class Handler:                                     # __init_subclass__ version (preferred)
    handlers: dict[str, type] = {}
    def __init_subclass__(cls, /, route: str, **kwargs):
        super().__init_subclass__(**kwargs)
        Handler.handlers[route] = cls

class Home(Handler, route="/"): ...

class RequiresRun:                                 # enforce a rule at class-creation time
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        if not callable(cls.__dict__.get("run")):
            raise TypeError(f"{cls.__name__} must define run()")
```

Output (`metaprogramming.py`):

```
Rex speaks | type(Dog): type | type(type): type
type(42): int | type(int): type
metaclass registry: {'csvexporter': <class '__main__.CsvExporter'>, 'jsonexporter': <class
'__main__.JsonExporter'>}
type(CsvExporter): RegistryMeta
__init_subclass__ routes: {'/': 'Home', '/about': 'About'}
GoodJob: ran
TypeError: BadJob must define run()
```

Note `type(CsvExporter)` is `RegistryMeta` — the metaclass is inherited by every subclass.

When to use what (least power first)

Need	Use
Add/modify methods of one class	Class decorator
Reusable attribute behaviour	Descriptor
React to / register / validate every subclass	`__init_subclass__`
Control the class namespace itself, customise `isinstance`, change how instances are created, or build framework-level DSLs	Metaclass

Python core developer Tim Peters famously remarked that metaclasses are deeper magic than 99% of users should worry about — and since 3.6, `__init_subclass__` and `__set_name__` have removed most remaining reasons.

Common mistakes

- **Metaclass conflict:** combining two base classes with unrelated metaclasses → `TypeError: metaclass conflict`. (One reason to avoid custom metaclasses in libraries.)
- Forgetting `super().__init_subclass__(**kwargs)` — breaks cooperative inheritance.
- Using a metaclass for something a decorator could do.

Real-world examples

`abc.ABCMeta` (enforces abstract methods), `enum.EnumType` (formerly `EnumMeta`, makes members), Django's `ModelBase` (turns field declarations into a model with a manager and metadata), Pydantic's `ModelMetaclass` (collects fields and builds validators). Many plugin systems use `__init_subclass__` registries.

In the interview

> **30-second answer:** "A metaclass is the class of a class – `type` by default – and it runs when a class statement executes, so it can inspect or change the class being created. Frameworks like Django's ORM and `abc` use them. In application code I almost never need one: class decorators, descriptors and `__init_subclass__` cover registration and validation with less magic and no metaclass conflicts."

- *What's `type(type)`?* – `type`.
- *What is `__init_subclass__` for?* – A parent hook that runs for each new subclass (registries, validation, per-subclass config).

TL;DR

Everything is an object; classes are made by metaclasses (`type`). Prefer class decorators → descriptors → `__init_subclass__` → metaclasses, in that order.

12.7 Monkey patching – caveats

What it is

Monkey patching is replacing attributes (functions, methods, classes) of modules or classes **at run time**, from outside, e.g. `Greeter.hello = lambda self: "patched!"`. Because Python classes and modules are mutable objects, this always works – which is exactly the danger.

Output (`descriptors_attrs.py`, last part):

```
monkey patched: patched! | existing instance too: patched!
restored: hello
```

Patching a class affects **every existing instance** too, because methods are looked up on the class.

Legitimate uses

- **Tests** – replacing slow or external dependencies (network, time, randomness). Use `unittest.mock.patch` or pytest's `monkeypatch` fixture, which **undo the change automatically**.
- **Hot-fixing a third-party bug** until an upstream fix ships (document it and pin the version).
- Framework-level instrumentation, e.g. gevent's `monkey.patch_all()` makes the standard library cooperative, and APM/tracing agents (OpenTelemetry auto-instrumentation) wrap library functions.

Caveats

1. **Global and invisible**: every user of the module/class sees the change, including other libraries and other threads.
2. **Patch where it's looked up, not where it's defined**: if `app.py` did `from utils import send_email`, you must patch `app.send_email`; patching `utils.send_email` doesn't affect the name already bound in `app`.
3. **Order-dependent**: code that imported or cached the original before the patch keeps it.
4. **Fragile across upgrades**: private internals you patch may change without notice.
5. **Always restore** (try/finally, or tools that do it for you), otherwise test pollution causes failures in unrelated tests.
6. Built-in types implemented in C (`str`, `int`, `list`) can't be patched: `str.upper = ...` raises `TypeError`.

In the interview

"Monkey patching is modifying classes or modules at run time. It's fine in tests via `mock.patch` or pytest's `monkeypatch`, which restore automatically and must target the name where it's used. In production I avoid it – it's global, order-dependent and breaks on upgrades – and prefer dependency injection, subclassing or a wrapper."

TL;DR

Monkey patching is powerful and global. Use it in tests with auto-restoring tools, patch where the name is looked up, and prefer dependency injection in production.

12.8 Introspection and dynamic code (brief)

Introspection means a program examining its own objects at run time. It is the everyday side of metaprogramming:

Tool	Does	Example
`getattr(obj, name, default)`, `setattr`, `hasattr`, `delattr`	Read/write attributes whose names are strings	`getattr(handler, f"do_{verb}")` to dispatch by name
`vars(obj)`, `dir(obj)`	Instance `__dict__`; all attribute names	Debugging, generic `__repr__`
`type(x)`, `isinstance`, `issubclass`, `callable`	Types and capabilities	
`inspect.signature(f)`	Parameter names, defaults, annotations	FastAPI and pytest read signatures to inject arguments/fixtures
`inspect.getsource(f)`, `inspect.getmembers(obj)`	Source text; all members	Documentation tools
`eval(expr)`, `exec(code)`	Run a string as Python	Almost never needed; **never on untrusted input** – it can run any code
`ast.literal_eval(s)`	Safely parse a literal (`"[1, 2, {'a': 3}]"`)	The safe replacement for `eval` on data

Rule of thumb: `getattr`-style introspection is normal Python; `eval`/`exec` on anything a user can influence is a security hole (the same class of bug as unpickling untrusted data, Chapter 9).

12.9 Chapter self-check

1. What does `@d` above `def f` do, and when? → `f = d(f)`, once at definition time.
2. Why `functools.wraps`? → Keeps `__name__`, `__doc__`, etc. and sets `__wrapped__`.
3. `@retry` vs `@retry()` for a factory decorator? → The factory needs to be called: `@retry()`.
4. Order of application/execution of stacked decorators? → Applied bottom-up, run top-down.
5. What's required of `lru_cache` arguments? → Hashable.
6. How does `__exit__` suppress an exception? → By returning a truthy value.
7. What must surround `yield` in a `@contextmanager`? → `try/finally` so clean-up runs on errors.
8. Data vs non-data descriptor precedence? → Data descriptor > instance dict > non-data descriptor.
9. When is `__getattr__` called? → Only when normal lookup fails.
10. Simplest way to register every subclass? → `__init_subclass__`.

Chapter TL;DR

- **Decorators:** function in, function out, at definition time. `*args, **kwargs`, return the result, `@functools.wraps`. Factories (three levels) for arguments; stacking applies bottom-up, runs top-down; class decorators modify classes; decorators can simply register.
- **Built-ins:** `property`, `classmethod`, `staticmethod`, `dataclass`, `total_ordering`, `lru_cache` / `cache` (hashable args, pure functions, beware methods), `cached_property`.
- **Context managers:** `__enter__` / `__exit__` guarantee clean-up; truthy `__exit__` suppresses. `@contextmanager` = setup, `yield` in `try`, clean-up in `finally`. `ExitStack` for dynamic resources (LIFO). `suppress`, `closing`, `nullcontext`, `redirect_stdout`, `chdir`.
- **Descriptors:** `__get__` / `__set__` / `__delete__` on class attributes; they power properties and methods. Data descriptors beat the instance dict.
- **Attribute hooks:** `__getattr__` for missing names (delegation), `__getattribute__` for everything (rare, use `super()`).
- **Class creation:** metaclasses (`type` by default) are rarely needed; prefer decorators, descriptors, `__init_subclass__`.
- **Monkey patching:** tests only (auto-restore, patch where used); avoid in production.

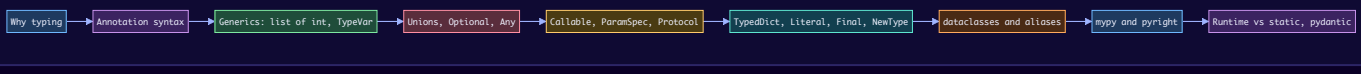
Chapter 13 — Type Hints and Static Typing

What you will learn. Why a dynamic language added optional types, and how to write them: annotation syntax, built-in generics such as `list[int]`, `Optional` / `Union` / `X | None`, `Any`, generics with `TypeVar` and the new 3.12 bracket syntax, `Callable` and `ParamSpec`, structural typing with `Protocol`, and the special forms `TypedDict`, `Literal`, `Final` and `NewType`. Then how types meet `dataclasses`, how to name types with aliases and the 3.12 `type` statement, how to run **mypy** and **pyright** and read their errors, what types do (and do not do) at runtime, and how **pydantic** turns hints into runtime validation.

Prerequisites. Functions (Chapter 7), classes and dataclasses (Chapter 10), decorators (Chapter 12).

Code. Every script is in `chapters/13-type-hints/code/` and runs on Python 3.12+. Script `04_run_mypy.py` needs `pip install mypy`:

```
python chapters/13-type-hints/code/02_generics_protocols.py
mypy --strict chapters/13-type-hints/code/03_special_forms.py
```



13.1 Why typing?

What it is

A **type hint** (or *annotation*) is a note attached to a variable, parameter or return value saying what kind of value it should hold: `def area(w: float, h: float) -> float`. Python itself **does not enforce** these notes. Separate programs called **static type checkers** (mypy, pyright) read your source code *without running it* and report places where the notes are contradicted.

Two words you will see everywhere:

- **Dynamic typing** — types belong to *values*, and are checked while the program runs (`"a" + 1` fails only when that line executes). Python is dynamically typed.
- **Static typing** — types are checked *before* the program runs, from the source text. Java, Go and Rust do this in the compiler.

Python's type hints give you **gradual typing**: you can add types to as much or as little of the code as you like, and un-annotated code is treated leniently.

Analogy: hints are road signs. The car (the interpreter) does not stop you from ignoring them; the type checker is the driving instructor who reads the route beforehand and says "this turn is a dead end".

Why it matters

1. **Bugs found early.** Passing `None` where a `str` is expected, misspelled attribute names, wrong argument order — caught before tests, before production.
2. **Documentation that cannot go stale.** A signature like `def fetch(ids: list[int]) -> dict[int, User]` explains itself, and the checker complains if the code drifts from it.
3. **Editor power.** Autocomplete, go-to-definition and safe renames all work much better when the editor knows the types (VS Code's Pylance is built on pyright).
4. **Safer refactoring** in large codebases: change a function's signature and the checker lists every caller that must change.

How it works

1. Annotations were added in Python 3.0 (PEP 3107) as free-form metadata; PEP 484 (Python 3.5) gave them standard meaning and created the `typing` module.

2. At runtime, annotations are stored on the object (`func.__annotations__`) and otherwise ignored.
3. A checker parses your code, infers the type of every expression, and compares it with what each annotation promises.
4. Un-annotated code is treated as `Any` (see §13.5), which is why you can adopt typing one file at a time.

Trade-offs

Benefit	Cost
Catches whole classes of bugs before running	Extra syntax to learn and write
Self-documenting APIs, better IDE support	Some dynamic patterns are awkward to type
Refactoring confidence at scale	False sense of safety if `Any` leaks everywhere
Zero runtime cost (hints are not checked)	…which also means no runtime protection

When not to use it

Throw-away scripts, notebooks exploring data, and tiny programs gain little. Types pay off in code that lives long, is shared by several people, or is a library others call.

Real-world examples

Dropbox type-checked about 4 million lines of Python with mypy (Guido van Rossum worked on mypy there). Instagram and Meta built the Pyre checker; Microsoft built pyright. **FastAPI** and **pydantic** read type hints at runtime to validate requests and generate API docs. The standard library's type information lives in the **typeshed** project, which every checker uses.

In the interview

"Python is dynamically typed, but since PEP 484 we can add optional type hints. They're ignored at runtime; a static checker like mypy or pyright verifies them before the code runs. It's gradual, so you can type a codebase incrementally. I use them on anything long-lived because they catch `None` bugs and make refactors safe."

- *Do hints make code faster?* No. CPython ignores them. (Tools like mypyc can compile typed code to C extensions, but that is a separate compiler.)
- *Does `x: int = "a"` raise?* No, it runs fine. Only the checker complains.

Self-check

1. Are annotations enforced when the program runs? → No; they are metadata for tools.
2. What does "gradual typing" mean? → Typed and untyped code can coexist; un-annotated parts are treated as `Any`.
3. Name two static checkers. → mypy, pyright (also Pyre; newer Rust-based ones like Astral's ty and Meta's Pyrefly).

TL;DR

Hints are optional, unenforced notes. A static checker reads them to find bugs before runtime. Adopt gradually; the biggest wins are in long-lived, shared code.

13.2 Annotation syntax

What it is

The places you can write an annotation:

```python
def greet(name: str, times: int = 1) -> str:   # parameters and return
    return ", ".join([f"hi {name}"] * times)

port: int = 8080                               # variable with value
scores: dict[str, list[int]] = {"asha": [90]}  # nested types
total: int                                     # bare annotation: declares, creates nothing

class User:
    name: str            # class-level annotation (dataclasses use these as fields)
    def rename(self, new: str) -> None: ...     # self is not annotated
```

How it works

1. `param: Type` annotates a parameter; with a default it is `param: Type = default` (spaces around `=`, unlike the un-annotated `param=default` – PEP 8).
2. `-> Type` annotates the return. A function that returns nothing is `-> None`.
3. `name: Type = value` annotates a variable (PEP 526). A bare `name: Type` creates **no** variable – reading `total` above raises `NameError`.
4. Annotations are stored in `__annotations__`; `typing.get_type_hints(obj)` returns them fully resolved (turning string forward references into real classes).
5. **Forward references.** If a type is not defined yet (a class referring to itself), write it as a string: `def clone(self) -> "Node":`. Since 3.11 you can also use `typing.Self` for "the current class".
6. **When are annotations evaluated?** Up to 3.13, annotations are evaluated when the `def` runs (unless the module has `from __future__ import annotations`, which stores them as strings). **Python 3.14** (PEP 649/749) makes evaluation *lazy* by default: annotations are computed only when someone asks for them, so forward references usually work without quotes. The new `annotationlib` module exposes the details.

Script `01_annotations_basics.py`:

```python
print("greet annotations:", greet.__annotations__)
print("get_type_hints    :", get_type_hints(greet))
print("greet(123, 2)     :", greet(123, 2))   # int passed for str - runs fine
```

Output:

```
greet annotations: {'name': <class 'str'>, 'times': <class 'int'>, 'return': <class 'str'>}
get_type_hints    : {'name': <class 'str'>, 'times': <class 'int'>, 'return': <class 'str'>}
greet(123, 2)     : hi 123, hi 123
```

The last line proves the point: Python happily ran `greet(123, 2)`. Running mypy on the same file reports it:

```
01_annotations_basics.py:34: error: Argument 1 to "greet" has incompatible type "int"; expected "str"  [arg-type]
```

What to annotate

- **Always:** function parameters and return types of public functions. This is where types help most.
- **Rarely:** local variables. The checker **infers** `x = 5` as `int`. Annotate locals only when inference cannot work, e.g. an empty container: `seen: set[str] = set()`.

Common mistakes

- Forgetting `-> None`. Under mypy's default settings a function with *no* annotations at all is not checked inside – adding `-> None` opts it in.
- Writing `def f(x: int=5)` – works, but style is `x: int = 5`.
- Expecting `total: int` to create `total`.

TL;DR

`param: T`, `-> T`, `var: T = value`. Annotate signatures, let inference handle locals. Hints live in `__annotations__`; read them with `get_type_hints`.

13.3 Built-in generics and the abstract collection types

What it is

A **generic type** is a container type with a *parameter* saying what it holds: `list[int]` is "a list of ints". Since Python 3.9 (PEP 585) the built-in classes accept brackets directly — you no longer need `typing.List`, `typing.Dict`, etc. (they still work but are deprecated aliases).

Type	Meaning
`list[int]`	list of ints
`dict[str, float]`	keys `str`, values `float`
`set[str]`, `frozenset[str]`	set of strings
`tuple[int, str]`	**exactly** two items: an int then a str
`tuple[int, ...]`	any length, all ints
`tuple[()]`	the empty tuple
`type[User]`	the class `User` itself (or a subclass), not an instance

Accept abstract, return concrete

`collections.abc` defines *abstract* collection types you can also parameterise:

Abstract type	What it promises	Accepts
`Iterable[T]`	can be looped over	list, tuple, set, dict, generator, file…
`Sequence[T]`	indexable, has `len`, read-only	list, tuple, str, range
`Mapping[K, V]`	read-only dict-like	dict, `MappingProxyType`, custom
`MutableSequence[T]`, `MutableMapping[K, V]`	can be modified	list / dict
`Iterator[T]`, `Generator[Y, S, R]`	produced by generators	generator functions

The rule (from *Robust Python* and *Fluent Python* alike): **parameters should ask for the least they need; return values should say exactly what they give.**

```python
from collections.abc import Iterable

def total(prices: Iterable[float]) -> float:   # works with list, tuple, generator
    return sum(prices)

def tokens(text: str) -> list[str]:            # caller knows they get a real list
    return text.split()
```

Variance in one paragraph

Is `list[int]` a `list[float]`? **No.** A function taking `list[float]` could append `2.5` to it, which would corrupt your `list[int]`. Mutable containers are **invariant** — the parameter must match exactly. Read-only ones like

`Sequence` are **covariant**: a `Sequence[int]` *is* accepted where `Sequence[float]` is expected, since nothing can be written into it. (Separately, the typing rules let a plain `int` be passed where a `float` is expected — a special "numeric promotion".) This is one more reason to prefer `Sequence` / `Iterable` for parameters.

Common mistakes

- `def f(xs: list)` — a bare `list` means `list[Any]`; you lose checking of the items.
- `tuple[int]` when you meant "tuple of ints" — that is a one-element tuple. Use `tuple[int, ...]`.
- Importing `List`, `Dict` from `typing` in new code (fine, just old style).

TL;DR

Use `list[int]`, `dict[str, int]`, `tuple[int, ...]` directly. For parameters prefer `Iterable` / `Sequence` / `Mapping` from `collections.abc`; mutable containers are invariant.

13.4 `Optional`, `Union` and `X | None`

What it is

- A **union** means "one of these types": `int | str` (3.10+, PEP 604) or `Union[int, str]` (older).
- `X | None` means "an `X` or `None`". `Optional[X]` is exactly the same type — the name is misleading: it does **not** mean "optional argument", it means "may be None".

```
def find_user(user_id: int) -> str | None:
    users = {1: "asha", 2: "ben"}
    return users.get(user_id)
```

Why it matters

`None` bugs (`'NoneType' object has no attribute 'upper'`) are among the most common production errors in Python. Tony Hoare called null references his "billion-dollar mistake". Marking the possibility in the type forces every caller to handle it.

How it works — narrowing

The checker tracks **narrowing**: after a check, it knows a more specific type in that branch.

```
def f(x: int | str | None) -> None:
    if x is None:
        return
    reveal_type(x)          # int | str
    if isinstance(x, int):
        reveal_type(x)      # int
    else:
        reveal_type(x)      # str
```

`reveal_type(...)` is a checker built-in (also importable from `typing` since 3.11) that makes mypy print the inferred type. Real mypy output for this snippet:

```
note: Revealed type is "int | str"
note: Revealed type is "int"
note: Revealed type is "str"
```

Things that narrow: `is None` / `is not None`, `isinstance`, truthiness (`if x:` — careful, it also removes `""` and `0`), `==` against a literal, `match` patterns, early `return` / `raise`, and your own functions returning `TypeGuard[T]` or (3.13+) `TypeIs[T]`.

Unions also work at runtime in `isinstance`: `isinstance(5, int | str)` → `True`.

Common mistakes

- `def f(x: str = None)` – the default is `None`, so the type must be `str | None`. mypy rejects the shorthand by default ("implicit Optional" is off since mypy 0.990).
- Using `if x:` to narrow when `0` or `""` are valid values – use `if x is not None:`.
- Returning `X | None` everywhere "just in case": every caller must then check. Raise an exception instead when absence is an error.

In the interview

"`Optional[X]` is just `X | None`. The checker forces callers to narrow with `is not None` before using it, which eliminates most `NoneType has no attribute` errors."

- *Is `Optional` about default arguments?* No – a parameter can have a default and not be Optional, and vice versa.

TL;DR

`A | B` = either. `X | None` = `Optional[X]` = may be None. Narrow with `is not None`, `isinstance`, or early return before use.

13.5 `Any` (and `object`)

What it is

`Any` is the escape hatch: a value of type `Any` is compatible with **every** type in both directions, and every operation on it is allowed. Checking is switched off for it. Un-annotated parameters are implicitly `Any`.

`object` is the opposite: it is the top of the class hierarchy. **Anything can be passed in**, but you can only do things every object supports (`print`, `==`, `hash`) until you narrow it.

	Accepts anything?	Can call any method on it?	Safety
`Any`	yes	yes	none
`object`	yes	no (must narrow with `isinstance`)	full

```python
def load(raw: Any) -> None:
    raw.whatever()        # checker says nothing - could crash at runtime

def show(x: object) -> None:
    x.upper()             # error: "object" has no attribute "upper"
    if isinstance(x, str):
        x.upper()         # fine after narrowing
```

When to use it

- Truly dynamic data at a boundary (a parsed JSON blob before validation), or a third-party library without types.
- Temporarily during migration.

Common mistakes

`Any` is **contagious**: `y = raw.field` is also `Any`, and so is everything computed from it. A few `Any`s at the edges can silently disable checking deep inside. Prefer `object` when you just mean "I accept anything"; validate and convert `Any` data into real types as early as possible (§13.14). `mypy --strict` includes flags that warn about `Any` leaking from untyped code.

TL;DR

`Any` turns checking off; `object` accepts everything but keeps checking. Contain `Any` at boundaries.

13.6 `TypeVar` and generics

What it is

A **type variable** is a placeholder that stands for "some type, the same one each time it appears". It lets you write one function that works for many types *without* losing precision.

The problem it solves: `def first(items: list[Any]) -> Any` works, but `first([1, 2]) + "x"` is not caught — the result is `Any`. With a type variable, the checker knows `first([1, 2])` is an `int`.

How it works

Old style (3.5–3.11), still common:

```
from typing import TypeVar
T = TypeVar("T")

def first_old(items: Sequence[T]) -> T:
    return items[0]
```

New style (3.12+, PEP 695) — declare type parameters in square brackets; no separate `TypeVar` line:

```
def first[T](items: Sequence[T]) -> T:
    return items[0]

def biggest[N: (int, float)](a: N, b: N) -> N:   # constrained: N is int or float
    return a if a > b else b

class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None:
        self._items.append(item)
    def pop(self) -> T:
        return self._items.pop()
```

1. At each call, the checker **solves** `T` from the arguments: `first("xyz")` → `T = str` → returns `str`.
2. **Bound**: `[T: Hashable]` (old: `TypeVar("T", bound=Hashable)`) means "T can be any subtype of Hashable".
3. **Constraints**: `[N: (int, float)]` means "`N` is solved to exactly `int` or exactly `float`, never a subclass or a mix". So `biggest(2, 7)` returns `int`, and `biggest(2, 7.0)` returns `float` (the `int` is accepted as a `float`, see *numeric promotion* in §13.3). The classic example is `AnyStr = (str, bytes)`: `concat("a", b"b")` is rejected because no single choice fits both.
4. **Generic class**: `Stack[int]()` or `s: Stack[int] = Stack()` fixes `T` for that instance; `s.push("a")` is then an error.
5. Old-style generic classes inherit `Generic[T]`: `class Stack(Generic[T]): ...`.
6. Since 3.13 (PEP 696), type parameters can have **defaults**: `class Box[T = int]: ...`.

Output of `02_generics_protocols.py` (generic part):

```
first_old([3,4]) : 3
first('xyz')     : x
biggest(2, 7)    : 7
Stack pop        : 2 | len 1
type params      : (T,) (T,)
```

The 3.12 syntax also records the parameters at runtime in `__type_params__`.

Trade-offs

	Old `TypeVar`	3.12 `[T]` syntax
Works on	3.5+	3.12+ only
Scope	module-global name, easy to misuse	scoped to the function/class
Variance	declared manually (`covariant=True`)	inferred automatically

Common mistakes

- Using a type variable only once in a signature (`def f(x: T) -> None`) – it adds nothing; use `object`.
- Reusing one old-style `T` across unrelated functions and being confused – each function solves it independently, which is fine, but the 3.12 syntax makes this clearer.

Real-world examples

`dict.get`, `list.__getitem__`, `max`, `sorted`, `functools.lru_cache` and `asyncio.Queue[T]` are all generic in typeshed. SQLAlchemy 2.0's `Mapped[int]` and Django-stubs use generics so ORM columns have precise types.

In the interview

"A TypeVar ties input and output types together, so `first(list_of_ints)` returns `int` instead of `Any`. Since 3.12 I write `def first[T](xs: Sequence[T]) -> T` and `class Stack[T]`."

- *Bound vs constrained?* Bound: any subtype of X. Constrained: exactly one of the listed types.
- *Why is `list[int]` not a `list[float]`?* Invariance – the callee could insert a float.

Self-check

1. What does `first([1, 2])` return to the checker with `-> T`? → `int`.
2. How do you declare a generic class in 3.12? → `class Stack[T]: ...`.
3. When is a TypeVar pointless? → When it appears only once in the signature.

TL;DR

Type variables connect types across a signature. Prefer `def f[T](...)` / `class C[T]` on 3.12+; use `bound=` or constraints to restrict them.

13.7 `Callable` and `ParamSpec`

What it is

`Callable[[Arg1, Arg2], Return]` is the type of "something you can call with these arguments that returns this". Import it from `collections.abc` (the `typing` version is a deprecated alias).

```
from collections.abc import Callable, Iterable

def apply_all(fn: Callable[[int], int], xs: Iterable[int]) -> list[int]:
    return [fn(x) for x in xs]
```

- `Callable[..., int]` – any arguments, returns int (loose).
- `Callable[[], None]` – no arguments, returns nothing (a callback).
- For keyword arguments, overloads or default values, `Callable` cannot express the signature; define a `Protocol` with a `__call__` method instead (§13.8).

ParamSpec – typing decorators (brief)

A decorator that wraps *any* function should keep that function's exact parameters. Before `ParamSpec` (3.10, PEP 612) people wrote `Callable[..., Any]` and lost all checking of the decorated function's arguments.

```python
P = ParamSpec("P")
R = TypeVar("R")

def timed(fn: Callable[P, R]) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            print(f"  {fn.__name__} took {time.perf_counter() - start:.4f}s")
    return wrapper

@timed
def add(a: int, b: int) -> int:
    return a + b
```

1. `P` captures the whole parameter list of `fn`; `R` captures its return type.
2. `*args: P.args, **kwargs: P.kwargs` says "the wrapper takes exactly what `fn` takes".
3. The result type `Callable[P, R]` means `add("x", 1)` is still flagged after decoration.
4. 3.12 syntax: `def timed[**P, R](fn: Callable[P, R]) -> Callable[P, R]:`.
5. `Concatenate[Request, P]` expresses decorators that add or remove a leading argument.

Output:

```
apply_all       : [1, 4, 9]
add(2, 3) ->
  add took 0.0000s
  result 5
```

TL;DR

`Callable[[args], ret]` types function values. `ParamSpec` lets decorators preserve the wrapped signature.

13.8 `Protocol` — structural typing

What it is

Nominal typing means "B is a subtype of A because B *inherits* from A". **Structural typing** means "B is compatible with A because it *has the right methods*", whatever its ancestry. A `Protocol` (PEP 544, 3.8) declares such a shape. It is duck typing — "if it walks like a duck…" — made checkable.

Why it matters

- You can type code against objects you do not control (a third-party class cannot be made to inherit your ABC, but it may already have a `close()` method).
- It decouples modules: the consumer defines the tiny interface it needs; producers need not import it.

How it works

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class SupportsClose(Protocol):
    def close(self) -> None: ...

class DbConnection:              # does NOT inherit from SupportsClose
    def close(self) -> None:
        print("  db closed")

def close_all(things: Iterable[SupportsClose]) -> None:
    for t in things:
        t.close()
```

1. A class body that inherits `Protocol` lists the required methods/attributes with `...` bodies.
2. Any class whose members match (names **and** compatible signatures) satisfies it — checked statically.
3. `@runtime_checkable` additionally allows `isinstance(x, SupportsClose)`; at runtime this checks only that the **names exist**, not their signatures.
4. Protocols can be generic: `class SupportsLessThan[T](Protocol): def __lt__(self, other: T) -> bool: ...`.

Output:

```
close_all ->
  db closed
  temp file removed
isinstance(DbConnection(), SupportsClose): True
isinstance(42, SupportsClose)            : False
```

Trade-offs — Protocol vs ABC

	`Protocol`	`abc.ABC`
Relationship	structural (shape)	nominal (must inherit / register)
Checked	by the type checker	at instantiation (`TypeError` if abstract methods missing)
Shares implementation	not intended	yes, concrete helper methods
Works with third-party classes	yes	only with `register()`

Common mistakes

- Trusting `isinstance` with a runtime-checkable protocol to verify signatures — it doesn't.
- Big protocols. Keep them small (one to three methods), like the standard ones: `Iterable`, `Sized`, `Hashable`, `SupportsInt`, `SupportsAbs`.

Real-world examples

`typing.SupportsIndex`, `collections.abc.Iterable` and typeshed's `SupportsRead[str]` (used for "anything with a `.read()` method", e.g. in `json.load`) are all structural.

In the interview

"A Protocol is a statically checked duck type: any class with matching methods satisfies it, with no inheritance. I use it to define the minimal interface a function needs, for example `SupportsClose`, which also makes testing with fakes easy."

- *Protocol or ABC?* ABC when you want to share code and force subclasses to implement methods; Protocol when you only describe a shape.

TL;DR

`Protocol` = interface by shape. Keep it small; `@runtime_checkable` only checks method names.

13.9 `TypedDict`

What it is

A `TypedDict` describes a dict with a fixed set of string keys, each with its own value type – the typical shape of JSON objects and configuration.

```
from typing import NotRequired, TypedDict

class Movie(TypedDict):
    title: str
    year: int
    rating: NotRequired[float]      # key may be absent (3.11+)

m: Movie = {"title": "Arrival", "year": 2016}
```

How it works

1. The checker verifies keys and value types in literals and on access: `m["yeer"]` and `m["year"] = "2016"` are errors.
2. By default every key is required. Use `NotRequired[...]` per key, or `class Movie(TypedDict, total=False)` to make all optional (then `Required[...]` for exceptions). `ReadOnly[...]` (3.13) forbids writes.
3. **At runtime it is just a `dict`**: no validation, no class instance. `isinstance(x, Movie)` raises `TypeError`.

Output:

```
TypedDict value  : {'title': 'Arrival', 'year': 2016} | type at runtime: dict
Movie keys       : frozenset({'title', 'year'}) frozenset({'rating'})
```

TypedDict vs dataclass

	TypedDict	dataclass
Runtime object	plain `dict`	instance of your class
Access	`m["title"]`	`m.title`
Methods	none	yes
Best for	JSON in/out, `**kwargs` typing (`Unpack[Opts]`)	domain objects in your code

TL;DR

TypedDict types dict-shaped data (JSON). It's static only – a normal dict at runtime.

13.10 `Literal` , `Final` , `NewType`

`Literal` – only these exact values

```
Mode = Literal["r", "w", "a"]
def open_log(path: str, mode: Mode = "r") -> str: ...
open_log("app.log", "x")      # error: expected "Literal['r', 'w', 'a']"
```

Good for string/int "flags" when an `Enum` feels heavy. `typing.get_args(Mode)` returns `('r', 'w', 'a')` at runtime, which you can use for validation. `match` / `==` against a literal narrows it. Standard library example: `open()`'s `mode` is typed with literals in typeshed so the checker knows `open(p, "rb")` returns a binary file.

`Final` – must not be reassigned

```
MAX_RETRIES: Final = 3        # inferred as int, and constant
MAX_RETRIES = 4               # error: Cannot assign to final name "MAX_RETRIES"
```

Also `@final` on a method (no overriding) or class (no subclassing). Not enforced at runtime. Related: `@typing.override` (3.12) marks a method that must override a parent method, catching typos in method names.

`NewType` – a distinct name for the same runtime type

```
UserId = NewType("UserId", int)
OrderId = NewType("OrderId", int)

def get_user(uid: UserId) -> str: ...
get_user(UserId(7))   # OK
get_user(42)          # error: expected "UserId"
```

1. For the checker, `UserId` is a *subtype* of `int`: you can pass a `UserId` where an `int` is expected, but not the reverse.
2. At runtime `UserId(7)` just returns `7` – `type(UserId(7)) is int` is `True`. Zero cost.
3. Use it to stop mixing up IDs, units (`Meters` vs `Feet`) or sanitised vs raw strings (`SafeHtml`).

Output of `03_special_forms.py`:

```
Literal args     : ('r', 'w', 'a')
open app.log in 'a'
NewType          : user#7 | UserId(7) is int: True
```

Common mistakes

- `NewType` vs alias: `UserId = int` is an alias (interchangeable – no protection); `NewType` makes a distinct type.
- Expecting `Final` or `Literal` to raise at runtime.

Other special forms you will meet

Form	Meaning	Example
`Annotated[T, meta]` (3.9)	Type `T` plus extra metadata the checker ignores but libraries read	FastAPI/pydantic: `age: Annotated[int, Field(gt=0)]`
`Self` (3.11)	"An instance of the current class" — for fluent methods and alternative constructors	`def with_name(self, n: str) -> Self:`
`NoReturn` / `Never` (3.11)	The function never returns normally (always raises or exits)	`def fail(msg: str) -> NoReturn: raise SystemExit(msg)`
`ClassVar[T]`	A class-level attribute, not a per-instance field	`count: ClassVar[int] = 0`
`@overload`	Several signatures for one function, when the return type depends on the arguments	`open(p, "rb")` → binary file, `open(p, "r")` → text file
`TypeGuard[T]` / `TypeIs[T]` (3.10 / 3.13)	A user function that narrows its argument when it returns `True`	`def is_str_list(x: list[object]) -> TypeIs[list[str]]:`
`LiteralString` (3.11)	Only strings written in the source code, not user input	SQL query builders defending against injection
`Unpack[TD]` (3.11/3.12)	Type `**kwargs` with a `TypedDict`	`def f(**kw: Unpack[Opts]):`

TL;DR

`Literal` = fixed values; `Final` = no reassignment; `NewType` = type-safe wrapper with no runtime cost. `Annotated`, `Self`, `Never`, `@overload`, `TypeIs` cover the remaining common cases.

13.11 dataclasses and typing

What it is

`@dataclass` (Chapter 10) reads the **class-level annotations** to decide which fields exist and in what order, and generates `__init__`, `__repr__` and `__eq__`. It is the best-known place in the standard library where annotations change runtime behaviour (others: `typing.NamedTuple`, `TypedDict`, and `functools.singledispatch`'s `register`, which can dispatch on a parameter's annotation) — but it uses them only as *field declarations*; it does not validate types.

```
@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float
    tags: tuple[str, ...] = ()

@dataclass
class Cart:
    owner: str
    items: list[str] = field(default_factory=list)
```

How it works

1. Each annotated class attribute becomes a field; the generated `__init__` has typed parameters, so checkers verify `Point("a", 2)`.
2. Attributes annotated `ClassVar[int]` are **not** fields; `InitVar[str]` is an `__init__`-only parameter passed to `__post_init__`.
3. Mutable defaults need `field(default_factory=list)` — a plain `= []` raises `ValueError` at class creation.

4. `frozen=True` makes instances immutable (checkers flag `p.x = 5`; at runtime it raises `FrozenInstanceError`).
5. `Point(x="oops", y=2)` **runs** — dataclasses do not check types.

Output:

```
dataclass        : Point(x=1.0, y=2.0, tags=()) | fields: ['x', 'y', 'tags']
default_factory  : ['pen'] []
frozen           : FrozenInstanceError
```

Related typed record types

Tool	Mutable	Validates types	Notes
`@dataclass`	yes (or `frozen`)	no	the default choice
`typing.NamedTuple`	no	no	a real tuple (unpacks, indexes)
`TypedDict`	yes	no	plain dict
`attrs` (third-party)	configurable	optional validators	dataclasses' ancestor
`pydantic.BaseModel`	yes	**yes**	for untrusted input (§13.14)

TL;DR

dataclasses turn annotations into fields but never validate. Use `field(default_factory=...)`, `ClassVar`, `frozen=True` as needed.

13.12 Type aliases and the `type` statement

What it is

A **type alias** is a short name for a long type, to make signatures readable.

```
Vector = list[float]                                  # implicit alias (any version)
from typing import TypeAlias
Headers: TypeAlias = dict[str, str]                   # explicit (3.10+, now deprecated style)
type JSON = dict[str, JSON] | list[JSON] | str | int | float | bool | None   # 3.12+
```

How it works

1. The `type` statement (3.12, PEP 695) creates a `TypeAliasType` object. Its value is evaluated **lazily**, so the alias can refer to itself (recursive types like `JSON`) or to names defined later, without quotes.
2. Aliases can be generic: `type Pair[T] = tuple[T, T]`, then `Pair[int]`.
3. An alias is interchangeable with what it names — unlike `NewType`, it gives no extra safety.
4. A `type` alias is **not** a class: `isinstance(x, JSON)` fails; you cannot call it. Use `JSON.__value__` to see the underlying type.

Output:

```
type alias       : JSON -> TypeAliasType
alias value      : dict[str, 'JSON'] | list['JSON'] | str | int | float | bool | None
scale            : [3.0, 6.0]
```

(The script quotes `"JSON"` inside the alias only for display; with the `type` statement quotes are optional.)

TL;DR

Name complicated types with `type Name = ...` (3.12+) or `Name = ...` (older). Aliases are for readability, not safety.

13.13 Running mypy and pyright; common errors

What it is

mypy is the original checker (reference implementation of PEP 484, written in Python and compiled with mypyc). **pyright** (Microsoft, written in TypeScript) is faster and powers VS Code's Pylance; it also infers more aggressively in un-annotated code. Both read the same annotations and typeshed stubs; they occasionally disagree on edge cases.

How it works — using mypy

```
pip install mypy
mypy src/                       # check a package
mypy --strict src/              # turn on all optional strictness flags
```

Configure once in `pyproject.toml`:

```
[tool.mypy]
python_version = "3.12"
strict = true                   # new code: go strict
warn_unused_ignores = true

[[tool.mypy.overrides]]
module = ["legacy.*"]
ignore_errors = true            # old code: switch off per module while migrating
```

pyright equivalent: `pip install pyright` (or `npm i -g pyright`), then `pyright src/`, configured in `[tool.pyright]` with `typeCheckingMode = "strict"`.

Gradual adoption plan (what Dropbox and others describe):

1. Run the checker in CI with default settings on everything; fix or silence the first errors.
2. Annotate the most-used modules and public functions first.
3. Turn on `strict` per module as it becomes fully typed.
4. Forbid new untyped functions (`disallow_untyped_defs`).
5. Install stubs for third-party libraries: `pip install types-requests` etc. (mypy suggests the package name when missing).

Reading real errors

`bad_types.py` contains seven deliberate mistakes; `04_run_mypy.py` runs mypy through its Python API (`mypy.api.run`):

```
if __name__ == "__main__":
    area("3", 4)                # 1 wrong argument type
    name = find(1)              # returns str | None
    print(name.upper())         # 2 Optional not narrowed
    get_user(42)                # 3 plain int is not a UserId
    mode("x")                   # 4 not an allowed literal
    LIMIT = 11                  # 5 reassigning a Final
    count: int = "three"        # 6 wrong assignment
    shout(name)                 # 7 str | None passed where str needed
```

Output (real mypy 2.x):

```
bad_types.py:29: error: Argument 1 to "area" has incompatible type "str"; expected "float"  [arg-type]
bad_types.py:31: error: Item "None" of "str | None" has no attribute "upper"  [union-attr]
bad_types.py:32: error: Argument 1 to "get_user" has incompatible type "int"; expected "UserId"  [arg-type]
bad_types.py:33: error: Argument 1 to "mode" has incompatible type "Literal['x']"; expected "Literal['r',
'w']"  [arg-type]
bad_types.py:34: error: Cannot assign to final name "LIMIT"  [misc]
bad_types.py:35: error: Incompatible types in assignment (expression has type "str", variable has type "int")
[assignment]
bad_types.py:36: error: Argument 1 to "shout" has incompatible type "str | None"; expected "str"  [arg-type]
exit status: 1
```

Each line is `file:line: error: message [error-code]`. The code in brackets is what you name in a targeted ignore.

Common errors and fixes

Error (code)	Usual cause	Fix
`Incompatible types in assignment` `[assignment]`	variable reused for a different type	new variable name, or a union type
`Argument 1 has incompatible type` `[arg-type]`	wrong type passed	convert (`float(s)`) or fix the signature
`Item "None" of "X | None" has no attribute` `[union-attr]`	Optional not narrowed	`if x is not None:` / early return / raise
`Need type annotation for "d"` `[var-annotated]`	empty container `d = {}`	`d: dict[str, int] = {}`
`Missing return statement` `[return]`	a branch falls off the end	return or raise in every branch
`Function is missing a type annotation` `[no-untyped-def]`	strict mode, un-annotated def	annotate it
`Library stubs not installed for "requests"` `[import-untyped]`	no type info for a package	`pip install types-requests`
`Returning Any from function declared to return "int"` `[no-any-return]`	value came from untyped code	`cast(int, ...)` after validating, or validate

Escape hatches (use sparingly)

- `# type: ignore[arg-type]` on one line – always include the code, so unrelated errors are not hidden; `warn_unused_ignores` finds stale ones.
- `typing.cast(T, value)` – tells the checker "trust me"; does nothing at runtime.
- `if TYPE_CHECKING:` – import only for the checker (avoids import cycles).
- `.pyi` **stub files** – type information kept separate from code (how typeshed works).
- `@overload` – several precise signatures for one function whose return type depends on arguments.

Common mistakes

- Blanket `# type: ignore` without a code.
- Assuming un-annotated functions are checked – mypy skips their bodies unless `check_untyped_defs = true` (included in `--strict`).
- Using `cast` instead of real validation on external data.

In the interview

"I run mypy or pyright in CI and pre-commit, strict for new code and per-module overrides for legacy code. Most errors are unhandled `None`, empty containers needing annotations, and missing stubs. I use `reveal_type` to debug inference and always put an error code on `type: ignore`."

Self-check

1. How do you see what mypy inferred? → `reveal_type(expr)`.
2. What does `cast` do at runtime? → Nothing; it returns the value unchanged.
3. `Library stubs not installed` – fix? → install the `types-…` stub package or the library's own typed version.
4. Why include the code in `# type: ignore[...]`? → To suppress only that error and let `warn_unused_ignores` catch stale ones.

TL;DR

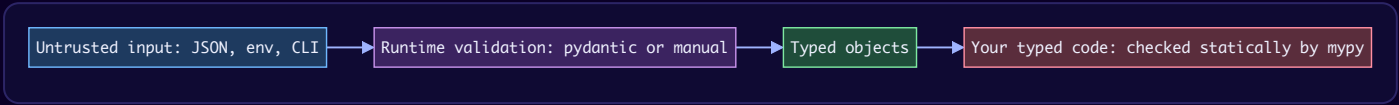
`mypy --strict` / `pyright` in CI; configure in `pyproject.toml`; adopt gradually. Read `file:line: error: msg [code]`; fix the cause; ignore only with a code.

13.14 Runtime vs static – and pydantic

What it is

- **Static** checking (mypy/pyright) reasons about your **code** before it runs. It knows nothing about the actual data that arrives at runtime.
- **Runtime validation** checks the **data** while the program runs – essential for anything from outside: HTTP requests, files, environment variables, user input, LLM output.

A program fully typed with mypy can still crash if a JSON payload has `"age": "thirty"`, because `json.loads` returns `Any`.



What Python does at runtime with hints

Feature	Runtime effect
Parameter/variable annotations	stored only; never checked
`@dataclass`	fields from annotations; **no** type check
`NewType`, `cast`, `Final`, `Literal`, `TypedDict`	no checks (NewType returns its argument)
`isinstance(x, int | str)`	works
`isinstance(x, list[int])`	`TypeError` – parameterised generics can't be checked with isinstance
`@runtime_checkable` Protocol	`isinstance` checks method names only
`get_type_hints`, `get_origin`, `get_args`	let libraries read hints and act on them

How runtime validation works – a tiny version

`05_runtime_validation.py` builds a mini validator with `get_type_hints` + `get_origin` / `get_args`, which is the idea behind pydantic:

```
def check(value: object, tp: object) -> bool:
    origin = get_origin(tp)
    if tp is NoneType or tp is None:
        return value is None
    if origin is UnionType:                       # X | Y
        return any(check(value, arg) for arg in get_args(tp))
    if origin is list:                            # list[T]
        (item_tp,) = get_args(tp)
        return isinstance(value, list) and all(check(v, item_tp) for v in value)
    if isinstance(tp, type):                      # plain class
        return isinstance(value, tp) and not (tp is int and isinstance(value, bool))
    raise TypeError(f"unsupported type {tp!r}")
```

Output:

```
Signup(email='a@x.io', age=30, referrer=None, tags=['beta'])
rejected: age='30' is not <class 'int'>
rejected: tags=['ok', 5] is not list[str] | None
```

(The `bool` special case exists because `bool` is a subclass of `int` – `isinstance(True, int)` is `True`.)

pydantic (brief, real-world)

pydantic is the most widely used runtime-validation library; version 2's core is written in Rust (`pydantic-core`). You declare a model with ordinary hints and it validates, converts and serialises data. (Needs `pip install pydantic` – not run here.)

```
from pydantic import BaseModel, ValidationError

class Signup(BaseModel):
    email: str
    age: int
    referrer: str | None = None

s = Signup(email="a@x.io", age="30")      # "lax" mode converts "30" -> 30
print(s.age + 1)                          # 31
print(s.model_dump())                     # {'email': 'a@x.io', 'age': 30, 'referrer': None}

try:
    Signup(email="b@x.io", age="thirty")
except ValidationError as e:
    print(e.error_count(), "error")       # 1 error: age is not a valid integer

s2 = Signup.model_validate_json('{"email": "c@x.io", "age": 41}')
```

Key facts:

1. By default pydantic is **lax**: it converts compatible input (`"30"` → `30`). `strict=True` (per model or field) rejects such conversions.
2. Errors are collected together in one `ValidationError` with the location of each problem.
3. `model_dump()` / `model_dump_json()` serialise; `model_json_schema()` produces JSON Schema.
4. Where it is used: **FastAPI** validates request bodies and generates OpenAPI docs from pydantic models; `pydantic-settings` loads typed config from environment variables; LLM SDKs (including Anthropic's and OpenAI's Python SDKs) use pydantic models for responses, and "structured output" tools validate model output against pydantic schemas.
5. Alternatives: `attrs` + `cattrs`, `msgspec` (very fast), `marshmallow` (schema objects separate from classes), `typeguard` / `beartype` (check function hints at call time).

When not to use it

Don't validate internal data that has already been validated and is covered by static types – it wastes time. Validate **once, at the boundary**, then trust the types inside ("parse, don't validate").

In the interview

"Type hints are static; Python never enforces them. For untrusted data I validate at the boundary with pydantic, which uses the same annotations to parse JSON into typed objects, then mypy guarantees the rest of the code uses those objects correctly."

- *Would mypy catch a bad API payload?* No — it never sees runtime data.
- *Why is pydantic fast in v2?* Its validation core is written in Rust.

Self-check

1. Does `@dataclass` check that `x: int` got an int? → No.
2. Can you `isinstance(x, list[int])`? → No, `TypeError`.
3. Where should runtime validation happen? → At system boundaries, once.
4. pydantic lax vs strict? → Lax converts compatible values (`"30"` → `30`); strict rejects them.

TL;DR

Static checking verifies code; runtime validation verifies data. Use both: pydantic (or similar) at the edges, type hints everywhere else.

13.15 Chapter self-check

1. Does `def f(x: int)` stop `f("a")` from running? → No; only a checker flags it.
2. `Optional[str]` equals? → `str | None`.
3. Parameter type for "anything I can loop over once"? → `Iterable[T]`.
4. Why is `list[int]` not a subtype of `list[float]`? → Lists are mutable, so invariant.
5. `Any` vs `object`? → `Any` disables checks; `object` accepts all but allows only universal operations.
6. 3.12 syntax for a generic function? → `def first[T](xs: Sequence[T]) -> T:`.
7. How do you keep a decorated function's signature? → `ParamSpec` (`Callable[P, R]`).
8. Class with `close()` but no inheritance — matches `SupportsClose` Protocol? → Yes, structurally.
9. What is a TypedDict at runtime? → A plain dict.
10. `UserId = int` vs `NewType("UserId", int)`? → Alias = interchangeable; NewType = distinct for the checker.
11. How do you write a recursive JSON type in 3.12? → `type JSON = dict[str, JSON] | list[JSON] | str | int | float | bool | None`.
12. Mypy says `Need type annotation for "d"`. Fix? → `d: dict[str, int] = {}`.
13. Where do you validate data from an API? → At the boundary, e.g. with pydantic.

Chapter TL;DR

- **Hints are optional and unenforced.** Checkers (mypy, pyright) read them before runtime; typing is gradual.
- **Syntax:** `param: T`, `-> T`, `var: T = v`. Annotate signatures; let inference handle locals. 3.14 evaluates annotations lazily.
- **Generics:** `list[int]`, `dict[str, int]`, `tuple[int, ...]`; parameters take `Iterable` / `Sequence` / `Mapping`; mutable containers are invariant.
- **Unions:** `A | B`; `X | None` = `Optional[X]`; narrow with `is not None` / `isinstance`. `Any` switches checking off, `object` doesn't.
- **Type variables:** `def f[T](...)`, `class C[T]` (3.12), bounds and constraints. `Callable` for function values, `ParamSpec` for decorators.
- `Protocol` = structural interface. `TypedDict` = typed dict. `Literal` = fixed values. `Final` = constant. `NewType` = zero-cost distinct type.
- **dataclasses** use annotations as fields but never validate. **Aliases** via `type X = ...`.
- **Tooling:** `mypy --strict` / pyright in CI, config in `pyproject.toml`, `reveal_type`, `# type: ignore[code]`, stubs (`types-*`).

- **Runtime vs static:** validate untrusted data once at the boundary (pydantic), trust the types inside.

Chapter 14 — A Tour of the Standard Library

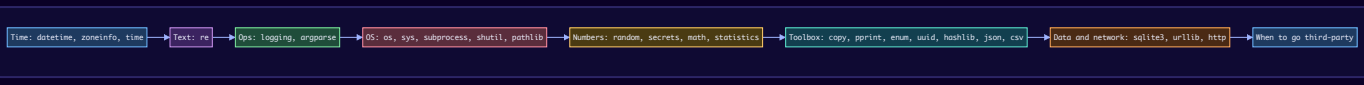
What you will learn. The modules you will reach for every week, and the traps in each: dates and time zones (`datetime` , `zoneinfo` , `time`), regular expressions in depth (`re`), professional logging (`logging`), command-line tools (`argparse`), talking to the operating system (`os` , `sys` , `subprocess` , `shutil` , `pathlib`), randomness done right (`random` vs `secrets`), numbers (`math` , `statistics` , exact `decimal` / `fractions`), a toolbox of small modules (`copy` , `pprint` , `enum` , `uuid` , `hashlib` , `json` , `csv`), an embedded database (`sqlite3`), basic HTTP (`urllib` , `http`), and when a third-party package (requests, rich, click …) is the better choice.

Prerequisites. Chapters 1–12. `collections` , `itertools` and `functools` were covered in Chapters 6, 7 and 11 — see the pointer in §14.10.

Code. Every script is in `chapters/14-standard-library-tour/code/` , uses only the standard library, needs no internet (the HTTP demo talks to a server it starts on `127.0.0.1`), and runs on Python 3.12+:

```
python chapters/14-standard-library-tour/code/01_datetime_time.py
```

"Batteries included" is Python's motto: the standard library ships with a few hundred modules. You don't need to know them all — you need to know which one to reach for and how to use it without falling into its known traps.



14.1 `datetime` and `zoneinfo` — naive vs aware

What it is

The `datetime` module has four main types:

Type	Holds	Example
`date`	year, month, day	`date(2026, 9, 28)`
`time`	hour … microsecond (no date)	`time(14, 30)`
`datetime`	date + time (+ optional time zone)	`datetime(2026, 9, 28, 14, 30)`
`timedelta`	a duration	`timedelta(days=2, hours=3)`

A datetime is **naive** if it has no time-zone info (`tzinfo is None`) — it's just "14:30 on some wall clock, somewhere". It is **aware** if it carries a `tzinfo` , so it refers to one exact instant. `zoneinfo.ZoneInfo("Asia/Kolkata")` (3.9+) gives real time-zone rules from the IANA time-zone database, including daylight saving time (**DST**). `datetime.UTC` (3.11+, alias of `timezone.utc`) is the UTC zone.

Analogy: a naive time is a note saying "meet at 9"; an aware time is "meet at 9 Tokyo time".

Why it matters

Time bugs are notoriously common: meetings an hour off after a DST change, reports that miss a day because a server runs in UTC and a user in IST, or crashes from comparing naive and aware values.

How it works

1. **Store and compute in UTC; convert to local time only for display.**
2. Create aware values: `datetime.now(UTC)` , or `datetime(..., tzinfo=ZoneInfo("Europe/London"))` . Avoid `datetime.utcnow()` — it returns a *naive* value and is deprecated since 3.12.

3. Convert with `.astimezone(ZoneInfo(...))` – same instant, different wall clock.
4. Parse and format: `datetime.fromisoformat("2026-09-28T09:15:00+05:30")` / `.isoformat()` for machine formats (ISO 8601); `strptime(text, fmt)` / `strftime(fmt)` for custom formats (`%Y-%m-%d %H:%M`, `%A` weekday name, `%I %p` 12-hour clock, `%Z` zone name).
5. Arithmetic: `datetime ± timedelta`, and `datetime − datetime → timedelta`. **Caution:** adding a `timedelta` to an aware datetime does *wall-clock* arithmetic in its zone – it does not adjust for a DST jump. For "exactly N hours later", convert to UTC, add, convert back.
6. Naive and aware values cannot be compared or subtracted (`TypeError`).
7. On Windows (no system time-zone database) install the `tzdata` package so `zoneinfo` works.

Script `01_datetime_time.py` (key parts):

```
naive = datetime(2026, 3, 8, 1, 30)
aware = datetime(2026, 3, 8, 1, 30, tzinfo=ZoneInfo("America/New_York"))
naive < aware                                  # TypeError

later = aware + timedelta(hours=1)             # wall-clock arithmetic
real = (aware.astimezone(UTC) + timedelta(hours=1)).astimezone(aware.tzinfo)

meeting_utc = datetime(2026, 9, 28, 14, 0, tzinfo=UTC)
for tz in ["Asia/Kolkata", "Europe/London", "America/Los_Angeles"]:
    print(f"  {tz:<20}", meeting_utc.astimezone(ZoneInfo(tz)).strftime("%a %H:%M %Z"))
```

Output:

```
naive tzinfo      : None
aware             : 2026-03-08T01:30:00-05:00
compare naive/aware: can't compare offset-naive and offset-aware datetimes
aware + 1h (wall) : 2026-03-08T02:30:00-05:00
aware + 1h (real) : 2026-03-08T03:30:00-04:00
  Asia/Kolkata         Mon 19:30 IST
  Europe/London        Mon 15:00 BST
  America/Los_Angeles  Mon 07:00 PDT
fromisoformat     : 2026-09-28 09:15:00+05:30 | utcoffset 5:30:00
strptime          : 2026-09-28 18:05:00 | strftime: 2026-09-28 06:05 PM
date + 30 days    : 2026-03-02 | weekday Monday
days until 2027   : 95
```

On 8 March 2026, New York clocks jump from 02:00 straight to 03:00. The "wall" result, 02:30, is a time that never existed on a New York clock; the UTC round-trip gives the correct 03:30 EDT.

Common mistakes

- Using naive datetimes in a server that talks to users in several zones.
- Hard-coding offsets (`+05:30`) instead of zone names – offsets change with DST and with politics; the zone name's rules are updated.
- Using `pytz` in new code – its `localize()` API is error-prone; `zoneinfo` replaced it.
- Expecting `date(2026, 1, 31) + "1 month"` – `timedelta` has no months (months vary in length). Use `dateutil.relativedelta` (third-party) for calendar months.

Real-world examples

Databases such as PostgreSQL store `timestamptz` as UTC; Django (`USE_TZ = True`) and most APIs exchange ISO 8601 strings in UTC; calendar apps store UTC plus the event's zone name so recurring meetings follow DST.

In the interview

"I keep all datetimes aware and in UTC internally, convert to the user's `ZoneInfo` zone only at display time, and serialise as ISO 8601. Naive datetimes are a bug source because they're ambiguous and can't be compared with aware ones."

- *Why not `utcnow()`?* It returns a naive value that looks local; use `datetime.now(UTC)`.

Self-check

1. What makes a datetime aware? → A non-None `tzinfo`.
2. How do you show a UTC time in Tokyo? → `dt.astimezone(ZoneInfo("Asia/Tokyo"))`.
3. Does `aware + timedelta(hours=1)` account for DST jumps? → No, it's wall-clock arithmetic; go via UTC.

TL;DR

Aware + UTC inside, `ZoneInfo` at the edges, ISO 8601 on the wire. `timedelta` has no months.

14.2 `time` — timestamps and timers

What it is

The `time` module is the lower-level clock interface.

Function	Returns	Use for
`time.time()`	seconds since the Unix epoch (1970-01-01 UTC), float	timestamps (can jump if the system clock is changed)
`time.perf_counter()`	highest-resolution counter, arbitrary start	measuring short durations
`time.monotonic()`	never goes backwards, arbitrary start	timeouts, rate limiters
`time.sleep(s)`	–	pause the thread
`time.time_ns()`, `perf_counter_ns()`	integer nanoseconds	avoid float precision loss

Convert a timestamp to a datetime with `datetime.fromtimestamp(ts, tz=UTC)` – e.g. `1_790_000_000` → `2026-09-21 14:13:20+00:00`.

Common mistakes

Measuring durations with `time.time()`: if NTP (the network time service) adjusts the clock mid-measurement, you get wrong or even negative durations. Use `perf_counter()` or `monotonic()`. For micro-benchmarks use the `timeit` module, which repeats the code and disables garbage collection during timing.

TL;DR

`time()` for "when", `perf_counter()` for "how long", `monotonic()` for timeouts.

14.3 `re` — regular expressions, deeper

Chapter 4 introduced `re` (`search`, `match`, `findall`, basic patterns). This section adds the parts used in real parsing work.

What it is

A **regular expression** (regex) is a small pattern language for matching text. Python's `re` module compiles a pattern into a matcher object and runs it with a backtracking engine.

How it works — the essentials

Feature	Syntax	Meaning
Raw strings	`r"\d+"`	always use them so `\` reaches `re` unchanged
Group	`( … )`	capture a part; `m.group(1)` or `m[1]`
Named group	`(?P<year>\d{4})`	`m["year"]`, `m.groupdict()`
Non-capturing	`(?: … )`	group without capturing
Alternation	`cat|dog`	either
Quantifiers	`* + ? {m,n}`	greedy by default
Lazy	`*? +? ??`	match as little as possible
Anchors	`^ $ \b \A \Z`	line start/end, word boundary, string start/end
Lookahead	`(?=…)`, `(?!…)`	require / forbid what follows, without consuming
Lookbehind	`(?<=…)`, `(?<!…)`	same, for what precedes (fixed width)
Backreference	`\1`, `(?P=name)`	the same text again

Functions: `re.search` (anywhere), `re.match` (only at the start), `re.fullmatch` (entire string — use it for validation), `re.findall` (list of strings, or tuples if there are groups), `re.finditer` (iterator of match objects — memory-friendly and gives positions), `re.sub(pattern, repl, s)` (repl may be a string with `\1` or a function of the match), `re.split`.

Flags: `re.IGNORECASE` (`re.I`), `re.MULTILINE` (`^` / `$` match at each line), `re.DOTALL` (`.` also matches newline), `re.VERBOSE` (`re.X`: whitespace and `#` comments allowed inside the pattern).

Compile once: `LINE = re.compile(...)` at module level. `re` also caches recently used patterns, so this is mainly for readability and reuse.

Script `02_regex_deeper.py` parses log lines with a verbose, named-group pattern:

```
LINE = re.compile(r"""
    ^(?P<date>\d{4}-\d{2}-\d{2})\s     # date
    (?P<time>[\d:]{8})\s+
    (?P<level>[A-Z]+)\s+
    \[(?P<svc>\w+)\]\s                 # service in brackets
    (?P<msg>.*?)\s                     # lazy: stop before the last field
    user=(?P<user>\d+)$
""", re.VERBOSE | re.MULTILINE)

for m in LINE.finditer(LOG):
    if m["level"] == "ERROR":
        print("error:", m.group("svc", "user"), "|", m["msg"])

re.sub(r"\d{12}(\d{4})", lambda m: "*" * 12 + m[1], card)   # function replacement
re.sub(r"(\w+), (\w+)", r"\2 \1", "Lovelace, Ada")          # backreferences
strong = re.compile(r"(?=.*\d)(?=.*[A-Z]).{8,}")            # two lookaheads
```

Output:

```
error: ('payments', '42') | card declined
error: ('auth', '7') | bad password
findall groups  : ['42', '7', '7']
greedy          : <a><b>
lazy            : <a>
masked          : pay ***********1111 now
swap            : Ada Lovelace
strong('password'  ): False
strong('Passw0rdX' ): True
IGNORECASE      : ['ERROR', 'ERROR']
split multi     : ['a', 'b', 'c ', 'd']
match vs search : None 1
```

Trade-offs / when not to use it

- **Structured formats have parsers**: JSON → `json`, CSV → `csv`, HTML → an HTML parser (BeautifulSoup/lxml), URLs → `urllib.parse`, emails → a validation library. Regex on these breaks on edge cases.
- Simple checks are clearer with string methods: `s.startswith("http")`, `"@" in s`, `s.isdigit()`.
- **Catastrophic backtracking**: nested quantifiers like `(a+)+$` on a long non-matching input take exponential time. This caused real outages (Stack Overflow in 2016, Cloudflare in 2019). Avoid nested/ambiguous quantifiers on untrusted input; since 3.11 `re` supports atomic groups `(?>…)` and possessive quantifiers `*+` to prevent backtracking.

Common mistakes

- Forgetting the `r` prefix (`"\b"` is a backspace character, not a word boundary).
- Using `match` when you meant `search` (or `fullmatch` for validation).
- Greedy `.*` swallowing too much — use `.*?` or a negated class like `[^\]]*`.
- `findall` with groups returns only the groups, not the whole match.

In the interview

"I compile patterns with named groups and `re.VERBOSE` so they're readable, use `finditer` for large inputs and `fullmatch` for validation, and I avoid regex for structured formats. I'm careful about nested quantifiers because of catastrophic backtracking on untrusted input."

TL;DR

Raw strings, named groups, `finditer`, `fullmatch`, `sub` with a function, `VERBOSE` for readability, lazy quantifiers when greedy grabs too much, and no regex for JSON/HTML/CSV.

14.4 `logging` — levels, handlers, formatters, best practice

What it is

`logging` is the standard way for programs and libraries to report what they are doing. Four building blocks:

Part	Job	Example
Logger	the object your code calls; named hierarchically with dots	`logging.getLogger("shop.orders")`
Handler	sends records somewhere	`StreamHandler` (console), `FileHandler`, `RotatingFileHandler`, `TimedRotatingFileHandler`, `SysLogHandler`, `QueueHandler`
Formatter	turns a record into text	`"%(asctime)s %(levelname)s %(name)s %(message)s"`
Filter	drops or edits records	add a request ID to each record

Levels (numbers in brackets): `DEBUG` (10) detail for developers · `INFO` (20) normal events · `WARNING` (30) something unexpected, still working · `ERROR` (40) an operation failed · `CRITICAL` (50) the program may not continue.

Why it matters (logging vs print)

`print` has no levels, no timestamps, no destinations, and cannot be switched off per module. With logging, the **same code** can be silent in tests, show INFO on the console in development, and send JSON to a log collector in production — only the configuration changes.

How it works

1. Each module does `log = logging.getLogger(__name__)`. Names form a tree: `shop.orders` → `shop` → root.
2. A call like `log.info("order placed qty=%d", qty)` creates a **LogRecord** if the logger's effective level allows INFO.
3. The record goes to the logger's handlers, then **propagates** up to the parent's handlers, and so on to the root.
4. Each handler has its own level and formatter, so one record can go to the console (INFO+) and a file (DEBUG+) in different formats.
5. With no configuration at all, WARNING and above are printed to stderr by a fallback handler ("last resort"); DEBUG/INFO are dropped because the root logger's default level is WARNING.
6. Configure **once, at program start** (in `main`), with `logging.basicConfig(level=..., format=...)` for simple apps or `logging.config.dictConfig({...})` for real ones. `basicConfig` does nothing if the root logger already has handlers, unless you pass `force=True`.

Script `03_logging_demo.py`:

```
log = logging.getLogger("shop.orders")      # normally getLogger(__name__)

def place_order(qty: int) -> None:
    log.debug("validating qty=%s", qty)
    if qty <= 0:
        log.warning("rejected order with qty=%d", qty)
        return
    try:
        1 / (qty - 3)
    except ZeroDivisionError:
        log.exception("pricing failed for qty=%d", qty)   # ERROR + traceback
        return
    log.info("order placed qty=%d", qty)

logging.config.dictConfig({
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {"short": {"format": "%(levelname)-7s %(name)s: %(message)s"},
                   "full":  {"format": "%(asctime)s %(levelname)s %(name)s "
                                       "%(filename)s:%(lineno)d %(message)s"}},
    "handlers": {"console": {"class": "logging.StreamHandler", "level": "INFO",
                             "formatter": "short", "stream": "ext://sys.stdout"},
                 "file": {"class": "logging.StreamHandler", "level": "DEBUG",
                          "formatter": "full", "stream": file_like}},
    "loggers": {"shop": {"level": "DEBUG", "handlers": ["console", "file"]}},
})
```

Output (console handler; path shortened):

```
INFO    shop.orders: order placed qty=2
WARNING shop.orders: rejected order with qty=0
ERROR   shop.orders: pricing failed for qty=3
Traceback (most recent call last):
  File ".../03_logging_demo.py", line 15, in place_order
    1 / (qty - 3)
    ~~^~~~~~~~~~
ZeroDivisionError: division by zero
--- file handler got 11 lines; DEBUG line present: True
logger chain: shop.orders -> shop -> root
effective level: DEBUG
```

The console showed INFO and above; the "file" handler (a `StringIO` here) received the DEBUG lines too.

Best practice

1. `getLogger(__name__)` in every module; never log on the root logger directly in libraries.
2. **Libraries never configure logging.** They only create loggers (optionally adding `logging.NullHandler()`); the application decides where logs go.
3. Pass values as arguments – `log.info("user %s", uid)` – not f-strings. Formatting is then skipped when the level is disabled, and log aggregators can group identical messages.
4. Inside `except`, use `log.exception(...)` (ERROR level + traceback) or `exc_info=True`.
5. Don't log secrets or personal data (passwords, tokens, card numbers).
6. In production prefer **structured** (JSON) logs with context fields (request ID, user ID), rotated files or stdout collected by the platform (the "12-factor" approach: log to stdout, let Docker/Kubernetes collect). `structlog` and `python-json-logger` are common helpers.
7. For high-throughput or async apps, `QueueHandler` + `QueueListener` moves slow I/O off the hot path.

Common mistakes

- Calling `basicConfig` in an imported module (it silently wins or silently does nothing).
- Adding handlers inside a function called repeatedly → each message printed many times. Configure once.
- Messages appear twice: a handler on both a child logger and the root, and `propagate` left True.
- `disable_existing_loggers` defaults to `True` in `dictConfig` – loggers created before configuration are silenced. Set it to `False`.

Real-world examples

Django and Flask configure logging through `dictConfig` (Django's `LOGGING` setting); `uvicorn`, `requests` / `urllib3` and `boto3` all emit through named loggers you can tune, e.g. `logging.getLogger("urllib3").setLevel(logging.WARNING)`.

In the interview

"Each module gets `getLogger(__name__)`. The application configures handlers and formatters once with `dictConfig`. Libraries don't configure logging. I use lazy `%s` arguments, `log.exception` in except blocks, and structured JSON logs to stdout in containers."

- *Why not print?* No levels, no routing, can't be turned off per module, no timestamps or context.
- *Why are my debug logs missing?* The logger or handler level is above DEBUG – the default root level is WARNING.

Self-check

1. Default level of the root logger? → WARNING.
2. Which call logs a traceback? → `log.exception(...)` inside `except` (or `exc_info=True`).
3. Why are messages duplicated? → Handlers on several loggers in the chain with propagation.
4. Should a library call `basicConfig`? → No.

TL;DR

Logger (per module) → handlers (where) → formatters (how). Configure once in the app, lazy args, `log.exception`, structured logs in production.

14.5 `argparse` — command-line interfaces

What it is

`argparse` turns `sys.argv` (the list of words typed after the program name) into a validated object, generates `--help`, and prints usage errors.

How it works

1. Create `ArgumentParser(prog=..., description=...)`.
2. `add_argument("title")` — a **positional** argument (required, by position).
3. `add_argument("--priority", type=int, choices=..., default=...)` — an **option**; `type` converts the text, `choices` restricts values.
4. `action="store_true"` for on/off flags, `action="count"` for `-vv`, `action="append"` for repeatable options, `nargs="+"` for one-or-more values, `required=True` to force an option.
5. `add_subparsers()` for git-style subcommands (`todo add`, `todo list`).
6. `parse_args()` returns a `Namespace`; on invalid input it prints usage to stderr and exits with status `2`. `-h/--help` is added automatically.
7. Make `main(argv=None)` accept a list so tests can call it without a real command line.

Script `04_argparse_cli.py`:

```
p = argparse.ArgumentParser(prog="todo", description="A tiny todo CLI.")
p.add_argument("-v", "--verbose", action="count", default=0)
sub = p.add_subparsers(dest="command", required=True)

add = sub.add_parser("add", help="add a task")
add.add_argument("title")
add.add_argument("--priority", type=int, choices=range(1, 4), default=2)
add.add_argument("--tag", action="append", default=[])

ls = sub.add_parser("list", help="list tasks")
ls.add_argument("--done", action="store_true")
ls.add_argument("--limit", type=int, default=10)
```

Output:

```
$ todo add Buy milk --priority 1 --tag home --tag shop
  parsed: {'verbose': 0, 'command': 'add', 'title': 'Buy milk', 'priority': 1, 'tag': ['home', 'shop']}
$ todo -vv list --done
  parsed: {'verbose': 2, 'command': 'list', 'done': True, 'limit': 10}
$ todo add x --priority 9
usage: todo add [-h] [--priority {1,2,3}] [--tag TAG] title
todo add: error: argument --priority: invalid choice: '9' (choose from '1', '2', '3')
  exit code: 2
usage: todo [-h] [-v] {add,list} ...
```

(`"Buy milk"` was one list item here; in a real shell you'd quote it: `todo add "Buy milk"`.)

Making it runnable

Add a `[project.scripts]` entry in `pyproject.toml` (`todo = "todo.cli:main"`) so `pip install` creates a `todo` command, or run as a module with `python -m todo` using a `__main__.py` (Chapter 8). Finish with `raise SystemExit(main())` so the return code reaches the shell.

Trade-offs

Library	Style	Good for
`argparse` (stdlib)	imperative, no dependency	scripts, tools that must not need installs
`click`	decorators, composable command groups	larger CLIs (Flask's `flask` command uses it)
`typer`	builds a click CLI from type hints	modern typed CLIs
`sys.argv` directly	manual	only for one or two trivial arguments

Common mistakes

- `type=bool` – `bool("False")` is `True`. Use `action="store_true"`.
- Parsing `sys.argv` by hand for anything non-trivial.
- Calling `parse_args()` at import time, which breaks tests; do it in `main`.

TL;DR

Positionals + options with `type` / `choices` / `action`; subparsers for commands; `main(argv=None)` for testability; exit code 2 on usage errors.

14.6 `os`, `sys`, `subprocess`, `shutil` (+ `pathlib` recap)

What it is

Module	Area	Most-used parts
`os`	the operating system	`os.environ`, `os.getenv`, `os.cpu_count()`, `os.getpid()`, `os.walk`, `os.sep`, `os.replace` (atomic rename)
`sys`	the running interpreter	`sys.argv`, `sys.exit()`, `sys.path`, `sys.version_info`, `sys.platform`, `sys.executable`, `sys.stdin/stdout/stderr`, `sys.modules`
`subprocess`	running other programs	`subprocess.run([...], capture_output=True, text=True, check=True, timeout=...)`
`shutil`	high-level file operations	`copy2`, `copytree`, `move`, `rmtree`, `make_archive`, `which`, `disk_usage`
`pathlib`	paths as objects (Chapter 9)	`Path`, `/`, `.read_text()`, `.glob()`, `.rglob()`, `.mkdir(parents=True)`
`tempfile`	throw-away files/dirs	`TemporaryDirectory()`, `NamedTemporaryFile()`

How it works – the key rules

1. **Configuration from the environment**: `os.environ.get("APP_ENV", "dev")` (never `os.environ["X"]` if it may be missing – that raises `KeyError`). Environment values are always strings.
2. **Paths**: use `pathlib.Path` for new code; `os.path` functions still work but are more verbose.
3. **Running commands** with `subprocess.run`:
   - Pass a **list** of arguments. Avoid `shell=True` with any user-supplied text – it enables shell injection (`"file.txt; rm -rf ~"`).
   - `capture_output=True, text=True` gives `result.stdout` as a `str`.
   - `check=True` raises `CalledProcessError` on a non-zero exit code; `timeout=` raises `TimeoutExpired`.
   - Use `sys.executable` to run "the same Python that runs me".
   - `os.system` and `os.popen` are older, less safe interfaces – prefer `subprocess`.

4. **Deleting trees** with `shutil.rmtree` is permanent — double-check the path. `os.replace(tmp, final)` gives an atomic "write then swap" for safe file updates.
5. **Exit codes:** `sys.exit(0)` success, non-zero failure (it raises `SystemExit`, so `finally` blocks still run).

Script `05_os_sys_subprocess_shutil.py` (key parts):

```
with tempfile.TemporaryDirectory() as tmp:
    root = Path(tmp)
    (root / "src" / "pkg").mkdir(parents=True)
    (root / "src" / "pkg" / "a.py").write_text("print('hi')\n")
    shutil.copytree(root / "src", root / "backup")
    archive = shutil.make_archive(str(root / "bundle"), "zip", root / "src")
    shutil.rmtree(root / "backup")
    r = subprocess.run([sys.executable, str(root / "src" / "pkg" / "a.py")],
                       capture_output=True, text=True, check=True, timeout=10)
```

Output:

```
python      : (3, 14) | platform: darwin
argv[0]     : 05_os_sys_subprocess_shutil.py
HOME set    : True | APP_ENV = dev (default)
cpu_count>0 : True
copied      : ['backup/notes.txt', 'backup/pkg', 'backup/pkg/a.py']
py files    : ['a.py', 'a.py']
archive     : bundle.zip True
after rmtree: ['bundle.zip', 'src']
child stdout: hi | returncode 0
failed child: returncode 3
which python: True
disk free GB: True
```

Common mistakes

- `subprocess.run("ls " + user_input, shell=True)` — injection risk.
- Ignoring the return code (forgetting `check=True`).
- `os.chdir` in library code — it changes the working directory for the whole process.
- String-concatenating paths with `"/"` instead of `Path` or `os.path.join`.

In the interview

"I use `subprocess.run` with a list of arguments, `check=True`, a timeout, and never `shell=True` on user input. Config comes from `os.environ.get` with defaults, paths are `pathlib.Path`, and temporary work goes in `tempfile.TemporaryDirectory`."

TL;DR

`os` = OS & env, `sys` = interpreter, `subprocess.run([...], check=True)` = other programs, `shutil` = copy/move/delete trees, `pathlib` = paths.

14.7 `random` vs `secrets`

What it is

- `random` produces **pseudo-random** numbers from the Mersenne Twister algorithm: fast, reproducible with a seed, statistically good — but **predictable** once enough outputs are observed.
- `secrets` (3.6+) draws from the operating system's cryptographically secure source (`os.urandom`): unpredictable, not reproducible.

Rule: `random` for simulations, games, sampling and tests; `secrets` for passwords, tokens, keys, password-reset links.

How it works

`random`: `random.Random(42)` (your own seeded generator – better than the global `random.seed` because other code can't disturb it), `randint(a, b)` (both ends included), `random()` in [0, 1), `uniform`, `choice`, `choices(pop, weights=, k=)` (with replacement), `sample(pop, k)` (without replacement), `shuffle` (in place), `gauss(mu, sigma)`.

`secrets`: `token_bytes(n)`, `token_hex(n)` (2n hex chars), `token_urlsafe(n)` (about 1.3 × n characters, URL-safe), `choice(seq)`, `randbelow(n)`, and `compare_digest(a, b)` – a constant-time comparison that prevents timing attacks when checking tokens.

Output of `06_random_secrets_math_stats.py` (first part):

```
randint      : [6, 1, 1, 6, 3]
choice       : red
sample       : [28, 17, 94]
shuffle      : [7, 2, 5, 6, 0, 3, 4, 9, 8, 1]
weighted     : ['A', 'A', 'A', 'B', 'B', 'A', 'A', 'A']
gauss        : 1.1491
token_urlsafe: 22 chars
token_hex(8) : 16 hex chars
compare_digest: True
```

Common mistakes

- `random` for security tokens (predictable).
- Comparing secrets with `==` (can leak timing information) – use `secrets.compare_digest`.
- For NumPy work, use `numpy.random.default_rng(seed)` instead (ML handbook).

TL;DR

`random` = reproducible simulation; `secrets` = anything an attacker must not guess.

14.8 `math`, `statistics`, `decimal`, `fractions`

What it is

`math` has fast functions on single floats/ints (C library wrappers). `statistics` has descriptive statistics on small-to-medium data lists – correct and readable, but slower than NumPy/pandas for big arrays.

How it works — highlights

Need	Function	Note
Compare floats	`math.isclose(a, b)`	default relative tolerance 1e-9; `0.1 + 0.2 == 0.3` is `False`
Rounding	`math.floor`, `math.ceil`, `math.trunc`, `round`	`round` uses **banker's rounding**: `round(2.5) == 2`, `round(3.5) == 4`
Roots, distances	`sqrt`, `isqrt` (exact int), `hypot`, `dist`	
Counting	`comb(n, k)`, `perm(n, k)`, `factorial`	5 choose 2 = 10
Number theory	`gcd`, `lcm`	`lcm` since 3.9
Logs, constants	`log(x, base)`, `log2`, `log10`, `exp`, `pi`, `e`, `inf`, `nan`	
Accurate sums	`math.fsum`	exact float sum
Averages	`statistics.mean`, `fmean`, `median`, `mode`, `multimode`	
Spread	`pstdev` / `pvariance` (population), `stdev` / `variance` (sample, divides by n−1)	
Quartiles	`statistics.quantiles(data, n=4)`	
Relations	`correlation`, `covariance`, `linear_regression`	3.10+

Output (second part):

```
isclose      : False True
floor/ceil   : -3 -2 2 4
sqrt, hypot  : 4.0 5.0
comb, perm   : 10 20 120
gcd, lcm     : 6 12
log          : 3.0 3.0 2.71828
fsum vs sum  : 1.0 1.0
mean/median  : 5 4.5
mode         : 4
pstdev/stdev : 2.0 2.1381
quantiles    : [4.0, 4.5, 6.5]
correlation  : 0.7746
linear_reg   : LinearRegression(slope=0.6, intercept=2.2)
```

Worked check for the spread: data `[2, 4, 4, 4, 5, 5, 7, 9]` has mean 5; squared deviations sum to 9+1+1+1+0+0+4+16 = 32. Population variance = 32/8 = 4 → `pstdev` = 2.0. Sample variance = 32/7 ≈ 4.571 → `stdev` ≈ 2.1381.

Note on `fsum vs sum`: since Python 3.12, the built-in `sum()` of floats uses compensated (Neumaier) summation, so `sum([0.1] * 10)` is `1.0`; on 3.11 and earlier it was `0.9999999999999999`. `math.fsum` remains the exactly-rounded option.

`decimal` and `fractions` — exact arithmetic

Floats are binary, so `0.1` cannot be stored exactly and money sums drift. Two stdlib types avoid this:

```
from decimal import Decimal, ROUND_HALF_UP
from fractions import Fraction

Decimal("0.1") + Decimal("0.2")                                  # Decimal('0.3') - exact
Decimal(0.1)                                                     # 0.1000000000000000055511... (built from the
float!)
Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)   # Decimal('2.68')
round(2.675, 2)                                                  # 2.67 - the float is really 2.67499999...
Fraction(1, 3) + Fraction(1, 6)                                  # Fraction(1, 2)
```

- `Decimal` is base-10 with a configurable precision (28 significant digits by default) and explicit rounding modes. Use it for money, and always build it from **strings**, not floats.
- `Fraction` stores an exact numerator/denominator — handy for maths puzzles and exact ratios, but slow if numbers grow large.
- Both are much slower than `float`; use them where exactness matters, not in numeric loops.

Common mistakes

- `math.floor(-2.5)` is `-3` (towards minus infinity), not `-2`; `int(-2.5)` is `-2` (towards zero).
- Using `==` on floats. Use `isclose`, or `decimal.Decimal` for money.
- Mixing up `stdev` (sample) and `pstdev` (population).

TL;DR

`math` for one-number maths, `statistics` for small data summaries; `isclose` for float comparison; `round` is banker's rounding.

14.9 A toolbox of small modules: `copy`, `pprint`, `enum`, `uuid`, `hashlib`, `json`, `csv`

`copy` — shallow vs deep

- `copy.copy(x)` — a **shallow** copy: a new outer container that shares the same inner objects.
- `copy.deepcopy(x)` — copies recursively (handles cycles with a memo dict). Slower; customise with `__copy__` / `__deepcopy__`.
- `copy.replace(obj, **changes)` (3.13+) — a modified copy of a dataclass, namedtuple, datetime, etc.

```
shallow sees change: ['a', 'b', 'c'] | deep: ['a', 'b']
```

The shallow copy shares the `hosts` list, so appending through `cfg` shows up in it.

`pprint` — readable nested data

`pprint.pp(obj, width=50)` (3.8+) prints nested structures across lines, keeping insertion order (`pprint.pprint` sorts dict keys by default). Output:

```
{'service': 'api',
 'replicas': 3,
 'env': {'DB_URL': 'postgres://db:5432/app',
         'DEBUG': False},
 'ports': [80, 443],
 'owners': ['platform-team', 'sre']}
```

Also `pprint.pformat` returns the string, handy in log messages.

`enum` – named constants

Covered in Chapter 10. `Enum` for a closed set of named values, `StrEnum` (3.11+) whose members *are* strings (great for JSON and databases), `IntEnum`, `Flag` for combinable bits, `auto()` for automatic values (for `StrEnum` it gives the lower-cased member name):

```
StrEnum: active BANNED | Enum: Color.GREEN ['RED', 'GREEN']
```

`uuid` – unique identifiers

A **UUID** is a 128-bit identifier, written as 36 characters, that can be generated anywhere without coordination.

Function	Based on	Use
`uuid4()`	random	the default for IDs
`uuid5(namespace, name)` / `uuid3`	SHA-1 / MD5 hash of a name	the same input always gives the same ID
`uuid1()`	time + MAC address	leaks the machine's MAC; avoid
`uuid7()` (3.14+)	Unix time + random	time-ordered, so database indexes stay compact

```
uuid4 version: 4 | uuid5 is deterministic: 6639460f-3425-5329-8097-a58f06127860
```

`hashlib` – cryptographic hashes

A **hash function** maps any bytes to a fixed-size digest; the same input always gives the same digest, and a good one makes it infeasible to find two inputs with the same digest.

```
hashlib.sha256(b"hello").hexdigest()      # '2cf24dba5fb0a30e...'
h = hashlib.sha256()
for chunk in (b"hel", b"lo"):              # hash large files chunk by chunk
    h.update(chunk)
key = hashlib.pbkdf2_hmac("sha256", b"s3cret", salt, 100_000)
```

Output:

```
sha256('hello'): 2cf24dba5fb0a30e ...
streamed equal : True
pbkdf2 key len : 32
```

Rules:

- Use SHA-256 (or BLAKE2, `hashlib.blake2b`) for integrity checks, cache keys, content addressing (Git identifies objects by SHA-1 and is adding SHA-256 support).
- MD5 and SHA-1 are broken for security; fine only for non-security checksums.
- **Never store passwords with a plain fast hash.** Use a slow, salted key-derivation function: `hashlib.scrypt` or `pbkdf2_hmac` with a random per-user salt (`secrets.token_bytes(16)`) and a high iteration count – or the third-party `argon2-cffi` / `bcrypt`. (The demo's fixed salt is only for reproducibility.)
- For message authentication use `hmac` (`hmac.new(key, msg, "sha256")`) and compare with `hmac.compare_digest`.
- Python's built-in `hash()` is **not** a stable hash: for strings it is randomised per process.

`json` and `csv` – recap (Chapter 9)

- `json.dumps(obj, default=str, sort_keys=True, indent=2)` – `default` is called for unsupported types (dates, sets, Decimal); tuples become lists; `json.loads` returns plain dicts/lists.
- `csv.DictWriter` / `DictReader` handle quoting (`"Li, Wei"` stays one field). Open files with `newline=""`.

```
json: {"id": 7, "tags": ["a", "b"], "when": "2026-09-28"} | back: ['a', 'b']
csv text: ['name,city', 'Asha,Pune', '"Li, Wei",Oslo']
csv read: {'name': 'Li, Wei', 'city': 'Oslo'}
```

More small modules worth knowing

Module	Does	Example
`tomllib` (3.11)	Read TOML config files (read-only)	`tomllib.load(open("pyproject.toml", "rb"))`
`zipfile`, `tarfile`, `gzip`	Read/write archives and compressed files	`gzip.open("log.gz", "rt")` streams a compressed text file
`base64`	Encode bytes as ASCII text	`base64.b64encode(b"hi")` → `b'aGk='`
`textwrap`	Wrap/indent/dedent paragraphs	`textwrap.dedent(help_text)`
`shlex`	Split a command line like a shell does	`shlex.split('git commit -m "fix bug"')` → `['git', 'commit', '-m', 'fix bug']`
`difflib`	Diffs and "did you mean" matches	`difflib.get_close_matches("pyhton", ["python", "java"])`
`getpass`	Read a password without echoing it	`getpass.getpass()`
`string`	Constants and simple templates	`string.ascii_lowercase`, `string.Template("$name")`
`timeit`, `cProfile`, `tracemalloc`	Timing, profiling, memory tracing	Chapter 17

TL;DR

`deepcopy` for independent nested copies; `pprint.pp` for readable dumps; `StrEnum` for string constants; `uuid4` / `uuid7` for IDs; SHA-256 for integrity, scrypt/pbkdf2/argon2 for passwords; `json` `default=`; csv via `DictReader/DictWriter`.

14.10 Pointer: `collections`, `itertools`, `functools`

These three are covered in depth elsewhere; here is where to look:

Module	Highlights	Chapter
`collections`	`Counter`, `defaultdict`, `deque`, `namedtuple`, `OrderedDict`, `ChainMap`	6
`itertools`	`chain`, `islice`, `groupby`, `product`, `permutations`, `combinations`, `accumulate`, `batched` (3.12), `pairwise`	11
`functools`	`lru_cache` / `cache`, `partial`, `reduce`, `wraps`, `total_ordering`, `singledispatch`, `cached_property`	7, 11, 12
`heapq`, `bisect`	priority queues, binary search in sorted lists	5, 6
`dataclasses`, `abc`, `contextlib`	records, abstract classes, context managers	10, 12

14.11 `sqlite3` — an embedded SQL database

What it is

SQLite is a complete SQL database stored in a single file (or in memory), running inside your process — no server to install. `sqlite3` is Python's built-in driver for it (following the DB-API 2.0 standard, PEP 249, shared with PostgreSQL/MySQL drivers).

How it works

1. `con = sqlite3.connect("app.db")` (or `":memory:"`).
2. `con.execute(sql, params)` runs one statement; `executemany` runs it for each row.
3. **Always use placeholders** (`?` or `:name`) for values — the driver escapes them, preventing **SQL injection**. Never build SQL with f-strings from user input.
4. `with con:` wraps a **transaction**: it commits on success and rolls back if an exception escapes (it does *not* close the connection — call `con.close()`).
5. `con.row_factory = sqlite3.Row` lets you read columns by name.
6. Results: `.fetchone()`, `.fetchall()`, or iterate the cursor.

Script `08_sqlite_demo.py`:

```
con = sqlite3.connect(":memory:")
con.row_factory = sqlite3.Row
con.execute("""CREATE TABLE expense (
                id INTEGER PRIMARY KEY,
                category TEXT NOT NULL,
                amount REAL NOT NULL CHECK (amount > 0))""")
with con:
    con.executemany("INSERT INTO expense (category, amount) VALUES (?, ?)", rows)

total = con.execute("SELECT SUM(amount) AS s FROM expense WHERE category = ?",
                    (cat,)).fetchone()["s"]

try:
    with con:                           # both inserts, or neither
        con.execute("INSERT INTO expense (category, amount) VALUES ('gift', 5)")
        con.execute("INSERT INTO expense (category, amount) VALUES ('bad', -1)")
except sqlite3.IntegrityError as e:
    print("rolled back:", e)
```

Output:

```
food total: 19.75
  travel  n=1 sum=40.0
  food    n=2 sum=19.75
  books   n=1 sum=18.0
rolled back: CHECK constraint failed: amount > 0
rows now: 4
```

The valid `gift` insert was rolled back together with the invalid one — that is atomicity.

When not to use it

Many concurrent writers (SQLite allows one writer at a time; WAL mode helps readers) or a database shared by several servers over a network — use PostgreSQL/MySQL. SQLite is excellent for apps, tests, caches, CLIs, prototypes and small-to-medium websites.

Real-world examples

SQLite is in every Android and iOS device, in browsers (Firefox and Chrome store history/bookmarks in it) and in many desktop apps. Django uses it as the default development database.

Common mistakes

- `f"... WHERE name = '{name}'"` – injection.
- Forgetting to commit (changes vanish when the connection closes).
- `(cat)` instead of `(cat,)` – parameters must be a sequence; a one-item tuple needs the comma.

TL;DR

`connect` → `execute` with `?` placeholders → `with con:` for transactions → `Row` for named columns.

14.12 `urllib` and `http` (brief)

What it is

- `urllib.parse` – split, build and encode URLs: `urlparse`, `parse_qs`, `urlencode`, `urljoin`, `quote`.
- `urllib.request` – a basic HTTP client: `urlopen(Request(url, headers=..., data=...), timeout=...)`.
- `urllib.error` – `HTTPError` (server replied with 4xx/5xx) and `URLError` (couldn't connect).
- `http.HTTPStatus` – named status codes (`HTTPStatus.NOT_FOUND == 404`).
- `http.server` – a minimal HTTP server (also `python -m http.server 8000` to share a folder). The docs warn it is **not for production**.

Script `09_urllib_http.py` starts a local server on `127.0.0.1` (port 0 = "any free port") in a background thread, then calls it:

```
u = urlparse("https://api.example.com:8443/v1/users?id=7&sort=name#top")
print("parsed :", u.scheme, u.hostname, u.port, u.path, parse_qs(u.query))

req = Request(f"{base}/greet?{urlencode({'name': 'Asha'})}",
              headers={"Accept": "application/json"})
with urlopen(req, timeout=5) as resp:
    print("GET    :", resp.status, resp.headers["Content-Type"], json.load(resp))
```

Output:

```
parsed : https api.example.com 8443 /v1/users {'id': ['7'], 'sort': ['name']}
encode : q=type+hints&page=2
urljoin: https://x.org/docs/b
GET    : 200 application/json {'message': 'hello Asha'}
error  : 404 Not Found
```

When not to use it

For real HTTP clients, `urllib.request` lacks connection pooling, convenient JSON/auth/retry handling and async. Use **requests** (synchronous, the most popular) or **httpx** (sync + async, HTTP/2). For servers, use a framework (FastAPI, Flask, Django) behind a production server (uvicorn, gunicorn).

Common mistakes

- No `timeout=` – a stuck server hangs your program forever.
- Building query strings by hand instead of `urlencode` (spaces and `&` break).

TL;DR

`urllib.parse` is great; `urllib.request` is fine for quick, dependency-free scripts; use requests/httpx for real clients.

14.13 When to reach for third-party packages

How to decide

1. **Is the stdlib version good enough?** For scripts and tools that must run anywhere without installs (CI helpers, system scripts), prefer the standard library.
2. **Is there a clear community standard?** If a package is the de-facto choice, well maintained, and widely used, the dependency is usually worth it.
3. **Cost of a dependency:** install size, security updates, licence, and supply-chain risk (typo-squatting packages exist). Pin versions and use a lock file (Chapter 8).

Need	stdlib	Common third-party choice	Why switch
HTTP client	`urllib.request`	**requests**, **httpx**	sessions/pooling, JSON, auth, retries, async (httpx)
CLI	`argparse`	**click**, **typer**	composable commands, less boilerplate, type-hint driven (typer)
Terminal output	`print`, `logging`	**rich**	colours, tables, progress bars, pretty tracebacks and logging handler
Dates	`datetime`, `zoneinfo`	`python-dateutil`, `pendulum`, `arrow`	month arithmetic, fuzzy parsing
Validation	dataclasses	**pydantic**	runtime validation (Chapter 13)
Data tables	`csv`, `statistics`	**pandas**, **polars**	vectorised analysis at scale
Numerics	`math`, `random`	**NumPy**	fast arrays
Testing	`unittest`	**pytest**	simpler tests, fixtures, plugins (Chapter 15)
Env config	`os.environ`	`python-dotenv`, `pydantic-settings`	`.env` files, typed settings
Passwords	`hashlib.scrypt`	`argon2-cffi`, `bcrypt`	modern password hashing
Database	`sqlite3`	**SQLAlchemy**, `psycopg`	ORM / PostgreSQL

Quick taste of what the three named in the outline look like (need `pip install requests rich click` – not run here):

```python
import requests
r = requests.get("https://api.github.com/repos/python/cpython", timeout=10)
r.raise_for_status()                # HTTPError on 4xx/5xx
print(r.json()["stargazers_count"])

from rich.console import Console
from rich.table import Table
t = Table("service", "status"); t.add_row("api", "[green]up[/]")
Console().print(t)                  # a coloured, boxed table

import click
@click.command()
@click.argument("name")
@click.option("--count", default=1, type=int)
def hello(name: str, count: int) -> None:
    for _ in range(count):
        click.echo(f"Hello {name}")
```

TL;DR

Default to the standard library; switch when a well-maintained community standard removes real pain (requests/httpx, click/typer, rich, pytest, pydantic, pandas). Pin what you add.

14.14 Chapter self-check

1. Naive vs aware datetime? → Aware has `tzinfo`; store aware UTC, convert with `astimezone(ZoneInfo(...))` for display.
2. Timer for measuring code? → `time.perf_counter()`.
3. Regex function for validating a whole string? → `re.fullmatch`.
4. Why `log.info("x=%s", x)` instead of an f-string? → Formatting is skipped when disabled; aggregators group by template.
5. What should a library do about logging configuration? → Nothing beyond `getLogger(__name__)` (and optionally `NullHandler`).
6. argparse error exit code? → 2.
7. Safest way to run a command with user input? → `subprocess.run([...list...], check=True, timeout=...)`, no `shell=True`.
8. `random` or `secrets` for a password-reset token? → `secrets`.
9. `round(2.5)`? → 2 (banker's rounding).
10. Shallow copy of `{"a": [1]}`, append to original's list — does the copy change? → Yes.
11. How to store passwords? → Salted slow KDF (scrypt/pbkdf2/argon2), never plain SHA-256.
12. How to pass values into SQL safely? → `?` placeholders with a parameters tuple.
13. Why prefer requests/httpx over urllib.request? → Pooling, JSON/auth/retries, async (httpx), nicer API.

Chapter TL;DR

- **Time:** aware datetimes in UTC, `ZoneInfo` for display, ISO 8601 on the wire; `perf_counter` for durations, `monotonic` for timeouts.
- **re:** raw strings, named groups, `VERBOSE`, `finditer`, `fullmatch`, `sub` with a function; beware catastrophic backtracking; don't regex JSON/HTML/CSV.
- **logging:** `getLogger(__name__)`, configure once with `dictConfig`, lazy args, `log.exception`, libraries don't configure, structured logs in production.
- **argparse:** positionals/options/subcommands, `main(argv=None)`, exit code 2 on errors.
- **OS:** `os.environ.get`, `sys.argv` / `sys.exit`, `subprocess.run([...], check=True, timeout=...)`, `shutil` for trees, `pathlib` for paths, `tempfile` for scratch space.
- **Numbers:** `random` (seeded `Random`) vs `secrets`; `math.isclose`, banker's `round`; `statistics` mean/median/stdev/quantiles/regression.
- **Toolbox:** `deepcopy`, `pprint.pp`, `StrEnum`, `uuid4` / `uuid7`, `hashlib` (SHA-256; KDFs for passwords), `json` `default=`, `csv.Dict*`.
- **sqlite3:** placeholders, `with con:` transactions, `sqlite3.Row`.
- **HTTP:** `urllib.parse` always; `urllib.request` for quick scripts; requests/httpx for real work.
- **Third-party** when a community standard clearly beats the stdlib — then pin it.

Chapter 15 — Testing and Debugging

What you will learn. Why automated tests pay for themselves, and how to write them: the built-in `unittest` style, then **pytest** — plain `assert`, fixtures, `parametrize`, `tmp_path`, marks and `conftest.py`. How to isolate code from slow or external dependencies with **test doubles** and `unittest.mock` (`Mock`, `patch`, `side_effect`, autospec). How to measure what your tests exercise with **coverage**, the red-green-refactor **TDD** loop, executable examples with **doctest**, and **property-based testing** with Hypothesis. Then debugging: reading tracebacks, `pdb` and `breakpoint()`, logging instead of `print`, and a checklist of the bugs Python programmers hit most.

Prerequisites. Functions, classes, exceptions (Chapters 7, 9, 10). Logging basics are in Chapter 14.

Code. All code is in `chapters/15-testing-and-debugging/code/`: a small module `inventory.py` plus its tests. Install the test tools once, then run everything from that folder:

```
pip install pytest pytest-cov hypothesis
cd chapters/15-testing-and-debugging/code
pytest -q                      # 31 passed, 1 skipped, 1 xfailed
python debugging_demo.py
```

15.1 Why write tests?

What it is

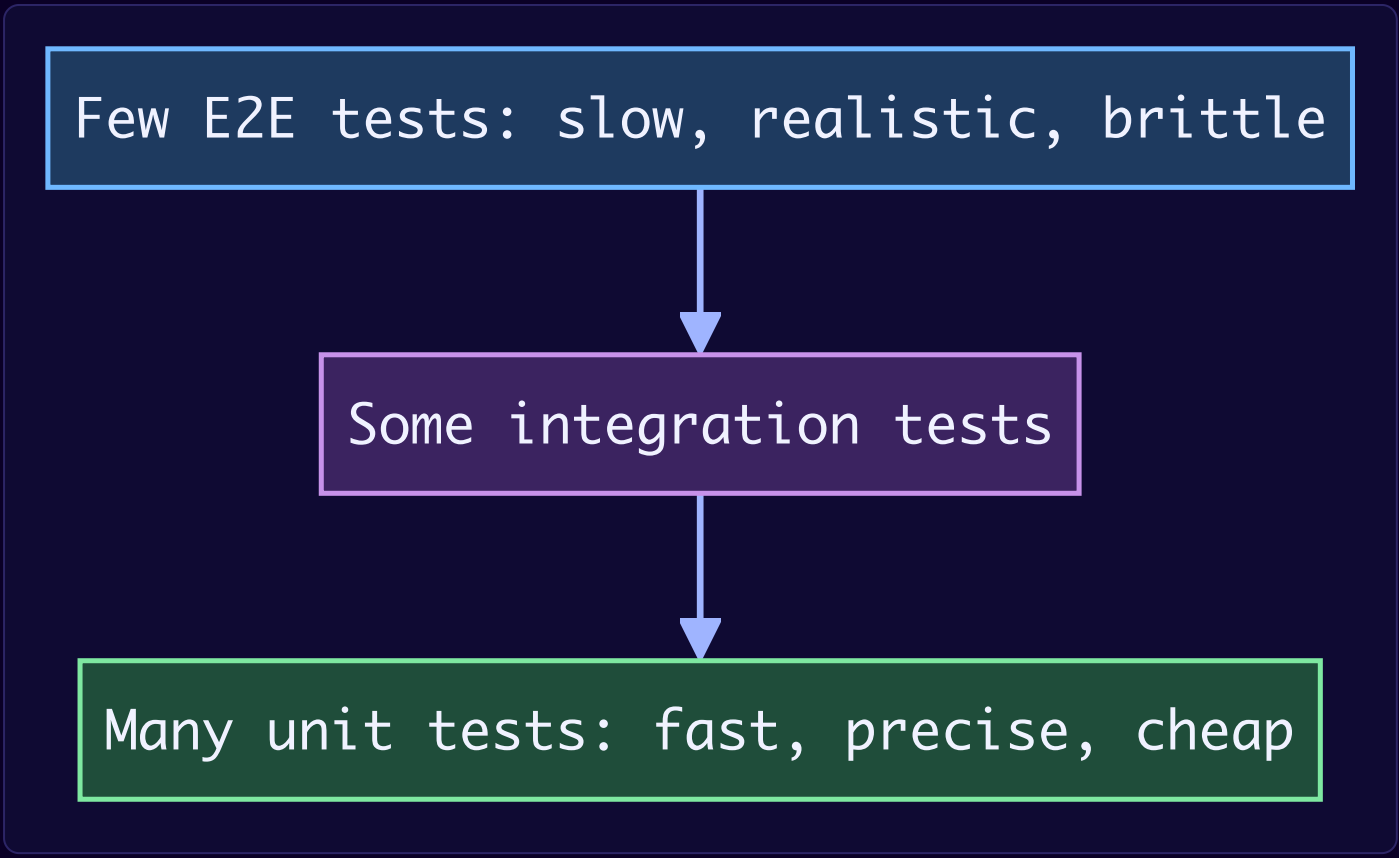
An **automated test** is code that runs your code with known inputs and checks the result. Vocabulary used in this chapter:

Term	Meaning
Unit test	tests one small piece (a function, a class) in isolation, in milliseconds
Integration test	tests several pieces together (your code + a real database or file system)
End-to-end (E2E) test	drives the whole system like a user (HTTP calls, browser)
Test suite	all the tests of a project
Regression	a bug that comes back, or a change that breaks something that used to work
SUT	*system under test* — the code being tested
Arrange–Act–Assert	the usual three-part shape of a test: set up, do the thing, check the outcome

Why it matters

1. **Confidence to change code.** Refactoring without tests is guessing. With a good suite, you change code and learn in seconds whether anything broke.
2. **Bugs are cheaper earlier.** A failing test on your laptop costs minutes; the same bug in production costs incidents, data fixes and trust.
3. **Tests are executable documentation** — they show exactly how the code is meant to be used.
4. **Better design.** Code that is hard to test is usually too coupled; writing tests pushes you toward small functions with explicit inputs.
5. In Python specifically, there is **no compiler** to catch a typo in a rarely-run branch — only running the code finds it (type checkers help, Chapter 13).

How it works — the test pyramid



Most tests should be fast unit tests; fewer integration tests; a handful of end-to-end tests for the critical paths. Tests run automatically in **CI** (continuous integration — e.g. GitHub Actions runs `pytest` on every push/pull request).

What makes a good test

- **Fast** (milliseconds), **isolated** (no dependence on other tests or their order), **repeatable** (same result every time — no real clock, network, or unseeded randomness), **self-checking** (pass/fail, no human reading output), and **focused** (one behaviour; the name says which).
- Test **behaviour, not implementation**: check what a function returns or changes, not which private helpers it calls. Then refactoring doesn't break tests.

Real-world examples

CPython's own suite (`python -m test`) is written with `unittest`; NumPy, pandas, pip and most modern projects use pytest; Django ships a `unittest`-based runner. Pull requests on these projects are merged only when CI is green.

TL;DR

Tests let you change code safely. Many fast unit tests, fewer integration and E2E tests, all run in CI. Test behaviour, keep tests isolated and deterministic.

15.2 The module under test

Every example tests the same small module, `inventory.py` (abridged — `# ...` marks omitted lines):

```python
class OutOfStock(Exception):
    """Raised when removing more units than are available."""

def slugify(name: str) -> str:
    """Lower-case, keep letters/digits, join words with '-'.

    >>> slugify("Hello, World!")
    'hello-world'
    """
    words = re.findall(r"[a-z0-9]+", name.lower())
    return "-".join(words)

def apply_discount(price: float, percent: float) -> float:
    if not 0 <= percent <= 100:
        raise ValueError(f"percent must be 0-100, got {percent}")
    return round(price * (1 - percent / 100), 2)

@dataclass
class Inventory:
    stock: dict[str, int] = field(default_factory=dict)

    def add(self, sku: str, qty: int) -> None:
        if qty <= 0:
            raise ValueError("qty must be positive")
        self.stock[sku] = self.stock.get(sku, 0) + qty

    def remove(self, sku: str, qty: int) -> None:
        have = self.stock.get(sku, 0)
        if qty > have:
            raise OutOfStock(f"{sku}: want {qty}, have {have}")
        self.stock[sku] = have - qty

    def low_stock(self, threshold: int) -> list[str]: ...   # built with TDD in §15.8
    def total_units(self) -> int: ...
    def save(self, path: Path) -> None:
        path.write_text(json.dumps(self.stock, sort_keys=True))
    @classmethod
    def load(cls, path: Path) -> "Inventory": ...

def price_in(currency: str, usd: float, client: RateClient) -> float:
    """Convert a USD price using an external rate service (mocked in tests)."""
    rate = client.get_rate(currency)
    if rate <= 0:
        raise ValueError("bad rate")
    return round(usd * rate, 2)
```

Note the design choice in `price_in`: the rate service is **passed in** (dependency injection) instead of created inside. That single decision makes it trivial to test with a fake (§15.5).

15.3 `unittest` basics

What it is

`unittest` is the standard library's test framework, modelled on Java's JUnit (the "xUnit" family). Tests are methods of a class that inherits `unittest.TestCase`.

How it works

1. Create `class TestX(unittest.TestCase)`.
2. Each method whose name starts with `test` is one test.
3. `setUp()` runs before **each** test, `tearDown()` after each; `setUpClass` / `tearDownClass` (class methods) run once per class.

4. Check results with assertion methods: `assertEqual`, `assertTrue`, `assertIn`, `assertIsNone`, `assertAlmostEqual` (floats, 7 decimal places by default), `assertRaises` (as a context manager), `assertLogs`.
5. Skip with `@unittest.skip(reason)` / `skipIf(cond, reason)`; `@unittest.expectedFailure` for known bugs.
6. Run: `python -m unittest` (auto-discovers `test*.py` files), `python -m unittest -v test_inventory_unittest`, or `unittest.main()` inside the file.

```
class TestInventory(unittest.TestCase):
    def setUp(self):                      # runs before EACH test method
        self.inv = Inventory()
        self.inv.add("mug", 5)

    def test_add_accumulates(self):
        self.inv.add("mug", 3)
        self.assertEqual(self.inv.stock["mug"], 8)

    def test_remove_too_many_raises(self):
        with self.assertRaises(OutOfStock):
            self.inv.remove("mug", 99)

    def test_discount(self):
        self.assertAlmostEqual(apply_discount(19.99, 10), 17.99)

    @unittest.skip("demo of skipping")
    def test_skipped(self):
        self.fail("never runs")
```

Output of `python test_inventory_unittest.py`:

```
test_add_accumulates (__main__.TestInventory.test_add_accumulates) ... ok
test_discount (__main__.TestInventory.test_discount) ... ok
test_remove_too_many_raises (__main__.TestInventory.test_remove_too_many_raises) ... ok
test_skipped (__main__.TestInventory.test_skipped) ... skipped 'demo of skipping'

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK (skipped=1)
```

Trade-offs — unittest vs pytest

	unittest	pytest
Install	built in	`pip install pytest`
Test shape	class + `self.assertX` methods	plain functions + `assert`
Setup	`setUp` / `tearDown`	fixtures (reusable, composable, scoped)
Data-driven tests	`self.subTest(...)` loop	`@pytest.mark.parametrize`
Failure messages	from the assert method	rewritten `assert` shows values and diffs
Plugins	few	large ecosystem (coverage, xdist, asyncio, django…)
Runs the other's tests	–	yes, pytest runs `unittest` tests unchanged

When to use it

When you can't add dependencies (stdlib-only tools, CPython itself), or in an existing unittest codebase. For everything else, pytest is the community default — and since pytest runs unittest tests, you can migrate gradually.

TL;DR

`unittest`: `TestCase` classes, `test_` methods, `setUp`, `assertEqual` / `assertRaises`; run with `python -m unittest`. Built in, more verbose than pytest.

15.4 pytest

What it is

pytest is the most widely used Python test framework. Tests are plain functions using plain `assert`; pytest discovers them, runs them, and explains failures in detail.

How it works – discovery and running

1. pytest collects files named `test_*.py` or `*_test.py`, functions named `test_*`, and classes named `Test*` (without an `__init__`).
2. Common commands:

Command	Does
`pytest`	run everything under the current folder
`pytest -q` / `-v`	quieter / one line per test
`pytest test_x.py::test_name`	one test
`pytest -k "discount and not bad"`	tests whose names match the expression
`pytest -m "not slow"`	select by mark
`pytest -x` / `--maxfail=2`	stop after the first/second failure
`pytest --lf`	rerun only last-failed tests
`pytest -s`	show `print` output (normally captured)
`pytest --pdb`	open the debugger at a failure
`pytest --durations=5`	list the slowest tests

3. Settings live in `pyproject.toml` under `[tool.pytest.ini_options]` (e.g. `testpaths = ["tests"]`, `markers = ["slow: ..."]`, `addopts = "-q"`).

15.4.1 Plain `assert` and assertion rewriting

pytest rewrites `assert` statements in test files so that on failure it prints the values involved. You write `assert result == expected`, and get a diff for free (see §15.8 for a real one). Helpers:

- `pytest.raises(Exc, match="regex")` – the block must raise `Exc` with a message matching the regex.
- `pytest.approx(x)` – float comparison with tolerance (relative 1e-6 by default): `0.1 + 0.2 == pytest.approx(0.3)`.
- `pytest.warns(Warning)` – the block must emit a warning.

15.4.2 Fixtures

A **fixture** is a function that prepares something a test needs (an object, a temp file, a database connection) and optionally cleans it up afterwards. A test requests a fixture simply by naming it as a parameter – pytest calls the fixture and passes in the result (**dependency injection**).

```python
# conftest.py
@pytest.fixture
def inv():
    """A fresh inventory with two products for every test that asks for it."""
    inventory = Inventory()
    inventory.add("mug", 5)
    inventory.add("pen", 10)
    return inventory

# test_inventory_pytest.py
def test_total_units(inv):
    assert inv.total_units() == 15

def test_remove_reduces_stock(inv):
    inv.remove("pen", 4)
    assert inv.stock == {"mug": 5, "pen": 6}

def test_remove_too_many_raises(inv):
    with pytest.raises(OutOfStock, match="want 6, have 5"):
        inv.remove("mug", 6)
```

Rules:

1. **Fresh per test by default** (`scope="function"`): `test_remove_reduces_stock` modifies its `inv`, and the next test still gets the original — tests stay isolated.
2. **Scopes**: `function` (default), `class`, `module`, `package`, `session`. Use wider scopes for expensive, read-only resources (e.g. one database engine per session).
3. **Teardown with `yield`**: code after `yield` runs after the test, even if it fails:

   ```python
   @pytest.fixture
   def audit_log(tmp_path):
       f = (tmp_path / "audit.txt").open("w")
       yield f          # the test runs here
       f.close()        # cleanup
   ```

4. Fixtures can use other fixtures (`audit_log` uses `tmp_path`).
5. `autouse=True` applies a fixture to every test in its scope without naming it — use sparingly.
6. Useful **built-in fixtures**: `tmp_path` (a unique temporary directory as a `Path`), `monkeypatch` (temporarily set attributes, env vars, dict items; undone automatically), `capsys` (capture stdout/stderr), `caplog` (capture log records), `request` (info about the requesting test).

15.4.3 `tmp_path` — real files, safely

```python
def test_save_and_load_roundtrip(inv, tmp_path):
    path = tmp_path / "stock.json"          # unique temp dir per test
    inv.save(path)
    assert Inventory.load(path) == inv
    assert path.read_text() == '{"mug": 5, "pen": 10}'
```

Each test gets its own directory, so tests never collide and never write into your project. This is often better than mocking the file system: it tests the real code path and is still fast.

15.4.4 `parametrize` — one test, many cases

```
@pytest.mark.parametrize(
    ("price", "percent", "expected"),
    [(100, 0, 100), (100, 25, 75), (19.99, 10, 17.99), (50, 100, 0)],
)
def test_discount(price, percent, expected):
    assert apply_discount(price, percent) == pytest.approx(expected)

@pytest.mark.parametrize(("raw", "slug"), [
    ("Blue Mug", "blue-mug"),
    ("  --Hello__World-- ", "hello-world"),
    ("", ""),
], ids=["simple", "punctuation", "empty"])
def test_slugify(raw, slug):
    assert slugify(raw) == slug
```

Each tuple becomes a **separate test** with its own ID, so one failing case doesn't hide the others. Include edge cases: empty input, zero, boundaries (0 and 100), negative numbers, Unicode.

15.4.5 Marks

A **mark** is a label on a test that changes how it's selected or run.

Mark	Effect
`@pytest.mark.skip(reason=...)`	never run
`@pytest.mark.skipif(sys.platform == "win32", reason=...)`	run only when the condition is false
`@pytest.mark.xfail(reason=..., strict=True)`	expected to fail; with `strict=True`, an unexpected *pass* is reported as a failure, so you notice when the bug is fixed
`@pytest.mark.parametrize(...)`	data-driven cases
`@pytest.mark.slow` (custom)	a label you select with `-m`; register it to avoid "unknown mark" warnings

```
@pytest.mark.slow
def test_slow_path():
    time.sleep(0.2)
    assert True

@pytest.mark.xfail(reason="float rounding: 2.675 is stored as 2.67499...; use Decimal",
                   strict=True)
def test_known_bug():
    assert apply_discount(2.675, 0) == 2.68
```

The xfail documents a genuine float issue: the literal `2.675` is stored in binary as slightly less than 2.675, so `round(2.675, 2)` gives `2.67`. The fix would be `decimal.Decimal`.

15.4.6 `conftest.py`

`conftest.py` is a special file that pytest loads automatically. Fixtures and hooks defined there are available to every test in that directory and below — **no import needed**. Nested folders can have their own `conftest.py`. Ours defines the `inv` fixture and registers the `slow` marker:

```
def pytest_configure(config):
    config.addinivalue_line("markers", "slow: marks a test as slow")
```

(In a real project you'd register markers in `pyproject.toml` instead.)

Output

`pytest -v test_inventory_pytest.py`:

```
test_inventory_pytest.py::test_total_units PASSED                        [  6%]
test_inventory_pytest.py::test_remove_reduces_stock PASSED               [ 12%]
test_inventory_pytest.py::test_remove_too_many_raises PASSED             [ 18%]
test_inventory_pytest.py::test_discount[100-0-100] PASSED                [ 25%]
test_inventory_pytest.py::test_discount[100-25-75] PASSED                [ 31%]
test_inventory_pytest.py::test_discount[19.99-10-17.99] PASSED           [ 37%]
test_inventory_pytest.py::test_discount[50-100-0] PASSED                 [ 43%]
test_inventory_pytest.py::test_discount_rejects_bad_percent[-1] PASSED   [ 50%]
test_inventory_pytest.py::test_discount_rejects_bad_percent[101] PASSED  [ 56%]
test_inventory_pytest.py::test_slugify[simple] PASSED                    [ 62%]
test_inventory_pytest.py::test_slugify[punctuation] PASSED               [ 68%]
test_inventory_pytest.py::test_slugify[empty] PASSED                     [ 75%]
test_inventory_pytest.py::test_save_and_load_roundtrip PASSED            [ 81%]
test_inventory_pytest.py::test_audit_written PASSED                      [ 87%]
test_inventory_pytest.py::test_slow_path PASSED                          [ 93%]
test_inventory_pytest.py::test_known_bug XFAIL (float rounding: 2.67...) [100%]

======================== 15 passed, 1 xfailed in 0.31s =========================
```

`pytest -q -m "not slow" test_inventory_pytest.py` → `14 passed, 1 deselected, 1 xfailed`. `pytest --durations=3` (whole folder):

```
0.20s call     test_inventory_pytest.py::test_slow_path
0.12s call     test_properties.py::test_stock_never_negative
0.04s call     test_properties.py::test_slugify_is_idempotent
```

Common mistakes

- Tests that depend on each other's side effects or order (use fresh fixtures).
- Asserting on floats with `==` (use `pytest.approx`).
- `pytest.raises` wrapping too much code — the exception might come from the setup line instead of the call you meant.
- Module-level mutable state shared between tests.
- Test files with the same basename in different folders without `__init__.py` — pytest reports an import-mismatch error.

In the interview

"I use pytest: plain asserts, fixtures for setup with `yield` for teardown, `parametrize` for edge cases, `tmp_path` and `monkeypatch` for files and environment, and marks to separate slow tests. Shared fixtures go in `conftest.py`."

- *Fixture scope?* Default is per function for isolation; session scope for expensive read-only resources.
- *xfail vs skip?* Skip = don't run; xfail = run, expect failure (strict flags an unexpected pass).

Self-check

1. How does a test get a fixture? → By naming it as a parameter.
2. Where does teardown code go? → After `yield` in the fixture.
3. What does `parametrize` with four tuples produce? → Four separate tests.
4. Does a test need to import fixtures from `conftest.py`? → No, they're discovered automatically.

TL;DR

pytest = functions + `assert`. Fixtures (scoped, `yield` teardown, in `conftest.py`), `parametrize` for cases, `tmp_path` / `monkeypatch` / `capsys` / `caplog` built-ins, marks for skip/xfail/selection.

15.5 Test doubles and `unittest.mock`

What it is

A **test double** is a stand-in for a real dependency (a network API, a payment gateway, the clock) during a test – like a stunt double in a film. The standard names (from Gerard Meszaros' *xUnit Test Patterns*, popularised by Martin Fowler):

Double	What it does	Example
Dummy	passed but never used	`None` for an unused logger argument
Stub	returns canned answers	`get_rate` always returns 83.5
Spy	records how it was called, checked afterwards	a list that collects sent emails
Mock	pre-programmed with expectations about calls, which the test verifies	"`get_rate` must be called once with `'INR'`"
Fake	a working, simplified implementation	an in-memory dict instead of a database

`unittest.mock.Mock` can play stub, spy and mock roles at once.

Why it matters

Real dependencies make tests **slow** (network), **flaky** (service down), **costly** (paid APIs) or **dangerous** (sending real emails, charging cards). They also make some cases hard to reach – how do you make a real service time out on demand?

How it works – `Mock`

1. `m = Mock()` – any attribute access returns another `Mock`, and calling it records the call.
2. `m.get_rate.return_value = 83.5` – **stub** the result.
3. `m.get_rate.side_effect = ...` – dynamic behaviour: an **exception** (raised when called), an **iterable** (one value per call), or a **function** (called with the same arguments).
4. Verify: `assert_called_once_with(...)`, `assert_called_with`, `assert_not_called`, `call_count`, `call_args`, `call_args_list`.
5. `MagicMock` also supports magic methods (`__len__`, `__iter__`, `__enter__` …); `AsyncMock` (3.8+) for `async def` functions.
6. **Autospec** – `create_autospec(RealClass, instance=True)` or `patch(..., autospec=True)` builds a mock that only has the real object's attributes and signatures. A plain `Mock` happily accepts misspelled methods, so a test can pass while the real code would crash.

```
class FakeRates:                              # hand-written fake
    def __init__(self, rates): self.rates = rates
    def get_rate(self, currency): return self.rates[currency]

def test_with_fake():
    assert price_in("EUR", 10, FakeRates({"EUR": 0.9})) == 9.0

def test_with_mock_stub_and_verify():
    client = Mock()
    client.get_rate.return_value = 83.5          # stub the answer
    assert price_in("INR", 2, client) == 167.0
    client.get_rate.assert_called_once_with("INR")   # verify the interaction

def test_side_effect_exception():
    client = Mock()
    client.get_rate.side_effect = TimeoutError("rate service down")
    with pytest.raises(TimeoutError):
        price_in("EUR", 10, client)

def test_side_effect_sequence():
    client = Mock()
    client.get_rate.side_effect = [0.9, -1.0]     # one value per call
    assert price_in("EUR", 10, client) == 9.0
    with pytest.raises(ValueError, match="bad rate"):
        price_in("EUR", 10, client)
    assert client.get_rate.call_args_list == [call("EUR"), call("EUR")]

def test_autospec_catches_typos():
    client = create_autospec(FakeRates, instance=True)
    with pytest.raises(AttributeError):
        client.get_rates("EUR")                   # misspelled: plain Mock would allow it
```

How it works — `patch`

When code creates or imports its dependency itself (instead of receiving it), use `patch` to swap a name **temporarily**:

```
def test_patch_where_it_is_used(tmp_path):
    # inventory.py did `import json`; patch the name inventory.json.dumps
    with patch("inventory.json.dumps", return_value="{}") as fake_dumps:
        inventory.Inventory({"a": 1}).save(tmp_path / "x.json")
    fake_dumps.assert_called_once()
    assert (tmp_path / "x.json").read_text() == "{}"
```

1. `patch("module.name")` replaces the attribute for the duration of a `with` block, a decorated test function, or between `start()` / `stop()`; the original is always restored.
2. **Patch where the name is looked up, not where it is defined.** If `shop.py` does `from requests import get`, patch `"shop.get"` — patching `"requests.get"` won't affect the copy of the name already bound in `shop`. (In the example, `inventory.json` *is* the `json` module, so this patch actually affects `json.dumps` everywhere during the block — one reason to keep patches short.)
3. `patch.object(obj, "attr")` patches an attribute of an object you have; `patch.dict(os.environ, {...})` patches dictionary entries.
4. pytest's `monkeypatch` does the same for simple cases: `monkeypatch.setenv("APP_ENV", "test")`, `monkeypatch.setattr(module, "name", value)`.

`pytest -v test_mocking.py`:

```
test_mocking.py::test_with_fake PASSED                          [ 14%]
test_mocking.py::test_with_mock_stub_and_verify PASSED          [ 28%]
test_mocking.py::test_side_effect_exception PASSED              [ 42%]
test_mocking.py::test_side_effect_sequence PASSED               [ 57%]
test_mocking.py::test_autospec_catches_typos PASSED             [ 71%]
test_mocking.py::test_patch_where_it_is_used PASSED             [ 85%]
test_mocking.py::test_monkeypatch_env PASSED                    [100%]
```

Trade-offs

Approach	Pros	Cons
Real dependency (tmp_path, SQLite in memory, local server)	tests the real thing	slower, needs setup
Fake	realistic behaviour, reusable, readable	you must write and maintain it
Mock/stub	quick, can simulate errors/timeouts	couples tests to call details; can drift from the real API
`patch`	works without changing the code	fragile if imports move; hints that dependency injection would be cleaner

When not to mock

- Don't mock what you own and can run cheaply (pure functions, dataclasses, files in `tmp_path`, in-memory SQLite).
- Don't mock the thing you are testing.
- Don't assert every internal call — you'll lock in the implementation. Mock at the **boundary** (network, clock, randomness, third-party SDKs), and prefer checking outcomes over interactions.

Common mistakes

- Patching the wrong path (where defined instead of where used).
- Plain `Mock()` hiding typos — use `autospec` / `spec`.
- Misspelled assertions such as `m.called_once_with(...)` or `m.assret_called()` — on old Pythons some silently "passed" (they just created a child mock). Mock now raises `AttributeError` for names that look like assertions but aren't (typo prefixes like `assret_` since 3.5, and assertion names missing the `assert_` prefix since 3.12), unless created with `unsafe=True`.
- Tests that only prove the mock returns what you told it to.

Real-world examples

Libraries exist for common boundaries: `responses` / `respx` fake HTTP for requests / httpx, `freezegun` / `time-machine` freeze the clock, `moto` fakes AWS services, `fakeredis` fakes Redis.

In the interview

"I prefer dependency injection so I can pass a fake. At external boundaries I use `unittest.mock` with autospec: `return_value` to stub, `side_effect` for errors or sequences, and `assert_called_once_with` only when the interaction itself matters. With `patch` I patch where the name is looked up."

- *Mock vs stub?* A stub supplies answers; a mock also verifies how it was called.
- *Why autospec?* So the mock fails on methods or signatures the real object doesn't have.

Self-check

1. `side_effect = [1, 2]` — what do three calls do? → Return 1, then 2, then raise `StopIteration`.
2. `shop.py` has `from requests import get`. What do you patch? → `"shop.get"`.
3. What's a fake? → A simplified working implementation, e.g. an in-memory repository.

TL;DR

Doubles replace slow/external dependencies: dummy, stub, spy, mock, fake. `Mock` + `return_value` / `side_effect` + `assert_called_*`; `patch` where used; autospec to stay honest; mock only at boundaries.

15.6 Coverage

What it is

Code coverage measures which lines (and branches) of your code ran during the tests. `coverage.py` is the standard tool; `pytest-cov` integrates it with pytest.

- **Line (statement) coverage** – was each line executed?
- **Branch coverage** – did each `if` go both ways? A line can run while one side of its `if` is never taken.

How it works

```
pytest --cov=inventory --cov-branch --cov-report=term-missing
# or without the plugin:
coverage run --branch -m pytest && coverage report -m && coverage html   # htmlcov/index.html
```

Output:

```
Name           Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
inventory.py      48      7     10      2    84%   48, 81-86
-----------------------------------------------------------
TOTAL             48      7     10      2    84%
```

Reading it: 48 statements, 7 never ran; 10 branch points, 2 partially taken. `Missing` lists the lines: **48** is `raise ValueError("qty must be positive")` in `Inventory.add` – no test ever passes a non-positive quantity, so that validation is untested. **81-86** is the `if __name__ == "__main__":` demo block (fine to exclude with a `# pragma: no cover` comment or config).

Trade-offs / how to use the number

- Coverage shows what is **not** tested. It does **not** show that tested code is **correct** – a test that runs a line without asserting anything still "covers" it.
- Many teams require a threshold in CI (`--cov-fail-under=85`). 100% is rarely worth the effort except in small critical libraries; aim for high coverage of business logic and error paths.
- Watch the *missing* list during code review more than the percentage.
- **Mutation testing** (e.g. `mutmut`) goes further: it changes your code (`<` to `<=`) and checks that some test fails.

TL;DR

`pytest --cov=pkg --cov-branch --cov-report=term-missing`. Use it to find untested lines, not as proof of quality.

15.7 doctest – examples that are tests

What it is

`doctest` finds text in docstrings that looks like an interactive session (`>>> expr` followed by the expected output) and checks that running it produces that output.

```
def slugify(name: str) -> str:
    """Lower-case, keep letters/digits, join words with '-'.

    >>> slugify("Hello, World!")
    'hello-world'
    """
```

How it works

- `python -m doctest inventory.py -v`, or inside the module `doctest.testmod()`, or with pytest: `pytest --doctest-modules`.
- Output is compared as **text**, so it must match `repr` exactly.

Output of `python inventory.py` and `pytest --doctest-modules inventory.py`:

```
stock: {'mug': 3} | units: 3
doctest: TestResults(failed=0, attempted=3)
..                                                                   [100%]
2 passed in 0.07s
```

(3 examples in 2 docstrings; pytest counts each docstring as one test.)

Trade-offs / when not to use it

Great for keeping **documentation examples correct**. Poor for real test suites: text comparison is brittle (dict order, float digits, memory addresses in reprs), and there are no fixtures or parametrization. Use doctests for docs, pytest for tests.

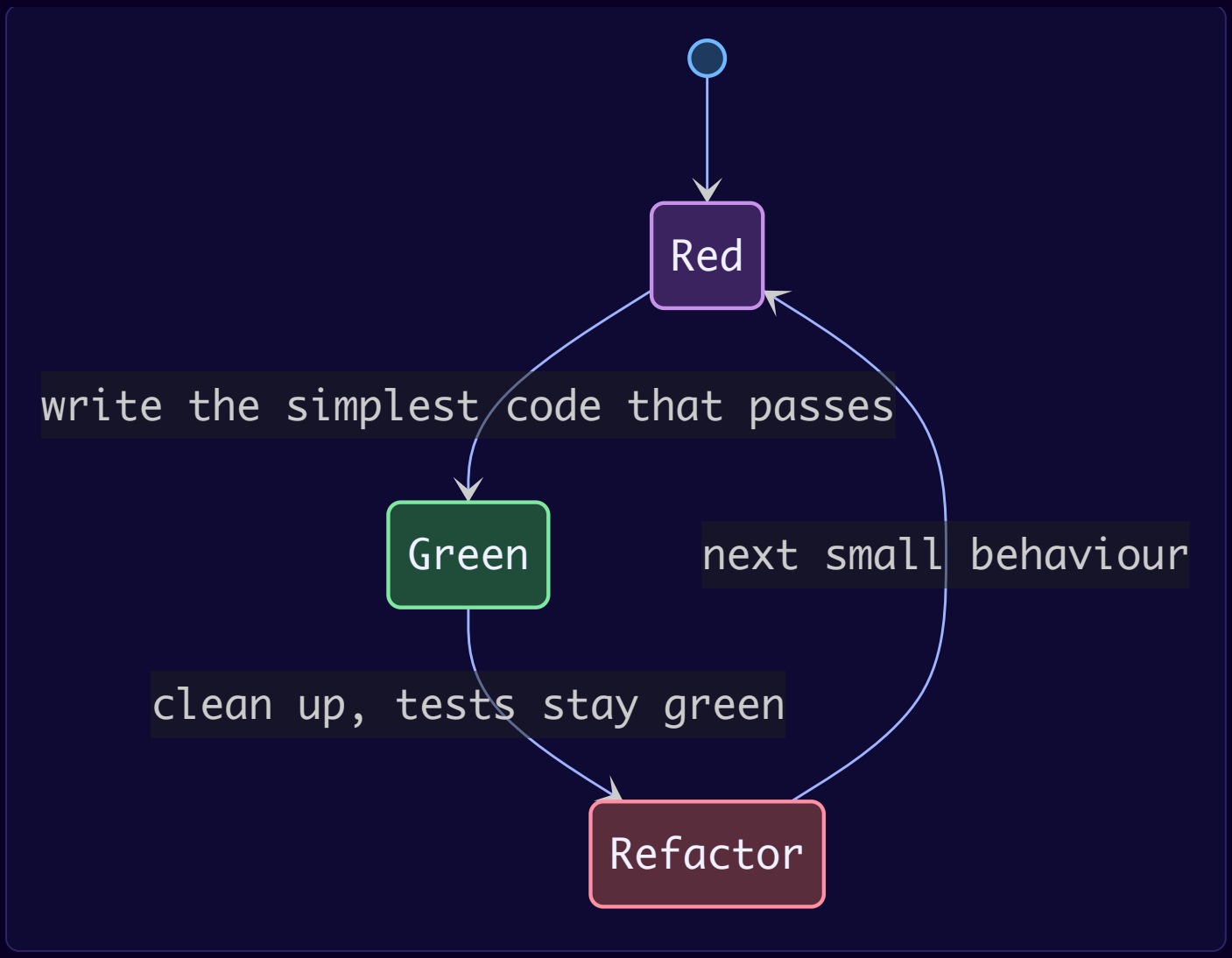
TL;DR

Docstring examples that run. Keep them short; `pytest --doctest-modules` runs them with the rest.

15.8 The TDD loop (red, green, refactor)

What it is

Test-driven development (popularised by Kent Beck) is writing the test **before** the code, in tiny cycles:



1. **Red** – write a small failing test for the next behaviour. Run it; see it fail *for the right reason*.
2. **Green** – write the minimum code to pass.
3. **Refactor** – improve names and structure; the tests guard you.

How it works – a real cycle

Feature: `Inventory.low_stock(threshold)` → SKUs whose quantity is **at or below** the threshold, sorted.

Red. Tests first (`test_tdd_low_stock.py`):

```
def test_low_stock_lists_skus_below_threshold(inv):      # inv: mug=5, pen=10
    assert inv.low_stock(threshold=6) == ["mug"]

def test_low_stock_sorted_and_inclusive(inv):
    inv.add("cap", 1)
    assert inv.low_stock(threshold=10) == ["cap", "mug", "pen"]
```

Run → both fail because the method doesn't exist yet:

```
>       assert inv.low_stock(threshold=6) == ["mug"]
               ^^^^^^^^^^^^
E       AttributeError: 'Inventory' object has no attribute 'low_stock'
...
2 failed in 0.07s
```

Green, first try. A quick implementation `return [sku for sku, q in self.stock.items() if q < threshold]` passes the first test but not the second. pytest's assertion rewriting shows exactly why:

```
>       assert inv.low_stock(threshold=10) == ["cap", "mug", "pen"]
E       AssertionError: assert ['mug', 'cap'] == ['cap', 'mug', 'pen']
E
E         At index 0 diff: 'mug' != 'cap'
E         Right contains one more item: 'pen'
E         Use -v to get more diff
1 failed, 1 passed in 0.07s
```

Two bugs are visible: the order isn't sorted, and `pen` (exactly 10) is missing, so the comparison must be `<=`.

Green.

```python
def low_stock(self, threshold: int) -> list[str]:
    """SKUs with quantity <= threshold, alphabetically."""
    return sorted(sku for sku, qty in self.stock.items() if qty <= threshold)
```

Whole suite: `31 passed, 1 skipped, 1 xfailed`.

Refactor. Names are clear already; nothing else to do — move to the next behaviour.

Trade-offs

Pros	Cons
Every line has a reason (a test)	Slower start; feels unnatural at first
Forces small, testable design	Hard when exploring an unknown problem/API (spike first, then test)
Instant regression safety	Can over-fit tests to one design if cycles are too big

Many teams use TDD for bug fixes at least: **first write a test that reproduces the bug**, see it fail, then fix — the bug can never silently return.

TL;DR

Red (failing test) → Green (simplest fix) → Refactor. At minimum, reproduce every bug with a failing test before fixing it.

15.9 Property-based testing with Hypothesis (brief)

What it is

In an **example-based** test you pick inputs. In a **property-based** test you state a rule that must hold for *all* inputs of some kind, and a library generates hundreds of inputs — including nasty ones (empty strings, huge numbers, Unicode, NaN) — trying to break it. **Hypothesis** is Python's main library for this (the idea comes from Haskell's QuickCheck).

When it finds a failing input it **shrinks** it to the smallest, simplest failing example, and stores it in a local database (`.hypothesis/`) so it's retried first next time.

How it works

```
from hypothesis import given, settings, strategies as st

@given(st.text())
def test_slugify_is_idempotent(s):            # applying it twice changes nothing
    assert slugify(slugify(s)) == slugify(s)

@given(st.text())
def test_slug_has_only_safe_chars(s):
    assert all(c.isascii() and (c.isalnum() or c == "-") for c in slugify(s))

@given(price=st.floats(0, 1e6), pct=st.floats(0, 100))
def test_discount_never_increases_price(price, pct):
    assert apply_discount(price, pct) <= round(price, 2)

@settings(max_examples=200)
@given(st.lists(st.tuples(st.sampled_from(["add", "remove"]), st.integers(1, 20))))
def test_stock_never_negative(ops):           # random sequences of operations
    inv = Inventory()
    for op, qty in ops:
        try:
            getattr(inv, op)("mug", qty)
        except OutOfStock:
            pass
        assert inv.stock.get("mug", 0) >= 0
```

All four pass on our `slugify`. To see shrinking, run the same "safe characters" property against a naive version `name.strip().lower().replace(" ", "-")`:

```
E       Failing test case: test_slug_has_only_safe_chars(
E           s=':',
E       )
```

Out of all the random strings it tried, Hypothesis reduced the failure to the single character `':'`.

Common properties to test: **round-trip** (`load(save(x)) == x`, `decode(encode(x)) == x`), **idempotence** (`f(f(x)) == f(x)`), **invariants** (stock never negative, output sorted, length preserved), **oracle** (fast version equals a slow obviously-correct version).

When not to use it

When you can't state a general rule, or when inputs are expensive to create. Use it alongside example tests, not instead of them.

TL;DR

State properties, let Hypothesis generate and shrink counter-examples. Great for parsers, serializers and invariants.

15.10 Reading tracebacks

What it is

A **traceback** is Python's report of an uncaught exception: the chain of function calls (the **call stack**) that led to the error, oldest call at the top, the failing line at the bottom.

How it works – read it bottom-up

1. **Last line first**: the exception type and message – `ValueError: invalid literal for int() with base 10: 'two'`.

2. **The frame just above it**: the file, line and function where it happened. Since 3.11, `~~~^^^` markers point at the exact sub-expression (PEP 657).
3. **Walk upward** until you reach *your* code (skip library frames) — often the bug is where your code passed bad data in.
4. **Chained exceptions**: "The above exception was the direct cause of the following exception" means the lower exception was raised with `raise NewError(...) from original`. "During handling of the above exception, another exception occurred" means a new error happened *inside* an `except` block (implicit chaining).

From `debugging_demo.py` (a CSV-like line with `"two"` as the quantity):

```
Traceback (most recent call last):
  File "debugging_demo.py", line 19, in load_orders
    orders.append(parse_order(line))
                  ~~~~~~~~~~~^^^^^^
  File "debugging_demo.py", line 12, in parse_order
    return {"sku": sku, "qty": int(qty), "price": float(price)}
                               ~~~^^^^^
ValueError: invalid literal for int() with base 10: 'two'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "debugging_demo.py", line 35, in <module>
    load_orders(["mug,2,4.50", "pen,two,1.20"])
    ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "debugging_demo.py", line 21, in load_orders
    raise RuntimeError(f"bad order on line {n}: {line!r}") from e
RuntimeError: bad order on line 2: 'pen,two,1.20'
```

The final error tells *which input* was bad (line 2); the cause above it tells *what* was wrong with it (`int('two')`). Good error handling (`raise ... from e` with context) makes tracebacks like this.

Common exception → usual cause

Exception	Usual cause
`NameError`	typo, or used before assignment
`AttributeError: 'NoneType' object has no attribute ...`	a function returned `None` (forgot `return`, or an in-place method like `list.sort()`)
`TypeError: ... takes 2 positional arguments but 3 were given`	forgot `self`, or wrong call
`KeyError` / `IndexError`	missing dict key / off-by-one
`UnboundLocalError`	assigning to a name inside a function makes it local (use `nonlocal` / `global` or rename)
`ModuleNotFoundError`	wrong virtual environment, or a local file shadows the module (e.g. your own `random.py`)
`RecursionError`	missing or unreachable base case

TL;DR

Read bottom-up: message, failing line, then up to your code. "Direct cause" = explicit `from`; "during handling" = error inside `except`.

15.11 Debugging with `pdb` and `breakpoint()`

What it is

`pdb` is Python's built-in interactive **debugger**: it pauses the program at a line and lets you inspect variables, step line by line, and move up and down the call stack. `breakpoint()` (3.7+, PEP 553) is the built-in way to pause there — it calls `pdb.set_trace()` by default.

How it works — ways in

Way	When
Put `breakpoint()` in the code	pause at a specific spot
`python -m pdb script.py`	start the whole script under the debugger
`pdb.pm()` after a crash (in the REPL) or `python -m pdb` (it enters post-mortem on an uncaught exception)	inspect the frame where it died
`pytest --pdb`	debugger at the failing assertion
`pytest --trace`	debugger at the start of each test
IDE (VS Code, PyCharm) breakpoints	same ideas, with a GUI

The environment variable `PYTHONBREAKPOINT` controls what `breakpoint()` does: `PYTHONBREAKPOINT=0` disables all of them (safety net in CI), `PYTHONBREAKPOINT=ipdb.set_trace` switches to another debugger.

The commands you need

Command	Does
`l` / `ll`	list code around the current line / the whole function
`n` (next)	run the current line, stepping *over* calls
`s` (step)	step *into* a call
`r` (return)	run until the current function returns
`c` (continue)	run until the next breakpoint
`unt N` (until)	run until line N
`p expr` / `pp expr`	print / pretty-print an expression
`w` (where)	show the call stack
`u` / `d`	move up / down the stack to inspect callers
`b file:line` / `b func` / `b line, cond`	set a (conditional) breakpoint
`cl N`	clear breakpoint N
`display expr`	show `expr` each time execution stops if it changed
`interact`	a full Python REPL in the current scope
`q`	quit

Tip: a bare expression also works, but if a variable is named like a command (`n`, `c`, `l`…), use `p n`.

A real session (scripted, so it can be shown)

`debugging_demo.py` drives `pdb` with a list of commands instead of the keyboard. The function under investigation crashes on an empty list:

```
def average_value(orders: list[dict]) -> float:
    total = 0.0
    for o in orders:
        total += o["qty"] * o["price"]
    return total / len(orders)          # bug when orders is empty
```

Commands: `b average_value`, `c`, `p orders`, `n`, `n`, `p total, len(orders)`, `c`. Transcript:

```
(Pdb) Breakpoint 1 at debugging_demo.py:26
(Pdb) > debugging_demo.py(26)average_value()
-> total = 0.0
(Pdb) []
(Pdb) > debugging_demo.py(27)average_value()
-> for o in orders:
(Pdb) > debugging_demo.py(29)average_value()
-> return total / len(orders)          # bug when orders is empty
(Pdb) (0.0, 0)
(Pdb)
then it raised: ZeroDivisionError('division by zero')
post-mortem locals: {'orders': [], 'total': 0.0}
breakpoint() skipped because PYTHONBREAKPOINT=0
```

Stepping shows the loop body is skipped and `len(orders)` is 0 at the return — the fix is to handle the empty case (`if not orders: return 0.0` or raise a clear error). The "post-mortem" line shows the same information recovered from the traceback's innermost frame after the crash, which is what `pdb.pm()` gives you interactively.

A debugging method (not just a tool)

1. **Reproduce** reliably — ideally as a failing test.
2. **Read** the traceback and the relevant code; state a hypothesis.
3. **Observe**: `breakpoint()`, logging, or `p` in pdb to check the hypothesis.
4. **Narrow down**: halve the input, comment out half the code, or use `git bisect` to find the commit that introduced the bug.
5. **Fix** the cause, not the symptom; keep the test so it can't regress.
6. Explaining the code line by line to someone (or a rubber duck) surprisingly often reveals the bug.

Common mistakes

- Committing a `breakpoint()` (CI hangs waiting for input — set `PYTHONBREAKPOINT=0` there, and use a linter rule such as Ruff's `T100`).
- Changing several things at once while debugging.

TL;DR

`breakpoint()` → `n` / `s` / `c` / `p` / `w` / `u` / `d`. `python -m pdb`, `pytest --pdb`, `pdb.pm()` for post-mortem. Reproduce, hypothesise, observe, narrow, fix, keep the test.

15.12 Logging vs `print` for debugging

What it is

`print` debugging is fine for a five-minute investigation. For anything that stays in the code, use **logging** (Chapter 14): it has levels, timestamps, module names and configurable destinations, and can be turned up to

DEBUG only where you need it.

	`print`	`logging`
Levels / filtering	no	DEBUG…CRITICAL, per module
Timestamp, module, line	manual	via formatter
Destination	stdout	console, files, syslog, log services
Safe to leave in code	no	yes (DEBUG off in production)
In tests	captured by pytest (`-s` shows)	`caplog` fixture lets tests assert on logs

```
log = logging.getLogger(__name__)
log.debug("pricing order=%s rate=%s", order_id, rate)    # visible only at DEBUG
```

Tips: temporarily raise one module's level (`logging.getLogger("shop.pricing").setLevel(logging.DEBUG)`); f-string `=` specifier for quick prints — `print(f"{total=}")` prints `total=0.0`; `python -X dev` enables extra runtime checks and warnings; `faulthandler` (`python -X faulthandler`) dumps tracebacks on hard crashes or hangs.

TL;DR

`print` for a quick look, then delete it; logging for anything that stays. Test logs with `caplog`.

15.13 Common bugs checklist

Run through this list when something "can't possibly be wrong":

#	Bug	Example	Fix
1	Mutable default argument	`def f(items=[])` shares one list across calls	`items=None` then `items = [] if items is None else items`
2	Late-binding closures	`[lambda: i for i in range(3)]` all return 2	`lambda i=i: i`
3	Aliasing / shallow copies	`grid = [[0]*3]*3` – one row shared three times	`[[0]*3 for _ in range(3)]`, `copy.deepcopy`
4	Modifying a list while iterating	removing items inside `for x in xs` skips items	iterate over a copy or build a new list
5	`is` vs `==`	`x is 1000` depends on object identity	`==` for values; `is` only for `None`, sentinels
6	Float equality	`0.1 + 0.2 == 0.3` is False	`math.isclose`, `pytest.approx`, `Decimal` for money
7	Integer vs float division	`7 / 2 == 3.5`, `7 // 2 == 3`, `-7 // 2 == -4`	pick the right operator
8	Methods returning `None`	`xs = xs.sort()` makes `xs` None	`xs.sort()` or `xs = sorted(xs)`
9	Swallowed exceptions	`except Exception: pass` hides the real error	catch specific exceptions, log, re-raise
10	Off-by-one	`range(1, n)` excludes `n`	test the boundaries
11	Truthiness	`if count:` is False for `0`	`if count is not None:`
12	Naive datetimes / time zones	comparing local and UTC times	aware UTC datetimes (Chapter 14)
13	Unclosed resources	files/connections left open	`with` statements
14	Wrong environment	`ModuleNotFoundError` though "it's installed"	check `sys.executable`, activate the venv, `python -m pip`
15	Module name shadowing	your `random.py` or `test.py` hides the stdlib	rename the file
16	Global/shared state between tests	tests pass alone, fail together	fresh fixtures, no module-level mutable state
17	Encoding	`UnicodeDecodeError` reading files	`open(p, encoding="utf-8")`
18	Unseeded randomness / real clock in tests	flaky tests	seed, inject the clock, freeze time

TL;DR

Most Python bugs are a handful of classics: mutable defaults, aliasing, `None` returns, float equality, swallowed exceptions, wrong venv. Check the list before a long debugging session.

15.14 Chapter self-check

1. Name the three parts of a well-shaped test. → Arrange, Act, Assert.
2. Why do most tests belong at the bottom of the pyramid? → Unit tests are fast, precise and cheap.
3. How does pytest show the values in a failing `assert`? → Assertion rewriting.
4. How do you add cleanup to a fixture? → Code after `yield`.
5. What does `tmp_path` give you? → A unique temporary directory (`Path`) per test.
6. `xfail(strict=True)` and the test passes – result? → Reported as a failure (XPASS strict), so you notice the fix.

7. Stub vs mock vs fake? → Canned answers / verified interactions / working simplified implementation.
8. Why `create_autospec`? → Mocks reject attributes and signatures the real object lacks.
9. What does 100% line coverage prove? → Only that every line ran — not that results were checked.
10. TDD cycle? → Red, green, refactor.
11. What does Hypothesis do after finding a failing input? → Shrinks it to a minimal example and remembers it.
12. How do you read a traceback? → Bottom-up: message, failing line, then up to your code.
13. How do you disable every `breakpoint()` in CI? → `PYTHONBREAKPOINT=0`.
14. `xs = xs.sort()` — what is `xs`? → `None`.

Chapter TL;DR

- **Why:** tests make change safe. Many fast unit tests, fewer integration/E2E; deterministic, isolated, behaviour-focused; run in CI.
- **unittest:** `TestCase`, `setUp`, `assertEqual` / `assertRaises`; built in.
- **pytest:** plain `assert` (with rewritten failure diffs), fixtures (`yield` teardown, scopes, `conftest.py`), `parametrize`, `tmp_path`, `monkeypatch`, `capsys`, `caplog`, marks (`skip`, `skipif`, `xfail`, custom + `-m`), `-k`, `-x`, `--lf`, `--pdb`.
- **Doubles:** dummy, stub, spy, mock, fake. `Mock` + `return_value` / `side_effect` / `assert_called_once_with`; `patch` where the name is used; autospec; mock only at boundaries; prefer dependency injection.
- **Coverage:** `pytest --cov --cov-branch --cov-report=term-missing` finds untested code; it doesn't prove correctness.
- **TDD:** red, green, refactor; always reproduce a bug with a failing test first.
- **doctest** keeps doc examples true; **Hypothesis** checks properties and shrinks counter-examples.
- **Debugging:** read tracebacks bottom-up; `breakpoint()` / `pdb` (`n s c p w u d b`), `pytest --pdb`, post-mortem; reproduce → hypothesise → observe → narrow → fix → keep the test.
- **Logging over print** for anything that stays; and check the **common bugs list** first.

Chapter 16 — Concurrency and Async

What you will learn. The difference between concurrency and parallelism, how to tell I/O-bound work from CPU-bound work, and Python's four main tools for doing several things at once: **threads** (`threading`, locks, `queue.Queue`), **processes** (`multiprocessing`), the high-level **executors** (`concurrent.futures`) and **async/await** (`asyncio`: event loop, coroutines, tasks, `gather`, `TaskGroup`, timeouts, async iteration). You will also learn what the **GIL** is, why it exists, and where the **free-threaded** (no-GIL) CPython stands in 3.13 and 3.14. The chapter ends with a "which tool?" table and the common pitfalls: blocking calls inside async code, race conditions and deadlocks.

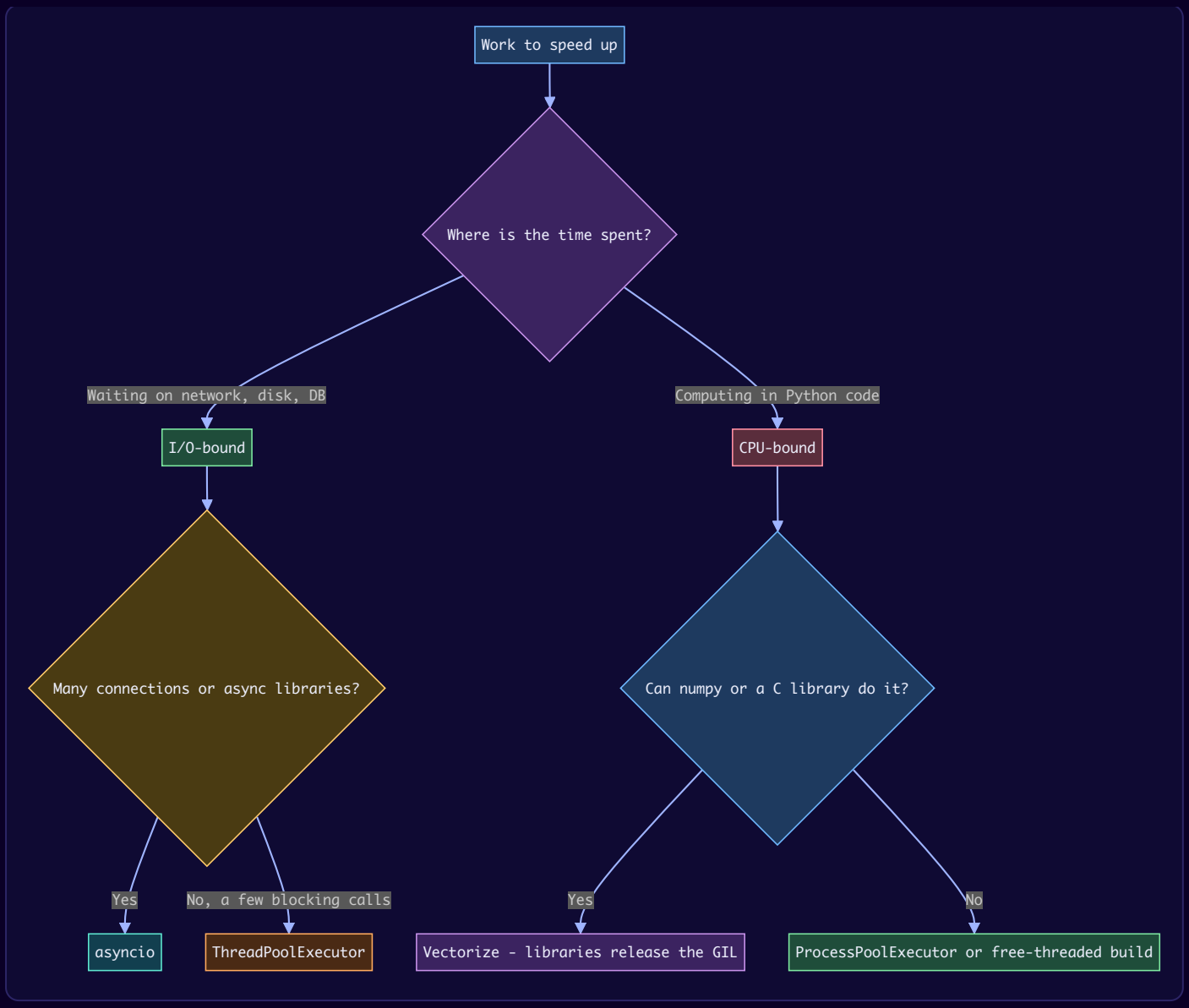
Prerequisites. Functions and closures (Ch 7), generators (Ch 11), context managers (Ch 12).

Code. `chapters/16-concurrency-and-async/code/` — `threads_basics.py`, `gil_benchmark.py` (makes the plot in `img/`), `multiprocessing_demo.py`, `futures_demo.py`, `asyncio_demo.py`, `pitfalls_demo.py`. All outputs below come from Python 3.14.7 (default build, GIL on) on a 14-core Mac. Timings will differ on your machine; the *shape* of the results will not.

Java side. The general theory — memory models, `synchronized`, `ReentrantLock`, semaphores, latches, atomics, concurrent collections, deadlock/livelock/starvation and the classic LLD problems — is in the LLD Handbook, Chapter 12 — Concurrency concepts. This chapter covers what is specific to Python.

Words used everywhere in this chapter (defined once):

Term	Meaning
Process	A running program with its own memory space. Two processes do not see each other's variables.
Thread	A path of execution *inside* a process. Threads of one process share its memory.
Task (asyncio)	A coroutine scheduled on an event loop. Many tasks share one thread.
Blocking call	A call that makes the caller wait (reading a socket, `time.sleep`, a disk read).
Race condition	A bug where the result depends on the unlucky timing of two threads touching shared data.
Critical section	Code that must be run by only one thread at a time.
Lock / mutex	An object that lets only one holder into a critical section.
Deadlock	Two or more threads each waiting forever for a lock the other holds.
GIL	Global Interpreter Lock: in standard CPython, only one thread runs Python bytecode at a time.



16.1 Concurrency vs parallelism

What it is

- **Concurrency** = *dealing with* many things at once: several tasks are in progress and their steps interleave. It can happen on a single CPU core.
- **Parallelism** = *doing* many things at the same instant: tasks literally run simultaneously on different cores.

Analogy: one chef cooking three dishes – putting pasta on to boil, then chopping while it boils – is concurrency. Three chefs each cooking one dish is parallelism. Concurrency is about *structure* (not sitting idle while waiting); parallelism is about *execution* (more hands).

Why it matters

Picking the wrong model wastes effort. Adding threads to a CPU-heavy pure-Python loop in standard CPython does nothing (the GIL allows one thread at a time), while adding threads to a program that spends 95 % of its time waiting on HTTP responses can make it 10–50× faster.

How it works

	Concurrency	Parallelism
Needs multiple cores?	No	Yes
Goal	Stay busy while waiting	Finish computation sooner
Python tools	threads, asyncio	multiprocessing, free-threaded threads, C extensions that release the GIL
Typical workload	Web servers, scrapers, chat apps	Image processing, simulations, number crunching

TL;DR

Concurrency overlaps waiting; parallelism overlaps computing. Parallelism is a kind of concurrency, but not the other way round.

16.2 I/O-bound vs CPU-bound

What it is

- **I/O-bound**: the program mostly *waits* – for the network, disk, a database, a user. The CPU is idle during the wait. Example: downloading 100 web pages.
- **CPU-bound**: the program mostly *computes*. The CPU is at 100 %. Example: computing SHA-256 of a billion strings in a Python loop, resizing images in pure Python.

How to tell (3 quick checks)

1. **Watch CPU usage** (`top`, Activity Monitor). ~100 % of one core → CPU-bound. Low CPU but slow → I/O-bound.
2. **Profile** (Ch 17): if the top functions are `socket.recv`, `select`, `read` – I/O. If they are your own loops – CPU.
3. **Compare wall time and CPU time**: `time.perf_counter()` (wall clock) vs `time.process_time()` (CPU time used). If wall ≫ CPU, you are waiting.

Measured

`gil_benchmark.py` runs 4 CPU tasks (a pure-Python loop of 15 million iterations) and 4 I/O tasks (`time.sleep(0.5)`, standing in for a network wait) three ways:

```python
def cpu_task(n: int = 15_000_000) -> int:
    total = 0
    for i in range(n):                    # pure-Python loop: holds the GIL the whole time
        total += i * i
    return total

def io_task(_: int = 0) -> None:
    time.sleep(0.5)                       # sleeping releases the GIL, like waiting on a socket
```

Output:

```
Python 3.14.7, GIL enabled: True
CPU-bound: sequential 1.94s  threads 1.83s  processes 0.87s
I/O-bound: sequential 2.02s  threads 0.51s  processes 0.92s
saved gil_benchmark.png
```

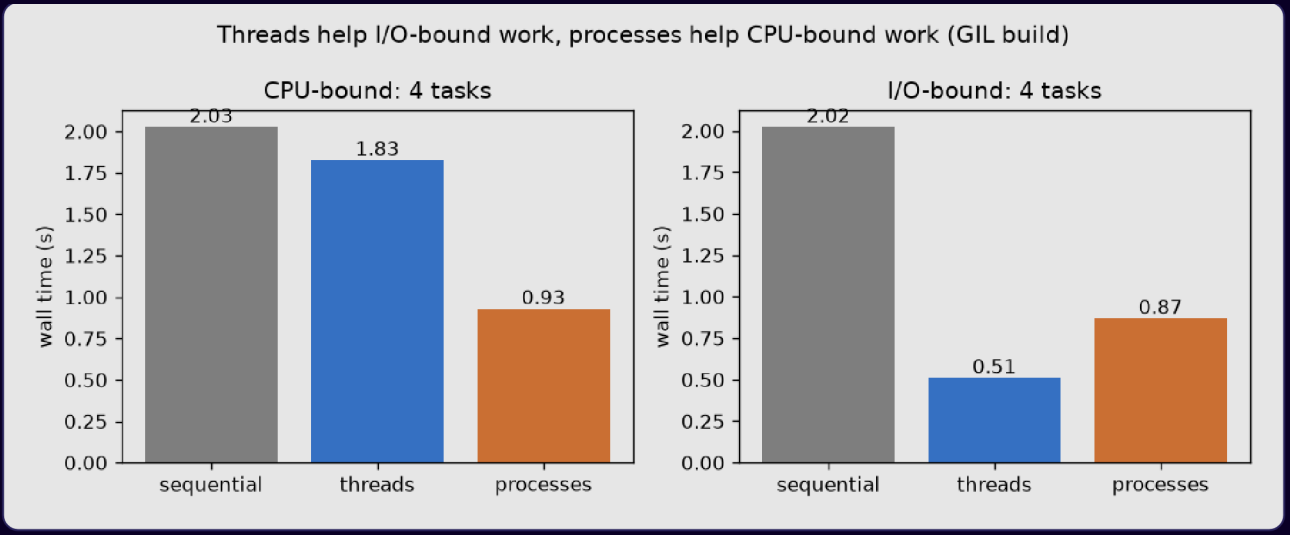


Figure – CPU-bound work only speeds up with processes; I/O-bound work speeds up with threads (processes also help but pay ~0.3–0.4 s start-up cost).

What it shows:

- **CPU-bound + threads ≈ sequential** (1.83 s vs 1.94 s is noise-level): the GIL lets only one thread run Python code at a time.
- **CPU-bound + processes ≈ 2.2× faster**: each process has its own interpreter and its own GIL, so they run on separate cores. It is not 4× because starting processes (the `spawn` method re-imports the module) and sending data back costs time.
- **I/O-bound + threads ≈ 4× faster**: while one thread sleeps (waits) it releases the GIL, so all four waits overlap.

TL;DR

Measure first. Waiting → threads or asyncio. Computing → processes, vectorized libraries, or (new) the free-threaded build.

16.3 Threads with `threading`

What it is

The `threading` module starts extra threads inside your process. All threads share global variables and objects on the heap. Each thread has its own call stack.

How it works

1. Create: `t = threading.Thread(target=func, args=(...), name="T1")`.
2. `t.start()` – the OS starts a new thread that calls `func(*args)`.
3. `t.join()` – the caller waits until `t` finishes (optionally `join(timeout)`).
4. A **daemon** thread (`daemon=True`) is killed abruptly when the main program exits; a normal thread keeps the program alive until it finishes. Use daemons only for work that can be lost (a heartbeat), never for writing files.

```
def worker(name: str, delay: float, results: dict) -> None:
    time.sleep(delay)                    # stands in for waiting on I/O
    results[name] = threading.current_thread().name

threads = [threading.Thread(target=worker, args=(f"job{i}", 0.2, results), name=f"T{i}")
           for i in range(3)]
for t in threads: t.start()              # all three begin waiting at once
for t in threads: t.join()               # wait for all to finish
```

Output (`threads_basics.py`, line 1):

```
3 x 0.2 s sleeps in threads took 0.21 s -> {'job0': 'T0', 'job1': 'T1', 'job2': 'T2'}
```

Three 0.2 s waits took 0.21 s in total because they overlapped. Threads return nothing from `target`; to get results, write to a shared structure (as here), use a `queue.Queue`, or — better — use `ThreadPoolExecutor` (16.7), which gives you return values and exceptions.

An alternative style is subclassing `threading.Thread` and overriding `run()`; it works, but passing a `target` function is simpler and more common.

Per-thread data. Sometimes each thread needs its *own* copy of a value (a DB connection, the current request ID). `threading.local()` gives an object whose attributes are separate per thread. In asyncio code, where many tasks share one thread, use `contextvars.ContextVar` instead — each task sees its own value (web frameworks store the current request this way).

TL;DR

`Thread(target=..., args=...)`, `start()`, `join()`. Threads share memory — which is both their convenience and their danger.

16.4 Race conditions and locks

What it is

A **race condition** happens when two threads read-modify-write the same data and their steps interleave. Even `counter += 1` is not one step: CPython executes it as *load counter → add 1 → store counter*. If thread A loads 5, then thread B loads 5, both store 6 — one increment is lost.

A **lock** (`threading.Lock`) fixes it: only one thread at a time can hold it, so the read-modify-write becomes a **critical section** that cannot interleave.

The race, demonstrated

```
def unsafe_increment(n: int) -> None:
    global counter
    for _ in range(n):
        current = counter                # read
        time.sleep(0)                    # yield: lets another thread run (widens the window)
        counter = current + 1            # write back a stale value

lock = threading.Lock()

def safe_increment(n: int) -> None:
    global counter
    for _ in range(n):
        with lock:                       # only one thread inside at a time
            current = counter
            time.sleep(0)
            counter = current + 1
```

Output (4 threads × 1,000 increments each):

```
unsafe counter (expected 4000): 1000
safe counter   (expected 4000): 4000
```

The unsafe number changes from run to run (other runs printed 2000 and 1031). The `time.sleep(0)` just makes the bug show up every time; without it the same bug still exists, it only appears rarely — which is worse, because it passes your tests and fails in production.

How it works — the synchronization toolbox

Primitive	What it does	Typical use
`Lock`	One holder at a time. Not re-entrant: the same thread acquiring twice blocks itself.	Protect shared state
`RLock`	Re-entrant lock: the owning thread may acquire it again (must release as many times).	Recursive code that re-enters a locked method
`Semaphore(n)`	Up to *n* holders at once.	Limit concurrent DB connections to 10
`Event`	A flag threads can `wait()` on until someone calls `set()`.	"Start now" / "shut down" signal
`Condition`	Wait until a condition becomes true, woken by `notify()`.	Building your own queues
`Barrier(n)`	Blocks until *n* threads arrive, then releases all.	Phased simulations
`queue.Queue`	Thread-safe FIFO with blocking `get` / `put`.	Producer–consumer (next section)

Always use locks through `with lock:` — it releases the lock even if an exception is raised. Manual `acquire()` / `release()` is useful only with a timeout: `if lock.acquire(timeout=1): ...`.

Common mistakes

- Believing "the GIL makes my code thread-safe". The GIL protects the *interpreter's* internals (so a `list` is never corrupted), not *your* multi-step logic. `if key not in d: d[key] = compute()` can still run twice.
- Protecting writes but not reads of a multi-field invariant (reading `balance` while another thread is half-way through a transfer).
- Holding a lock while doing slow I/O — other threads queue behind it.

In the interview

> **30-second answer:** "A race condition is when the outcome depends on thread timing, usually from an unprotected read-modify-write. Even `x += 1` is several bytecodes, so it's not atomic. I protect shared state with a `threading.Lock` used as a context manager, keep critical sections small, or avoid sharing entirely by passing messages through `queue.Queue`."

- *Is `list.append` thread-safe?* — A single `append` on a built-in list is safe in CPython (with the GIL, and the free-threaded build uses per-object locks for built-ins), but a sequence of operations is not. Don't rely on it for logic.
- *Lock vs RLock?* — RLock lets the same thread re-acquire; Lock would deadlock on itself.

Self-check

1. Why is `counter += 1` unsafe across threads? → It is load, add, store; another thread can run between them.
2. What does `with lock:` guarantee on an exception? → The lock is released.
3. Which primitive limits concurrency to N? → `Semaphore(N)`.

TL;DR

Shared mutable state + threads = races. Protect with `Lock` (via `with`), keep critical sections tiny, or share nothing and communicate through queues.

16.5 Producer–consumer with `queue.Queue`

What it is

Producer–consumer splits work into threads that *make* items and threads that *process* them, connected by a queue. `queue.Queue` is a thread-safe FIFO (first in, first out) with all locking done for you.

Why it matters

It removes most manual locking: threads never touch each other's data, they only `put` and `get`. A **bounded** queue (`maxsize=N`) also gives **backpressure** — a fast producer blocks on `put` instead of filling memory.

How it works

1. `q = queue.Queue(maxsize=5)`.
2. Producers call `q.put(item)` (blocks while full).
3. Consumers call `q.get()` (blocks while empty), process, then `q.task_done()`.
4. To stop, each producer puts a **sentinel** (a special "no more work" value, here `None`); the consumer stops after seeing one sentinel per producer. (Python 3.13 also added `Queue.shutdown()` for the same purpose.)

```
SENTINEL = None

def producer(q, items):
    for item in items:
        q.put(item)                      # blocks if the queue is full (backpressure)
    q.put(SENTINEL)

def consumer(q, out, n_producers):
    finished = 0
    while finished < n_producers:
        item = q.get()                   # blocks until an item is available
        if item is SENTINEL:
            finished += 1
        else:
            out.append(item * item)
        q.task_done()
```

Output:

```
consumer squared 10 items, sorted: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

`q.join()` (not used here) blocks until every item put has been matched by a `task_done()` — handy when the main thread wants to wait for "all work processed" without stopping the workers.

Queue class	Order
`queue.Queue`	FIFO
`queue.LifoQueue`	Stack (last in, first out)
`queue.PriorityQueue`	Smallest item first (put `(priority, item)` tuples)
`queue.SimpleQueue`	Unbounded FIFO, no `task_done` / `join`
`multiprocessing.Queue`	Between processes (items are pickled)
`asyncio.Queue`	Between asyncio tasks (not thread-safe)

Real-world examples

Web crawlers (fetcher threads → parser threads), logging (`logging.handlers.QueueHandler` + `QueueListener` move slow log I/O off request threads), and PyTorch's `DataLoader`, which uses worker *processes* feeding batches through queues.

TL;DR

Share a queue, not variables. Bound it for backpressure. Stop with sentinels or `shutdown()`.

16.6 The GIL (Global Interpreter Lock)

What it is

In the standard CPython build, the **GIL** is a single lock that a thread must hold to execute Python bytecode. So in one process, **only one thread runs Python code at any instant**, even on a 64-core machine.

Why it exists

CPython manages memory with **reference counts** (every object counts how many references point to it; Ch 17). Updating those counts from several threads at once would need a lock on every object or atomic operations everywhere, which was slower and much harder to get right. One big lock made the interpreter simple, single-threaded code fast, and C extensions easy to write — which is a big reason numpy, and the rest of the scientific stack, exist.

How it works

1. A thread holds the GIL while it runs bytecode.
2. It **releases** the GIL when it does blocking I/O (`socket.recv`, file reads, `time.sleep`) and when a C extension explicitly releases it (numpy on large arrays, `hashlib` on big buffers, `zlib`, many image and DB libraries).
3. A thread that wants the GIL waits; after the **switch interval** (`sys.getswitchinterval()`, default 0.005 s = 5 ms) the holder is asked to drop it so others get a turn.

Consequences (the benchmark in 16.2):

Workload	Threads with the GIL
I/O-bound	Scale well — the GIL is released while waiting
CPU-bound pure Python	No speed-up; can even be slightly slower (switching overhead)
CPU-bound in numpy / C that releases the GIL	Can scale across cores

Free-threaded CPython (PEP 703) — status

- **Python 3.13 (Oct 2024):** an *optional, experimental* build with the GIL disabled ships (installers call it "free-threaded"; the executable is `python3.13t`). PEP 703 is the design.
- **Python 3.14 (Oct 2025):** per **PEP 779**, the free-threaded build is **officially supported** (no longer experimental) but **still optional — not the default**. `python3.14` still has a GIL; `python3.14t` does not. The docs put the single-threaded slowdown at roughly 5–10 % (depending on platform and compiler), and memory use is higher.
- **Extensions:** a C extension that is not marked free-threading-compatible makes the interpreter **re-enable the GIL** on import (with a warning). You can also force it: `PYTHON_GIL=1` or `python -X gil=1`. Many major packages (numpy among them) now publish free-threaded wheels, but check before you rely on it.
- **Check at runtime:**

```
import sys, sysconfig
sysconfig.get_config_var("Py_GIL_DISABLED")  # 1 on a free-threaded build, else 0/None
sys._is_gil_enabled()                        # False when running without the GIL (3.13+)
```

- **What changes for you:** on a free-threaded build, the CPU-bound thread benchmark above can scale across cores — but races become *more* likely, not less, so all the locking rules of 16.4 still apply. Built-in `list` / `dict` / `set` use internal per-object locks so they are not corrupted, but multi-step logic still needs your locks.

Other ways around the GIL: **processes** (each has its own GIL), **subinterpreters** — since 3.12 each interpreter can have its own GIL (PEP 684) and 3.14 adds the `concurrent.interpreters` module and `InterpreterPoolExecutor` (PEP 734) — and native code (numpy, Cython, Rust via PyO3) that releases the GIL.

Common mistakes

- "Python can't do concurrency." It does I/O concurrency very well; the GIL limits *parallel Python bytecode*.
- "The GIL makes code thread-safe." See 16.4.
- Assuming other implementations behave the same: Jython and IronPython have no GIL; PyPy has one.

In the interview

> **30-second answer:** "The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It exists because CPython's reference counting isn't thread-safe without it, and it keeps single-threaded code and C extensions simple. It's released during blocking I/O and by many C extensions, so threads are fine for I/O-bound work; for CPU-bound Python I use multiprocessing or push the work into numpy. Since 3.13 there's an optional free-threaded build, officially supported in 3.14 but not the default."

- *Does the GIL make `x += 1` atomic?* – No. The thread can be switched between the load and the store.
- *Why doesn't numpy suffer from the GIL?* – Its heavy loops run in C and release the GIL.
- *Should I switch to the free-threaded build today?* – Only after checking that all your C-extension dependencies support it and benchmarking; single-threaded code gets a bit slower.

Self-check

1. Default switch interval? → 5 ms.
2. Name two ways to get CPU parallelism in CPython with a GIL. → Processes; C extensions that release the GIL (numpy).
3. How do you tell if the GIL is on at runtime? → `sys._is_gil_enabled()`.
4. Is the free-threaded build the default in 3.14? → No – supported but optional (`python3.14t`).

TL;DR

One thread runs Python bytecode at a time in standard CPython. I/O releases it; pure-Python CPU work doesn't. Free-threading: experimental in 3.13, officially supported but optional in 3.14.

16.7 `multiprocessing`

What it is

`multiprocessing` runs functions in separate **processes**. Each process has its own interpreter, memory and GIL, so CPU-bound Python code runs truly in parallel. The API mirrors `threading`.

How it works

1. **Start method** – how the child process is created:

Method	How	Default on
`spawn`	Fresh interpreter; re-imports your main module; arguments are pickled	Windows, macOS
`fork`	Copies the parent process (fast, but unsafe if the parent has threads)	– (was Linux's default up to 3.13)
`forkserver`	Forks from a clean single-threaded server process	Linux and other POSIX except macOS, from 3.14

Because `spawn` / `forkserver` re-import your script, **always guard the start-up code with `if __name__ == "__main__":`**, or each child would try to start children of its own.

2. **Pickling** – arguments and return values travel between processes by `pickle` (serialization). Lambdas, nested functions and open sockets can't be pickled; use top-level functions.

3. **No shared memory by default** — each process has its own copy of globals.

```
counter = 0                              # each process gets its OWN copy

def bump_global(n: int, parent_pid: int) -> None:
    global counter
    for _ in range(n):
        counter += 1
    print(f"  child: different pid from parent={os.getpid() != parent_pid}, sees counter={counter}")

def bump_shared(shared, n: int) -> None:
    for _ in range(n):
        with shared.get_lock():          # Value comes with its own lock
            shared.value += 1

if __name__ == "__main__":
    p = mp.Process(target=bump_global, args=(1000, os.getpid()))
    p.start(); p.join()
    shared = mp.Value("i", 0)            # 'i' = C int in shared memory
    # ... start 4 processes running bump_shared(shared, 1000) ...
    with mp.Pool(processes=4) as pool:
        pool.map(square, range(10))
```

Output (`multiprocessing_demo.py`):

```
start method: spawn, CPUs: 14
  child: different pid from parent=True, sees counter=1000
parent counter after child ran: 0  (not shared!)  exitcode=0
shared Value after 4 x 1000 increments: 4000
pool.map squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
pool.map chunksize=3: [0, 1, 4, 9, 16] ...
lambda in Pool -> PicklingError
```

Shared-state options and caveats

Tool	What it is	Caveat
Return values / `Queue` / `Pipe`	Message passing (pickled)	Preferred; cost grows with data size
`Value`, `Array`	A C number/array in shared memory	Needs `get_lock()` for read-modify-write
`multiprocessing.shared_memory.SharedMemory`	Raw shared bytes (wrap in a numpy array)	You manage layout, lifetime and `unlink()`
`Manager()`	A server process holding a dict/list proxies talk to	Easy but slow (every access is IPC)

`Pool` methods: `map` (ordered, blocks), `imap` / `imap_unordered` (lazy, stream results), `starmap` (multiple arguments), `apply_async` (one call, returns an `AsyncResult`). The `chunksize` argument batches items per message — large chunks cut overhead for many small tasks.

When not to use it

- Tiny tasks: pickling and process start-up (tens to hundreds of ms under `spawn`) outweigh the work.
- Huge inputs that must be copied to every worker — use shared memory, or let each worker load its own slice from disk.
- I/O-bound work — threads or asyncio are lighter.

Common mistakes

- Missing `if __name__ == "__main__":` → `RuntimeError` or runaway process creation under `spawn`.
- Expecting a global modified in a child to change in the parent.

- Passing lambdas/local functions → `PicklingError` / `AttributeError`.
- Mixing `fork` with threads → child can inherit a lock held by a thread that no longer exists, and hang.

Real-world examples

Gunicorn and uWSGI run several **worker processes** of a Django/Flask app to use all cores; Celery's default worker pool is "prefork" (processes); PyTorch `DataLoader(num_workers=4)` loads batches in worker processes.

TL;DR

Processes = real CPU parallelism at the cost of start-up, pickling and no shared memory. Guard `__main__`, use top-level functions, prefer message passing.

16.8 `concurrent.futures` — executors

What it is

A single high-level API for "run these calls in the background and give me the results": `ThreadPoolExecutor` (threads) and `ProcessPoolExecutor` (processes) share the same interface. You get a **`Future`** — an object representing a result that will exist later.

How it works

1. `with ThreadPoolExecutor(max_workers=8) as ex:` — creates a pool; leaving the `with` waits for all work (`shutdown(wait=True)`).
2. `fut = ex.submit(fn, *args)` → a `Future`. `fut.result(timeout=None)` waits and returns the value **or re-raises the exception** from the worker.
3. `ex.map(fn, iterable)` → results in **input order**, lazily.
4. `as_completed(futures)` → yields futures in **completion order**.
5. `wait(futures, return_when=FIRST_COMPLETED | FIRST_EXCEPTION | ALL_COMPLETED)` → `(done, pending)`.
6. `fut.cancel()` works only if the call hasn't started yet.

Default pool sizes (3.13+): threads `min(32, os.process_cpu_count() + 4)`; processes `os.process_cpu_count()`.

```
with ThreadPoolExecutor(max_workers=3) as ex:
    futures = {ex.submit(fake_download, u): u for u in urls}
    for fut in as_completed(futures):
        try:
            print(f"done {futures[fut]} -> {fut.result()}")
        except ConnectionError as e:
            print(f"failed {futures[fut]} -> {e!r}")

with ThreadPoolExecutor(max_workers=3) as ex:
    it = ex.map(fake_download, urls)
    try:
        for r in it:
            print(f"map result {r}")
    except ConnectionError as e:
        print(f"map raised {e!r}")
```

Output (`futures_demo.py`; delays a=0.3 s, b=0.1 s, c=0.2 s and c fails):

```
done b.com -> ('b.com', 100)
failed c.com -> ConnectionError('c.com timed out')
done a.com -> ('a.com', 300)
map result ('a.com', 300)
map result ('b.com', 100)
map raised ConnectionError('c.com timed out')
first completed: 1 done, 1 cancelled
primes: [True, True, True, False] in 0.32s with processes
```

What it shows: `as_completed` hands back the fastest first; `map` keeps input order and raises when it *reaches* the failed item; with one worker and three 0.2 s jobs, after the first finishes the second is already running (can't be cancelled) and only the third is cancelled; switching to processes for CPU work is a one-word change.

Trade-offs

	Raw `threading` / `multiprocessing`	`concurrent.futures`
Return values & exceptions	Manual (queues, shared dicts)	Built in (`Future.result()`)
Thread ↔ process switch	Rewrite	Change the class name
Fine control (daemons, custom sync)	Yes	Limited
Best for	Long-lived workers, pipelines	"Run N calls, collect results"

In the interview

30-second answer: "`concurrent.futures` gives one API over thread and process pools. `submit` returns a Future; `result()` returns the value or re-raises the worker's exception. `map` preserves order, `as_completed` yields in finish order. I use `ThreadPoolExecutor` for blocking I/O and `ProcessPoolExecutor` for CPU-bound pure Python."

- *What happens to an exception in a worker?* – Stored in the Future; raised when you call `result()` (or iterate `map`). If you never check, it is silently lost.
- *How do you add a timeout?* – `fut.result(timeout=5)` or `as_completed(fs, timeout=5)` → `TimeoutError`.

TL;DR

Default choice for simple parallel work. Always call `result()` so exceptions surface.

16.9 `asyncio` – async/await

What it is

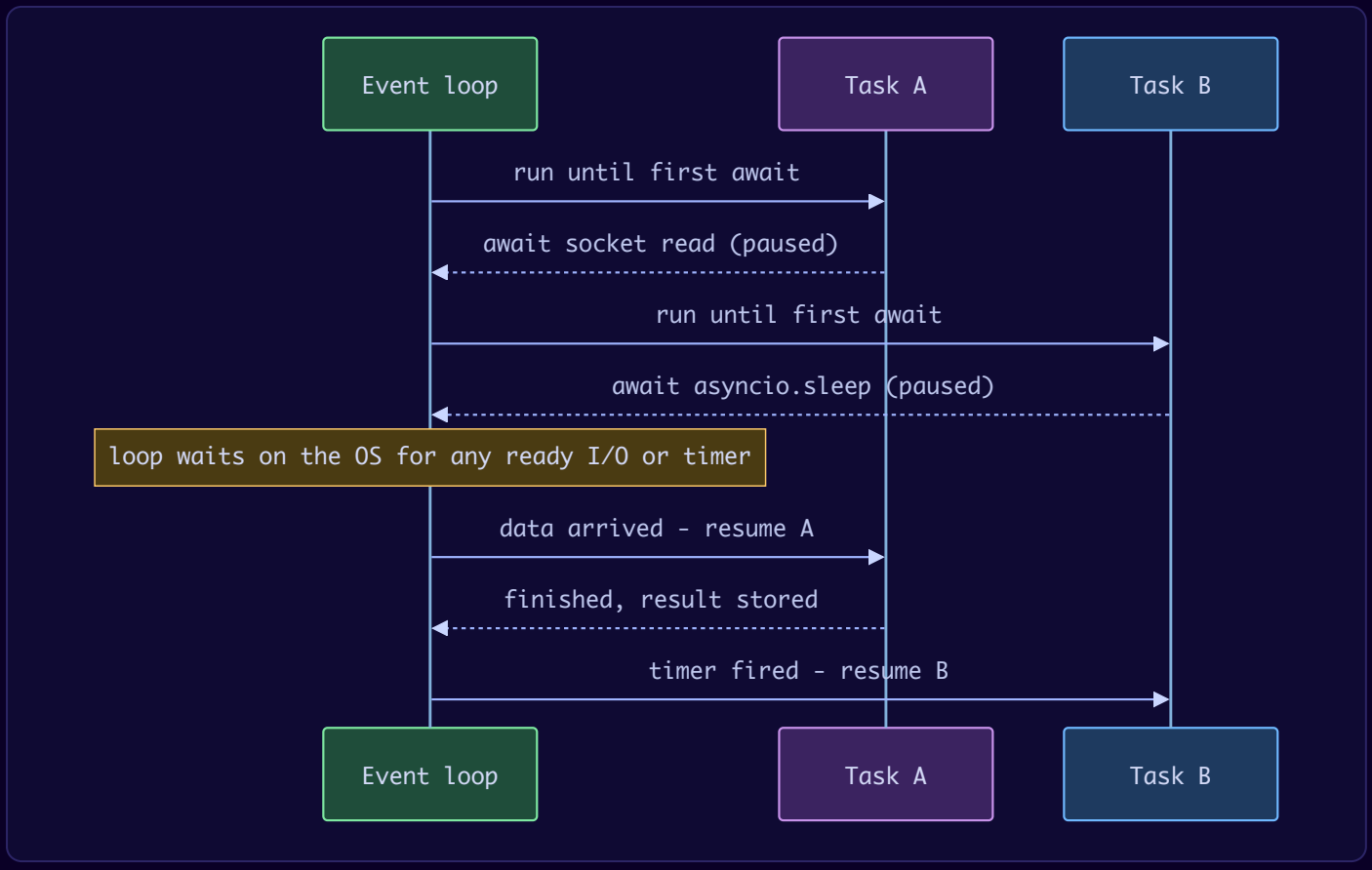
`asyncio` runs many tasks concurrently in **one thread** using an **event loop**. A **coroutine** (`async def`) is a function that can pause at each `await` and let the loop run something else while it waits for I/O. Nothing runs in parallel; the speed comes from never sitting idle.

Analogy: one waiter serving 20 tables. They take an order, hand it to the kitchen (`await`), and serve other tables instead of standing at the kitchen door.

Why it matters

A thread costs memory (its stack) and OS scheduling; 10,000 threads is painful. 10,000 asyncio tasks waiting on sockets is normal. Switching only happens at `await`, so there are far fewer race windows than with threads.

How it works



1. `asyncio.run(main())` creates an event loop, runs `main()` to completion and closes the loop. Call it once, at the top.
2. Calling `fetch()` creates a **coroutine object**; nothing runs until it is awaited or wrapped in a task.
3. `await x` pauses the current coroutine until `x` (a coroutine, Task or Future) is done.
4. `asyncio.create_task(coro)` schedules a coroutine to run **concurrently**, returning a `Task`.
5. The loop uses the OS's readiness APIs (epoll/kqueue) to know which sockets and timers are ready and resumes the corresponding tasks.

Core API with real output (`asyncio_demo.py`)

```python
async def fetch(name: str, delay: float) -> str:
    await asyncio.sleep(delay)           # non-blocking wait
    return f"{name}:{delay}"

await fetch("a", 0.2); await fetch("b", 0.2)                      # sequential: 0.4 s
results = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.2), fetch("c", 0.1))  # 0.2 s

task = asyncio.create_task(fetch("bg", 0.2))    # starts running now
await asyncio.sleep(0.1)
print(task.done())                              # False
value = await task
```

```
type before await: coroutine
awaited: x:0
[0.40s] two sequential awaits
[0.20s] gather -> ['a:0.2', 'b:0.2', 'c:0.1']  (order = argument order)
[0.10s] task done yet? False
[0.20s] task result bg:0.2
```

Key lesson: `await a(); await b()` is **sequential**. Concurrency only happens with `gather`, `create_task` or a `TaskGroup`.

`gather` vs `TaskGroup` (3.11+)

```
res = await asyncio.gather(fetch("ok", 0.1), failing(0.05), return_exceptions=True)

try:
    async with asyncio.TaskGroup() as tg:
        slow = tg.create_task(fetch("slow", 1.0))
        tg.create_task(failing(0.1))
except* ValueError as eg:                       # except* handles an ExceptionGroup
    print(eg.exceptions, slow.cancelled())
```

```
gather with return_exceptions -> ['ok:0.1', ValueError('boom')]
[0.10s] TaskGroup raised (ValueError('boom'),); slow cancelled=True
```

	`asyncio.gather`	`asyncio.TaskGroup`
On first failure	Raises it, but the **other tasks keep running** unattended	**Cancels** the siblings, waits for them, raises an `ExceptionGroup`
Collect errors as values	`return_exceptions=True`	Catch with `except*`
Results	List in argument order	`task.result()` on each task
Recommended for new code	Simple fan-out	Yes – "structured concurrency": no task outlives its block

Timeouts, limits and async iteration

```
async with asyncio.timeout(0.1):                # 3.11+, can wrap several awaits
    await fetch("too-slow", 1.0)
await asyncio.wait_for(fetch("too-slow", 1.0), timeout=0.1)   # one awaitable

sem = asyncio.Semaphore(2)                      # at most 2 at a time
async def limited(i):
    async with sem:
        await asyncio.sleep(0.1)
        return i

async def ticker(n):                            # async generator
    for i in range(n):
        await asyncio.sleep(0.05)
        yield i

async with AsyncResource():                     # __aenter__ / __aexit__
    squares = [i * i async for i in ticker(4)] # async comprehension
```

```
asyncio.timeout -> TimeoutError
wait_for -> TimeoutError
[0.31s] 6 jobs, 2 at a time -> [0, 1, 2, 3, 4, 5]
[0.00s] open resource
[0.20s] close resource
async comprehension -> [0, 1, 4, 9]
```

On timeout the inner task is **cancelled** (a `CancelledError` is raised inside it) and the caller sees the built-in `TimeoutError` (since 3.11, `asyncio.TimeoutError` is the same class). Six jobs with a limit of two took 3 rounds × 0.1 s.

Async protocol summary: `async for` uses `__aiter__` / `__anext__`; `async with` uses `__aenter__` / `__aexit__`. Other useful pieces: `asyncio.Queue` (producer–consumer between tasks), `asyncio.Event`, `asyncio.Lock`, `asyncio.as_completed`.

Mixing sync and async

```
def blocking_io() -> str:
    time.sleep(0.3)                 # a sync library call (e.g. requests.get)
    return "sync result"

await asyncio.gather(asyncio.to_thread(blocking_io), asyncio.to_thread(blocking_io))
async def bad():
    return blocking_io()            # time.sleep inside a coroutine blocks EVERYTHING
await asyncio.gather(bad(), bad())
```

```
[0.31s] two blocking calls via to_thread ran together
[0.60s] two blocking calls inside coroutines ran one after another
```

One blocking call inside a coroutine freezes the **entire** loop — every other task, every connection. Use async libraries (`httpx` / `aiohttp` instead of `requests`, `asyncpg` instead of `psycopg2` sync, `asyncio.sleep` instead of `time.sleep`), or push sync calls to a thread with `asyncio.to_thread` (3.9+) / `loop.run_in_executor`. CPU-heavy work goes to a `ProcessPoolExecutor` via `run_in_executor`.

When to use asyncio

- **Yes:** many simultaneous network connections — web servers/APIs (FastAPI, Starlette, aiohttp), websockets, chat, crawlers, calling many microservices or LLM APIs concurrently.
- **No:** CPU-bound work (single thread!), a script that makes three HTTP calls (threads are simpler), or a codebase whose key libraries are all synchronous.

Common mistakes

- Forgetting `await`: `fetch()` without `await` just creates a coroutine; Python warns *"coroutine ... was never awaited"*.
- `create_task` and dropping the reference: the loop keeps only a weak reference, so an unreferenced task can be garbage-collected mid-run. Keep tasks in a set or use a `TaskGroup`.
- Calling `asyncio.run()` inside an already-running loop (e.g. Jupyter) → `RuntimeError`; in notebooks just `await main()`.
- Catching `CancelledError` and not re-raising it — breaks timeouts and TaskGroup cancellation.
- Sharing an `asyncio.Queue` / `Lock` with other threads — asyncio objects are not thread-safe; use `loop.call_soon_threadsafe` or `asyncio.run_coroutine_threadsafe`.

Real-world examples

FastAPI/Starlette serve async endpoints on the uvicorn server (uvloop, a faster event loop written on libuv, is optional); Discord bots (`discord.py`), Home Assistant, and the official Anthropic and OpenAI Python SDKs ship `Async...` clients for concurrent requests.

In the interview

30-second answer: "asyncio is single-threaded cooperative concurrency. Coroutines pause at `await`, and the event loop resumes whichever task's I/O is ready. It scales to thousands of connections cheaply, but anything blocking stalls everything, so I use async libraries or `asyncio.to_thread`. For fan-out I prefer `TaskGroup` over `gather` because a failure cancels the siblings."

- *Is asyncio parallel?* — No, one thread; concurrency only.
- *`gather` vs `TaskGroup`?* — `gather` leaves siblings running on error unless you handle it; `TaskGroup` cancels them and raises an `ExceptionGroup`.
- *How do you call sync code from async?* — `await asyncio.to_thread(fn, *args)`.

Self-check

1. What runs when you call an `async def` function? → Nothing; you get a coroutine object.

2. Two `await`s in a row — concurrent? → No, sequential.
3. How do you cap concurrency at 10? → `asyncio.Semaphore(10)`.
4. What exception does `asyncio.timeout` raise to the caller? → `TimeoutError`.
5. Why is `time.sleep` in a coroutine bad? → It blocks the whole event loop.

TL;DR

One thread, one loop, many tasks; switch only at `await`. `asyncio.run` once, `TaskGroup` / `gather` for concurrency, `timeout` for deadlines, `Semaphore` for limits, `to_thread` for sync calls. Never block the loop.

16.10 Choosing the right tool

Situation	Use	Why
A few to hundreds of blocking I/O calls, sync libraries	`ThreadPoolExecutor`	Simple, GIL released during I/O
Thousands of concurrent connections, async libraries	`asyncio`	Tasks are cheap, no thread per connection
CPU-bound pure Python	`ProcessPoolExecutor` / `multiprocessing.Pool`	Each process has its own GIL
CPU-bound numeric work	numpy/pandas (vectorize, Ch 17–18)	C loops, often multi-threaded internally
CPU-bound, want shared memory, deps support it	Free-threaded build (3.14t) + threads	Real parallel threads
Long-running pipeline stages	Threads/processes + `Queue`	Backpressure, clean shutdown
Async app needs a sync library	`asyncio.to_thread`	Keeps the loop responsive
Async app needs heavy CPU	`loop.run_in_executor(ProcessPoolExecutor(), ...)`	Moves CPU work off the loop
Scale beyond one machine	Celery, RQ, Ray, Dask, Spark	Distributed task queues/clusters

	Threads	Processes	asyncio
Parallel CPU (GIL build)	No	Yes	No
Memory per unit	~MB stack (virtual)	Whole interpreter (tens of MB)	~KB per task
Shared memory	Yes (needs locks)	No (pickle/IPC)	Yes (single thread)
Switching	Pre-emptive (any time)	OS	Cooperative (at `await`)
Race risk	High	Low	Low
Works with sync libs	Yes	Yes	Needs `to_thread`

16.11 Common pitfalls

Blocking calls inside async code

Shown in 16.9: `time.sleep`, `requests.get`, a sync DB driver, big file reads, or a long CPU loop in a coroutine halts every other task. Diagnose with **debug mode** (`asyncio.run(main(), debug=True)` or `PYTHONASYNCIODEBUG=1`), which logs callbacks slower than 100 ms. Python 3.14 also adds `python -m asyncio ps PID` / `pstree PID` to inspect the tasks of a running process.

Deadlocks

A **deadlock** happens when thread 1 holds lock A and waits for B, while thread 2 holds B and waits for A. Both wait forever.

```
def transfer(first, second, name, log):
    with first:
        time.sleep(0.05)                          # make the bad interleaving likely
        if second.acquire(timeout=0.5):           # a real deadlock would wait forever
            second.release()
            log.append(f"{name} ok")
        else:
            log.append(f"{name} gave up: DEADLOCK")

run((lock_a, lock_b), (lock_b, lock_a))   # opposite order
run((lock_a, lock_b), (lock_a, lock_b))   # same order
```

Output (`pitfalls_demo.py`):

```
opposite order : ['T1 gave up: DEADLOCK', 'T2 ok']
same order     : ['T1 ok', 'T2 ok']
RLock re-entered by the same thread: fine
plain Lock re-acquire (timeout 0.1): False
```

The timeout lets the demo finish; without it both threads would hang. Fixes:

1. **Global lock ordering** – always acquire locks in the same order (e.g. by account id).
2. **Fewer locks** – one coarse lock, or no shared state (queues, immutable data).
3. **Timeouts** on `acquire` to detect and back off.
4. `RLock` when the same thread legitimately re-enters (a plain `Lock` deadlocks on itself, as the last line shows).
5. In asyncio: a task awaiting something that only it could produce, or a TaskGroup child waiting on its parent – same idea, same cure (clear ownership).

Other pitfalls

Pitfall	Symptom	Fix
Unprotected shared state	Wrong totals, rare	Lock, or message passing
Unhandled exception in a `Thread`	Printed to stderr, thread dies silently	Use executors and call `result()`; or set `threading.excepthook`
Too many threads	Memory, context-switch overhead	Bounded pool
Missing `__main__` guard	Recursive spawning / `RuntimeError`	Guard the entry point
Huge arguments to processes	Slow pickling	Shared memory, or load inside the worker
Forgotten `await` / lost task reference	Work silently never runs / vanishes	Linters (ruff), `TaskGroup`
`fork` after threads started	Random hangs in child	`spawn` / `forkserver` (defaults on macOS/Windows, and Linux from 3.14)

In the interview

- *Four conditions for deadlock?* – Mutual exclusion, hold-and-wait, no pre-emption, circular wait. Break any one (lock ordering breaks circular wait). Details: LLD Ch 12.
- *How would you find a hung thread?* – `faulthandler.dump_traceback_later()` or `py-spy dump --pid` show every thread's stack.

16.12 Chapter self-check

1. Concurrency vs parallelism in one line each? → Overlapping waits vs simultaneous execution on several cores.
2. Why do threads speed up downloads but not a pure-Python prime search? → I/O releases the GIL; bytecode execution doesn't.
3. What does `q.task_done()` pair with? → `q.join()`, which waits until every item is marked done.
4. Why must `multiprocessing` code have `if __name__ == "__main__":`? → `spawn` / `forkserver` re-import the main module in each child.
5. Which default start method does Linux use from 3.14? → `forkserver`.
6. `ex.map` vs `as_completed` ordering? → Input order vs completion order.
7. What happens to siblings when one TaskGroup task fails? → They are cancelled; an `ExceptionGroup` is raised.
8. How do you run `requests.get` from async code without freezing the loop? → `await asyncio.to_thread(requests.get, url)`.
9. What's the status of free-threaded CPython in 3.14? → Officially supported (PEP 779), optional, not the default.
10. The simplest way to prevent lock-ordering deadlocks? → Always acquire locks in one global order.

Chapter TL;DR

- **First classify the work**: waiting (I/O-bound) or computing (CPU-bound). Measure.
- **Threads**: cheap concurrency for I/O; share memory → protect with `Lock`, or pass messages through `queue.Queue` (bounded, with sentinels).
- **GIL**: one thread runs Python bytecode at a time in standard CPython; released on I/O and by C extensions. Free-threaded build: experimental in 3.13, supported but optional in 3.14 (`python3.14t`, ~5–10 % single-thread cost).
- **Processes**: real parallelism; no shared globals; arguments are pickled; guard `__main__`; `spawn` on macOS/Windows, `forkserver` on Linux from 3.14.
- **Executors**: `ThreadPoolExecutor` / `ProcessPoolExecutor`, `submit` → `Future.result()`, `map` (ordered), `as_completed` (finish order). The default choice for "run N things".
- **asyncio**: one thread, event loop, coroutines pause at `await`. `gather` / `TaskGroup` for concurrency, `asyncio.timeout`, `Semaphore`, async generators/context managers, `to_thread` for sync code. Never block the loop.
- **Pitfalls**: races, deadlocks (fix with lock ordering), blocking calls in async, lost exceptions, missing `__main__` guard.

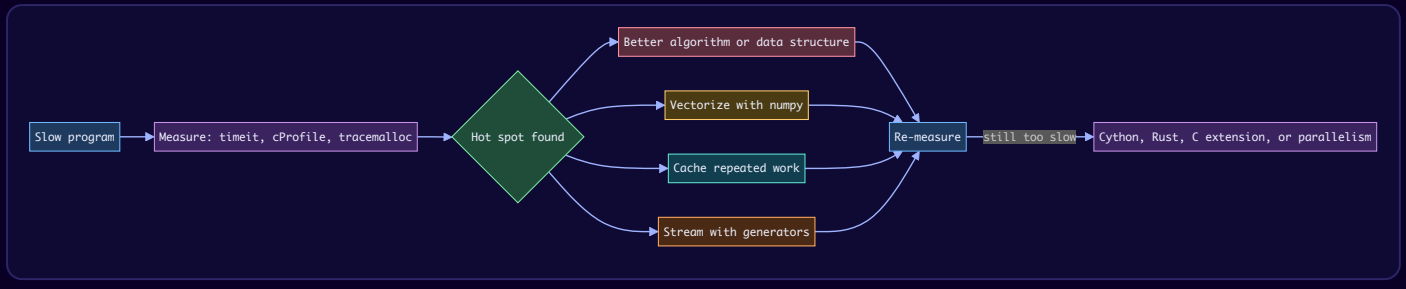
Chapter 17 — Performance, Memory and CPython Internals

What you will learn. How CPython represents every value as an object (`PyObject` : type + reference count + value), how memory is freed (reference counting plus the cyclic garbage collector), what weak references are for, why `256 is 256` but maybe not `257 is 257` (small-int cache and string interning), a recap of mutability and copying, the **big-O cost of built-in operations** (list, dict, set, deque), how to **profile** (`timeit` , `cProfile` , `tracemalloc` , line-level tools) and the optimisations that actually pay off — the right data structure, numpy vectorisation, caching, generators, local lookups, `str.join` , `__slots__` — plus when to drop to C/Cython/Rust and what the "Faster CPython" releases (3.11+) changed.

Prerequisites. Ch 2 (names and objects), Ch 5–6 (containers), Ch 10 (classes), Ch 11 (generators), Ch 16 (the GIL).

OS fundamentals. What sits underneath every Python process — processes vs threads, CPU scheduling, virtual memory and page faults, system calls, file descriptors, blocking vs `epoll` / `kqueue` I/O (the mechanism under `asyncio` 's event loop and `selectors`), and the page cache — is explained once, language-neutrally, in the Java handbook's §16.13 Operating-system fundamentals for engineers. The Python equivalents: `os.getpid()` / `multiprocessing` (processes), `threading` (threads under the GIL, Ch 16), `mmap` , `os.fsync` , `resource.getrlimit(resource.RLIMIT_NOFILE)` and the `selectors` module.

Code. `chapters/17-performance-memory-internals/code/` — `object_model.py` , `gc_weakref.py` , `complexity_bench.py` (plot), `profiling_demo.py` , `optimizations.py` (plot). Output from CPython 3.14.7, 64-bit macOS. Byte sizes and timings are CPython implementation details and can differ between versions and machines.



17.1 The CPython object model

What it is

In Python **everything is an object**: numbers, strings, functions, classes, modules, even `None` and types themselves. Every object has three things:

Part	Meaning	How to see it
Identity	Which object it is; never changes during its life. In CPython it is the memory address.	`id(x)` , `x is y`
Type	What it can do; also never changes.	`type(x)`
Value	Its data. Changeable for *mutable* objects (list, dict, set), fixed for *immutable* ones (int, str, tuple, frozenset).	`==` compares values

Inside CPython (written in C) every object begins with a header struct called `PyObject` holding two fields: `ob_refcnt` (the **reference count**, how many references point at it) and `ob_type` (a pointer to its type object). A variable is just a **name** pointing to an object — the object lives on the heap, the name holds a reference.

Analogy: objects are houses, names are address labels written on sticky notes. Several notes can carry the same address; when no note carries it any more, the house is demolished.

How it works — demo

```
data = [1, 2, 3]
sys.getrefcount(data)            # 1
alias = data; holder = {"k": data}
sys.getrefcount(data)            # 3  (two more references)
del alias, holder                # back to 1

x = 10; before = id(x); x += 1    # int is immutable -> x now names a NEW object
lst = [1]; before = id(lst); lst += [2]   # list is mutable -> same object, changed in place
```

Output (`object_model.py`):

```
refcount of new list: 1  (exact numbers are an implementation detail)
after alias + dict entry: 3
after del: 1
refcount of None: 3221225472  (immortal object, 3.12+)
a is b: True, a == c: True, a is c: False
type(a): list, type(type(a)): type, isinstance(len, object): True
int += changes id: True
list += keeps id: True
```

Notes on the output:

- `sys.getrefcount` traditionally reports one extra (the temporary reference held by its own argument), yet here a fresh list shows 1: newer interpreters skip some reference-count updates, so the exact value varies by version. The docs say not to rely on the number beyond 0 or 1 — use it only to watch it go **up and down**.
- `None`'s huge count: since 3.12 (PEP 683), objects like `None`, `True`, small ints and some strings are **immortal** — their refcount is never changed, which avoids cache-line traffic and helps the free-threaded build.
- `type(type(a))` is `type`: classes are objects too, and their type is `type`.

Object sizes (64-bit CPython 3.14)

```
  getsizeof(int 0       ) =   28 bytes
  getsizeof(int 1       ) =   28 bytes
  getsizeof(int 2**30   ) =   32 bytes
  getsizeof(int 2**100  ) =   40 bytes
  getsizeof(float       ) =   24 bytes
  getsizeof(str ''      ) =   41 bytes
  getsizeof(str 'abc'   ) =   44 bytes
  getsizeof(str 'é'     ) =   61 bytes
  getsizeof(tuple ()    ) =   48 bytes
  getsizeof(list []     ) =   56 bytes
  getsizeof(list 3 items) =   88 bytes
  getsizeof(dict {}     ) =   64 bytes
  getsizeof(set()       ) =  216 bytes
  getsizeof(None        ) =   16 bytes
```

Lessons: a Python `int` costs ~28 bytes versus 8 bytes in a numpy `int64` array; ints grow as needed (arbitrary precision); an ASCII string costs ~41 bytes + 1 byte per char, and one non-ASCII character switches the whole string to a wider storage (PEP 393). `sys.getsizeof` is **shallow**: a list's size counts its pointer array (8 bytes per slot), not the objects inside.

Common mistakes

- Using `is` to compare values (`if x is 1000`). `is` checks identity; use `==`. Only use `is` for singletons: `None`, `True` / `False`, sentinels.
- Treating `id()` as unique forever: after an object dies, its id (address) can be reused.

In the interview

> **30-second answer:** "In CPython every value is a heap object starting with a `PyObject` header – a reference count and a type pointer. Variables are names bound to objects. Identity is `id` / `is`, type is fixed, and the value may or may not be mutable. Memory is freed when the refcount hits zero, with a cycle collector for reference cycles."

- *Difference between `==` and `is`?* – Value equality (`__eq__`) vs same object.
- *Why is a list of a million ints ~36 MB, but a numpy array ~8 MB?* – The list stores 8-byte pointers to separate ~28-byte int objects; numpy stores raw 8-byte values contiguously.

Self-check

1. Three properties of every object? → Identity, type, value.
2. What two fields start every `PyObject`? → Reference count and type pointer.
3. Does `x += 1` on an int change the object? → No, it rebinds `x` to a new int.
4. Is `sys.getsizeof([big objects])` the total memory? → No, it's shallow.

TL;DR

Everything is an object with identity, type and value; names are references. CPython tracks each object's references in a refcount; immortal objects (3.12+) skip that.

17.2 Memory management and garbage collection

What it is

CPython frees memory in two ways:

1. **Reference counting** – the main mechanism. Each new reference increments the count, each dropped one decrements it; when it hits 0 the object is freed **immediately**.
2. **Cyclic garbage collector (`gc` module)** – a backup that finds groups of objects that reference each other (a **reference cycle**) but that nothing else can reach. Their counts never reach 0 on their own, so refcounting alone would leak them.

Under both sits **pymalloc**, CPython's allocator for small objects (≤ 512 bytes), which carves memory into pools and arenas so tiny allocations are fast. (Freed memory usually returns to pymalloc's pools rather than straight to the OS, which is why a process's memory can stay high after you delete data.)

How it works

- **Refcounting** is deterministic: files closed by the last reference going away, `__del__` running at a predictable point (in CPython – other implementations differ, so still use `with`).
- **Cyclic GC** groups container objects (lists, dicts, class instances …) into **generations**: new objects start young; survivors get promoted. Young objects are checked often, old ones rarely – "most objects die young". A collection starts when allocations minus deallocations exceed a threshold (`gc.get_threshold()`). It only tracks containers; ints and strings can't form cycles.
- Version note: 3.14.0–3.14.4 shipped an *incremental* collector (it did a slice of the old-generation work at a time to shorten pauses); 3.14.5 (May 2026) reverted to the 3.13-style generational collector because of memory-pressure problems reported in production. The ideas above hold for both.

```
class Node:
    def __init__(self, name): self.name, self.other = name, None

n = Node("solo")
weakref.finalize(n, print, "  solo freed as soon as last reference went away")
del n                                   # refcount -> 0 -> freed right here

gc.collect(); gc.disable()
a, b = Node("a"), Node("b")
a.other, b.other = b, a                 # reference cycle
weakref.finalize(a, print, "  cycle freed by gc.collect()")
del a, b                                # refcounts stay at 1 -> not freed yet
found = gc.collect()
```

Output (`gc_weakref.py`):

```
gc thresholds: (2000, 10, 10), gc enabled: True
  solo freed as soon as last reference went away
  (after del solo)
  (after del a, b: cycle still alive)
  cycle freed by gc.collect()
  gc.collect() found 3 unreachable objects
```

The single node disappears exactly at `del`; the cycle survives `del` and is only reclaimed when the collector runs.

Weak references

A **weak reference** points to an object *without* increasing its refcount, so it doesn't keep the object alive. `weakref.ref(obj)()` returns the object or `None` once it has died.

```
r = weakref.ref(obj);  del obj;  r()          # -> None
cache = weakref.WeakValueDictionary()          # entries disappear with their values
```

```
  weakref alive: True
  weakref after del: None
  cache keys while x alive: ['x']
  cache keys after del x: []
```

Uses: caches that shouldn't pin big objects in memory, parent pointers in trees (child → parent weak, avoiding cycles), observer lists. `weakref.finalize(obj, fn)` is the safe way to run clean-up when an object dies (better than `__del__`). Not every type can be weakly referenced (`int`, `list`, `tuple` can't; ordinary class instances can).

Trade-offs

	Reference counting	Cyclic GC
When memory is freed	Immediately	Periodically
Handles cycles	No	Yes
Cost	Small, on every reference change	Pauses proportional to tracked objects
Control	–	`gc.collect()`, `gc.disable()`, `gc.freeze()`, `gc.set_threshold()`

Common mistakes

- Relying on `__del__` for important clean-up (files, locks) – use `with` or `weakref.finalize`.

- Accidental cycles in long-lived caches or through closures/tracebacks (storing an exception object keeps its frames alive).
- Calling `gc.collect()` in a loop "to be safe" — it costs time and doesn't fix leaks caused by references you still hold.

Real-world examples

Instagram disabled, and later **froze**, the cyclic GC in its Django worker processes to stop GC touching pre-forked memory pages and destroying copy-on-write sharing; that work led to `gc.freeze()` (3.7). ORMs like SQLAlchemy use weak references in their identity map so loaded rows can be freed when your code stops using them.

In the interview

> **30-second answer:** "CPython primarily uses reference counting: when an object's count reaches zero it's freed immediately. Reference cycles never reach zero, so a generational cyclic garbage collector periodically finds unreachable groups of container objects. Weak references let you refer to an object without keeping it alive, which is useful for caches and back-pointers."

- *Can Python leak memory?* — Yes: objects still referenced by a global cache, an ever-growing list, or a cycle involving objects the GC can't collect safely.
- *How do you find a leak?* — `tracemalloc` snapshots compared over time; `gc.get_referrers`; tools like `objgraph`.

Self-check

1. Which mechanism frees most objects? → Reference counting.
2. What does the cyclic GC find? → Unreachable reference cycles among container objects.
3. Does a weak reference keep an object alive? → No.
4. Why do generations exist? → Most objects die young; scan young ones often, old ones rarely.

TL;DR

Refcount frees most objects immediately; the generational cyclic GC cleans up cycles; `weakref` refers without owning.

17.3 Small-int caching and string interning

What it is

Two memory optimisations that make some *different-looking* values share one object:

- **Small-int cache**: CPython pre-creates the ints **−5 to 256**; every result in that range reuses those objects.
- **String interning**: keeping one shared copy of a string so equal strings are the same object. CPython automatically interns identifiers (variable, attribute, function names) and many short, identifier-like literal strings in your code; you can force it with `sys.intern(s)`.

```
a, b = int("256"), int("256")           # built at run time, not constants
c, d = int("257"), int("257")
s1 = "".join(["hello", "_", "world"])     # built at run time: not interned automatically
s2 = "hello_world"
```

```
256 is 256: True   257 is 257: False   (small ints -5..256 are cached)
runtime str is literal: False, after sys.intern: True
```

(Numbers are built with `int("...")` because equal constants in the same code block may be merged by the compiler, which would hide the effect.)

Why it matters

- Speed: dict lookups on interned keys can succeed with a pointer comparison.
- Memory: millions of repeated strings (e.g. a "country" column read from a file) can be deduplicated with `sys.intern`.
- Interview trap: `a is b` for ints/strings gives results that depend on the implementation. **Never use `is` for value comparison.**

TL;DR

Ints −5…256 and many identifier-like strings are shared objects. It's an optimisation, not a language rule — compare with `==`.

17.4 Mutability and copying (recap)

Chapters 2 and 5 covered this in detail; here is the memory-level view in one example.

```
grid = [[0, 0], [0, 0]]
alias = grid                    # same object
shallow = grid.copy()           # new outer list, SAME inner lists
deep = copy.deepcopy(grid)      # new everything
grid[0][0] = 99
bad = [[0] * 2] * 2             # two references to ONE inner list
def append_to(item, bucket=[]):  # default evaluated once, shared across calls
```

```
alias [99, 0], shallow [99, 0], deep [0, 0]
[[0]*2]*2 after bad[0][0]=1 -> [[1, 0], [1, 0]]
mutable default after two calls: [1, 2]
```

Operation	New outer object?	New inner objects?
`b = a`	No	No
`a.copy()`, `list(a)`, `a[:]`, `dict(a)`	Yes	No (shallow)
`copy.deepcopy(a)`	Yes	Yes (recursive, handles cycles)

Performance angle: copying is O(n) — shallow copies copy pointers, deep copies copy everything and are much slower. Immutable objects are safe to share (no copies needed), are hashable (usable as dict keys), and are thread-safe to read. Use `[[0] * cols for _ in range(rows)]` for grids and `None` as the default for mutable parameters.

17.5 Big-O of built-in operations

What it is

Big-O describes how an operation's cost grows with the container size *n*: O(1) = constant, O(n) = proportional to n, O(log n) = grows very slowly. "Average" for dict/set assumes a good hash spread; "amortised" means occasional expensive resizes averaged over many cheap calls.

The table (CPython)

Operation	list	dict	set	deque
Index `x[i]` / key lookup `d[k]`	O(1)	O(1) avg	–	O(1) at ends, O(n) in middle
`x in c`	O(n)	O(1) avg	O(1) avg	O(n)
Append / add at right end	O(1) amortised	O(1) avg	O(1) avg	O(1)
Pop from right end	O(1)	`popitem` O(1)	–	O(1)
Insert / pop at left end	O(n) (`insert(0,x)`, `pop(0)`)	–	–	O(1) (`appendleft`, `popleft`)
Insert / delete in middle	O(n)	O(1) avg (`del d[k]`)	O(1) avg (`remove`)	O(n)
`len`	O(1)	O(1)	O(1)	O(1)
Iterate	O(n)	O(n)	O(n)	O(n)
Copy	O(n)	O(n)	O(n)	O(n)
Sort	O(n log n) (Timsort)	–	–	–
Slice `x[a:b]`	O(k) (k = slice length)	–	–	–
`min` / `max` / `sum`	O(n)	O(n)	O(n)	O(n)
Union / intersection	–	`d1 | d2` O(len d1 + len d2)	`s | t` O(len s + len t); `s & t` O(min) avg	–

Others: `heapq.heappush/heappop` O(log n), `heapq.heapify` O(n); `bisect` search O(log n) (but `insort` is O(n) because of the list shift); string concatenation `a + b` O(len a + len b); `str.join` O(total length). Dict/set worst case is O(n) when many keys collide, which is rare with built-in hashes (string hashing is randomised per process to resist deliberate collisions).

Measured

```
setup = f"n={n}; L=list(range(n)); S=set(L); D=deque(L)"
per_op_us("(n-1) in L", setup, 200)               # worst case: last item
per_op_us("(n-1) in S", setup, 200_000)
per_op_us("L.append(L.pop(0))", setup, 2_000)
per_op_us("D.append(D.popleft())", setup, 200_000)
```

Output (`complexity_bench.py`, microseconds per operation):

```
         n            x in list            x in set    list.pop(0)+append  deque.popleft+append
     1,000                5.346               0.022                 0.098                 0.018
    10,000               54.597               0.023                 0.788                 0.017
   100,000              541.614               0.023                13.024                 0.018
 1,000,000             5346.586               0.022               132.574                 0.017
```

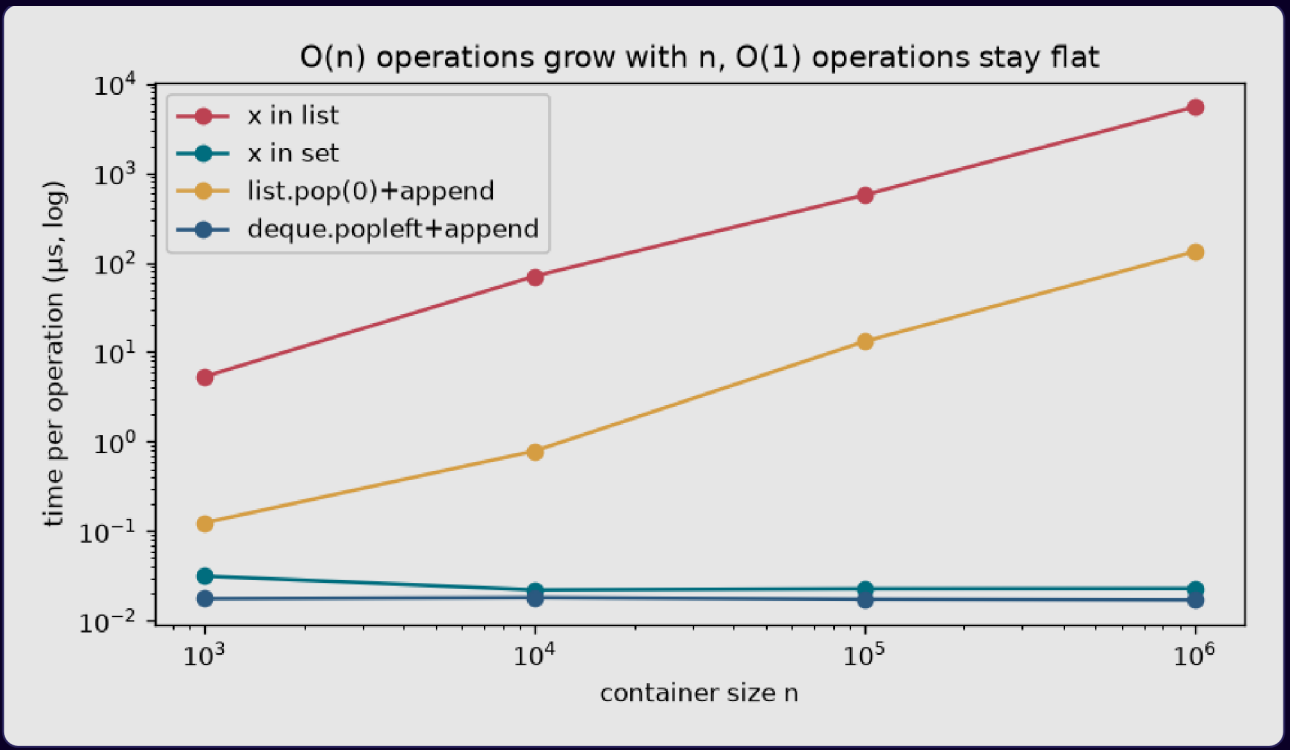


Figure — each 10× larger list makes `in` ~10× slower; set and deque don't care about n. At a million items, set lookup is ~250,000× faster.

Common mistakes

- `if x in some_list` inside a loop → O(n²). Build a `set` once.
- Using a list as a queue (`pop(0)`) → use `collections.deque`.
- Repeated `sorted()` inside a loop when you need a running minimum → `heapq`.
- Forgetting that building a set is O(n): for a *single* lookup, the list is fine.

In the interview

> **30-second answer:** "Lists are dynamic arrays: O(1) index and append, O(n) insert/pop at the front and O(n) membership. Dicts and sets are hash tables with O(1) average lookup, insert and delete. Deque gives O(1) at both ends. The classic fix for slow code is turning repeated list membership into a set lookup."

- *Why is `list.append` "amortised" O(1)?* — The array over-allocates; occasional resizes copy everything, but averaged over many appends each costs O(1).
- *When is dict lookup O(n)?* — Pathological hash collisions.

TL;DR

List: fast at the end, slow at the front and for `in`. Dict/set: O(1) average lookups. Deque: O(1) both ends. Pick by the operations you do most.

17.6 Profiling — measure before you optimise

What it is

Profiling measures where a program spends time or memory. Guessing is usually wrong; the hot spot is often one innocent-looking line.

Tool	Answers	How
`timeit`	"How fast is this small snippet?"	`python -m timeit "sum(range(1000))"` or `timeit.repeat(...)`; in Jupyter `%timeit`
`cProfile` + `pstats`	"Which *functions* take the time?" (deterministic: every call counted)	`python -m cProfile -s cumtime script.py`
`line_profiler` (3rd-party)	"Which *lines* inside a function?"	decorate with `@profile`, run `kernprof -l -v script.py`; Jupyter `%lprun`
`py-spy`, `scalene`, `pyinstrument` (3rd-party)	Sampling profilers: low overhead, work on running processes; scalene also profiles memory/CPU vs native time	`py-spy top --pid 1234`
`tracemalloc`	"Which lines allocate memory?"	snapshots and `statistics("lineno")`
`time.perf_counter()`	Quick wall-clock timing of a block	subtract two readings

Python 3.15 (release candidates since August 2026; final due October 2026) adds a statistical sampling profiler to the standard library, `profiling.sampling` (PEP 799); on 3.14 and earlier, use the tools above.

How it works – a real hunt

```python
def is_vip(customer_id: int, vip_ids: list[int]) -> bool:
    return customer_id in vip_ids            # O(len(vip_ids)) per call!

def build_report(n: int = 20_000) -> dict:
    records = load_records(n)
    vip_ids = list(range(0, n, 7))           # a list, not a set
    ...
prof = cProfile.Profile(); prof.enable(); build_report(); prof.disable()
pstats.Stats(prof).strip_dirs().sort_stats("cumulative").print_stats(4)
```

Output (`profiling_demo.py`):

```
timeit sum(range(1000)): 3.79 µs per call
         22861 function calls in 0.287 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.003    0.003    0.287    0.287 profiling_demo.py:19(build_report)
    20000    0.281    0.000    0.281    0.000 profiling_demo.py:15(is_vip)
        1    0.003    0.003    0.003    0.003 profiling_demo.py:10(load_records)
     2858    0.000    0.000    0.000    0.000 {method 'get' of 'dict' objects}
list lookup: 278 ms  set lookup: 2.8 ms  speed-up x101  same result: True
records: 50,000, current 11.2 MB, peak 11.2 MB
top allocation site: line 11, 9.6 MB
```

How to read the columns: **ncalls** = number of calls; **tottime** = time inside the function itself; **cumtime** = including everything it called; **percall** = per-call averages. Here `is_vip` has 98 % of the time → change `vip_ids` to a `set` → ~100× faster. `tracemalloc` points at line 11 (the list of dicts) as the memory hog.

Steps that work every time:

1. Reproduce with realistic input size.
2. Profile the whole run (`cProfile`, or `py-spy` on a live server).
3. Sort by `cumtime` to find the expensive branch, then `tottime` to find the leaf.
4. Fix the **algorithm/data structure** first; micro-optimise last.
5. Re-measure with `timeit` and check the result is unchanged.

Common mistakes

- Timing with a single run (noise) – use `timeit.repeat` and take the minimum.
- Profiling tiny inputs where start-up dominates.

- Forgetting that `cProfile` adds overhead to every call, so many tiny functions look worse than they are; sampling profilers avoid this.

TL;DR

`timeit` for snippets, `cProfile` for functions, `line_profiler` for lines, `py-spy` / `scalene` for live/low-overhead, `tracemalloc` for memory. Measure → fix the biggest thing → re-measure.

17.7 Optimisation techniques

Measured on 3.14

```
def concat_plus():              s = ""; for i in range(N): s += str(i)
def concat_join():              return "".join(str(i) for i in range(N))
def global_lookup(): ... total += math.sqrt(i)          # attribute lookup each time
def local_lookup():  sqrt = math.sqrt; ... total += sqrt(i)
def numpy_sum_squares():        a = np.arange(N, dtype=np.int64); return int((a * a).sum())
@lru_cache(maxsize=None)
def fib_cached(n): return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)
```

Output (`optimizations.py`, N = 100,000):

```
string += vs join            6.33 ms ->    5.11 ms   x  1.2
for+append vs listcomp       2.45 ms ->    2.15 ms   x  1.1
math.sqrt vs local sqrt      3.27 ms ->    2.83 ms   x  1.2
python sum vs numpy          2.70 ms ->    0.04 ms   x 61.1
fib(25) vs lru_cache         5.85 ms ->   0.002 ms   x 2419   CacheInfo(hits=23, misses=26, maxsize=None,
currsize=26)
sum over 1M: list peak 40.4 MB vs generator peak 0.4 KB
100k objects: regular class 12.8 MB vs __slots__ 8.8 MB
__slots__ blocks new attributes: 'PointSlots' object has no attribute 'z' and no __dict__ for setting new
attributes
```

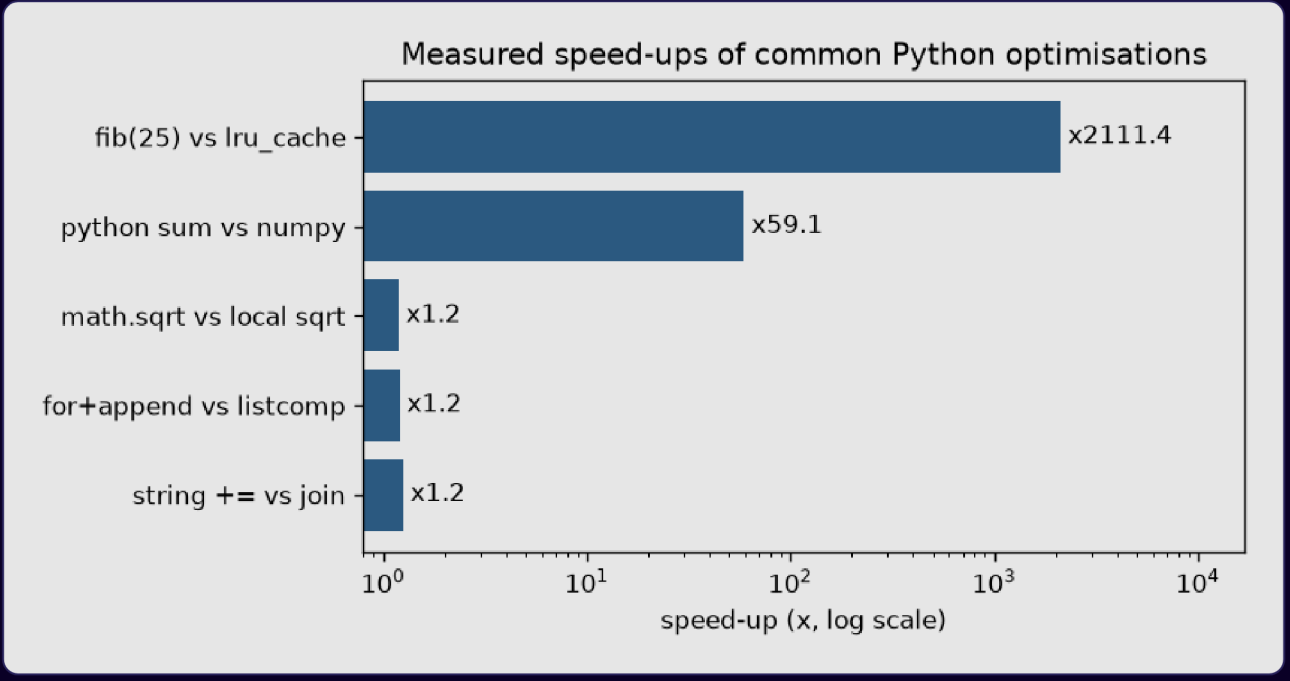


Figure — algorithmic and vectorisation changes win by orders of magnitude; micro-tweaks give 10–20 %.

The techniques, ranked by typical pay-off

#	Technique	Why it helps	Typical gain
1	**Better algorithm / data structure** (set for membership, dict for lookup, deque for queues, heapq for top-k)	Changes the big-O	10×–100,000×
2	**Caching** (`functools.lru_cache` / `cache` , memoisation dicts)	Don't recompute the same answer	Up to exponential → linear
3	**Vectorise with numpy/pandas**	One C loop over raw numbers instead of a Python loop over objects	10×–100×
4	**Generators / streaming**	Constant memory, earlier results	Memory: GB → KB
5	**Built-ins and comprehensions** (`sum` , `max` , `sorted` , `"".join` , list comprehensions)	Loops run in C / specialised bytecode	1.1×–3×
6	**Local variable lookups** (alias `math.sqrt` before a hot loop)	Local access is an array index; globals/attributes are dict lookups	~1.1× on 3.11+
7	`__slots__` on classes with millions of instances	No per-instance `__dict__`	Less memory per object (31 % less total here)
8	**Parallelism** (Ch 16)	Use more cores	≤ number of cores

Details worth knowing:

- `str.join` : repeated `s += piece` can be O(n²) in general, because strings are immutable and each `+=` may copy. CPython has a special in-place optimisation when the string has only one reference, which is why the gap above is small – but it's an implementation detail (not on PyPy, and it breaks as soon as another reference exists). `"".join(parts)` is always linear. Same idea for bytes: use `bytearray` or `io.BytesIO` .
- `lru_cache` : arguments must be hashable; `maxsize=None` (or `@functools.cache`) means unbounded – a memory leak risk for many distinct inputs. It caches per function, not per instance; on methods it keeps `self` alive (use `functools.cached_property` for per-instance values).
- **Local lookups**: the 3.11+ specialising interpreter already caches global and attribute lookups, so this trick gives little today. Apply it only in measured hot loops.
- **Avoid needless work in loops**: hoist invariant computations, avoid repeated `len()` in conditions you can compute once, avoid exceptions for normal control flow in very hot paths (though `try` itself is zero-cost in 3.11+ when nothing is raised).

`__slots__`

Normally each instance stores its attributes in a per-instance dict. `__slots__ = ("x", "y")` replaces it with fixed slots:

```
class PointSlots:
    __slots__ = ("x", "y")              # fixed attributes, no per-instance __dict__
    def __init__(self, x, y):
        self.x, self.y = x, y
```

Measured: 100,000 points took 12.8 MB vs 8.8 MB (the total includes the list and the int objects, so the per-object saving is larger than the ratio suggests). Also slightly faster attribute access. Costs: you can't add new attributes, no `__dict__` (breaks some tools that expect it), no weak references unless you add `"__weakref__"` to the slots, and each class in an inheritance chain needs its own `__slots__` to get the benefit. `@dataclass(slots=True)` (3.10+) generates them for you. Since 3.11–3.13, CPython also stores ordinary instance attributes more compactly ("inline values"), so the gap is smaller than in old blog posts – measure.

When to drop to C, Cython or Rust

When profiling shows a hot loop that can't be expressed with numpy/pandas and runs too slow in Python:

Option	What it is	Good for
numba	JIT-compiles numeric Python functions (`@njit`) to machine code	Loops over numpy arrays
Cython	Python-like language with C types, compiled to a C extension	Gradual speed-up of existing Python; wrapping C libraries
Rust + PyO3/maturin	Write the extension in Rust	Safe, fast new libraries (pydantic-core, polars, ruff, tokenizers are written this way)
C/C++ + pybind11 / C API	Classic native extension	Existing C/C++ code
PyPy	Alternative Python with a tracing JIT	Pure-Python long-running programs (check C-extension support)
mypyc	Compiles type-annotated Python to C extensions	Typed codebases (mypy and black use it)

Rule: exhaust algorithmic fixes and vectorisation first; native code adds a build step and a second language to maintain.

Python version performance (Faster CPython)

Version	What changed (from the official "What's New" notes)
3.11 (2022)	Specialising adaptive interpreter (PEP 659): bytecode rewrites itself for the types it sees. Cheaper frames, inlined Python-to-Python calls, "zero-cost" exceptions. Official figure: **10–60 % faster than 3.10, ~1.25× on average** on the pyperformance suite.
3.12 (2023)	Inlined comprehensions (PEP 709, up to 2× faster for comprehension-heavy code), immortal objects (PEP 683), per-interpreter GIL (PEP 684).
3.13 (2024)	Experimental JIT compiler (PEP 744, off by default), experimental free-threaded build (PEP 703), improved interactive REPL.
3.14 (2025)	Free-threaded build officially supported (PEP 779); optional tail-calling interpreter (~3–5 % faster when built with recent Clang); experimental JIT included in the Windows and macOS installers, still off by default.

Simply upgrading from 3.10 to 3.11+ is often the cheapest speed-up you'll ever get.

When not to optimise

Code that runs once, code that is already I/O-bound (a 1 ms CPU saving next to a 200 ms network call), and anything not shown hot by a profiler. Readability usually wins.

Common mistakes

- Micro-optimising before fixing an $O(n^2)$ algorithm.
- Vectorising by calling numpy element-by-element in a Python loop (slower than plain Python!).
- Unbounded caches that grow forever in long-running services.
- Premature `__slots__` on classes with 50 instances.

Real-world examples

pandas and scikit-learn are fast because their inner loops are C/Cython; polars and pydantic v2 got large speed-ups by moving their cores to Rust; Django and many web frameworks use `functools.lru_cache` / `cached_property` for settings and URL resolution; mypyc was built by the mypy team (originally at Dropbox) to compile mypy itself, making it several times faster.

In the interview

> **30-second answer:** "First I profile – cProfile or py-spy – to find the hot spot. The biggest wins are algorithmic: the right data structure, avoiding repeated work with caching, and vectorising numeric loops with numpy. Then smaller things: built-ins, comprehensions, `join` instead of `+=`, generators for memory, `__slots__` for many small objects. If that's still not enough, I'd move the hot loop to numba, Cython or Rust, or parallelise."

- *Why is numpy faster than a Python loop?* – Contiguous typed memory, one C loop, no per-element object creation or dispatch, often SIMD.
- *What's the downside of `lru_cache`?* – Memory growth and stale results; arguments must be hashable.

Self-check

1. Biggest source of speed-ups? → Better algorithms/data structures.
2. Why is `''.join` preferred over `+=`? → Guaranteed linear; `+=` can be quadratic.
3. What does `__slots__` remove? → The per-instance `__dict__`.
4. Name two ways to speed up a numeric loop numpy can't express. → numba, Cython (or Rust/PyO3).
5. Roughly how much faster was 3.11 than 3.10? → ~25 % on average (10–60 %).

TL;DR

Measure → fix the algorithm → cache → vectorise → stream → micro-tweak → native code/parallelism. Upgrade Python.

17.8 Chapter self-check

1. What is in every `PyObject` header? → Refcount and type pointer.
2. When is an object freed in CPython? → When its refcount reaches zero, or when the cyclic GC finds it unreachable in a cycle.
3. Which ints are cached? → −5 to 256.
4. What does `weakref.WeakValueDictionary` do when a value dies? → Removes the entry.
5. Cost of `x in list` vs `x in set`? → O(n) vs O(1) average.
6. Best structure for a FIFO queue? → `collections.deque`.
7. `tottime` vs `cumtime`? → Own time vs including callees.
8. Why does `sum(i*i for i in range(10**6))` use almost no memory? → Generator streams values.
9. Downside of `__slots__`? → No new attributes, no `__dict__`, no weakrefs unless listed.
10. First thing to try before rewriting in Rust? → Profile and fix the algorithm / vectorise.

Chapter TL;DR

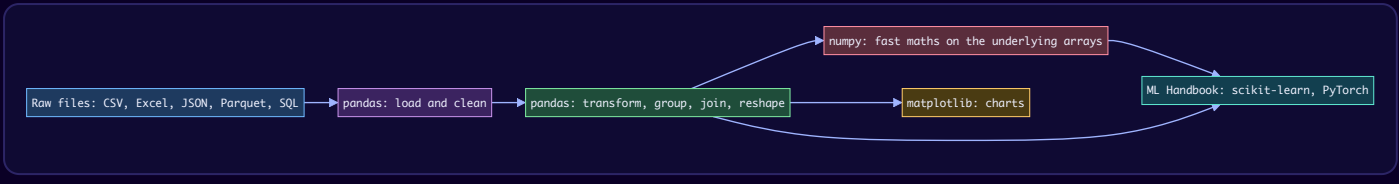
- **Objects**: identity (`id`, `is`), type, value; `PyObject` = refcount + type pointer; names are references; immortal objects since 3.12.
- **Memory**: refcounting frees immediately; generational cyclic GC handles cycles; `weakref` refers without owning; `gc.freeze()` helps pre-fork servers.
- **Caching quirks**: small ints −5…256 and interned strings are shared – always compare with `==`.
- **Copying**: assignment shares, `.copy()` is shallow, `deepcopy` is deep; immutables are safe to share.
- **Big-O**: list O(1) end / O(n) front and `in`; dict/set O(1) average; deque O(1) both ends; heapq O(log n).
- **Profile**: `timeit`, `cProfile` / `pstats`, `line_profiler`, `py-spy` / `scalene`, `tracemalloc`.
- **Optimise**: data structure → caching → numpy → generators → built-ins/join/locals → `__slots__` → numba/Cython/Rust → parallelism. Newer CPython (3.11+) is substantially faster for free.
- **OS layer** (processes, threads, paging, syscalls, fds, epoll, page cache): see Java handbook §16.13.

Chapter 18 — The Data Stack: numpy, pandas, matplotlib

What you will learn. The three libraries almost every Python data, science and ML project stands on. **numpy**: fast n-dimensional arrays, dtypes, indexing and views, broadcasting, vectorisation, aggregation along axes, basic linear algebra and the modern random API. **pandas**: `Series` and `DataFrame`, reading and writing files, `loc` / `iloc`, filtering, `groupby` / `agg` / `transform`, `merge` / `concat`, `pivot_table` / `melt`, missing data, datetimes, why `apply` is slow, and performance tips — including what changed in **pandas 3.0** (string dtype, Copy-on-Write). **matplotlib**: Figure and Axes, the four workhorse charts, subplots, labels and saving images. The chapter ends with a small end-to-end analysis: messy CSV → clean → analyse → chart.

Where this leads. The ML Handbook, Chapter 3 — The Python ML stack, takes these tools into machine learning (feature matrices, scikit-learn pipelines, train/test splits). This chapter stays with general data work and does not cover scikit-learn.

Code. `chapters/18-data-stack/code/` — `numpy_basics.py`, `pandas_basics.py`, `matplotlib_demo.py`, `end_to_end.py`, and `plot_style.py` (a shared chart style). Plots are saved to `img/`. Versions used: Python 3.14.7, numpy 2.5, pandas 3.0.6, matplotlib 3.11.



Library	Core object	Think of it as
numpy	`ndarray`	A grid of numbers of one type, stored in one block of memory
pandas	`DataFrame` / `Series`	A spreadsheet table with labelled rows and columns; columns can have different types
matplotlib	`Figure` / `Axes`	A canvas with one or more plots drawn on it

18.1 numpy arrays

What it is

numpy (Numerical Python) gives you the `ndarray`: an n-dimensional array whose elements all share one **dtype** (data type such as `int64`, `float64`, `bool`) and sit side by side in one contiguous block of memory. Its **shape** is a tuple of sizes per dimension (`(3, 4)` = 3 rows, 4 columns); `ndim` is the number of dimensions.

Analogy: a Python list is a shelf of labelled boxes, each box holding any kind of object somewhere else in the warehouse; a numpy array is an egg carton — every slot the same size, all in one piece, so a machine can process a whole carton at once.

Why it matters

- **Speed**: operations run as compiled C loops over raw numbers — typically 10–100× faster than Python loops (measured below: 35×).
- **Memory**: 8 bytes per `float64` versus ~24–32 bytes per Python float object plus an 8-byte pointer.
- **Ecosystem**: pandas columns, scikit-learn inputs, images in OpenCV/Pillow, and PyTorch tensors (which convert to and from numpy) all build on this idea.

How it works — creating and inspecting

```
a = np.array([1, 2, 3])                       # from a list
np.zeros((2, 3)); np.ones(3, dtype=np.int32)  # filled arrays
np.arange(0, 10, 3); np.linspace(0, 1, 5)     # ranges: step-based vs count-based
m = np.arange(12).reshape(3, 4)               # 3 rows x 4 cols
np.array([1, 2.5]).dtype                      # mixed -> upcast to float64
np.array([1.7, -1.7]).astype(int)             # truncates toward zero
```

Output (`numpy_basics.py`):

```
[1 2 3] int64 (3,) 1
[[0. 0. 0.]
 [0. 0. 0.]]
[1 1 1]
[0 3 6 9]
[0.   0.25 0.5  0.75 1.  ]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
m.shape=(3, 4) m.size=12 m.dtype=int64 m.nbytes=96 itemsize=8
mixed ints and floats -> float64; astype -> [ 1 -1]
```

`reshape` needs the same total size (12 = 3 × 4); pass `-1` for one dimension to let numpy compute it (`reshape(-1, 2)`). Other creators: `np.full` , `np.eye` (identity), `np.empty` (uninitialised — fast, contains garbage), `np.zeros_like(m)` .

Indexing, slicing, views and masks

```
m[1, 2]; m[0]; m[:, 1]; m[-1, ::2]            # element, row, column, strided
view = m[:2, :2]; view[0, 0] = 100            # slicing returns a VIEW -> m changes too
mask = m % 2 == 0; m[mask]                    # boolean mask selects matching elements
m[[2, 0]]                                     # fancy (list) indexing returns a COPY
np.where(m > 5, m, -1)                        # vectorised if/else
```

```
m[1, 2]=6  m[0]=[0 1 2 3]  m[:, 1]=[1 5 9]  m[-1, ::2]=[ 8 10]
after view[0,0]=100, m[0,0]=100  shares memory: True
even values: [ 0  2  4  6  8 10]
fancy rows [2, 0]:
[[ 8  9 10 11]
 [ 0  1  2  3]]
np.where(m > 5, m, -1):
[[-1 -1 -1 -1]
 [-1 -1  6  7]
 [ 8  9 10 11]]
```

Indexing kind	Example	Returns
Basic (ints, slices)	`m[1:, ::2]`	**View** (shares memory)
Boolean mask	`m[m > 5]`	Copy (1-D)
Fancy (int lists/arrays)	`m[[2, 0]]`	Copy

Unlike Python lists, a numpy slice is a **view**: changing it changes the original. Call `.copy()` when you need independence. Combine masks with `&` , `|` , `~` and parentheses: `m[(m > 2) & (m < 9)]` (`and` / `or` raise an error on arrays).

Vectorised maths and aggregation along an axis

```
x*2=[2. 4. 6. 8.]  x**2=[ 1.  4.  9. 16.]  np.sqrt(x)=[1.    1.414 1.732 2.   ]  x>2=[False False  True  True]
m.sum()=66  m.sum(axis=0)=[12 15 18 21]  m.sum(axis=1)=[ 6 22 38]
m.mean()=5.50  m.max(axis=1)=[ 3  7 11]  m.argmax()=11  cumsum=[ 1  3  6 10]
```

Arithmetic and **ufuncs** (universal functions: `np.sqrt`, `np.exp`, `np.log`, `np.abs` …) apply element-wise. For reductions, `axis=0` means "collapse the rows" – one result **per column**; `axis=1` collapses the columns – one result **per row**. `argmax` gives the position of the maximum (in the flattened array unless you pass `axis`).

Vectorisation, measured

```
[v * 2 + 1 for v in big]      # Python: one bytecode loop, one int object per element
arr * 2 + 1                   # numpy: two C loops over raw int64s
```

```
1M elements: list comprehension 25.8 ms, numpy 0.75 ms, x35
```

Vectorisation means expressing a computation as whole-array operations instead of a Python loop. The rule of thumb: if you are writing `for i in range(len(arr))` over a numpy array, look for the array operation (`np.where`, masks, `cumsum`, `np.diff`, `np.clip`, broadcasting).

Other everyday array functions: **join** with `np.concatenate([a, b])` (along an existing axis) or `np.stack([a, b])` / `np.vstack` / `np.hstack` (new or row/column axis); **sort** with `np.sort(a)` (returns a copy) or `a.sort()` (in place), and `np.argsort(a)` for the order of indices (e.g. to sort one array by another); **deduplicate/count** with `np.unique(a, return_counts=True)`; **reshape views** with `a.T`, `a.ravel()`, `a[:, None]`.

Common mistakes

- Expecting a slice to be a copy (it's a view) – or the reverse with fancy indexing.
- Integer overflow: fixed-size dtypes wrap around (`np.int8(127) + np.int8(1)` → −128 – numpy warns for scalars, but array arithmetic wraps silently); Python ints never overflow.
- Comparing floats with `==` – use `np.isclose` / `np.allclose`.
- Building an array by `np.append` in a loop (copies each time, $O(n^2)$) – collect in a list, convert once.
- Mixing up `axis=0` and `axis=1`.

TL;DR

One dtype, one memory block, shape tuple. Slices are views; masks and fancy indexing copy. Whole-array operations replace loops and run tens of times faster.

18.2 Broadcasting

What it is

Broadcasting is numpy's rule for combining arrays of different shapes without copying data: the smaller array is (virtually) stretched to match the larger one.

How it works – the rule

Compare the two shapes **from the right-most dimension leftwards**:

1. If one shape is shorter, pad it on the left with 1s.
2. Two dimensions are compatible if they are **equal** or **one of them is 1**.
3. A size-1 dimension is stretched to the other size. Any other mismatch → `ValueError`.

Worked example: `(3, 1) + (4,)` → pad → `(3, 1)` vs `(1, 4)` → result `(3, 4)`.

```
col = np.array([[10], [20], [30]])                # shape (3, 1)
row = np.array([1, 2, 3, 4])                      # shape (4,)
col + row                                         # shape (3, 4)
z = (data - data.mean(axis=0)) / data.std(axis=0)  # standardise each column: (3,2) op (2,)
np.ones((3, 2)) + np.ones(3)                      # (3,2) vs (3,) -> last dims 2 vs 3 -> error
```

```
(3,1) + (4,) -> (3, 4)
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
column-standardised:
[[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
(3,2) + (3,) -> ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

To subtract a per-row value, make it a column: `data - data.mean(axis=1, keepdims=True)` (or `[:, None]` / `np.newaxis` to add a dimension).

Real-world examples

Standardising features (subtract column means, divide by column stds), normalising images (`img / 255.0`, or subtracting a per-channel mean of shape `(3,)` from an `(H, W, 3)` image), pairwise distances (`a[:, None, :] - b[None, :, :]`).

Self-check

1. Shape of `(5, 1, 3) + (4, 1)`? → `(5, 4, 3)`.
2. Why does `(3, 2) + (3,)` fail? → Right-most dims 2 and 3 differ and neither is 1.
3. How do you subtract each row's mean? → `x - x.mean(axis=1, keepdims=True)`.

TL;DR

Align shapes from the right; equal or 1 is fine; size 1 stretches. `keepdims=True` or `[:, None]` fixes most shape errors.

18.3 Linear algebra and random numbers

Linear algebra basics

```
A = np.array([[3.0, 1.0], [1.0, 2.0]]); b = np.array([9.0, 8.0])
A @ A                       # matrix multiplication (same as np.matmul); A * A is element-wise!
np.linalg.solve(A, b)       # solve A x = b
np.linalg.det(A); np.linalg.inv(A); np.linalg.eigh(A); np.linalg.norm(b); A.T
```

```
A @ A =
[[10.  5.]
 [ 5.  5.]]
solve A x = b -> x = [2. 3.], check A @ x = [9. 8.]
det = 5.000, inv =
[[ 0.4 -0.2]
 [-0.2  0.6]]
eigenvalues = [1.382 3.618], norm of b = 12.042, A.T =
[[3. 1.]
 [1. 2.]]
```

Check by hand: 3·2 + 1·3 = 9 and 1·2 + 2·3 = 8. ✓ det = 3·2 − 1·1 = 5. ✓ The norm is √(9² + 8²) = √145 ≈ 12.042. ✓

Need	Use	Note
Matrix product	`A @ B`	`*` is element-wise
Solve `A x = b`	`np.linalg.solve`	Faster and more accurate than `inv(A) @ b`
Least squares	`np.linalg.lstsq`	Over-determined systems (more equations than unknowns)
Eigen-decomposition	`np.linalg.eigh` (symmetric), `eig` (general)	With numpy 2.5 here, `eig` returned a complex dtype even for this real-valued case; `eigh` returns real values sorted ascending
SVD	`np.linalg.svd`	Basis of PCA
Dot product / length	`a @ b`, `np.linalg.norm(a)`	

The maths behind these (vectors, matrices, eigenvalues) is covered in the ML Handbook's maths chapter.

Random numbers

```
rng = np.random.default_rng(seed=42)        # modern Generator API
rng.integers(0, 10, size=5); rng.normal(0, 1, size=3); rng.choice(["a", "b", "c"], size=4)
rng.shuffle(arr5)                           # in place
```

```
integers: [0 7 6 4 4]
normal:   [ 0.941 -1.951 -1.302]
choice:   ['c' 'c' 'c' 'c']
shuffle:  [1 0 2 4 3]
same seed, same numbers: [0 7 6 4 4]
```

Use `np.random.default_rng(seed)` and pass the `rng` object around. The old global functions (`np.random.seed`, `np.random.rand`) still work but share hidden global state, which makes reproducibility across modules fragile. `rng.integers(low, high)` excludes `high`. Other useful methods: `random`, `uniform`, `permutation`, `binomial`, `poisson`. (Four 'c's in a row is just chance — with this seed.)

In the interview

30-second answer: "A numpy array is a typed, contiguous block of memory with a shape, so operations run as C loops — vectorised code is often tens of times faster than Python loops. Slices are views, fancy indexing copies. Broadcasting lines up shapes from the right, and a dimension of 1 stretches. I use `@` for matrix multiply, `linalg.solve` rather than inverting, and `default_rng(seed)` for reproducible randomness."

- *Why is numpy fast?* — Contiguous typed memory, compiled loops, no per-element Python objects, SIMD, and BLAS/LAPACK for linear algebra.
- *`a * b` vs `a @ b`?* — Element-wise vs matrix product.
- *How do you make results reproducible?* — Seeded `Generator` passed explicitly.

TL;DR

`@` for matrices, `solve` not `inv`, `eigh` for symmetric matrices; `default_rng(seed)` for randomness.

18.4 pandas: Series and DataFrame

What it is

- A **Series** is a one-dimensional labelled array: values plus an **index** (the row labels).
- A **DataFrame** is a table: an ordered collection of Series (columns) sharing one index. Each column has its own dtype.

Analogy: a DataFrame is a spreadsheet in which each column is typed and every row has a name tag.

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="score")
s["b"]; s.iloc[0]; s[s > 15]; s.mean()
```

```
a    10
b    20
c    30
Name: score, dtype: int64
s['b']=20  s.iloc[0]=10  s[s > 15].tolist()=[20, 30]  s.mean()=20.0
```

Operations between Series **align on the index**: `s1 + s2` adds values with matching labels; labels present in only one become `NaN`. This automatic alignment is pandas' biggest difference from numpy.

Reading data and first look

```
df = pd.read_csv(io.StringIO(CSV), parse_dates=["date"])   # usually pd.read_csv("orders.csv", ...)
df.head(3); df.dtypes; df.shape; df.describe()
```

```
   order_id       date customer   city   product   qty  price
0         1 2026-01-03      ana   Pune       pen  10.0    1.5
1         2 2026-01-03      ben  Delhi  notebook   2.0    4.0
2         3 2026-01-10      ana   Pune  notebook   NaN    4.0
order_id             int64
date        datetime64[us]
customer               str
city                   str
product                str
qty                float64
price              float64
shape=(8, 7)
       order_id                 date        qty      price
count       8.0                    8   7.000000   7.000000
mean        4.5  2026-02-03 00:00:00   6.142857   5.928571
min         1.0  2026-01-03 00:00:00   1.000000   1.500000
max         8.0  2026-03-05 00:00:00  20.000000  25.000000
```

What to notice:

- `qty` became `float64` because it has a missing value: `NaN` (Not a Number, numpy's missing marker) is a float. Use the nullable dtype `"Int64"` (capital I) to keep integers with missing values (shown as `<NA>`).
- Text columns show dtype `str` – new in pandas 3.0. Earlier versions used the generic `object` dtype. The `str` dtype holds only strings (and missing values); it is backed by PyArrow when installed, otherwise by Python objects.
- Dates parsed from strings default to **microsecond** resolution (`datetime64[us]`) in pandas 3.0 (earlier: nanoseconds).
- First-look checklist: `head()`, `info()` (dtypes, non-null counts, memory), `describe()` (count/mean/std/min/quartiles/max), `value_counts()` on categorical columns, `isna().sum()`.

Input and output

Format	Read	Write	Notes
CSV	`pd.read_csv(path, usecols=, dtype=, parse_dates=, chunksize=)`	`df.to_csv(path, index=False)`	Universal, slow, loses dtypes
Excel	`pd.read_excel`	`df.to_excel`	Needs `openpyxl`
JSON	`pd.read_json(lines=True)`	`df.to_json(orient="records")`	APIs, logs
Parquet	`pd.read_parquet`	`df.to_parquet`	Columnar, compressed, keeps dtypes, fast; needs `pyarrow`. Preferred for anything big.
SQL	`pd.read_sql(query, conn)`	`df.to_sql(name, conn)`	Via SQLAlchemy or `sqlite3`
Clipboard / HTML	`read_clipboard`, `read_html`		Quick grabs

`chunksize=100_000` in `read_csv` returns an iterator of DataFrames — process files larger than memory piece by piece.

TL;DR

Series = labelled 1-D; DataFrame = table of Series. Operations align on labels. pandas 3: text columns are `str`, dates default to microseconds. Parquet for storage, CSV for exchange.

18.5 Selecting data: `loc`, `iloc` and filters

What it is

Accessor	Selects by	Slice end	Example
`df["col"]` / `df[["a", "b"]]`	Column name(s)	–	one Series / a DataFrame
`df.loc[rows, cols]`	**Labels** (index values, column names) or boolean masks	**Inclusive**	`df.loc[1:2, ["customer", "qty"]]` → rows labelled 1 **and** 2
`df.iloc[rows, cols]`	**Integer positions**	Exclusive (like Python)	`df.iloc[0, 2]` → first row, third column
`df.at[r, c]` / `df.iat[i, j]`	One cell, fast	–	

```
df.loc[0, "customer"]; df.iloc[0, 2]; df.loc[1:2, ["customer", "qty"]]
df[(df["city"] == "Pune") & (df["product"] == "pen")]       # boolean mask, parentheses required
df.query("qty >= 5 and price < 2")                          # same idea as a string expression
df[df["product"].isin(["bag", "notebook"])]
```

```
ana | ana | {'customer': ['ben', 'ana'], 'qty': [2.0, nan]}
{'order_id': [1, 8], 'qty': [10.0, 20.0]}
[1, 4, 8]
[2, 3, 5, 6, 7]
```

Other handy filters: `.between(a, b)`, `.str.contains("x")`, `.str.startswith`, `.isna()` / `.notna()`, `~mask` for NOT.

Sorting and ranking: `df.sort_values("revenue", ascending=False)` (or a list of columns), `df.sort_index()`, `df.nlargest(3, "revenue")` for a quick top-N, and `df["city"].value_counts()` (frequency of each value, most

common first; `normalize=True` for shares).

Setting values and Copy-on-Write

Always set with **one** `.loc` call:

```
df.loc[df["revenue"] >= 20, "tier"] = "big"      # correct
# df["tier"][df["revenue"] >= 20] = "big"       # chained assignment: never modifies df in pandas 3
```

In **pandas 3.0, Copy-on-Write (CoW)** is always on: any result of indexing *behaves as a copy*. Changing it never changes the original DataFrame, and "chained assignment" (`df[a][b] = v`) does nothing to `df` (pandas warns). The old `SettingWithCopyWarning` is gone. Internally pandas still shares memory until one side is modified, so it stays fast.

```
sub = df[df["city"] == "Pune"]
sub["qty"] = 0                                  # modifies only `sub`, never df
```

```
df Pune qty still: [10, 0, 2, 20]
```

(The 0 is the order whose missing quantity was filled with 0 earlier.)

Common mistakes

- Using `and` / `or` instead of `&` / `|` → `ValueError: The truth value of a Series is ambiguous`.
- Forgetting parentheses around each condition (`&` binds tighter than `==`).
- Assuming `loc` slices exclude the end label.
- Relying on chained assignment (silently ineffective in pandas 3).

TL;DR

`loc` = labels (inclusive), `iloc` = positions (exclusive). Filter with masks joined by `&` / `|`. Assign with a single `.loc[mask, col] = value`.

18.6 Cleaning: missing data and new columns

Missing data

pandas marks missing values as `NaN` (floats), `NaT` (datetimes) or `pd.NA` (nullable dtypes); `None` in input becomes one of these.

Task	Method
Find	`df.isna()`, `df.isna().sum()`, `df.info()`
Drop	`df.dropna()` (any NaN in row), `dropna(subset=["qty"])`, `dropna(thresh=5)`
Fill with constant	`df["qty"].fillna(0)`
Fill with statistic	`fillna(df["price"].median())` or per group with `transform` (below)
Fill from neighbours	`ffill()`, `bfill()`, `interpolate()` (time series)
Replace sentinel values	`df.replace(-1, np.nan)`

```
df["qty"] = df["qty"].fillna(0).astype(int)
df["price"] = df["price"].fillna(df.groupby("product")["price"].transform("median"))
```

```
{'qty': 1, 'price': 1}
after fill: nulls=0, order 6 (missing price, bag) filled with bag median=25.0
```

The missing bag price was filled with the median price of *bags*, not of all products – group-aware filling is usually more sensible. Choosing between dropping and filling is a modelling decision: drop when few rows are affected at random; fill (and possibly add an "was_missing" flag column) when missingness is common or informative. Aggregations skip `NaN` by default (`sum`, `mean`), and `NaN != NaN`, so always test with `isna()`.

New columns, vectorised

```
df = df.assign(revenue=df["qty"] * df["price"], month=df["date"].dt.to_period("M"))
df.loc[df["revenue"] >= 20, "tier"] = "big"
df["tier"] = df["tier"].fillna("small")
```

```
   order_id  revenue    month   tier
0         1     15.0  2026-01  small
1         2      8.0  2026-01  small
2         3      0.0  2026-01  small
3         4      7.5  2026-02  small
```

`assign` returns a new DataFrame, so it chains nicely. For several conditions use `np.select([cond1, cond2], [val1, val2], default=...)`; for bins use `pd.cut` (fixed edges) or `pd.qcut` (quantiles). Other cleaning tools: `astype`, `pd.to_numeric(errors="coerce")` (bad strings → NaN), `.str.strip().str.lower()`, `rename(columns=...)`, `drop_duplicates()`, `drop(columns=...)`.

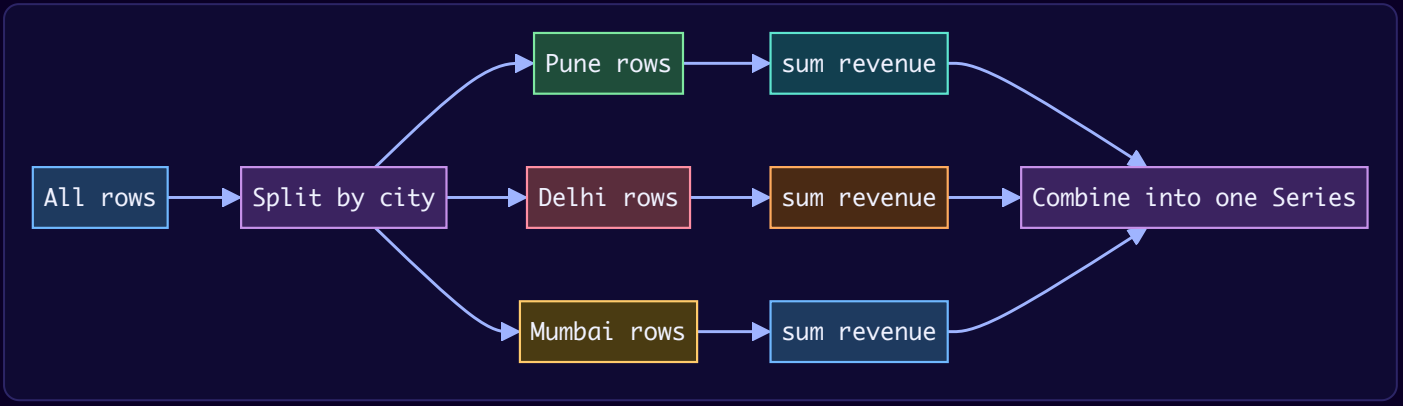
TL;DR

Find with `isna().sum()`, then drop or fill thoughtfully (group medians, forward-fill for time series). Build columns with vectorised expressions and `.loc`; never loop row by row.

18.7 `groupby`: split-apply-combine

What it is

`groupby` implements **split-apply-combine**: split rows into groups by key(s), apply a function to each group, combine the results. It's SQL's `GROUP BY`.



How it works

```
df.groupby("city")["revenue"].sum().sort_values(ascending=False)
df.groupby("customer").agg(orders=("order_id", "count"),          # named aggregation
                           revenue=("revenue", "sum"),
                           avg_qty=("qty", "mean"))
df["share_of_city"] = df["revenue"] / df.groupby("city")["revenue"].transform("sum")
```

```
city
Pune      95.0
Delhi     33.0
Mumbai    19.5
Name: revenue, dtype: float64
          orders  revenue  avg_qty
customer
ana            3     45.0     10.0
ben            2     33.0      1.5
cara           2     19.5      4.0
dev            1     50.0      2.0
   order_id   city  share_of_city
0         1   Pune           0.16
1         2  Delhi           0.24
2         3   Pune           0.00
```

Method	Output shape	Use
`agg` / `sum` / `mean` / `size` / `count`	One row per group	Summaries
`transform`	Same length as the input (group result broadcast back to each row)	Group-relative features: share of total, deviation from group mean, group-wise fill
`filter`	Subset of original rows	Keep groups meeting a condition (`lambda g: len(g) > 1`)
`apply`	Anything	Flexible but slowest – last resort

`size()` counts rows including NaN; `count()` counts non-null values. Group by several keys with a list (`["city", "product"]`) → a MultiIndex result; `.reset_index()` flattens it, or pass `as_index=False`. With categorical keys, pass `observed=True` to skip category combinations that never occur.

Real-world examples

Revenue per region per month, average session length per user cohort, top product per store, feature engineering such as "user's average order value" for ML models.

TL;DR

`groupby(keys)[col].agg(...)` for summaries; `transform` to broadcast group stats back to rows; avoid `apply` when a built-in aggregation exists.

18.8 Combining tables: `merge` , `join` , `concat`

What it is

- `merge` joins two tables on key columns, like SQL `JOIN` .
- `join` is a shortcut for merging on the index.
- `concat` stacks tables vertically (more rows) or side by side (`axis=1`).

```
customers = pd.DataFrame({"customer": ["ana", "ben", "cara", "eve"],
                          "segment": ["retail", "business", "retail", "business"]})
df.merge(customers, on="customer", how="inner")      # only matching keys
df.merge(customers, on="customer", how="left")       # keep all orders
customers.merge(df[["customer"]].drop_duplicates(), on="customer", how="outer", indicator=True)
pd.concat([df, april], ignore_index=True)            # stack rows; missing columns -> NaN
```

```
inner rows=7, left rows=8, dev's segment=nan
  customer   segment      _merge
0      ana    retail        both
1      ben  business        both
2     cara    retail        both
3      dev       NaN  right_only
4      eve  business   left_only
concat rows: 9
```

how=	Keeps	SQL
`inner` (default)	Keys in both	`INNER JOIN`
`left`	All left rows	`LEFT JOIN`
`right`	All right rows	`RIGHT JOIN`
`outer`	All keys from both	`FULL OUTER JOIN`
`cross`	Every pair	`CROSS JOIN`

`indicator=True` adds a `_merge` column that tells you where each row came from — a quick data-quality check ("which orders have no customer record?"). Use `validate="many_to_one"` to make pandas raise if the right table unexpectedly has duplicate keys, which would otherwise **multiply** rows silently. Different key names: `left_on=` / `right_on=`. Overlapping column names get `suffixes=("_x", "_y")`.

Common mistakes

- Row explosion from duplicate keys (check `len` before and after, or `validate=`).
- Keys with different dtypes (`int` vs `str`) → no matches.
- `concat` in a loop (quadratic) — collect DataFrames in a list and concat once.

TL;DR

`merge` = SQL join (choose `how`, check with `indicator` / `validate`); `concat` = stack.

18.9 Reshaping: `pivot_table` and `melt`

What it is

- **Wide** format: one row per entity, one column per variable (a spreadsheet report).
- **Long** ("tidy") format: one row per observation — `(city, product, revenue)`. Plotting and grouping libraries usually prefer long.
- `pivot_table` goes long → wide (aggregating duplicates); `melt` goes wide → long.

```
pivot = df.pivot_table(index="city", columns="product", values="revenue",
                       aggfunc="sum", fill_value=0)
long = pivot.reset_index().melt(id_vars="city", var_name="product", value_name="revenue")
```

```
product   bag  notebook   pen
city
Delhi    25.0       8.0   0.0
Mumbai    0.0      12.0   7.5
Pune     50.0       0.0  45.0
     city product  revenue
0   Delhi     bag     25.0
1  Mumbai     bag      0.0
2    Pune     bag     50.0
```

Related: `pivot` (no aggregation – fails on duplicate pairs), `stack` / `unstack` (move a level between index and columns), `pd.crosstab` (frequency tables), `margins=True` in `pivot_table` for totals.

TL;DR

`pivot_table` = Excel pivot (long → wide with aggregation); `melt` = unpivot (wide → long).

18.10 Dates and times

How it works

- Parse: `pd.to_datetime(col, format="%Y-%m-%d", errors="coerce")` or `parse_dates=` in `read_csv`. Giving `format` is faster and avoids day/month ambiguity.
- The `.dt` accessor: `.dt.year`, `.dt.month`, `.dt.day_name()`, `.dt.hour`, `.dt.to_period("M")`, `.dt.floor("h")`.
- Arithmetic: `date + pd.Timedelta(days=7)`, `df["end"] - df["start"]` → timedelta, `pd.date_range("2026-01-01", periods=3, freq="D")`.
- Time series: set a `DatetimeIndex`, then `resample("ME")` (month end), `"W"`, `"D"`, `"h"` to aggregate by period, and `rolling(7).mean()` for moving averages.
- Time zones: `tz_localize("Asia/Kolkata")` then `tz_convert("UTC")`; pandas 3 uses the standard-library `zoneinfo`.

```
df["date"].dt.day_name().head(5); df["date"].dt.month.unique()
df.set_index("date")["revenue"].resample("ME").sum()
```

```
['Saturday', 'Saturday', 'Saturday', 'Sunday', 'Saturday'] [1, 2, 3]
date
2026-01-31    23.0
2026-02-28    82.5
2026-03-31    42.0
Freq: ME, Name: revenue, dtype: float64
```

Version note: pandas 3.0 removed the old frequency aliases – month end is `"ME"` (not `"M"`), quarter end `"QE"`, year end `"YE"`, hour `"h"`. Old tutorials using `resample("M")` now fail.

TL;DR

Parse early with an explicit format, use `.dt` for parts, `resample` for calendar buckets, `rolling` for moving windows. Use `"ME"` / `"h"` aliases in pandas 3.

18.11 `apply` vs vectorised, and performance tips

Measured

```
big.apply(lambda r: r["qty"] * r["price"], axis=1)   # a Python function call per row
big["qty"] * big["price"]                            # one C loop per column
```

```
100k rows: apply(axis=1) 210 ms, vectorised 0.08 ms, x2523
```

`apply(axis=1)` builds a Series object for every row and calls your Python function on it – it's a loop with extra overhead. Row iteration with `iterrows()` is even slower; if you truly must loop, `itertuples()` is much faster.

Performance checklist

Tip	Why
Vectorise: arithmetic on columns, `np.where` / `np.select`, `.str` and `.dt` methods	C loops instead of Python calls
Right dtypes: `category` for low-cardinality text, smaller numeric types (`float32`, `int32`) where precision allows	Less memory, faster group-bys
Read less: `usecols=`, `dtype=`, `nrows=`, `chunksize=`	Don't load what you don't need
Store as Parquet	Faster reads, keeps dtypes, column pruning
Use built-in aggregations (`"sum"`, `"mean"`) rather than `lambda` in `agg`	Built-ins run in optimised (Cython) code
Avoid growing DataFrames in a loop	Collect rows/frames in a list, build once
`merge` on the right keys, with `validate`	Avoid accidental row explosions
Check `df.memory_usage(deep=True)`	Find heavy columns
When data outgrows pandas	Polars, DuckDB, Dask or Spark

```
cities = pd.Series(rng.choice(["Pune", "Delhi", "Mumbai"], 100_000))
cities.memory_usage(deep=True); cities.astype("category").memory_usage(deep=True)
```

```
dtype str: 5.40 MB, category: 0.10 MB
```

A **categorical** stores each distinct value once plus a small integer code per row – here 54× less memory. (With PyArrow installed, the `str` dtype itself is also far more compact than Python string objects.)

In the interview

30-second answer: "pandas is fast when you stay vectorised: column arithmetic, `np.where`, `.str` / `.dt` accessors and built-in group aggregations all run in compiled code. `apply(axis=1)` and `iterrows` are Python loops. For memory I use categoricals and smaller dtypes, read only needed columns and store Parquet. In pandas 3, Copy-on-Write means every indexing result behaves as a copy, so I always assign with a single `.loc`."

- *`loc` vs `iloc`?* – Labels (inclusive end) vs positions (exclusive end).
- *`transform` vs `agg`?* – `transform` returns one value per original row; `agg` one per group.

- *How do you find orders with no matching customer?* – Left merge with `indicator=True` and filter `left_only`.

Self-check

1. Why does `qty` load as float? → It contains `NaN`, which is a float.
2. What does `how="left"` keep? → Every row of the left table.
3. What changed about chained assignment in pandas 3? → It never updates the original (Copy-on-Write).
4. Month-end frequency alias in pandas 3? → `"ME"`.
5. Faster way to compute `qty * price` than `apply`? → `df["qty"] * df["price"]`.

TL;DR

Vectorise, pick dtypes, read less, store Parquet, don't loop. `apply` is the last resort.

18.12 matplotlib: Figure, Axes and the four workhorse charts

What it is

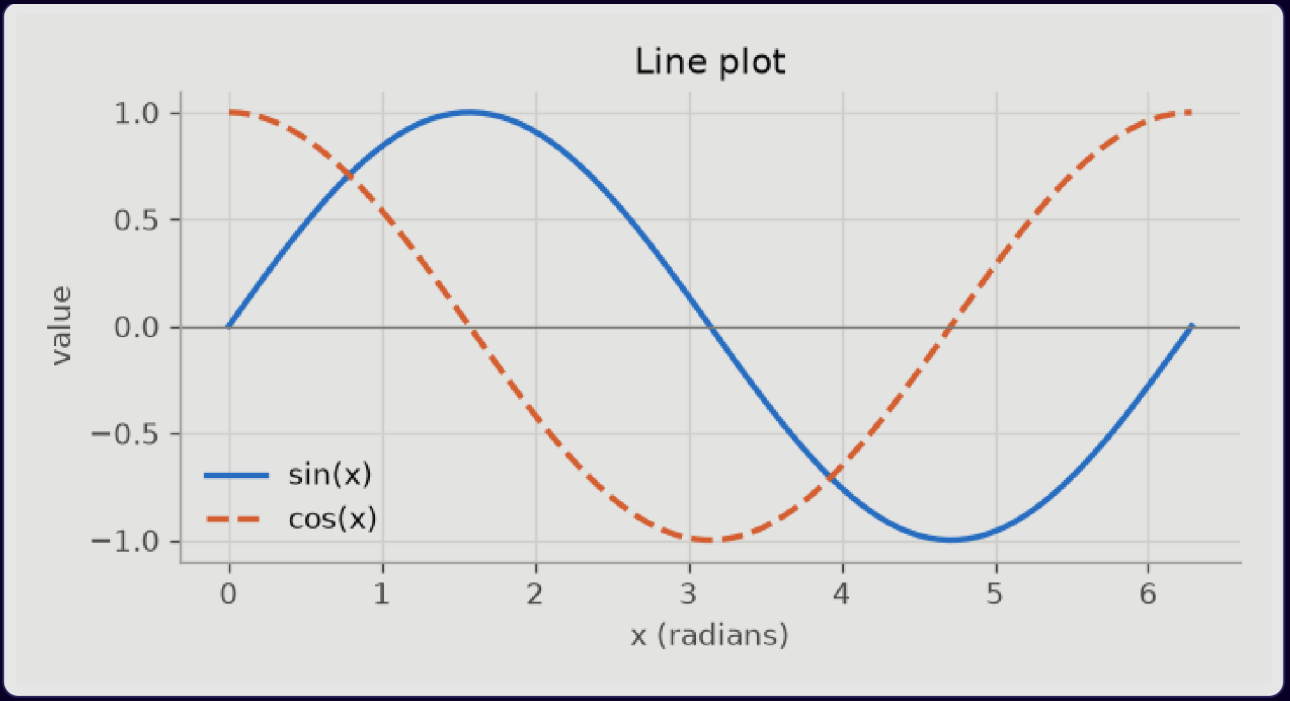
matplotlib is Python's foundational plotting library (pandas' `.plot()` and seaborn are built on it).

- A **Figure** is the whole canvas/image.
- An **Axes** is one plot inside it – with its own x/y axis, title, labels and legend. A figure can hold many axes.
- Two styles: the **pyplot state machine** (`plt.plot(...)`, acts on "the current axes" – quick in notebooks) and the **object-oriented (OO) API** (`fig, ax = plt.subplots(); ax.plot(...)` – explicit, recommended for scripts and multiple plots). Use the OO API.

How it works

```
import matplotlib
matplotlib.use("Agg")                          # render to files only (servers, scripts)
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 3.2))       # Figure = canvas, Axes = one plot
ax.plot(x, np.sin(x), label="sin(x)")
ax.plot(x, np.cos(x), label="cos(x)", linestyle="--")
ax.set(title="Line plot", xlabel="x (radians)", ylabel="value")
ax.legend(frameon=False)
fig.tight_layout()                             # stop labels overlapping
fig.savefig(IMG / "line_plot.png", dpi=120)    # format from the extension: png, svg, pdf
plt.close(fig)                                 # free memory in scripts/loops
```




```
fig, axes = plt.subplots(2, 2, figsize=(9, 6))              # axes is a 2x2 numpy array
axes[0, 0].plot(months, sales, marker="o")                  # line: change over time
axes[0, 1].bar(["pen", "notebook", "bag"], [95, 20, 75])    # bar: compare categories
axes[1, 0].scatter(h, w, s=14, alpha=0.6)                   # scatter: relationship
axes[1, 1].hist(rng.normal(50, 10, 1_000), bins=30)         # histogram: distribution
fig.suptitle("Four basic chart types")
```

Output (`matplotlib_demo.py`):

```
corr(height, weight) = 0.80
saved line_plot.png, four_charts.png
```

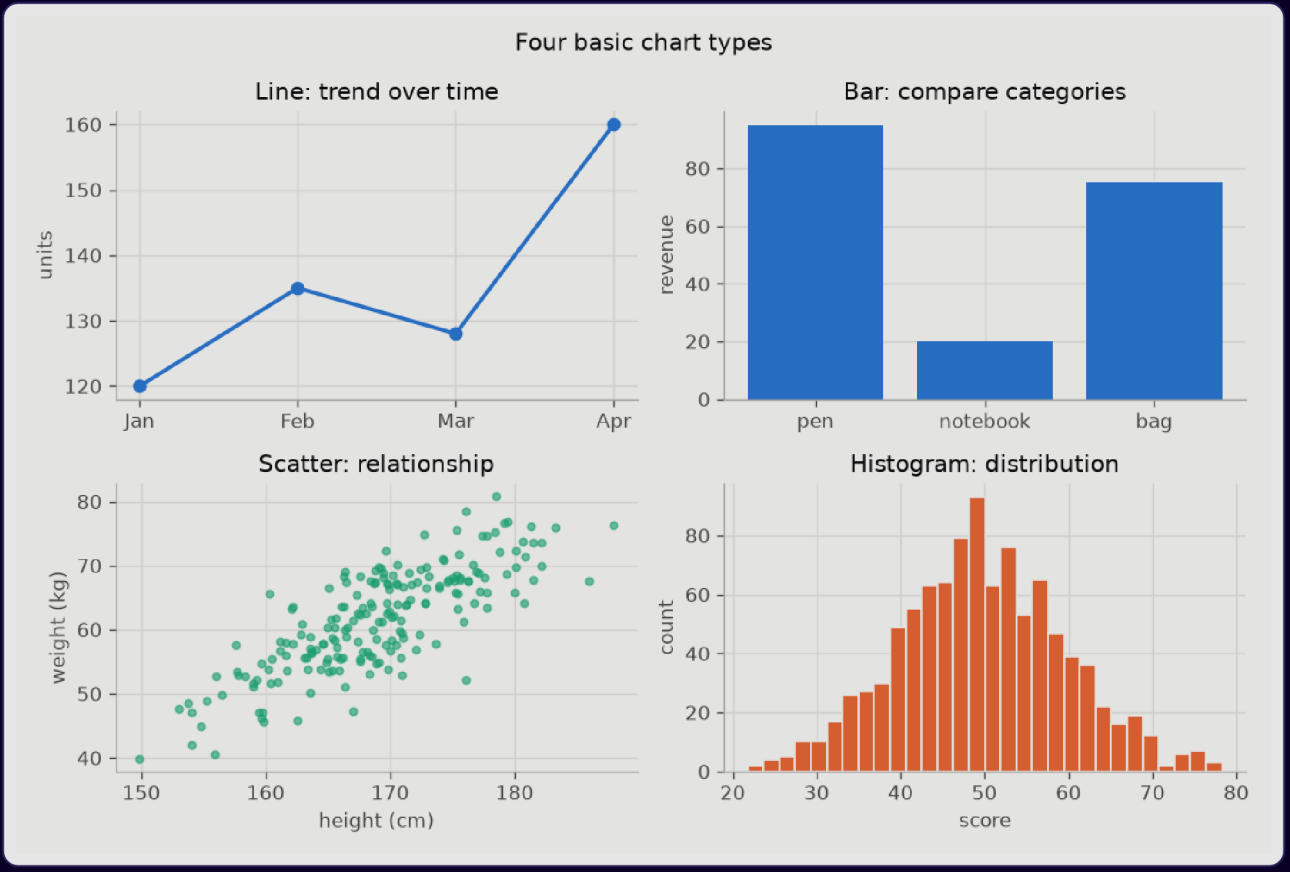


Chart	Answers	Call	Tip
Line	How does it change over an ordered x (time)?	`ax.plot(x, y)`	Don't use for unordered categories
Bar	How do categories compare?	`ax.bar` / `ax.barh`	Start the y-axis at 0; horizontal bars for long labels
Scatter	Are two numbers related?	`ax.scatter(x, y)`	`alpha` for overplotting
Histogram	How are values distributed?	`ax.hist(values, bins=30)`	Try several `bins` values
Box plot	Spread and outliers by group	`ax.boxplot`	

Labelling essentials: `ax.set_title`, `ax.set_xlabel` / `ax.set_ylabel` (include units), `ax.legend()` whenever there are two or more series, `ax.tick_params(axis="x", rotation=30)` for crowded dates, `ax.set_xlim` / `set_ylim`, `ax.axhline` for reference lines. `savefig(..., bbox_inches="tight")` trims margins; `dpi=150–300` for print. Shortcut from pandas: `df.plot(kind="bar", ax=ax)`.

Common mistakes

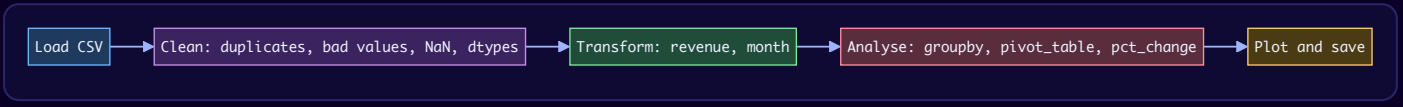
- Mixing pyplot and OO calls, so labels land on the wrong subplot.
- Forgetting `plt.close(fig)` in loops → memory grows and matplotlib warns about too many open figures.
- Calling `plt.show()` in a headless script instead of `savefig` (use the `Agg` backend).
- Two y-axes (`twinx`) for unrelated measures — usually misleading; use two panels.
- Rainbow colour maps for ordered data — use a single-hue sequential map (e.g. `viridis`).

TL;DR

`fig, ax = plt.subplots()`, draw on `ax`, label everything, `fig.savefig(path, dpi=...)`, `plt.close(fig)`. Line = trend, bar = compare, scatter = relationship, histogram = distribution.

18.13 Mini end-to-end analysis

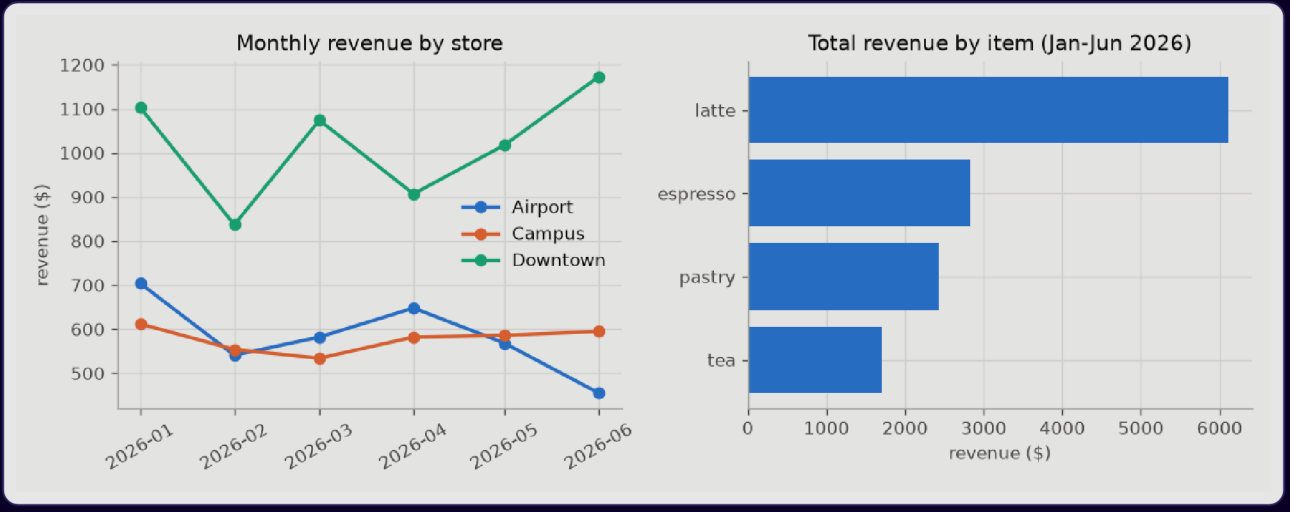
A coffee chain with three stores wants to know: which store earns most, how revenue moved month by month, and which item matters most. The script generates a realistic messy CSV (6 months, 2,000 orders, 3 % missing quantities, a negative price, 10 duplicated rows) and then runs the standard pipeline.



```
# 1. Load
raw = pd.read_csv(io.StringIO(make_raw_csv()), parse_dates=["date"])
# 2. Clean
df = (raw.drop_duplicates()
         .query("unit_price > 0")
         .assign(qty=lambda d: d["qty"].fillna(d["qty"].median()).astype(int),
                 store=lambda d: d["store"].astype("category"),
                 item=lambda d: d["item"].astype("category")))
# 3. Transform
df["revenue"] = df["qty"] * df["unit_price"]
df["month"] = df["date"].dt.to_period("M")
# 4. Analyse
by_store = df.groupby("store", observed=True).agg(orders=("qty", "size"),
                                                  revenue=("revenue", "sum"),
                                                  avg_ticket=("revenue", "mean")).round(2)
monthly = df.pivot_table(index="month", columns="store", values="revenue",
                         aggfunc="sum", observed=True).round(0)
growth = monthly.sum(axis=1).pct_change().mul(100).round(1)
# 5. Plot  (line per store + horizontal bar per item, saved to img/coffee_analysis.png)
```

Output (`end_to_end.py`):

```
raw: 2010 rows, missing qty=65, duplicates=10, negative prices=2
clean: 1999 rows, nulls=0
          orders  revenue  avg_ticket
store
Downtown    1010   6112.0        6.05
Airport      412   3497.2        8.49
Campus       577   3461.5        6.00
store    Airport  Campus  Downtown
month
2026-01    704.0   612.0    1103.0
2026-02    541.0   554.0     837.0
2026-03    582.0   534.0    1074.0
2026-04    648.0   582.0     907.0
2026-05    568.0   586.0    1018.0
2026-06    455.0   595.0    1173.0
best item per store: {'Airport': 'latte', 'Campus': 'latte', 'Downtown': 'latte'}
month-over-month total revenue change %: {'2026-02': -20.1, '2026-03': 13.4, '2026-04': -2.4, '2026-05': 1.6,
'2026-06': 2.3}
saved coffee_analysis.png
```



Reading the results:

1. **Data quality** – 10 duplicates removed (the duplicated block contained the bad row, hence 2 negative prices, 1 after dedup); 1 negative price dropped; 65 missing quantities filled with the median. Always report what cleaning did.

2. **Stores** – Downtown has the most orders and revenue; Airport has the fewest orders but the highest average ticket (8.49 vs ~6), which matches the 40 % airport markup built into the data.
3. **Trend** – February dropped 20 % (it has 28 days, and our synthetic data is uniform per day – a reminder to normalise by days or compare like periods); Airport declines towards June while Downtown grows.
4. **Items** – lattes lead in every store: highest price among drinks and most frequent item.

This is the workflow of most analytics tasks and the first half of any ML project; the ML Handbook's Chapter 3 continues from a clean DataFrame to features, models and evaluation.

18.14 Chapter self-check

1. Why is `arr * 2` faster than a list comprehension? → One compiled loop over contiguous typed memory, no per-element Python objects.
2. Is `m[:2, :2]` a copy? → No, a view.
3. Result shape of `(4, 1) * (3,)`? → `(4, 3)`.
4. Better than `inv(A) @ b`? → `np.linalg.solve(A, b)`.
5. How do you get reproducible random numbers? → `rng = np.random.default_rng(seed)`.
6. `df.loc[1:3]` vs `df.iloc[1:3]` with a default index? → Labels 1–3 (3 rows) vs positions 1–2 (2 rows).
7. How do you add each row's share of its group total? → `col / df.groupby(key)[col].transform("sum")`.
8. How do you detect keys missing from a lookup table after a merge? → `indicator=True`, look for `left_only`.
9. Long → wide? Wide → long? → `pivot_table`; `melt`.
10. Figure vs Axes? → The canvas vs one plot on it.

Chapter TL;DR

- **numpy**: typed contiguous arrays; shape/dtype; slices are views; masks and fancy indexing copy; vectorise instead of looping (35× here); `axis=0` per column, `axis=1` per row; broadcasting aligns shapes from the right; `@`, `solve`, `eigh`; `default_rng(seed)`.
- **pandas**: Series/DataFrame with label alignment; `read_csv` / `read_parquet`; `loc` (labels, inclusive) vs `iloc` (positions); masks with `&` / `|`; fill missing values thoughtfully; `groupby` + `agg` / `transform`; `merge` with `how`, `indicator`, `validate`; `pivot_table` / `melt`; `.dt` and `resample("ME")`.
- **pandas 3.0**: `str` dtype for text, Copy-on-Write always on (assign with one `.loc`), microsecond datetimes, new frequency aliases.
- **Performance**: vectorise (2,500× over `apply` here), categoricals (54× less memory here), Parquet, read less.
- **matplotlib**: OO API (`fig, ax = plt.subplots()`), line/bar/scatter/hist, label everything, `savefig` + `close`.
- **Workflow**: load → clean → transform → analyse → plot → explain, then hand the clean table to ML.

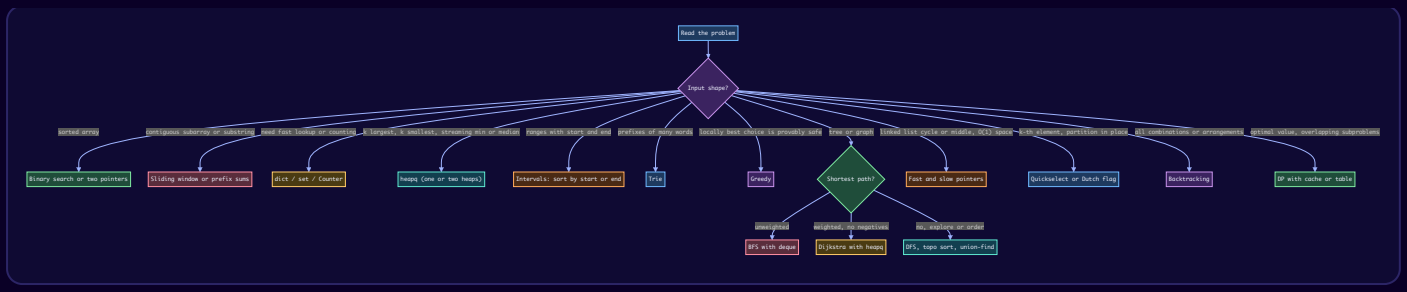
Chapter 19 – Python for Coding Interviews

Coding interviews (the "LeetCode-style" rounds at most tech companies) test two things at once: whether you know the classic **data-structure and algorithm (DSA) patterns**, and whether you can express them cleanly in a language. Python is the most popular interview language because it is short: a heap, a hash map, a queue and a binary search are each one import away. But that shortness hides costs – `list.pop(0)` looks innocent and is O(n), `[[0]*n]*m` looks like a grid and is a bug.

This chapter is the Python toolkit for DSA. For each pattern you get the idea, the Python tools that implement it, a tiny runnable example with `assert` tests, its complexity, and the mistakes people make. It assumes Chapters 5–7 (lists, dicts, functions) and 11 (iterators). All scripts are in `code/`; they were run with Python 3.14 (everything is 3.12-compatible) and the output shown is real.

What you'll learn

#	Topic	Script
19.1	Big-O refresher and the complexity cheat table for built-ins	`01_builtins_complexity.py`
19.2	Arrays & strings: two pointers, sliding window, prefix sums	`02_arrays_strings.py`
19.3	Hashing: `dict`, `set`, `Counter`, `defaultdict`	`03_hashing.py`
19.4	Stacks and queues: `list`, `deque`, monotonic stack/deque	`04_stacks_queues.py`
19.5	Heaps: `heapq`, the max-heap trick, top-k, merge k, two heaps (running median, maximise capital)	`05_heaps.py`
19.6	Sorting with keys; binary search by hand and with `bisect`; weighted random pick	`06_sorting_bisect.py`
19.7	Linked lists and trees as Python classes; serialize/deserialize a tree	`07_linked_lists_trees.py`
19.8	Graphs: adjacency lists, BFS/DFS, grids, Dijkstra, topological sort, recursion limit	`08_graphs.py`
19.9	Union-Find (disjoint sets), Kruskal	`09_union_find.py`
19.10	Dynamic programming: `@cache` (top-down) and tabulation (bottom-up)	`10_dynamic_programming.py`
19.11	Backtracking	`11_backtracking.py`
19.12	Bit tricks and math (`gcd`, `pow(b, e, m)`, `isqrt`, `comb`, sieve)	`12_bits_math.py`
19.13	Tries: prefix search, autocomplete, wildcard search, word search on a board	`13_tries.py`
19.14	Greedy: jump game, gas station, candy, interval scheduling	`14_greedy.py`
19.15	Intervals: merge, insert, intersect, meeting rooms (heap and sweep line)	`15_intervals.py`
19.16	Fast & slow pointers: cycle start, middle, palindrome list, happy number, find duplicate	`16_fast_slow_pointers.py`
19.17	Sort & search: merge sort on a list, quicksort, quickselect, Dutch national flag	`17_sort_search.py`
19.18	Math & geometry: spiral matrix, rotate, reverse a 32-bit int, collinear points, Josephus	`18_math_geometry.py`
19.19	Common pitfalls	`19_pitfalls.py`
19.20	How to talk through a solution	–



19.1 Big-O and the built-ins cheat table

What it is

Big-O notation describes how running time (or memory) grows with the input size *n*, ignoring constant factors — the fuel-consumption class of a car, not the litres for today's trip. O(1) does not grow with n, O(n) grows in proportion, O(n log n) is good sorting, O(n²) is "every pair". An **amortised** O(1) operation is occasionally expensive but averages out to constant time: `list.append` sometimes reallocates and copies, but rarely enough that the average stays O(1).

Why it matters

Interviewers always ask "what's the complexity?", and Python hides loops inside innocent-looking calls: `x in my_list` scans, `my_list.insert(0, x)` shifts every element, `s = s + ch` copies the string.

How it works — the cheat table

CPython's `list` is a dynamic array of pointers; `dict` and `set` are hash tables; `collections.deque` is a doubly-linked list of fixed-size blocks; `heapq` works on a plain list arranged as a binary heap. The costs follow from those designs (average case; hash tables can degrade to O(n) with pathological collisions):

Structure	Operation	Cost	Note
`list`	`a[i]`, `a[i] = x`, `len(a)`	O(1)	array indexing
	`a.append(x)`, `a.pop()`	O(1) amortised	end of the array
	`a.insert(i, x)`, `a.pop(i)`, `del a[i]`	O(n)	shifts the tail; `pop(0)` is the trap
	`x in a`, `a.index(x)`, `a.count(x)`, `min/max/sum`	O(n)	linear scan
	`a[i:j]`, `a.copy()`, `a + b`	O(k)	slicing **copies** k elements
	`a.sort()`, `sorted(a)`	O(n log n)	Timsort, stable; O(n) if already sorted
	`a.reverse()`, `a[::-1]`	O(n)	
`deque`	`append`, `appendleft`, `pop`, `popleft`	O(1)	the right queue type
	`dq[i]` in the middle	O(n)	O(1) at either end
`dict`	`d[k]`, `d[k] = v`, `del d[k]`, `k in d`, `d.get`	O(1) avg	keys must be hashable
	iterate over keys/values/items	O(n)	insertion-ordered (3.7+)
`set`	`add`, `remove`, `discard`, `x in s`	O(1) avg	
	`s | t`, `s - t`	O(len s + len t) / O(len s)	
	`s & t`	O(min(len s, len t))	
`heapq`	`heappush`, `heappop`, `heappushpop`	O(log n)	min-heap on a list
	`h[0]` (peek)	O(1)	
	`heapify(list)`	O(n)	faster than n pushes
	`nlargest(k, it)`, `nsmallest(k, it)`	O(n log k)	
`bisect`	`bisect_left/right`	O(log n)	list must be sorted
	`insort`	O(n)	search is log n, the insert shifts
`str`	`s[i]`, `len(s)`	O(1)	immutable
	`s + t`, slicing, `s.replace`, `s.split`	O(n)	always builds a new string
	`"".join(parts)`	O(total length)	the right way to build strings
`Counter`	`Counter(iterable)`	O(n)	`most_common(k)` O(n log k)

Python `int`s have **arbitrary precision** (no overflow), so `(lo + hi) // 2` is safe — but arithmetic on huge numbers is not O(1); multiplying two d-digit numbers costs more than constant time. For interview-sized numbers, treat it as O(1).

The script measures the two most important rows:

```
def pop_front_times(n):
    def drain_list():
        items = list(range(n))
        while items:
            items.pop(0)        # O(n) each: every other element shifts left
    def drain_deque():
        items = deque(range(n))
        while items:
            items.popleft()     # O(1) each
    return timeit.timeit(drain_list, number=1), timeit.timeit(drain_deque, number=1)
```

Output (your machine's numbers will differ; the shape will not):

```
     n |   x in list |  x in set |  pop(0) all | popleft all
  1000 |      5.3 us |   0.03 us |     0.07 ms |     0.02 ms
  5000 |     26.3 us |   0.04 us |     1.06 ms |     0.13 ms
 20000 |    108.3 us |   0.03 us |    24.50 ms |     0.51 ms
 50000 |    321.3 us |   0.18 us |   174.89 ms |     1.47 ms
```

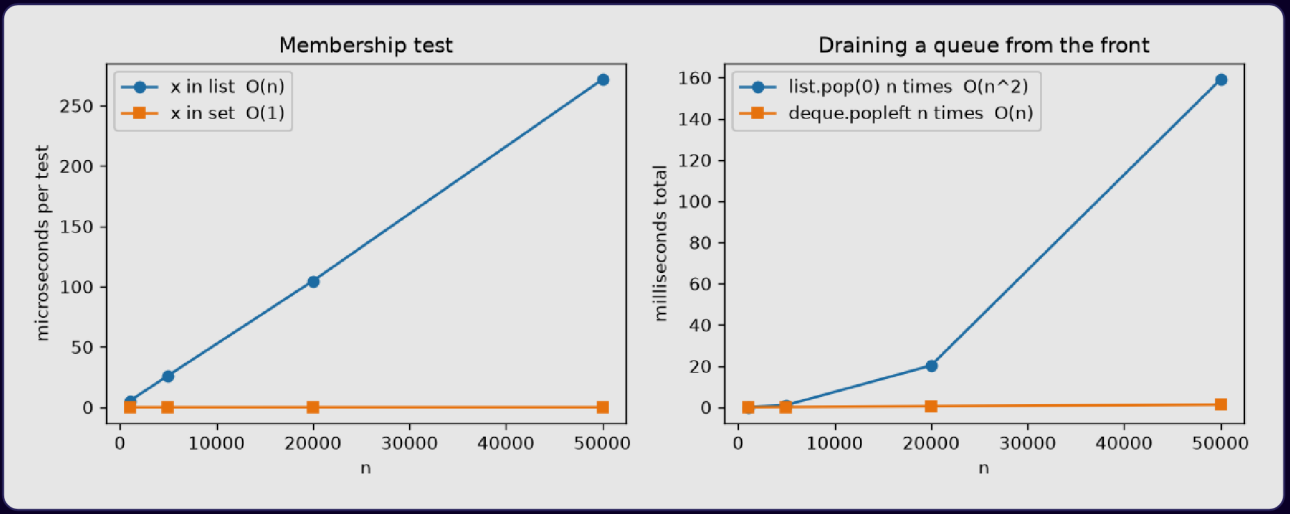


Figure — `x in list` grows linearly while `x in set` stays flat; draining a list from the front is quadratic, a deque is linear.

Common mistakes

- `x in list` inside a loop (O(n²) total) when a `set` makes it O(n); a list as a queue with `pop(0)`.
- Forgetting that slicing copies: `helper(nums[1:])` in recursion adds O(n) per call — pass indices instead.
- Building a string with `+=` in a loop (CPython sometimes optimises it in place, but you cannot rely on it) — collect pieces and `"".join` once.

In the interview

30-second answer: "Lists are dynamic arrays — O(1) index and append, O(n) insert/delete at the front and membership. Dicts and sets are hash tables with O(1) average lookup. For a queue I use `deque`, for a priority queue `heapq`, and sorting is O(n log n) Timsort."

- *Why is `append` amortised O(1)?* → The array over-allocates; when full it grows by a proportion of its size and copies, so the total copying over n appends is O(n).
- *Worst case for a dict lookup?* → O(n) if many keys collide; in practice with good hashes it's O(1).
- *Space complexity of `sorted(a)`?* → O(n) for the new list (Timsort also uses up to n/2 temporary space).

19.2 Arrays and strings: two pointers, sliding window, prefix sums

These three patterns turn many "check every pair/subarray" O(n²) problems into O(n).

What it is

- **Two pointers** — two indices move through the data, usually from both ends toward each other (sorted pair sum, palindrome) or both forward at different speeds (a *slow* "write" pointer and a *fast* "read" pointer for in-place filtering).
- **Sliding window** — indices `left` and `right` bound a contiguous window, like a caterpillar: `right` grows it one step at a time, `left` shrinks it when a rule is broken. Each index only moves forward, so the total work is O(n).
- **Prefix sums** — precompute `prefix[k] = sum of the first k elements`. Then the sum of any range `nums[i:j]` is `prefix[j] - prefix[i]` in O(1).

How it works

Two pointers on a sorted array (pair with sum = target): start at both ends. If the sum is too small, only moving `lo` up can increase it; if too big, only moving `hi` down can decrease it. Each step discards one element forever → O(n).

Variable sliding window (longest substring with no repeated character): expand `right`, remembering each character's last index in a dict; if the new character was already seen inside the window, jump `left` past its earlier copy. The answer is the largest `right - left + 1`.

Prefix sums + hash map (count subarrays summing to k, negatives allowed): a subarray `(i, j]` sums to k exactly when `prefix[j] - prefix[i] = k`, i.e. `prefix[i] = prefix[j] - k`. Walk once, keep a count of prefix values seen so far, and add `seen[running - k]` at each step. Worked numbers: for `[1, 1, 1]` and k = 2, running sums are 1, 2, 3; at 2 we look up 0 (seen once, the empty prefix) → 1; at 3 we look up 1 (seen once) → 1; total 2.

```python
def two_sum_sorted(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target:
            return lo, hi
        if s < target:
            lo += 1          # need a bigger sum -> move the small end up
        else:
            hi -= 1          # need a smaller sum -> move the big end down
    return None

def longest_unique_substring(s):
    last_seen = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1      # shrink: jump past the earlier copy
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best

def min_window_at_least(nums, target):   # positive numbers only
    left = total = 0
    best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total >= target:            # shrink while still valid
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else int(best)

class RangeSum:
    def __init__(self, nums):
        self.prefix = [0, *accumulate(nums)]   # prefix[k] = sum(nums[:k])
    def query(self, i, j):                     # half-open [i, j)
        return self.prefix[j] - self.prefix[i]

def count_subarrays_sum_k(nums, k):
    seen = defaultdict(int)
    seen[0] = 1           # the empty prefix
    running = count = 0
    for x in nums:
        running += x
        count += seen[running - k]
        seen[running] += 1
    return count

assert two_sum_sorted([1, 3, 4, 6, 9], 10) == (0, 4)
assert longest_unique_substring("pwwkew") == 3
assert min_window_at_least([2, 3, 1, 2, 4, 3], 7) == 2
assert RangeSum([3, -1, 4, 1, 5]).query(1, 4) == 4
assert count_subarrays_sum_k([1, 1, 1], 2) == 2
```

The script also has `is_palindrome` (two pointers from both ends, skipping non-alphanumerics) and `remove_duplicates_sorted` (slow/fast pointers, in place). Output:

```
prefix: [0, 3, 2, 6, 7, 12]
two_sum_sorted([1,3,4,6,9], 10) -> (0, 4)
longest_unique_substring('pwwkew') -> 3
min_window_at_least([2,3,1,2,4,3], 7) -> 2
count_subarrays_sum_k([3,4,7,-2,2,1,4,2], 7) -> 6
all asserts passed
```

Python helpers worth knowing

Need	Python
index + value	`for i, x in enumerate(nums)`
adjacent pairs	`for a, b in itertools.pairwise(nums)` (3.10+) or `zip(nums, nums[1:])`
running totals	`itertools.accumulate(nums)`; `accumulate(nums, max)` for running max
reverse	`nums[::-1]` (copy) or `nums.reverse()` (in place)
letter → 0..25	`ord(ch) - ord("a")`; back with `chr(i + ord("a"))`
character class	`ch.isalnum()`, `ch.isdigit()`, `ch.isalpha()`, `ch.lower()`
strings are immutable	convert to `list(s)`, edit, then `"".join(chars)`

Trade-offs

Pattern	Needs	Time	Space
Two pointers (pair sum)	sorted input	O(n) (+ O(n log n) to sort)	O(1)
Hash map (pair sum, 19.3)	nothing	O(n)	O(n)
Sliding window	a condition that stays monotonic as the window grows/shrinks (e.g. positive numbers)	O(n)	O(1)–O(alphabet)
Prefix sums	static array	O(n) build, O(1) query	O(n)

When not to use it

A shrinking window relies on "adding makes it bigger, removing makes it smaller". With **negative numbers** that breaks (shrinking may *increase* the sum) — use prefix sums + hash map instead, as in `count_subarrays_sum_k`. If the array changes between queries, prefix sums must be rebuilt; use a Fenwick/segment tree.

Common mistakes

- Off-by-one in window length (it is `right - left + 1`); forgetting `seen[0] = 1` (misses subarrays starting at index 0).
- Moving `left` backwards in the unique-substring problem — the `last_seen[ch] >= left` check prevents it.

In the interview

> **30-second answer:** "When I see contiguous subarrays or substrings I think sliding window — expand right, shrink left while invalid, O(n). For pairs in a sorted array, two pointers from both ends. For range sums or 'subarray sums to k', prefix sums with a hash map of seen prefixes."

- *Why is the window O(n) when it has a nested `while`?* → Each index enters and leaves the window at most once; the inner loop's total iterations across the whole run are ≤ n.
- *Two-sum on unsorted data?* → Hash map in O(n) time/O(n) space; sorting + two pointers is O(n log n) and loses original indices unless you sort `(value, index)` pairs.
- *What is `prefix` for `[2, 4, 1]`?* → `[0, 2, 6, 7]`.
- *Sum of `nums[1:3]` from it?* → `prefix[3] - prefix[1] = 5`.

19.3 Hashing: `dict`, `set`, `Counter`, `defaultdict`

What it is

A **hash table** stores items in buckets chosen by a **hash** of the key, giving O(1) average insert and lookup. Python's four flavours: `dict` (key → value), `set` (keys only), `collections.Counter` (a dict subclass that counts; missing keys read as 0) and `collections.defaultdict(factory)` (creates `list()`, `int()`, `set()`… the first time a missing key is accessed). Only **hashable** objects can be keys – `int`, `str`, `tuple` of hashables, `frozenset`; convert a list to a `tuple` first.

Why it matters

A large share of easy/medium problems are "trade O(n) memory for O(1) lookup": two-sum, anagrams, duplicates, frequency counting, "have I seen this before?".

How it works

```
def two_sum(nums, target):
    index_of = {}
    for i, x in enumerate(nums):
        if target - x in index_of:        # the complement was seen earlier
            return index_of[target - x], i
        index_of[x] = i
    return None

def is_anagram(a, b):
    return Counter(a) == Counter(b)

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)   # key must be hashable -> tuple
    return list(groups.values())

def longest_consecutive(nums):
    values = set(nums)
    best = 0
    for x in values:
        if x - 1 not in values:           # x starts a run -> count it once
            length = 1
            while x + length in values:
                length += 1
            best = max(best, length)
    return best

assert two_sum([2, 7, 11, 15], 9) == (0, 1)
assert longest_consecutive([100, 4, 200, 1, 3, 2]) == 4

c = Counter("mississippi")
print("Counter:", c)
print("most_common(2):", c.most_common(2))
print("missing key:", c["z"])
print("c - Counter('sip'):", c - Counter("sip"))
```

Output:

```
Counter: Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
most_common(2): [('i', 4), ('s', 4)]
missing key: 0
c - Counter('sip'): Counter({'i': 3, 's': 3, 'm': 1, 'p': 1})
group_anagrams: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
all asserts passed
```

`longest_consecutive` looks like a nested loop but is O(n): the inner `while` only runs from the start of each run, so every number is visited by it at most once.

Handy API

Want	Code
read with default	`d.get(k, 0)`
count / group	`Counter`, `defaultdict(int)`, `defaultdict(list)` + `.append`
insert if missing, return value	`d.setdefault(k, [])`
top k counts	`Counter(xs).most_common(k)`
counter arithmetic	`c1 + c2`, `c1 - c2` (drops ≤ 0), `c1 & c2` (min), `c1 | c2` (max)
fixed alphabet count	`[0] * 26` indexed by `ord(ch) - 97` — also hashable as `tuple(...)`
set algebra	`a & b`, `a | b`, `a - b`, `a ^ b`

Trade-offs

Hashing costs O(n) extra memory and loses order-based tricks. If the input is sorted, two pointers use O(1) space. For keys drawn from a tiny fixed range (26 letters), an array is faster and simpler than a dict.

Common mistakes

- Using a list as a key (`TypeError: unhashable type: 'list'`) — use `tuple(lst)`.
- Reading a `defaultdict` with `d[k]` just to check it: that **inserts** the key. Use `k in d`.
- Mutating a dict while iterating over it (`RuntimeError: dictionary changed size during iteration`) — iterate over `list(d)`.

In the interview

> **30-second answer:** "I use a dict when I need to remember something about values I've already seen — index, count, group — which turns an O(n²) pair search into O(n). Keys must be hashable, so I use tuples for composite keys. `Counter` and `defaultdict` remove boilerplate."

- *How would you make a dict key from a list of counts?* → `tuple(counts)`.
- *`Counter.most_common()` with no argument?* → All items sorted by count, O(n log n); with k it uses a heap, O(n log k).
- *Can a `set` be a dict key?* → No; a `frozenset` can.

19.4 Stacks and queues: `list` and `deque`

What it is

- A **stack** is last-in, first-out (LIFO), like a pile of plates: a `list` with `append` (push) and `pop()` — both O(1) at the end.
- A **queue** is first-in, first-out (FIFO), like a checkout line: `collections.deque` with `append` and `popleft` — both O(1). (`queue.Queue` is a slower thread-safe queue for concurrency, Chapter 16 — not what interviews want.)
- A **monotonic stack** keeps its elements in increasing (or decreasing) order by popping anything that breaks the order before pushing — "next greater/smaller element" in O(n). A **monotonic deque** also drops elements that leave a window from the front — sliding-window maximum in O(n).

How it works

Valid parentheses: push openers; on a closer, the top of the stack must be its matching opener; at the end the stack must be empty.

Next greater element: walk left to right holding indices whose values have not yet found a bigger element. When a new value `x` arrives, it *is* the answer for every smaller value on top of the stack — pop them and record `x`. Each index is pushed and popped once → O(n).

```
def valid_parentheses(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif not stack or stack.pop() != pairs[ch]:
            return False
    return not stack

def next_greater(nums):
    result = [-1] * len(nums)
    stack = []                          # indices, values decreasing
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            result[stack.pop()] = x
        stack.append(i)
    return result

def sliding_window_max(nums, k):
    dq = deque()                        # indices, values decreasing
    out = []
    for i, x in enumerate(nums):
        if dq and dq[0] <= i - k:
            dq.popleft()                # front fell out of the window
        while dq and nums[dq[-1]] <= x:
            dq.pop()                    # smaller values can never be the max
        dq.append(i)
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

assert valid_parentheses("([]{})") and not valid_parentheses("(]")
assert next_greater([2, 1, 2, 4, 3]) == [4, 2, 4, -1, -1]
assert sliding_window_max([1, 3, -1, -3, 5, 3, 6, 7], 3) == [3, 3, 5, 5, 6, 7]
```

The script also has `daily_temperatures` (same monotonic idea, storing distances), a `MinStack` (each entry stores `(value, min_so_far)` so `get_min` is O(1)), and `eval_rpn` for Reverse Polish Notation. Note the division there: `int(a / b)` truncates toward zero as the problem requires, while `a // b` floors (`-7 // 2 = -4`). Deque tour output:

```
deque: [0, 1, 2, 3, 4]
popleft -> 0 | pop -> 4 | now deque([1, 2, 3])
after rotate(1): deque([3, 1, 2])
deque(maxlen=3) after 0..5: deque([3, 4, 5], maxlen=3)
sliding_window_max: [3, 3, 5, 5, 6, 7]
all asserts passed
```

`deque(maxlen=k)` silently drops from the opposite end when full — a ready-made "last k items" buffer.

Common mistakes

- `stack.pop()` or `stack[-1]` on an empty list raises `IndexError` — check `if stack` first (the `not stack or ...` short-circuit above).
- Storing values instead of indices in a monotonic stack when you later need distances or window positions.

In the interview

> **30-second answer:** "Stack = list with append/pop. Queue = deque with append/popleft, never list.pop(0). When the question is 'next greater/smaller' or 'span', I use a monotonic stack of indices — O(n) because each index is pushed and popped once."

- *Implement a queue with two stacks?* → Push to `inbox`; to dequeue, if `outbox` is empty move everything from `inbox` to `outbox` (reversing it), then pop `outbox`. Amortised O(1).
- *Why does `sliding_window_max` pop smaller values from the back?* → A smaller value to the left of a bigger one can never be the maximum of any future window.
- *Which end does `deque.append` add to?* → Right. `appendleft` adds to the left.
- *What does `deque.rotate(1)` do?* → Moves the last element to the front.

19.5 Heaps with `heapq`

What it is

A **binary heap** is a complete binary tree stored in an array where every parent is ≤ its children (a **min-heap**), so the smallest item is at index 0. `heapq` provides functions that treat a plain `list` as a min-heap — there is no heap class. A heap is the standard **priority queue** (always serve the most urgent item next).

How it works

- Children of index i live at 2i+1 and 2i+2. `heappush` appends and "sifts up"; `heappop` moves the last item to the root and "sifts down" — both O(log n).
- **Max-heap trick:** `heapq` only does min-heaps, so push `-x` and negate on the way out. (Python 3.14 adds public `heapq.heappush_max` / `heappop_max` / `heapify_max`, but most interview platforms run older versions, so the negation trick is the portable answer.)
- **Tuples as entries:** `(priority, item)` compares by priority first. If priorities tie, Python compares the next field — add a unique counter `(priority, count, item)` when items themselves aren't comparable.
- **Top-k pattern:** to find the k largest of n items, keep a **min-heap of size k**; whenever it exceeds k, pop the smallest. The root is then the k-th largest. O(n log k) time, O(k) space.

```python
def kth_largest(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)       # drop the smallest
    return heap[0]

def top_k_frequent(words, k):
    counts = Counter(words)
    return heapq.nsmallest(k, counts, key=lambda w: (-counts[w], w))  # ties alphabetical

def merge_k_sorted(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        val, i, j = heapq.heappop(heap)       # i breaks ties
        out.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out

class RunningMedian:
    """low = max-heap (negated) of the smaller half, high = min-heap of the larger half."""
    def __init__(self):
        self.low, self.high = [], []
    def add(self, x):
        heapq.heappush(self.low, -x)
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))
    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2
```

Output of the tour in `__main__`:

```
heapified: [1, 2, 8, 3, 5] smallest: 1
pop order: [1, 2, 3, 5, 8]
max via negation: 8
priority queue first: (1, 'fix prod bug')
nlargest(2): [9, 7]
running medians: [5.0, 10.0, 5.0, 4.0]
max capital (k=2): 4
all asserts passed
```

Notice `heapified` is **not** sorted – only `h[0]` is guaranteed smallest. Popping repeatedly gives sorted order (that's heapsort).

The two-heaps pattern

Use **two heaps facing each other** when you must repeatedly split data into a "smaller" group and a "larger" group, and you need the boundary between them fast.

- **Running (streaming) median** – `RunningMedian` above. `low` is a max-heap holding the smaller half, `high` a min-heap holding the larger half. Two invariants: every value in `low` ≤ every value in `high`, and `len(low)` is equal to `len(high)` or one more. Then the median is `low`'s top (odd count) or the average of both tops (even count). `add` keeps both invariants by always passing the new value through `low` into `high`, then moving one back if `high` got bigger. Trace for 5, 15, 1, 3:

add	`low` (max-heap)	`high` (min-heap)	median
5	{5}	{}	5
15	{5}	{15}	(5 + 15) / 2 = 10
1	{1, 5}	{15}	5
3	{1, 3}	{5, 15}	(3 + 5) / 2 = 4

- **Maximise capital** (`max_capital` in the script): you may do at most k projects; each needs `cost` capital to start and returns `profit`. One heap holds **locked** projects by cost (min-heap); the other holds **affordable** ones by profit (max-heap). Each round, move every project you can now afford from the first heap to the second, then take the most profitable. It is a greedy choice (19.14) that heaps make O(n log n).
- **Sliding-window median** is the hard follow-up: same two heaps, but values also leave the window. Heaps cannot delete from the middle cheaply, so you use **lazy deletion** – remember "remove x later" in a `Counter` and discard x when it reaches a top.

Trade-offs

Need	Best tool	Cost
k largest once, small k	`heapq.nlargest(k, xs)`	O(n log k)
k close to n	`sorted(xs)[-k:]`	O(n log n)
k-th element, average fast	quickselect	O(n) average, $O(n^2)$ worst
repeated "give me the min" with inserts	heap	O(log n) each
delete arbitrary item	heap + "lazy deletion" (mark, skip when popped)	amortised

Common mistakes

- Assuming the heap list is sorted; forgetting to negate **both** on push and pop in a max-heap.
- Pushing `(priority, dict)` – on a tie, Python tries to compare dicts → `TypeError`. Add a tie-breaker counter.
- `heappush` onto a list that already held unordered items without `heapify`-ing it first.

In the interview

> **30-second answer:** "`heapq` is a min-heap on a list: push/pop O(log n), peek `h[0]` O(1), `heapify` O(n). For a max-heap I push negatives. For top-k I keep a size-k min-heap, O(n log k). Dijkstra, merge-k-lists and running median are all heap problems."

- *Why a min-heap for k largest?* → The root is the weakest of the current top k, so it's the one to evict.
- *Running median complexity?* → O(log n) per insert, O(1) per median.
- *Why not one sorted list with `insort`?* → The search is O(log n) but the insert shifts items, O(n) per add; two heaps are O(log n).

19.6 Sorting with keys; binary search and `bisect`

What it is

- `sorted(iterable, key=..., reverse=...)` returns a new list; `list.sort(...)` sorts in place and returns `None`. Both use **Timsort** – O(n log n) worst case, O(n) on already-sorted input, and **stable** (equal keys keep their original order).
- A **key function** maps each item to the value to compare (`key=len`, `key=str.lower`). Tuples compare element by element, so a tuple key gives multi-level sorting; negate a numeric field for "descending on this field

only". `functools.cmp_to_key` adapts a pairwise comparator for the rare problems where order depends on the *pair* (e.g. "largest number").

- **Binary search** halves a sorted range each step: O(log n). `bisect_left(a, x)` = first index with `a[i] >= x`; `bisect_right(a, x)` = first index with `a[i] > x`; `insort` inserts keeping order. Since 3.10 they accept `key=`.

How it works

Binary search on the answer is the pattern interviewers love: when a yes/no test `ok(v)` is *monotonic* (false, …, false, true, …, true), binary-search the smallest v that is true. Example: the minimum eating speed to finish banana piles in h hours — if speed s works, any faster speed works too.

```
people = [("ana", 31), ("bo", 25), ("cy", 31), ("di", 25)]
print(sorted(people, key=lambda p: (p[1], p[0])))      # age, then name
print(sorted(people, key=lambda p: (-p[1], p[0])))     # age desc, name asc

def merge_intervals(intervals):
    out = []
    for start, end in sorted(intervals):
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])
    return out

def lower_bound(nums, target):        # same as bisect_left
    lo, hi = 0, len(nums)              # half-open [lo, hi)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo

def min_eating_speed(piles, hours):
    def ok(speed):
        return sum(ceil(p / speed) for p in piles) <= hours
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid          # mid works -> try smaller
        else:
            lo = mid + 1
    return lo

a = [1, 3, 3, 3, 7, 9]
assert lower_bound(a, 3) == bisect.bisect_left(a, 3) == 1
assert bisect.bisect_right(a, 3) == 4
assert min_eating_speed([3, 6, 7, 11], 8) == 4
```

Output:

```
by age then name: [('bo', 25), ('di', 25), ('ana', 31), ('cy', 31)]
age desc, name asc: [('ana', 31), ('cy', 31), ('bo', 25), ('di', 25)]
itemgetter(1) (stable): [('bo', 25), ('di', 25), ('ana', 31), ('cy', 31)]
case-insensitive: ['Apple', 'banana', 'cherry']
by length desc: ['banana', 'cherry', 'Apple']
count of 3s via bisect: 3
insort 5: [1, 3, 3, 3, 5, 7, 9]
first age>=31 at index 2
weighted pick shares (expect ~0.17, 0.50, 0.33): [0.17, 0.5, 0.33]
all asserts passed
```

The script also contains `largest_number` (uses `cmp_to_key`: put `a` before `b` if `a+b > b+a` as strings) and `search_rotated` (binary search in a rotated sorted array: one half is always sorted, check whether the target lies in it).

Weighted random selection

Problem: given weights `w = [1, 3, 2]`, return index i with probability `w[i] / sum(w)` (1/6, 3/6, 2/6). Used for load balancing across servers of different sizes and for sampling.

Idea: lay the weights end to end on a number line using **prefix sums**: `[1, 4, 6]` means index 0 owns [0, 1), index 1 owns [1, 4), index 2 owns [4, 6). Draw a random integer r in [0, 6) and find which slot it falls in – the first prefix value **greater than** r, which is exactly `bisect_right`. Building is O(n), each pick O(log n).

```
class WeightedPicker:
    def __init__(self, weights, seed=None):
        self.prefix = list(accumulate(weights))     # [1, 3, 2] -> [1, 4, 6]
        self.rng = random.Random(seed)
    def pick(self):
        r = self.rng.randrange(self.prefix[-1])     # integer in [0, total)
        return bisect.bisect_right(self.prefix, r)
```

60,000 seeded picks gave shares `[0.17, 0.5, 0.33]` (output above). Python also has `random.choices(range(n), weights=w)`, which does the same prefix-sum + bisect internally – mention it, but interviewers usually want the hand-written version. Using `bisect_left` here is an off-by-one: r = 1 would map to index 0.

The two binary-search templates

Template	Loop	Update	Returns
Exact match	`while lo <= hi` on `[lo, hi]`	`lo = mid + 1` / `hi = mid - 1`	index or –1
Boundary (first true)	`while lo < hi` on `[lo, hi)`	`lo = mid + 1` / `hi = mid`	first position where condition holds

Pick one template and use it every time; most binary-search bugs come from mixing them (infinite loops when `lo = mid`).

Common mistakes

- `nums = nums.sort()` sets `nums` to `None`; `reverse=True` reverses the whole key, not one field.
- `bisect` on an unsorted list (no error, just wrong answers); thinking `insort` is O(log n) – the search is, the insertion is O(n).

In the interview

30-second answer: "Python sorts with Timsort, stable, O(n log n). I pass a `key` – tuples for multi-level ordering and negation for descending fields. For binary search I use `bisect_left/right` or a half-open `lo < hi` template, and for 'minimum value that satisfies X' I binary-search the answer space when X is monotonic."

- *Why does stability matter?* → You can sort by a secondary key first, then by the primary; equal primaries keep the secondary order.
- *Is there a TreeMap/sorted set in Python?* → Not built in. Use a sorted list + `bisect` (O(n) inserts), a heap, or the third-party `sortedcontainers.SortedList` (some platforms, such as LeetCode, preinstall it).
- *`bisect_left([1,2,2,3], 2)` and `bisect_right`?* → 1 and 3.
- *Sort strings by length, ties alphabetical?* → `sorted(ws, key=lambda w: (len(w), w))`.

19.7 Linked lists and trees in Python

What it is

Python has no built-in linked list or tree for interviews; you define tiny node classes. A **singly linked list** node has `val` and `next`; a **binary tree** node has `val`, `left`, `right`. A **binary search tree (BST)** additionally keeps every left-subtree value < node < every right-subtree value. `@dataclass` (Chapter 10) writes the `__init__` for you; `from __future__ import annotations` lets a class mention itself in type hints.

How it works — linked lists

- **Dummy head:** start with a fake node so you never special-case "the list is empty" when building or merging.
- **Reversal:** walk with `prev`; point each node back at `prev`. Python's tuple assignment evaluates the whole right side first, so `head.next, prev, head = prev, head, head.next` is a correct one-liner.
- **Fast/slow pointers (Floyd):** `slow` moves 1, `fast` moves 2. When `fast` reaches the end, `slow` is at the middle; if there is a cycle, they eventually meet. Compare nodes with `is` (identity).

```
@dataclass
class ListNode:
    val: int
    next: ListNode | None = None

def reverse(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False

def merge_two(a, b):
    dummy = tail = ListNode(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

How it works — trees

- **DFS traversals:** pre-order (node, left, right), in-order (left, node, right — gives sorted order for a BST), post-order (left, right, node). Recursive versions are 3 lines; iterative versions use an explicit stack.
- **BFS / level order:** a `deque`; process `len(q)` nodes per outer loop to separate levels.
- **Validate a BST:** pass down the allowed `(lo, hi)` range — checking only parent vs child is a classic bug.
- **Lowest common ancestor:** recurse; if both sides return something, the current node is the LCA.

```
@dataclass
class TreeNode:
    val: int
    left: TreeNode | None = None
    right: TreeNode | None = None

def inorder_iterative(root):
    out, stack, node = [], [], root
    while stack or node:
        while node:                 # go left as far as possible
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.append(node.val)
        node = node.right
    return out

def level_order(root):
    if not root:
        return []
    levels, q = [], deque([root])
    while q:
        levels.append([])
        for _ in range(len(q)):
            node = q.popleft()
            levels[-1].append(node.val)
            q.extend(c for c in (node.left, node.right) if c)
    return levels

def max_depth(root):
    return 0 if not root else 1 + max(max_depth(root.left), max_depth(root.right))

def is_valid_bst(root, lo=float("-inf"), hi=float("inf")):
    if not root:
        return True
    if not lo < root.val < hi:
        return False
    return is_valid_bst(root.left, lo, root.val) and is_valid_bst(root.right, root.val, hi)

def lca(root, p, q):
    if root is None or root.val in (p, q):
        return root
    left, right = lca(root.left, p, q), lca(root.right, p, q)
    return root if left and right else left or right
```

`build_tree([3, 9, 20, None, None, 15, 7])` in the script turns LeetCode's level-order list format into nodes so you can test locally. Output:

```
reversed list: [5, 4, 3, 2, 1]
inorder: [9, 3, 15, 20, 7] preorder: [3, 9, 20, 15, 7]
levels: [[3], [9, 20], [15, 7]] depth: 3
serialized: 3,9,#,#,20,15,#,#,7,#,#
all asserts passed
```

Serialize and deserialize a binary tree

Serialize = turn a tree into a string (to store or send it); **deserialize** = rebuild the identical tree. A plain pre-order list of values is not enough – `[1, 2]` could be 2 as a left or a right child. The fix is to also write a marker (`#`) for every missing child. Then pre-order is unambiguous: read a value, build the node, and the next tokens are *exactly* its left subtree followed by its right subtree.

```python
def serialize(root):
    out, stack = [], [root]
    while stack:                        # iterative pre-order
        node = stack.pop()
        if node is None:
            out.append("#"); continue
        out.append(str(node.val))
        stack.append(node.right)        # right first, so left is popped first
        stack.append(node.left)
    return ",".join(out)

def deserialize(data):
    tokens = iter(data.split(","))      # a shared iterator = "read the next token"
    def build():
        tok = next(tokens)
        if tok == "#":
            return None
        node = TreeNode(int(tok))
        node.left = build()
        node.right = build()
        return node
    return build()
```

Both are O(n) time and space; for a tree with n nodes the string has 2n + 1 tokens (n values + n + 1 `#`). The script checks the round trip `deserialize(serialize(t)) == t`. The recursive `build` has the usual depth limit – for very deep trees use the level-order format that `build_tree` reads instead.

Complexity

All traversals are O(n) time. Space is O(h) for DFS (h = tree height: log n if balanced, n if a degenerate "linked-list" tree) and O(w) for BFS (w = widest level, up to n/2).

Common mistakes

- A mutable default like `def f(node, path=[])` in tree recursion (19.19).
- `if node.val:` when the value can be `0` – test `if node:` / `is None`.
- Recursing on a 10^5-deep degenerate tree → `RecursionError`; use the iterative version.

In the interview

30-second answer: "I define small `ListNode` / `TreeNode` classes. For lists: dummy head, fast/slow pointers, in-place reversal with three pointers. For trees: recursive DFS for clarity, an explicit stack if depth can be large, and BFS with a deque for level order. Space is O(height)."

- *Detect where a cycle starts?* → After slow and fast meet, move one pointer to the head; advance both by 1; they meet at the cycle start.
- *Inorder successor in a BST?* → If there's a right child, the leftmost node of it; else the nearest ancestor you reached via a left edge.

19.8 Graphs: adjacency lists, BFS, DFS, Dijkstra, topological sort

What it is

A **graph** is a set of **nodes** (vertices) joined by **edges**, which may be **directed** (one-way) and **weighted** (have a cost). The standard Python representation is an **adjacency list**: a `defaultdict(list)` mapping node → neighbours (or `(neighbour, weight)` pairs). A **grid** is an implicit graph whose cells neighbour the 4 (or 8) adjacent cells.

- **BFS (breadth-first search)** visits nodes in rings of increasing distance using a queue → shortest paths in **unweighted** graphs.
- **DFS (depth-first search)** goes as deep as possible before backtracking (recursion or a stack) → connectivity, cycles, topological order, flood fill.
- **Dijkstra's algorithm** finds shortest paths with **non-negative** weights using a min-heap of `(distance, node)`.
- **Topological sort** orders a **DAG** (directed acyclic graph) so every edge goes from earlier to later — course prerequisites, build order.

How it works

BFS: put the start in a deque and in `visited`; repeatedly pop the front and push unvisited neighbours, **marking them visited when enqueued** (marking on pop lets the same node be enqueued many times).

Dijkstra (lazy version):

1. `dist = {src: 0}`, heap = `[(0, src)]`.
2. Pop the smallest `(d, u)`. If `d` is larger than the best known `dist[u]`, it's a stale entry — skip it.
3. For each edge `u → v` with weight w: if `d + w < dist[v]`, update and push `(d + w, v)`.
4. Complexity O((V + E) log V).

Topological sort, Kahn's algorithm: count in-degrees; start with all nodes of in-degree 0; pop one, append it to the order, decrement its neighbours' in-degrees and enqueue any that reach 0. If the order has fewer than V nodes, there is a cycle.

Topological sort, DFS: three colours — white (unvisited), grey (on the current path), black (finished). Meeting a grey node means a back edge = cycle. Append each node when it finishes (post-order) and reverse at the end.

```python
def bfs_shortest_path(g, start, goal):
    parent = {start: None}          # doubles as the visited set
    q = deque([start])
    while q:
        node = q.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nxt in g.get(node, []):
            if nxt not in parent:   # mark when ENQUEUED
                parent[nxt] = node
                q.append(nxt)
    return None

def dijkstra(g, src):
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, float("inf")):
            continue                    # stale entry
        for v, w in g.get(u, []):
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist

def topo_sort_kahn(n, prereqs):         # (a, b): b before a
    g = defaultdict(list)
    indeg = [0] * n
    for a, b in prereqs:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else None   # None -> cycle
```

The script also has `dfs_recursive`, `dfs_iterative` (pushes neighbours in reverse so the order matches the recursive one), `num_islands` (grid flood fill with a stack and the four direction offsets), and `topo_sort_dfs`. Output:

```
adjacency: {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C', 'E'], 'E': ['D']}
dfs recursive: ['A', 'B', 'D', 'C', 'E'] | iterative: ['A', 'B', 'D', 'C', 'E']
dijkstra from A: {'A': 0, 'B': 3, 'C': 1, 'D': 4}
course order (Kahn): [0, 1, 2, 3] | DFS: [0, 2, 1, 3]
recursion limit: 1000
depth 5000 -> RecursionError
after setrecursionlimit(10_000): depth 5000
all asserts passed
```

Both topological orders are valid — a DAG usually has several.

The recursion limit

CPython refuses to nest calls deeper than `sys.getrecursionlimit()` (1000 by default) and raises `RecursionError`. Python has **no tail-call optimisation**, so even tail-recursive code counts. Options:

1. Prefer the **iterative** version (explicit stack/deque) for graphs/trees that may be deep — safest.
2. `sys.setrecursionlimit(10**6)` — accepted on many judges, but very deep recursion can still crash the interpreter by exhausting the real C stack in some situations, so treat it as a workaround, not a fix.

Grid template

```
for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == "1" and (nr, nc) not in seen:
        ...
```

Python's chained comparison `0 <= nr < rows` keeps bounds checks readable.

Trade-offs

Problem	Algorithm	Complexity
Shortest path, unweighted	BFS	O(V + E)
Shortest path, weights 0/1	0-1 BFS (deque, `appendleft` for 0-edges)	O(V + E)
Shortest path, non-negative weights	Dijkstra	O((V + E) log V)
Negative weights	Bellman–Ford	O(V·E)
All pairs, small V	Floyd–Warshall	$O(V^3)$
Order with dependencies	Topological sort	O(V + E)
Connected components, dynamic unions	Union-Find (19.9)	~O(α(n)) per op

Common mistakes

- Marking visited on pop in BFS (duplicates in the queue, sometimes TLE); a list instead of a set for `visited`.
- Using Dijkstra with negative edges (wrong answers).
- Forgetting isolated nodes that never appear in an edge list (loop over `range(n)`, not just over `g`).

In the interview

30-second answer: "I store graphs as `defaultdict(list)` adjacency lists. Unweighted shortest path → BFS with a deque, marking visited on enqueue. Weighted non-negative → Dijkstra with heapq, skipping stale entries. Dependencies → topological sort with Kahn's algorithm, which also detects cycles. For deep graphs I use iterative DFS because of Python's recursion limit."

- *Why doesn't Dijkstra work with negative edges?* → It finalises a node when popped assuming no later path can be shorter; a negative edge can break that.
- *Detect a cycle in an undirected graph?* → DFS that ignores the edge back to the parent, or union-find (an edge joining two nodes already in the same set).

19.9 Union-Find (disjoint set union)

What it is

Union-Find (a.k.a. **DSU**, disjoint set union) keeps track of which items belong to the same group while groups merge. It supports two operations: `find(x)` → the representative ("root") of x's group, and `union(a, b)` → merge the two groups. Analogy: companies merging — to know whether two employees work for the same firm, follow "who owns whom" up to the top parent company.

How it works

1. `parent[i] = i` initially (everyone is their own root).
2. `find` follows parents to the root. **Path compression** then points every node on that path directly at the root, flattening the tree.
3. `union` finds both roots; if different, attach the **smaller** tree under the larger (**union by size**) and decrement the component count. Return `False` if already joined — that is how you detect a cycle-closing edge.
4. With both optimisations, each operation is $O(\alpha(n))$ amortised, where α is the inverse Ackermann function — ≤ 4 for any realistic n, so "effectively constant".

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.components = n

    def find(self, x):
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:          # path compression (iterative)
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.components -= 1
        return True

def kruskal_mst(n, edges):              # edges (weight, u, v)
    dsu, total = DSU(n), 0
    for w, u, v in sorted(edges):
        if dsu.union(u, v):
            total += w
    return total

assert kruskal_mst(4, [(1, 0, 1), (4, 0, 2), (3, 1, 2), (2, 2, 3), (5, 1, 3)]) == 6
```

Output: `parent: [0, 0, 0, 3, 3, 5] components: 3` then `all asserts passed`. The script also solves "number of connected components" and "redundant connection" (the first edge whose `union` returns `False`). The iterative `find` avoids the recursion limit; in `self.parent[x], x = root, self.parent[x]` the right side (the old parent) is evaluated before `x` is rebound.

When not to use it

Union-Find cannot **split** groups and cannot give you paths — use BFS/DFS for those. For a one-off "count components" on a static graph, BFS/DFS is equally good.

In the interview

> **30-second answer:** "Union-Find tracks groups under merging. `find` with path compression and `union` by size give near-constant amortised time. I use it for connected components, cycle detection in undirected graphs, Kruskal's MST, and 'accounts merge'-type problems."

- *Why union by size?* → Keeps trees shallow (height O(log n) even without compression).
- *Kruskal's complexity?* → O(E log E) for the sort; the DSU part is near-linear.

19.10 Dynamic programming: `@cache` and tabulation

What it is

Dynamic programming (DP) combines answers to smaller **overlapping subproblems**, computing each once. It applies when the problem has **optimal substructure** (the best answer is built from best answers to subproblems).

- **Top-down (memoisation):** write the natural recursion, then cache it with `functools.cache` (3.9+) or `lru_cache(maxsize=None)`.
- **Bottom-up (tabulation):** fill an array from the smallest subproblems upward with loops; no recursion, and you can often keep only the last row.

How it works — the recipe

1. **State:** what parameters describe a subproblem? (`i` = index, `(i, j)` = prefixes of two strings, `(i, capacity)` …)
2. **Transition:** how does a state's answer come from smaller states?
3. **Base cases.**
4. **Order** (bottom-up) or **memoise** (top-down).
5. **Answer location** (`dp[n]`, `dp[m][n]`, `max(dp)` …).

Worked example — coin change, coins `[1, 2, 5]`, amount 11: `dp[a] = 1 + min(dp[a − c])` over coins `c ≤ a`, `dp[0] = 0`. `dp[5] = 1` (one 5), `dp[10] = 2`, `dp[11] = 1 + min(dp[10], dp[9], dp[6]) = 1 + 2 = 3`.

```python
from functools import cache

@cache
def climb(n):                    # 1 or 2 steps at a time
    return 1 if n <= 1 else climb(n - 1) + climb(n - 2)

def coin_change(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1

def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]    # NOT [[0]*(n+1)]*(m+1)
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

def edit_distance(a, b):
    @cache
    def go(i, j):
        if i == len(a):
            return len(b) - j
        if j == len(b):
            return len(a) - i
        if a[i] == b[j]:
            return go(i + 1, j + 1)
        return 1 + min(go(i + 1, j), go(i, j + 1), go(i + 1, j + 1))
    return go(0, 0)

def knapsack_01(weights, values, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(weights, values):
        for c in range(cap, w - 1, -1):    # backwards: each item used once
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]
```

Output:

```
climb.cache_info(): CacheInfo(hits=8, misses=11, maxsize=None, currsize=11)
coin_change([1,2,5], 11) = 3
edit_distance('horse','ros') = 3
knapsack = 9
all asserts passed
```

`cache_info()` shows only 11 real computations for `climb(10)` – without the cache the naive recursion makes 177 calls, and the count grows exponentially with n.

The script also includes `house_robber` (O(1)-space DP with two variables), `unique_paths` (one-row grid DP) and `lis_length` – the O(n log n) longest increasing subsequence that keeps `tails[k]` = smallest tail of an increasing run of length k+1 and uses `bisect_left` to place each number.

Common DP families

Family	State	Examples
1-D sequence	`dp[i]`	climbing stairs, house robber, decode ways, LIS
Unbounded choice	`dp[amount]`	coin change, combination sum count
0/1 knapsack	`dp[capacity]` iterated backwards	subset sum, partition equal subset
Two strings	`dp[i][j]`	LCS, edit distance, regex/wildcard matching
Grid	`dp[r][c]`	unique paths, minimum path sum
Intervals	`dp[i][j]` over lengths	burst balloons, palindrome partitioning

Top-down vs bottom-up

	`@cache` recursion	Tabulation
Writing it	closest to the recurrence; quick	must choose iteration order
Computes	only reachable states	all states
Limits	recursion limit; function-call overhead	none
Memory tricks	hard	easy to keep one row (O(n) instead of O(n·m))

Common mistakes

- `@cache` on a function taking a **list** → `TypeError: unhashable type`. Pass tuples or indices.
- A module-level `@cache` function reused across test cases with different global data returns stale answers — define the cached helper **inside** the solving function (as `edit_distance` does) or call `.cache_clear()`.
- Building 2-D tables with `[[0] * n] * m` (all rows are the same list — see 19.19).
- Iterating 0/1 knapsack capacity **forwards** (turns it into unbounded knapsack).

In the interview

> **30-second answer:** "I first write the recurrence and state, then memoise it with `functools.cache` — that's top-down. If recursion depth or memory is a concern I convert to a bottom-up table and often compress it to one row. Complexity is number of states × work per state."

- *Complexity of edit distance?* → O(m·n) states × O(1) = O(m·n) time; O(m·n) space top-down, O(min(m, n)) with rolling rows.
- *`lru_cache` vs `cache`?* → `cache` is exactly `lru_cache(maxsize=None)`: unbounded, and cheaper than a bounded LRU because it never tracks recency; `lru_cache(maxsize=k)` evicts least-recently-used entries.
- *How to see how many states `@cache` stored?* → `f.cache_info().currsize`.

19.11 Backtracking

What it is

Backtracking builds candidate solutions step by step and abandons ("prunes") a partial candidate as soon as it cannot lead to a valid answer. It is DFS over a tree of choices. Template: **choose → explore → un-choose**.

How it works

1. Keep a mutable `path` (current partial solution) and maybe a `used` array/sets for constraints.
2. At each level, loop over the available choices; skip invalid ones (prune).
3. Append the choice, recurse, then pop it to restore the state.

4. When the path is complete, record **a copy** (`path[:]`).

```
def subsets(nums):
    out, path = [], []
    def go(start):
        out.append(path[:])              # copy! path keeps changing
        for i in range(start, len(nums)):
            path.append(nums[i])         # choose
            go(i + 1)                    # explore
            path.pop()                   # un-choose
    go(0)
    return out

def combination_sum(cands, target):      # reuse allowed
    cands = sorted(cands)
    out, path = [], []
    def go(start, remaining):
        if remaining == 0:
            out.append(path[:])
            return
        for i in range(start, len(cands)):
            if cands[i] > remaining:
                break                    # prune: later ones are bigger
            path.append(cands[i])
            go(i, remaining - cands[i])  # i, not i+1: reuse allowed
            path.pop()
    go(0, target)
    return out

def n_queens(n):
    cols, diag, anti = set(), set(), set()
    def go(r):
        if r == n:
            return 1
        count = 0
        for c in range(n):
            if c in cols or r - c in diag or r + c in anti:
                continue
            cols.add(c); diag.add(r - c); anti.add(r + c)
            count += go(r + 1)
            cols.remove(c); diag.remove(r - c); anti.remove(r + c)
        return count
    return go(0)
```

Output:

```
subsets: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
combination_sum([2,3,6,7], 7): [[2, 2, 3], [7]]
8-queens solutions: 92
all asserts passed
```

The tests check `subsets` / `permute` against `itertools.combinations` / `permutations`, and `word_search` marks grid cells visited by overwriting them with `"#"` and restoring afterwards.

`itertools` shortcuts

When you just need to *enumerate*, the standard library already does it (Chapter 11): `permutations(xs, r)`, `combinations(xs, r)`, `combinations_with_replacement`, `product(xs, repeat=k)`. Interviewers usually want the hand-written version when pruning matters, but mentioning these shows fluency.

Complexity

Exponential by nature: subsets $O(2^n \cdot n)$, permutations $O(n! \cdot n)$. Pruning shrinks the real work but not the worst case.

Common mistakes

- Appending `path` instead of `path[:]` – every stored result becomes the same (final, usually empty) list.
- Forgetting to undo the choice (sets/`used` flags drift).
- Duplicates in input (e.g. subsets II): sort first and skip `nums[i] == nums[i-1]` when `i > start`.

In the interview

> **30-second answer:** "Backtracking is DFS over choices: choose, recurse, undo. I store a copy of the path at the leaves and prune early – sort candidates so I can break, and use sets for O(1) constraint checks, like columns and diagonals in N-Queens."

- *Why `r - c` and `r + c` for diagonals?* → Cells on the same "" diagonal share r – c; on the same "/" diagonal they share r + c.
- *How many subsets does a 4-element set have?* → $2^4 = 16$.

19.12 Bit tricks and math

What it is

Python `int`s are arbitrary-precision and behave like infinite two's-complement numbers for bitwise operators: `&` (and), `|` (or), `^` (xor), `~` (not: `~x == -x - 1`), `<<`, `>>` (arithmetic shift, floors for negatives). The `math` module and built-in `pow` cover interview number theory.

Bit cheat table (x = 180 = 0b10110100, from the real output)

Trick	Expression	Result
test bit i	`x >> i & 1`	bit 2 → 1
set bit i	`x | (1 << i)`	set bit 0 → `0b10110101`
clear bit i	`x & ~(1 << i)`	
toggle bit i	`x ^ (1 << i)`	
lowest set bit	`x & -x`	4
clear lowest set bit	`x & (x - 1)`	`0b10110000`
power of two?	`x > 0 and x & (x - 1) == 0`	
count 1-bits	`x.bit_count()` (3.10+) or `bin(x).count("1")`	4
bits needed	`x.bit_length()`	8
format	`f"{x:08b}"`, `bin(x)`, `int("1011", 2)`	

XOR identities power "single number": `a ^ a = 0`, `a ^ 0 = a`, order doesn't matter. A bitmask `mask` from `0` to $2^n - 1$ enumerates all subsets (bit i set ⇒ item i chosen) – the basis of bitmask DP.

Math toolkit

Need	Python
gcd / lcm	`math.gcd(a, b)`, `math.lcm(a, b)` (3.9+, both accept many args)
modular power	`pow(b, e, m)` – fast square-and-multiply, O(log e)
modular inverse (m prime or gcd = 1)	`pow(a, -1, m)` (3.8+)
integer square root	`math.isqrt(n)` – exact, no float error
combinations	`math.comb(n, k)`, `math.perm(n, k)`, `math.factorial(n)`
floor/ceil division	`a // b`, `-(-a // b)` or `math.ceil(a / b)` (float – careful with huge ints)
quotient and remainder	`divmod(a, b)`
infinity	`float("inf")`, `math.inf`; `sys.maxsize` is just a big int
big modulus	`MOD = 10**9 + 7`

Output of the script (excerpt):

```
~5 = -6  (two's complement: -(5+1))
-7 >> 1 = -4  (arithmetic shift floors)
gcd(84, 36) = 12 | lcm(4, 6) = 12
pow(2, 100, MOD) = 976371285
comb(5,2) = 10 | perm(5,2) = 20 | factorial(5) = 120
isqrt(10**20 + 1) = 10000000000
primes <= 30: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
2**100 = 1267650600228229401496703205376 (arbitrary precision)
divmod(-7, 2) = (-4, 1) | math.fmod(-7, 2) = -1.0
```

The script also writes `fast_pow` by hand (square-and-multiply: square the base each step and multiply it into the result when the current exponent bit is 1) and a Sieve of Eratosthenes (cross out multiples starting at i^2, O(n log log n)).

Common mistakes

- Expecting 32-bit overflow: problems that say "return the 32-bit result" need masking, e.g. `x & 0xFFFFFFFF` and converting back if the top bit is set.
- `~x` is not "flip bits of an unsigned number"; it is `-x - 1`.
- Using `math.sqrt` on big integers (float precision) – use `math.isqrt`.
- `(a ** b) % m` builds a giant number first – use `pow(a, b, m)`.

In the interview

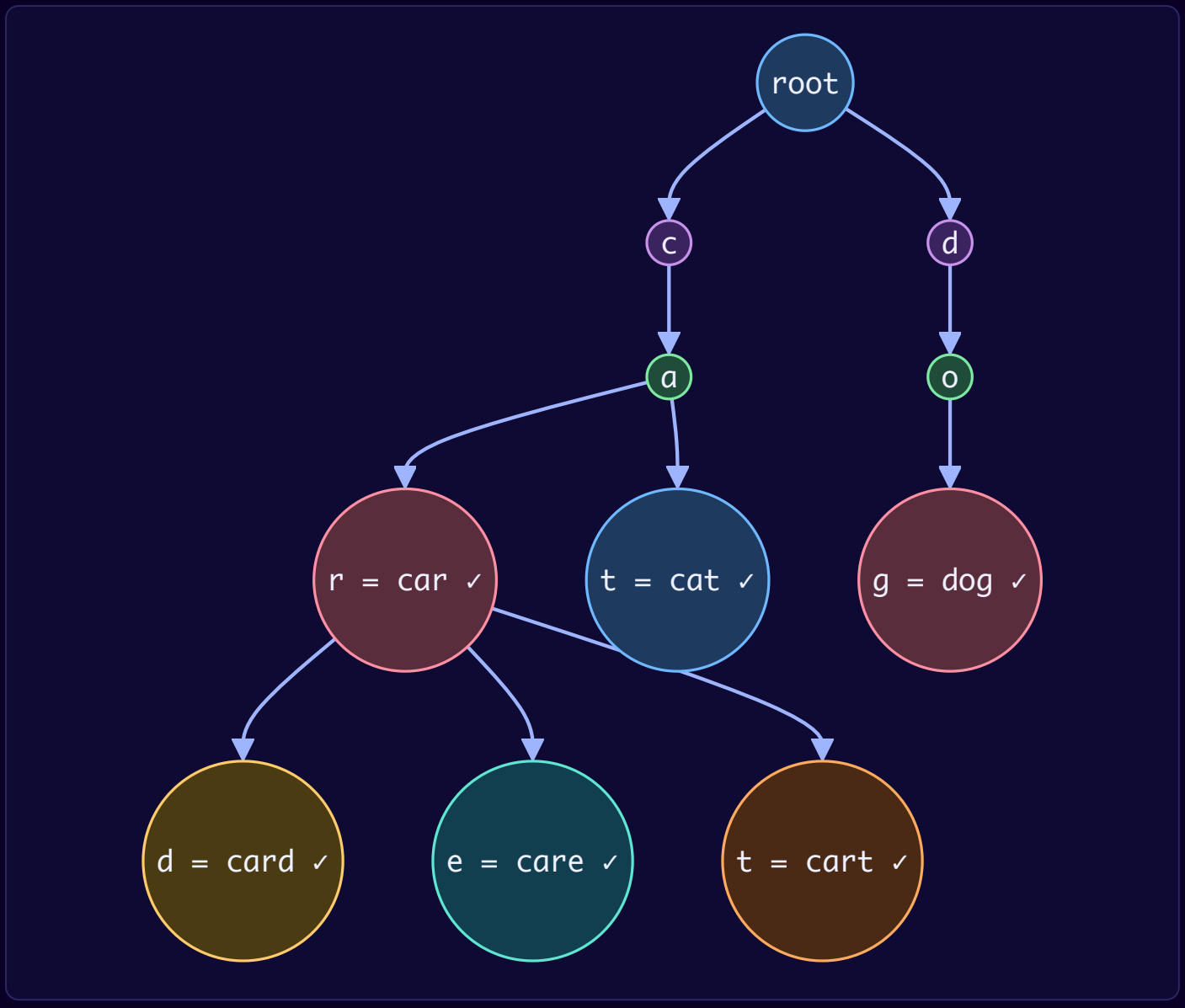
30-second answer: "Python ints never overflow, so I use `pow(b, e, m)` for modular exponentiation, `math.gcd`, `math.isqrt` and `math.comb` instead of hand-rolling. For bits: `x & (x-1)` drops the lowest set bit, `x & -x` isolates it, XOR cancels pairs."

- *Check power of four?* → Power of two and the single bit sits at an even position: `n > 0 and n & (n-1) == 0 and n & 0x55555555 != 0` (32-bit range), or replace the mask test with `n.bit_length() % 2 == 1` for any size. (In Python `&` binds tighter than `==`, unlike C, so no extra parentheses are needed.)
- `6 & 5`, `6 | 5`, `6 ^ 5`? → 4, 7, 3.
- *Modular inverse of 3 mod 7?* → `pow(3, -1, 7) = 5` ($3 \cdot 5 = 15 \equiv 1$).

19.13 Tries (prefix trees)

What it is

A **trie** (say "try", from re*trie*val) is a tree that stores strings letter by letter. Each node represents a **prefix** (the first few letters of some word); each edge adds one letter; a flag on the node says "a whole word ends here". All words that start with "car" share the path c → a → r, so a trie answers **"which words start with this prefix?"** by walking the prefix once — no scanning of the whole word list.



The problem it solves

With a `set` of words, "is `car` a word?" is O(L) (L = word length, for hashing), but "does any word start with `ca`?" means checking every word — O(n·L). A trie makes both O(L), and it lets a search **stop early** the moment a path is not a prefix of anything, which is what makes word search on a board fast.

How it works

In Python a node is just a `dict` of children plus a field for the word that ends there. `setdefault(ch, TrieNode())` walks or creates in one step.

```python
class TrieNode:
    __slots__ = ("children", "word")        # many small nodes -> save memory
    def __init__(self):
        self.children = {}                   # letter -> TrieNode
        self.word = None                     # full word if one ends here

class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.word = word
    def _walk(self, s):
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node
    def search(self, word):
        node = self._walk(word)
        return node is not None and node.word is not None
    def starts_with(self, prefix):
        return self._walk(prefix) is not None
```

Problem 1 — autocomplete. Walk to the prefix node, then DFS below it, visiting children in alphabetical order, and collect the first `limit` words (`words_with_prefix` in the script).

Problem 2 — search with a wildcard. `.` matches any single letter. At a normal letter follow one child; at a `.` try **every** child (`any(...)` stops at the first success):

```python
def search_wildcard(self, pattern):
    def dfs(node, i):
        if i == len(pattern):
            return node.word is not None
        if pattern[i] == ".":
            return any(dfs(child, i + 1) for child in node.children.values())
        child = node.children.get(pattern[i])
        return child is not None and dfs(child, i + 1)
    return dfs(self.root, 0)
```

Problem 3 — word search on a board. Given a letter grid and a list of words, find every word you can trace through horizontally/vertically adjacent cells without reusing a cell. Running a separate grid search per word repeats work; instead put all words in one trie and run **one backtracking DFS from each cell that walks the trie at the same time**. A path that leaves the trie is abandoned at once.

```python
def find_words(board, words):
    trie = Trie()
    for w in words:
        trie.insert(w)
    rows, cols, found = len(board), len(board[0]), []

    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent.children.get(ch)
        if node is None:
            return                          # no word continues this way
        if node.word is not None:
            found.append(node.word)
            node.word = None                # report each word once
        board[r][c] = "#"                   # mark visited in place
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                dfs(nr, nc, node)
        board[r][c] = ch                    # undo (backtracking, 19.11)
        if not node.children and node.word is None:
            del parent.children[ch]         # prune a finished branch

    for r in range(rows):
        for c in range(cols):
            dfs(r, c, trie.root)
    return sorted(found)
```

Output of `13_tries.py`:

```
autocomplete 'car': ['car', 'card', 'care']
wildcard 'ca.e': True | '.o': False
words on board: ['eat', 'oat', 'oath']
all asserts passed
```

Complexity

Operation	Time	Notes
insert / search / starts_with	O(L)	L = length of the word or prefix
wildcard search	O(L) with no dots; up to O(26^d · L) with d dots	every dot branches
build for n words	O(total letters)	space: up to one node per letter
word search on an r × c board, max word length L	O(r·c·4·3^(L−1)) worst case	pruning makes it far faster in practice

When not to use it

- Only exact lookups ("is this a word?") → a `set` is simpler and uses less memory.
- Only one prefix query on a static list → `sorted` + `bisect` on the prefix is enough.

Common mistakes

- Treating "the path exists" as "the word exists" — `search("ca")` must be `False` when only "car" was inserted; that is what the end-of-word field is for.
- Word search: forgetting to restore the cell after the DFS, or reporting the same word twice when two paths spell it (clear `node.word` after the first find).
- Using `defaultdict(TrieNode)`-style auto-creation in `search` — a *read* then creates nodes and the trie grows on every miss.

In the interview

> **30-second answer:** "A trie stores words letter by letter; each node is a prefix, so insert, search and prefix checks are O(L). In Python a node is a dict of children plus an end-of-word marker. For a wildcard I branch into every child; for word search on a board I put all words in one trie and backtrack from every cell while walking the trie, pruning as soon as the path isn't a prefix."

- *Why use a trie instead of a hash set of all prefixes?* → A set of all prefixes costs O(total L^2) memory and cannot enumerate "words under this prefix"; a trie shares common prefixes.
- *How to return the top 3 suggestions by popularity?* → Store at each node the top-3 words for its prefix (updated on insert), making a lookup O(L).
- *Memory is too high?* → Use a **radix tree** (compress chains of single-child nodes into one edge), or an array of 26 children instead of a dict for lowercase-only input.

19.14 Greedy

What it is

A **greedy algorithm** builds the answer one step at a time and at each step takes the choice that looks best **right now**, never going back. It is correct only when you can argue that the local best choice never blocks the global best – typically with an **exchange argument**: "take any optimal answer; swapping its first choice for the greedy choice keeps it optimal."

The problem it solves

Many optimisation problems look like they need DP (19.10) over every combination. When a greedy rule is provably safe, the answer drops to a single pass, O(n) or O(n log n) after sorting, with O(1) extra space.

How it works

There is no single template; look for "keep only the best thing seen so far" or "sort, then take whatever still fits". Four classics:

Problem 1 – jump game. `nums[i]` is the maximum jump length from index i. *Can you reach the last index?* Track `farthest`, the furthest index reachable so far; if you ever stand beyond it, you are stuck. *Fewest jumps?* Think of BFS levels (19.8) without a queue: all indices reachable with j jumps form a range; when you reach the end of the current range you must jump once more, and the new range ends at the farthest point seen.

```python
def can_jump(nums):
    farthest = 0
    for i, n in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + n)
    return True

def min_jumps(nums):
    jumps = end = farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == end:                # finished this level -> jump again
            jumps += 1
            end = farthest
    return jumps
```

Problem 2 – gas station. Stations on a circle; station i gives `gas[i]` and driving to the next costs `cost[i]`. Find the start index that completes the loop (–1 if none). Two facts: if total gas < total cost, no start works; and if the tank goes negative at station i after starting at s, then **no** station between s and i can

work either: starting at one of them means arriving there with an empty tank, whereas the run from s arrived with a tank ≥ 0 and still failed at i. So jump the start to i + 1.

```
def gas_station(gas, cost):
    if sum(gas) < sum(cost):
        return -1
    start = tank = 0
    for i, (g, c) in enumerate(zip(gas, cost)):
        tank += g - c
        if tank < 0:
            start, tank = i + 1, 0
    return start
```

Problem 3 — candy. Children stand in a line with ratings; each gets at least one candy and a child with a higher rating than a neighbour must get more than that neighbour. Minimise the total. One left-to-right pass satisfies the "left neighbour" rule, one right-to-left pass the "right neighbour" rule, and taking the `max` keeps both.

```
def candy(ratings):
    n = len(ratings)
    c = [1] * n
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            c[i] = c[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            c[i] = max(c[i], c[i + 1] + 1)
    return sum(c)
```

Problem 4 — interval scheduling (`max_non_overlapping`): to fit the most non-overlapping talks in one room, sort by **end** time and take every talk that starts after the last one you took. The talk that ends first leaves the most room for the rest — that is the exchange argument. Sorting by start time or by length both fail on small counter-examples.

Output of `14_greedy.py`:

```
can_jump [3,2,1,0,4]: False | min_jumps [2,3,1,1,4]: 2
gas station start: 3
candy [1,3,4,5,2]: 11
max non-overlapping talks: 4
all asserts passed
```

Trade-offs

	Greedy	DP
Speed	usually O(n) or O(n log n)	states × work per state
Correctness	needs a proof; a plausible rule is often wrong	follows from the recurrence
Example of greedy failing	coins {1, 3, 4}, amount 6: greedy 4+1+1 = 3 coins; best 3+3 = 2	DP gets 2

Common mistakes

- Using a greedy rule without checking a small counter-example (the coin example above). If you cannot justify it in one sentence, consider DP.
- Candy: doing only one pass, or setting `c[i] = c[i+1] + 1` in the second pass and breaking the first pass's result (use `max`).
- Min jumps: looping to the last index inclusive, which counts one extra jump.

In the interview

> **30-second answer:** "Greedy takes the locally best choice and never revisits it, so it's usually one pass after a sort. I only use it when I can justify it – typically an exchange argument, like 'the interval that ends first never hurts'. Otherwise I fall back to DP. Jump game tracks the farthest reachable index; gas station resets the start when the tank goes negative; candy is two passes."

- *How do you know greedy is right?* → Exchange argument, or "greedy stays ahead": after every step, greedy's partial answer is at least as good as any other.
- *Jump game with minimum jumps – why is it O(n) and not BFS O(n²)?* → Each level is a contiguous range, so you track only its end and the farthest reach.

19.15 Intervals

What it is

An **interval** is a pair `[start, end]` – a meeting, a booking, a range of IDs. Interval problems ask whether they overlap, how to merge them, or how many are active at once. Two intervals `[a, b]` and `[c, d]` **overlap** when `max(a, c) <= min(b, d)` (use `<` instead of `<=` if touching ends do not count – ask).

The problem it solves

Comparing every pair is O(n²). **Sorting** puts overlapping intervals next to each other, so most problems become one O(n) pass after an O(n log n) sort. Which key to sort by matters: **start** time for merging, **end** time for "keep as many as possible" (19.14).

How it works

Problem 1 – merge. Sort by start; each interval either overlaps the last kept one (extend its end) or starts a new one. (This is the snippet from 19.6.)

Problem 2 – insert into a sorted, non-overlapping list. No re-sort needed: copy the intervals that end before the new one starts, absorb every interval that overlaps it, then copy the rest. O(n).

```
def insert(intervals, new):
    out, i, n = [], 0, len(intervals)
    s, e = new
    while i < n and intervals[i][1] < s:          # entirely before
        out.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= e:         # overlapping -> absorb
        s, e = min(s, intervals[i][0]), max(e, intervals[i][1]); i += 1
    out.append([s, e])
    out.extend(intervals[i:])                     # entirely after
    return out
```

Problem 3 – intersection of two sorted lists. Two pointers (19.2): the overlap of the current pair is `[max(starts), min(ends)]` if non-empty; then advance whichever interval **ends first**, because it cannot overlap anything later in the other list.

Problem 4 – minimum meeting rooms. How many meetings run at the same time at the peak? Two standard answers:

```python
def min_rooms_heap(meetings):          # heap of end times of meetings in progress
    ends = []
    for start, end in sorted(meetings):
        if ends and ends[0] <= start:  # the earliest-ending room is free again
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)

def min_rooms_sweep(meetings):         # sweep line: +1 at a start, -1 at an end
    events = sorted([(s, 1) for s, _ in meetings] + [(e, -1) for _, e in meetings])
    return max(accumulate(d for _, d in events), default=0)
```

The **sweep line** idea: turn each interval into two events and walk them in time order, keeping a running count. Sorting the tuples puts `(t, -1)` before `(t, 1)`, so a meeting ending at 10 frees its room for one starting at 10.

Output of `15_intervals.py`:

```
merged: [[1, 6], [8, 10], [15, 18]]
insert [4,8]: [[1, 2], [3, 10], [12, 16]]
intersection: [[1, 2], [5, 5], [8, 10], [15, 23], [24, 24], [25, 25]]
rooms needed: 2
all asserts passed
```

The script also has `erase_overlaps` (fewest intervals to delete so the rest don't overlap = n − the greedy maximum from 19.14).

Trade-offs

Question	Sort by	Technique	Cost
merge / union	start	extend last kept	O(n log n)
insert one	already sorted	three phases	O(n)
overlap of two lists	already sorted	two pointers	O(m + n)
max active at once	start (heap) or event time (sweep)	min-heap of ends / running sum	O(n log n)
keep the most / delete the fewest	end	greedy	O(n log n)

Common mistakes

- Not asking whether `[1, 4]` and `[4, 5]` overlap; the `<=` vs `<` choice changes the answer.
- Merge: `out[-1][1] = end` instead of `max(out[-1][1], end)` — breaks when an interval is fully inside the previous one (`[1, 10]`, `[2, 3]`).
- Sweep line: letting starts sort before ends at the same time, which over-counts rooms.

In the interview

> **30-second answer:** "I sort the intervals — by start to merge or count overlaps, by end to keep as many as possible — and then make one pass. Two intervals overlap when max of starts ≤ min of ends. For meeting rooms I use a min-heap of end times or a sweep line of +1/−1 events; both are O(n log n)."

- *Which room does each meeting get?* → Keep `(end, room_id)` in the heap; reuse the popped room's id.
- *Intervals arrive as a stream?* → Keep them in a sorted structure keyed by start (`sortedcontainers.SortedList` or `bisect` on a list) and merge with neighbours on insert.

19.16 Fast & slow pointers

What it is

Two pointers walk the **same** sequence at different speeds: `slow` moves one step, `fast` moves two. This is **Floyd's cycle-finding algorithm** (also called "tortoise and hare"). It works on anything where "next" is defined — a linked list, or an implicit sequence like x → f(x) — using O(1) extra memory. 19.7 introduced it for the middle and a yes/no cycle check; here are the full set of uses.

The problem it solves

Detecting a loop or finding a midpoint usually needs a `set` of visited nodes (O(n) memory) or a first pass to count the length. Fast/slow does both in one pass with two variables.

How it works

- **Middle:** when `fast` reaches the end, `slow` has done half the steps. For an even length this returns the second middle.
- **Cycle start:** if there is a loop, `fast` gains one step per move on `slow` inside it, so they must meet. Then put one pointer back at the head and move both one step at a time: they meet exactly at the loop's entry. (Why: let a = distance from head to the entry, and let the meeting point be b steps into a loop of length c. `fast` walked twice as far as `slow`, which gives a = k·c − b for some whole number k. So a steps from the head and a steps from the meeting point land on the same node, the entry.)

```
def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:                    # `is`: same node, not equal values
            slow = head
            while slow is not fast:
                slow, fast = slow.next, fast.next
            return slow
    return None
```

Problem 1 — palindrome linked list in O(1) space. Find the middle, reverse the second half in place (19.7's reversal), compare the halves, then reverse it back so the caller's list is unchanged.

Problem 2 — happy number. Replace n by the sum of the squares of its digits (19 → $1^2 + 9^2$ = 82 → 68 → 100 → 1). A number is **happy** if this reaches 1. The sequence always either hits 1 or loops forever, so treat `f(n)` as "next" and run Floyd:

```
def is_happy(n):
    def f(x):
        return sum(int(d) ** 2 for d in str(x))
    slow, fast = n, f(n)
    while fast != 1 and slow != fast:
        slow, fast = f(slow), f(f(fast))
    return fast == 1
```

Problem 3 — find the duplicate number. An array of n + 1 values, each in 1..n, one repeated, read-only, O(1) space. Treat index i → `nums[i]` as a "next" pointer: two indices point to the same value, so that value is the entry of a cycle — `cycle_start` on an array (`find_duplicate` in the script).

Output of `16_fast_slow_pointers.py`:

```
cycle starts at value: 2
palindrome [1,2,3,2,1]: True
happy numbers <= 30: [1, 7, 10, 13, 19, 23, 28]
all asserts passed
```

Complexity

All of these are O(n) time and O(1) extra space (n = list length, or the number of distinct values before the sequence repeats).

Common mistakes

- Loop condition `while fast.next and fast` – checks `fast.next` before knowing `fast` isn't `None`. Always `while fast and fast.next`.
- Comparing nodes with `==` on a `@dataclass` node: dataclasses generate a value-based `__eq__`, so two different nodes with equal fields compare equal (and on a cyclic list the comparison may never finish). Use `is`, or declare the node with `@dataclass(eq=False)` as the script does.
- Palindrome check that leaves the list half-reversed – interviewers ask "did you modify the input?"

In the interview

> **30-second answer:** "Slow moves one, fast moves two. When fast ends, slow is at the middle; if they meet, there's a cycle, and resetting one to the head and stepping both by one finds the entry. It's O(n) time and O(1) space, and it also works on implicit sequences – happy number, or find-the-duplicate where index → value is the next pointer."

- *Cycle length?* → After they meet, keep one still and count steps until the other returns.
- *Why not a set of visited nodes?* → Also O(n) time and simpler, but O(n) memory; mention it as the first solution, then optimise.

19.17 Sort & search: quicksort, quickselect, Dutch flag, merge sort on a list

What it is

You will normally call `sorted` (19.6), but interviewers still ask for the ideas inside sorting algorithms because they solve other problems:

- **Partition** – rearrange an array around a **pivot** value so smaller items come first and larger items after. It is the core of **quicksort** (partition, then sort both sides) and **quickselect** (partition, then continue into **one** side to find the k-th element).
- **Three-way partition** (Dutch national flag) – sort an array of only three distinct values in one pass.
- **Merge sort** – split in half, sort each half, merge. On a linked list it needs no extra arrays, which is why it is the list sorting algorithm.

The problem it solves

"Find the k-th largest" by sorting costs O(n log n); a heap (19.5) costs O(n log k); quickselect costs **O(n) on average** because each partition throws away one side. "Sort 0s, 1s and 2s" by counting takes two passes; the Dutch flag takes one.

How it works

Lomuto partition with a random pivot (a random pivot makes the $O(n^2)$ worst case – always picking the smallest item – extremely unlikely):

```python
def partition(nums, lo, hi, rng):
    p = rng.randint(lo, hi)
    nums[p], nums[hi] = nums[hi], nums[p]    # move the pivot to the end
    pivot, store = nums[hi], lo
    for i in range(lo, hi):
        if nums[i] < pivot:                  # grow the "smaller" region [lo, store)
            nums[i], nums[store] = nums[store], nums[i]
            store += 1
    nums[store], nums[hi] = nums[hi], nums[store]
    return store                             # pivot is now in its final sorted place

def kth_largest(nums, k, seed=0):
    nums, rng = nums[:], random.Random(seed)
    target = len(nums) - k                   # k-th largest = index n-k when sorted
    lo, hi = 0, len(nums) - 1
    while True:
        p = partition(nums, lo, hi, rng)
        if p == target:
            return nums[p]
        if p < target:
            lo = p + 1
        else:
            hi = p - 1
```

`quicksort` in the script uses the same `partition` with an explicit stack of `(lo, hi)` ranges instead of recursion.

Dutch national flag – three regions grow while an "unknown" region shrinks. Invariant: `[0, low)` are 0s, `[low, mid)` are 1s, `(high, end]` are 2s, `[mid, high]` not yet seen.

```python
def sort_colors(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1; mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:                                # a 2 goes to the back; the value swapped
            nums[mid], nums[high] = nums[high], nums[mid]   # in is unseen, so mid stays
            high -= 1
```

Merge sort on a linked list – find the middle with fast/slow pointers (19.16; start `fast` at `head.next` so `slow` stops at the end of the left half), cut, sort both halves recursively, then merge with a dummy head (19.7). `sort_list` in the script.

Output of `17_sort_search.py` (the script also checks `quicksort` and `kth_largest` against `sorted` on 200 random lists):

```
merge-sorted list: [1, 2, 2, 3, 4, 5]
quicksort: [1, 2, 4, 7, 8, 9] | 2nd largest of [3,2,1,5,6,4]: 5
sort colors: [0, 0, 1, 1, 2, 2]
all asserts passed
```

Trade-offs

Algorithm	Time (avg / worst)	Extra space	Stable?	Use when
`sorted` (Timsort)	O(n log n) / O(n log n)	O(n)	yes	always, in real code
quicksort	O(n log n) / O(n²)	O(log n) stack	no	asked to implement in place
merge sort	O(n log n) / O(n log n)	O(n) array, O(log n) list	yes	linked lists; external sorting of huge files
quickselect	O(n) / O(n²)	O(1)	–	one k-th element, input may be reordered
heap top-k	O(n log k)	O(k)	–	streaming data, or k ≪ n
Dutch flag	O(n)	O(1)	no	exactly three key values

Common mistakes

- Always choosing the first or last element as pivot – O(n²) on sorted input. Randomise.
- Many equal values: Lomuto puts all equals on one side and degrades; a three-way partition (Dutch flag around the pivot) fixes it.
- Dutch flag: advancing `mid` after swapping with `high`.
- Quickselect mutating the caller's list without saying so – copy, or ask.

In the interview

> **30-second answer:** "Partition puts a pivot in its final place with smaller items left and larger right. Quicksort recurses on both sides – O(n log n) average; quickselect only follows the side containing index n−k – O(n) average; random pivots avoid the worst case. Three distinct values: Dutch flag, one pass with low/mid/high. Linked lists: merge sort, splitting with fast/slow pointers."

- *Guaranteed O(n) selection?* → Median of medians, O(n) worst case but slower in practice; mention it, don't code it.
- *Quickselect vs heap for top-k?* → Quickselect is O(n) average but needs all data in memory and reorders it; a size-k heap works on a stream in O(k) memory.

19.18 Math & geometry

What it is

A family of problems where the trick is a small mathematical fact rather than a data structure: walking a matrix in a pattern, handling integer limits, comparing slopes exactly, or replacing a simulation with a formula. 19.12 has the number-theory toolkit (`gcd`, `pow`, `isqrt`, `comb`); this section applies it.

How it works

Problem 1 – spiral order. Keep four boundaries – `top`, `bottom`, `left`, `right` – and peel one ring at a time: along the top row, down the right column, back along the bottom row, up the left column, shrinking the matching boundary after each side. The two `if` checks stop a single remaining row or column from being read twice.

```
def spiral_order(matrix):
    out = []
    if not matrix:
        return out
    top, bottom, left, right = 0, len(matrix) - 1, 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        out += [matrix[top][c] for c in range(left, right + 1)];  top += 1
        out += [matrix[r][right] for r in range(top, bottom + 1)]; right -= 1
        if top <= bottom:
            out += [matrix[bottom][c] for c in range(right, left - 1, -1)]; bottom -= 1
        if left <= right:
            out += [matrix[r][left] for r in range(bottom, top - 1, -1)]; left += 1
    return out
```

`rotate` in the script turns an n × n matrix 90° clockwise in place: **transpose** (swap `m[i][j]` and `m[j][i]`), then reverse each row.

Problem 2 — reverse an integer with 32-bit overflow. Reverse the digits of x, but return 0 if the result falls outside the signed 32-bit range $[-2^{31}, 2^{31} - 1]$. Python ints never overflow (19.12), so you must check the range yourself. Interviewers often want the check a 32-bit language needs — test **before** multiplying: `res > (INT_MAX - digit) // 10`.

```
INT_MIN, INT_MAX = -2**31, 2**31 - 1

def reverse_int(x):
    sign = -1 if x < 0 else 1
    x, res = abs(x), 0
    while x:
        x, digit = divmod(x, 10)
        if res > (INT_MAX - digit) // 10:
            return 0
        res = res * 10 + digit
    res *= sign
    return res if INT_MIN <= res <= INT_MAX else 0
```

(Taking `abs(x)` first avoids a trap: `-123 % 10` is 7 in Python, not −3, because `%` follows the sign of the divisor — see 19.19.)

Problem 3 — most points on one line (collinear points). For each anchor point, group the other points by the slope of the line through the anchor; the biggest group, plus the anchor and any exact duplicates of it, is the best line through that anchor. Store the slope as a **reduced fraction** `(dx // g, dy // g)` with `g = gcd(dx, dy)` and one fixed sign, not as the float `dy / dx` — floats can round two different slopes to the same value, and vertical lines would divide by zero.

Problem 4 — Josephus. n people stand in a circle; counting round, every k-th person leaves; who survives? Simulating with `list.pop` is $O(n^2)$. The recurrence $J(1) = 0$, $J(m) = (J(m-1) + k) \bmod m$ (0-based) gives it in $O(n)$: after the first removal the circle is a problem of size m − 1 whose numbering is shifted by k.

```
def josephus(n, k):
    survivor = 0
    for m in range(2, n + 1):
        survivor = (survivor + k) % m
    return survivor
```

Output of `18_math_geometry.py` (the script checks `josephus` against the simulation for every n < 30, k < 8):

```
spiral: [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
rotated: [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
reverse 1534236469 -> 0 (overflow)
max collinear: 4
Josephus n=7, k=3 survivor index: 3
all asserts passed
```

Complexity

Problem	Time	Space
spiral / rotate	O(r·c)	O(1) extra (rotate in place)
reverse integer	O(number of digits)	O(1)
max points on a line	O(n²) (the gcd adds a log factor)	O(n) per anchor
Josephus	O(n)	O(1)

Common mistakes

- Spiral: missing the two `if` guards, so a middle row or column is output twice on non-square matrices.
- Slopes as floats, or not normalising the sign – `(1, -2)` and `(-1, 2)` are the same direction.
- Forgetting duplicates of the anchor point in "max points on a line".
- Relying on Python's big ints when the problem says "assume a 32-bit environment".

In the interview

30-second answer: "For matrix walks I keep explicit boundaries and shrink them; rotation is transpose plus reverse rows. Python ints don't overflow, so for 32-bit problems I check the range explicitly – before multiplying, as a C or Java solution would. For geometry I compare slopes as reduced integer fractions via `gcd`, never floats. When a simulation is slow, I look for a recurrence, like Josephus's `(J + k) % m`."

- *Rotate counter-clockwise?* → Transpose, then reverse each column (or reverse each row first, then transpose).
- *Why does Josephus's recurrence work?* → After removing person k–1, renumber the remaining m–1 people starting from the next one; the survivor of that smaller circle, shifted back by k (mod m), is the survivor of the larger one.

19.19 Common pitfalls

Each of these is demonstrated (bug, then fix) in `19_pitfalls.py`. Chapter 20 covers general Python gotchas in more depth; these are the ones that bite in timed coding.

```
1. [1, 2] [1, 2] <- shared list
   fixed: [1] [2]
2. [[0]*3]*2 -> [[9, 0, 0], [9, 0, 0]] | comprehension -> [[9, 0, 0], [0, 0, 0]]
3. shallow sees change: 99 | deep does not: 1
4. stored references: [[1, 2], [1, 2]] | use path[:] -> [[1], [1, 2]]
5. default limit: 1000
   depth(3000) -> RecursionError; use an explicit stack or raise the limit
6. 7 // 2 = 3 | -7 // 2 = -4 | int(-7 / 2) = -3 | -7 % 3 = 2
   7 / 2 = 3.5 (always float) | round(2.5) = 2 | round(3.5) = 4
7. 0.1 + 0.2 == 0.3: False | isclose: True
8. a == b: True | a is b: False | use `is` only for None/sentinels
9. remove while iterating -> [1, 2, 3] | comprehension -> [1, 3]
10. ''.join(parts) = 01234 (O(n); repeated += can be O(n^2))
11. xs.sort() returns None - xs is now [1, 2, 3] | sorted() returns a new list
12. list still callable: ['a', 'b'] [1, 2]
```

#	Pitfall	Why	Fix
1	Mutable default `def f(x, acc=[])`	defaults are evaluated **once** when `def` runs; every call shares the list	`acc=None`, then `if acc is None: acc = []`
2	`[[0] * n] * m`	the outer `*` repeats a **reference** to one row	`[[0] * n for _ in range(m)]`
3	Shallow copy of nested data	`a[:]`, `list(a)`, `a.copy()`, `copy.copy` copy only the outer list	`copy.deepcopy`, or `[row[:] for row in grid]` for 2-D
4	`res.append(path)` in backtracking	stores the same list object	`res.append(path[:])`
5	Deep recursion	limit 1000, no tail-call optimisation	iterative stack/deque; `sys.setrecursionlimit` as last resort
6	Integer division	`//` **floors** (toward $-\infty$); `int(a / b)` truncates toward 0; `%` takes the sign of the divisor; `/` always returns float; `round` uses banker's rounding (ties to even)	choose deliberately; for "truncate toward zero" use `int(a / b)` (small ints) or sign-aware `//`
7	Float equality	binary floats can't represent 0.1 exactly	`math.isclose`, or stay in integers
8	`is` vs `==`	`is` compares identity	`==` for values; `is` only for `None` and sentinels
9	Mutating a list while iterating	the iterator's index skips elements	build a new list, or iterate over a copy
10	String `+=` in loops	may copy each time → $O(n^2)$	collect parts, `"".join`
11	`nums = nums.sort()`	in-place methods return `None`	`nums.sort()` alone or `nums = sorted(nums)`
12	Shadowing built-ins (`list`, `dict`, `sum`, `max`, `input`, `id`)	later calls hit your variable	pick other names (`nums`, `total`, `list_`)

Two more to remember: `float("inf")` compares fine with ints but `int(float("inf"))` raises `OverflowError`; and `@cache` on a helper with global state gives stale answers across test cases (19.10).

19.20 How to talk through a solution

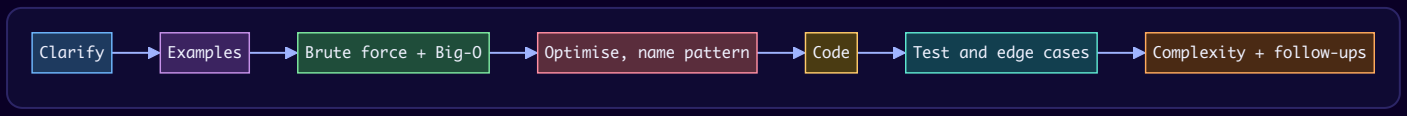
What it is

Interviewers score **communication and process** as well as the final code; a predictable structure keeps you calm and shows how you think.

How it works — the seven steps

1. **Clarify** (1–2 min): input sizes, ranges (negatives? duplicates? empty?), output format, may you modify the input? Size hints the target: $n \le 20$ → exponential is fine; $n \le 10^3$ → $O(n^2)$; $n \le 10^5$–10^6 → $O(n \log n)$ or $O(n)$.
2. **Examples:** one normal case and one edge case by hand.
3. **Brute force:** state it and its complexity in one sentence — proof you understand the problem, and a fallback.
4. **Optimise:** name the pattern ("sorted → two pointers", "k-th → heap"), explain *why* it's faster, agree before coding.
5. **Code:** clear names, small helpers, idioms (`enumerate`, `defaultdict`, unpacking); narrate briefly.

6. **Test:** trace your example, then edge cases — empty, single element, all equal, negatives, maximum sizes.
7. **Analyse:** time and space, and what changes for follow-ups (streaming data, memory limits).



Phrases that help

- "Before I optimise, the brute force is $O(n^2)$ because we check every pair."
- "Since the array is sorted, I can discard half each step — binary search, $O(\log n)$."
- "I'll use a `deque` here rather than a list because `popleft` is $O(1)$."
- "Space is $O(n)$ for the hash map; if memory mattered, I could sort and use two pointers at $O(1)$ extra space but $O(n \log n)$ time."

Python-specific tips

- Using `Counter`, `heapq`, `bisect`, `itertools` is expected — but know each call's complexity and be ready to hand-write it ("could you do this without `Counter`?").
- Type hints in the signature (`def solve(nums: list[int]) -> int:`) read well; don't golf — a clear four-line loop beats a one-liner you can't explain.
- Know which Python version the platform runs; newer helpers (`itertools.pairwise` / `math.lcm` need 3.9–3.10, `itertools.batched` 3.12, `heapq.heappush_max` 3.14) may not exist there.

Chapter TL;DR

Pattern	Reach for it when…	Python tool
Costs	always state them	list = array (O(1) end, O(n) front/search); dict/set O(1) avg; slices and `+` copy
Two pointers / window / prefix sums	sorted pairs; contiguous + monotonic rule; range sums or "sum = k"	indices, `accumulate`, dict of prefixes
Hashing	seen-before, counts, groups	`set`, `dict`, `Counter`, `defaultdict` (hashable keys)
Stack / queue / monotonic	brackets, "next greater", window max	`list`, `deque`
Heap	top-k, merge-k, two heaps (running median, capital), Dijkstra	`heapq` (negate for max, tie-breakers)
Sort / binary search	ordering, "first position", monotonic feasibility, weighted random	`key=` tuples, `bisect_left/right`, prefix sums
Lists / trees / graphs	pointers, traversals, shortest paths, dependencies	node classes, `deque` BFS, `heapq` Dijkstra, Kahn, DSU
DP / backtracking	overlapping subproblems; enumerate with pruning	`@cache` or tables; choose → explore → un-choose
Bits / math	masks, parity, number theory	`x & (x-1)`, `pow(b, e, m)`, `gcd`, `isqrt`, `comb`
Tries	prefix search, autocomplete, wildcard, many words on a grid	dict-of-children nodes + end-of-word field
Greedy	a locally best choice is provably safe (exchange argument)	one pass, often after `sorted(key=...)`
Intervals	merge, insert, overlap, rooms needed	sort by start (merge) or end (keep most); heap of ends; sweep line
Fast & slow pointers	cycle and its entry, middle, O(1)-space list checks, implicit sequences	`slow`, `fast`; compare with `is`
Sort & search	k-th element, partition in place, three values, sort a linked list	quickselect, Dutch flag, merge sort
Math & geometry	matrix walks, 32-bit limits, collinear points, circle elimination	boundaries, explicit range checks, `gcd` slopes, recurrences

Avoid the pitfalls of 19.19 (mutable defaults, `[[0]*n]*m`, shallow copies, the 1000-frame recursion limit, `//` flooring), and narrate: clarify → brute force → optimise → code → test → analyse.

Chapter 20 — Pythonic Idioms, Gotchas and Interview Q&A

Pythonic code is code that uses the language the way its designers intended: readable, short without being clever, and leaning on built-in protocols (iteration, context managers, unpacking) instead of re-implementing them. Reviewers and interviewers notice it immediately — `for i in range(len(xs))` or `if x == None` marks someone who learned Python from another language. The flip side is a handful of **gotchas**: behaviours that surprise even experienced developers (mutable default arguments, late-binding closures, shallow copies).

This chapter pulls the whole book together for review: first the idioms, then the gotchas (each with a demo and a fix), then about 45 classic Python interview questions with short, correct answers that point back to the chapter where the topic is taught in depth. Scripts live in `code/` and were run with Python 3.14 (all 3.12-compatible); outputs are real.

What you'll learn

#	Topic	Script
20.1	Pythonic idioms: EAFP, unpacking, `enumerate` / `zip`, comprehensions, truthiness, f-strings, context managers and `with open`, `pathlib`, and smaller habits	`01_idioms.py`
20.2	13 gotchas, each demonstrated and fixed	`02_gotchas.py`
20.3	Interview Q&A: language basics, data structures, functions, OOP, internals and memory, concurrency, typing/tooling	`03_interview_qa.py`

> The Zen of Python (`import this`, PEP 20) summarises the taste behind the idioms: *explicit is better than implicit, simple is better than complex, readability counts, errors should never pass silently, there should be one obvious way to do it.* PEP 8 is the style guide (4-space indents, `snake_case` functions, `CapWords` classes, lines ≤ 79–99 chars); tools such as `ruff` and `black` enforce it automatically.

20.1 Pythonic idioms

What it is

An **idiom** is the conventional way a community expresses a common task. In Python, idioms usually replace manual bookkeeping (indexes, flags, cleanup calls) with a language feature that does it for you.

Why it matters

- Fewer lines → fewer places for bugs (no off-by-one on an index you didn't need).
- Other Python developers read idiomatic code faster.
- Many idioms are also faster, because the work runs inside C-implemented built-ins (`sum`, `join`, comprehensions).

The idioms at a glance

Instead of…	Write…	Section
`if key in d: v = d[key] else: v = default`	`v = d.get(key, default)` or `try/except KeyError`	EAFP
`tmp = a; a = b; b = tmp`	`a, b = b, a`	Unpacking
`for i in range(len(xs)): x = xs[i]`	`for i, x in enumerate(xs)`	enumerate
`for i in range(len(a)): a[i], b[i]`	`for x, y in zip(a, b)`	zip
`out = []` + loop + `append`	`[f(x) for x in xs if cond(x)]`	Comprehensions
`if len(xs) == 0` / `if flag == True`	`if not xs` / `if flag`	Truthiness
`"Hi " + name + ", you are " + str(age)`	`f"Hi {name}, you are {age}"`	f-strings
`f = open(p); ...; f.close()`	`with open(p) as f:`	Context managers
`os.path.join(d, "x.txt")`	`Path(d) / "x.txt"`	pathlib
`s = ""` + `s += piece` in a loop	`"".join(pieces)`	Strings
`x == None`	`x is None`	Identity
`found = False` flag + loop	`for ... else:` or `any(...)`	Loops

EAFP vs LBYL

LBYL ("look before you leap") checks conditions first: `if "port" in config: ...`. **EAFP** ("easier to ask forgiveness than permission") just tries and handles the failure: `try: port = int(config["port"]) except (KeyError, ValueError): port = 80`. Python favours EAFP because:

1. It avoids **race conditions** – between "check the file exists" and "open it", another process can delete it; opening inside `try` has no gap.
2. One `try` can cover several failure modes (missing key *and* bad number above).
3. It keeps the happy path unindented and readable.

Use LBYL when failure is common and cheap to check (then exceptions would be slow and noisy), or when the check has no side effects to race with. For "ignore this particular error", `contextlib.suppress(FileNotFoundError)` is a tidy EAFP shortcut. Never write a bare `except:` (see gotcha 11).

Unpacking

Any iterable can be unpacked into names; `*name` collects "the rest" into a list; `**` unpacks mappings.

```
a, b = b, a                              # swap, no temp variable
first, *middle, last = [10, 20, 30, 40, 50]
(x, y), z = (1, 2), 3                    # nested
merged = {**point, "z": 5}               # dict unpacking (or point | {"z": 5}, 3.9+)
both = [*range(3), *"ab"]
_, ext = "report.final.pdf".rsplit(".", 1)  # _ = "don't care"
```

```
swap: 2 1 | star: 10 [20, 30, 40] 50 | nested: 1 2 3
dict merge: {'x': 3, 'y': 4, 'z': 5} | point | {'z': 5} -> {'x': 3, 'y': 4, 'z': 5} | list: [0, 1, 2, 'a',
'b']
rsplit + _: pdf
```

Unpacking also works in `for name, score in pairs:` and in function calls (`f(*args, **kwargs)`, Chapter 7).

`enumerate` and `zip`

`enumerate(xs, start=1)` yields `(index, item)` pairs; `zip(a, b)` yields tuples of items at the same position and **stops at the shortest** input. Add `strict=True` (3.10+) to get a `ValueError` when lengths differ – a cheap

guard against silent data loss. `zip(*rows)` "unzips" (transposes).

```
for rank, name in enumerate(names, start=1):
    print(f"  {rank}. {name}")
dict(zip(names, scores))
list(zip(names, [1, 2], strict=True))   # ValueError
nums, letters = zip(*[(1, "a"), (2, "b")])
```

```
  1. ann
  2. bob
  3. cy
zip: [('ann', 91), ('bob', 78), ('cy', 85)] | dict(zip): {'ann': 91, 'bob': 78, 'cy': 85}
zip(strict=True): zip() argument 2 is shorter than argument 1
unzip: (1, 2) ('a', 'b')
```

Comprehensions and generator expressions

A **comprehension** builds a list, dict or set from an iterable in one expression: `[expr for x in it if cond]`, `{k: v for ...}`, `{x for ...}`. A **generator expression** `(expr for x in it)` produces items lazily — pass it straight to `sum`, `any`, `max`, `"".join` to avoid building a list at all.

```
[n * n for n in range(6)]                          # [0, 1, 4, 9, 16, 25]
[n for n in range(10) if n % 2 == 0]               # filter at the end
["even" if n % 2 == 0 else "odd" for n in range(4)] # if-else goes BEFORE for
{w: len(w) for w in ["hi", "hello"]}
sum(n * n for n in range(1_000_000))               # no million-item list
[x for row in [[1, 2], [3]] for x in row]          # loops read left-to-right
```

Rules of thumb: one `for` and one condition is ideal; two `for`s is the maximum; if it no longer fits on a line or two, write a normal loop. Don't use a comprehension just for side effects (`[print(x) for x in xs]`) — that builds a useless list.

Truthiness

Every object has a truth value. **Falsy**: `None`, `False`, zero of any numeric type (`0`, `0.0`, `0j`), and empty containers (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`). Everything else is truthy unless its class defines `__bool__` / `__len__` otherwise. So write `if items:` and `if not name:`.

The trap: `if count:` is also false when `count == 0`. When `0` (or `""`) is a valid value and you mean "was it provided?", write `if count is not None:`.

`and` / `or` return one of their **operands**, not a bool: `name or "anonymous"` gives a default for any falsy name.

f-strings

f-strings (3.6+) evaluate expressions inside `{}` and apply the **format mini-language** after a colon:

```
print(f"{name:<8}|{price:>12,.2f}|{ratio:.1%}|{42:05d}|{255:#x}|{255:b}")
print(f"{price=}")          # 3.8+: prints the expression and its value
```

```
Widget  |    1,234.57|45.7%|00042|0xff|11111111
price=1234.5678
```

`<` / `>` / `^` align, the number is the width, `,` adds thousands separators, `.2f` fixes decimals, `%` multiplies by 100, `x` / `b` are hex/binary. `!r` uses `repr()`. For logging, pass arguments lazily instead (`log.info("user %s", uid)`) so formatting only happens if the message is emitted.

Context managers and `with open`

A **context manager** is an object whose `__enter__` runs at the start of a `with` block and whose `__exit__` runs at the end — **even if an exception is raised** (Chapter 12). Files, locks, database transactions, temporary directories and timers all use it, so cleanup can't be forgotten.

```
with open(path, "w", encoding="utf-8") as f:    # closed even on error
    f.write("line one\nline two\n")
with open(path, encoding="utf-8") as f:
    for line_no, line in enumerate(f, 1):       # stream: one line in memory at a time
        print(line_no, line.rstrip())

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                    # the with-block body runs here
    finally:
        print(f"{label}: done")
```

Always pass `encoding="utf-8"` for text files: the default comes from the platform's locale (for example, often cp1252 on Windows), which causes "works on my machine" bugs. Several managers can share one `with`: `with open(a) as src, open(b, "w") as dst:`.

`pathlib`

`pathlib.Path` (Chapter 9) represents file-system paths as objects: `/` joins, and methods replace scattered `os.path` functions.

```
base = Path(tmp)
notes = base / "notes" / "day1.txt"
notes.parent.mkdir(parents=True, exist_ok=True)
(base / "b.csv").write_text("a,b\n1,2\n", encoding="utf-8")
notes.name, notes.stem, notes.suffix     # 'day1.txt', 'day1', '.txt'
[p.name for p in base.glob("*.csv")]     # ['b.csv']
[p.relative_to(base).as_posix() for p in base.rglob("*.txt")]
```

```
name/stem/suffix: day1.txt day1 .txt
glob *.csv: ['b.csv']
rglob *.txt: ['notes/day1.txt']
exists: True | read_text: 'line one'
```

Smaller habits (all in `01_idioms.py`)

- **Chained comparisons:** `0 < x < 10`.
- **Conditional expression:** `"big" if x > 3 else "small"`.
- `any` / `all` **with generators** instead of flag loops; `max(xs, key=len, default=None)` for empty-safe maxima.
- `for … else`: the `else` runs only if the loop didn't `break` — the "search failed" branch.
- **Walrus** `:=` (3.8+) assigns inside an expression: `if (m := len(s)) > 3:` — handy in `while` loops reading chunks.
- `defaultdict` / `Counter` instead of "if key not in d: d[key] = …".
- `"sep".join(strings)` to build strings.
- `dataclass` instead of hand-written `__init__` / `__repr__` / `__eq__` (Chapter 10).
- `if __name__ == "__main__":` so importing a module doesn't run its script code (Chapter 8).

Common mistakes

- Comprehensions nested three deep — readable code beats short code.
- `except Exception: pass` — EAFP means *handling* a specific failure, not hiding every failure.
- Truthiness tests where `0` / `""` are valid values.

In the interview

> **30-second answer:** "Pythonic code uses the language's protocols: iterate directly with `enumerate` / `zip`, build collections with comprehensions, use truthiness, `with` for any resource, f-strings for formatting, `pathlib` for paths, and EAFP – try the operation and catch the specific exception, which also avoids check-then-use races."

- *EAFP or LBYL for checking a file exists before opening it?* → EAFP: `try: open(...) except FileNotFoundError:` – the file could vanish between the check and the open.
- *List comprehension or generator expression?* → Generator when you consume once (e.g. `sum`), especially for large data; list when you need to index, reuse or `len()` it.

Self-check

1. What does `zip([1, 2, 3], "ab")` produce? → `[(1, 'a'), (2, 'b')]` – it stops at the shortest.
2. Write "the first item and the rest" in one line. → `first, *rest = items`.
3. Why pass `encoding="utf-8"` to `open`? → The default text encoding is platform-dependent.
4. When does a `for` loop's `else` run? → When the loop ends without `break`.

TL;DR

Iterate directly (`enumerate`, `zip`), build with comprehensions, test truthiness, format with f-strings, manage resources with `with`, use `pathlib`, and prefer EAFP with specific exceptions.

20.2 Top gotchas – demo and fix

Each block below is printed by `02_gotchas.py`; the section explains *why* it happens and the fix.

1. Mutable default arguments

```
def add_item_bad(item, bucket=[]):
    bucket.append(item)
    return bucket
```

```
bad : ['a', 'b'] ['a', 'b'] | defaults: (['a', 'b'],)
good: ['a'] ['b']
```

Why: default values are evaluated **once**, when the `def` statement runs, and stored on the function (`__defaults__`). Every call that omits `bucket` gets the *same* list. **Fix:** default to `None` and create the list inside: `if bucket is None: bucket = []`. (The same applies to `{}`, `set()`, and any mutable object.) Linters flag this (ruff `B006`).

2. Late-binding closures

```
fns = [lambda: i for i in range(3)]
[f() for f in fns]                      # [2, 2, 2]
fns = [lambda i=i: i for i in range(3)]     # [0, 1, 2]
```

Why: a closure stores a reference to the *variable* `i`, not its value at creation time; `i` is looked up when the function is **called**, after the loop has finished (so it's 2). **Fix:** bind the current value via a default argument (`i=i`), or use `functools.partial(f, i)`, or a factory function `def make(i): return lambda: i`.

3. `is` vs `==`

```
a == b: True | a is b: False
x is None: True (the right way to test for None)
```

Why: `==` calls `__eq__` and compares **values**; `is` compares **identity** (same object in memory, `id(a) == id(b)`). **Fix:** use `==` for values; use `is` only for singletons – `None`, `True` / `False` in rare cases, and your own sentinel objects (`_MISSING = object()`).

4. Integer caching

```
256 is 256: True | 257 is 257: False | 257 == 257: True
```

Why: CPython pre-creates the integers –5 to 256 and reuses them, so any two 256s are the same object; bigger ints built at runtime are separate objects. (Literals in the same compiled code block may be shared by the compiler, which is why a naive `a = 257; b = 257; a is b` can print `True` – it's an implementation detail either way.) Similarly, some strings are **interned** (identifier-like literals). **Fix:** never use `is` to compare numbers or strings.

5. Modifying a list while iterating

```
bad : [1, 2, 3]
good: [1, 3] | or iterate a copy: for n in nums[:]
dict: dictionary changed size during iteration
dict good: {'b': 2}
```

Why: the list iterator walks by index; removing an item shifts the rest left, so the next item is skipped (the second `2` survived). Dicts and sets detect size changes and raise `RuntimeError`. **Fix:** build a new collection with a comprehension, iterate over a copy (`nums[:]`, `list(d)`), or collect keys to delete and delete afterwards.

6. Shallow copy

```
original after shallow edit: {'name': 'svc', 'ports': [80, 443]} | deep copy: {'name': 'svc', 'ports': [80]}
alias is config: True
```

Why: assignment (`alias = config`) copies nothing – both names point at one object. A **shallow copy** (`d.copy()`, `list(x)`, `x[:]`, `copy.copy`) makes a new outer container but shares the inner objects. A **deep copy** (`copy.deepcopy`) recursively copies everything (and handles cycles with a memo dict). **Fix:** use `deepcopy` for nested mutable data you intend to change independently – or better, design with immutable values (tuples, frozen dataclasses). For a 2-D list, `[row[:] for row in grid]` suffices.

7. Shared mutable values: `dict.fromkeys` and `defaultdict` reads

```
fromkeys bad : {'a': [1], 'b': [1]}
fromkeys good: {'a': [1], 'b': []}
defaultdict after a read: {'missing': []} | use `'missing' in dd` or dd.get()
```

Why: `dict.fromkeys(keys, [])` stores the **same** list object under every key (just like `[[]] * n`). And a `defaultdict` creates and inserts a default on *any* `d[key]` access, even a read, so checking `if dd["x"]:` silently grows the dict. **Fix:** a dict comprehension (`{k: [] for k in keys}`); test membership with `in` or use `dd.get(key)`, which does **not** insert.

8. Class attributes vs instance attributes

```
bad : sam.tricks = ['roll']
good: sam.tricks = []
assignment shadows: wolf | canis familiaris | canis familiaris
```

Why: attributes defined in the class body belong to the class and are shared by all instances. Mutating one through an instance (`r.tricks.append`) changes it for everyone. *Assigning* through an instance (`r.species = "wolf"`) creates a new instance attribute that shadows the class one — only for `r`. **Fix:** create per-instance mutable state in `__init__` (`self.tricks = []`); keep class attributes for constants and genuinely shared data. `dataclass` refuses a mutable default like `tricks: list = []` and makes you write `field(default_factory=list)`.

9. Circular imports

```
bad : ImportError: cannot import name 'helper_a' from 'mod_a'
good: ba
```

Why: importing a module executes it top to bottom. `mod_a` starts, hits `from mod_b import helper_b`, so `mod_b` runs — and its `from mod_a import helper_a` looks for a name `mod_a` hasn't defined yet (it's only *partially initialised*). The exact message wording varies by Python version. **Fixes, best first:**

1. Restructure: move the shared code into a third module both import (usually the real fix — a cycle often signals muddled responsibilities).
2. `import mod_a` (the module) and access `mod_a.helper_a` at call time, as the "good" version does.
3. Move the import inside the function that needs it.
4. For imports needed only by type hints: `from typing import TYPE_CHECKING` + `if TYPE_CHECKING: import ...`.

10. Floating point

```
0.1 + 0.2 = 0.30000000000000004 | == 0.3: False
math.isclose: True
Decimal('0.1') + Decimal('0.2') = 0.3 | Decimal(0.1) = 0.1000000000000000555...
Fraction(1, 10) + Fraction(2, 10) = 3/10
round(2.675, 2) = 2.67 | round(0.5), round(1.5) = 0 2
float('nan') == float('nan'): False
```

Why: `float` is IEEE-754 binary64; most decimal fractions (0.1, 2.675) have no exact binary representation, so they are stored as the nearest representable value. `round` uses **round-half-to-even** ("banker's rounding"), and `2.675` is really `2.67499999…`. `NaN` is unequal to everything, including itself. **Fix:** compare with `math.isclose` (or an explicit tolerance); use `decimal.Decimal` **constructed from strings** for money; `fractions.Fraction` for exact rationals; integers of cents for currency; `math.isnan` to test NaN.

11. Bare `except:`

```
bad : bare except swallowed KeyboardInterrupt
good: `except Exception` let KeyboardInterrupt through
```

Why: a bare `except:` catches `BaseException`, which includes `KeyboardInterrupt` (Ctrl+C) and `SystemExit` (`sys.exit()`), so your program can't be stopped and real bugs vanish. **Fix:** catch the narrowest exception you can handle (`except KeyError:`); at the top level use `except Exception:` and **log** it (`logger.exception(...)`); re-raise with `raise` if you can't handle it.

12. Shadowing built-ins

```
bad : 'list' object is not callable
good: ['a', 'b', 'c']
```

Why: names are looked up Local → Enclosing → Global → **Built-in** (LEGB, Chapter 7). A variable called `list`, `dict`, `str`, `id`, `input`, `sum`, `max`, `type`, `file` or `len` hides the built-in for the rest of that scope. **Fix:** choose descriptive names (`items`, `user_id`) or a trailing underscore (`type_`); linters (ruff `A001`) catch it. Also don't name your *files* after standard modules (`random.py`, `json.py`) – they shadow the real module on import.

13. Bonus: return values that surprise

- `list.sort()`, `list.append()`, `random.shuffle()`, `dict.update()` modify in place and return `None` – so `xs = xs.sort()` loses your list.
- `and` / `or` return an operand: `"" or "default"` → `'default'`, `0 and 1` → `0`.

Gotcha summary

Gotcha	Rule to remember
Mutable default	Defaults are evaluated once → use `None`
Late binding	Closures see variables, not values → `i=i`
`is` vs `==`	`is` only for `None`/sentinels
Int caching / interning	Implementation detail – never rely on it
Mutating while iterating	Build a new collection or iterate a copy
Shallow copy	Copies the container, shares the contents
`fromkeys` / `defaultdict`	Shared default object; reads insert
Class vs instance attrs	Mutable state goes in `__init__`
Circular imports	Restructure, or import the module not the name
Floats	`isclose`, `Decimal("…")`, `Fraction`, ints of cents
Bare `except`	Catch specific exceptions; log the rest
Shadowing built-ins	Don't name things `list`, `id`, `sum` …

In the interview

> **30-second answer:** "The classic ones: mutable default arguments are evaluated once, so use `None`; closures bind late, so capture loop variables with a default; `is` is identity, not equality; copying is shallow unless you `deepcopy`; don't mutate a collection while iterating over it; floats are binary, so use `isclose` or `Decimal`; and never use a bare `except`."

- *Why does Python evaluate defaults once?* → Defaults are part of the function object, created when `def` runs; re-evaluating on every call would be slower and would change the semantics of defaults that refer to outer names.
- *How do you make a closure capture the current loop value?* → Default argument `lambda i=i: i`, `functools.partial`, or a factory function.

Self-check

1. `def f(x, seen=set())` – what's wrong? → `seen` is shared between calls.
2. `a = [[0] * 2] * 2; a[0][0] = 1` → `a`? → `[[1, 0], [1, 0]]`.
3. How do you compare two floats for "equality"? → `math.isclose(a, b)` (optionally with `abs_tol` near zero).
4. Which exceptions does bare `except:` catch that `except Exception:` doesn't? → e.g. `KeyboardInterrupt`, `SystemExit`, `GeneratorExit` – the `BaseException` subclasses outside `Exception`.

TL;DR

Defaults once, closures late, `is` for identity, copies shallow, don't mutate while iterating, floats inexact, catch specific exceptions, don't shadow built-ins, break import cycles.

20.3 Python interview Q&A

Short, correct answers you can say out loud in 20–40 seconds. "→ Ch N" points to where the topic is taught in full. Demos come from `03_interview_qa.py`; its full output follows the relevant answers.

A. Language basics

Q1. Is Python compiled or interpreted? Both, in stages. CPython (the reference implementation, written in C) **compiles** source to **bytecode** (cached as `.pyc` files in `__pycache__`), then its virtual machine **interprets** the bytecode. There is no ahead-of-time compilation to machine code by default (CPython 3.13+ ships an experimental JIT that is off by default). `dis` shows the bytecode:

```
== bytecode: CPython compiles to bytecode, then interprets it
167           RESUME                   0
              LOAD_FAST_BORROW         0 (x)
              LOAD_SMALL_INT           1
              BINARY_OP                0 (+)
              RETURN_VALUE
```

(Exact opcodes change between versions.) Other implementations: **PyPy** (tracing JIT, often much faster for pure-Python loops), MicroPython (microcontrollers), and historically Jython/IronPython. → Ch 1, 17

Q2. Python is dynamically and strongly typed — what does that mean? *Dynamic*: types belong to objects, not variables; a name can be rebound to any type and types are checked at runtime. *Strong*: Python won't silently convert between unrelated types — `"1" + 1` raises `TypeError` (unlike JavaScript). → Ch 2

Q3. Mutable vs immutable types? Immutable objects can't change after creation: `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`. Mutable: `list`, `dict`, `set`, `bytearray`, most user classes. "Changing" an immutable value (`s += "x"`) creates a new object and rebinds the name. Only immutables (whose contents are hashable) can be dict keys or set members. Note a tuple *containing* a list is immutable itself but its list can change — and then it isn't hashable:

```
== mutable vs immutable, list vs tuple
   tuple holding a list: (1, [2, 3, 4])
   hash(tuple with list): unhashable type: 'list'
   sizes: list 88 bytes | tuple 72 bytes
```

→ Ch 2, 5

Q4. How are arguments passed — by value or by reference? Neither exactly: **by object reference** ("call by sharing"). The function's parameter becomes a new name bound to the *same object*. Mutating that object is visible to the caller; rebinding the parameter is not.

```
== pass by object reference
   after mutate: ['orig', 'changed']
```

(`lst.append` was visible; `lst = ["rebound"]` was not.) → Ch 7

Q5. `is` vs `==`? `==` compares values via `__eq__`; `is` compares identity (same object). Use `is` only for `None` and sentinels. See gotchas 3–4. → Ch 2

Q6. What does `if __name__ == "__main__":` do? Every module has `__name__`; it's `"__main__"` when the file is run directly and the module's import name when imported. The guard runs code only in the first case — so a file can be both an importable module and a script, and multiprocessing child processes (which re-import the main module on spawn) don't re-run your program. → Ch 8

Q7. Module vs package; what is `__init__.py`? A **module** is one `.py` file. A **package** is a directory of modules; a *regular* package has an `__init__.py` (run on first import, often used to expose a public API); without it you get a *namespace package*. Imports are cached in `sys.modules`, so a module body runs once per process. → Ch 8

Q8. What is a virtual environment and why use one? An isolated directory with its own interpreter link and `site-packages`, created with `python -m venv .venv`. It lets each project pin its own dependency versions without conflicts or touching the system Python. Tools: `pip` + `requirements.txt`, or `uv` / `poetry` with `pyproject.toml` and lock files. → Ch 8

Q9. What's new that interviewers like to hear about? `match` statement (structural pattern matching, 3.10), `X | Y` union types (3.10), exception groups and `except*` (3.11), big speed-ups from the "Faster CPython" work (3.11+), type-parameter syntax `def f[T](x: T)` (3.12), an optional free-threaded (no-GIL) build (experimental in 3.13, officially supported in 3.14), and template strings (t-strings, 3.14). → Ch 1, 3, 13, 16

B. Data structures

Q10. List vs tuple?

	list	tuple
Mutable	yes	no
Hashable (dict key)	no	yes, if its items are
Typical meaning	many items of one kind (a collection)	a fixed record of positions (`(lat, lon)`)
Memory / speed	over-allocates to allow growth	smaller and slightly faster to create
Tuples also signal intent: "this won't change". → Ch 5		

Q11. How is a list implemented? A dynamic array of pointers to objects. Index access is O(1); `append` is amortised O(1) because the array over-allocates and occasionally grows and copies; insert/delete at the front is O(n). → Ch 5, 19

Q12. How does a dict work internally? A **hash table**. `hash(key)` picks a slot; collisions are resolved by **open addressing** (probing other slots in a pseudo-random sequence). Since CPython 3.6 the layout is *compact*: a small index table points into a dense array of entries, which saves memory and preserves **insertion order** – guaranteed by the language since 3.7. The table resizes when it gets about two-thirds full, so operations stay O(1) on average (O(n) worst case with many collisions). Keys must be hashable, and objects that compare equal must have equal hashes. → Ch 6

Q13. What makes an object hashable? What if I define `__eq__`? It needs `__hash__` returning an int that never changes during its lifetime, and consistent `__eq__` (equal ⇒ same hash). If a class defines `__eq__` but not `__hash__`, Python sets `__hash__ = None` → instances become unhashable. A `@dataclass(frozen=True)` generates both. → Ch 6, 10

Q14. Set vs list for membership? Set: O(1) average `in` via hashing, no duplicates, unordered. List: O(n) scan, keeps order and duplicates. Convert to a set when you test membership repeatedly. → Ch 6, 19

Q15. Shallow vs deep copy? Shallow copies the container and shares the elements; deep copies recursively.

```
== copy vs deepcopy
   shallow: {'tags': ['a', 'b']} | deep: {'tags': ['a']}
```

Classes can customise with `__copy__` / `__deepcopy__`. → Ch 5, gotcha 6

Q16. `array` vs `list` vs NumPy array? `list` holds pointers to arbitrary objects. `array.array` stores raw C numbers of one type compactly. A NumPy `ndarray` stores a typed, contiguous block *and* provides vectorised operations implemented in C – orders of magnitude faster for numeric work. → Ch 5, 18

Q17. What's in `collections`? `Counter` (counting), `defaultdict` (auto-default values), `deque` (O(1) both ends), `namedtuple` (tuple with named fields), `OrderedDict` (order-aware equality, `move_to_end` – handy for LRU caches), `ChainMap` (layered lookups). → Ch 6

C. Functions

Q18. What are `*args` and `**kwargs`? In a definition, `*args` collects extra positional arguments into a **tuple**, `**kwargs` extra keyword arguments into a **dict** (the names are convention). At a call site, `*` and `**` **unpack** an iterable / mapping into arguments. Order in a signature: positional-only (before `/`), normal, `*args`, keyword-only, `**kwargs`.

```
== *args / **kwargs
   show(1, 2, a=3) -> ((1, 2), {'a': 3}) | show(*[1, 2], **{'b': 4}) -> ((1, 2), {'b': 4})
```

Typical use: wrappers/decorators that forward arguments unchanged. → Ch 7

Q19. What is a closure? A function that remembers variables from the enclosing function's scope even after that function has returned. The variables live in *cells* (`f.__closure__`). Use `nonlocal` to rebind them. Closures bind late (gotcha 2). → Ch 7

Q20. Explain LEGB and `global` / `nonlocal`. Name lookup order: **L**ocal, **E**nclosing functions, **G**lobal (module), **B**uilt-ins. Assigning to a name anywhere in a function makes it local for the whole function (hence `UnboundLocalError` if read first). `global x` rebinds the module variable; `nonlocal x` rebinds the nearest enclosing function's variable. → Ch 7

Q21. What is a decorator? Write one with arguments. A callable that takes a function and returns a replacement; `@d` above `def f` means `f = d(f)`. With arguments you need one more layer: `@retry(3)` calls `retry(3)`, which returns the actual decorator. Always use `functools.wraps` so the wrapper keeps the original `__name__`, `__doc__` and `__wrapped__`.

```python
def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ValueError:
                    print(f"   attempt {attempt} failed")
            raise RuntimeError("gave up")
        return wrapper
    return decorator
```

```
== decorators
   attempt 1 failed
   attempt 2 failed
    ok | name kept by wraps: flaky | Fails twice, then works.
```

Real uses: `@app.get("/")` (FastAPI/Flask routing), `@functools.cache`, `@pytest.fixture`, `@login_required` (Django). → Ch 12

Q22. Iterator vs iterable vs generator? An **iterable** has `__iter__` returning an iterator (lists, dicts, files). An **iterator** has `__next__` (returns the next item or raises `StopIteration`) and an `__iter__` returning itself; it's single-pass. A **generator** is an iterator produced by a function containing `yield` (or a generator expression): its state is saved between `next` calls. Every generator is an iterator; not every iterator is a generator.

```
== generators vs iterators
   iter(gen) is gen: True | values: [3, 2, 1] | second pass: []
   list 85176 bytes vs generator 208 bytes
```

The second pass is empty — iterators are exhausted once consumed. The generator's size doesn't depend on how many items it will produce. → Ch 11

Q23. `yield` vs `return`; what is `yield from`? `return` ends the function with one value. `yield` produces a value and **pauses**, resuming on the next `next()`. `yield from sub` delegates to a sub-iterator (re-yielding all

its values and passing `send()` / `throw()` through). Generators also support `send(value)` to push values in – the historical basis of coroutines. → Ch 11

Q24. What is a lambda and when shouldn't you use it? An anonymous single-expression function: `lambda x: x * 2`. Good for short `key=` arguments. Don't assign it to a name (`f = lambda ...` – use `def`, which gives a proper name in tracebacks), and don't cram logic into it. → Ch 7

Q25. `map` / `filter` vs comprehensions? Both are lazy in Python 3 (`map` / `filter` return iterators). Comprehensions are usually preferred for readability, especially when a `lambda` would be needed; `map(str, xs)` with an existing function is fine. → Ch 11

D. Object-oriented programming

Q26. `__init__` vs `__new__`? `__new__(cls, …)` is a static method that **creates** and returns the instance; `__init__(self, …)` **initialises** an already-created instance and returns `None`. You override `__new__` only for immutable types (subclassing `int`, `str`, `tuple`), singletons/instance caching, or metaclass work. Note `__init__` still runs on every call:

```
== __new__ vs __init__
   __new__: creating the object
   __init__: setting value=1
   __init__: setting value=2
   same object: True | value: 2
```

→ Ch 10

Q27. `@staticmethod` vs `@classmethod` vs instance method?

Kind	First parameter	Can access	Typical use
Instance method	`self`	instance and class	normal behaviour
`@classmethod`	`cls`	the class (and subclasses correctly)	alternative constructors (`dict.fromkeys`, `datetime.fromtimestamp`, `from_fahrenheit`)
`@staticmethod`	none	nothing implicit	a helper that logically belongs to the class

```
== classmethod / staticmethod / property / __slots__
    Temperature(100.0) | is_freezing(-3): True
```

→ Ch 10

Q28. What does `@property` do? Turns a method into a computed attribute, with optional setter/deleter – so you can start with a plain attribute and later add validation without changing callers (`t.celsius = -300` → `ValueError: below absolute zero` above). Python has no need for Java-style getters/setters. → Ch 10

Q29. What is the MRO and how does `super()` work? The **Method Resolution Order** is the order Python searches classes for an attribute, computed with the **C3 linearisation**: children before parents, and the left-to-right order of bases is preserved. `super()` means "the next class in the MRO of the *instance's* class", not "my parent" – which makes cooperative multiple inheritance (diamonds) call each class once:

```
== MRO / super()
   D.__mro__: ['D', 'B', 'C', 'A', 'object'] | D().hello(): D>B>C>A
```

`B`'s `super()` went to `C`, not `A`, because the instance is a `D`. → Ch 10

Q30. `__str__` vs `__repr__`? `__repr__` is for developers – unambiguous, ideally valid code to recreate the object (`Temperature(100.0)`); used by the REPL, debuggers and inside containers. `__str__` is for end users (`print`, `str()`, f-strings); if missing, it falls back to `__repr__`. Always define `__repr__` first. → Ch 10

Q31. What are dunder (magic) methods? Special methods named `__x__` that Python calls for syntax and built-ins – the **data model**: `len(x)` → `__len__`, `a + b` → `__add__`, `x[i]` → `__getitem__`, `for` → `__iter__`, `with` →

`__enter__` / `__exit__`, `==` → `__eq__`, `bool(x)` → `__bool__`, calling → `__call__`. Implementing them makes your classes behave like built-ins. → Ch 10

Q32. What does `__slots__` do? Declares a fixed set of attribute names, so instances store them in a compact array instead of a per-instance `__dict__` – less memory per object (helpful for millions of small objects) and slightly faster attribute access, but you can't add new attributes:

```
__slots__ blocks new attributes: 'Temperature' object has no attribute 'colour' and no __dict__ for setting new attributes
```

`@dataclass(slots=True)` (3.10+) generates it. → Ch 10, 17

Q33. Abstract base classes vs duck typing vs `Protocol`? Duck typing: if it has the method you call, it works – no inheritance needed. An **ABC** (`abc.ABC` + `@abstractmethod`) enforces at instantiation time that subclasses implement methods (nominal: you must inherit). A **`typing.Protocol`** describes the required methods structurally, checked by static type checkers without inheritance – "duck typing that mypy can verify" (`Square` satisfies `HasArea` above without inheriting it). → Ch 10, 13

Q34. What is a dataclass? `@dataclass` generates `__init__`, `__repr__` and `__eq__` from annotated fields; options add ordering (`order=True`), immutability + hashing (`frozen=True`), `slots=True`, and `field(default_factory=list)` for mutable defaults. Use it for plain data records; use Pydantic when you need runtime validation/parsing. → Ch 10

Q35. What are descriptors and metaclasses (briefly)? A **descriptor** is an object with `__get__` / `__set__` / `__delete__` stored as a class attribute; attribute access on instances is routed through it. `property`, methods (functions bind `self` via `__get__`), `classmethod` and `staticmethod` are all descriptors. A **metaclass** is the class of a class (default `type`); it controls class *creation*. ORMs (Django models), `abc.ABCMeta` and `enum.EnumMeta` / `EnumType` use metaclasses; for most needs, a class decorator or `__init_subclass__` is simpler. → Ch 12

E. Errors, resources, memory and internals

Q36. Explain `try / except / else / finally` and `raise … from`. `except` handles matching exceptions; `else` runs only if the `try` raised nothing (keep code that shouldn't be protected there); `finally` always runs (cleanup). `raise NewError(...) from err` chains exceptions, storing the cause in `__cause__` so the traceback shows both. Define custom exceptions by subclassing `Exception`. → Ch 9

Q37. How does a context manager work? `with obj as x:` calls `obj.__enter__()` (its return value is bound to `x`), runs the block, then always calls `obj.__exit__(exc_type, exc, tb)`; if `__exit__` returns true, the exception is suppressed. `contextlib.contextmanager` builds one from a generator: code before `yield` is enter, after it (in `finally`) is exit. → Ch 12

Q38. How does Python manage memory?

1. **Reference counting**: every object counts references to it; when the count reaches zero it is freed **immediately** (deterministic cleanup).
2. A **cyclic garbage collector** (`gc` module, generational) periodically finds groups of objects that only reference each other and frees them.
3. Small objects come from CPython's own allocator (**pymalloc**) that pools memory to avoid many small `malloc` calls.

```
== memory: reference counting + cycle collector
   refcount (includes getrefcount's own arg): 2
   freed immediately by refcounting: True
   cycle alive before gc.collect(): True
   cycle freed after gc.collect(): True
```

Memory "leaks" in Python are usually lingering references: growing global caches/lists, unbounded `lru_cache`s, or closures holding big objects. Diagnose with `tracemalloc`. → Ch 17

Q39. What is pickling? Is it safe? `pickle` **serialises** Python objects to bytes (`dumps` / `dump`) and back (`loads` / `load`); used by `multiprocessing` to send data between processes, and to save scikit-learn models

(joblib builds on it). Functions and classes are pickled *by reference* (module + name), so lambdas and locally defined functions can't be pickled:

```
== pickling
   round trip: {'model': 'v1', 'weights': [0.1, 0.2]} | 61 bytes
   pickle a lambda: PicklingError
```

Never unpickle untrusted data – loading a pickle can execute arbitrary code. For data exchange use JSON (or formats like Parquet/ `safetensors` for arrays). → Ch 9

Q40. How do you find and fix slow Python code? Measure first: `cProfile` / `pstats` (which functions take time), `timeit` (micro-benchmarks), `tracemalloc` (memory). Then: better algorithms/data structures (sets for membership, deque for queues), move loops into built-ins or NumPy vectorised operations, cache (`functools.cache`), avoid repeated attribute lookups in hot loops, and only then consider PyPy, Cython/Rust extensions, or processes for parallelism. → Ch 17

F. Concurrency

Q41. What is the GIL? The **Global Interpreter Lock** is a mutex in CPython that lets only one thread execute Python bytecode at a time. It keeps reference counting and the interpreter's internals simple and fast in single-threaded code. Consequences:

- **CPU-bound** pure-Python code does **not** speed up with threads – use `multiprocessing` / `ProcessPoolExecutor` (separate processes, separate GILs) or C extensions that release the GIL (NumPy does for many operations).
- **I/O-bound** code *does* benefit from threads, because the GIL is released while a thread waits on I/O.
- The GIL does not make your own code thread-safe: `counter += 1` is several bytecodes and can still race – use `threading.Lock` .

```
== GIL: threads vs processes on CPU-bound work
   GIL enabled: True
   CPU-bound x4  serial 0.95s | 4 threads 1.02s | 4 processes 0.56s
```

(Timings are machine-dependent; the pattern is what matters.) Threads gave no speed-up; processes did (less than 4× because of process start-up and data transfer). PEP 703's **free-threaded** CPython build (`python3.13t` / `3.14t`) removes the GIL; it's optional, and the default build still has it. → Ch 16

Q42. Threads vs processes vs asyncio – which when?

Workload	Choose	Why
CPU-bound (math, parsing, image processing)	`multiprocessing` / `ProcessPoolExecutor`	true parallelism, one GIL per process
I/O-bound, moderate concurrency, blocking libraries	`threading` / `ThreadPoolExecutor`	GIL released during I/O; simple
I/O-bound, thousands of connections, async libraries	`asyncio`	one thread, very cheap tasks
Numeric arrays	NumPy/vectorisation	C loops, often release the GIL
→ Ch 16		

Q43. How does `asyncio` work? A single-threaded **event loop** runs **coroutines** (`async def`). At each `await` on something not ready (network, timer), the coroutine yields control back to the loop, which runs another ready task – **cooperative multitasking**. `asyncio.gather` runs many concurrently:

```
== asyncio: many waits overlap on one thread
   10 x 0.2s waits concurrently -> 0.20s, ['req0', 'req1', 'req2']...
```

Ten 0.2-second waits took 0.2 s total, not 2 s. Pitfall: a blocking call (`time.sleep` , `requests.get` , heavy CPU) inside a coroutine freezes the whole loop – use async libraries (`httpx` , `aiohttp`) or `await`

`asyncio.to_thread(func)`. → Ch 16

G. Typing and tooling

Q44. Are type hints enforced? No. Annotations are stored (`__annotations__`) but the interpreter ignores them at runtime:

```
== typing is not enforced at runtime; Protocol = structural typing
   double('ab') runs anyway: abab
   total_area: 13
```

Static checkers (**mypy**, **pyright**) and IDEs use them to catch bugs before running; libraries like **Pydantic** and **FastAPI** read them to validate data at runtime. Key vocabulary: `list[int]`, `dict[str, float]`, `X | None` (= `Optional[X]`), `Callable[[int], str]`, `TypeVar`/generics (`def first[T](xs: list[T]) -> T`, 3.12), `Protocol`, `TypedDict`, `Literal`, `Any`. → Ch 13

Q45. How do you test Python code? `pytest` is the de-facto standard: plain `assert` statements, fixtures for setup, `@pytest.mark.parametrize` for many cases; `unittest` is the standard-library xUnit style. Use `unittest.mock` (`patch`, `Mock`) to replace external dependencies, `coverage` to measure what ran, and `hypothesis` for property-based tests. → Ch 15

Q46. What is monkey patching? Replacing an attribute of a module or class at runtime (`module.func = fake`). Useful in tests — `unittest.mock.patch("pkg.mod.name")` does it safely and restores the original afterwards — but dangerous in production code because it changes behaviour globally and invisibly. Patch where the name is **looked up**, not where it's defined. → Ch 15

Rapid-fire round

Question	One-line answer
`range` in Python 3?	A lazy, immutable sequence; O(1) `len` and `in` for ints.
Are strings mutable?	No; build with `join`, or `io.StringIO`.
What does `pass` do?	Nothing — a placeholder where a statement is required.
`append` vs `extend`?	`append(x)` adds one item; `extend(it)` adds each item of an iterable.
`sorted` vs `.sort()`?	New list vs in-place (returns `None`); both stable Timsort.
`del` vs `remove` vs `pop`?	By index/slice/name; by value (first match); by index returning the item.
What is `None`?	The single instance of `NoneType`, meaning "no value"; test with `is None`.
Why `self`?	Methods receive the instance explicitly; `obj.m()` is `Class.m(obj)`.
What is `__call__`?	Makes instances callable like functions.
What is a namedtuple?	A tuple subclass with named fields: lightweight immutable records.
`@functools.cache` vs `lru_cache`?	Unbounded vs size-limited memoisation.
`copy.copy` on a tuple of ints?	Returns the same tuple (immutable — no need to copy).
What is `__pycache__`?	Cached bytecode (`.pyc`) to skip recompiling unchanged modules.
What is `*` alone in a signature?	Everything after it is keyword-only.
What does `/` in a signature mean?	Everything before it is positional-only.
`match` statement?	Structural pattern matching (3.10): destructures sequences, mappings, classes.
What does `enumerate` return?	A lazy iterator of `(index, item)` tuples.

How to answer well

1. **Lead with the one-sentence definition**, then the mechanism, then a use case or trade-off.

2. **Show you know the edges**: "…except the free-threaded build", "…O(1) on average, O(n) worst case".
3. **Offer a tiny example** ("e.g. `@app.get` in FastAPI is a decorator").
4. **Say "I'd check the docs"** rather than invent a detail — interviewers value accuracy over bluffing.

Self-check

1. Why doesn't threading speed up a CPU-bound pure-Python loop in standard CPython? → The GIL lets only one thread run bytecode at a time.
2. What does `super()` resolve to in a diamond? → The next class in the instance's MRO.
3. When is `__new__` worth overriding? → Immutable subclasses, singletons/caching, metaclass-level control.
4. Name the two parts of CPython's memory reclamation. → Reference counting + cyclic GC.
5. Is `pickle.loads` on a user upload safe? → No — it can run arbitrary code.

TL;DR

Know the mechanism behind each feature: bytecode + VM; call by sharing; hash-table dicts (ordered since 3.7); iterators/generators as lazy protocols; decorators as `f = d(f)`; `__new__` creates, `__init__` initialises; MRO via C3; refcounting + cycle GC; the GIL blocks CPU parallelism in threads (processes or asyncio instead); hints are for tools, not the runtime.

Chapter TL;DR

- **Idioms:** EAFP with specific exceptions; unpacking (`a, b = b, a`, `first, *rest`); `enumerate` / `zip(strict=True)`; comprehensions and generator expressions; truthiness (`if not xs`, but `is not None` when `0` is valid); f-strings with format specs and `{x=}`; `with` for every resource; `open(..., encoding="utf-8")`; `pathlib.Path`; `"".join`; `for … else`; `:=`.
- **Gotchas:** mutable defaults (use `None`), late-binding closures (`i=i`), `is` vs `==`, int caching/interning (don't rely), mutating while iterating (new list or copy), shallow vs deep copy, `dict.fromkeys` / `defaultdict` surprises, class vs instance attributes, circular imports (restructure / import the module), float precision (`isclose`, `Decimal("…")`), bare `except` (catch specific), shadowing built-ins.
- **Interview core:** bytecode + interpreter; dynamic + strong typing; call by object reference; list = dynamic array, dict = ordered hash table; hashable = immutable + `__hash__` / `__eq__`; decorators, closures, generators; `__new__` / `__init__`; static vs class methods; MRO/C3 and `super()`; dunder methods; `__slots__`; ABC vs Protocol; refcounting + GC; pickle is unsafe for untrusted data; GIL → processes for CPU, threads/asyncio for I/O; type hints checked by tools, not at runtime.

Chapter 21 — Cheat Sheets and Glossary

This last chapter compresses Chapters 1–20 into pages you can scan in a minute: one-page **cheat sheets** by topic, a **"which tool for which job"** table, and an alphabetical **glossary** of 300+ terms, each pointing to the section where it is explained properly. Nothing here is new — if a line doesn't make sense, follow its section reference (written like **§6.2** = Chapter 6, section 2) back to the full explanation.

Every snippet with a visible result was checked by `code/cheatsheet_checks.py` (Python 3.14; everything shown is valid on 3.12 unless a version is noted).

```
OK  sheet_21_2_types_operators
OK  sheet_21_4_strings
OK  sheet_21_5_methods
OK  sheet_21_6_iteration
OK  sheet_21_7_functions
OK  sheet_21_8_oop
OK  sheet_21_9_exceptions
OK  sheet_21_11_tools
OK  sheet_21_18_copy
OK  sheet_21_19_numpy_pandas
all 10 cheat-sheet checks passed
```

What's inside

#	Sheet	Built from
21.1	Syntax and control flow	Ch 1–3
21.2	Types, numbers and operators	Ch 2
21.3	Data structures and big-O	Ch 5, 6, 17, 19
21.4	String methods and formatting	Ch 4
21.5	List, tuple, dict and set methods	Ch 5, 6
21.6	Comprehensions, iterators and generators	Ch 5, 6, 11
21.7	Functions and arguments	Ch 7
21.8	OOP and dunder methods	Ch 10, 12
21.9	Exceptions	Ch 9
21.10	Files, paths and data formats	Ch 9, 14
21.11	`itertools`, `functools`, `collections`	Ch 6, 7, 11, 12
21.12	Decorators, context managers, metaprogramming	Ch 12
21.13	Type hints	Ch 13
21.14	Standard-library quick picks	Ch 14
21.15	Modules, packages and environments	Ch 8
21.16	Testing and debugging	Ch 15
21.17	Concurrency: which tool?	Ch 16
21.18	Performance and internals	Ch 17
21.19	numpy, pandas, matplotlib essentials	Ch 18
21.20	Interview patterns and gotchas	Ch 19, 20
21.21	Which tool for which job	all
21.22	Glossary (A–Z)	all

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19 · 20

21.1 Syntax and control flow (§1.9–1.11, Ch 3)

Concept	Syntax / rule
Blocks	A line ending in `:` opens a block; its body is indented **4 spaces**; no braces (§1.9)
Comments / docstrings	`# comment` (discarded); `"""Docstring."""` as the first statement → `__doc__` (§1.10)
Line continuation	Implicit inside `()`, `[]`, `{}`; avoid backslashes (§1.9)
Names	Case-sensitive; `snake_case` variables/functions, `CapWords` classes, `UPPER_CASE` constants (§1.11)
Input / output	`input(prompt)` always returns `str`; `print(*values, sep=" ", end="\n")` (§1.7, §1.9)
Script entry point	`if __name__ == "__main__": main()` (§8.5)
`if`	`if c1: … elif c2: … else: …` – first truthy branch wins (§3.2)
Conditional expression	`x = a if cond else b` (§3.3)
`match` (3.10+)	`match cmd:` / `case "quit": …` / `case ["go", direction]: …` / `case {"id": id_}:` / `case Point(x=0):` / `case _:` – no fall-through; bare names **capture** (§3.4)
`for`	`for x in iterable:` – any iterable; `range(start, stop, step)` stop excluded (§3.6)
`while`	`while cond:`; `while True: … break` for loop-and-a-half (§3.7)
Loop control	`break` (leave innermost loop), `continue` (next iteration), `pass` (do nothing) (§3.8)
Loop `else`	Runs only if the loop finished **without** `break` (§3.9)
Walrus	`while (line := f.readline()):` – assign inside an expression, 3.8+ (§2.8)

```
for i, name in enumerate(names, start=1):      # index + value
for a, b in zip(xs, ys, strict=True):          # parallel, error on length mismatch
for k, v in d.items():                          # dict pairs
for x in reversed(seq): ...                     # backwards
for x in sorted(xs, key=len): ...               # sorted view, original unchanged
```

Zen and style: PEP 20 (`import this`) – readability counts, explicit beats implicit, errors shouldn't pass silently. PEP 8 is the style guide; let a formatter (Black/Ruff) apply it (§1.11).

21.2 Types, numbers and operators (Ch 2)

Type	Example	Mutable?	Notes
`int`	`42`, `0b101`, `0xff`, `1_000_000`	no	unbounded precision (§2.5)
`float`	`3.14`, `1e-3`, `float("inf")`	no	IEEE-754 double, ~15–17 significant digits (§2.5)
`complex`	`2 + 3j`	no	`.real`, `.imag`
`bool`	`True`, `False`	no	subclass of `int` (`True + 1 == 2`) (§2.6)
`str`	`"hi"`, `'hi'`, `"""multi"""`, `r"\d"`, `f"{x}"`	no	Unicode text (Ch 4)
`bytes` / `bytearray`	`b"\x00"`	no / yes	raw binary (§4.9)
`list`	`[1, "a"]`	yes	dynamic array (Ch 5)
`tuple`	`(1, 2)`, `1, 2`, `(1,)`	no	the comma makes the tuple (§5.9)
`range`	`range(0, 10, 2)`	no	lazy sequence
`dict`	`{"k": 1}`	yes	insertion-ordered (Ch 6)
`set` / `frozenset`	`{1, 2}`, `set()`	yes / no	`{}` is an empty **dict** (§6.6)
`NoneType`	`None`	no	the single "no value" object; test with `is None`

Falsy values: `None`, `False`, `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`. Everything else is truthy (§2.6).

Conversions (§2.7): `int("42")`, `int("ff", 16)`, `int(3.9)` → `3` (truncates), `float("1.5")`, `str(3)`, `bool([])` → `False`, `list("ab")` → `['a', 'b']`, `ord("A")` → `65`, `chr(65)` → `'A'`. Bad input raises `ValueError`; wrong type raises `TypeError`.

Operators (§2.8):

Kind	Operators	Watch out
Arithmetic	`+ - * / // % **`	`/` always float; `//` floors (`-7 // 2 == -4`); `%` takes the divisor's sign (`-7 % 3 == 2`); `**` is right-associative and binds tighter than unary minus (`-2 ** 2 == -4`)
Comparison	`== != < <= > >=`, chains `0 < x < 10`	compare floats with `math.isclose`
Identity / membership	`is`, `is not`, `in`, `not in`	`is` only for `None`/singletons
Logical	`and`, `or`, `not`	short-circuit; return an **operand** (`"" or "x"` → `'x'`)
Bitwise	`& | ^ ~ << >>`	`~x == -x - 1`
Assignment	`=`, `+=` …, `:=`	`+=` mutates lists in place, rebinds immutables
Built-in maths	`abs`, `round` (half-to-even: `round(2.5) == 2`), `divmod`, `pow(b, e, m)`, `min`, `max`, `sum`	

Money and exactness: `Decimal("0.1")` (from a string!) for money, `Fraction(1, 3)` for exact ratios, `math.isclose` for float comparison (§2.5, §14.8).

21.3 Data structures and big-O (§5.3, §6.2, §6.12, §17.5, §19.1)

Structure	Use for	Fast (avg)	Slow
`list`	ordered, growable sequence	`a[i]`, `a[i]=x`, `append`, `pop()` → O(1)	`insert(0,x)`, `pop(0)`, `x in a`, `remove`, `index` → O(n)
`tuple`	fixed record, dict key	`t[i]` O(1)	no mutation
`dict`	key → value	`d[k]`, `d[k]=v`, `del d[k]`, `k in d` → O(1)	worst case O(n) with collisions
`set`	uniqueness, membership	`add`, `discard`, `x in s` → O(1)	no indexing
`collections.deque`	queue / both-ends stack / last N	`append`, `appendleft`, `pop`, `popleft` → O(1)	middle index O(n)
`heapq` on a list	priority queue, top-k	push/pop O(log n), peek `h[0]` O(1), `heapify` O(n)	search O(n)
`bisect` on a sorted list	ordered search	search O(log n)	`insort` O(n)
`array.array`	compact numbers	O(1) index	Python-level maths not faster
`numpy.ndarray`	numeric arrays	vectorised maths in C	appending is O(n) (copies)

Operation	Cost	Operation	Cost
`sorted(xs)` / `xs.sort()`	O(n log n), O(n) if already sorted, **stable**	`a[i:j]` slice	O(j − i) − copies
`len(x)` (built-ins)	O(1)	`"".join(parts)`	O(total length)
`s1 | s2`	O(len s1 + len s2)	`s1 & s2`	O(min(len s1, len s2))
`min` / `max` / `sum`	O(n)	`Counter(xs)`	O(n)
`heapq.nlargest(k, xs)`	O(n log k)	`list(dict)`	O(n)

Rules: membership tests in a loop → use a `set`; queues → `deque`; repeated "smallest next" → `heapq`; never build strings with `+=` in a loop → `"".join` (§4.8, §17.5).

21.4 String methods and formatting (Ch 4)

Strings are **immutable**: every method returns a new string (§4.2). Index from 0; negative indexes count from the end; slices `s[start:stop:step]` exclude `stop` and never raise (§4.4).

Group	Methods (result for `s = " Hello, World "` where shown)
Case	`lower()`, `upper()`, `title()`, `capitalize()`, `swapcase()`, `casefold()` (aggressive lower for comparisons)
Strip	`strip()` → `'Hello, World'`, `lstrip()`, `rstrip()`, `strip("xy")` removes those *characters*; `removeprefix("ab")`, `removesuffix(".py")` remove an exact prefix/suffix (3.9+)
Split / join	`split()` (any whitespace), `split(",")`, `split(",", 1)`, `rsplit`, `splitlines()`, `partition(":")` → 3-tuple; `", ".join(items)`
Search	`x in s` (preferred test), `find(sub)` → index or `-1`, `index(sub)` → index or `ValueError`, `rfind`, `count(sub)`, `startswith(("a", "b"))`, `endswith`
Replace	`replace(old, new, count)`, `translate(str.maketrans("abc", "xyz"))`, `expandtabs()`
Checks	`isdigit()`, `isdecimal()`, `isnumeric()`, `isalpha()`, `isalnum()`, `isspace()`, `isupper()`, `islower()`, `isidentifier()`
Align	`center(20, "*")`, `ljust(10)`, `rjust(10)`, `zfill(5)` (`"42".zfill(5)` → `'00042'`)
Encode	`s.encode("utf-8")` → `bytes`; `b.decode("utf-8")` → `str` (§4.9)

Escapes: `\n \t \\ \' \" \xhh \uXXXX \N{name}`; raw strings `r"C:\path\n"` keep backslashes (use for regex) (§4.5).

f-string format spec – `{value:[[fill]align][sign][#][0][width][,][.precision][type]}` (§4.6):

Spec	Result	Spec	Result
`f"{3.14159:.2f}"`	`3.14`	`f"{1234567:,}"`	`1,234,567`
`f"{0.256:.1%}"`	`25.6%`	`f"{42:05d}"`	`00042`
`f"{'hi':>6}"`	`hi`	`f"{'hi':*^6}"`	`**hi**`
`f"{255:#x}"` / `f"{255:b}"`	`0xff` / `11111111`	`f"{1234.5:e}"`	`1.234500e+03`
`f"{x=}"` (3.8+)	`x=…`	`f"{obj!r}"`	uses `repr()`

Other formatting: `"{} {}".format(a, b)` for stored templates, `"%s" % x` in `logging` calls, `string.Template` for user-supplied templates, t-strings `t"…"` (3.14) produce a `Template` object for safe processing (§4.6).

Regex (§4.10, §14.3): `re.search` (anywhere), `re.match` (at start), `re.fullmatch` (whole string), `re.findall`, `re.finditer`, `re.sub(pattern, repl, s)`, `re.split`, `re.compile`; groups `(...)`, named `(?P<name>...)`; always raw strings; beware catastrophic backtracking on nested quantifiers.

21.5 List, tuple, dict and set methods (Ch 5, 6)

List (§5.4, §5.5):

Method	Does	Returns
`append(x)`	add one item at the end	`None`
`extend(iterable)`	add each item	`None`
`insert(i, x)`	insert before index i	`None`
`remove(x)`	delete first `x` (`ValueError` if absent)	`None`
`pop(i=-1)`	remove and return item i	the item
`clear()`	empty the list	`None`
`index(x[, start[, end]])`	first position (`ValueError` if absent)	int
`count(x)`	occurrences	int
`sort(key=None, reverse=False)`	sort in place (stable)	`None`
`reverse()`	reverse in place	`None`
`copy()`	shallow copy (= `a[:]`)	new list

Slicing: `a[1:3]`, `a[::-1]`, `a[::2]`; slice assignment `a[1:3] = [9, 9, 9]` can resize; `del a[::2]`. Copies: `=` aliases, `a.copy()` / `a[:]` / `list(a)` shallow, `copy.deepcopy(a)` deep; grids with `[[0] * n for _ in range(m)]`, never `[[0] * n] * m` (§5.8).

Tuple (§5.9–5.13): `count(x)`, `index(x)`; packing `t = 1, 2`; unpacking `a, *rest = t`; `namedtuple("P", "x y")` / `typing.NamedTuple` for named fields (`_asdict()`, `_replace()`).

Dict (§6.1, §6.4):

Method	Does
`d[k]` / `d[k] = v` / `del d[k]`	read (`KeyError` if missing) / insert-or-update / delete
`get(k, default=None)`	read without error
`setdefault(k, default)`	return `d[k]`, inserting `default` first if missing
`update(other)` / `d |= other` / `d1 | d2`	merge (right side wins); `|` returns a new dict (3.9+)
`pop(k[, default])`	remove and return
`popitem()`	remove and return the **last** inserted pair
`keys()`, `values()`, `items()`	live **views**; `keys()` / `items()` support set operations
`dict.fromkeys(keys, value)`	new dict — the *same* `value` object for every key
`copy()`, `clear()`	shallow copy, empty

Iterating a dict yields keys, in insertion order (guaranteed since 3.7). Don't add/remove keys while iterating — loop over `list(d)`.

Set (§6.6–6.8):

Operator	Method	Meaning
`a | b`	`union`	in either
`a & b`	`intersection`	in both
`a - b`	`difference`	in a, not b
`a ^ b`	`symmetric_difference`	in exactly one
`a <= b`, `a < b`	`issubset`	subset, proper subset
`a >= b`	`issuperset`	superset
–	`isdisjoint(b)`	no common items

Mutators: `add(x)`, `update(it)`, `remove(x)` (raises if absent), `discard(x)` (doesn't), `pop()` (arbitrary item), `clear()`, and in-place `|=`, `&=`, `-=`, `^=`. Methods accept any iterable; operators need sets. `frozenset` is the immutable, hashable version.

21.6 Comprehensions, iterators and generators (§5.6, §6.5, Ch 11)

```
[x * x for x in xs]                       # list
[x for x in xs if x > 0]                  # filter: `if` at the END
["pos" if x > 0 else "neg" for x in xs]   # choose: if-else BEFORE `for`
[y for row in grid for y in row]          # nested: same order as the loops
{k: v for k, v in pairs}                  # dict
{w.lower() for w in words}                # set
(x * x for x in xs)                       # generator expression — lazy
sum(x * x for x in xs)                    # parentheses can be dropped as sole argument
```

Concept	Rule
Iterable	has `__iter__` → returns an iterator (list, str, dict, file, range) (§11.1)
Iterator	has `__next__` (raises `StopIteration` at the end) and `__iter__` returning itself; **single-pass**
`for` loop	`it = iter(x)` then `next(it)` until `StopIteration`
Generator function	contains `yield`; calling it returns a generator (an iterator) without running the body (§11.2)
`yield from sub`	delegate to a sub-iterator (§11.3)
`send`, `throw`, `close`	two-way generators (§11.3)
Lazy vs eager	generators/ `map` / `filter` / `zip` / `range` produce on demand → constant memory, early exit (§11.4)
Built-in consumers	`list`, `tuple`, `set`, `dict`, `sum`, `min`, `max`, `any`, `all`, `sorted`, `"".join`, `next(it, default)`

```
def read_records(path):          # a pipeline stage: constant memory
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.strip():
                yield line.rstrip("\n").split(",")
```

21.7 Functions and arguments (Ch 7)

```
def f(pos_only, /, normal, *args, kw_only, **kwargs) -> None:
    """Docstring."""
```

Part	Meaning
before `/`	positional-only (§7.2)
normal	positional or keyword
`*args`	extra positionals → **tuple**
after `*` or `*args`	keyword-only
`**kwargs`	extra keywords → **dict**
defaults `x=1`	evaluated **once** at `def` time → use `None` for mutable defaults
call-site unpacking	`f(*seq, **mapping)`
return several	`return a, b` returns a tuple; no `return` → `None`

Topic	One-liner
Scope	**LEGB**: Local → Enclosing → Global → Built-in; assignment makes a name local for the whole function; `global x`, `nonlocal x` to rebind outer names (§7.3)
Closure	inner function remembering outer variables; binds **late** (§7.4)
First-class	functions are objects: pass, return, store; `key=` functions (§7.5)
`lambda`	`lambda x: x * 2` – one expression; don't assign to a name
`map` / `filter`	lazy iterators; comprehensions usually clearer (§7.5, §11.7)
Decorator	`@d` over `def f` ≡ `f = d(f)`; use `functools.wraps` (§7.6, §12.1)
Recursion	base case + progress; limit ~1000 frames (`sys.getrecursionlimit()`); no tail-call optimisation (§7.7)
Memoisation	`@functools.cache` / `@lru_cache(maxsize=128)`; arguments must be hashable (§7.6)
Docs and hints	docstring → `help(f)`; annotations are not enforced (§7.8)

21.8 OOP and dunder methods (Ch 10, §12.4–12.6)

```python
from dataclasses import dataclass, field

class Account:
    bank = "PyBank"                       # class attribute (shared)
    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner                # instance attribute
        self._balance = balance           # _ = internal by convention
    @property
    def balance(self) -> float:           # computed/validated attribute
        return self._balance
    @classmethod
    def empty(cls, owner):                # alternative constructor
        return cls(owner)
    @staticmethod
    def valid_amount(x) -> bool:          # helper, no self/cls
        return x > 0
    def __repr__(self):
        return f"Account({self.owner!r}, {self._balance})"

@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float = 0.0
    tags: tuple[str, ...] = ()            # mutable defaults need field(default_factory=list)
```

Syntax / built-in	Dunder	Syntax / built-in	Dunder
`Cls(...)` create / init	`__new__` / `__init__`	`repr(x)` / `str(x)`	`__repr__` / `__str__`
`x == y`, `x < y`	`__eq__`, `__lt__` …	`hash(x)`	`__hash__`
`x + y`, `y + x` fallback, `x += y`	`__add__`, `__radd__`, `__iadd__`	`bool(x)`	`__bool__` (else `__len__`)
`len(x)`	`__len__`	`x[k]`, `x[k] = v`, `del x[k]`	`__getitem__`, `__setitem__`, `__delitem__`
`for a in x`	`__iter__` / `__next__`	`a in x`	`__contains__`
`x(...)`	`__call__`	`with x:`	`__enter__` / `__exit__`
`x.attr` missing	`__getattr__`	every `x.attr`	`__getattribute__`
descriptor protocol	`__get__`, `__set__`, `__delete__`	subclass hook	`__init_subclass__`
`f"{x:spec}"`	`__format__`	pattern matching positions	`__match_args__`

Rule	Section
Always write `__repr__`; `__str__` falls back to it	§10.2
Defining `__eq__` without `__hash__` makes instances unhashable	§10.2
Return `NotImplemented` (not `False`) for unsupported operand types	§10.3
`super()` = next class in the instance's `MRO` (C3 linearisation); `Cls.__mro__`	§10.4, §10.5
Mixins add small capabilities via multiple inheritance	§10.6
Duck typing by default; ABCs enforce methods at instantiation; `Protocol` for structural static typing	§10.7
Prefer composition (has-a) unless it's a genuine is-a	§10.8
`_x` internal by convention; `__x` is name-mangled to `_Cls__x`, not private	§10.9
`@dataclass` options: `frozen`, `order`, `slots`, `kw_only`, `field(default_factory=…)`	§10.10
`enum.Enum`, `IntEnum`, `StrEnum`, `Flag`, `auto()`	§10.11
`__slots__` removes the per-instance `__dict__` → less memory, no new attributes	§10.12
`functools.total_ordering` fills in comparisons from `__eq__` + one of `__lt__` …	§10.2, §12.2

21.9 Exceptions (Ch 9)

```
try:
    value = int(text)
except ValueError as e:               # specific first; the first matching except wins
    log.warning("bad input %r: %s", text, e)
    value = 0
except (TypeError, KeyError):
    raise                             # re-raise unchanged
else:
    print("parsed OK")                # only if no exception
finally:
    cleanup()                         # always

class AppError(Exception): ...        # custom hierarchy
class NotFound(AppError): ...
raise NotFound("user 42") from err    # chain: keeps the cause (__cause__)
e.add_note("while loading config")    # 3.11+
```

Hierarchy (partial)	Raised when
`BaseException` → `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, `Exception`	root; don't catch it
`Exception` → `ArithmeticError` → `ZeroDivisionError`, `OverflowError`	maths
`LookupError` → `IndexError`, `KeyError`	bad index / missing key
`ValueError` (→ `UnicodeError`)	right type, bad value
`TypeError`	wrong type
`AttributeError`, `NameError` (→ `UnboundLocalError`)	missing attribute / name
`OSError` → `FileNotFoundError`, `PermissionError`, `TimeoutError`, `ConnectionError`	OS / I/O
`ImportError` → `ModuleNotFoundError`	import failed
`RuntimeError` → `RecursionError`, `NotImplementedError`	generic runtime problems
`StopIteration`	iterator exhausted
`ExceptionGroup` + `except*` (3.11+)	several errors at once, e.g. from `asyncio.TaskGroup` (§9.6)

Rules: keep `try` blocks small; never bare `except:` (catches Ctrl+C); prefer EAFP (§9.5); `assert` is for internal invariants only — it is removed by `python -O` (§9.7); read tracebacks bottom-up (§9.1, §15.10).

21.10 Files, paths and data formats (§9.8–9.11, §14.6)

```python
from pathlib import Path
p = Path("data") / "sales.csv"
p.parent.mkdir(parents=True, exist_ok=True)
text = p.read_text(encoding="utf-8");  p.write_text("a,b\n", encoding="utf-8")
with p.open(encoding="utf-8") as f:       # stream big files line by line
    for line in f: ...
with open("img.png", "rb") as f: data = f.read()        # binary: bytes, no encoding
for q in Path(".").rglob("*.py"): print(q.name, q.stem, q.suffix)
```

Mode	Meaning	Mode	Meaning
`"r"`	read text (default)	`"rb"` / `"wb"`	binary read / write
`"w"`	write, **truncates**	`"a"`	append
`"x"`	create, fail if it exists	`"r+"`	read and write

Task	Tool
Paths, globbing, read/write small files	`pathlib.Path` (§9.10)
Copy/move/delete trees, archives	`shutil.copytree`, `shutil.move`, `shutil.rmtree`, `shutil.make_archive`
Scratch files/dirs	`tempfile.TemporaryDirectory()`, `NamedTemporaryFile`
Atomic write	write to a temp file in the same folder, then `os.replace(tmp, final)`
CSV	`csv.reader/writer`, `csv.DictReader/DictWriter`; open with `newline=""`; values come back as `str` (§9.11)
JSON	`json.load/dump` (files), `json.loads/dumps` (strings), `indent=2`, `default=str` for unknown types
Python objects	`pickle` – Python-only, **never load untrusted data**
Tabular analytics	pandas `read_csv` / `read_parquet` (§18.4)
Embedded SQL	`sqlite3` with `?` placeholders (§14.11)

Always pass `encoding="utf-8"` for text files (§9.9).

21.11 `itertools`, `functools`, `collections` (§6.9, §7.6, §11.5, §12.2)

itertools (all lazy) – results for small inputs:

Function	Example → result
`count(start, step)`	`count(10, 5)` → 10, 15, 20, … (infinite)
`cycle(it)` / `repeat(x, n)`	`cycle("ab")` → a, b, a, …; `repeat(0, 3)` → 0, 0, 0
`chain(a, b)` / `chain.from_iterable(xss)`	`chain([1, 2], [3])` → 1, 2, 3
`islice(it, stop)` / `islice(it, start, stop, step)`	`islice(count(), 3)` → 0, 1, 2
`takewhile` / `dropwhile`	`takewhile(lambda x: x < 3, [1, 2, 5, 1])` → 1, 2
`groupby(it, key)`	groups **consecutive** equal keys – sort first
`accumulate(it[, func])`	`accumulate([1, 2, 3])` → 1, 3, 6
`pairwise(it)` (3.10+)	`pairwise("abc")` → ('a','b'), ('b','c')
`batched(it, n)` (3.12+)	`batched("abcde", 2)` → ('a','b'), ('c','d'), ('e',)
`zip_longest(a, b, fillvalue=None)`	pads the shorter input
`product(a, b)` / `product(a, repeat=2)`	Cartesian product
`permutations(it, r)`	`permutations("abc", 2)` → 6 ordered pairs
`combinations(it, r)`	`combinations("abc", 2)` → ab, ac, bc
`combinations_with_replacement`	allows repeats
`starmap(f, pairs)`	`f(*pair)` for each
`tee(it, n)`	n independent iterators from one

functools:

Tool	Use
`@cache` / `@lru_cache(maxsize=…)`	memoise pure functions; `.cache_info()`, `.cache_clear()`
`@cached_property`	compute an attribute once per instance
`@wraps(func)`	keep name/docstring in decorators
`partial(f, *args, **kw)`	pre-fill arguments
`reduce(f, it, initial)`	fold to one value (prefer `sum`, `max`, `math.prod` when they fit)
`@total_ordering`	derive comparison methods
`cmp_to_key(cmp)`	use an old-style comparator as `key=`
`@singledispatch`	overload a function by the first argument's type

collections:

Type	One-liner
`Counter(it)`	counts; `most_common(k)`; missing key → 0; `+ - & |`
`defaultdict(list)`	auto-creates missing values (a *read* inserts)
`deque(maxlen=n)`	O(1) at both ends; `rotate`; bounded buffer
`namedtuple("P", "x y")`	tuple with field names
`OrderedDict`	`move_to_end`, `popitem(last=False)`; order-sensitive `==`
`ChainMap(a, b)`	look up keys in several mappings in order

21.12 Decorators, context managers, metaprogramming (Ch 12)

```python
import functools, time
from contextlib import contextmanager

def timed(func):                                  # plain decorator
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__}: {time.perf_counter() - t0:.3f}s")
    return wrapper

def retry(times):                                 # decorator factory (3 levels)
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except OSError:
                    if attempt == times - 1:
                        raise
        return wrapper
    return decorator

@contextmanager
def opened(path):                                 # generator-based context manager
    f = open(path, encoding="utf-8")
    try:
        yield f                                   # value bound by `as`
    finally:
        f.close()                                 # always runs
```

Fact	Section
Stacked decorators are applied bottom-up, and the wrappers run top-down	§12.1
Decorators run at **definition** time; wrappers run at **call** time	§12.1
`__exit__` returning a truthy value suppresses the exception	§12.3
`contextlib` : `suppress` , `closing` , `nullcontext` , `redirect_stdout` , `chdir` (3.11+), `ExitStack` (dynamic number of managers, closes LIFO)	§12.3
Async versions: `async with` , `@asynccontextmanager`	§12.3
Descriptors (`__get__` / `__set__`) power `property` , methods, `classmethod` , `staticmethod` ; data descriptors beat the instance `__dict__`	§12.4
`__getattr__` runs only when normal lookup fails; `__getattribute__` runs for every access	§12.5
Metaclass = class of a class (`type` by default); prefer class decorators or `__init_subclass__`	§12.6
Monkey patching: fine in tests with automatic restore (`mock.patch` , `monkeypatch`); avoid in production	§12.7
Introspection: `type` , `isinstance` , `dir` , `vars` , `getattr` / `setattr` / `hasattr` , `inspect` ; never `eval` / `exec` untrusted input	§12.8

21.13 Type hints (Ch 13)

```python
from typing import Protocol, TypedDict, Literal, Final, NewType, Callable, Any

def greet(name: str, times: int = 1) -> str: ...
scores: dict[str, list[float]] = {}
maybe: int | None = None                        # = Optional[int]
def first[T](xs: list[T]) -> T: ...             # 3.12 generic syntax
type Matrix = list[list[float]]                 # 3.12 type alias statement
Handler = Callable[[str, int], bool]
Mode = Literal["r", "w"]
MAX: Final = 100
UserId = NewType("UserId", int)

class SupportsClose(Protocol):                  # structural: no inheritance needed
    def close(self) -> None: ...

class Movie(TypedDict):
    title: str
    year: int
```

Concept	Meaning	Section
Not enforced	the interpreter ignores hints; mypy/pyright check them statically	§13.1
Gradual typing	add hints where they pay off; unannotated code is `Any`	§13.1
Accept abstract, return concrete	parameters `Iterable` / `Sequence` / `Mapping` (from `collections.abc`), return `list` / `dict`	§13.3
Invariance	`list[bool]` is not a `list[int]`; `Sequence` is covariant	§13.3
Narrowing	`if x is not None:` / `isinstance` refine a union	§13.4
`Any` vs `object`	`Any` switches checking off; `object` allows only what every object supports	§13.5
`TypeVar` / bounds / constraints	generic functions and classes	§13.6
`ParamSpec`	type decorators that preserve signatures	§13.7
`Protocol`, `TypedDict`, `Literal`, `Final`, `NewType`	structural interface, typed dict shape, fixed values, constant, distinct zero-cost type	§13.8–13.10
Tools	`mypy --strict`, `pyright`, `reveal_type(x)`, `# type: ignore[code]`, stub packages `types-*`	§13.13
Runtime validation	pydantic / FastAPI at the boundary	§13.14

21.14 Standard-library quick picks (Ch 14)

Need	Module / call	Section
Dates and times	`datetime.now(UTC)`, `ZoneInfo("Europe/Paris")`, `isoformat()`, `fromisoformat()`, `timedelta` – store aware UTC	§14.1
Durations / timeouts	`time.perf_counter()` / `time.monotonic()`; `time.time()` = epoch seconds	§14.2
Regex	`re` (see 21.4)	§14.3
Logging	`log = logging.getLogger(__name__)`; configure once (`basicConfig` / `dictConfig`); `log.info("x=%s", x)`; `log.exception(...)` in `except`	§14.4
CLI arguments	`argparse.ArgumentParser`, `add_argument`, subcommands	§14.5
Environment, argv, exit	`os.environ.get("KEY", default)`, `sys.argv`, `sys.exit(code)`	§14.6
Run commands	`subprocess.run(["git", "status"], check=True, capture_output=True, text=True, timeout=30)` – a list, not a shell string	§14.6
Random	`random.Random(42)` for reproducible simulations; `secrets.token_urlsafe()` for tokens/passwords	§14.7
Maths / stats	`math.isclose`, `math.gcd`, `math.isqrt`, `math.comb`; `statistics.mean/median/stdev/quantiles`	§14.8
Exact numbers	`decimal.Decimal("0.10")`, `fractions.Fraction(1, 3)`	§14.8
Copy / pretty print	`copy.deepcopy`, `pprint.pp`	§14.9
IDs / hashes	`uuid.uuid4()`; `hashlib.sha256(b).hexdigest()` (use a KDF such as `hashlib.scrypt` for passwords)	§14.9
Embedded DB	`sqlite3.connect`, `with con:` transactions, `?` placeholders	§14.11
URLs / HTTP	`urllib.parse`; `urllib.request` for quick scripts; `requests` / `httpx` for real work	§14.12

21.15 Modules, packages and environments (Ch 8)

Task	Command / rule	Section
Import styles	`import json`, `from pathlib import Path`, `import numpy as np`; avoid `import *`	§8.1
How import works	check `sys.modules` → search `sys.path` → create module, register it, run it once → bind the name	§8.2
Package	folder with `__init__.py` (regular) or without (namespace); `__all__` controls `import *`	§8.3
Relative import	`from .utils import x` – only inside packages; run with `python -m pkg.mod`	§8.4
Script vs module	`if __name__ == "__main__":`	§8.5
Circular import fix	restructure; `import module` instead of `from module import name`; import inside the function	§8.6
Virtual env	`python -m venv .venv` → `source .venv/bin/activate` (Windows: `.venv\Scripts\activate`) → `python -m pip install pkg`	§8.7
Freeze / reproduce	`pip freeze > requirements.txt`, `pip install -r requirements.txt`	§8.8
Modern projects	`pyproject.toml` (metadata, dependencies, tool config) + lock file; `uv add pkg`, `uv run`, `uv sync`; Poetry similar	§8.8
Layout and publishing	`src/` layout; build sdist + wheel (`python -m build` / `uv build`); TestPyPI → PyPI (Trusted Publishing)	§8.9

21.16 Testing and debugging (Ch 15)

```python
# test_prices.py – run with: pytest -q
import pytest
from prices import apply_discount

@pytest.fixture
def cart():
    return {"apple": 1.0, "pear": 2.0}

@pytest.mark.parametrize("pct, expected", [(0, 3.0), (50, 1.5)])
def test_discount(cart, pct, expected):
    assert apply_discount(cart, pct) == pytest.approx(expected)

def test_rejects_negative(cart):
    with pytest.raises(ValueError, match="negative"):
        apply_discount(cart, -5)
```

Tool / idea	Key facts	Section
Test types	unit (one piece, fast), integration (pieces together), end-to-end (whole system); Arrange–Act–Assert	§15.1
`unittest`	`class T(unittest.TestCase)`, `setUp`, `assertEqual`, `assertRaises`; `python -m unittest`	§15.3
pytest	plain `assert`; fixtures (`yield` for teardown, `scope=`), `conftest.py`, `parametrize`, `tmp_path`, `monkeypatch`, `capsys`, `caplog`; marks `skip` / `skipif` / `xfail`	§15.4
pytest CLI	`-q`, `-x` (stop at first failure), `-k expr`, `-m mark`, `--lf` (last failed), `--pdb`	§15.4
Test doubles	dummy, stub, spy, mock, fake; `Mock(return_value=…, side_effect=…)`, `assert_called_once_with`; `patch("where.it.is.used")`; `autospec=True`	§15.5
Coverage	`pytest --cov=pkg --cov-branch --cov-report=term-missing`; coverage ≠ correctness	§15.6
doctest	examples in docstrings run as tests	§15.7
TDD	red → green → refactor	§15.8
Hypothesis	property-based tests: `@given(st.lists(st.integers()))`; shrinks failures	§15.9
Tracebacks	read the last line first, then walk up the frames	§15.10
Debugger	`breakpoint()` → pdb: `n` next, `s` step, `c` continue, `p expr`, `w` where, `u` / `d` up/down, `b` breakpoint, `q` quit	§15.11
Logging vs print	logging for anything that stays in the code	§15.12

21.17 Concurrency: which tool? (Ch 16)

Situation	Use	Why
Tens–hundreds of blocking I/O calls (HTTP, DB, files), sync libraries	`ThreadPoolExecutor` / `threading`	GIL is released while waiting
Thousands of concurrent connections, async libraries	`asyncio`	one thread, cheap tasks
CPU-bound pure Python	`ProcessPoolExecutor` / `multiprocessing`	one GIL per process → real parallelism
CPU-bound numeric work	numpy/pandas vectorisation	C loops
CPU-bound threads with shared memory, compatible deps	free-threaded build (`python3.14t`)	no GIL (optional build)
Pipeline stages	threads/processes + `queue.Queue` (bounded)	backpressure, sentinels for shutdown
Blocking call inside async code	`await asyncio.to_thread(func, ...)`	keeps the event loop responsive
Beyond one machine	Celery, RQ, Dask, Ray, Spark	distributed

	Threads	Processes	asyncio
Parallel CPU (standard build)	no	yes	no
Shared memory	yes (needs locks)	no (pickled messages)	yes (one thread)
Switching	pre-emptive	OS	cooperative at `await`
Cost per unit	~MB stack	whole interpreter	~KB

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
with ThreadPoolExecutor(max_workers=8) as ex:
    futures = [ex.submit(fetch, url) for url in urls]
    for fut in as_completed(futures):          # completion order; ex.map keeps input order
        print(fut.result())                    # re-raises the worker's exception

import asyncio
async def main():
    async with asyncio.TaskGroup() as tg:      # 3.11+: structured concurrency
        tasks = [tg.create_task(fetch_async(u)) for u in urls]
    return [t.result() for t in tasks]
asyncio.run(main())
```

Synchronisation (§16.4): `Lock` (mutual exclusion; `with lock:`), `RLock` (re-entrant), `Semaphore(n)` (at most n holders), `Event`, `Condition`. Pitfalls (§16.11): races on shared state, deadlocks (take locks in a global order, use timeouts), blocking calls in coroutines, lost exceptions in fire-and-forget tasks, missing `if __name__ == "__main__":` with processes.

21.18 Performance and internals (Ch 17)

Fact	Section
Everything is an object with identity (`id`), type and value; CPython objects carry a reference count and a type pointer	§17.1
Memory: reference counting frees objects immediately; a generational cyclic GC collects reference cycles; `weakref` refers without owning	§17.2
Small ints −5…256 and some strings are cached/interned → never compare values with `is`	§17.3
Assignment shares; `.copy()` is shallow; `deepcopy` is deep	§17.4

Measure first (§17.6):

Question	Tool
How fast is this snippet?	`python -m timeit "..."`, `timeit.timeit(fn, number=…)`, `%timeit` in Jupyter
Which functions are slow?	`python -m cProfile -s cumtime script.py`, `pstats` (ncalls, tottime, cumtime)
Which lines?	`line_profiler` (`kernprof -l -v`)
Live / low-overhead	`py-spy top --pid N`, `scalene`, `pyinstrument`
Which lines allocate memory?	`tracemalloc` snapshots

Optimise in this order (§17.7): better algorithm/data structure (set for membership, dict for lookup, deque for queues) → cache repeated work (`functools.cache`) → vectorise with numpy/pandas → stream with generators → built-ins, comprehensions, `str.join`, local variables in hot loops → `__slots__` for many small objects → compile hot spots (numba, Cython, Rust/PyO3, mypyc) or use PyPy → parallelise (processes). Newer CPython releases (3.11+) are faster for free.

21.19 numpy, pandas, matplotlib essentials (Ch 18)

numpy (§18.1–18.3):

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])       # shape (2, 3), dtype int64
a.shape, a.ndim, a.dtype, a.size
np.zeros((2, 3)); np.ones(3); np.arange(0, 10, 2); np.linspace(0, 1, 5); np.eye(3)
a[0, 1]; a[:, 1]; a[a > 2]                  # index, column slice (view), boolean mask (copy)
a.sum(axis=0)                               # per column -> [5, 7, 9]
a.mean(axis=1)                              # per row    -> [2., 5.]
a * 10; a + np.array([10, 20, 30])          # vectorised + broadcasting
a @ a.T                                     # matrix product
a.reshape(3, 2); a.T; np.concatenate([a, a]); np.vstack; np.hstack
rng = np.random.default_rng(42); rng.normal(size=3); rng.integers(0, 10, 5)
np.linalg.solve(A, b); np.linalg.eigh(S)
```

Rules: slices are **views** (changing them changes the original; `.copy()` to detach); masks and fancy indexing return copies; broadcasting compares shapes from the right – dimensions must be equal or one of them 1; `axis=0` collapses rows (result per column), `axis=1` collapses columns (result per row); vectorise instead of Python loops.

pandas (§18.4–18.11, pandas 3.x):

Task	Code
Load / save	`pd.read_csv(path, parse_dates=["date"])`, `pd.read_parquet`, `df.to_parquet`
First look	`df.head()`, `df.info()`, `df.describe()`, `df["col"].value_counts()`, `df.isna().sum()`
Select	`df["col"]`, `df[["a", "b"]]`, `df.loc[row_labels, cols]` (labels, end **inclusive**), `df.iloc[0:5, 1]` (positions, end exclusive)
Filter	`df[(df.qty > 5) & (df.city == "Oslo")]` (use `&`, `|`, `~` with parentheses), `df.query("qty > 5")`
New columns	`df["rev"] = df.qty * df.price`, `df.assign(rev=lambda d: d.qty * d.price)`, `np.where`, `np.select`, `pd.cut`
Missing data	`isna`, `dropna`, `fillna`, `pd.to_numeric(errors="coerce")`
Group	`df.groupby("city")["rev"].agg(["sum", "mean"])`, named aggregation `agg(total=("rev", "sum"))`, `transform` keeps the original shape
Combine	`pd.merge(a, b, on="id", how="left", validate="m:1", indicator=True)`, `pd.concat([a, b])`
Reshape	`pivot_table(index=, columns=, values=, aggfunc=)`, `melt` (wide → long)
Dates	`pd.to_datetime(col, format=...)`, `.dt.month`, `set_index("date").resample("ME").sum()` (`"ME"` month end in pandas ≥ 2.2/3.0), `rolling(7).mean()`
Sort / top-N	`sort_values("rev", ascending=False)`, `nlargest(3, "rev")`
Speed / memory	vectorise instead of `apply`; `astype("category")` for repeated strings; Parquet over CSV

pandas 3.0 notes (§18.4–18.5): text columns use a dedicated `str` dtype; **Copy-on-Write** is always on → assign with a single `df.loc[mask, "col"] = value`, never chained indexing.

matplotlib (§18.12):

```
import matplotlib
matplotlib.use("Agg")                       # headless scripts
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6, 4))      # object-oriented API
ax.plot(x, y, label="sales"); ax.bar(cats, vals); ax.scatter(x, y); ax.hist(data, bins=20)
ax.set(title="Sales", xlabel="Month", ylabel="EUR"); ax.legend()
fig.tight_layout(); fig.savefig("img/sales.png", dpi=120); plt.close(fig)
```

21.20 Interview patterns and gotchas (Ch 19, 20)

Pattern → Python tool (Ch 19):

Signal in the problem	Pattern	Python
sorted array, pair/triple sums	two pointers	indices `lo`, `hi` (§19.2)
contiguous subarray/substring	sliding window / prefix sums	`dict` of last seen, `accumulate`, `defaultdict(int)` (§19.2)
"seen before?", counts, groups	hashing	`set`, `Counter`, `defaultdict(list)` (§19.3)
matching brackets, "next greater"	stack / monotonic stack	`list.append/pop` (§19.4)
FIFO, level order, window max	queue / monotonic deque	`collections.deque` (§19.4)
k largest / smallest, merge k, running median	heap / two heaps	`heapq` (negate for max-heap) (§19.5)
sorted search, minimum feasible value, weighted random pick	binary search	`bisect_left/right`, `lo < hi` template, prefix sums (§19.6)
linked list, tree, serialize a tree	node classes, DFS/BFS	`@dataclass` nodes, pre-order with `#` markers (§19.7)
shortest path unweighted / weighted / dependencies	BFS / Dijkstra / topological sort	`deque`, `heapq`, in-degree counts (§19.8)
dynamic connectivity, cycles, MST	union-find	`parent` list + path compression (§19.9)
optimal value with overlapping subproblems	DP	`@cache` or a table (§19.10)
all combinations / permutations / placements	backtracking	choose → recurse → undo; `path[:]` (§19.11)
bits, parity, powers	bit tricks, number theory	`x & (x-1)`, `pow(b, e, m)`, `math.gcd`, `math.isqrt` (§19.12)
prefix lookups, autocomplete, wildcard, words on a grid	trie	`dict` of children + end-of-word field (§19.13)
locally best choice is provably safe	greedy	one pass after `sorted(key=...)`; justify by exchange argument (§19.14)
merge / insert / overlap ranges, rooms needed	intervals	sort by start (merge) or end (keep most); `heapq` of ends; sweep line (§19.15)
cycle entry, middle, O(1)-space list checks	fast & slow pointers	`slow`, `fast`, compare with `is` (§19.16)
k-th element, three values, sort a linked list	quickselect / Dutch flag / merge sort	random-pivot partition; `low` / `mid` / `high` (§19.17)
spiral/rotate matrix, 32-bit limits, collinear points	math & geometry	boundaries; explicit range check; `gcd`-reduced slopes (§19.18)

Top gotchas (§19.19, §20.2): mutable default args (`None` sentinel) · late-binding closures (`lambda i=i: i`) · `is` vs `==` · int caching/interning · mutating a list/dict while iterating · shallow copies and `[[0]*n]*m` · `dict.fromkeys(k, [])` shares one list; a `defaultdict` read inserts · mutable class attributes shared by instances · circular imports · float precision (`isclose`, `Decimal("…")`) · bare `except:` · shadowing built-ins (`list = …`) · `xs.sort()` returns `None` · `//` floors toward −∞ · recursion limit ~1000.

21.21 Which tool for which job

Job	Reach for	Not	Section
Ordered collection you append to	`list`	`tuple` (immutable)	§5.2
Fixed record / multiple return values / dict key	`tuple`, `NamedTuple`, frozen `dataclass`	`list`	§5.9, §5.13, §10.10
Look up by key	`dict`	list of pairs + search	§6.1
Membership / de-duplication	`set` (`dict.fromkeys` to keep order)	`x in list` in a loop	§6.6
Counting	`Counter`	manual dict with `if k in d`	§6.9
Grouping	`defaultdict(list)`	`setdefault` everywhere	§6.9
Queue / sliding buffer	`deque` (`maxlen`)	`list.pop(0)`	§6.9
Priority queue / top-k	`heapq`	repeated `sort`	§6.10
Sorted inserts and range queries	`bisect` (or `sortedcontainers`)	re-sort after each insert	§6.11
Numeric arrays / maths	NumPy	Python loops over lists	§5.14, §18.1
Tables, group-by, joins	pandas	nested dicts	§18.4
Simple data class	`@dataclass`	hand-written `__init__` / `__repr__` / `__eq__`	§10.10
Validate untrusted input (JSON, API)	pydantic	trusting type hints	§13.14
Named constants / fixed choices	`enum.Enum` / `StrEnum`	bare strings	§10.11
Interface without inheritance	`typing.Protocol`	ABC (when you don't control the classes)	§10.7, §13.8
Reuse behaviour	composition, small mixins	deep inheritance trees	§10.6, §10.8
Cross-cutting behaviour (timing, retry, auth)	decorator	copy-pasted code	§12.1
Setup/teardown that must happen	`with` + context manager	manual `close()`	§9.8, §12.3
Huge or infinite sequences	generators, `itertools`	building full lists	§11.2, §11.5
Paths	`pathlib`	string concatenation, `os.path` sprawl	§9.10
Config / data exchange	JSON (or TOML for config)	pickle	§9.11
Persist Python objects you trust	`pickle` / `joblib`	pickle from strangers	§9.11
Small local database	`sqlite3`	ad-hoc files	§14.11
Diagnostics in running code	`logging`	`print`	§14.4, §15.12
CLI arguments	`argparse` (or click/typer)	parsing `sys.argv` by hand	§14.5
External commands	`subprocess.run([...], check=True)`	`os.system`, `shell=True` with user input	§14.6

Job	Reach for	Not	Section
Tokens / passwords	`secrets`	`random`	§14.7
Money	`Decimal("…")` or integer cents	`float`	§2.5, §14.8
Timestamps	aware `datetime` in UTC + `zoneinfo`	naive local times	§14.1
Durations	`time.perf_counter()`	`time.time()`	§14.2
Tests	pytest (+ fixtures, parametrize)	manual prints	§15.4
Replace a dependency in a test	`unittest.mock.patch` / `monkeypatch`	editing production code	§15.5
Many I/O waits, sync libraries	threads / `ThreadPoolExecutor`	processes	§16.8
Many I/O waits, async libraries	`asyncio`	one thread per connection	§16.9
CPU-heavy pure Python	`ProcessPoolExecutor`	threads (standard build)	§16.7
Find the slow part	`cProfile`, `py-spy`	guessing	§17.6
Memoise a pure function	`functools.cache`	hand-rolled global dict	§7.6
Dependencies per project	`venv` + `pyproject.toml` (+ `uv`)	global `pip install`	§8.7, §8.8
Type-check a codebase	mypy / pyright in CI	runtime `isinstance` everywhere	§13.13

21.22 Glossary (A–Z)

Each entry: **term** — plain meaning — section where it is explained. Links go to the chapter README.

Term	Meaning	Where
ABC (abstract base class)	A class (from `abc`) with `@abstractmethod`s that subclasses must implement before they can be instantiated	10.7
Absolute import	Import by full path from the project/package root (`from pkg.sub import x`)	8.4
Accumulate	`itertools.accumulate`: running totals (or running results of any two-argument function)	11.5
Aliasing	Two names bound to the same object, so a change through one is seen through the other	5.8
Amortised O(1)	An operation that is occasionally expensive but constant on average over many calls (e.g. `list.append`)	5.3
Annotation	A type hint attached to a parameter, return value or variable; stored in `__annotations__`, not enforced	13.2
`Any`	Type-hint escape hatch that disables checking for a value	13.5
argparse	Standard-library module for command-line options, arguments and subcommands	14.5
Argument	The actual value passed at a call (vs the parameter name in the definition)	7.1
`*args`	Parameter collecting extra positional arguments into a tuple	7.2
Arrange–Act–Assert	Test structure: set up, perform the action, check the result	15.1
`array.array`	Compact, typed array of C numbers from the standard library	5.1
`assert`	Statement checking an internal invariant; removed when Python runs with `-O`	9.7
AST (abstract syntax tree)	Tree form of parsed source code that CPython compiles to bytecode	1.8
asyncio	Standard library for single-threaded concurrency with an event loop and `async` / `await`	16.9
Attribute	A name attached to an object (`obj.name`): data or method	10.1
Augmented assignment	`x += 1` and friends; mutates in place for mutable objects that support it	2.8
Autospec	Mock option that makes a mock follow the real object's signature	15.5
Aware datetime	A `datetime` that carries a time zone (`tzinfo`); comparable across zones	14.1
Backpressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	16.5
Backtracking	Search that builds a candidate step by step and undoes steps that can't succeed	19.11
Banker's rounding	Round-half-to-even, used by `round()` (`round(2.5) == 2`)	2.5
Base case	The input where a recursive function stops recursing	7.7
`BaseException`	Root of all exceptions, including `SystemExit` and `KeyboardInterrupt`; catch `Exception` instead	9.1
BFS (breadth-first search)	Graph/tree traversal level by level with a queue; shortest paths in unweighted graphs	19.8
Big-O	Notation for how cost grows with input size, ignoring constants	17.5
Binary search	Finding a position in sorted data by halving the range each step, O(log n)	19.6
bisect	Standard-library binary search and sorted insertion on lists	6.11
Block	Group of statements under a line ending in `:`, marked by indentation	1.9
Bound method	A function retrieved through an instance, with `self` already filled in	10.1
Branch coverage	Coverage metric that checks both outcomes of each decision were executed	15.6

Term	Meaning	Where
`breakpoint()`	Built-in that drops into the debugger (pdb by default) at that line	15.11
Broadcasting	NumPy rule for combining arrays of different shapes by stretching size-1 dimensions	18.2
Bytecode	The low-level instructions CPython compiles source into and then interprets	1.8
`bytes`	Immutable sequence of raw bytes (0–255); text must be encoded to become bytes	4.9
`cached_property`	Decorator computing an attribute once per instance and storing it	12.4
Call stack	The stack of active function frames; its depth is limited for recursion	7.7
`Callable`	Type hint for function values, e.g. `Callable[[int], str]`	13.7
Capture pattern	A bare name in a `case` pattern that binds the matched value	3.4
Categorical (dtype)	pandas type storing repeated strings as small integer codes — saves memory	18.11
Chained comparison	`a < b < c` meaning `a < b and b < c`, evaluating `b` once	2.8
ChainMap	`collections` mapping that searches several dicts in order	6.9
Circular import	Two modules importing each other so one sees the other half-initialised	8.6
Class	Blueprint that bundles data (attributes) and behaviour (methods)	10.1
Class attribute	Attribute defined in the class body, shared by all instances	10.1
Class decorator	A decorator applied to a class instead of a function	12.1
`classmethod`	Method receiving the class (`cls`) — used for alternative constructors	10.1
Closure	Inner function that remembers variables from its enclosing function	7.4
Code object	Compiled bytecode plus constants and names for one function/module	1.8
Code point	The number Unicode assigns to a character (`ord("A") == 65`)	4.9
Comprehension	Expression building a list/dict/set from an iterable: `[f(x) for x in xs if c]`	5.6
Composition	Building objects that *have* other objects instead of inheriting from them	10.8
Concurrency	Making progress on several tasks by interleaving them (not necessarily at the same instant)	16.1
`concurrent.futures`	High-level executors (`ThreadPoolExecutor`, `ProcessPoolExecutor`) returning futures	16.8
`conftest.py`	pytest file whose fixtures are shared by tests in its directory	15.4
Context manager	Object with `__enter__` / `__exit__` used by `with` to guarantee setup and cleanup	9.8
`contextmanager`	`contextlib` decorator turning a one-`yield` generator into a context manager	12.3
Copy-on-Write	pandas 3 rule: derived objects behave as copies; modify with one `.loc` assignment	18.5
Coroutine	The object returned by calling an `async def` function (a *coroutine function*); the event loop runs it, pausing at each `await`	16.9
Counter	`collections` dict subclass for counting hashable items	6.9
Coverage	Measurement of which lines/branches the tests executed	15.6
cProfile	Built-in deterministic profiler reporting time per function	17.6
CPU-bound	Work limited by computation speed rather than waiting	16.2
CPython	The reference Python implementation, written in C	1.8
Critical section	Code touching shared state that only one thread may run at a time	16.4

Term	Meaning	Where
CSV	Comma-separated text tables; `csv` module; all values read back as strings	9.11
dataclass	`@dataclass` generates `__init__`, `__repr__`, `__eq__` from annotated fields	10.10
DataFrame	pandas 2-D labelled table of columns (each column a Series)	18.4
Deadlock	Threads waiting for each other's locks forever	16.11
`Decimal`	Exact decimal arithmetic type for money; construct from strings	14.8
Decorator	Callable that takes a function (or class) and returns a replacement; `@d` means `f = d(f)`	12.1
Deep copy	Recursive copy of an object and everything it contains (`copy.deepcopy`)	5.8
`defaultdict`	Dict that creates a default value for a missing key on access	6.9
deque	Double-ended queue with O(1) appends/pops at both ends	6.9
Descriptor	Object with `__get__` / `__set__` / `__delete__` that controls attribute access on a class	12.4
DFS (depth-first search)	Traversal that goes as deep as possible before backtracking (recursion or stack)	19.8
dict	Mutable mapping from hashable keys to values; hash table, insertion-ordered	6.1
Dict view	Live, set-like object from `keys()` / `items()` / `values()`	6.4
Dijkstra's algorithm	Shortest paths with non-negative weights using a min-heap	19.8
Docstring	String literal as the first statement of a module/class/function, shown by `help()`	1.10
doctest	Runs examples written in docstrings as tests	15.7
dtype	NumPy/pandas element type (`int64`, `float64`, `bool`, `category`…)	18.1
Duck typing	Using an object by what it can do, not by its class	10.7
Dunder method	"Double underscore" special method like `__len__` that Python calls for syntax	10.2
Dynamic array	Array that over-allocates and grows by copying — how `list` works	5.3
Dynamic programming (DP)	Solving overlapping subproblems once and combining their answers	19.10
Dynamic typing	Types belong to objects; names can be rebound to any type	2.3
EAFP	"Easier to ask forgiveness than permission": try, then handle the exception	9.5
Editable install	Installing a project so source edits take effect without reinstalling (`pip install -e .`)	8.8
Encoding	Rule turning text into bytes (UTF-8 is the standard)	4.9
Enum	Class of named constant members (`enum.Enum`, `StrEnum`, `auto()`)	10.11
`enumerate`	Built-in yielding `(index, item)` pairs	3.6
Epoch	Reference time 1970-01-01 UTC that `time.time()` counts seconds from	14.2
Event loop	The asyncio scheduler that runs ready coroutines and waits for I/O	16.9
Exception	Object representing an error, raised to unwind the stack until handled	9.1
Exception chaining	Linking a new exception to its cause with `raise … from …`	9.4
ExceptionGroup	Exception bundling several errors, handled with `except*` (3.11+)	9.6
Executor	Pool that runs callables on threads or processes and returns futures	16.8
ExitStack	`contextlib` tool managing a dynamic number of context managers	12.3

Term	Meaning	Where
f-string	String literal `f"…{expr:spec}…"` that evaluates and formats expressions	4.6
Fake	Test double with a working lightweight implementation (e.g. in-memory DB)	15.5
Falsy	Value treated as false in a condition: `None`, `False`, zeros, empty containers	2.6
Fast and slow pointers	Two pointers moving at speeds 1 and 2 to find a list's middle or a cycle's entry in O(1) space (Floyd)	19.16
Figure / Axes	matplotlib's whole image / a single plot area within it	18.12
`Final`	Type hint marking a name that must not be reassigned	13.10
First-class function	Functions are objects: they can be stored, passed and returned	7.5
Fixture	pytest function providing setup (and teardown) injected into tests by name	15.4
float	IEEE-754 double-precision number; approximate for most decimals	2.5
Floor division	`//`: divide and round toward negative infinity	2.5
for-else	`else` block that runs when a loop ends without `break`	3.9
Format spec	Mini-language after `:` in f-strings (`>10`, `,.2f`, `%`, `x`)	4.6
`Fraction`	Exact rational number type (`fractions.Fraction(1, 3)`)	14.8
Frame	Per-call workspace holding a function's local variables	7.1
Free-threaded build	Optional CPython build without the GIL (experimental in 3.13, supported in 3.14)	16.6
frozenset	Immutable, hashable set	6.8
functools	Module of higher-order tools: `cache`, `lru_cache`, `partial`, `reduce`, `wraps`	7.6
Future	Placeholder for a result that will be available later (`.result()`)	16.8
Garbage collector (cyclic)	CPython component that finds and frees reference cycles	17.2
Generator	Iterator produced by a function containing `yield`; pauses between values	11.2
Generator expression	Lazy comprehension in parentheses: `(x * x for x in xs)`	11.2
Generic type	Type parameterised by other types (`list[int]`, `dict[str, float]`)	13.3
GIL (Global Interpreter Lock)	CPython lock letting one thread run Python bytecode at a time	16.6
`global`	Statement letting a function rebind a module-level name	7.3
Gradual typing	Adding type hints incrementally; unannotated code is treated as `Any`	13.1
Greedy algorithm	Builds an answer by always taking the locally best choice; correct only when an exchange argument proves it	19.14
groupby (itertools)	Groups *consecutive* items with equal keys – sort first	11.5
groupby (pandas)	Split–apply–combine: aggregate or transform rows per group	18.7
Guard (match)	`if` condition attached to a `case` pattern	3.4
Guard clause	Early `return` / `continue` for special cases to keep code flat	3.2
Hash	Integer computed from a value by `hash()`, used to place it in a hash table	6.2
Hash table	Array of slots indexed by hash – the structure behind dict and set	6.2
Hashable	Has a hash that never changes and consistent `__eq__`; can be a dict key or set item	6.2

Term	Meaning	Where
hashlib	Standard-library cryptographic hash functions (SHA-256…)	14.9
Heap / heapq	Binary min-heap stored in a list; priority queues with O(log n) push/pop	6.10
Higher-order function	Function that takes or returns functions (`map` , `sorted(key=…)` , decorators)	7.5
Hypothesis	Library for property-based testing that generates and shrinks inputs	15.9
I/O-bound	Work that spends most of its time waiting on network, disk or other I/O	16.2
IDE	Integrated development environment (VS Code, PyCharm)	1.6
Identifier	A name for a variable, function, class, etc.	1.9
Identity	Which object a name refers to; compared with `is` , shown by `id()`	2.2
`iloc` / `loc`	pandas selection by integer position / by label (label slices include the end)	18.5
Immortal object	Object (like `None` or small ints) whose refcount never changes, since 3.12	17.1
Immutable	Cannot change after creation (`int` , `str` , `tuple` , `frozenset`)	2.2
Import system	Machinery that finds, loads, caches (`sys.modules`) and binds modules	8.2
Inheritance	A subclass reuses and extends a base class ("is-a")	10.4
Instance	An object created from a class	10.1
Instance attribute	Attribute stored on one object (usually set in `__init__`)	10.1
Integration test	Test of several components working together	15.1
Interning (strings)	Reusing one object for equal strings (identifier-like literals, `sys.intern`)	17.3
Introspection	Examining objects at runtime (`type` , `dir` , `getattr` , `inspect`)	12.8
Invariance	Generic container types don't accept subtypes: `list[bool]` is not `list[int]`	13.3
Iterable	Object that can return an iterator (`__iter__`): anything you can loop over	11.1
Iterator	Object producing items one at a time with `__next__` ; single-pass	11.1
itertools	Standard module of lazy iterator building blocks	11.5
JSON	Text data format; `json` module; safe and portable, limited types	9.11
Keyword	Reserved word of the language (`if` , `def` , `class` …)	1.9
Keyword argument	Argument passed by name (`f(x=1)`)	7.2
Keyword-only parameter	Parameter after `*` or `*args` that must be passed by name	7.2
`**kwargs`	Parameter collecting extra keyword arguments into a dict	7.2
`lambda`	Anonymous one-expression function	7.5
Late binding	Closures look up variables when called, not when created	7.4
Lazy evaluation	Computing values only when requested (generators, `range` , `map`)	11.4
LBYL	"Look before you leap": check conditions before acting	9.5
LEGB	Name lookup order: Local, Enclosing, Global, Built-in	7.3
Linked list	Nodes each pointing to the next; built with small classes in Python	19.7
Linter	Tool that flags bugs and style problems without running code (Ruff, Flake8)	1.6
list	Mutable, ordered sequence of any objects (dynamic array)	5.2
`Literal`	Type hint restricting a value to specific constants	13.10

Term	Meaning	Where
Lock	Synchronisation primitive allowing one thread into a critical section	16.4
Lock file	File pinning exact versions of every dependency for reproducible installs	8.8
Logger / Handler / Formatter	logging's named source / output destination / message layout	14.4
`lru_cache`	Memoising decorator with a size limit, evicting least-recently-used entries	7.6
`match` statement	Structural pattern matching on a subject's shape and values (3.10+)	3.4
Memoization	Caching a function's results by its arguments	7.7
merge (pandas)	SQL-style join of two DataFrames on key columns	18.8
Metaclass	The class of a class (default `type`); controls class creation	12.6
Method	Function defined in a class and called on its instances	10.1
Mixin	Small class adding one capability through multiple inheritance	10.6
Mock	Test double that records calls and returns configured values	15.5
Module	A `.py` file whose names can be imported	8.1
Monkey patching	Replacing attributes of modules/classes at runtime	12.7
Monotonic stack	Stack kept in sorted order to answer "next greater/smaller" in O(n)	19.4
MRO (method resolution order)	Order Python searches classes for attributes, computed by C3 linearisation	10.5
multiprocessing	Running work in separate processes, each with its own interpreter and GIL	16.7
Mutable	Can be changed in place (`list`, `dict`, `set`)	2.2
Mutable default argument	Default like `[]` created once and shared by all calls — use `None`	7.2
mypy	Static type checker for Python type hints	13.13
Naive datetime	A `datetime` without time-zone information	14.1
Name mangling	`__attr` inside a class is renamed to `_Class__attr` to avoid subclass clashes	10.9
namedtuple	Tuple subclass with named fields	5.13
Namespace	Mapping from names to objects (module globals, a function's locals…)	7.3
Namespace package	Package directory without `__init__.py`, possibly spread over several folders	8.3
Narrowing	A type checker refining a union after a check like `is not None`	13.4
ndarray	NumPy's typed, contiguous, n-dimensional array	18.1
`NewType`	Creates a distinct type for checkers with zero runtime cost	13.10
`None`	The single "no value" object; functions without `return` give it	2.4
`nonlocal`	Statement letting an inner function rebind a variable of its enclosing function	7.3
Normalization (Unicode)	Converting equivalent character sequences to one form (NFC/NFD)	4.9
Object	Any value in Python: has identity, type and value	17.1
Open addressing	Hash-collision strategy that probes other slots in the same table	6.2
Operator overloading	Defining dunder methods so operators work on your types	10.3
`Optional[X]`	Type hint meaning `X | None`	13.4
OrderedDict	Dict with `move_to_end` and order-sensitive equality; handy for LRU caches	6.9

Term	Meaning	Where
Package	Directory of modules importable as a unit	8.3
Parallelism	Literally running several computations at the same instant on multiple cores	16.1
Parameter	Name in a function definition that receives an argument	7.1
`ParamSpec`	Type-hint variable capturing a function's full parameter list (for decorators)	13.7
Parametrize	`@pytest.mark.parametrize`: run one test with many inputs	15.4
Parquet	Columnar, compressed file format that keeps dtypes; fast with pandas	18.4
`partial`	`functools.partial`: new callable with some arguments pre-filled	7.6
pathlib	Object-oriented file-system paths (`Path("a") / "b.txt"`)	9.10
pdb	Python's built-in command-line debugger	15.11
PEP	Python Enhancement Proposal — design documents for the language	1.2
PEP 8	Python's style guide	1.11
PEP 20 (Zen of Python)	Aphorisms describing Python's design philosophy (`import this`)	1.11
pickle	Serialises Python objects to bytes; unsafe to load from untrusted sources	9.11
pip	The standard package installer (`python -m pip install …`)	8.7
pivot_table / melt	pandas reshaping: long → wide summary / wide → long	18.9
Positional-only parameter	Parameter before `/` that can't be passed by keyword	7.2
Prefix sum	Running totals that give any range sum in O(1)	19.2
Priority queue	Queue that always serves the smallest (most urgent) item next — `heapq`	6.10
Producer–consumer	Pattern where producers put work on a queue and consumers take it	16.5
Profiling	Measuring where a program spends time or memory	17.6
`property`	Decorator turning methods into a managed attribute (getter/setter)	10.1
Property-based testing	Testing general properties over many generated inputs	15.9
`Protocol`	Structural interface for type checkers — match by methods, not inheritance	13.8
Pydantic	Library validating and parsing data at runtime from type hints	13.14
PyPI	The Python Package Index, where `pip` downloads packages	8.7
pymalloc	CPython's allocator for small objects	17.2
PyPy	Alternative Python implementation with a JIT compiler	1.8
`pyproject.toml`	Standard project file: metadata, dependencies, build system, tool config	8.8
pyright	Fast static type checker (also powers VS Code's Pylance)	13.13
pytest	The de-facto Python testing framework	15.4
Quickselect	Partition-based search for the k-th smallest/largest element, O(n) average	19.17
Race condition	Bug where the result depends on the timing of threads/processes	16.4
Raw string	`r"…"` literal where backslashes are not escapes	4.5
Recursion	A function calling itself on a smaller input	7.7
Recursion limit	Maximum call depth (default 1000), `sys.getrecursionlimit()`	7.7
Reference count	Number of references to an object; freed when it reaches zero	17.2

Term	Meaning	Where
Reflected operator	`__radd__` etc., tried when the left operand doesn't support the operation	10.3
Regression test	Test added so a fixed bug can't silently return	15.1
Regular expression	Pattern language for matching text (`re` module)	4.10
Relative import	Import relative to the current package (`from . import x`)	8.4
REPL	Read-eval-print loop: the interactive `>>>` prompt	1.5
`__repr__` / `__str__`	Developer-facing / user-facing text for an object	10.2
resample	pandas time-series grouping by calendar period (`"ME"`, `"W"`, `"D"`)	18.10
RLock	Re-entrant lock the same thread may acquire several times	16.11
`secrets`	Module for cryptographically secure random tokens	14.7
Semaphore	Lock allowing up to n holders at once	16.4
Sentinel	Special marker value, e.g. `None` default or an end-of-queue object	16.5
Series	pandas 1-D labelled array	18.4
set	Mutable, unordered collection of unique hashable items	6.6
Shallow copy	New outer container sharing the same inner objects	5.8
Short-circuit evaluation	`and` / `or` stop as soon as the result is known	2.8
Slicing	`seq[start:stop:step]` — a new sequence (or view in NumPy)	4.4
Sliding window	Two indices bounding a contiguous range that moves in O(n)	19.2
`__slots__`	Class attribute fixing allowed instance attributes, removing `__dict__`	10.12
Small-int cache	CPython reuses the int objects −5 to 256	17.3
Soft keyword	Word that is a keyword only in context (`match`, `case`, `type`, `_`)	1.9
sqlite3	Standard-library interface to the embedded SQLite database	14.11
src layout	Project layout with code under `src/pkg/` so tests use the installed package	8.9
Stable sort	Equal keys keep their original relative order (Timsort is stable)	5.7
Standard library	Modules shipped with Python ("batteries included")	8.1
`staticmethod`	Method with no implicit first argument; a plain function in the class namespace	10.1
str	Immutable sequence of Unicode code points	4.1
Strong typing	No silent conversion between unrelated types (`"1" + 1` fails)	2.3
Structural typing	Compatibility decided by available methods/attributes, not by class name	13.8
Stub	Test double returning canned answers	15.5
Stub file (`.pyi`)	File with only type signatures for a module	13.13
subprocess	Module for running external commands (`subprocess.run([...])`)	14.6
`super()`	Proxy to the next class in the MRO, used to call overridden methods	10.4
Sweep line	Turning intervals into +1/−1 events sorted by time and keeping a running count	19.15
`sys.modules`	Cache of already-imported modules	8.2
`sys.path`	List of directories searched for imports	8.2

Term	Meaning	Where
t-string	Template string literal `t"…"` (3.14) producing a `Template` object instead of a `str`	4.6
TaskGroup	asyncio structured concurrency: runs tasks and cancels siblings on failure (3.11+)	16.9
TDD (test-driven development)	Red → green → refactor: write a failing test first	15.8
tempfile	Module creating temporary files/directories that clean themselves up	9.10
Test double	Stand-in for a real dependency in tests: dummy, stub, spy, mock, fake	15.5
Thread	Separate flow of execution sharing the process's memory	16.3
timeit	Module for timing small snippets accurately	17.6
Timsort	Python's stable, adaptive merge/insertion sort, O(n log n)	5.7
Topological sort	Ordering of a DAG's nodes so every edge points forward	19.8
tracemalloc	Standard module tracing memory allocations by line	17.6
Traceback	Report of the call stack when an exception is not handled	15.10
Trie	Prefix tree storing strings letter by letter; insert, search and prefix checks are O(word length)	19.13
Truthy	Value treated as true in a condition (anything not falsy)	2.6
try / except / else / finally	Exception-handling statement; `else` on success, `finally` always	9.2
tuple	Immutable ordered sequence; hashable if its items are	5.9
Tuple packing	Commas create a tuple: `t = 1, 2`	5.9
Two pointers	Two indices moving through data to avoid nested loops	19.2
Type alias	Name for a type expression (`type Vector = list[float]`, 3.12)	13.12
Type conversion	Changing a value's type (`int("3")`); implicit only within the numeric tower	2.7
Type hint	Annotation describing expected types for tools and readers	13.1
TypedDict	Type hint for dicts with a fixed set of typed keys	13.9
TypeVar	Placeholder type for generic functions/classes	13.6
ufunc	NumPy universal function operating element-wise in C (`np.sqrt`, `+`)	18.1
Unicode	Standard assigning a number (code point) to every character	4.9
Union type	`A | B`: a value that may be either type	13.4
Union-Find (DSU)	Structure tracking merging groups with near-O(1) find/union	19.9
Unit test	Fast test of one small piece in isolation	15.1
unittest	Standard-library xUnit-style testing framework	15.3
`unittest.mock`	Standard library for mocks and `patch`	15.5
Unpacking	Assigning an iterable's items to several names (`a, *rest = xs`)	5.10
UTF-8	Variable-length Unicode encoding, 1–4 bytes per character; the default to use	4.9
uuid	Module generating universally unique identifiers	14.9
uv	Fast tool managing Python versions, venvs, dependencies and lock files	8.8
Vectorisation	Operating on whole arrays in compiled code instead of Python loops	18.1

Term	Meaning	Where
View (NumPy)	Array sharing memory with another (e.g. a slice)	18.1
Virtual environment (venv)	Isolated per-project interpreter environment and package folder	8.7
Virtual machine (CPython)	The bytecode interpreter loop that executes compiled code	1.8
Walrus operator	`:=` assigns inside an expression (3.8+)	2.8
Weak reference	Reference that doesn't keep an object alive (`weakref`)	17.2
Wheel	Pre-built package archive (`.whl`) that installs without building	8.9
Wildcard pattern	`case _:` – matches anything without binding	3.4
`with` statement	Runs a block inside a context manager's setup and guaranteed cleanup	9.8
`yield`	Produces a value from a generator and pauses it	11.2
`yield from`	Delegates to a sub-generator/iterator	11.3
`zip`	Pairs items from several iterables, stopping at the shortest (`strict=True` to check)	3.6
zoneinfo	Standard-library IANA time zones (`ZoneInfo("Asia/Kolkata")`)	14.1

Chapter TL;DR

- Use the sheets in 21.1–21.19 as a quick reminder; follow the § references for full explanations, examples and pitfalls.
- 21.20 maps interview problem signals to patterns and lists the gotchas that cost the most points.
- 21.21 answers "what should I reach for?" in one table.
- The glossary defines 300+ terms in plain words and points to where each is taught.